Emergence of Intelligent Software Agents: From Semantic Reasoning via Real-World Datasets towards General Artificial Intelligence

Tapio Pitkäranta
Aalto University
tapio.pitkaranta@iki.fi

Helsinki, November 11, 2026

Doctoral thesis submitted in partial fulfillment of the requirements for the degree of Doctor of Science in Technology.

Supervisor:	Professor Petri Vuorimaa (since 2023) Professor Eljas Soisalon-Soininen (2004–2014)
Instructor:	Senior Research Scientist (Dr) Timo Laakko (2001–2005), Senior Research Scientist Santtu Toivonen (2001–2005)

Abstract

AALTO UNIVERSITY ABSTRACT OF THE DOCTORAL THESIS

Author:	Tapio Pitkäranta
Name of the thesis:	Emergence of Intelligent Software Agents: From Semantic Reasoning via Real-World Datasets towards General Artificial Intelligence
Date:	November 2026
School:	School of Science **Professorship:** Computer Science

Spanning 2001–2025, this thesis synthesizes research on Artificial Intelligence (AI) with a sustained focus on software agents, and organizes it into three phases. Its original contribution is the cross-phase synthesis of this previously published work into one cumulative argument, together with the public artefact releases, rather than new empirical results.

Context. Phase I began in the Semantic Web era, when agents were expected to interpret machine-readable content and coordinate via shared ontologies and protocol specifications. In Phase II, the emphasis shifted to applied machine learning and information retrieval over large, coded healthcare datasets. In Phase III, advances in Generative AI (GenAI) and Large Language Models (LLMs) reframed agents as conversational, tool-using systems, raising the question of how their behavior can be kept aligned with human values and organizational objectives.

Approach. The research follows a modified Design Science Research (DSR) process across three phases: (i) modeling and prototyping ontology-driven agents and interaction protocols on Web standards (Phase I); (ii) designing vector-space and structure-aware retrieval pipelines that exploit existing domain taxonomies and sparse codes without requiring complete redesign (Phase II); and (iii) engineering LLM agent patterns, AI alignment and governance mechanisms, and evaluation artefacts that emphasize transparency, auditability, and human-in-command oversight (Phase III). Artefacts are evaluated primarily by demonstration on implemented prototypes and case studies, with quantitative measurement where a meaningful reference exists (Phase II accuracy, the NordDRG benchmark).

Findings. Phase I shows that, despite technical promise, practical convergence between Semantic Web formalisms and multi-agent protocols was limited within this research program: heterogeneity of real-world ecosystems, modeling overhead, and brittle runtime assumptions constrained adoption. Phase II demonstrates that agents orchestrating retrieval over a combination of domain-specific encodings (e.g., clinical codes) and vector representations enable efficient analysis and retrieval over sparse, large-scale corpora (accuracy measured on real hospital data; scale tested for query latency on synthetic databases of up to 10^8 episodes), without changing established taxonomies. Phase III shows that LLM agents can be placed inside auditable, human-in-command architectures. The thesis consolidates a public *NordDRG AI Benchmark* for hospital-funding logic, released with its 2025 papers, on which top 2025 models master the payment-rule Logic tasks (best 13/13) while exact grouper emulation stays hard (best 7/13), and proposes *a reference development process* (CCL), demonstrated on real NordDRG change tickets, for human–AI rule alignment and maintenance. It introduces, as the breadth counterpart of CCL under the same governance layer, a multi-agent architecture (HADA) that places stakeholder agents over LLMs and legacy logic, translating objectives into vendor-neutral controls; in a retail-banking credit-scoring sandbox, it achieved four of its six design objectives fully and two partially. Unlike capability-focused approaches such as ReAct, AutoGen, and MetaGPT, HADA and CCL target governed, auditable decision-making; unlike horizontal benchmarks such as HELM, the benchmark tests executable hospital-payment rules. Together, these results treat alignment as an engineering discipline requiring instrumentation, versioning, and verifiable governance.

Summary. Read together, the three phases form this cumulative argument, the governable-agency through-line: each era's substrate exposes a facet of the demand for interoperable, auditable, human-in-command agent behavior that it cannot satisfy, and the next era takes it up. The results trace the emergence of intelligent software agents, from semantic reasoning through real-world data pipelines toward the general-purpose LLM agents of the mid-2020s, to which this work redirected its research program in 2023, advocating systems that pair LLM adaptability with explicit governance and audit trails.

Prior work on which the three phases build (peer-reviewed papers, theses, and preprints):
Phase I — 2001–2006: [1, 2, 3, 4, 5, 6, 7, 8]
Phase II — 2007–2013: [9, 10, 11, 12, 13, 14, 15, 16, 17]
Phase III — 2023–2025: [18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30]

Keywords: Artificial Intelligence (AI), Generative AI (GenAI), Large Language Models (LLMs), Machine Learning (ML), Multi-Agent Systems (MAS), Natural Language Processing (NLP), Semantic Web, Software Agents

Language: English

AALTO-YLIOPISTO VÄITÖSKIRJAN TIIVISTELMÄ

Tekijä:	Tapio Pitkäranta
Väitöskirjan nimi:	Emergence of Intelligent Software Agents: From Semantic Reasoning via Real-World Datasets towards General Artificial Intelligence
Päivämäärä:	marraskuu 2026
Korkeakoulu:	Perustieteiden korkeakoulu **Professuuri:** Tietotekniikka

Tämä väitöskirja kokoaa yhteen vuosina 2001–2025 tehtyä tekoälytutkimusta (AI), jonka kestävänä painopisteenä ovat ohjelmistoagentit, ja jäsentää sen kolmeen vaiheeseen. Alkuperäinen kontribuutio on aiemmin julkaistujen töiden vaiheet ylittävä synteesi yhdeksi kumulatiiviseksi argumentiksi sekä julkiset artefaktijulkaisut, ei uudet empiiriset tulokset.

Tausta. Vaihe I sijoittui semanttisen webin aikakauteen, jolloin agenttien odotettiin tulkitsevan koneluettavaa sisältöä ja toimivan yhteen jaettujen ontologioiden ja protokollien avulla. Vaiheessa II painopiste siirtyi soveltavaan koneoppimiseen ja tiedonhakuun laajoista, koodatuista terveydenhuollon aineistoista. Vaiheessa III generatiivinen tekoäly (GenAI) ja suuret kielimallit (LLM) tekivät agenteista keskustelevia ja työkaluja käyttäviä järjestelmiä, mikä nosti esiin kysymyksen niiden toiminnan linjaamisesta ihmisen arvoihin ja organisaation tavoitteisiin.

Lähestymistapa. Tutkimus noudattaa kaikissa vaiheissa mukautettua suunnittelutieteen tutkimusprosessia (Design Science Research, DSR): (i) ontologiavetoisten agenttien ja protokollien mallinnus ja prototypointi web-standardien varaan (vaihe I); (ii) vektoriavaruuteen ja rakenteeseen perustuvat tiedonhakuputket, jotka hyödyntävät olemassa olevia toimialaluokituksia ja niiden harvoja (engl. *sparse*) koodiesityksiä ilman täydellistä uudelleensuunnittelua (vaihe II); (iii) kielimalliagenttien suunnittelumallit, linjaus- ja hallintamekanismit sekä läpinäkyvyyttä, jäljitettävyyttä ja ihmisen komentovaltaa korostavat arviointiartefaktit (vaihe III). Arviointi perustuu pääosin prototyyppien ja tapaustutkimusten demonstraatioihin ja on kvantitatiivista, kun mielekäs vertailukohta on olemassa (vaiheen II tarkkuus, NordDRG-vertailuaineisto).

Tulokset. Vaihe I osoittaa, että teknisestä lupaavuudesta huolimatta semanttisen webin formalismien ja moniagenttiprotokollien käytännön yhteensovittaminen jäi tässä tutkimusohjelmassa rajalliseksi: todellisten ekosysteemien heterogeenisuus, mallinnuksen työläys ja hauraat ajonaikaiset oletukset rajoittivat käyttöönottoa. Vaihe II osoittaa, että agentit, jotka ohjaavat tiedonhakua toimialakoodausten (esim. kliiniset koodit) ja vektoriesitysten varassa, mahdollistavat tehokkaan analyysin ja tiedonhaun suurista aineistoista, joiden data-avaruus on harva (koodiyhdistelmiä on valtavasti enemmän kuin havaintoja), muuttamatta vakiintuneita luokituksia; tarkkuus mitattiin todellisella sairaala-aineistolla ja vasteajat synteettisillä, enintään 10^8 hoitojakson tietokannoilla. Vaihe III osoittaa, että kielimalliagentit voidaan sijoittaa jäljitettäviin, ihmisen komennossa oleviin arkkitehtuureihin. Vuoden 2025 julkaisujen yhteydessä avatun sairaalarahoituksen logiikan *NordDRG AI Benchmark* -vertailuaineiston sääntötason logiikkatehtävät vuoden 2025 parhaat kielimallit hallitsevat (paras 13/13), mutta DRG-ryhmittelijän (grouper) päätöslogiikan tarkka jäljittely on yhä vaikeaa (paras 7/13). Väitöskirja esittää ihmisen ja tekoälyn yhteistyönä tehtävään sääntöjen linjaukseen ja ylläpitoon *kehitystyön viiteprosessin* (CCL), joka demonstroidaan todellisilla NordDRG-muutospyynnöillä, sekä CCL:n leveyssuuntaisena vastinparina saman hallintakerroksen alla moniagenttiarkkitehtuurin (HADA), joka asettaa sidosryhmäagentit kielimallien ja vanhan päätöslogiikan päälle ja muuntaa tavoitteet toimittajariippumattomiksi kontrolleiksi; vähittäispankin luottoluokituksen kokeiluympäristössä se saavutti neljä kuudesta suunnittelutavoitteestaan täysin ja kaksi osittain. ReActin, AutoGenin ja MetaGPT:n kaltaiset kehykset keskittyvät siihen, mitä agentit osaavat tehdä, ja HELMin kaltaiset vertailuaineistot mittaavat yleisiä kyvykkyyksiä; tämä työ tähtää hallittuun ja jäljitettävään päätöksentekoon ja testaa todellisia, suoritettavia sairaalarahoituksen sääntöjä. Yhdessä tulokset käsittelevät tekoälyn linjausta mittarointia, versiointia ja todennettavaa hallintaa edellyttävänä insinööritieteenalana.

Yhteenveto. Yhdessä luettuina kolme vaihetta muodostavat kumulatiivisen argumentin, hallittavan toimijuuden (governable agency) läpileikkaavan linjan: kunkin aikakauden esitysmuoto paljastaa puolen yhteentoimivan, jäljitettävän ja ihmisen komennossa olevan agenttitoiminnan vaatimuksesta, jota se ei itse pysty täyttämään, ja seuraava aikakausi tarttuu siihen. Tulokset jäljittävät älykkäiden ohjelmistoagenttien kehittymisen semanttisesta päättelystä datavirtojen kautta 2020-luvun puolivälin yleiskäyttöisiin kielimalliagentteihin, joihin työ suuntautui vuonna 2023, ja puoltavat järjestelmiä, joissa kielimallien mukautuvuus yhdistyy eksplisiittiseen hallintaan ja jäljitysketjuihin.

Vaiheiden pohjana olevat aiemmat julkaisut (vertaisarvioidut artikkelit, opinnäytteet ja esijulkaisut):
Vaihe I — 2001–2006: [1, 2, 3, 4, 5, 6, 7, 8]
Vaihe II — 2007–2013: [9, 10, 11, 12, 13, 14, 15, 16, 17]
Vaihe III — 2023–2025: [18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30]

Avainsanat: tekoäly (AI), generatiivinen tekoäly (GenAI), suuret kielimallit (LLM), koneoppiminen (ML), moniagenttijärjestelmät (MAS), luonnollisen kielen käsittely (NLP), semanttinen web, ohjelmistoagentit

Kieli: englanti (tämä tiivistelmä: suomi)

Contents

Abbreviations . . . 8
List of Figures . . . 12
List of Tables . . . 17
PREFACE . . . 19
Acknowledgments . . . 21
1 **INTRODUCTION** . . . 22
1.1 Research Method . . . 22
1.1.1 Research Process . . . 22
1.1.2 Research Phases, Contributions, and Communication . . . 23
1.2 Phase I Problem and Motivation: Early Web Evolution—From Information Space to Service Platform . . . 24
1.2.1 Interaction-Protocol-Based Service Adaptation . . . 25
1.2.2 Ontology-Driven Software Agents for B2B Purchasing Automation . . . 27
1.2.3 Matching Service Descriptions in a Context-Aware Environment . . . 28
1.2.4 Phase I from a 2026 perspective . . . 29
1.3 Phase II Problem and Motivation: Software Agents and ML Healthcare Item Sets . . . 29
1.3.1 Assisting Clinical Decision Support with ML and Software Agents . . . 30
1.3.2 Phase II from a 2026 perspective . . . 31
1.4 Phase III Problem and Motivation: Software Agents in the Generative AI Era . . . 32
1.4.1 HADA: Human-AI Agent Decision Alignment Architecture . . . 33
1.4.2 NordDRG AI Benchmark: Teaching LLMs the Nuances of Hospital Funding Instruments 35
1.4.3 CCL: How Humans and AI Agents Align on Decision-Making Rules . . . 36
1.4.4 Phase III from a 2026 perspective . . . 38
1.5 Main Research Questions . . . 39
1.6 PhD Artefacts . . . 39
1.6.1 Main Artefacts . . . 39
1.6.2 Individual Artefacts per Phase . . . 40
1.6.3 Original Contributions . . . 40
1.7 Summary: Problem Identification and Objectives . . . 41
1.8 The Structure of the Thesis . . . 41
2 **BACKGROUND** . . . 44
2.1 A Short History of AI: AI Summers and Winters . . . 44
2.1.1 The Hype Cycle . . . 45
2.1.2 Review and Synthesis of AI Winters and AI Summers . . . 46
2.2 History of the Web . . . 47
2.2.1 The Hypermedia Concept (1945) . . . 47
2.2.2 Prototypes and Concept Development (1960s) . . . 47
2.2.3 Hypermedia Research (1980s) . . . 47

2.2.4 The Invention of the World Wide Web (1989) . . . 48
2.2.5 Research Boost (1990–1994) . . . 49
2.2.6 The Web: From Document Collection to Application Platform . . . 49
2.3 The Semantic Web and Its Vision of AI Software Agents . . . 49
2.3.1 Software Agents Before the Semantic Web . . . 49
2.3.2 The Vision of the Semantic Web and Intelligent Agents . . . 50
2.3.3 Key Enabling Technologies and the Semantic Web Stack . . . 51
2.3.4 Semantic Web Influence on Modern AI/ML Data Integration . . . 52
2.3.5 Could Semantic Web Principles Complement LLMs? . . . 52
2.3.6 How AI Has Shaped Real-World Semantic Web Applications . . . 52
2.4 Software Agents . . . 53
2.4.1 Agents and Distributed Artificial Intelligence (DAI) . . . 53
2.4.2 Hype Cycles: The Ever-Promising Agent Technology . . . 53
2.4.3 Definitions . . . 54
2.4.4 Single-Agent Architectures . . . 55
2.4.5 Multi-Agent Architectures . . . 55
2.4.6 Multi-Agent Communication . . . 56
2.4.7 Multi-Agent Communication Languages . . . 56
2.4.8 Interaction Protocols . . . 57
2.4.9 Content Languages . . . 57
2.4.10 Critical Synthesis and Positioning . . . 58
2.5 LLM Agents . . . 59
2.5.1 Emerging Field of LLM Agents . . . 59
2.5.2 Single LLM Agent . . . 60
2.5.3 Emerging LLM Agent Protocols . . . 61
2.5.4 Multi-Agent Conversation Patterns in LLM Era . . . 62
2.5.5 Prominent LLM Agent Frameworks and Libraries . . . 63
2.5.6 Critical Synthesis and Positioning . . . 63
2.6 LLM and DRG Related Studies . . . 65
2.6.1 LLMs . . . 65
2.6.2 AI Alignment and Safety . . . 65
2.6.3 LLM Benchmarks and Leaderboards . . . 65
2.6.4 Version Control Systems for Governed Change . . . 66
2.6.5 MLOps: Operating ML under Software Discipline . . . 67
2.6.6 NordDRG CaseMix System . . . 67
2.6.7 Related Work: DRGs and LLMs . . . 67
2.6.8 DRG Grouper Software . . . 68
2.7 Chapter Summary: Background . . . 68
3 **FORMAL DEFINITIONS** . . . 70

3.1 Deep Learning Analytical Framework and Core Definitions . . . 70
3.1.1 Core Concepts and Notation . . . 70
3.1.2 Transformer Definitions . . . 73
3.2 LLM Agents . . . 74
4 **PHASE I: DESIGN & DEVELOPMENT AND DEMONSTRATIONS** . . . 77
4.1 Interaction-Protocol-Based Service Adaptation . . . 77
4.1.1 Design and Development . . . 78
4.1.2 Demonstration . . . 81
4.2 Ontology-Driven Software Agents for B2B Purchasing Automation . . . 82
4.2.1 Design and Development . . . 83
4.2.2 Demonstration . . . 84
4.3 Matching Service Descriptions in a Context-Aware Environment . . . 85
4.3.1 Design and Development . . . 85
4.3.2 Demonstration . . . 88
5 **PHASE II: DESIGN & DEVELOPMENT AND DEMONSTRATIONS** . . . 89
5.1 Assisting Clinical Decision Support with ML and Software Agents . . . 89
5.1.1 Design and Development . . . 90
5.1.2 Demonstration and Results . . . 94
6 **PHASE III: DESIGN & DEVELOPMENT AND DEMONSTRATIONS** . . . 98
6.1 HADA: Human-AI Agent Decision Alignment Architecture . . . 98
6.1.1 Design & Development . . . 99
6.1.2 Demonstration . . . 105
6.2 NordDRG AI Benchmark for LLMs: Teaching LLMs the Nuances of Hospital Funding Instruments . . . 110
6.2.1 Design & Development: NordDRG AI Benchmark for LLMs . . . 110
6.2.2 Demonstration and Results . . . 116
6.3 CCL: How Humans and AI Agents Align on Decision-Making Rules . . . 118
6.3.1 Design & Development: Collaborative Curated Learning (CCL) . . . 120
6.3.2 Demonstration: Collaborative Curated Learning . . . 129
7 **EVALUATION AND DISCUSSION** . . . 135
7.1 Phase I: Interaction Protocol Adaptation with Software Agents . . . 135
7.2 Phase I: Ontology-Driven Software Agents for B2B Purchasing Automation . . . 136
7.3 Phase I: Matching Service Descriptions in a Context-Aware Environment . . . 138
7.4 Phase II: Assisting Clinical Decision Support with ML and Software Agents . . . 139
7.5 Phase III: Human-AI Agent Decision Alignment (HADA) . . . 142
7.6 Phase III: Evaluation of the NordDRG AI Benchmark . . . 144
7.7 Phase III: Evaluation of the CCL . . . 146
7.7.1 Evaluation of the NordDRG Decision-Tree Abstraction . . . 147
7.7.2 Evaluation of Preparing NordDRG for CCL . . . 147
7.7.3 Evaluation of the NordDRG Contributor and Reviewer Agents . . . 148

7.7.4 Discussion . . . 148
7.8 Cross-Phase Threats to Validity, Scalability, and Generalizability . . . 151
8 **CONCLUSIONS** . . . 153
8.1 Phase I: The History of the Semantic Web and Software Agents . . . 153
8.2 Phase I: Software Agents and Semantic Web Integration . . . 154
8.2.1 Phase I: Run-Time Protocol Adaptation . . . 154
8.2.2 Phase I: B2B Negotiation Systems . . . 155
8.2.3 Phase I: Matching Service Descriptions in a Context-Aware Environment . . . 156
8.2.4 Phase I: Irreducible Integration Gaps . . . 157
8.2.5 Phase I Transition: Toward Data-Driven Approaches . . . 158
8.3 Phase II: Assisting Clinical Decision Support with ML and Software Agents . . . 159
8.4 Phase III: Human-AI Agent Decision Alignment (HADA) Architecture . . . 160
8.5 Phase III: NordDRG AI Benchmark . . . 163
8.6 Phase III: Collaborative Curated Learning (CCL) . . . 165
8.7 Summary . . . 166
8.8 Toward an Exact Science of Agents . . . 169
9 **EPILOGUE** . . . 170
9.1 Agents are (finally) useful . . . 170
A **SUPPLEMENTARY FIGURES AND TABLES** . . . 171
References . . . 219

Abbreviations

Organizations

AAAI	Association for the Advancement of Artificial Intelligence
ACM	Association for Computing Machinery
CERN	Conseil Européen pour la Recherche Nucléaire
CHIRA	International Conference on Computer-Human Interaction Research and Applications
DARPA	Defense Advanced Research Projects Agency
FIPA	Foundation for Intelligent Physical Agents
IETF	Internet Engineering Task Force
ISO	International Organization for Standardization
NCSA	National Center for Supercomputing Applications
NOMESCO	Nordic Medico-Statistical Committee
OECD	Organisation for Economic Co-operation and Development
PCSI	Patient Classification Systems International
SRI	Stanford Research Institute
VTT	Valtion Teknillinen Tutkimuskeskus (Technical Research Centre of Finland)
W3C	World Wide Web Consortium

Technical

A2A	Agent2Agent
AADAR	Agentic Algorithmic Decision Alignment with RAG
ACL	Agent Communication Language
ADK	Agent Development Kit
AI	Artificial Intelligence
AJAX	Asynchronous JavaScript and XML
AMS	Agent Management System
AOSE	Agent-Oriented Software Engineering
AOV	Activity-on-Vertex
AP	Agent Platform
API	Application Programming Interface
AR-DRG	Australian Refined Diagnosis-Related Group
B2B	Business-to-Business
B2C	Business-to-Consumer
BDI	Belief, Desire, Intention
BERT	Bidirectional Encoder Representations from Transformers
BIG-bench	Beyond the Imitation Game Benchmark
C2C	Consumer-to-Consumer
CAC	Computer-Assisted Coding
CAD	Cosine Angle Distance
CaseMix	Case Mix (hospital patient classification)
CC	Complication or Comorbidity
CCL	Collaborative Curated Learning
ChatGPT	Chat Generative Pre-trained Transformer
CI/CD	Continuous Integration / Continuous Delivery
CORBA	Common Object Request Broker Architecture
CoT	Chain of Thought
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
DAI	Distributed Artificial Intelligence
DAML	DARPA Agent Markup Language
DAML-S	DARPA Agent Markup Language for Services
DCOM	Distributed Component Object Model
DF	Directory Facilitator
DNS	Domain Name System

DRG	Diagnosis-Related Group
DSR	Design Science Research
DSRM	Design Science Research Methodology
DSRP	Design Science Research Process
DYNAMOS	Dynamic Composition and Sharing of Context-Aware Mobile Services
ERP	Enterprise Resource Planning
FIPA-ACL	FIPA Agent Communication Language
FIPA-SL	FIPA Semantic Language
FSM	Finite State Machine
FTP	File Transfer Protocol
GB	Gigabyte
GDPR	General Data Protection Regulation
GELU	Gaussian Error Linear Unit
Gemini	Google's large language model
GenAI	Generative AI; foundation-model-based systems (e.g., LLMs) that generate novel content
GLUE	General Language Understanding Evaluation
GPRS	General Packet Radio Service
GPS	Global Positioning System
GPT	Generative Pre-trained Transformer
GPU	Graphics Processing Unit
gRPC	Remote Procedure Call framework
GUI	Graphical User Interface
HADA	Human-AI Agent Decision Alignment
HELM	Holistic Evaluation of Language Models
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
ICD	International Classification of Diseases
ICD-10	International Classification of Diseases, 10th Revision
ICD-11	International Classification of Diseases, 11th Revision
IDE	Integrated Development Environment
IDF	Inverse Document Frequency
IP	Internet Protocol
IPPS	Inpatient Prospective Payment System
IR	Information Retrieval
IT	Information Technology
ITSM	IT Service Management
JADE	Java Agent DEvelopment Framework
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
KIF	Knowledge Interchange Format
KPI	Key Performance Indicator
KQML	Knowledge Query and Manipulation Language
kNN	k-Nearest Neighbors
KR	Knowledge Representation
LangChain	Large language model framework
LangGraph	Graph-based language model framework
LEAP	Lightweight Extensible Agent Platform
LLaMA	Large Language Model Meta AI
LlamaIndex	Data framework for LLMs
LLM	Large Language Model
LMSYS	Large Model Systems
MAS	Multi-Agent System
MB	Megabyte
MCC	Major Complication or Comorbidity
MCP	Model Context Protocol
MDC	Major Diagnostic Category
ML	Machine Learning
MMLU	Massive Multitask Language Understanding

MRQ	Main Research Question
MS-DRG	Medicare Severity Diagnosis-Related Group
MTS	Message Transport System
NCSP	NOMESCO Classification of Surgical Procedures
NDMS	NordDRG Maintenance System
NL	Natural Language
NLG	Natural Language Generation
NLP	Natural Language Processing
NLU	Natural Language Understanding
NordDRG	Nordic Diagnosis-Related Groups
OCI	Open Container Initiative
OKR	Objectives and Key Results
OLAP	Online Analytical Processing
OpenAPI	OpenAPI Specification
OPT	Open Pre-trained Transformer
ORD	Execution order (priority) in `drg_logic`
OWL	Web Ontology Language
OWL-S	Web Ontology Language for Services
PaLM	Pathways Language Model
PDF	Portable Document Format
PIM	Product Information Management
PR	Pull Request
Pre-LN	Pre-Layer Normalization (layer normalization applied before each sublayer)
PROCPRO	Procedure property table in NordDRG
PWA	Progressive Web App
RACI	Responsible, Accountable, Consulted, Informed
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
RDF	Resource Description Framework
RDFS	RDF Schema
RDQL	RDF Data Query Language
ReLU	Rectified Linear Unit
REST	Representational State Transfer
RFQ	Request for Quote
RIF	Rule Interchange Format
RL	Reinforcement Learning
RLHF	Reinforcement Learning from Human Feedback
RMI	Remote Method Invocation
RPC	Remote Procedure Call
RuleML	Rule Markup Language
SaaS	Software as a Service
SDK	Software Development Kit
SEO	Search Engine Optimization
SGML	Standard Generalized Markup Language
SHACL	Shapes Constraint Language
SL	Semantic Language (FIPA)
SMS	Short Message Service
SOAP	Simple Object Access Protocol
SOM	Self-Organizing Map
SOP	Standard Operating Procedure
SPARQL	SPARQL Protocol and RDF Query Language
SQL	Structured Query Language
SRQ	Supporting Research Question
SSO	Single Sign-On
SuperGLUE	Super General Language Understanding Evaluation
SVG	Scalable Vector Graphics
SVM	Support Vector Machine
SWRL	Semantic Web Rule Language
T5	Text-to-Text Transfer Transformer

TCP	Transmission Control Protocol
TF-IDF	Term Frequency–Inverse Document Frequency
ToT	Tree of Thoughts
UDDI	Universal Description, Discovery and Integration
UI	User Interface
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
VCS	Version Control System
WLAN	Wireless Local Area Network
WSDL	Web Services Description Language
WWW	World Wide Web
WYSIWYG	What You See Is What You Get
XLSX	Office Open XML Spreadsheet
XML	Extensible Markup Language
XP	Extreme Programming
ZIP	Zone Improvement Plan (postal codes)

List of Figures

1 Design Science Research Process (DSRP) [31] model depicting a nominal six-step flow: problem identification and motivation (define and justify the problem), objectives of a solution (what an improved artefact should achieve), design and development (build the artefact), demonstration (apply the artefact in an appropriate context), evaluation (assess effectiveness and iterate back to design), and communication (report results to scholarly and professional audiences). The figure further highlights that studies may start from different entry points, including problem-, objective-, design/development-, or observation-centered approaches. The figure was redrawn from [31]. 22

2 Phase III at a glance: governable agency over decision-making models. The *capability premise* (can a large language model understand a large decision-making model?) is measured by the NordDRG AI Benchmark; on it rest two *harnesses* sharing one governance layer—CCL maintains *one* such large model (depth; rule change) while HADA deploys distributed agentic decisioning across an organization over *many* models (breadth; rule use). NordDRG is one running example. The symbols $\mathcal{R}$, $\mathcal{N}$, $\mathcal{O}$, and $\mathcal{L}$ are those of Definition 3.2.5, under which each agent's control policy Π operates ("policy-controlled"); $\boldsymbol{\theta}$ denotes the parameters (the model of record) of the decision-making model $\hat{f}_{\boldsymbol{\theta}}$ (Definition 3.1.17; Remark A.2). 33

3 Categorization issue: the Semantic Web research vision had a strong AI focus, but it is categorized under the WWW branch. 45

4 History of AI as illustrated by previous studies. The period of active Semantic Web development is typically identified as an AI winter period [25]. 46

5 Excerpt of the ACM Computing Classification System: the Distributed artificial intelligence subtree under Artificial intelligence [32]. 53

6 Agents and their environments [33]. 54

7 Single LLM agent core components. 60

8 Deep learning core concepts and their relationships. In the electronic version, each box links to its definition. 70

9 Transformer ecosystem concepts and their relationships (an LLM is created from the Transformer Mathematical Model that its Transformer Architecture uses; Definition 3.1.21). In the electronic version, each box links to its definition. 73

10 Progression of the expediting process from the contractor's point of view. 79

11 Core concepts of the interaction protocol ontology [34]. 80

12 The protocol state machine. 81

13 Overall purchasing process (RFQ, quote negotiation, and contract-making) with decision outcomes. 83

14 Hybrid approach for service provision, combining provider-to-user (B2C, top left) and user-to-user (C2C, top right) content sharing (own drawing, adapted from the model in [35]). 86

15 Overall architecture of the DYNAMOS system: client, distribution servers, and content servers. 87

16 Recommendation accuracy (share of cases in which the removed fact is among the topN proposals) of the multi-agent system with the SHOULD-occur and MUST-occur fetching strategies (x axes categorical, not to scale): (a) accuracy vs. topN (kNN=5); (b) accuracy vs. kNN (topN=1); (c) accuracy vs. topN with low-quality data (kNN=5). Redrawn from the original charts of [14]; values digitized from them (about ±0.01). 96

17 Query execution time (seconds) with the SHOULD-occur and MUST-occur fetching strategies when scaling the database size (kNN=5, topN=5). Redrawn from the original chart of [14]; values digitized from it (about ±0.005 s). 97

18 Bank strategy process: time horizons and stakeholder coverage. 100

19 HADA high-level architecture: the Agent Layer (controller, interaction agents, agent capabilities) and the Tools Layer (AI Tools, Data Tools, Standard Tools), connected via MCP or direct API calls. . . . 104

20 HADA prototype user interface: the multi-agent chat pane with role selection (left), the Governance catalog menu (top), and the AI Algorithm, Stakeholder Agents, AI Metadata, and Recent Decisions panels (right). . . . 106

21 Dialogue: Business Manager (BM) Shifting the AI Tool Objective to Minimizing Credit Risk. 109

22 **CCL PR state machine as implemented in the CCL Platform.** A PR enters *Draft*, where an agentic loop (implement → agent review → optional tests/verification → human review) iterates on requested changes; it then awaits role-based human approval and is *Merged*, or exits as *Rejected* or *Withdrawn*. Release tagging and *Rollback* on monitoring alerts are shown in Figure 23; merges require mandated human sign-off. . . . 124

23 Roles and human–AI hand-offs across the PR timeline (the "Final AI approval" of step 8 is advisory only; the human sign-off that follows is mandatory). Agents collaborate during an initial *agentic review* phase; escalation to human sign-off preserves the human-in-command invariant. The label `tau` in the figure denotes the risk tier κ, not the typing function τ of Definition 6.3.2; R2 and R3 are drawn together. . . . 125

24 **How the agent plugs into CCL.** The Contributor NordDRG Agent ingests NordDRG Forum tickets (via pasted text), grounds them in the official specification and guidelines, and emits explanation-rich atomic diffs; it then opens a pull request against the model-of-record, where CI runs the CCL gates (G1–G4) and, after the Reviewer agent's pre-review, the risk-tiered human approvals of Section 6.3.1 are collected (in the diagram, the R1 branch draws only the agent pre-review (the single human approval that R1 requires, Section 6.3.1.3, is not drawn separately), and the R2/R3 branch adds the Human Expert Reviewer's safety review and sign-off, Step 11) before a maintainer performs a merge, tags a release, and writes an immutable audit trail to the ledger with canary/rollback safeguards. The diagram shows the full CCL flow; in the demonstration evaluated here, tickets were supplied as pasted text rather than by forum notification, and CI (Steps 6–8) and the post-merge Steps 14–16 were not executed. The diagram shows the agent as a first-line contributor under "human-in-command" governance. 149

25 Human-in-the-loop or agentic workflow: grammar checking of this manuscript with two LLM agents. Left: the human dispatches every step and approves each change (A–E). Right: the agents hand off to each other, Agent 2 applies the corrections (C), and the human only reviews the updated manuscript (D) and approves it (E). . . . 170

26 Structure and contributions of the thesis. . . . 171

27 Hype Cycle: the Hype Level and the Engineering or Business Maturity curves [36]. . . . 171

28 Web architecture by Tim Berners-Lee in 1990 [37]. . . . 173

29 The Semantic Web as envisioned by Tim Berners-Lee: (a) the layered technology stack (2006) and (b) the 2003 rollout-vision "wave." Both panels are redrawn from the original sources. . . . 174

30 Relationships between communicative acts defined by FIPA [38]. . . . 175

31 Phase I: Semantic Web Stack [39] and Intelligent Web Services [40]. . . . 176

32 The FIPA Contract Net Interaction Protocol [41]. . . . 184

33 Agent Management Reference Model (redrawn from [42]). . . . 185

34 Simple illustration of the learning settings *Supervised Learning*, *Unsupervised Learning*, and *Reinforcement Learning*, each of which induces an *Optimization Problem*. . . . 186

35 **Left:** Block-level view of a GPT that is a Transformer Mathematical Model with L layers from input to output. Dataflow: input tokens are embedded, combined with positional encodings, passed through a stack of L decoder Transformer Blocks, layer-normalized, and mapped by a final linear+softmax head to token probabilities. **Right:** Internal structure of a single Block_ℓ with nodes drawn from the Transformer Mathematical Functions: masked multi-head self-attention f_{MHA} with h heads and a causal mask, followed by a position-wise feed-forward network f_{FFN} (GELU). Both sublayers use residual connections ($\oplus$), LayerNorm f_{LN}, and dropout (a training-time regularization procedure, part of Ψ in Definition 3.1.10, and the identity at inference). 189

36 Contract Net request for quotes in the B2B negotiation prototype. 190

37 Proposing a lower price. 191

38 DYNAMOS components on the Fuego Core event-based infrastructure: mobile client (left) and DYNAMOS Event Server (right). 191

39 Comparison of single-path vs. multi-path reasoning (redrawn from [43]). 192

40 Comparison between Reason Only, Act Only, Reason and Act, and Reflexion frameworks (redrawn from [44, 45]). 192

41 Development timeline of large language models, agent frameworks, agent protocols, and applications (redrawn from [46]; item placement largely follows the source and is approximate). 193

42 MCP general architecture [47]. 193

43 In the example library AutoGen, agents are conversable, customizable, and can be based on LLMs, tools, humans, or even a combination of them (redrawn from [48]). 194

44 A tool for communicating with the software agents. 196

45 A tool for monitoring the state and information the agent receives and sends. 197

46 Agents and messages. 198

47 Process flow of the protocol-adaptation demonstrator. 198

48 Proposal ontology. 199

49 Product ontology. 199

50 Phase I B2B negotiation prototype GUI: product catalog tables. 200

51 User Profile and User Context. 201

52 DYNAMOS System Overview. 201

53 DYNAMOS client screenshots from the August 2005 user trial (a trial participant's phone number, e-mail address, and face are masked). 202

54 The hierarchy of classification. 202

55 Primary and secondary business processes and data warehousing. 203

56 A sample coded dataset network interlinked by a coding scheme. Redrawn from [12, 13]. . . . 203

57 AI tool `getLoanDecision`, built on a Kaggle dataset: its decision trees and OpenAPI specifications (Versions 1.0 and 1.1). 204

58 Dialogue: Data Scientist delivering a new AI tool version for business approval. 204

59 Dialogue: Business Manager approves version 1.1 for production. 205

60 Dialogue: Customer applying for a personal loan. 205

61 Dialogue: Customer questioning the use of ZIP codes in lending. 206

62 Dialogue: Value and Ethics Manager (DVEM) evaluating ZIP codes. 206

63 Materials used: the NordDRG definition tables [49], shown as the list of the 20 tables and an excerpt of `drg_logic` (own typesetting from the published Combined 2024PL CSV release). . 207

64 Materials used: the NordDRG definition-table expert documentation [49], summarized in the author's words (own drawing; no pages of the guides are reproduced). 207

65 Learning paradigms create a decision-making model $\hat{f}_{\boldsymbol{\theta}}$ from data. Supervised, unsupervised, reinforcement, and transfer learning differ chiefly in the form and timing of the signals they exploit. 208

66 High-level architecture of the CCL Platform. The browser-based front end communicates with the FastAPI back end over REST. The back end integrates with PostgreSQL for persistence, a git repository (local, with a GitHub remote) for version control, and multiple LLM providers for agent capabilities. 208

67 **CCL workflows.** (a) an example process slice typed by participants, and (b) the participant set and the agentic, hybrid, and human workflow types. 209

68 **Governed Version Control history for CCL.** A feature branch (`feat-ccl-edit`) opens a PR with an Impact Brief and CI (tier R2), iterates agentically, then merges with `-no-ff` into *main* after required human approvals; an unrelated hotfix and tagged releases (`v1.4`, `v1.5`) are shown. A second branch (`feat-rejected`) illustrates a PR that is rejected (no merge). Only humans may merge to *main*. 210

69 Illustrative fragment of the first-layer decision logic for DRG 373X. Top: excerpt of the custom NordDRG format where each row expands to a path condition (rows that are identical in the columns shown collapse to one path). Bottom: the corresponding path-wise view as a canonical decision tree compiled for analysis. 211

70 NordDRG as decision trees: illustrative slices with 10, 25, and 100 root→leaf paths. Only small portions of the full structure are readable as static graphics. In practice: *10 paths* are fully legible at print scale and useful for explaining predicate structure and local flow; *25 paths* remain readable on screen with moderate zoom and are borderline for print without enlarging; *100 paths* are overcrowded in static form and are only useful with interactive panning/zoom, making them unsuitable for print. The full specification contains nearly 9,000 paths; it can be rendered in the browser but is difficult to navigate. 211

71 Preparing the NordDRG specification for CCL: (left) the CSV tables that constitute the agent-editable decision model; (middle/right) the two PDF guides from the NordDRG Forum used to ground agents in the custom notation and change process (own drawing; guide contents summarized). 212

72 The Overview page of the CCL Platform. The top section displays live system metrics (pull requests, pending reviews, specification tables, decision rules). Below, the quality-gate pipeline (G1–G4) and decision-model summary provide at-a-glance situational awareness. The five-step workflow diagram illustrates the change lifecycle, and navigation cards link to all platform sections. 212

73 The Agentic Workflow Configurator. Administrators define multi-step pipelines specifying which agents participate, the iteration limit, and optional test and verification procedures. The lower panel lists existing workflow configurations with their status and execution counts. 213

74 The Agent Configurator tab within Agentic Workflows. Each agent configuration specifies the role (Contributor, Reviewer, or Verifier), the LLM provider and model, a system prompt, and behavioral parameters. Configurations can be selected per workflow step and per Agent Chat session, allowing different tasks to use different models. 214

75 The Decision Tree page in SVG Graph mode, showing NordDRG decision logic as a flowchart. Rounded blue nodes represent decision points on input features (diagnosis codes, procedures, age, sex), and rectangular green leaf nodes represent DRG group outputs. The toolbar provides view-mode switching, DRG code filtering, layout orientation controls, and SVG export. 215

76 Contributor Agent processing a ticket from the NordDRG Forum. The agent proposes a change to the NordDRG specification, including atomic diffs and a rationale. The user can then review and export the suggestions for further processing. 216

77 NordDRG Reviewer Agent taking in the implementation by the NordDRG Contributor Agent together with the original ticket. The Reviewer Agent proposes changes to the implementation to align with the NordDRG decision model coding standards and guidelines. 216

List of Tables

1 DSRP process sequence (Figure 1) linked to Phases and Structure of this Thesis. 24

2 Main Research Questions by phase. 39

3 Original contributions mapped to the research questions they answer. Each contribution (**C1**–**C8**) is summarized in Section 1.6.3 and developed in full in Appendix A; artefact codes **A1**–**A7** follow Table 5. 40

4 Research questions mapped to the artefact(s) that answer them, the evaluation outcome and its level of evidence, and the publication(s) reporting that result. Artefact codes **A1**–**A7** follow Table 5; the evidence levels are developed in full in Section 7.8. 42

5 Main artefacts created across the three research phases. 43

6 Comparison of Prominent LLM Agent Frameworks. 63

7 Positioning recent LLM-agent approaches against the governance dimension this thesis targets. The agent works optimize capability, while the Tropos and Constitutional AI rows mark design-time and training-time governance for contrast; the thesis artefacts (HADA, CCL) add governed, auditable, human-aligned decision-making. Yes = enforced by the architecture at run time; Partial = supported but optional or unrecorded; Limited = present but not exposed; Design-time or Training-time = addressed only before run time; No = not addressed. HADA and CCL ratings are evaluated in Sections 7.5 and 7.7. 64

8 Materials used in the scalability experiment (Figure 17): number of patient care episodes $|D|$ and database size. 96

9 Stakeholder map for the banking use case `getLoanDecision`. 101

10 RACI matrix for the `getLoanDecision` AI Tool 103

11 Stakeholder agents (see Table 9), capabilities, and A2A contracts for the `getLoanDecision` prototype. 106

12 Publicly released NordDRG benchmark resources. 111

13 CaseMix questions with automatically verifiable answers (Logic-1–Logic-13), reproduced as released with the benchmark [24]. 113

14 CaseMix questions for emulating the official NordDRG grouper (Grouper-1–Grouper-13). All 13 prompts share one template and differ only in the test-case ID. 114

15 Configuration types for the NordDRG benchmark datasets. 115

16 Accuracy on the 13 automatically verifiable NordDRG Logic tasks (Logic-1–Logic-13). All runs used the **Definition Tables + PDF Instructions** bundle; each cell is a single scored answer (no repeated runs). (✓=correct, ✗=incorrect) 117

17 DRG grouper emulation results on thirteen case-level tasks (Grouper-1–Grouper-13). All runs used the **Definition Tables + PDF Instructions** bundle and the structured test cases. (✓=correct; exact match on *DRG and drg_logic.id*; ✗=incorrect) 118

18 Quantitative metrics of the CCL Platform implementation (approximate counts at the time of writing). 130

19 Scale and structural statistics for the NordDRG decision model. The "first-layer" tree refers to the top-level grouping layer. 132

20 Outcome of each of the five NordDRG Forum tickets (Cases A–E) under two workflow modes across three LLM backends (accepted in total: Agentic 8/15, Single-Agent 1/15). A check mark (✓) indicates that, on the author's manual review (a single, non-blinded reviewer), the generated pull request reproduced the change the NordDRG experts implemented in the NDMS production system for the same ticket—the reference standard; a cross (✗) indicates that it did not. Automated CI (Steps 6–8) was not executed in this demonstration. 149

21 Consolidated results-and-evidence summary across the three phases. Quantitative figures are reproduced from the per-artefact evaluation sections (Sections 7.1–7.7); "reference standard / baseline status" records what each result is measured against and, for the governance processes of the novel architectures, why no baseline can exist. 152

22 Agent characteristics [50]. 172

23 Candidate's individual role per main artefact and its lead publication (Art. = main artefact, Table 5; Ph. = phase). 177

24 Phase I individual artefacts and their mapping to the main artefacts (Table 5). 178

25 Phase II individual artefacts and their mapping to the main artefacts (Table 5). 178

26 Phase III individual artefacts and their mapping to the main artefacts (Table 5). 179

27 Core symbols of the Transformer architecture, with the values of GPT-3 175B [51] as an example. 185

28 Key matrices and intermediate representations in a Transformer block (for the activation matrices, row = token and column = feature). 186

29 Gate policy by risk tier κ for CCL pull requests. Each required gate is fail-closed (a failing gate blocks the PR); higher tiers enlarge evidence requirements and mandate additional human sign-offs, and only humans approve merges. In the terms of Definition 3.2.5, G_1 realizes the objective check ($\mathcal{O}$), G_2–G_4 the norm checks ($\mathcal{N}$), and the required human approvers the accountable-human clause of $\mathcal{R}$. 210

30 Reference answers for each CaseMix question (Logic-1–Logic-13). 217

31 Reference answers for each CaseMix grouper question (Grouper-1–Grouper-13). 217

PREFACE

Agents are (finally) here

During the course of this thesis research, intelligent software agents have evolved from theoretical Distributed Artificial Intelligence (DAI) concepts into integral components of practical, everyday applications through the emergence of *LLM agents*. This advancement is exemplified by the grammar review process for this document (then over 150 pages), which was completed within a few hours using a small Multi-Agent System (MAS) of two agents, with one human—myself—in command.[1] Setting up such agents can now be accomplished in minutes with modern tools. More importantly, this foregrounds the question of what role humans should assume in these workflows. Should the entire grammar perfection task be delegated to *LLM agents*, and what role should remain for the human? See Section 9.1 for how I chose to allocate the responsibilities in the case of this document.

My experience leaves two lingering questions: why have these agents reached practical utility only now, and why did I not adopt this application earlier?

Use of AI tools. In preparing this thesis, I have used AI agents and large language models from OpenAI, Anthropic, and Google as working tools—for example, in language revision, consistency checking, literature and citation verification, and manuscript revision—and, as coding assistants, in developing the Phase III software demonstrators. I have reviewed all such contributions, and I alone am responsible for the content of this thesis.

From Surprises Toward an Exact Science

In 2023, I watched a four-year-old child in my family speak Finnish to a phone. The AI in the phone took the child's spoken idea and turned it into pictures and bedtime stories, which the phone then read aloud. What surprised me was that the AI understood the child, and that the two understood each other, although the child could neither read nor write. I wanted to find out precisely where the surprise came from. I came to the conclusion that the surprise was not mine alone: in practice, the Transformer [52] surprised the whole field, and the surprises have continued since the release of ChatGPT.

In my experience, computing and AI as a science have been a combination of abstract engineering and separate mathematical methods, and even basic concepts such as "learning" and "agent" have been imprecisely defined; the agent literature I reviewed in Phase I already acknowledged that no consensus existed (Section 2.4). The development of AI has been so fast that it should be carried out safely, and I believe one way to do so is to move the field toward a more precisely defined science, an exact science. To that end, I have tried to express the Phase III work in defined terms and formulas (Chapter 3); I return to this at the end of the Conclusions (Section 8.8).

About the research period of this thesis

Spanning more than two decades, my career in software development has been bound up with the practical application of AI and Machine Learning (ML). It began in the early 2000s. Much of my career has gone to deploying AI and ML algorithms in diverse industrial settings—healthcare, retail, international tax, and human resources—which gave me experience with real-world applications. This industry experience has strongly shaped the direction of my doctoral thesis, which moved from the complexities of Web infrastructure and semantics to the tangible analysis of real-world datasets.

Advances in AI over this period include the creation of highly accurate, large-scale search engines, the development of neural-network-based algorithms achieving superhuman performance in complex games [53], and the recent emergence of LLMs that convincingly simulate human conversation, demonstrating a significant leap in Natural Language Processing (NLP). Even young children, regardless of literacy, can now utilize

[1] **A note on language conventions.** This manuscript follows US English spelling (Merriam-Webster), with one deliberate exception: *artefact/artefacts* retains the British/European spelling that is standard in the Design Science Research literature on which the thesis builds. It uses the serial (Oxford) comma and standardizes key terms—LLM, LLM agent, and decision-making model against the formal definitions in Chapter 3, and NordDRG and CaseMix as introduced in Section 2.6.6 and the list of abbreviations. The agent-assisted language review described above was limited to grammar and this kind of consistency; the wider use of AI tools in this work is described below.

current AI applications to generate creative content, such as bedtime stories [30]. A 2025 study [54] reports that some LLMs passed a controlled three-party Turing test.

The contemporary discourse surrounding AI and ML has consequently shifted toward profound, almost philosophical, inquiries. These include examining the alignment of human values and objectives with the decisions made by algorithms [55] and contemplating the potential for AI to surpass human capabilities [56]. This thesis compiles my research papers and contributions, spanning a period of more than two decades. My own public research trajectory commenced with investigations into software agents and the Semantic Web, progressing toward the development of flexible ML, NLP, and Information Retrieval (IR) algorithms, and more recently, exploring the integration of software agents with LLMs to enable more autonomous and intelligent systems to align human and AI decisions.

Today, AI is advancing at a historically rapid pace and keeps changing our sense of what is possible: problems long considered out of reach have become tractable, and widely used AI benchmarks have saturated one after another [57]. This thesis is no exception: the single-question tasks of the 2024 NordDRG studies approached saturation as 2025 frontier models caught up (Chapter 6). The final phase of this work was carried out mainly in 2023–2025 and describes that period; already in 2026, much has changed. I do not attempt to rewrite the history of the research from today's vantage point; instead, I examine each phase in the light of its own time and keep today's reassessments apart, in the clearly marked "from a 2026 perspective" subsections (Sections 1.2.4, 1.3.2, and 1.4.4). Were I to begin this thesis today, I would approach many questions differently, assisted by continually evolving agentic AI.

Acknowledgments

This dissertation has unfolded over more than two decades, and it owes much to the people who accompanied it through its three phases. Long before any of them, my father, Juhani Pitkäranta, showed me what scientific work looks like; I thank him for our many mathematical discussions over the years and for being a scientific role model to me.

In the early Semantic Web years I was fortunate to work alongside Santtu Toivonen, whose continuous interest, advice, and generous pointers to the literature shaped my first steps as a researcher. I thank Dr Timo Laakko for his thoughtful comments on the early demonstrations, and Heikki Helin and Mikko Laukkanen for many stimulating discussions on software agents and the Semantic Web. I am grateful, too, to Oriana Riva, whose co-authored Semantic Web work dates to this formative period. Much of this early work unfolded in the inspiring research environment of VTT Technical Research Centre of Finland, where I was fortunate to be surrounded by talented colleagues—among them Seppo Linnainmaa, whose pioneering contributions to computing I greatly looked up to.

As the research moved toward machine learning and real-world clinical data, I had the privilege of collaborating with a wider circle of co-authors. I warmly thank Minna-Liisa Sjöblom and Petra Kokko for their contributions to our joint publications. I owe a special debt of gratitude to Martti Virtanen, whose collaboration on NordDRG and the broader Nordic CaseMix work was central to that phase of the research. Across these first two phases I was fortunate to be supervised by Professor Eljas Soisalon-Soininen, to whom I am indebted for his valuable comments and unfailingly supportive attitude throughout those years.

In the most recent phase, centered on generative AI and LLM agents, I again had the good fortune of excellent collaborators on our joint publications. I warmly thank Eero Hyvönen and Natalia Vuori for their contributions, and I am especially grateful to my wife, Leena Pitkäranta, who is herself a co-author of much of this phase's work (my individual role in each co-authored publication is stated in Table 23).

I also thank Kristiina Kahur for her valuable comments on the NordDRG AI Benchmark and Collaborative Curated Learning (CCL) papers. I am grateful to Patient Classification Systems International (PCSI) for the best-paper award at its 2025 conference in Quebec City, given to our paper *Towards Curated Learning*, which introduced CCL, and for the invitation to give a keynote talk at the PCSI 2027 conference. I am likewise grateful for the second-best-paper recognition that my paper *NordDRG AI Benchmark for Large Language Models* received at the CHIRA 2025 conference in Marbella, Spain.

Above all, I wish to express my heartfelt gratitude to my doctoral supervisor, Professor Petri Vuorimaa, for his guidance, encouragement, and steady support in bringing this thesis to completion. I also warmly thank Juho Vepsäläinen, who read the manuscript through several times and whose careful comments helped me polish and improve it.

Beyond academia, my thinking has been sharpened by many talented colleagues in industry whose collaboration, though not directly part of this thesis, gave me brilliant people to work with and learn from. I warmly thank Mikko Kärkkäinen (RELEX), Hannu-Tapani Leppänen (Aibidia), Markus Suomi (Tietoevry), and Markku Savusalo, Maria Niiniharju, and Tomi Pienimäki (Siili Solutions). I am also grateful to Mikko Rotonen (HUS), an industry lead in the healthcare sector and a genuine driving force for change.

Finally, my deepest thanks go to my wife, Leena Pitkäranta—a constant source of support from the very beginning—and to our whole family, whose patience and love made everything else possible. I also thank our children for their inspiration: the open mind and the high expectations with which they approach AI have continually sharpened my own thinking.

1 INTRODUCTION

This chapter traces a two-decade research program investigating how software agents can enable automated, auditable interaction across heterogeneous systems and domains. The research unfolds across three distinct phases that mirror the evolution of AI technology: from Semantic Web services (2001–2006), through ML and data integration (2007–2013), to generative AI and LLMs (2023–2025). Throughout these phases, software agents provide the architectural through-line, while the underlying representational substrates evolve from ontologies and protocols to statistical models and now foundation models. The chapter first presents the research methodology and process that guided this longitudinal study, then details each phase's context, problems, and contributions. Each phase section states its main research question in context; the research questions, artefacts, and original contributions are then consolidated (Sections 1.5 and 1.6.3).

1.1 Research Method

1.1.1 Research Process

This dissertation employs a *modified Design Science Research (DSR)* methodology (DSRM) and process (DSRP) illustrated in Figure 1 that adheres to the canonical six-step process of Peffers et al. [58, 31] while tailoring each stage to more than twenty individual studies conducted over two decades. The modification lies in how the six activities are applied, not in the activities themselves: the studies entered the process at different points; demonstration, complemented by the peer review of the underlying publications, is the principal evaluation mode, with quantitative validation where a reference exists; and the evaluations of all three phases are collected in Chapter 7. Guided by the relevance-rigor balance articulated in Hevner et al.'s [59] DSR guidelines, every study executed the downstream phases of *artefact design, demonstration, evaluation, and communication*. By contrast, the upstream *entry points*—whether problem-, objective-, or design-centered—varied from study to study, as Peffers et al. explicitly allow.

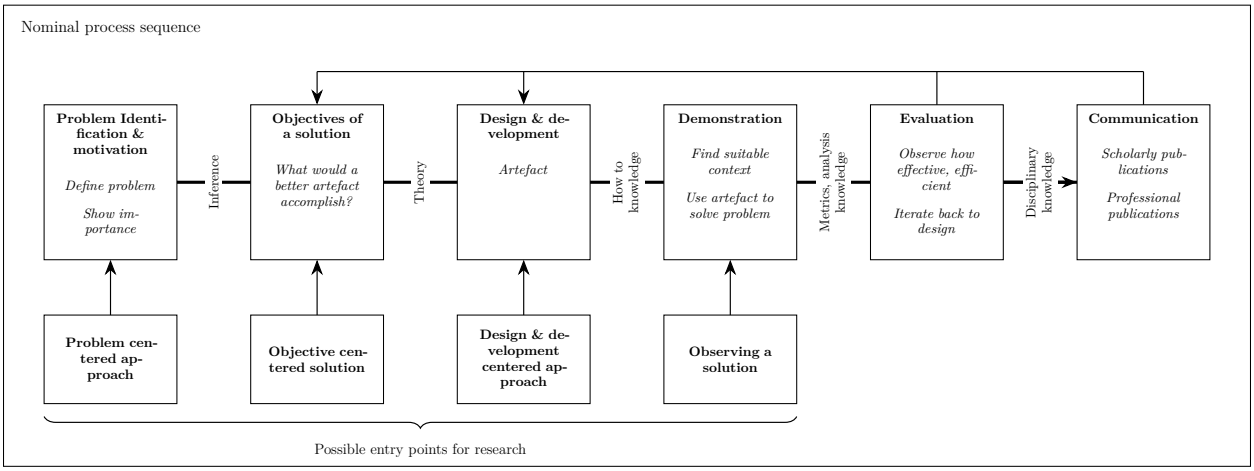


Figure 1: Design Science Research Process (DSRP) [31] model depicting a nominal six-step flow: problem identification and motivation (define and justify the problem), objectives of a solution (what an improved artefact should achieve), design and development (build the artefact), demonstration (apply the artefact in an appropriate context), evaluation (assess effectiveness and iterate back to design), and communication (report results to scholarly and professional audiences). The figure further highlights that studies may start from different entry points, including problem-, objective-, design/development-, or observation-centered approaches. The figure was redrawn from [31].

Inspired by Gregor and Jones's [60] anatomy of *mid-range design theory*, the program sought to encapsulate the *constructs, design principles, and justificatory knowledge* of its technical artefacts. Each iteration began with a problem motivation or design opportunity, proceeded to explicit objectives that set success criteria, and advanced through incremental development cycles. Formative evaluations—both analytical and prototypical—were used to refine successive iterations in accordance with Hevner et al.'s [59] recommendations.

Because large-scale field deployments were beyond the scope, the resulting artefacts were disseminated primarily as *proof-of-concept, conceptual demonstrations*—scenario walk-throughs and storyboarded simulations built

from representative data. Rather than relying on internal expert panels, the publications underlying the artefacts were, in most cases, double-blind peer-reviewed at specialist conferences in their domain (e.g., health-informatics venues for hospital decision support, enterprise-systems tracks for SaaS artefacts); the exceptions are the ERCIM News note related to artefacts A1 and A2 (Table 5), the Master's and Licentiate monographs, and the Phase II supporting items (a medical-journal editorial, a poster, and two conference presentation decks), which are not independent contributions. Such external vetting—aligned with DSR's emphasis on independent evaluation [59, 31]—complements, but does not replace, artefact evaluation, whose limits are discussed in Section 7.8.

By *demonstration* we mean applying an artefact in a representative context—scenario walk-throughs and storyboarded simulations on real or realistic data—to show feasibility, in the sense of Peffers et al. [31]. By *validation* we mean a controlled or comparative empirical study that measures effectiveness against a baseline or reference. *Comparative, quantitative validation is reported wherever a meaningful reference exists*—most directly for the NordDRG AI Benchmark, which is scored by exact match against published gold keys, for Grouper the official grouper output (Section 7.6), and for the Phase II retrieval pipeline, which is measured against historical coded documentation (Section 7.4). The outputs of the Collaborative Curated Learning process (CCL) also have a reference standard, the expert change implemented in the NordDRG Maintenance System (NDMS) for the same change ticket, but are judged against it only at demonstration scale (Section 7.7). For the governance *processes* of the Phase III architectures (HADA and CCL), *no established real-world baseline exists*, because they introduce a governance layer that current production systems and agent frameworks do not implement. These processes are therefore evaluated by demonstration (for CCL, complemented by a single-agent ablation) and positioned against what existing frameworks omit, with full-scale field validation deliberately left out of scope (Chapter 7).

Interim results were also consolidated into two milestone monographs: a *Master's thesis* that captured the earliest design cycles and a subsequent *Licentiate thesis* that documented an expanded artefact portfolio and its preliminary evaluations. Both monographs served as formal checkpoints before the research progressed to later phases.

In sum, this research operationalizes the Peffers et al. process model [58, 31], embodies Hevner et al.'s [59] design guidelines and March and Smith's [61] build-evaluate framing, and draws on Gregor and Jones's [60] anatomy to articulate design principles (notably the governable-agency through-line, Section 1.1.2), contributing primarily to practice and toward mid-range theory in AI-enabled decision systems.

1.1.2 Research Phases, Contributions, and Communication

Figure 26 (Appendix A) and Table 1 trace the dissertation's three phases, which mirror both the wider evolution of AI technology and the author's own professional arc from Web-centric infrastructure to modern generative models.

The dissertation synthesizes this two-decade body of previously published agent research into one analytical scheme—seven artefacts, seven research questions, and a single through-line; its original contribution is that synthesis, together with the public artefact releases, rather than new empirical results (Section 1.6.3).

This dissertation reads its three phases as *one cumulative argument*: in each era the agents inherit a limitation the previous era could not overcome. Phase I's symbolic, knowledge-engineered agents could coordinate services but could not learn from data or scale their ontologies. Phase II answers this with data-centric machine learning, yet its bespoke statistical models could neither explain themselves nor keep human experts in command at scale. Phase III answers *that* with large-language-model reasoning placed inside auditable, human-governed multi-agent architectures. Seen from the technology side, the same line is a "then versus now" arc: the early agent vision was held back by immature tooling, fragmented standards, and resource-limited devices. Today's foundation models—paired with connectivity, hyperscale infrastructure, and large community-generated data streams—remove many of those barriers, which is why agents have become practical only now, as the Epilogue illustrates hands-on (Chapter 9).

We name this cumulative argument the *governable-agency through-line* and use it as the unifying analytical lens of the dissertation. Its claim is that the true constant across the three phases is not the representational substrate—which shifts from ontologies and interaction protocols, through statistical and information-retrieval models, to large language models—but the recurring demand for *governable agency*: agent behavior that is interoperable, auditable, and aligned with human command. The lens runs along two axes. The first is *limitation inheritance*, the era-by-era cascade traced above, in which each phase inherits and advances the same problem rather than restating it. The second is *technology enablement*, the "then versus now" arc

noted above. Read along these two axes, the three phases become one progression of knowledge rather than a retrospective collection of projects; its framing is published in the two positioning papers [25, 30]; the synthesis itself is developed only in this dissertation (Section 1.6.3).

Table 1: DSRP process sequence (Figure 1) linked to Phases and Structure of this Thesis.

DSRP	Time Frame	1, 2	3, 4	5	6
DSRP Labels		Problem Identification, Objectives	Design & Development, Demonstration	Evaluation	Communication
Thesis Phase I	2001–2006	**Section 1.2**	**Chapter** 4: Section 4.1, Section 4.2, Section 4.3	**Chapter** 7: Section 7.1, Section 7.2, Section 7.3	Publications: [1, 2, 3, 4, 5, 6, 7, 8]
Thesis Phase II	2007–2013	**Section 1.3**	**Chapter** 5: Section 5.1	**Chapter** 7: Section 7.4	Publications: [9, 10, 11, 12, 13, 14, 15, 16, 17]
Thesis Phase III	2023–2025	**Section 1.4**	**Chapter** 6: Section 6.1, Section 6.2, Section 6.3	**Chapter** 7: Section 7.5, Section 7.6, Section 7.7	Publications: [18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30]

Phase I—2001–2006. The work begins with agent-mediated *Semantic Web* services: ontological descriptions, agent communication languages, and MAS coordination for intelligent service matching. Findings from this formative stage were consolidated into a Master's thesis and expanded in a Licentiate thesis, alongside a cluster of early conference papers. Together they establish the semantic foundations and messaging patterns that Phase II reuses and Phase III revisits.

Phase II—2007–2013. Because, within this research program, agentic AI found no viable route through the combination of Semantic Web formalisms and symbolic agent architectures, the research pivots to *data-centric ML*. Kernel methods and vector-space information retrieval over large healthcare item sets are layered beneath the existing agent framework, allowing agents to orchestrate decisions over high-dimensional coded healthcare datasets. This hybrid symbolic–statistical stance is documented in a suite of peer-reviewed healthcare-informatics publications.

Phase III—2023–2025. The final phase integrates GenAI (foundation-model-based systems, such as LLMs, that generate content from prompts) into multi-agent architectures, enabling agents to offload portions of planning, explanation, and natural-language interaction to foundation models. Contributions include the *NordDRG AI Benchmark for LLMs*, the Collaborative Curated Learning (CCL) process, and the Human-AI Agent Decision Alignment (HADA) architecture, each vetted through specialist conference venues.

Across the three phases, communication outlets evolve from theses and early Semantic Web conferences to domain-specific ML venues and, most recently, specialist conferences—yet each contribution passes through the same DSR activities, with the level of evaluation evidence stated per artefact in Table 4.

1.2 Phase I Problem and Motivation: Early Web Evolution—From Information Space to Service Platform

Research Context.

The World Wide Web, conceived in 1989 [62] and popularized during 1991–1995 [63], began as a decentralized, hyperlinked information space. In the early 2000s (circa 2000–2005)—which largely overlaps the initial phase of this study (2001–2006)—the Web evolved from a predominantly human-readable document repository toward a platform for distributed services, adding programmatic access[2] alongside traditional browser-based interaction. This period also saw mobile phones emerge as endpoints for Web access (feature-phone Internet usage circa 2003–2006; smartphone adoption 2007–2010), and context-aware information (user location, device capabilities, and preferences) gained importance for tailoring services and interactions.

[2] e.g., SOAP (Simple Object Access Protocol) and REST (Representational State Transfer) APIs (Application Programming Interfaces)

1.2 PHASE I PROBLEM AND MOTIVATION: EARLY WEB EVOLUTION—FROM INFORMATION SPACE TO SERVICE PLATFORM

Web Services vs. Semantic Web (synthesis). By the early 2000s, industry-grade Web services delivered pragmatic, platform-neutral interfaces for Remote Procedure Call (RPC)-style integration, whereas the Semantic Web vision [64] aimed for machine-interpretable data and agent-level reasoning over shared ontologies. Figure 31 (Appendix A) situates both: Web services operationalized interoperability at the *message* layer; the Semantic Web targeted *meaning*-level interoperability and autonomous discovery/coordination by software agents. This thesis treats them as complementary layers—transport/API standardization below, semantics and agent protocols above—asking where they converge in practice and where irreducible gaps remain.

What is new here. Phase I contributes no new formalism: its components were previously published and examined (the Licentiate thesis largely corresponds to this phase), and the dissertation's role is to *synthesize and position* them. Its honest value is an early, concrete probe of *where* Semantic Web formalisms and FIPA software-agent protocols practically converge—runtime RDF-driven protocol adaptation, ontology-driven B2B negotiation, and context-aware mobile service matching—and where they do not: in these prototypes, the agent tooling used here and mainstream Web standards showed no sign of converging, the gap that motivated the data-centric pivot of Phase II.

Main RQ.I.1 (Table 2): *What are the historical origins of the Web and of software agents?*

Rationale. Both paradigms emerged from parallel efforts to enable autonomous yet interoperable systems on the Internet. Clarifying their shared lineage explains why the Semantic Web vision and agent architectures were repeatedly intertwined, and why their integration remains relevant for runtime interoperability and adaptation today.

Operationalization via three scenarios. RQ I.2 and RQ I.3 below examine how the two paradigms complement each other by implementing three "Intelligent Web Services" scenarios that bind semantic descriptions and agent interaction protocols to concrete tasks:

a) **Interaction-protocol adaptation (construction expediting).** At runtime, agents ingest Resource Description Framework (RDF)-serialized agent protocol descriptions and execute conversations across partners on an agent platform, aiming to reduce status latency and reconcile reports (see Section 1.2.1, Section 2.4).

b) **Ontology-driven purchasing.** Buyer/supplier agents use shared domain ontologies to automate Request for Quote (RFQ) dissemination, compare quotes expressed in a shared proposal ontology, and produce first-round triage prior to human decision in Business-to-Business (B2B) contexts (see Section 1.2.2).

c) **Context-aware service matching (mobile/sailing).** A hybrid model aligns multi-modal context (time, activity, location) with Business-to-Consumer (B2C) service descriptions and Consumer-to-Consumer (C2C) annotations to return only "here-and-now" relevant results (see Section 1.2.3).

1.2.1 Interaction-Protocol-Based Service Adaptation

1.2.1.1 Problem Identification and Motivation

Business context. Large general contractors coordinate multi-tier supply networks of subcontractors, suppliers, and carriers. To control schedule and cost risk, they run an *expediting process*: soliciting progress reports, validating them with third parties, and acting on discrepancies. In practice, this coordination still occurs over ad-hoc channels (phone and e-mail), which introduce latency and poor auditability—conditions that fuel disputes in construction projects [65].

Potential solution. A run-time-adaptive, agent-mediated solution that interoperates with heterogeneous partner protocols could (i) shorten the confirm-deny feedback loop, (ii) reduce manual effort, and (iii) yield a machine-traceable audit trail.

Assumptions. The following are assumed: (i) shared formal protocol descriptions, (ii) run-time ingestion/-compilation into executable conversations, and (iii) auditable execution on an agent platform, in the presence of realistic heterogeneity (partners, payloads, and protocol variants).

1.2.1.2 Research Gap

There are standards for agent interaction patterns, and RDF provides a machine-readable substrate, yet it remains unclear how far these can be *practically* combined for *run-time* adaptation (protocol variants published at, or close to, execution time) without hard-coded flows. The construction expediting scenario—where partner-specific protocol variants appear late and differ subtly—offers a concrete probe of both possibilities and limits of semantic–agent integration.

1.2.1.3 Research Questions

Main RQ.I.2 (Table 2): To what extent can Semantic Web formalisms and software-agent protocols be integrated to deliver runtime interoperability and adaptation on the Web, and what irreducible gaps prevent practical convergence?

Through the lens of a construction expediting scenario, what does the contractor agent's ability to parse RDF-serialized protocols reveal about the extent and limits of integrating Semantic Web formalisms with software-agent protocols for runtime adaptation? The following Supporting Research Questions (SRQs) examine specific integration points and the gaps they expose (SRQ and objective numbers in Phase I follow the three studies—I.1 protocol adaptation, I.2 B2B negotiation, I.3 DYNAMOS—so SRQ.I.1.* and SRQ.I.2.* both support Main RQ.I.2, and SRQ.I.3.* supports Main RQ.I.3):

SRQ.I.1.1 Surface integration versus semantic understanding. While RDF successfully serializes Foundation for Intelligent Physical Agents (FIPA) protocols for runtime parsing, how does the reliance on pre-programmed state machines rather than semantic reasoning reveal a fundamental gap between syntactic data exchange and the Semantic Web vision of machine understanding?

SRQ.I.1.2 Platform-specific versus Web-scale interoperability. The agent platform enables agent message exchange in a controlled Java environment, but what does this platform dependency reveal about the irreducible gap between agent-platform protocols and true Web-based interoperability?

SRQ.I.1.3 Constrained adaptation versus open-world flexibility. The expeditor adapts to conforming protocols within a fixed ontology, but how does this limited variability expose the practical convergence barrier when facing the Web's heterogeneous services and evolving semantics?

SRQ.I.1.4 Data extraction versus semantic integration. Basic report parsing and parameter passing work within the scenario, but where does the absence of inference and context reasoning demonstrate that current integration remains far from autonomous semantic service composition?

1.2.1.4 Objectives of the Solution

O.I.1.1 Protocol parsing from RDF serialization. Demonstrate that the expeditor agent can parse RDF-serialized FIPA protocol descriptions at runtime using a pre-programmed state machine that recognizes the interaction protocol ontology structure (cf. Section 4.1.1.2, Section 4.1.1.4).

O.I.1.2 Message exchange on agent platform. Exchange FIPA Agent Communication Language (FIPA-ACL) messages between contractor, subcontractor, supplier, and carrier agents on the agent platform, demonstrating basic message routing and visualization through agent platform debugging tools (cf. Section 4.1.2.1).

O.I.1.3 Scenario-specific protocol handling. Enable the expeditor to interact with supplier (FIPA Query) and carrier (similar protocol) agents in the construction scenario without hard-coding these specific interactions, though limited to protocols conforming to the predefined ontology (cf. Section 4.1.1.3).

O.I.1.4 Report extraction and comparison. Extract order identifiers from RDF-formatted status reports and use them to query related information from supplier and carrier agents, enabling basic cross-validation of delivery information (cf. Section 4.1.1.3).

Design, demonstration, and evaluation. Design of the artefacts and demonstration can be found in Section 4.1. Discussion and evaluation can be found in Section 7.1. This study was originally published and communicated in [7, 1, 2].

1.2.2 Ontology-Driven Software Agents for B2B Purchasing Automation

1.2.2.1 Problem Identification and Motivation

Context. This second demonstration focuses on application-domain ontologies and negotiation processes within B2B purchasing scenarios. In this approach, intelligent services are implemented as software agents representing participants across various business sectors, communicating according to predefined interaction protocols.

The scenario centers on purchasing, where a buyer requires products or services and must negotiate with multiple potential suppliers.

Business pain-point. Even though electronic data interchange and Web-based portals have streamlined many B2B activities, the *purchasing process* itself remains labor-intensive and slow. A buying firm must (i) locate suitable suppliers, (ii) exchange multiple rounds of RFQ messages, and (iii) manually compare heterogeneous quotes—often via e-mail or proprietary portals that provide little semantic structure. These repetitive tasks tie up highly paid personnel, delay production schedules, and raise transaction costs.

Root causes. 1. Syntactic rather than semantic integration. Web-service integration of the early 2000s exchanged messages that machines could route but not *understand*; product descriptions and commercial terms were still interpreted by humans. **2. Static bindings.** Service endpoints are hard-coded through bilateral contracts, and dynamic discovery of alternative suppliers is rare, limiting competition and resilience. **3. Lack of autonomous negotiation.** No common negotiation protocol or decision logic exists that would let software agents filter, rank, or counter-propose quotes without human intervention.

Motivation for a design artefact. If routine steps—supplier discovery, RFQ dissemination, first-round quote evaluation, and simple counter-offers—could be delegated to ontology-driven software agents, buyer personnel would engage only at value-adding decision points. This would shorten purchasing lead-time, widen the supplier pool, and reduce clerical errors. By grounding agent communication in shared domain ontologies and agent interaction protocols, the resulting solution could generalize across industries and interoperate with existing enterprise IT system backends.

1.2.2.2 Research Questions

The following SRQs, which support Main RQ.I.2 (Table 2), guided the investigation of ontology-driven software agents for B2B purchasing automation:

SRQ.I.2.1 How can product and process knowledge in B2B procurement be formalized using domain ontologies to enable machine-interpretable purchasing requests and offers?

SRQ.I.2.2 Which agent communication and coordination mechanisms best support autonomous matching and negotiation based on shared ontological vocabularies?

SRQ.I.2.3 How does ontology-based reasoning contribute to interoperability and automation across heterogeneous enterprise systems?

1.2.2.3 Objectives of the Solution

O.I.2.1 Automate routine purchasing tasks. Enable services that can autonomously discover potential suppliers and conduct preliminary negotiations in the background.

O.I.2.2 Realize automation through software agents. Use software agents that represent buyer and supplier organizations and interact via predefined FIPA-compliant protocols.

O.I.2.3 Maintain human oversight. Let agents handle routine interactions while critical pricing and contractual decisions are escalated to company personnel.

O.I.2.4 Leverage application-domain ontologies. Provide shared semantics for product descriptions and negotiation messages throughout the purchasing process.

Design, demonstration, and evaluation. Design of the artefacts and demonstration can be found in Section 4.2. Discussion and evaluation can be found in Section 7.2. This study was originally published and communicated in [1, 2]; an early conceptual sketch of the B2B scenarios appeared in the ERCIM News note [5] (not peer-reviewed).

1.2.3 Matching Service Descriptions in a Context-Aware Environment

1.2.3.1 Problem Identification and Motivation

Context. Mobile users—such as recreational boaters—frequently switch location, activity, and social setting. The DYNAMOS research project therefore targets *context-aware, hybrid service provision* that integrates: (i) third-party B2C service descriptions, and (ii) C2C user annotations and notes, all expressed through shared ontologies. Traditional Web / service-discovery mechanisms are designed for stationary desktop use and therefore *fail to surface only the services that matter "here & now"*. Consequently, end-users must either (i) sift through irrelevant results or (ii) miss safety- and mission-critical information (e.g., submerged rocks, guest-harbor availability, race checkpoints).

1.2.3.2 Research Questions

Main RQ.I.3 (Table 2): Which end-to-end architecture best supports an intelligent model that fuses context-aware personalization with community-shared annotations of services, so it can operate on resource-limited smartphones and still provide proactive recommendations?

To operationalize the MRQ in these settings, we decompose it into the following focused SRQs, informed by our performance measurements and field-trial feedback:

SRQ.I.3.1 End-to-end bottlenecks. Which pipeline stages dominate latency in practice, and do the results require special technology choices?

SRQ.I.3.2 Field usability and resilience. In high-workload, adverse conditions, which UI and interaction choices preserve proactive value while minimizing attention demand, crashes, and battery drain?

SRQ.I.3.3 Lightweight context ontology. What is the minimal ontology (profiles + time/location/activity) that preserves recommendation relevance while remaining feasible on smartphones and constrained links?

SRQ.I.3.4 First-class collaborative annotations. When user notes/ratings are modeled and matched like services, which matching/indexing and event-distribution choices keep real-time delivery and throughput acceptable?

1.2.3.3 Research Gap

Prior context-aware prototypes focus almost exclusively on *location* and treat service provision as a *Business-to-Consumer* (B2C) pipeline. They ignore two dimensions:

G.I.3.1 Rich, multi-modal context (time, activity, social setting, sensor feeds) that fluctuates on a minute-by-minute scale; and

G.I.3.2 C2C knowledge — ratings, warnings, and free-form notes — which is a primary driver of trust and adoption in mobile scenarios.

1.2.3.4 Objectives of the Solution

Guided by the identified gap, this study pursues the following design objectives:

O.I.3.1 Context Modeling – Create an ontology-based model that unifies relatively static *user profiles* with highly dynamic *user context* (time, location, activity) while remaining lightweight enough for mobile devices.

O.I.3.2 Matching Mechanism with Minimal Resource Overhead – Design and implement an automated matcher that links available service descriptions and user-generated annotations to the current context model, delivering only the most relevant items to the user in real time *while minimizing user attention and energy overhead on smartphones.*

O.I.3.3 Collaborative Annotation Support – Enable end-users to contribute, rate, and share annotations and free-form notes, and ensure that these contributions are themselves treated as first-class, context-matched artefacts.

Motivation for a design artefact. A *hybrid, ontology-driven service-matching mechanism* that blends B2C service descriptions with crowd-sourced C2C annotations *and* aligns them automatically with the user's real-time context would

- increase personal safety and decision speed for sailors and other mobile communities;
- expand the commercial reach of micro-service providers without spamming uninterested users;
- supply researchers with a reusable testbed for evaluating context models, matching algorithms, and event-based distribution at scale.

Design, demonstration, and evaluation. Design of the artefacts and demonstration can be found in Section 4.3. Discussion and evaluation can be found in Section 7.3. This study was originally published and communicated in [2, 8, 3, 4], building on the context-sensitive agent-interaction work of [6].

1.2.4 Phase I from a 2026 perspective

While the Phase I research and demonstrations (2001–2006) (Section 1.1.2) predate the current GenAI revolution by two decades, they identified three fundamental challenges that remain central to the LLM agent deployments of 2026. *Run-time protocol adaptation* (Section 1.2.1) resurfaces as LLM agents invoke tools through OpenAPI specifications, the Model Context Protocol (MCP), and Google's Agent2Agent (A2A) protocol (see Section 2.5), with MCP and A2A now under vendor-neutral Linux Foundation governance [66, 67]: explicit, machine-readable interface descriptions are once again the lingua franca and only the interpreter has changed, while the failure modes have moved from "cannot parse the interface" to "cannot be trusted to act under it". *Ontology-driven service discovery and negotiation* (Section 1.2.2) reappears as context engineering, agent cards, and skill manifests; the tension between semantic precision through explicit ontologies and practical scalability through statistical methods has not been resolved but *relocated*. *Context-aware service matching* (Section 1.2.3) prefigures today's personal agents, whose memory-plus-context split reproduces the Phase I insight that context requires both a static profile and a dynamic state, and whose inference-cost, latency, and interruption budgets are the 2026 form of the field trial's battery, latency, and attention constraints. The open questions of Phase I—who defines the shared vocabulary, how a runtime interpreter is held to it, and how its actions are made accountable—are therefore again the active questions of the field; the full reassessment is given in Appendix A ("The three phases reassessed in 2026").

1.3 Phase II Problem and Motivation: Software Agents and ML Healthcare Item Sets

Research Context. Healthcare organizations face a knowledge-fragmentation problem that undermines clinical decision-making. It manifests through three dimensions that together prevent the use of accumulated clinical experience and patient data. Phase II confronts these dimensions in one concrete, reimbursement-critical instance: DRG coding and clinical documentation support.

Problem Dimension 1: Information System Fragmentation. Modern hospitals operate many legacy information systems—from dozens to hundreds of specialized platforms—each designed for a specific clinical, administrative, or laboratory function. This fragmentation creates data silos in which patient information is distributed across heterogeneous databases with incompatible interfaces and terminologies. The resulting architecture forces clinicians to interrogate multiple systems in sequence to construct a comprehensive patient view, introducing delays, cognitive overhead, and the potential for incomplete retrieval at critical decision points.

Problem Dimension 2: Limited Real-time Clinical Intelligence. Current healthcare information systems offer limited capability for real-time pattern recognition and case-based recommendation. Although hospitals accumulate vast repositories of clinical cases and outcomes, this institutional knowledge remains largely inaccessible for immediate decision support. Clinicians cannot efficiently leverage similar historical cases to inform current treatment decisions.

Problem Dimension 3: Scalability Constraints for Intelligent Systems. While software agent architectures and machine learning techniques have been proposed for healthcare data integration and recommendation systems, their deployment in production hospital environments remains limited. This

implementation gap stems from challenges in achieving real-time performance with large-scale healthcare datasets while maintaining acceptable accuracy levels and operating within the hardware constraints typical of clinical environments.

Theoretical Foundation and Research Gap. Existing approaches typically address individual dimensions in isolation—data integration solutions focus on technical interoperability without intelligent analysis capabilities, while clinical decision support systems operate on limited data sources without comprehensive patient context.

This research identifies a need for an integrated artefact that simultaneously addresses:

- **Real-time heterogeneous data integration** across multiple clinical systems.
- **Intelligent pattern recognition** for case-based recommendation generation.
- **Scalable performance** suitable for production healthcare environments.

What is new here. The constituent methods were previously published and peer-reviewed; the dissertation's contribution is their *synthesis* into one artefact (A4) under consistent terminology and the agents-over-substrates through-line, not a new result; Section 1.6.3 sets out how the synthesis itself constitutes the contribution. A4 addresses the three problem dimensions above through one lens—DRG coding and clinical documentation support, delivered as top-N recommendations over coded episodes (accuracy measured on real hospital data; query latency tested on synthetic databases of up to 10^8 episodes)—so the general decision-support framing and the DRG-specific results describe the same artefact. Concretely, Phase II embeds information-retrieval and recommender machinery (vector-space TF–IDF, cosine similarity, kNN top-N ranking) inside a JADE/FIPA multi-agent system to recast clinical coding and documentation support as top-N recommendation over *coded* DRG/CaseMix episodes. It reaches 53–66% top-1 and ∼90% top-8–10 accuracy with sub-second queries on commodity hardware and graceful degradation under documentation defects, all without modifying the underlying taxonomies. This is an integration over coded healthcare data rather than free-text natural-language processing, and agent-based information retrieval itself long predates it (e.g., SAIRE, 1997 [68]); the neural-infeasibility argument advanced at the time was later partly superseded (Section 1.3.2).

1.3.1 Assisting Clinical Decision Support with ML and Software Agents

1.3.1.1 Problem Identification and Motivation

This research addresses the following design problem: How can an artefact be designed and implemented that provides clinicians with unified, real-time access to comprehensive patient data across heterogeneous information systems while delivering intelligent, case-based treatment recommendations that scale to institutional and national-level data volumes? This problem statement spans two challenges—the technical one of real-time heterogeneous data integration and the analytical one of generating actionable clinical intelligence from large-scale healthcare datasets—both under the practical constraints of production healthcare environments.

1.3.1.2 Research Questions

Main RQ.II.1 (Table 2): How can software agents perform ad-hoc, flexible analysis over large healthcare item sets on commodity hardware under real-world data-quality defects while meeting explicit bounds on latency, memory, and accuracy to support clinical decision-making and hospital funding decisions (DRG coding)?

To operationalize the MRQ in these settings, we decompose it into the following focused SRQs, informed by our performance measurements:

SRQ.II.1.1 Robust ML architectures. What ML/IR model characteristics are essential for agent-based systems operating on large, imperfect healthcare datasets with limited hardware resources?

SRQ.II.1.2 Recommendation accuracy thresholds. What recommendation accuracy levels are achievable with real healthcare data using vector space models in multi-agent architectures?

SRQ.II.1.3 Robustness to data-quality defects. How do realistic documentation defects affect recommendation accuracy, and which retrieval constraints mitigate degradation?

SRQ.II.1.4 Real-time scalability on commodity hardware. What maximum healthcare data volumes can a MAS architecture handle while maintaining sub-second query response times on standard hardware?

1.3.1.3 Objectives of the Solution

The design problem stated above is operationalized through the following specific solution objectives:

O.II.1.1 Robust ML/IR Architecture (Arch)—Design vector space model implementations and kernel-based similarity methods that efficiently handle high-dimensional, sparse healthcare data spaces while leveraging semantic relationships inherent in clinical classification systems (ICD-10, NCSP).

O.II.1.2 Clinically Viable Recommendation Accuracy (Acc)—Achieve recommendation systems delivering >50% first-proposal accuracy and >90% top-10 accuracy for clinical coding suggestions, with flexible fetching strategies that maintain quality under varying data conditions.

O.II.1.3 Robustness to Data-Quality Defects (Rob)—Implement flexible retrieval strategies that maintain recommendation quality when confronted with incomplete, inconsistent, or erroneous healthcare documentation, demonstrating graceful degradation under realistic data defects.

O.II.1.4 Real-time Scalability on Commodity Hardware (Scale)—Develop distributed MAS architecture capable of processing queries against datasets containing up to 10^8 patient episodes within sub-second response times on standard hardware, with linear scaling as data volume and system diversity increase.

Design, demonstration, and evaluation. Design of the artefacts and demonstration can be found in Section 5.1. Discussion and evaluation can be found in Section 7.4. Conclusions can be found in Section 8.3. This study was originally published and communicated in [14], developed from [10, 11, 12, 13].

1.3.2 Phase II from a 2026 perspective

Examining this 2007–2013 research (Section 1.1.2) from a 2026 perspective shows which of its insights held and which were overtaken.

During 2009–2010, I experimented with and demonstrated ontology-aware query-projection methods to enhance healthcare NLP/IR agents. The approach preserved minimal datasets in orthogonal, high-dimensional sparse bases and projected queries through clinical taxonomies (e.g., ICD-10, NCSP, DRG) to improve recall while trying to maintain precision. These results were implemented as demonstration prototypes but were not published in peer-reviewed conferences at the time, and they are not claimed as a contribution of this dissertation. In spirit, this work sits within the long vector-space and distributional tradition of information retrieval and is conceptually related to later formal work on distributional representations such as word2vec (published in 2013) [69] and dual-encoder ("two-tower") retrieval architectures—e.g., DSSM (2013) and large-scale recommendation two-tower models (2016)—in which separate encoders map queries and items into a shared embedding space for efficient matching [70, 71].

By 2026, AI has moved from experimental implementations into routine clinical workflows, but unevenly and with modest measured gains. Ambient documentation ("AI scribes") is the first LLM application to reach standard practice at many health systems, and it is among the first to be evaluated in pragmatic randomized trials in routine practice: a three-arm pragmatic trial of 238 outpatient physicians found reductions in documentation time of at most around ten percent, possible benefits for burnout, and occasional clinically significant inaccuracies in the drafts [72]. Automated coding, the task closest to Phase II, is less settled. Studies published in 2024–2026 report that LLMs used on their own assign ICD-10 and DRG codes with precision well below what coders require [73], while reinforcement-learning-tuned models reach only mid-fifty-percent DRG accuracy on public benchmarks [74]; the configurations that work in practice draft codes with a retrieval or classification model and let an LLM or a human audit them. That draft-and-audit shape resembles the Phase II design—a retrieval substrate proposing top-N candidates with a coordinating layer around it—so the original research's focus on healthcare data fragmentation and real-time clinical intelligence anticipated the problems and, in outline, an architecture that some contemporary implementations use.

The technological foundation has shifted twice since Phase II: the Java Agent DEvelopment Framework (JADE) gave way to cloud-native services, and those in turn are being wrapped, as of 2026, behind standardized agent protocols such as MCP (Section 2.5). An MCP server exposing one hospital system to an agent is the

direct descendant of the Phase II wrapper agent that exposed one legacy database to the JADE platform; the integration problem is the same, and the wrapper has become a commodity.

The research's mathematical analysis demonstrating the infeasibility of neural networks for sparse healthcare data spaces has since been partly superseded by technological advances. Contemporary AI-driven clinical decision support systems employ deep learning architectures, including convolutional neural networks and transformers; I have reflected on how abruptly the Transformer breakthrough overturned such era-bounded expectations in [22]. The 2.8×10^{230} combinatorial complexity that originally justified information retrieval approaches over neural networks is now addressed through embedding techniques, transfer learning, and large-scale pre-trained models that handle sparse, high-dimensional clinical data—the very class of advances taken up in the Phase III work of this thesis (Chapter 6; see also the LLM-agent background in Section 2.5). Read through the dissertation's agents-over-substrates lens, this marks the inflection point of the through-line: in Phase II the measured value came from the information-retrieval and machine-learning substrate, with the software-agent layer serving mainly as orchestration around it. Only once the Transformer made language models capable reasoners did capability move *into* the agent itself—which is why agents become genuinely useful only in Phase III (Chapter 9). The infeasibility argument was, however, superseded only in part: embedding-based nearest-neighbor retrieval over sparse code spaces—the Phase II mechanism—is now the retrieval half of retrieval-augmented systems, so the method survived even though the reason given for preferring it did not.

Yet the problems identified in Phase II persist. Healthcare organizations in 2026 continue to struggle with fragmented information systems, data quality defects, and the need for real-time integration across heterogeneous platforms. The original research's emphasis on performance bounds (sub-second response times, commodity hardware constraints) reappears as the cost and latency budget of LLM inference, now the binding constraint on which clinical tasks can be automated at all. The regulatory environment has also matured: the EU AI Act's obligations for high-risk systems, which include AI used in medical devices, were deferred by the 2026 Digital Omnibus to December 2027 and August 2028 [75]. The deferral lengthens the compliance window without removing it, and the obligations place a premium on precisely the properties Phase II measured—documented accuracy bounds and robustness to data-quality defects.

1.4 Phase III Problem and Motivation: Software Agents in the Generative AI Era

Research Context. The third and most recent phase of this research revisits the concept of software agents after the emergence of LLMs and GenAI. Whereas the earlier phases of the project concentrated on infrastructural concerns and domain-specific data analytics, Phase III shifts the focus toward reimagining agent-based systems through the capabilities offered by LLM-powered multi-agent architectures.

What is new here. The HADA alignment architecture (artefact A5) was presented at a peer-reviewed venue (AGENTICS 2025) prior to this dissertation, so it is consolidated here rather than presented as a new result. What the dissertation foregrounds are the *public releases*: the NordDRG AI Benchmark (A6), an open, rule-complete evaluation suite for hospital-funding logic released to the field with its 2025 papers and consolidated here, and the Collaborative Curated Learning process (A7), the one case the dissertation develops substantially beyond its originating paper into a governed, human-in-command rule-maintenance process. The underlying building blocks—LLM agents, tool use, and AI-alignment patterns—are widely studied; the contribution is therefore architectural and process-level, demonstrated through working prototypes (a banking-sandbox demonstration for HADA and a governed rule-maintenance demonstrator for CCL) and quantitatively validated only for the benchmark, against published gold keys (Grouper: the official grouper output), rather than at production scale.

Phase III at a glance: two harnesses over decision-making models. The three Phase III artefacts are best read not as three separate systems but as governable agency over *decision-making models* (Definition 3.1.17) along two complementary directions (Figure 2). The *capability premise* is logically prior: can a large language model read and faithfully execute a large decision-making model at all? The *NordDRG AI Benchmark* (artefact A6, Section 1.4.2) tests this on one concrete example. On that premise rest two *harnesses*. In *depth*, CCL (artefact A7, Section 1.4.3) is the harness that keeps *one* such large model correct and evolving, with humans and agents together. In *breadth*, HADA (artefact A5, Section 1.4.1) is the harness that deploys distributed agentic decisioning across an organization, over *many* decision-making models, large or small. Both apply one shared governance layer—accountable humans, value constraints, objective

1.4 PHASE III PROBLEM AND MOTIVATION: SOFTWARE AGENTS IN THE GENERATIVE AI ERA

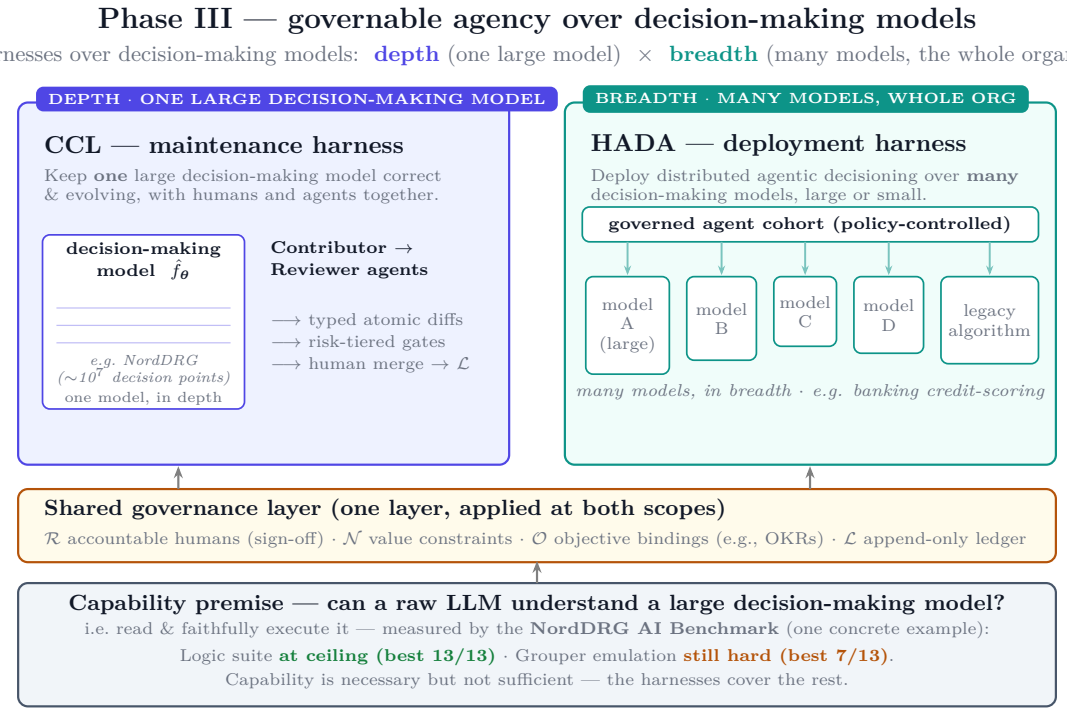


Figure 2: Phase III at a glance: governable agency over decision-making models. The *capability premise* (can a large language model understand a large decision-making model?) is measured by the NordDRG AI Benchmark; on it rest two *harnesses* sharing one governance layer—CCL maintains *one* such large model (depth; rule change) while HADA deploys distributed agentic decisioning across an organization over *many* models (breadth; rule use). NordDRG is one running example. The symbols $\mathcal{R}$, $\mathcal{N}$, $\mathcal{O}$, and $\mathcal{L}$ are those of Definition 3.2.5, under which each agent's control policy Π operates ("policy-controlled"); $\boldsymbol{\theta}$ denotes the parameters (the model of record) of the decision-making model $\hat{f}_{\boldsymbol{\theta}}$ (Definition 3.1.17; Remark A.2).

bindings, and an append-only audit ledger—at their two scopes; together with the capability premise, the three artefacts answer research questions RQ III.1–III.3. NordDRG serves throughout as one running example. The subsections below, like Chapters 6–8, take the artefacts in the numbering order of their research questions (HADA, the benchmark, CCL); the argument they jointly make reads premise, depth, breadth, as in Figure 2.

1.4.1 HADA: Human-AI Agent Decision Alignment Architecture

1.4.1.1 Problem Identification and Motivation

Despite the rapid adoption of LLM agents, most organizations still lack a rigorous, end-to-end mechanism for ensuring that these autonomous components — together with the legacy decision algorithms they increasingly wrap — remain aligned with measurable business targets, codified organizational values, and evolving regulation. The human-in-the-loop control (here in the stronger sense of *human-in-command*: exactly one accountable role per governed activity, which is always human; Definition 3.2.5) and conversational interaction this demands are the objects of recent human–AI decision-making research, which shows empirically that a conversational interface materially shifts user trust in a decision-support system [76] and that conversational explainability raises users' understanding and trust without yielding appropriate reliance, with LLM-powered explanation dialogues even amplifying over-reliance [77] — evidence this work builds on rather than re-establishes.

1.4.1.2 Research Questions

Main RQ.III.1 (Table 2): How could we leverage LLM agents to facilitate human–AI decision alignment in large corporate settings?

Building on this **MRQ**, this study asks the following SRQs:

1.4 PHASE III PROBLEM AND MOTIVATION: SOFTWARE AGENTS IN THE GENERATIVE AI ERA

SRQ.III.1.1 What LLM agent protocol-level gaps hinder human–AI decision alignment in enterprise settings?

SRQ.III.1.2 What integration issues does large-scale LLM-agent integration reveal for human–AI decision alignment?

SRQ.III.1.3 What human-in-the-loop control mechanisms are required in practice to keep decision responsibilities clear?

1.4.1.3 Research Gap

Three gaps underlie the deficit identified above:

G.III.1.1 Alignment Gap — Goal-oriented requirements methods such as Tropos [78, 79] refine objectives into agent designs primarily at design time (requirements-based run-time adaptation exists [80]), but no systematic mechanism binds high-level objectives and ethical principles to the run-time policies of individual LLM agents and algorithms (Section 2.5).

G.III.1.2 Containment Gap — Enterprises lack dependable circuit-breakers and "kill switches" to restrain agents that deviate, self-modify, or hallucinate in production environments (Section 2.5.6).

G.III.1.3 Integration Gap — Practitioners have few vendor-agnostic blueprints for embedding heterogeneous decision algorithms into multi-agent, LLM-powered workflows while preserving end-to-end auditability.

Regulatory pressure (e.g., the EU AI Act), reputational risks from misaligned decisions, and the strategic value of autonomous, conversational systems together motivate the search for a design solution that can close these gaps.

1.4.1.4 Objectives of the Solution

Guided by DSRM, the research sets six design objectives that together define the required capabilities of a Human-AI Agent Decision Alignment (HADA) architecture:

O.III.1.1 Natural-language interaction across planning horizons — Enable every stakeholder (C-suite, risk officers, data scientists, auditors, customers) to steer goals and examine decisions through conversational interfaces, bridging annual strategy and daily operations.

O.III.1.2 Multi-dimensional alignment (targets & values) — Bind each algorithm's optimization logic simultaneously to quantitative *Objectives & Key Results* (OKRs) and to machine-readable organizational values, maintaining a verifiable audit chain from principles to actions.

O.III.1.3 Reference architecture for hierarchical, hybrid AI systems — Provide a modular, protocol-agnostic stack that separates strategy, coordination, and execution layers, allowing LLM agents and classical algorithms to coexist under common governance.

O.III.1.4 Stakeholder alignment across time-scales — Offer repeatable methods for eliciting, reconciling, and propagating stakeholder objectives at yearly, quarterly, and operational cadences.

O.III.1.5 Scalability, auditability, and near-real-time propagation — Support thousands of agents and algorithms, propagate objective or policy changes within hours, and preserve immutable, explainable decision logs suitable for external audit.

O.III.1.6 Framework-agnostic, policy-driven design — Remain independent of any single LLM, agent library, or orchestration tool by exposing open standards so that strategic edits flow to heterogeneous runtimes.

Design, demonstration, and evaluation. Design of the artefacts and demonstration can be found in Section 6.1. Discussion and evaluation can be found in Section 7.5. Conclusions can be found in Section 8.4. This study was published and communicated in [28], building on the RAGADA and AADAR architectures [26, 27], of which I am first author, and on the management-seed papers [18, 19], where I served as a supporting co-author.

1.4 PHASE III PROBLEM AND MOTIVATION: SOFTWARE AGENTS IN THE GENERATIVE AI ERA

1.4.2 NordDRG AI Benchmark: Teaching LLMs the Nuances of Hospital Funding Instruments

1.4.2.1 Problem Identification and Motivation

LLMs are increasingly piloted for clinical coding and decision support, yet almost no open evaluation targets the *hospital-funding* layer where Diagnosis-Related Groups (DRGs) determine reimbursement. The stakes are macroeconomic: global health spending is on the order of US$10 trillion annually, with hospitals absorbing the largest share [81]; in DRG-based systems a sizeable portion of these flows is routed through governed grouper software. Transparency and auditability are therefore first-order requirements, not implementation details.

In most OECD settings, a DRG grouper is a governed decision-making model (Definition 3.1.17) that maps a patient episode's minimum dataset to a single payment group via an ordered rule graph with explicit inclusions, exclusions, and national activation flags. NordDRG makes this logic inspectable through $\sim$20 interlinked definition tables released annually alongside expert manuals and change templates. For non-specialists, the rule graph remains opaque; for experts, verifying end-to-end control flow and its governance trace is laborious.

This line of inquiry began in 2024: to the author's knowledge, the PCSI 2024 and CHIRA 2024 studies [20, 21] were the first to probe whether LLMs understand CaseMix (DRG) specifications, posing questions about the rule tables one at a time. During 2025, frontier models reached the point of answering such single questions efficiently—the question-level tasks no longer separated the strongest models—and that observed capability gain motivated extending the benchmark to *full grouper emulation* [24] (the initial arXiv version of [23] contained only question-level tasks). The open question moved from whether a model can read a rule to whether it can execute the specification's entire control flow.

1.4.2.2 Research Questions

Main RQ.III.2 (Table 2): To what extent can LLMs emulate, explain, and apply DRG-based hospital payment rules?

The following questions support RQ III.2 and align one-to-one with the objectives of the solution:

SRQ.III.2.1 Artefact design. What *minimal, versioned* artefact bundle and schema are necessary and sufficient to encode the complete NordDRG rule graph and its governance semantics in a public, machine-readable form?

SRQ.III.2.2 Benchmark operationalization. How should tasks, inputs, and exact-match scoring be structured so that (i) the *Logic* suite measures rule-level reasoning over the definition tables and manuals, and (ii) the *Grouper* suite enforces *full emulation* of the official control flow—returning both `drg_nat` and the triggering `drg_logic.id`?

SRQ.III.2.3 Baseline capability and failure modes. When constrained to the released artefacts (no external web access), how do representative LLM endpoints perform on the *Logic* and *Grouper* suites, and which systematic errors account for misses (e.g., cross-table joins, `ORD` priority, CC/MCC exclusions, national activation)?

1.4.2.3 Research Gap

Generic LLM leaderboards neither encode this rule graph nor enforce the audit trail that payers require. This yields two persistent gaps:

G.III.2.1 Scope Gap — the absence of a public, *rule-complete*, machine-readable artefact that bundles the full NordDRG specification (tables *and* governance documents) with stable identifiers.

G.III.2.2 Benchmark Gap — the absence of a standardized, *governance-grade* benchmark that tests both rule-level CaseMix reasoning and faithful emulation of the grouper's control flow with strict, auditable outputs (DRG and triggering `drg_logic.id`).

As a result, the community lacks a reproducible yardstick for measuring whether LLMs can read the DRG rule graph, apply governed control flow, and return decisions traceable to specific rule rows.

1.4.2.4 Objectives of the Solution

Guided by the DSRM, this work translates the above gaps into a rule-complete artefact and two evaluation suites with exact-match scoring.

O.III.2.1 Artefact construction *(addresses the Scope Gap; DSRM: Design & Development).* Release an open, machine-readable, versioned benchmark (*NordDRG AI Benchmark*) that captures the *complete* NordDRG rule graph—definition tables, diagnosis/procedure property tables, age/sex splits, national activation flags, and governance manuals—with schema-stable identifiers and modular packaging for reuse and extension (see Section 6.2.1).

O.III.2.2 Benchmark operationalization *(bridges both gaps; DSRM: Demonstration).* Instantiate two complementary test suites with exact-match scoring: (i) the *NordDRG Logic Benchmark*—13 automatically verifiable rule-level questions reflecting coder, clinician, and policy-analyst workflows, with public answer keys for exact-match scoring (Section 6.2.1.3); and (ii) the *NordDRG Grouper Benchmark*—13 case-level tasks that require faithful execution of the specification's control flow and return both `drg_nat` and the triggering `drg_logic.id` (Section 6.2.1.4).

O.III.2.3 Baseline evaluation *(targets the Benchmark Gap; DSRM: Evaluation).* Provide first-cut, fully reproducible results for representative state-of-the-art LLM endpoints across providers, including prompts, context bundles, and model outputs — establishing the initial yardstick for head-to-head comparisons and longitudinal tracking in hospital-funding contexts (results in Section 6.2.2; evaluated in Section 7.6).

Design, demonstration, and evaluation. Design of the artefacts and demonstration can be found in Section 6.2. Discussion and evaluation can be found in Section 7.6. Conclusions can be found in Section 8.5. This study was published and communicated in [23, 24], with earlier work appearing in [20, 21]; the CHIRA 2025 paper (9th CHIRA Conference, Marbella, Spain, October 2025) received the conference's second-best-paper recognition.

1.4.3 CCL: How Humans and AI Agents Align on Decision-Making Rules

1.4.3.1 Problem Identification and Motivation

Context. In safety-critical domains (healthcare, finance, public policy), the decision-making model $\hat{f}_{\boldsymbol{\theta}} : \mathcal{X} \to \mathcal{Y}$ (Definition 3.1.17) is often implemented as an *explicit, human-auditable* artefact—rule bases, scorecards, or multi-objective algorithms—so that every transformation step can be inspected and justified. NordDRG, whose hand-curated specification unrolls path-wise to tens of millions of decision points (Table 19), exemplifies this approach. The very safeguards that make such logic trustworthy—line-by-line review, change control, traceable audit trails, and formal sign-off—also slow release cadence to a handful of updates per year, while coding standards, clinical practice, and policy evolve continuously. The result is policy–practice drift and accumulating maintenance debt.

Problem statement.

> *When AI agents are introduced into regulated decision systems, there is no governed, scalable workflow that allows those agents to propose fine-grained edits for expert curation and ratification—end to end with an auditable trail—while preserving the accountability required by governance frameworks.*

Scope. This problem statement applies to AI-augmented workflows; it does not claim that governed human-only workflows are unavailable.

Illustrative challenges of introducing AI agents into the NordDRG development cycle.

1 **Release cadence constraints.** Annual release trains remain the norm; under current QA and sign-off, continuous delivery of AI-assisted micro-edits is difficult in practice. This study acknowledges that many processes—such as cost-weight calculation and budgetary allocation—are not amenable to continuous delivery, but the focus here is on the decision logic itself.

2 **Scale and cognitive load.** As the rule base expands, this study observes difficulties with conflict resolution, and onboarding remains hard because of substantial context dependencies.

3 **Limited agent–expert loop.** Across the tools and processes this study had access to, no production-grade, bidirectional workflow was observed in which AI agents propose well-typed diffs and certified experts curate and ratify them with guardrails and auditability.

4 **Opacity of AI suggestions.** Off-the-shelf ML/LLM outputs this study examined or piloted tended to lack line-level provenance, counterfactual evidence, and legally admissible documentation typically required for acceptance.

Scope and basis of observations. These points pertain only to AI-augmented workflows. They synthesize the author's limited interactions with NordDRG stakeholders across selected release cycles (e.g., participation in some specification review discussions and a small set of change-ticket exchanges), complemented by analysis of definition tables and publicly available change artefacts, and several semi-structured conversations. They are offered with respect for maintainers' expertise and constraints, as indicative observations rather than comprehensive claims about all NordDRG practices.

Motivation for research and practice. Addressing these challenges would (i) operationalize CCL as a governed development process for decision logic and (ii) yield a template transferable to other safety-critical domains where algorithmic decisions must remain both explainable and current (Section 7.7.4).

1.4.3.2 Research Questions

Main RQ.III.3 (Table 2): What end-to-end governance requirements and workflow properties enable AI agents to contribute to decision-model development without eroding human expert accountability?

Building on this **MRQ**, the research questions of this study are articulated at the *process* level. They derive from the gap analysis (Section 1.4.3.3) and map directly onto the design objectives (Section 1.4.3.4); this study asks:

SRQ.III.3.1 How should a decision model on the order of $\approx 10^7$ decision points, like NordDRG, best be represented for AI agents to support safe, atomic diffs?

SRQ.III.3.2 Under which constraints do LLM agents produce reviewer-acceptable change proposals for decision models at $\approx 10^7$-point scale?

The main research question is intentionally framed to test *any* governed end-to-end workflow that enables agent contributions while preserving expert accountability; the CCL process introduced later is evaluated as one candidate instantiation rather than presupposed by the question. The "AI agents" of the question are LLM agents in the sense of Definition 3.2.1; the Contributor and Reviewer agents of O.III.3.3 are their instances.

1.4.3.3 Research Gap

A review of the XAI, MLOps, and health-informatics literature (Section 2.6) finds *no governance-grade, end-to-end framework* that allows AI agents and certified experts to co-maintain a safety-critical decision model at the scale of NordDRG. Four gaps motivate this study:

G.III.3.1 Governed co-development loop is missing. Conventional ML practice typically positions human validation as a terminal "approval step," the pattern interactive machine learning set out to change [82]; even interpretability research focuses on making a finished model inspectable [83], rather than embedding experts in an *iterative* proposer–reviewer cycle where agents submit fine-grained diffs and experts curate them under explicit roles and gates.

G.III.3.2 Change-level auditability is insufficient. Popular explainability methods justify a *static* model post hoc [84, 85], but rarely capture the lineage, test evidence, and reviewer sign-offs needed to defend each micro-update to the decision logic.

G.III.3.3 Software-engineering mechanics are underused in AI governance. Version control, pull requests, and automated test suites are standard in code and ML development [86], yet are seldom operationalized in human–AI governance of decision rules.

G.III.3.4 Empirical evidence at operational scale is scarce. Published case studies do not reach NordDRG scale: NordDRG spans $\approx 10^7$ decision points when its full specification is unrolled path-wise (Table 19), outscaling the largest documented human-in-the-loop experiments by orders of magnitude and leaving scalability claims untested.

The closest prior art falls short on exactly these axes rather than being absent. Work on ontology evolution [87], including approaches that predict *where* an expert-governed (e.g., biomedical) ontology will next be extended from its real version history, focuses human effort but does not collaboratively *produce* the change under governance. Approaches that replay a large real system's change history—as in database schema evolution [88]—are automated and lack human-in-command authority; and agentic ontology curation, a further related direction, has to the author's knowledge not been reported with a governed, human-in-command loop or with review against real historical changes at this scale. To the best of the author's knowledge, CCL's novelty is therefore the *conjunction*—governed, human-in-command co-maintenance, review against real expert changes, and a regulated decision model on the order of $\approx 10^7$ decision points—rather than any single ingredient.

This study addresses the gap by specifying a governed co-maintenance process (CCL; Section 6.3.1.3), providing an agent-operable model-of-record for NordDRG (Section 6.3.1.4), and demonstrating agent roles within that process (Section 6.3.2), with evaluation in Section 7.7.

1.4.3.4 Objectives of the Solution

To answer the research questions (Section 1.4.3.2), this study defines three objectives for the solution. Each objective notes where it is specified, instantiated, and evaluated.

O.III.3.1 Governed workflow. Operationalize maintenance-time alignment by specifying the *governance requirements and workflow properties* (roles, risk tiers, CI gates, immutable ledger, human-in-command) that enable agent contributions without eroding expert accountability. Specified in Section 6.3.1.3, instantiated in Section 6.3.2.1, evaluated in Section 7.7.3.

O.III.3.2 Agent-operable model-of-record. Provide a line-addressable representation for decision models on the order of $\approx 10^7$ *decision points*, enabling safe, atomic diffs and reviewer-legible slices. Specified in Section 6.3.1.4, instantiated in Section 6.3.2.3, evaluated in Sections 7.7.1 and 7.7.2.

O.III.3.3 LLM agents under constraints. Implement Contributor/Reviewer agents under explicit *constraints* (grounding, role restrictions, gate policy) and measure reviewer acceptance (whether a proposed change reproduces the expert change implemented in NDMS for the same ticket, the reference standard); CI pass rates, error severity, and reviewer effort are specified as further measures for a full deployment and were not collected in the demonstration (Section 7.7.3). Specified in Section 6.3.1.5, instantiated in Section 6.3.2.4, evaluated in Section 7.7.3.

Design, demonstration, and evaluation. Design of the artefacts and demonstration can be found in Section 6.3. The CCL Platform, a full-stack web application implementing these artefacts, is described in Section 6.3.2.1. Discussion and evaluation can be found in Section 7.7. Conclusions can be found in Section 8.6. This study—the decision-tree analysis of NordDRG and the curated-learning process with its evaluation—was published in [29] as a peer-reviewed article at the 37th PCSI Conference (Quebec City, September 2025), where it received the conference's best-paper award; its preliminary counts (about 58,000 DRG-logic-level nodes, and about 2,000,000 nodes with the primary classification level) are replaced here by the decision-point counts of Table 19, computed as defined in Section 6.3.2.2, and this thesis gives the full account of the CCL process, platform, and evaluation.

1.4.4 Phase III from a 2026 perspective

The Phase III studies were carried out in 2023–2025, so a 2026 reassessment covers a short interval. It is nonetheless necessary, because the substrate under the three artefacts—frontier models and the agent harnesses around them—changed faster during 2025–2026 than during any earlier phase of this dissertation: coding agents matured into autonomous workers [89, 90], *harness engineering* became an industry term for the idea this dissertation applies in CCL and HADA [90], and the interface layer standardized under neutral governance (Section 1.2.4). For the NordDRG AI Benchmark (A6), the mid-2025 results stand as reported: the strongest models mastered the rule-level *Logic* suite (13/13) while exact, traceable emulation of the grouper's control flow stayed hard (best 7/13; Section 7.6). They now motivate a coding-agent track that would ask an agent to *construct* executable grouper software from the released artefacts alone, with the scoring cases held out and the agent kept off the public Web, where open grouper implementations would contaminate the result. In my assessment, CCL (A7) has become easier to instantiate, since 2026 coding agents ship most of what the CCL Platform had to build, but its oracle, representation, and accountability problems remain. Of HADA's (A5) three gaps, the Integration Gap has narrowed most, while the Alignment Gap is essentially untouched by the standards work. The 2026 substrate overturns none of the Phase III

claims, which were deliberately scoped to what the demonstrators showed (Section 7.8). The full reassessment is given in Appendix A ("The three phases reassessed in 2026").

1.5 Main Research Questions

The main research questions and objectives of the thesis are summarized in Table 2. The research is divided into the three phases introduced in Section 1.1.2 (see Table 1), each corresponding to a distinct period with its own research questions. Table 3 then maps each original contribution (**C1**–**C8**, summarized in Section 1.6.3 and set out in full in Appendix A) to the research question it answers and the artefact that delivers it. Table 4 completes the chain in the opposite direction: for each research question it names the artefact(s) that answer it, the evaluation outcome obtained, the level of evidence behind that outcome, and the publication(s) reporting it—distinguishing quantitatively validated results from prototype-level demonstrations (developed in full in Section 7.8).

Table 2: Main Research Questions by phase.

Phase	MRQ #	Main Research Question
I	I.1	What are the historical origins of the Web and of software agents?
	I.2	To what extent can Semantic Web formalisms and software-agent protocols be integrated to deliver runtime interoperability and adaptation on the Web, and what irreducible gaps prevent practical convergence?
	I.3	Which end-to-end architecture best supports an intelligent model that fuses context-aware personalization with community-shared annotations of services, so it can operate on resource-limited smartphones and still provide proactive recommendations?
II	II.1	How can software agents perform ad-hoc, flexible analysis over large healthcare item sets on commodity hardware under real-world data-quality defects while meeting explicit bounds on latency, memory, and accuracy to support clinical decision-making and hospital funding decisions (DRG coding)?
III	III.1	How could we leverage LLM agents to facilitate human–AI decision alignment in large corporate settings?
	III.2	To what extent can LLMs emulate, explain, and apply DRG-based hospital payment rules?
	III.3	What end-to-end governance requirements and workflow properties enable AI agents to contribute to decision-model development without eroding human expert accountability?

1.6 PhD Artefacts

1.6.1 Main Artefacts

As summarized in Table 5, this dissertation produced a portfolio of seven main design artefacts that map to the three research phases. Phase I delivers the agent-level innovations—run-time protocol adaptation, ontology-driven negotiation, and context-aware mobile service matching—that probe the limits of early Semantic Web tooling. Phase II contributes a large-scale, machine-learning–enabled clinical recommendation system, demonstrating that statistical retrieval can extend symbolic hierarchies without redesigning them. Phase III contributes three artefacts: the NordDRG AI Benchmark, to the author's knowledge the first public evaluation suite for hospital funding logic (which tests the capability premise), the CCL process enabling governed human–AI co-maintenance of safety-critical decision-making models (the depth harness), and the HADA alignment architecture for translating organizational values into auditable multi-agent controls (the breadth harness), the two harnesses sharing one governance layer (Figure 2). Taken as a whole, the artefact set illustrates the thesis's longitudinal claim: the software-agent abstraction remains a viable design unit across successive AI paradigms while progressively tightening the link between transparency, governance, and deployability in regulated settings (demonstrated in sandbox and benchmark settings).

Candidate's role. To make the candidate's individual contribution unambiguous in the co-authored work, Table 23 (Appendix A) states his role per main artefact and names the lead publication.

Table 3: Original contributions mapped to the research questions they answer. Each contribution (**C1**–**C8**) is summarized in Section 1.6.3 and developed in full in Appendix A; artefact codes **A1**–**A7** follow Table 5.

C#	Phase	Artefact	RQ	What it answers
C1	I	A1	I.2	Run-time interaction-protocol adaptation: the contractor's agent drives interactions from RDF/agent protocol specifications instead of hard-coded workflows (one adaptive role, fixed protocol ontology).
C2	I	A2	I.2	Ontology-driven B2B negotiation: a Contract-Net multi-agent prototype automating RFQ, one quote-revision round, and contract closure within personnel-set limits.
C3	I	A3	I.3	DYNAMOS context-aware service matcher: proactive, context-aware recommendations on resource-limited smartphones (three objectives, one partially).
C4	II	A4	II.1	ML/IR clinical recommendation multi-agent system over coded episodes: 53–66% top-1 / ∼90% top-8–10 accuracy on real hospital data; sub-second on commodity hardware on synthetic databases of up to 10^8 episodes.
C5	III	A5	III.1, III.3	HADA human–AI decision-alignment architecture: met six design objectives (two partially) in a banking sandbox.
C6	III	A6	III.2	NordDRG AI Benchmark: to the author's knowledge the first public hospital-funding-logic suite; top models master rule-level Logic while exact grouper emulation stays hard.
C7	III	A7	III.3	CCL process: typed atomic diffs, risk-tiered CI gates, and an immutable ledger keep rule maintenance human-in-command.
C8	I–III	—	cross-phase	Cross-phase synthesis: the longitudinal through-line binding the 2001–2025 work into one cumulative argument.

1.6.2 Individual Artefacts per Phase

The detailed per-phase breakdown of the individual artefacts and their mapping to the seven main artefacts (Table 5) is collected in Appendix A: Phase I in Table 24, Phase II in Table 25, and Phase III in Table 26.

1.6.3 Original Contributions

The original contribution of this dissertation is not new empirical results—the technical and empirical findings were, with few exceptions, *previously published and, in most cases, peer-reviewed*—but their *synthesis* into a single analytical scheme: the seven main artefacts (Table 5), the research questions (Table 2), and one longitudinal through-line, together with the cross-phase argument and the public artefact releases.

Concretely, the synthesis yields *eight contributions* (**C1**–**C8**), best read *by phase and as a single dependency line* rather than as eight independent results [91]. They comprise the Phase I agent and Semantic Web framing—run-time protocol adaptation, ontology-driven negotiation, and context-aware service matching (**C1**–**C3**); the Phase II pivot to data-centric machine learning over large coded datasets (**C4**); the Phase III governed-LLM architectures and, to the author's knowledge, the first public benchmark for hospital-funding (DRG grouping) logic—HADA, the NordDRG AI Benchmark, and the CCL maintenance process (**C5**–**C7**); and the cross-phase synthesis that binds them into one cumulative argument (**C8**). Each contribution names its primary publication, the predecessor work it develops, the candidate's role, and what is new *in this dissertation*; the full per-contribution detail is collected in Appendix A ("Original contributions in detail"), and Table 3 gives the contribution-to-question mapping.

Scope and non-claims. To keep these contribution claims calibrated, three boundaries deserve explicit statement up front; each is examined in depth in the cross-phase threats analysis (Section 7.8). First, HADA and CCL are proof-of-concept governance patterns demonstrated in sandbox settings; no claim of production readiness is made. Second, the NordDRG AI Benchmark certifies hospital-funding (CaseMix/DRG) rule reasoning; no generality beyond that domain is claimed. Third, the Phase I and Phase II artefacts are demonstrations of feasibility under their eras' constraints, not systematic controlled trials.

1.7 Summary: Problem Identification and Objectives

Context. This chapter frames the three phases (2001–2006, 2007–2013, 2023–2025) as one cumulative argument, with software agents as the architectural through-line and governable agency as the unifying lens (Section 1.1.2), and states each phase's problem.

Problem. The core problem is governable agency across changing substrates: how agent behavior can be kept interoperable, auditable, and aligned with human command as the substrate shifts from ontologies and protocols, through statistical models, to large language models.

Research questions. The chapter defines seven main research questions with study-level sub-questions (Table 2), specifying what must be demonstrated through artefact design and evaluation.

Objectives. The problem is decomposed into per-study solution objectives; where a measurable target exists (e.g., O.II.1.2) it is stated, and the level of evidence obtained for each research question (demonstration for most artefacts; quantitative validation where a reference exists) is summarized in Table 4.

Scope and assumptions. The scope and non-claims are stated in Section 1.6.3, and their consequences for validity are examined in Section 7.8.

Transition. The next chapter, Background (Chapter 2), provides the theoretical and technical foundations for these objectives.

1.8 The Structure of the Thesis

The rest of this thesis is organized as follows (Table 1 links the DSRP process sequence to the phases and to this structure):

Chapter 2 – Provides background information relevant to the thesis and the upcoming chapters.

Chapter 3 – Collects the formal definitions that underpin the analytical framework of the thesis: deep learning, the decision-making model (Definition 3.1.17), the Transformer architecture, LLM agents, and governed LLM agents (Definition 3.2.5).

Chapter 4 – Describes the design, development, and demonstrations of Phase I.

Chapter 5 – Describes the design, development, and demonstrations of Phase II.

Chapter 6 – Describes the design, development, and demonstrations of Phase III.

Chapter 7 – Discusses and evaluates the design, development, and demonstrations from Phases I, II, and III.

Chapter 8 – Summarizes the main findings and presents the overall conclusions of the thesis.

Chapter 9 – Illustrates hands-on why software agents have become practical only now.

Appendix A – Collects supplementary figures and tables, the per-phase artefact breakdowns, the original contributions in detail, the 2026 reassessment of Phases I and III, and the Epilogue's agent prompts.

Table 4: Research questions mapped to the artefact(s) that answer them, the evaluation outcome and its level of evidence, and the publication(s) reporting that result. Artefact codes **A1**–**A7** follow Table 5; the evidence levels are developed in full in Section 7.8.

RQ	Artefact	Evaluation outcome (evidence level)	Publication(s)	Evaluated in
I.1	—	Historical origins of the Web and of software agents synthesized in Chapter 2, showing their convergence in contemporary AI *(literature synthesis; contextual question, no design artefact).*	[25, 1, 2]	Section 8.1
I.2	A1, A2	RDF-driven run-time protocol adaptation and ontology-driven B2B negotiation shown feasible (A2: routine automation partial, human oversight not demonstrated); syntactic interoperability achieved, but the agent tooling used here and Web standards did not converge *(demonstration: peer-reviewed prototype for A1; Licentiate-thesis prototype for A2).*	[7, 2, 5]	Sections 7.1 and 7.2
I.3	A3	DYNAMOS delivered proactive, context-aware service recommendations on resource-limited smartphones, with O.I.3.1 and O.I.3.3 achieved and O.I.3.2 (resource-efficient matching) only partially achieved because efficiency was brittle on the mobile clients (Aug. 2005 field demonstration, reported in [2] and Section 4.3.2.4) *(demonstration: peer-reviewed prototype design; field trial reported in the Licentiate thesis).*	[3, 4, 2]	Section 7.3
II.1	A4	ML/IR clinical recommendation over coded episodes: 53–66% top-1 / ∼90% top-8–10 accuracy on real hospital data (hundreds of thousands of episodes); sub-second queries on synthetic databases of up to 10^8 episodes on commodity hardware; robust to documentation defects *(quantitative: measured against historical coded documentation, with ground-truth caveats).*	[14]	Section 7.4
III.1	A5	HADA met its six predefined design objectives O.III.1.1–O.III.1.6 (natural-language interaction, targets & values alignment, reference architecture, stakeholder alignment across time-scales, scalability & auditability, framework agnosticism; Section 1.4.1.4), two of them (O.III.1.4, O.III.1.5) only partially, in a banking sandbox *(demonstration: no comparable governed baseline exists).*	[28, 26, 27]	Section 7.5
III.2	A6	NordDRG AI Benchmark: top models master rule-level Logic (best 13/13) while exact grouper emulation stays hard (best 7/13), scored by exact match against published gold keys (Grouper: the official grouper's decision and rule row) *(quantitative: baseline-anchored, single-run snapshot; the strongest empirical evidence in the thesis).*	[24] (precursors: [21, 23])	Section 7.6
III.3	A7, A5	CCL governs regulated rule changes through typed atomic diffs, risk-tiered CI gates, and an immutable ledger, keeping merges human-in-command (A5 contributes the role-bound human sign-off, $\mathcal{R}$; Section 7.5); judged against the expert change implemented in the NordDRG Maintenance System (NDMS) for each of five tickets and compared against a single-agent workflow *(demonstration against a reference standard plus ablation: single run per cell, one non-blinded reviewer, gates applied by manual review; no baseline exists for the governance process).*	[29]	Section 7.7

Table 5: Main artefacts created across the three research phases.

Phase	Years	#	Main Artefact	Purpose / Contribution
I	2001–2006	A1	Run-time interaction-protocol adaptation engine.	Dynamically retrieves RDF/agent protocol specifications and drives interactions from them instead of hard-coded workflows; demonstrated for one role within a fixed protocol ontology. Problem: Section 1.2.1, Design and Demonstration: Section 4.1, Artefacts: Section 4.1.1
		A2	Ontology-driven B2B negotiation prototype.	Contract-Net multi-agent demonstrator that automates RFQ, one quote-revision round, and contract closure within personnel-set limits in supply-chain scenarios. Problem: Section 1.2.2, Design and Demonstration: Section 4.2, Artefacts: Section 4.2.1
		A3	*DYNAMOS* context-aware service matcher.	Intelligent mobile service matching that fuses B2C descriptions with C2C annotations via shared ontologies and real-time context. Problem: Section 1.2.3, Design and Demonstration: Section 4.3, Artefacts: Section 4.3.1
II	2007–2013	A4	ML-enabled clinical recommendation MAS.	Crawler–integrator–commander–aggregator pipeline; IR vector engine imputes missing medical codes over coded patient episodes on commodity hardware (latency tested on synthetic databases of up to 10^8 episodes). Problem: Section 1.3, Design and Demonstration: Section 5.1, Artefacts: Section 5.1.1
III	2023–2025	A5	*HADA* architecture.	A protocol-agnostic, multi-agent reference architecture that translates organizational OKRs/values into auditable, vendor-neutral controls by orchestrating stakeholder agents over LLMs and legacy decision logic (demonstrated on retail-bank credit scoring). Problem: Section 1.4.1, Design and Demonstration: Section 6.1, Artefacts: Section 6.1.1
		A6	*NordDRG AI Benchmark*.	To the author's knowledge, the first public, open hospital-funding-logic AI benchmark for LLMs uniting the Nordic DRG definition tables with 13 CaseMix logic tasks (Logic-1–Logic-13) and 13 CaseMix grouping tasks (Grouper-1–Grouper-13). Problem: Section 1.4.2, Design and Demonstration: Section 6.2, Artefacts: Section 6.2.1
		A7	*CCL* process.	Reference development process for automated decision-making logic enabling human–AI alignment and collaboration in maintenance of decision rules. Problem: Section 1.4.3, Design and Demonstration: Section 6.3, Artefacts: Section 6.3.1

2 BACKGROUND

This chapter provides essential background for the subsequent chapters of this thesis. The history and evolution of AI, software agents, the World Wide Web, and the Semantic Web are presented and discussed.

This chapter is structured as follows:

- Section 2.1 presents an overview of the technological hype cycles, highlighting the recurring patterns of "AI summers" and "AI winters". Parts of this section have been published in [25].
- Section 2.2 outlines the foundational concepts and historical development of the Web. Parts of this section have been published in [1, 2].
- Section 2.3 explores the concept of the Semantic Web, its original vision, and its relationship with AI-driven Software Agents. Parts of this section have been published in [25].
- Section 2.4 provides an overview of Software Agents and MAS. Parts of this section have been published in [1, 2].
- Section 2.5 introduces emerging technologies related to LLM-based agents. Parts of this section have been published in [23, 24, 25, 26, 27, 28].
- Section 2.6 reviews LLM, alignment, benchmark, version-control, MLOps, and DRG studies relevant to Phase III. Parts of this section have been published in [24].

2.1 A Short History of AI: AI Summers and Winters

This section traces AI's history through its boom-and-bust cycles to ground a claim developed below: the early-2000s Semantic Web research, though rarely classified as AI, belongs in that history. The notion of AI as a distinct discipline within Computer Science was introduced in 1956 at a workshop at Dartmouth College by early pioneers such as John McCarthy, Marvin Minsky, and others [92]. Since then, the history of AI has experienced recurring cycles of enthusiasm and subsequent disillusionment, commonly referred to as "AI summers" and "AI winters" [92, 93]. An AI winter is a period of reduced funding, interest, and progress in AI research, typically following unmet expectations or technological limitations; an AI summer, by contrast, is a phase of heightened enthusiasm, investment, and innovation, often driven by breakthroughs in algorithms, computing power, or applications. These cycles are associated with two main approaches to creating AI: symbolic ("white box") approaches, based on explicit knowledge representation, logic, reasoning, and search, and sub-symbolic ("black box") approaches, based on neural networks, ML, and statistical models.

Following the first AI summer in the 1960s, the first AI winter occurred in the 1970s after [94] proved fundamental limitations of single-layer perceptrons (e.g., parity and connectedness) and cast doubt on multi-layer extensions, a result widely read as a verdict against neural networks. As a response, the symbolic approach, including knowledge-based expert systems and logic programming, fueled the next AI summer in the 1980s. However, challenges in, e.g., knowledge representation and acquisition [95] soon led to another AI winter. Research on multi-layer sub-symbolic neural models and learning systems continued despite—or perhaps because of—the shortcomings of the knowledge-based approach. By expanding ML models through deep learning, employing new training algorithms such as backpropagation, introducing novel neural architectures and nonlinear activation functions, leveraging Big Data from the Web and available databases for training, and using new efficient AI chips for parallel neural computations, AI has entered an unprecedented period of advancement [96] that has accelerated in the 2020s [51]. A prominent research topic today, aimed at addressing challenges in neural systems such as hallucinations, is the combination of symbolic and sub-symbolic approaches in neuro-symbolic (hybrid) AI systems [97, 98].

The early 2000s are often labeled an AI winter, with little recognition of the relevant AI research conducted then. Yet this narrative overlooks significant developments of the era, including the emergence of the Semantic Web, which gained momentum in 2001 [64, 99]. Although rooted in knowledge representation, logic, reasoning, and ontologies—core subjects of traditional symbolic AI—this branch of research is typically not classified as part of AI. For example, in the authoritative ACM Computing Classification System[3] of Computer Science, AI is a major first-level category under Computing Methodologies (*Computing Methodologies* > *Artificial Intelligence*), while the Semantic Web is mentioned only in a minor category under Information Systems (*Information Systems* > *World Wide Web* > *Web data description languages* > *Semantic Web description languages*); see Figure 3.

[3]ACM Computing Classification System: `https://dl.acm.org/ccs`

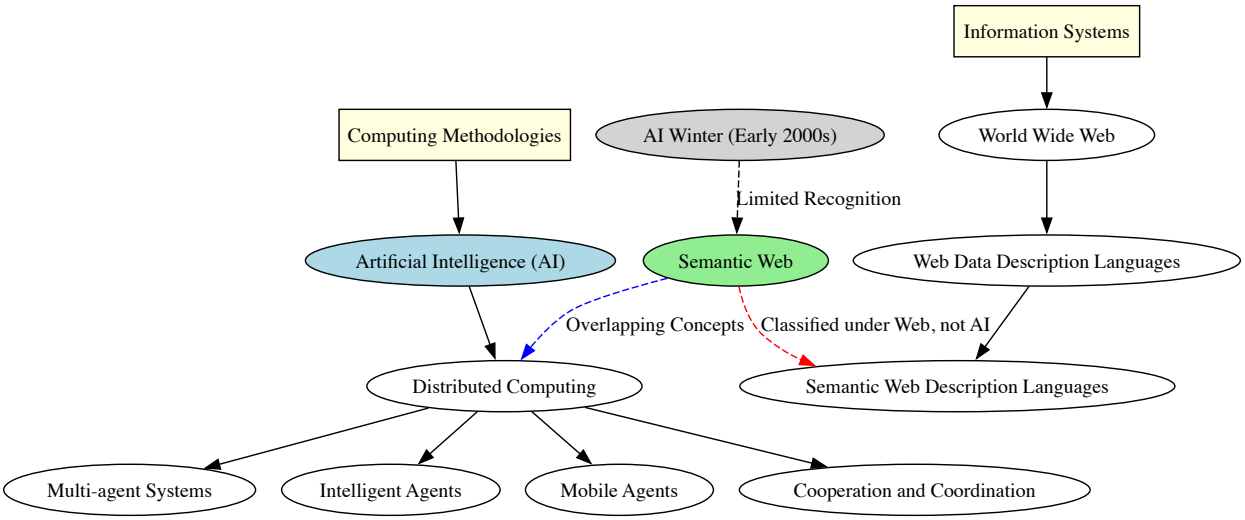


Figure 3: Categorization issue: the Semantic Web research vision had a strong AI focus, but it is categorized under the WWW branch.

According to the original Semantic Web vision outlined in 2001 [64], the Semantic Web is centered around intelligent agents on the Web, yet Intelligent Agents appear only as a minor subcategory under AI (*Computing Methodologies* > *Artificial Intelligence* > *Distributed artificial intelligence* > *Intelligent agents*, alongside Multi-agent systems (MAS) and Mobile agents; Figures 3 and 5).

Even comprehensive reviews of the Semantic Web [100], while acknowledging its overlap with AI and its agent-centered origins, foreground data-interoperability technologies (RDF, OWL, SPARQL) and consolidation challenges; AI-history accounts, in turn, rarely include this research (with one exception noted in Section 2.1.2).

The rapid advancement of GenAI and the widespread adoption of LLMs were catalyzed by the Transformer architecture, which enabled more efficient training and superior contextual understanding in deep learning models [52, 96]. This breakthrough laid the foundation for applications such as ChatGPT, whose ability to generate human-like text demonstrated the potential of LLMs and triggered a surge in research and commercial interest. More recently, the focus has expanded beyond standalone LLMs to the integration of Software Agents and agentic capabilities, where models exhibit autonomy, goal-directed behavior, and enhanced reasoning abilities [101, 44]. These agentic systems, which incorporate planning, memory, and interaction mechanisms, are, as of 2025, being extensively studied to augment the functionality of LLMs, enabling them to operate dynamically in complex environments, solve multi-step problems, and interact more effectively with users and digital ecosystems.

Section 2.1.1 discusses the general hype cycle model—the waves of enthusiasm and subsequent disappointment observed across many industries and technologies—and Section 2.1.2 reviews and synthesizes illustrations of AI history, placing its alternating "AI winters" and "AI summers" on a normalized scale, from the early promises of symbolic AI to the breakthroughs of the deep learning revolution.

2.1.1 The Hype Cycle

The hype cycle model, introduced by Gartner Inc. in 1995 [36], illustrates the evolution of technological expectations over time. As shown in Figure 27 (Appendix A) [36], the model is formed by merging two distinct curves: a human-centric expectation curve, represented by the blue bell-shaped curve, and a classical technology maturity S-curve, represented by the orange S-shaped curve. The first curve, the hype level, follows a bell-shaped curve driven by excitement, social contagion, and heuristic decision-making. This curve captures the tendency of people to overestimate the potential of a technology, leading to an initial peak of inflated expectations, followed by a sharp decline when real-world limitations become evident.

The second curve, the S-curve of technological maturity, represents the gradual advancement of a technology over time. Initially, progress is slow due to limited understanding and minimal performance gains from early investments. However, as knowledge accumulates and improvements accelerate, the technology experiences

rapid development until it eventually stabilizes at a plateau defined by its inherent limitations. Together, these two curves form the hype cycle, which models the typical trajectory of emerging technologies, highlighting both the initial overhype and the eventual stabilization at a realistic level of maturity; how the two curves interact governs both the tempo and the amplitude of successive boom-and-bust phases.

The cycle runs through five stages [36]: an **Innovation Trigger** (early excitement before practical use is understood), a **Peak of Inflated Expectations** (overhyped predictions, most projects failing), a **Trough of Disillusionment** (skepticism as limits surface and investment falls), a **Slope of Enlightenment** (realistic use cases drive renewed, practical adoption), and a **Plateau of Productivity** (mainstream, stable use delivering real benefits).

2.1.2 Review and Synthesis of AI Winters and AI Summers

This section synthesizes evidence from peer-reviewed papers, technical blogs, and public presentations to trace the recurring boom-and-bust pattern (AI Summers and AI Winters) that has characterized AI research and commercialization. The AI-history illustrations analyzed in [25] include the following fourteen, cited here (the reproduced Figure 4 aggregates the full set): [102, 93, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114].

To quantify "summer" and "winter", each illustration was normalized to a grading scale from +10 to -10: a figure receives +10 where it visually reaches its peak and -10 where it reaches its lowest point. This grading serves as an indication of "temperature," verbally expressed as "AI summer" (hot) or "AI winter" (cold). The grading is a single-coder, qualitative reading and is indicative only. The aggregated illustration is presented in Figure 4. This figure also indicates that the period of active Semantic Web research tends to be depicted as an AI winter. Furthermore, with one exception not noted in [25]—Francesconi [105], who attributes the end of the AI winter to AI meeting the Web and calls the Semantic Web an essential infrastructure for AI—none of the studies mention either the topic *Semantic Web* or *Software Agents* as representing any form of AI boom in historical AI illustrations.

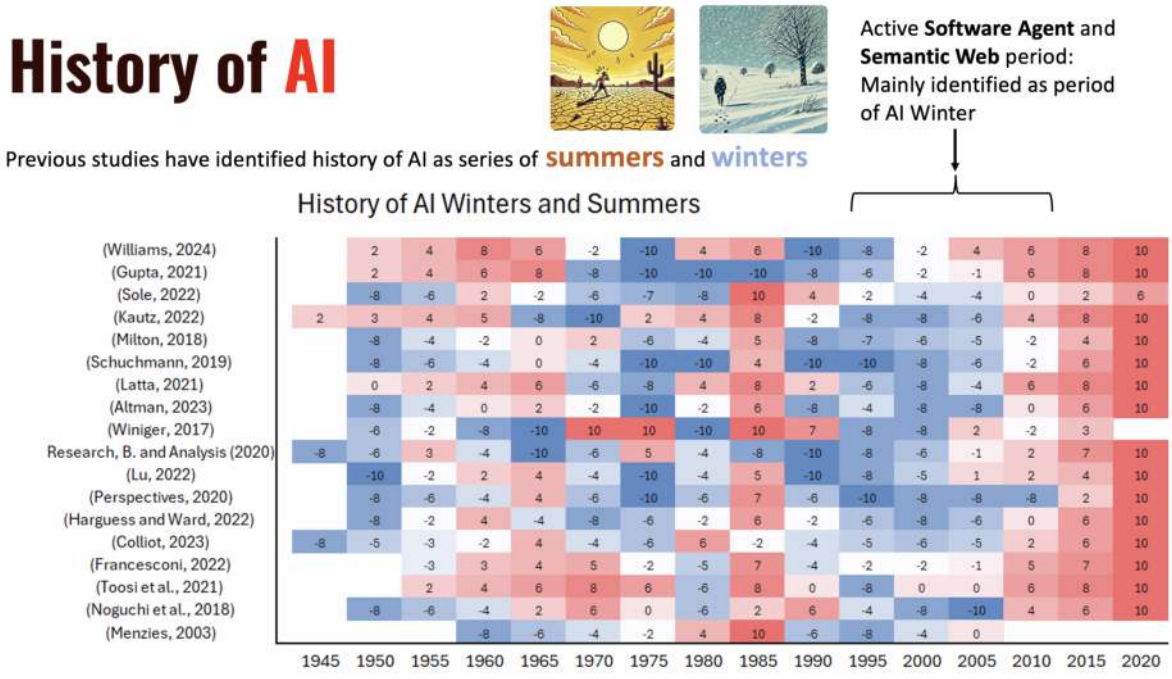


Figure 4: History of AI as illustrated by previous studies. The period of active Semantic Web development is typically identified as an AI winter period [25].

AI is not a single invention but a shifting portfolio of techniques, constantly redefined over time — from symbolic reasoning and expert systems to ML, deep learning, and the large-scale generative models of the 2020s. Each element follows its own trajectory, so the broad history of AI is best understood as the superposition of many overlapping hype cycles rather than one monolithic wave. Recognizing AI as such a

constellation of evolving technologies helps clarify why optimism and disappointment can coexist—and why practical value often emerges only after multiple passes through the cycle.

Take *neural networks* and *software agents* as illustrative examples. For a long time, neural networks were not considered mainstream AI [115]; however, especially during the fifteen years to 2025, that has changed. Similarly, agent technology, introduced in the early 1990s, rose to widespread media enthusiasm, hit technical and market barriers [116, 117], resurfaced with the rise of Web services, and, as of 2025, is enjoying renewed interest through autonomous AI assistants (Section 2.4). In other words, the same underlying concept has already traversed the Gartner hype cycle several times. Read forward rather than as a chronicle, these cycles trace one cumulative progression: each era's software agents were limited by the representational substrate their era could supply, and only the recent convergence of compute, ubiquitous connectivity, hyperscale infrastructure, and community-generated data lifts those limits [25, 30]—the through-line this thesis develops. The thesis's corollary, argued in Section 2.5.6, is that once capability stops being the bottleneck, governance becomes the binding constraint.

2.2 History of the Web

This section traces the origins of the Web—RQ I.1 (Table 2)—through the thesis's Phase I lens (Section 1.1.2), and reads them *critically* rather than as a chronicle. The argument is this: the Web succeeded precisely because it was deliberately *simple, decentralized, and human-facing*—a hyperlinked document system that grew into a general application and service substrate—but for that same reason it carried *no machine-understandable semantics*. That absence is exactly the gap Phase I's software agents and the Semantic Web (Section 2.3) set out to close, and it frames the Phase I problem (Section 1.2). Accordingly, the history below is told only as far as it explains that design choice and its consequences; lower-layer network and protocol mechanics are omitted.

2.2.1 The Hypermedia Concept (1945)

Vannevar Bush is widely acknowledged to be the inventor of hypermedia. As early as 1945, he wrote a famous article called *As We May Think* in the Atlantic Monthly [118], where he broadly discussed the problems of information management in the research world of those days. He described the latest technical achievements and ended up proposing a photo-electrical-mechanical device called *memex* ("memory extension") for personal information storage and management.

The idea behind memex was not to structure information as it was typically organized then, *i.e.*, in hierarchical, tree-like data structures, but to draw inspiration from the associative nature of the human mind:

> "The human mind . . . operates by association . . . Selection by association, rather than indexing, may yet be mechanized . . . The process of tying two items together is the important thing." [118]

The decisive idea was *associative linking*—tying any two items together—rather than hierarchical indexing: the conceptual seed of hypertext, which would influence later pioneers such as Douglas Engelbart.

2.2.2 Prototypes and Concept Development (1960s)

Ted Nelson coined the term *hypertext* in 1965 [119, 120]; his far more ambitious *Xanadu* vision was never fully realized [121]—an early sign that associative linking was easy to imagine but hard to build at scale. Douglas Engelbart's team at SRI publicly demonstrated its working hypertext system, NLS, in 1968—"The Mother of All Demos"—introducing the mouse, hypertext, object addressing, dynamic linking, and networked shared-screen collaboration [122].

2.2.3 Hypermedia Research (1980s)

Despite continued research after the initial prototypes, the 1980s were still characterized by incompatible networks, disk formats, data formats, and character-encoding schemes [37], which made transferring information between systems daunting and impractical. The situation was frustrating: computer usage was increasing and a large amount of important information was already stored on computers, many of them networked, yet the systems in use remained fundamentally incompatible.

2.2.4 The Invention of the World Wide Web (1989)

The development of the World Wide Web (WWW) began with a famous project proposal written in 1989 by Tim Berners-Lee [62]. In the proposal, Berners-Lee discussed how many information management systems organized information hierarchically, as trees, although a graph structure would be more suitable. To illustrate the limitations of tree-like data structures, Berners-Lee used newsgroup discussions as an example. Newsgroups are typically hierarchical discussion systems, yet discussions within a newsgroup often diverge from the original topic, fragmenting into subtopics and entirely different subjects. At that point, part of the discussion should logically reside in a different part of the tree. Even splitting the tree into multiple trees, or dividing discussions into different newsgroups based on these topics, would not fully resolve the issue: a tree-like structure lacks the necessary flexibility for this purpose.

Berners-Lee argued that CERN, at the time, was losing information due to the inflexibility of its information systems. He proposed that systems should be organized to allow direct links between documents. This structure would be more dynamic and more natural for users who prefer to find information related to a specific topic rather than navigating hierarchical file systems. It is important to note that the problem was precisely the same one that Bush had addressed 44 years earlier. All the necessary concepts[4] and data structures had been invented long before 1989. Berners-Lee simply applied hypertext to this new environment in a straightforward manner. As Berners-Lee himself stated:

> "I happened to come along with time, and the right interest and inclination, after hypertext and the Internet had come of age. The task left to me was to marry them together." [63]

The analytically important point is not the accolades that followed but *what* was married together: existing hypertext and Internet ideas, combined under a deliberately minimal design.

2.2.4.1 The Architecture of the Web

The Web's design followed a few principles [37]: it had to record arbitrary associations between objects; a cross-system link had to be an incremental effort (no merging of link databases); and information had to be available on every platform and easy to enter and correct, without forcing users into particular languages, operating systems, or machine-oriented ways of working—flexibility being the key goal.

Figure 28 (Appendix A) depicts the Web's architecture, which was designed to meet the aforementioned criteria and adhere to established principles of software design adapted to the network environment [37]. There should be as few specifications as possible, because every specification needed to ensure interoperability simultaneously constrained the Web's implementation. An important principle of Berners-Lee's proposal was that the client software program's user interface should be consistent across all types of computer platforms, allowing users to access information from a wide variety of computers.

Three new technologies were created in the project that emerged from the proposal. Briefly, these were the HyperText Markup Language (HTML), used to create Web documents; the HyperText Transfer Protocol (HTTP), used to transmit the pages; and the Uniform Resource Identifier (URI)[5], used for addressing. These technologies will now be briefly examined before continuing with the historical overview.

Uniform Resource Identifier (URI). Unlike the centralized link databases of 1980s hypertext systems—which guaranteed link consistency but could not scale—the Web deliberately dropped that guarantee, letting anyone create a reference without consulting its destination, and so achieved scalability [37]. A link can point to any resource because URIs provide a single global, extensible identifier space [125, 124]: any naming or addressing syntax can be mapped behind a prefix (e.g. *http:*) and folded into it.

HTTP. HTTP [126] was developed because File Transfer Protocol (FTP) was insufficiently feature-rich and too slow to be optimal for the Web [37]. HTTP URIs are resolved into the addressed document by splitting them into two parts. The first part is used to query the Domain Name System (DNS) to discover a suitable server, and the second part is passed to the server to identify the specific resource.

[4] Such as the Internet (TCP/IP, DNS), hypermedia, various lower-level protocols, and operating systems

[5] Originally named Universal Document Identifier (UDI) by Tim Berners-Lee, the term evolved within the IETF to Uniform Resource Locator (URL) and later to Uniform Resource Identifier [63, 123, 124]. In practice, the terms "uniform" and "universal" are often used interchangeably, and URI and URL are treated as synonyms.

HTML. HTML [127] was the hypertext interchange format. Its SGML lineage eased adoption by the documentation and hypertext communities, though reconciling full SGML compatibility with HTML's ease of use long challenged experts [63, 37].

2.2.5 Research Boost (1990–1994)

From the first server and WYSIWYG browser/editor in 1990 [63, 37], the Web spread through European universities and then exploded with NCSA *Mosaic* (1993) and Netscape, with the W3C founded in 1994 [128]. The analytically decisive point is not the statistics but their cause: this explosive, *unmanaged* growth—no central registry, no link-consistency guarantee—validated the original bet on simplicity, loose coupling, and decentralization over control. The same property made the Web essentially *unmanageable* to snapshot or govern centrally, a tension the later Semantic Web effort would inherit.

2.2.6 The Web: From Document Collection to Application Platform

The Web is not only about information but about *services*—tasks that can be invoked not just by users through browsers but by other programs. As services proliferated, the boundary between what is "on the Web" and what lies beyond it on the Internet blurred [129]—and, crucially for this thesis, the question of how *programs* (and agents) discover and invoke those services moved to the foreground.

Early visions of the Web as a hypertext library have evolved into the present-day reality of the browser as a general-purpose runtime: the advent of asynchronous JavaScript and XML (AJAX) first enabled desktop-like responsiveness in the mid-2000s [130], the "Web 2.0" paradigm framed the Web itself as a software platform, with applications delivered as continuously updated services rather than static documents [131], and subsequent standardization—HTML5 APIs, Service Workers, and WebAssembly—has delivered offline execution, background sync, and near-native performance [132, 133]. These capabilities underpin the Progressive Web App (PWA) model adopted by products such as Google Docs and Figma, illustrating how the Web now functions as a cross-device application substrate that abstracts away operating-system heterogeneity while retaining the universal reach of URLs.

For this thesis the lesson is twofold. First, the Web won through radical *simplicity and loose coupling*—the very property that left content human-readable but not machine-understandable, the design tension the Semantic Web and software agents would later struggle against (the convergence gap analyzed in Section 2.3). Second, its evolution from document collection into an application and service substrate is exactly what Phase I's mobile, context-aware service agents set out to exploit (Section 1.2). The history of the Web is thus not background color but the origin of the problem this dissertation attacks.

2.3 The Semantic Web and Its Vision of AI Software Agents

This section is read through the Phase I lens—how the Web's future was envisioned with Semantic-Web-native Software Agents—and it provides the conceptual baseline used in Section 1.2. The focal inquiry is RQ I.2 (Table 2): to what extent Semantic Web formalisms and agent protocols can be combined to deliver runtime interoperability and adaptation on the open Web, and where irreducible gaps prevent practical convergence. This section examines the integration surface between Web Knowledge Representation (KR) standards (RDF/OWL, rules, service ontologies) and FIPA-ACL, anticipating which promises proved workable and which remained aspirational. See also the Phases and thesis structure overview in Section 1.1.2.

2.3.1 Software Agents Before the Semantic Web

Long before the Web era, the conceptual foundations for software agents were established by research in *Distributed AI* (DAI) and the *Actor model* [134]. DAI cast intelligence as an *emergent* phenomenon arising from multiple, loosely coupled problem solvers that cooperate across a network, thereby elevating autonomy and negotiation to primary design concerns [135, 136]. In parallel, the Actor model supplied a concrete computational metaphor of independent, concurrently executing entities that communicate solely through asynchronous message passing. Together these two traditions contributed the twin conceptual pillars of **distribution** and **concurrency**, which later researchers distilled into the now-standard agent properties of social ability and proactive–reactive behavior [137].

In the 1990s software agents emerged as a concept in computer science, driven by the need for autonomous and intelligent systems in increasingly complex environments. This decade consolidated the core agent

characteristics of autonomy, proactivity, reactivity, and social ability [116, 138]. Interface agents that reduce users' information overload were proposed at the same time [139], and the rapid expansion of the World Wide Web around 1994 widened the space in which such agents could operate. Foundational research explored diverse aspects of agent technology, from theoretical underpinnings and formal methods to practical applications for information management and personalized assistance [138, 139].

The development of various agent architectures, such as reactive, deliberative, and hybrid models, further illustrated the field's commitment to creating sophisticated autonomous entities [140]. Researchers also focused on defining the boundaries of agency, distinguishing agents from conventional programs through taxonomies and formal definitions [141], and traced the field's roots to DAI and Carl Hewitt's Actor model of the 1970s [116, 134, 137].

From Definitions to Standards. The explicit taxonomies and formal definitions produced during this period played a pivotal role in *standardizing* the concept of agency. This shared vocabulary informed FIPA, whose specifications defined FIPA-ACL [142] and an agent-platform reference model [42] that was adopted by the main agent platforms of the era (e.g., JADE [143] and FIPA-OS; Section 2.4.5.3). These efforts, together with subsequent W3C initiatives (e.g., the *Agent Markup Language* submission), aimed to let later Semantic-Web agents interoperate across organizational and domain boundaries on the basis of well-defined semantics rather than ad-hoc conventions—an aim only partly met (Section 2.4.5.3).

2.3.1.1 Impact of the World Wide Web's Expansion on Software Agents

The exponential expansion of the World Wide Web in the mid-1990s profoundly altered the landscape in which software agents were conceived and applied. Originally developed in the context of distributed AI and local computation environments, software agents found new significance as the Web became a vast, decentralized, and heterogeneous information space. The growing complexity of navigating, searching, and processing this information catalyzed the demand for autonomous systems capable of filtering, retrieving, and reasoning over Web-based content on behalf of users.

Seminal work by Pattie Maes and others [139] recognized this shift early, introducing agents as personalized assistants that could reduce information overload in the emerging Web era. This period also saw the rise of "information agents"—autonomous programs designed specifically to operate over Web protocols, harvest content, and interact with Web services. The scalability and openness of the Web provided a compelling environment for agents to demonstrate their core properties: autonomy, proactivity, and social interaction, especially as HTML and HTTP standards facilitated agent-based crawling and retrieval.

Moreover, the ubiquity of browsers and online services created a new user expectation: that software should act intelligently in navigating and managing Web interactions. This sociotechnical transformation fueled both academic research and industrial exploration of agent-based architectures, eventually motivating the integration of agents into the foundational vision of the Semantic Web [64]. The Web's growth thus did not merely provide a backdrop—it actively shaped the functionality, relevance, and perceived utility of software agents during this foundational decade.

2.3.2 The Vision of the Semantic Web and Intelligent Agents

Tim Berners-Lee, James Hendler, and Ora Lassila, in their seminal 2001 paper "The Semantic Web" [64], envisioned an evolution of the World Wide Web into a more intelligent and interconnected system. This Semantic Web would be an infrastructure in which information has explicitly defined meaning, facilitating human-computer cooperation and enabling machines to understand and process Web data through a universal framework of metadata and ontologies. Realizing it meant enriching Web resources with semantic markup in languages like RDF and OWL, making content machine-processable [144]. The goal was a transition from unstructured documents to interconnected data, enhancing knowledge sharing, reuse, and integration [145], aligning directly with AI's goals of machine understanding and reasoning [64]. As of 2025, this highly influential paper had garnered over 30,000 citations.

Intelligent agents were conceived as crucial actors within this vision, able to automatically access, interpret, and reason about the semantically marked-up information [146]. These agents would act on behalf of users, performing tasks such as information retrieval, service discovery, automated decision-making (e.g., scheduling meetings or making purchases), and seamless interaction across different platforms [147]. Personal assistants, guided by rules, ontologies, and user profiles, were highlighted as prime examples of potential applications [64, 146].

Realizing this vision depended, in the Phase I reading, on the *ability of autonomous agents to converse*. The FIPA-ACL specification was taken to meet this requirement by supplying a performative-based message envelope whose semantics are defined in a modal logic of communicative acts [148, 149]. When the `content` field of such messages was expressed in RDF or OWL, the resulting utterances were to be simultaneously interpretable at the *data* and the *speech-act* levels, which was expected to enable richly contextualized dialogues among heterogeneous Web agents. Accordingly, ACLs were seen as the critical bridge between the Semantic Web's data layer and the multi-agent community's interaction layer—a bridge that, as discussed later in this section, was not operationally completed in that era.

The paper's emphasis on AI Software Agents, and the backgrounds of Hendler and Lassila in intelligent agents and knowledge representation, underscore the Semantic Web's connection to advancing autonomous systems capable of reasoning, decision-making, and communication [64]. By leveraging standardized semantics, agents could interpret and act upon information with greater sophistication. This vision linked the future of the Semantic Web to developments in AI and positioned Software Agents as a critical component in realizing the Web's full potential. Although the full vision remains unrealized, it is still a cornerstone at the intersection of Semantic Web technologies and AI Software Agent development.

2.3.3 Key Enabling Technologies and the Semantic Web Stack

Several technologies were crucial for realizing the vision of the Semantic Web and its integration with intelligent agents. RDF provided a standard model for data interchange on the Web, enabling the expression of relationships between resources [150]. OWL, built upon RDF, was designed to represent rich and complex knowledge about entities, their classifications, and relationships [144, 151]. Ontologies, formal and explicit specifications of shared conceptualizations, provided a common vocabulary and understanding of specific domains, which was crucial for enabling agents to interpret information consistently [152, 153].

Agent Communication Languages (ACLs), such as the FIPA-ACL [149], were recognized as essential for facilitating interaction and information exchange between autonomous agents. Furthermore, the *DAML-S*[6] ontology—renamed *OWL-S* after the release of OWL—[154] was created to add a machine-interpretable semantic layer to conventional Web-service descriptions. It exposes three mutually supportive components: a *Service Profile* that advertises what the service does, a *Process Model* that specifies how the service can be executed, and a *Grounding* that binds abstract inputs and outputs to concrete message exchanges. Together these components allow an intelligent agent to match its goals to advertised profiles and thus discover relevant services, to use the process model and grounding information to construct and send the correct messages for direct invocation, and to chain multiple process models into coherent workflows for automatic composition of complex tasks. The development of DAML-S/OWL-S therefore represents a focused effort to supply the semantic infrastructure—rooted in ontologies, RDF, and OWL—along with the inter-agent communication mechanisms required for truly autonomous agents on the Semantic Web.

Figure 29(a) (Appendix A) illustrates the 2006 version of the "Semantic Web Stack" (an earlier version appears in Figure 31), a layered architecture proposed by Tim Berners-Lee for organizing the technologies of machine-readable Web data [145]. URIs and Unicode provide the foundation for identification and character encoding. XML offers a syntax for structured documents, while RDF and RDF Schema (RDFS) define a standardized way to describe resources and their relationships [155]. OWL builds on this to enable the creation of complex ontologies. Rule languages such as SWRL (Semantic Web Rule Language) [156] and, later, RIF (Rule Interchange Format) [157], together with the SPARQL query language [158], were intended to support inference and querying over this semantically rich data.

OWL's expressivity is intentionally constrained for decidable, tractable reasoning; rule-based extensions such as SWRL add Datalog-style Horn rules (*if antecedent then consequent*) for greater inferencing power, with decidability preserved by DL-safe rules that restrict variables to named individuals [156, 159, 160].

Higher layers of the stack were envisioned to handle logical reasoning, proof verification, and trust. These once-aspirational *proof* and *trust* layers have since been partly operationalized by concrete primitives—graph canonicalization and Data-Integrity signatures, Verifiable Credentials, and Decentralized Identifiers—that let agents verify data provenance and attach signed proofs to their own outputs [161, 162, 163, 164].

Semantic Web Rollout Vision. Figure 29(b) (Appendix A) depicts Tim Berners-Lee's 2003 "rollout vision" for the Semantic Web, projecting the integration of layered technologies from markup (SGML, XML)

[6]DAML-S: Defense Advanced Research Projects Agency (DARPA) Agent Markup Language for Services

and data (RDF, RDF-S) through ontology (DAML, OIL, OWL) to rules and query (Prolog, RuleML), culminating in proof and trust mechanisms for automated reasoning and secure interoperability [99].

However, the agent-centric vision was not operationally realized in that era, and it is worth stating plainly *why* rather than merely noting that it did not succeed. While RDF and OWL saw adoption in niche areas, the higher, logic-based *proof* and *trust* layers on which autonomous agency depended saw only limited practical uptake, for three compounding reasons. First, OWL's Open World Assumption (unknown facts are not presumed false) and its lack of a Unique Names Assumption (identical names need not denote identical entities) clashed with the closed-world, unique-key assumptions of the database and enterprise systems that agents would have had to act upon [165]. Second, ontology engineering scaled poorly: the vision presupposed *universal, a priori semantic agreement*, which open, decentralized environments did not produce [145, 100]. Third—and, in this thesis's reading, decisively—the era's computing substrate was simply too thin to carry autonomous Web agents: the runtime the vision assumed did not yet exist, and the barriers that held it back were lifted only by the later technology shift traced as the "then versus now" arc in Section 1.1.2. The vision was thus sound but premature; the *standards* endured while the *autonomous-agent* program achieved only limited adoption [25]. This thesis accordingly does not attempt to revive the maximal stack: its Phase I artefacts opt instead for pragmatic, server-centric integration with lightweight semantics, and the irreducible gaps that blocked Semantic-Web/agent convergence in these prototypes are taken up directly as the answer to RQ I.2 in Section 8.1.

2.3.4 Semantic Web Influence on Modern AI/ML Data Integration

Although the Semantic Web's most ambitious goals remain aspirational, its foundational interoperability standards—such as RDF, OWL, SPARQL, and Shapes Constraint Language (SHACL)—have significantly influenced how structured data is integrated in modern AI and ML systems. These standards provided early blueprints for achieving semantic consistency across heterogeneous datasets, a challenge that remains central to AI pipelines in the LLM era.

Linked RDF graphs were designed as instruments for incremental data integration across sources [166]; in large-scale AI systems, knowledge graphs—many with RDF- or OWL-based schemas—now play a similar role, supporting entity resolution and data integration. Companies such as Google, Microsoft, and Facebook have built large internal knowledge graphs and ontologies to normalize concepts across business domains, supporting data integration for ML [167]. SPARQL's graph-query paradigm also anticipates the emerging need for expressive retrieval interfaces in RAG systems.

Furthermore, approaches since the mid-2010s to embedding knowledge graphs into vector spaces for use in ML models can use schema-level type constraints to guide embedding learning [168]. Validation languages, such as SHACL, parallel the schema-checking layers of AI pipelines, which aim at ensuring that training and inference data adhere to expected semantic contracts.

Thus the Semantic Web's *standards* outlived its agent vision: though seldom cited by name, RDF/OWL design patterns, shared vocabularies, and SHACL-style validation now underpin scalable AI data infrastructures—a telling split between what endured (declarative interoperability) and what did not (autonomous agency).

2.3.5 Could Semantic Web Principles Complement LLMs?

LLM pipelines still lack explicit semantics, provenance, and verifiability, whereas an ontology offers a canonical, machine-interpretable vocabulary with sound (if manually modeled and decidability-limited) inference. Combining the two pairs the symbolic rigor of OWL/RDF with the statistical reach of LLMs through patterns such as *ontology-constrained decoding* (pruning generations to ontology-resolvable entities), *knowledge-graph RAG* (grounding answers in explicit triples with inspectable subgraph rationales), and *neuro-symbolic post-hoc explanation* (aligning answers to entailment paths). All three aim to reduce hallucination and supply user-facing rationales [169, 170, 171].

2.3.6 How AI Has Shaped Real-World Semantic Web Applications

Reciprocally, AI advances drove real-world uptake where they offered scale: knowledge graphs behind Google's Knowledge Panel [172] and Facebook's Entity Graph [173] link entities into large-scale entity graphs, schema.org converged SEO with semantic markup, and SHACL-style validation now guards data integrity in regulated domains. The original agent-centric vision thus remains aspirational, but AI redefined the Semantic Web into a practical substrate for large-scale, machine-interpretable data.

2.4 Software Agents

Read through the thesis Phase I lens (Section 1.1.2), this section establishes the conceptual baseline for *software agents* as individual computational entities. Definitional criteria, core architectures, and the ambitions and limitations documented in the classic agent literature are synthesized. This baseline motivates the Phase I problem framing in Section 1.2 and anchors (i) the "software agents" half of RQ I.1 and (ii) the integration question RQ I.2 (Table 2) by clarifying which capabilities and abstractions must be made interoperable on the Web. Consistent with this scope, emphasis is placed on agent-level models and decision/interaction semantics rather than platform-specific middleware or lower-layer protocol mechanisms.

2.4.1 Agents and Distributed Artificial Intelligence (DAI)

The concept of agents emerged as a construct in AI, particularly within the subdomain of DAI. The ACM Computing Classification System identifies DAI as an area of AI research encompassing MAS, Intelligent Agents, Mobile Agents, and Co-operation and Coordination. These categories collectively represent a range of approaches and methodologies dedicated to creating systems where multiple autonomous entities, or agents, interact to achieve individual or shared goals, as illustrated in Figure 5.

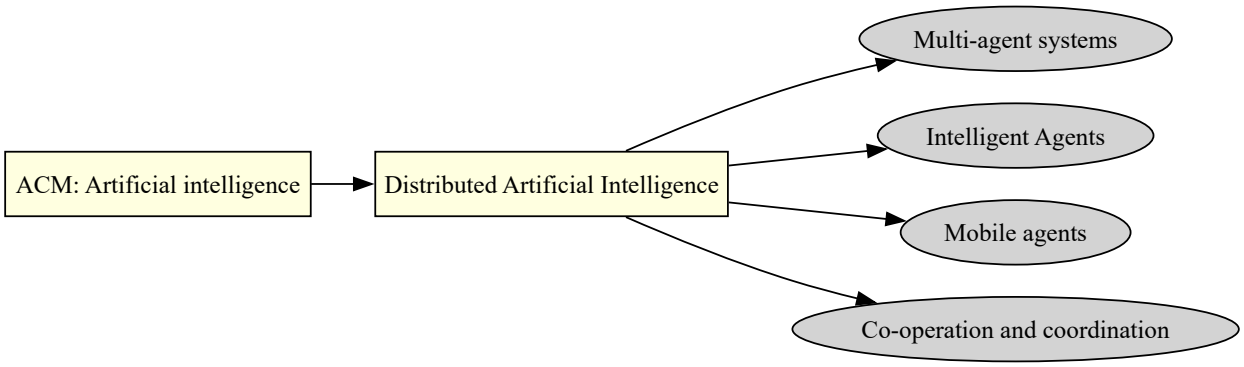


Figure 5: Excerpt of the ACM Computing Classification System: the Distributed artificial intelligence subtree under Artificial intelligence [32].

Agents combine perception, reasoning, learning, and action and—through autonomy, proactiveness, reactivity, and social ability—suit applications from robotics and distributed control to collaborative decision-making. In DAI the central question is how multiple agents interact (cooperation, coordination, negotiation, competition) within a shared environment, which requires explicit frameworks and protocols; MAS denote such structured collections of interacting agents, with intelligent agents adding reasoning and mobile agents adding migration across networks. The following sections develop the underlying principles and architectures.

2.4.2 Hype Cycles: The Ever-Promising Agent Technology

Software agent technology has gone through a hype cycle (Section 2.1.1) a couple of times during its history. The following quote is from 1996:

> "Agents are here to stay, not least because of their diversity, their wide range of applicability and the broad spectrum of companies investing in them. As we move further and further into the information age, any information-based organization which does not invest in agent technology may be committing commercial hara-kiri." [116]

A few years later Nwana and Ndumu wrote another article [117] stating that the expectations had been too high.

In December 2024, Microsoft CEO Satya Nadella argued that AI-driven software agents will transform B2B applications. According to Nadella, AI-driven agents will take over business logic and enable new AI-powered applications, placing software agents at the heart of the current AI hype. In an episode of the BG2 Pod series [174], he stated:

> "The notion that business applications exist—that is probably where they all collapse in the agent era. ... They are essentially CRUD-databases with a bunch of business logic; the business logic will go to these agents ... All the logic will be in the AI-tier. Once the AI-tier becomes the place where all the logic is, people will start replacing the backends ... so the logic tier can be orchestrated by AI agents in a very seamless way." [174]

Obviously, noting that agent technology has had inflated expectations in the past does not prove anything about its future. What is different now is concrete rather than rhetorical: many of the technology constraints that throttled 1990s agents (Sections 2.3 and 1.1.2) have since lifted, so the substrate on which agents run has changed even where the ambitions echo. It also bears remembering that neural networks themselves endured several hype cycles before their major breakthrough in the 2010s.

2.4.3 Definitions

Although there have been many discussions about the definition of a software agent (e.g., [175, 141, 176, 177, 116, 178, 179, 180, 181, 138]), there was little consensus on the matter in that literature. If there was any consensus, it was the acknowledgment that no consensus existed [50]. Russell and Norvig [178] have provided a well-known definition of an agent as *anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators*. A simplified version of this abstract definition is illustrated in Figure 6. It is evident that the definition is overly broad and could encompass various entities, such as a regular Unix process.

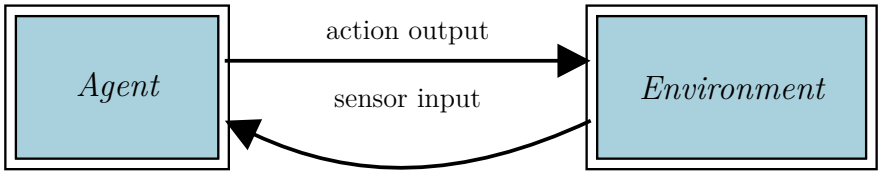


Figure 6: Agents and their environments [33].

Typically, properties associated with (intelligent) agents include proactivity, reactivity, sociality, and autonomy [33]. Some researchers argue that if a system does not require these properties, it is better to design and implement it without agents [38].

One way to view software agent technology is as a higher level of abstraction compared to, for example, object-oriented technology. It is said to provide the system designer with a natural abstraction for modeling an entire system or its components [182]. This is partly why agent technology has been considered a promising approach for the analysis, specification, and implementation of complex software systems. More broadly, this approach is referred to as Agent-Oriented Software Engineering (AOSE) [177], and mature methodologies such as *Tropos* carried the agent abstraction through every development phase, from early requirements through architectural and detailed design to implementation [78]. Such methodologies, however, addressed the development lifecycle rather than runtime interoperability on the open Web, which was left to platform standards (Section 2.4.5).

But why is it so hard to define an agent? The answer is at least partly because the *intelligent* and *collective* dimensions of agents have led them to be influenced by results stemming from extremely different disciplines [50]. Examples given by [183] span biological analogies (artificial life, genetic programming), computational linguistics (Speech Act theory [184]), cognitive science (the *Belief–Desire–Intention* architecture [185, 186]), machine learning (adaptation to users and context), database and knowledge-base technology, distributed computing (concurrency and interaction protocols), and econometric models of collective behavior.

With such a diversity of contributions, the agent and MAS paradigm was, in that literature, diluted in a multitude of perspectives, so no single definition can cover all the aspects that have been related to agents. Some characteristics that have been related to agents are listed in Table 22 (Appendix A)[7]; comparing them with the disciplines listed above shows how those disciplines have shaped the characteristics bound to agents. Some definition is nonetheless needed in this thesis so that the reader has a general understanding of agents.

[7]The characteristics are adopted from [138, 116, 177, 180, 141, 175] and the table from [50].

Definition Used in Phases I and II. In Phases I and II of this thesis, however, the focus is more on how the agents communicate than on how the actual agent is internally implemented. Therefore, this thesis adopts FIPA's definition, quoted here with minor omissions [42]:

An agent is a computational process that implements the autonomous, communicating functionality of an application. Agents communicate using an Agent Communication Language (ACL). An Agent is the fundamental actor on an Agent Platform (AP) which combines one or more service capabilities, as published in a service description, into a unified and integrated execution model. An agent must support at least one notion of identity that labels an agent so that it may be distinguished unambiguously within the Agent Universe. An agent may be registered at a number of transport addresses at which it can be contacted.

For Phase III of this thesis the focus is placed on *LLM agents* as introduced in Section 2.5 and formally defined in Definition 3.2.1.

2.4.4 Single-Agent Architectures

Of course, differing agent definitions led to differing internal architectures, spread along a continuum [50]: from purely **reactive** designs—a direct sense–act loop giving fast, stateless responses but no planning or learning [187, 188]—to **deliberative/symbolic** ones that reason over an internal world model, the best known being the **BDI** (Belief–Desire–Intention) architecture [185, 186]. Intermediate forms include knowledge-based (rule/inference) agents and finite-state machines, which add a decision step but stay rigid and goal-agnostic, while **learning** agents use ML/RL to improve from experience [189, 53]. **Hybrid** architectures stack behavior layers—either hierarchically (one- or two-pass control) or in parallel with subsumption, meta-control, or a shared blackboard to arbitrate conflicts [50].

2.4.5 Multi-Agent Architectures

The preceding section delineated definitions and architectural theories pertaining to individual agents. However, as this thesis addresses MAS, the mechanisms of inter-agent interaction are arguably more critical than the internal architecture of a single agent. Consequently, the following subsections detail the agent communication protocols of the Phase I era (2001–2006) and present the multi-agent communication model employed in the Phase I demonstrations (Chapter 4).

2.4.5.1 FIPA Architecture

FIPA[8] was founded in 1996 as a non-profit organization. FIPA produced the standards for heterogeneous and interacting agents and agent-based systems across multiple vendors' platforms on which the Phase I work of this thesis relies. The official mission statement was stated as follows [191]: "The promotion of technologies and interoperability specifications that facilitate the end-to-end interworking of intelligent agent systems in modern commercial and industrial settings. The emphasis here is on practical commercial and industrial uses of agent systems. However, we are also focusing on intelligent or cognitive agents, that is, software systems that may have the potential for reasoning about themselves and/or other systems that they encounter."

2.4.5.2 FIPA Agent Platform

In the Phase I era, agent platforms were implemented as middleware running on top of an operating system; they were needed because the operating systems of the time lacked the services that agents required. FIPA did not specify the internal design of an agent platform, because FIPA's main concern was achieving interoperability between agent platforms [38]. However, FIPA did specify a reference model [42] for agent management (see Figure 33 in Appendix A), which the remainder of this subsubsection describes as it stood then.

On a conceptual level the reference model is very simple and there are only three essential components:

- The Message Transport System (MTS) routes messages both between agents within the FIPA agent platform and between agents residing on different FIPA agent platforms.

[8] As noted by Labrou et al. [190], the word *physical* in FIPA's name was originally included to cover agents of the robotic variety; over time, however, the adjective's presence has come to serve as a reminder that physical—that is, human—agents and interaction with them are part of the association's scope.

- The Agent Management System (AMS) manages the life cycles of the agents, such as starting, stopping, and quitting agents. The AMS also maintains the mapping between an agent's identifier and its transport addresses, thus acting as an agent naming service. Furthermore, the AMS maintains the platform profile, which describes the platform properties such as communication capabilities.
- The Directory Facilitator (DF) plays a role in the agent world somewhat similar to that of UDDI in Web Services. The DF is an agent that provides yellow pages for other agents, i.e., it maintains information about the skills they have advertised with it.

2.4.5.3 FIPA Architecture Implementations

In the Phase I era there were several implementations of the FIPA Architecture, listed at [192]. An evaluation of FIPA-compliant agent platforms of that time is found in [193]. The evaluation shows that interoperability was not fully achieved[9] even though it was the most important goal of FIPA.

One of the most active projects in that era was JADE [143], which is also the basis of the Phase I demonstrations of this thesis (Chapter 4). One interesting FIPA implementation of the time was the Lightweight Extensible Agent Platform (LEAP) [194], which could be used in the resource-limited devices of the day, such as mobile phones.

2.4.6 Multi-Agent Communication

Russell and Norvig [178] have defined communication as *"intentional exchange of information brought about by the production and perception of signs drawn from a shared system of conventional signs"*. The most important system of signs for humans is the language that enables us to communicate with each other and a similar communication model based on a shared language has also been proposed for agent communication [38].

As noted by Helin [38], groups of agents are capable of achieving outcomes beyond the capabilities of any single agent. However, to perform their tasks effectively, agents depend heavily on expressive communication with one another: to make requests, to propagate their information capabilities, and to negotiate [50].

Agent communication can be divided into two fundamental parts. First, agents have to agree on a common ACL, which defines the types of the message performatives and their meanings. Second, agents must have a common understanding of the knowledge that is exchanged within the messages. This includes both a shared ontology and the content that is defined by a content language.

Modern distributed computing leverages architectural styles and frameworks, such as the REST architectural style [195], gRPC [196], and message queues—e.g., Kafka [197] and RabbitMQ [198]—to achieve interoperability in heterogeneous environments, thereby superseding earlier middleware technologies, such as Web Services [199], Java RMI [200], CORBA [201], and DCOM [202]. However, like their Phase I-era predecessors, they only provide interface description languages and services that allow distributed objects to be defined, located, and invoked, whereas, as Gandon argued of those predecessors, complex cooperation and heterogeneous information require knowledge-level interactions among agents [50]. This goes beyond the method-invocation paradigm and requires communication languages that provide a structural and semantic grounding[10].

2.4.7 Multi-Agent Communication Languages

The ACL defines the types of messages with their meaning. A number of inter-agent communication languages were proposed, but by the Phase I era only two had gained traction: Knowledge Query and Manipulation Language (KQML) [203] and FIPA-ACL [148]. In that era they remained the only agent communication languages that were widely implemented and used [50]; the LLM-era protocols that later took up the same interoperability role (MCP, A2A) are surveyed in Section 2.5. These languages are based on speech act theory and performatives [184], which were originally developed to model human communication.

In this thesis, FIPA-ACL is briefly presented because it has a more important role than KQML in the Phase I demonstrations (Chapter 4). Since both languages are based on speech act theory, they have many similarities.

[9] The evaluation was done in 1999 with early versions of FIPA-compliant platforms, among them FIPA-OS and JADE.

[10] This is the first part where the Semantic Web environment and MAS have similar interests; the need for a common language in MAS meets the W3C initiative of XML and RDF languages.

For example, if an agent wants to inform another agent about some fact, it may send an *INFORM* message if it is using FIPA-ACL and a *TELL* message if it is using KQML.

2.4.7.1 FIPA-ACL

FIPA-ACL is the communication language for FIPA agents. It has formal semantics and several concrete syntaxes, and its specification is split across several documents. First, FIPA Communicative Act Library [149] gives the communicative acts and introduces the semantic model behind the language. Second, FIPA ACL Message Structure Specification [148] specifies the abstract structure of an ACL message. Third, specifications [204] define concrete transport encoding schemes for FIPA-ACL. In addition to these, there are two sets of specifications that are not strictly part of the FIPA-ACL specification but closely related: the FIPA Content Language Library Specification [205] gives a registry for content languages and the FIPA Interaction Protocol Library Specification [206] gives a registry for interaction protocols.

A FIPA-ACL message contains a set of message elements. The only mandatory element is the communicative act type, but typically a message contains additionally at least the sender and receiver information, and the content of the message expressed in some content language.

As depicted in Figure 30 (Appendix A), FIPA-ACL has 22 predefined communicative acts, four of which are primitive communicative acts[11] and the rest are composite communicative acts. Composite communicative acts are composed of primitive communicative acts either by substitution or sequencing. A given agent has to implement only those communicative acts it needs and the *NOT-UNDERSTOOD* act which every FIPA agent must implement.

2.4.8 Interaction Protocols

Agent communication typically involves more than sending a single message with the correct syntax and semantics; it usually comprises several messages that form a dialogue. These dialogues frequently follow commonly occurring patterns, or conversation policies, and communication is enhanced when both participants are explicitly aware of the particular pattern in which they are engaged [207].

Thus, above the syntactic and semantic aspects is the pragmatic aspect of communication protocols, i.e., normative social behavior rules [50]. The protocols needed for agents go beyond the client-server model: in terms of high-level communication languages, they describe the sequences of messages that can occur and their meaning, ensuring the coherence required to initiate, carry out, and terminate a cooperation. Protocols have a normative force that requires that the agents behave felicitously, but in exchange they ensure a known and agreed set of possible events and outcomes.

The protocols define communication patterns that can be used to set up relationships that define a special organizational pattern [50]. For instance, Figure 32 (Appendix A) depicts one of the most well-known interaction protocols, i.e., the contract-net [208], where agents in need of services distribute calls for proposals to other agents. The recipients evaluate those requests and submit bids to the originating agents, which use the bids to choose and then award contracts to selected agents. Contract-net can be used in many cases, for instance, in a task allocation problem to decide which is the best agent to be assigned the responsibility of solving a problem. In institutionalizing interactions, the protocol generates an organizational structure by dynamically assigning two roles: manager or contractor.

2.4.9 Content Languages

As discussed earlier, agent communication languages typically define the outer language of agent communication but not the content of the message; content languages describe that content.

By the Phase I era many content languages had been proposed, but few had gained wide acceptance. The two most important for this thesis are briefly presented[12].

FIPA Semantic Language. FIPA Semantic Language (SL) [209] is a formal content language for the FIPA-ACL, expressing objects, propositions, and actions [38] across the standard communicative acts; it is

[11] *INFORM, REQUEST, CONFIRM, and DISCONFIRM*

[12] In the Phase I demonstrations, Semantic Web languages were used as content languages instead of agent content languages.

layered into three profiles (FIPA-SL0–SL2) of increasing expressivity, up to first-order predicate and some modal logic.

Knowledge Interchange Format (KIF). The Knowledge Interchange Format (KIF) [210] is a high-level interchange language for inter-program communication [38], with a Lisp-derived syntax close to that of FIPA-SL.

2.4.10 Critical Synthesis and Positioning

The classical agent program standardized *coordination and communication* well (FIPA architecture, ACL, interaction protocols, content languages) but left two decisive gaps. First, its deployed agents did not *learn*: learning architectures existed (Section 2.4.4), but behavior was hand-engineered through rules, ontologies, and fixed protocols. Second, the agent stack of that era, including the agent technologies used here, did not *converge with the open Web*, interoperating only within closed, pre-agreed deployments rather than the decentralized Web the Semantic-Web vision (Section 2.3) targeted. This thesis reuses the durable part—server-centric, protocol-disciplined integration—in its Phase I artefacts (A1, the interaction-protocol adaptation engine; A2, the ontology-driven B2B negotiator; and A3, DYNAMOS; Table 4), while its later phases address the two gaps: data-driven learning in Phase II, and in Phase III governed LLM agents (Definition 3.2.5) that, in HADA, are designed to connect through the open MCP/A2A protocol layer (Section 2.5; MCP is exercised in the prototype, A2A only in the reference design) and whose capability is matched to auditability and human accountability (contrast Section 2.5.6; the "absence of genuine learning" hinge is taken up in Section 8.1).

2.5 LLM Agents

Within the Phase III framing of this thesis (Section 1.1.2), this section defines the notion of an *LLM agent*: a software agent whose core decision or policy loop is mediated by an LLM and scaffolded with tool use, memory, and control mechanisms (Section 2.5.2). Building on the pre-LLM foundations in Sections 2.2, 2.3, and 2.4, it is outlined how classical agent capabilities are re-instantiated when the agent's reasoning substrate is a foundation model rather than a hand-coded planner or purely symbolic architecture. The design space is delineated—LLM planning and orchestration roles, RAG, function/tool invocation, external and episodic memory, reflection/critique loops, and multi-agent coordination—then emerging agent protocols (e.g., MCP, A2A) and developer frameworks that operationalize these patterns are surveyed. The distinctive affordances (rapid task adaptation, language-native interfacing) alongside new failure modes are surfaced. This baseline revisits the integration question of RQ I.2 (Table 2) for the LLM era by clarifying what must interoperate between LLM-centric agents and Web/agent protocols, and it motivates the governance and assurance concerns addressed later in Phase III.

2.5.1 Emerging Field of LLM Agents

The rise of LLMs following the introduction of the Transformer in 2017 [52] has dramatically reshaped software agent development, opening up new possibilities and challenges. Before LLMs, agents often relied on meticulously crafted rules, ontologies, and knowledge bases, which limited their adaptability and generalizability [137, 211]. While these approaches provided a degree of control and explainability, they struggled to handle the complexity and ambiguity of real-world scenarios. LLMs, trained on massive datasets, offer a fundamentally different approach: they possess an unprecedented ability to understand and generate natural language, reason, and even exhibit some degree of common sense, albeit imperfectly [51, 212, 213]. This inherent capability allows LLM-powered agents to interact with users and environments in a much more natural and flexible way, reducing the need for extensive manual programming and domain-specific knowledge engineering.

The integration of LLMs with agent frameworks has led to the emergence of "LLM-powered agents" or "LLM agents", capable of performing a wider range of tasks than their predecessors. A simple definition of an LLM agent is an artificial entity with prompt specifications (initial state), a conversation trace (state), and the ability to interact with its environment, such as tool usage (action) [214, 215] (formalized in Definition 3.2.1). These agents can leverage LLMs for various functions, including Natural Language Understanding (NLU), Natural Language Generation (NLG), planning, decision-making, and even tool use [216, 43, 217]. For instance, an LLM agent can be prompted to break down a complex task into smaller, manageable subtasks, search the Web for relevant information, interact with APIs to execute actions, and summarize the results for the user [44, 45]. Frameworks like LangChain [218] and AutoGen [48] provide tools and abstractions to facilitate the development and deployment of these agents, enabling developers to chain together LLM calls, external tools, and custom logic. This transition marks a significant shift from symbolic AI to a more data-driven, emergent approach to agent development.

Despite the significant advancements, LLM agents also present unique challenges. One major concern is the potential for "hallucinations," where the LLM generates plausible-sounding but factually incorrect or nonsensical information [219]. This phenomenon can lead to unreliable agent behavior and requires careful mitigation strategies, such as grounding the LLM's responses in external knowledge sources or employing techniques like RAG [220]. Other challenges include ensuring the safety and ethical behavior of agents, managing the computational cost of running large models, and addressing biases inherent in the training data [221, 222]. Furthermore, the "black box" nature of LLMs can make it difficult to understand the reasoning behind an agent's actions, hindering debugging and limiting trust in critical applications [223].

Ongoing research on LLM-powered agents focuses on addressing these limitations and exploring new capabilities within the emerging MAS, including methods for improving the factual accuracy and reliability of LLMs, more robust and explainable agent architectures, and effective evaluation metrics to assess agent performance across diverse tasks [224, 225, 226]. The convergence of LLMs and software agents represents a significant step toward building more general-purpose, adaptable, and intelligent systems, with the potential to transform how computers are interacted with and complex tasks are automated.

2.5.2 Single LLM Agent

Figure 7 illustrates a high-level architecture of an LLM-based agent[13], comprising several key components: **User Requests**, which serve as input from the user that initiates the agent's operations; the **Agent**, which is the central module acting as the coordinator and managing interactions with all other components; **Tools**, which are external modules or systems the agent can utilize to gather data or perform specific tasks; **Memory**, a mechanism for storing and retrieving past interactions or knowledge, either short-term or long-term; and **Planning**, a module responsible for devising a structured approach to solve complex tasks by breaking them into subtasks. Some sources, such as [46], list the underlying **LLM** (the foundation model, the core of the agent) and **Action Execution** (distinct from tool use) as separate components that are not shown in Figure 7.

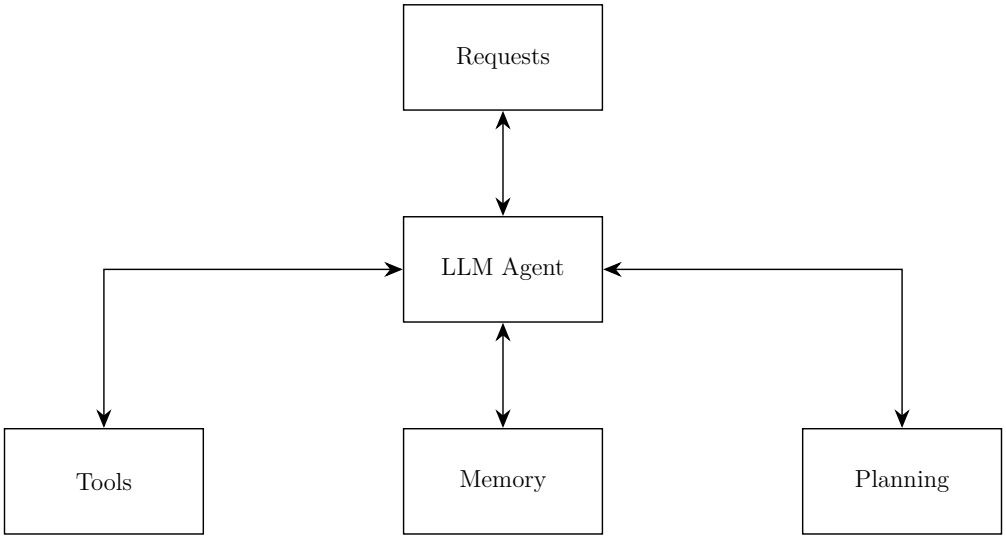


Figure 7: Single LLM agent core components.

Concrete instantiations of these components include long-term memory with reflection and planning, as in generative agents [227], and retrieval-augmented generation that grounds an agent's outputs in external sources [228].

2.5.2.1 Memory

The memory module stores the agent's internal logs, including prior thoughts, actions, observations, and interactions with users. Memory is generally categorized into **short-term memory**, which captures immediate context and is limited by the LLM's context window through in-context learning; **long-term memory**, which stores persistent knowledge and past experiences using external vector databases for scalable retrieval; and **hybrid memory**, which combines short-term and long-term memory to enhance reasoning and accumulate experiences over time. Memory can take various forms, such as natural language, embeddings, databases, or structured lists. Ghost in the Minecraft (GITM), for example, stores a text-based memory as a key-value dictionary whose keys are sub-goals and whose values are lists of successful action sequences [229].

2.5.2.2 Tools and Actions

Tools enable LLM agents to take actions and interact with external systems, such as APIs, databases, knowledge bases, and external models. They support workflows that help the agent gather information or complete subtasks, like using a code interpreter to generate charts in response to user queries [215].

Different frameworks and methods for tool integration include **MRKL**, which combines LLMs with external knowledge sources and discrete reasoning modules [230]; **Toolformer**, which self-trains LLMs to decide when and how to call external APIs [231]; **Function Calling**, which exposes predefined tool schemas the model can invoke programmatically [232]; and **HuggingGPT**, which uses an LLM as a planner/controller to orchestrate specialist models for complex, multi-step tasks [233].

[13]Image redrawn from: `https://www.promptingguide.ai/research/llm-agents`

2.5.2.3 Planning

Planning Without Feedback. The planning module is responsible for breaking down a complex task into smaller, manageable steps that the agent can address individually to fulfill the user request. This process enhances the agent's ability to reason effectively about the problem and reach a reliable solution. By utilizing an LLM, the planning module generates a detailed strategy comprising various subtasks necessary to address the user's query. Notable techniques for task decomposition include Chain of Thought (CoT) [213] and Tree of Thoughts (ToT) [234], which fall under single-path and multi-path reasoning, respectively, as categorized by [43]. As shown in Figure 39 (Appendix A), single-path methods (e.g., CoT) contrast with multi-path approaches (e.g., CoT-SC, ToT) that explore multiple trajectories before selection.

Beyond such single- and multi-path schemes, Meyerson et al. show that, once a complex problem is decomposed into near-atomic subtasks, reliability can be achieved by a Massively Decomposed Agentic Process (MDAP) that uses many small "micro-agents," voting, and simple structural filters, rather than ever-larger base models [235]. This suggests a distinction, offered here as an interpretation, between an unsolved phase, where strong reasoning models and recursive planning are needed to discover an effective decomposition, and a solved phase, where execution reduces to an engineering problem of extreme task decomposition, error-correcting orchestration, and strict rails—although orchestration alone cannot substitute for genuine reasoning while the decomposition itself is still unknown.

Planning With Feedback. The planning methods described above lack a feedback mechanism, making it difficult to handle long-term planning required for solving complex problems. To overcome this limitation, models can employ a system that iteratively revises and improves the execution plan by considering prior actions and observations. This process helps in correcting errors and refining the final output, which is especially crucial in challenging real-world tasks where iterative learning through trial and error is essential.

Two prominent approaches that incorporate feedback mechanisms are ReAct [44] and Reflexion [45] that are illustrated in Figure 40 (Appendix A). The left side of Figure 40 contrasts Reason Only and Act Only with ReAct, which combines reasoning and acting by alternating between three steps: Thought, Action, and Observation. This loop is repeated multiple times, allowing the model to incorporate feedback from the environment in the form of observations. Additional feedback can come from human inputs or model evaluations. This iterative approach enhances the model's capacity to tackle more complex questions effectively [44]. The right side of Figure 40 illustrates the Reflexion framework [45], which enhances language agents' performance through linguistic feedback instead of traditional weight updates: an Actor, an Evaluator, and a Self-reflection module (each an LM), with a Trajectory (short-term memory) and Experience (long-term memory), act on an external environment, receive observations or rewards, and turn external and internal feedback into reflective text stored in an episodic memory buffer, improving performance across tasks such as coding, reasoning, and decision-making.

These single-agent capabilities—reasoning, tool use, and self-reflection—are compared critically against the governance dimension this thesis targets in Section 2.5.6, together with the multi-agent frameworks introduced next.

2.5.3 Emerging LLM Agent Protocols

As illustrated in Figure 41 (Appendix A), an initial wave of agent protocols in 2024—model-to-tool (MCP) and agent-to-agent (the Agent Network Protocol, ANP, and Agora [236]) specifications—began to address the critical lack of standardized interfaces that had hindered LLM agents' ability to scale, interoperate, and integrate with external tools and data sources. In early 2025 this foundation was expanded by streamlined agent-to-API formats (e.g., *agents.json* [237]) and further inter-agent infrastructures supporting richer, structured messaging between agents. Looking ahead, the survey of [46] anticipates layered protocol architectures and group-centric protocols (an "Agent Mesh"), while new agent frameworks such as OpenManus and OWL (an agent framework, not the Web Ontology Language) continue to appear.

2.5.3.1 Model Context Protocol (MCP): LLM and Tools Integration

The Model Context Protocol (MCP), introduced in late 2024, is an open-source standard designed to link AI assistants with the systems where data resides—such as content management platforms, enterprise tools,

and software development environments[14]. Its primary goal is to enable advanced models to generate more accurate and contextually relevant outputs.

As AI assistants become more widely adopted, even the most advanced models remain limited by their lack of direct access to real-time data, and integrating each new data source typically demands a custom-built solution. MCP replaces such piecemeal, one-off integrations with a single, open standard for linking AI systems to data: acting as a middleware layer between AI assistants and diverse data environments, it abstracts away integration complexity so that developers can focus on higher-level application logic.

At its core, MCP follows a client-server architecture where a host application can connect to multiple servers, as illustrated in Figure 42 (Appendix A). The architecture includes **MCP Hosts**, which are programs, IDEs, AI tools, or LLM Agents that want to access data through MCP; **MCP Clients**, which are protocol clients that maintain 1:1 connections with servers; **MCP Servers**, which are lightweight programs that each expose specific capabilities through the standardized MCP; **Local Data Sources**, which refer to the local computer's files, databases, and services that MCP servers can securely access; and **Remote Services**, which are external systems available over the Internet (e.g., through APIs) that MCP servers can connect to.

2.5.3.2 Google Agent2Agent Protocol (A2A)

Agent2Agent (A2A) is an open protocol announced by Google in April 2025 that aims to complement the MCP, which provides helpful tools and context to agents [238]. The A2A protocol is designed to allow AI agents to communicate with each other, securely exchange information, and coordinate actions on top of various enterprise platforms or applications.

In its initial (2025) specification, the A2A protocol facilitates communication between independent AI agents through several key elements [239]: the **Agent Card** (public JSON metadata detailing an agent's capabilities, endpoint, and authentication for discovery); the **A2A Server** (an agent exposing an HTTP endpoint that implements the A2A protocol methods) and the **A2A Client** (an application or agent sending requests, e.g., tasks/send, to a server URL); the **Task**, the core work unit, with a unique ID and lifecycle states (submitted, working, input-required, completed, failed, canceled); and the **Message** (a communication turn between client and agent) and **Artifact** (agent-generated outputs such as files or structured data), both composed of **Parts** (TextPart, FilePart, or DataPart). For long tasks, **Streaming** via tasks/sendSubscribe delivers real-time progress as Server-Sent Events (TaskStatusUpdateEvent or TaskArtifactUpdateEvent), and **Push Notifications** send task updates to a client-provided webhook configured via *tasks/pushNotification/set*.

A different kind of specification, which packages procedural knowledge rather than an interface, followed in late 2025. An *Agent Skill*, introduced by Anthropic in October 2025 and published as an open specification in December 2025 [240, 241], is a folder holding a `SKILL.md` file—a name and a description, followed by instructions—and, optionally, scripts and other resources; the agent reads only the name and description of each installed skill and loads the rest when a task calls for it. Anthropic describes skills as a complement to MCP that can teach workflows using external tools [240]. In this thesis's reading, MCP supplies tools and context, A2A connects agents, and a skill describes how a task is done; the skills listed in an A2A Agent Card [239] advertise capabilities and are a different notion. A loaded skill becomes part of the agent's prompt specification (Definition 3.2.1; for its 2026 standardization, see Appendix A, "The three phases reassessed in 2026").

2.5.4 Multi-Agent Conversation Patterns in LLM Era

Figure 43 (Appendix A) illustrates the architecture of an agent-based conversation framework known as AutoGen, emphasizing three key capabilities: Conversable Agent, Agent Customization, and Flexible Conversation Patterns. A "Conversable Agent" is a modular entity with customizable components (e.g., OpenAI models, Python modules, and user-defined tools) tailored to specific tasks; in "Multi-Agent Conversations," agents interact collaboratively or competitively through directed communication channels; and "Flexible Conversation Patterns," such as joint chats and hierarchical chats, let agents either collaborate equally or follow structured command hierarchies.

Whereas AutoGen organizes collaboration through *free-form* conversation among configurable agents, MetaGPT [242] encodes the opposite discipline: human Standardized Operating Procedures (SOPs) are baked into the prompt sequence so that agents take fixed software-company roles—product manager, architect, engineer, QA—along an assembly line. Each role consumes and emits *structured intermediate artefacts* (user

[14] `https://modelcontextprotocol.io/introduction`

stories, API designs, design documents) rather than raw chat turns, which lets downstream agents verify upstream results and reduces error propagation in long, multi-step tasks. The two thus mark the ends of a spectrum of multi-agent orchestration, from emergent dialogue (AutoGen) to procedural, role-segmented workflows (MetaGPT). Both are positioned critically against the thesis's own governed artefacts (HADA, CCL) in Section 2.5.6.

2.5.5 Prominent LLM Agent Frameworks and Libraries

The rapid expansion of LLM-agent tooling is propelled by a diverse open-source ecosystem; the landscape summarized here is a snapshot as of mid-2025. LangChain (LangChain, Inc.) supplies modular building blocks for tool use and memory [243], while LangGraph (LangChain, Inc.) layers graph-based, stateful control on top [244]. AutoGen (Microsoft Research) enables multi-agent conversational collaboration [48, 245], and LlamaIndex (LlamaIndexAI, Inc.) focuses on data connectors and retrieval pipelines [246]. CrewAI (CrewAI) provides role-oriented orchestration [247], and Microsoft's Semantic Kernel targets enterprise integration [248]. Complementary projects—Haystack (deepset) [249], Embedchain (Embedchain) [250], Dify (LangGenius, Inc.) [251], and OpenAgents (XLang Lab, University of Hong Kong) [252]—extend the landscape across RAG pipelines, agent hosting, and end-to-end app platforms. Table 6 compares a representative subset of these frameworks, together with MetaGPT, ADK, Genkit, and Marvin, by core paradigm, key features, and typical use cases.

Table 6: Comparison of Prominent LLM Agent Frameworks.

Framework	Core Paradigm	Key Features	Best Use Cases
LangChain[243]	Composable agentic framework	Modular tools, memory management, large ecosystem	Chatbots, assistants, document analysis, recommendations, research
LangGraph[244]	Graph-based stateful agents	Graph workflows, branching/parallelism, step-level debugging	Planning, reflection, multi-agent coordination
AutoGen[48, 245]	Conversational multi-agent systems	Custom agents, auto-chat loops, tool use, feedback integration	Problem-solving, workflow automation, multi-agent conversations
LlamaIndex[246]	LLM-centric data layer	Data ingestion, retrieval, knowledge graphs, RAG pipelines	Q&A on private documents, summarization, specialized search
CrewAI[247]	Role-oriented MAS orchestration	Role-based agents, collaboration, flexible memory	Coordinated problem-solving, diverse expertise, simulations
MetaGPT[242]	Role/SOP-based meta-programming MAS	Assembly-line roles, structured intermediate artefacts (PRD/design docs), SOP-reduced hallucination	Code generation, structured multi-step software tasks
Semantic Kernel[248]	Enterprise AI orchestration	Plugins, multi-language SDKs, enterprise-grade security	AI integration in enterprise systems, high security/compliance
Agent Development Kit (ADK)[253]	Multi-agent development toolkit	Model abstraction, agent orchestration, evaluation suite, tool calling	Complex multi-agent workflows, research on agent behaviors, evaluation pipelines
Genkit[254]	Code-first GenAI workflow framework	Flow orchestration, prompt/version control, tool calling, RAG, observability, Cloud Run deployment	Content generation, RAG apps, end-to-end GenAI services with monitoring
Marvin[255]	Pythonic agent workflow library	Decorator-based tasks, threaded conversations, async support, minimal boilerplate	Developer-friendly agent workflows in Python apps, data/ML pipelines, rapid prototyping

Maintainers / organizations: LangChain—LangChain, Inc.; LangGraph—LangChain, Inc.; AutoGen—Microsoft (Research); LlamaIndex—LlamaIndexAI, Inc.; CrewAI—CrewAI (open-source; led by João Moura); MetaGPT—DeepWisdom (open-source); Semantic Kernel—Microsoft; ADK—Google; Genkit—Google (Firebase); Marvin—Prefect, Inc.

2.5.6 Critical Synthesis and Positioning

The approaches surveyed in this section are best read not as competitors but as advances along a single axis—raw agent *capability*: stronger reasoning (ReAct), self-correction (Reflexion), learned tool use (Toolformer), and the orchestration of several agents toward a goal (AutoGen, MetaGPT). The contribution of

this thesis lies on an *orthogonal* axis—governance, auditability, and human–AI alignment for decisions in regulated domains. Table 7 makes the contrast explicit along three dimensions that matter for such settings: whether the reasoning is inspectable, whether tool or role use is governed (constrained, permissioned, logged), and whether there is an audit trail tying a decision to an accountable human.

Table 7: Positioning recent LLM-agent approaches against the governance dimension this thesis targets. The agent works optimize capability, while the Tropos and Constitutional AI rows mark design-time and training-time governance for contrast; the thesis artefacts (HADA, CCL) add governed, auditable, human-aligned decision-making. Yes = enforced by the architecture at run time; Partial = supported but optional or unrecorded; Limited = present but not exposed; Design-time or Training-time = addressed only before run time; No = not addressed. HADA and CCL ratings are evaluated in Sections 7.5 and 7.7.

Approach	**Inspectable reasoning**	**Governed tool/role use**	**Audit trail & human accountability**
ReAct [44]	Yes — explicit thought/action/observation trace	No — unconstrained tool calls	No
Toolformer [231]	Limited — tool calls learned, not exposed	No	No
Reflexion [45]	Partial — verbal self-reflection	No	No
AutoGen [48]	Partial — visible conversation	No — orchestration without policy	Partial — optional human input, no accountability record
MetaGPT [242]	Partial — SOP/role artefacts	Role-structured, but not policy-governed	No
Tropos / Secure Tropos [78, 256, 257]	Design-time — goal, actor, security & trust models	Primarily design-time; requirements-based run-time adaptation exists [80], but no per-decision binding	Partial — delegation & trust modeled [257]; no run-time decision ledger
Constitutional AI [258]	No — values internalized in the model	Training-time value alignment, not run-time control	No
HADA (this thesis)	Yes — logged decisions and explanations	Yes — HADA Controller (policy-enforced orchestration), RACI roles, Values Catalogue	Yes — audit ledger, human-in-command
CCL (this thesis)	Yes — typed diffs + Impact Briefs	Yes — risk-tiered CI gates (by design; automated CI not executed in the demonstration)	Yes — immutable ledger, human merge

Two clusters dominate the recent literature, and both leave the same gap. Single-agent reasoning and tool-use methods (ReAct, Toolformer, Reflexion) sharpen how an agent thinks and self-corrects, yet they leave governance and logging of tool invocation largely to the application developer and do not model who is accountable for a resulting decision. Multi-agent orchestration (AutoGen, MetaGPT) coordinates several agents toward task completion and throughput and, although some frameworks provide human-input hooks or role definitions, again optimizes for getting the task done rather than for value-aligned, auditable decisions owned by a responsible human. In neither case is the agent's behavior bound to organizational objectives, permissions, or an immutable record of why an action was taken. Among the work surveyed here, closest to the thesis's concern is a run-time oversight line—agent visibility and logging [259], meaningful human control [260], and human-oversight requirements in regulation and voluntary frameworks [261, 262]—which states requirements rather than supplying an architecture; HADA and CCL bind them into one decision-level architecture and change process (Sections 6.1 and 6.3). A further line does engage governance explicitly, but *earlier in the lifecycle*: AOSE models goals, actors, security, and trust primarily at *design time*—Tropos and its security-oriented extension Secure Tropos [78, 256]—while Constitutional AI internalizes value constraints into the model at *training time* [258]. These lines are complementary rather than competing; the distinguishing move of this thesis is to bind such constraints to *each run-time decision*—with an accountable human and an immutable record—and to carry that binding into how the decision logic is *maintained*.

The Phase III artefacts of this thesis target precisely that gap. HADA (Section 6.1) wraps such agents in a policy-controlled, audit-logged governance layer with explicit human sign-off, and CCL (Section 6.3) governs how agent-proposed changes enter regulated decision logic without eroding human accountability. Interface standards (MCP and A2A) and packaging formats such as Agent Skills address interoperability and reuse rather than this gap: a skill is plain, versionable text that changes how an agent acts, yet the format itself neither tests the effect of a change nor records who approved it [241], and an early security analysis reports

that skills run with implicit trust and minimal vetting [263]. This positioning reflects the thesis's own reading of the field's history: earlier agent waves were bounded by their era's technology [25], and what this thesis treats as distinctive for the current wave is not raw capability alone but its deployment under governance and alignment constraints.

2.6 LLM and DRG Related Studies

Section 2.6.1 reviews core advances in LLMs relevant to the Phase III tasks. Section 2.6.2 motivates governance-grade evaluation through alignment and safety considerations. Section 2.6.3 surveys general LLM benchmarks and leaderboards, clarifying what they measure—and what they miss for CaseMix reasoning. Section 2.6.4 and Section 2.6.5 summarize the version-control and MLOps practices that CCL builds on. Section 2.6.6 introduces the NordDRG CaseMix system—the rule base the benchmark operationalizes—and establishes domain context. Section 2.6.7 synthesizes prior DRG-LLM studies to surface the remaining gaps. Section 2.6.8 describes production DRG grouper software and situates the benchmark within that ecosystem.

2.6.1 LLMs

The breakthrough Transformer architecture introduced by Vaswani *et al.* [52] ignited the post-2017 natural-language processing renaissance. Initial encoder models such as BERT [264] were quickly followed by vast autoregressive generators—GPT-3 (2020) [51], GPT-4 (2023), and others—whose parameter counts reached into the hundreds of billions by the early 2020s and which display emergent strengths in generation, retrieval, and reasoning. Concurrent strands of work explored alternative training frameworks (T5's text-to-text paradigm [265]), compute–data scaling laws (Chinchilla [266]), and extreme model sizes, exemplified by PaLM's 540-billion-parameter Pathways network [267]. Meanwhile, open-weight initiatives such as OPT [268] and Llama-2 [269] have broadened community access to state-of-the-art capabilities, with subsequent reinforcement-learned reasoning models pushing the reasoning frontier further [270]. Comprehensive surveys map this rapid trajectory from early generative models to ChatGPT, situating LLMs within the wider generative-AI landscape and cataloging key architectural and application trends [215, 271].

Since late 2022, chat-style interfaces—epitomized by ChatGPT—have moved LLMs beyond research into mainstream workflows. By masking low-level prompts behind natural conversation, these interfaces transform LLMs into a versatile cognitive layer that clinicians, lawyers, educators, and business users can harness with minimal technical overhead.

2.6.2 AI Alignment and Safety

As technical frontiers expand, the focus of responsible AI practice is shifting from *capability* to *governance*. Ensuring that AI model behavior aligns with human values and organizational objectives is a critical—and multidimensional—challenge [55], with well-documented risks of opacity, disparate impact, and governance failures in high-stakes domains [272]. The alignment problem is often decomposed into *outer alignment* (does the specified objective capture what we actually want?) and *inner alignment* (does the learned optimizer in fact pursue that objective, especially on inputs outside the training distribution?) [273, 274]. Practical failures arise from reward misspecification, specification gaming, and goal misgeneralization, particularly under non-stationarity [275, 274].

Recent advances combine data-driven optimization with human oversight to better shape training objectives. Preference learning and reward modeling translate human (or AI-assisted) judgments into training signals [276, 277]; instruction tuning with human or AI feedback (e.g., RLHF and Constitutional AI) further reduces harmful behaviors and improves adherence to stated guidelines [212, 258]. However, these training-time techniques alone do not meet the human-oversight and traceability requirements that regulation places on high-risk systems (e.g., EU AI Act, Article 14 [261]), which require that humans can effectively understand, monitor, and override such systems; this thesis reads that requirement as calling for an audit trail of decision logic, deliberation, and traceable approval of consequential changes.

2.6.3 LLM Benchmarks and Leaderboards

Benchmark suites and public leaderboards are central to tracking progress in LLMs. Early collections, such as GLUE [278] and SuperGLUE [57], gave way to broader, multi-skill batteries—e.g., BIG-bench [279] and MMLU [280]—probing factual knowledge, reasoning, multilinguality, and instruction following, while

efforts such as the BabyLM Challenge [281] target sample-efficient pretraining instead. Meta-evaluations like HELM [282] further standardize conditions and report robustness, bias, and efficiency alongside accuracy.

These suites are deliberately *horizontal*: HELM and its peers measure general-purpose competence (accuracy, robustness, bias, efficiency) across broad task batteries, and are by construction domain-agnostic. That breadth is also their limit for the present work—they neither encode nor certify the correctness of a specific regulated rule base, such as the control flow of a hospital-funding (CaseMix/DRG) grouper, where a single mis-applied exclusion changes the reimbursed group. The NordDRG AI Benchmark introduced in Section 6.2 is the *vertical* complement to this horizontal evaluation: a domain-specific, governance-grade benchmark that tests exactly the rule-level reasoning and faithful grouper emulation that general leaderboards leave unmeasured.

A second, more recent strand evaluates models not as static predictors but as *agents* that complete interactive, tool-using tasks: AgentBench exercises LLM agents across eight distinct environments [226]; GAIA poses real-world assistant questions whose unambiguous answers require tool use and multi-step reasoning [283]; SWE-bench scores whether an agent's patch makes a real repository's test suite pass [284]; τ-bench measures whether an agent follows *domain-specific policies and rules* under tool use and simulated-user interaction [285]; and LegalBench separates *stating* a rule from *applying* it to facts across collaboratively built legal-reasoning tasks [286]. These benchmarks share a design value with the present work—grounding the score in an objective, executable outcome rather than surface text—but remain *horizontal* in scope, certifying general task competence rather than the correctness of one regulated rule base. The NordDRG AI Benchmark is the vertical complement to *both* families: like the agent benchmarks it scores against an objective ground truth (the official grouper output), but unlike any general suite it tests faithful emulation of a specific hospital-funding control flow. It is, however, an LLM *benchmark*, not an agent architecture (unlike HADA and CCL in Sections 6.1 and 6.3): it measures rule-level reasoning and grouper emulation, not governed multi-agent orchestration.

A parallel concern is *evaluation reliability*. Many agent and open-ended benchmarks rely on an LLM acting as judge, which is known to carry position, verbosity, and self-preference biases [287, 288, 289], and static leaderboards are increasingly vulnerable to benchmark-data contamination as test items leak into pretraining corpora [290]. The NordDRG AI Benchmark sidesteps the first and mitigates the second: it is scored by *deterministic exact match* against the official grouper, so no model judge is interposed; its rule tables are public, so contamination cannot be ruled out, but scoring requires reproducing the triggering rule row for specific cases rather than recalling table text. Within the vertical, regulated-domain setting it sits alongside holistic clinical batteries such as MedHELM [291], filling the hospital-funding (CaseMix/DRG) slot that general medical-LLM evaluations leave open.

Results from these suites are surfaced on widely watched leaderboards. The Hugging Face *Open LLM Leaderboard*[15], in its original (2023) edition, aggregated automatic metrics (e.g., MMLU, ARC, HellaSwag) for open-weight models. The LMSYS/LMArena *Chatbot Arena* leaderboard ranks dialogue quality via blind, pairwise human preferences [292]. Multi-turn evaluations such as MT-Bench [287], graded by a strong LLM judge, probe conversational and instruction-following ability across categories including writing, reasoning, math, and coding.

2.6.4 Version Control Systems for Governed Change

Version Control Systems (VCSs) provide the provenance, review, and release mechanics that governed, audit-heavy work depends on. Centralized systems (e.g., CVS, Subversion) established change history and branching; distributed VCSs (DVCSs), such as Git and Mercurial, popularized immutable, content-addressed commits, offline branching, and robust merge workflows [293, 294]. Contemporary options extend this space with integrated platforms (Fossil, with built-in wiki/issue tracker) and patch-theoretic designs (Darcs; Pijul's formal patch model) [295, 296, 297]. Newer Git-compatible tools like Jujutsu (`jj`) focus on simpler UX while remaining compatible with Git storage/hosting [298, 299].[16]

[15] Open LLM Leaderboard: `https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard`

[16] This study uses "Git-style" to denote any VCS that offers immutable commit identifiers, branching/tags, reviewable change bundles, and CI hooks. Alternatives, such as Mercurial, Fossil, Pijul, or Jujutsu, can support the workflow proposed in this study so long as these properties are available. Git is used here pragmatically due to ubiquity in partner environments and mature integrations with common forges (GitHub/GitLab/Gitea), enterprise CI/CD, signed commits/tags, branch protections, and audit logging.

Across the VCS ecosystems surveyed above, governance-relevant capabilities are broadly available: (i) immutable, addressable commits; (ii) reviewable change bundles (PRs/MRs) with recorded approvals; (iii) CI gate integration; (iv) authenticated authorship and protected branches (e.g., signed commits/tags; required reviewers); (v) reproducible rebuilds from a commit hash; and (vi) long-term retention and audit-trail export.

2.6.5 MLOps: Operating ML under Software Discipline

Machine Learning Operations (MLOps) extends Development and Operations (DevOps) with practices and platforms that make model-centric systems reproducible, observable, and continuously deployable in production [86]. Empirical studies [300] from large engineering organizations show that ML work follows a multi-stage, experiment-heavy workflow (data collection/labeling, feature engineering, training, evaluation, deployment, monitoring) with numerous feedback loops and hand-offs between roles. In contrast to conventional software, behavior in ML systems depends not only on code but also on mutable data, configurations, and environments; consequently, operational quality hinges on disciplined tracking, testing, and change management across *all* these artefacts [301, 302].

Core concerns and recurring risks. Foundational accounts highlight "hidden technical debt" specific to ML: boundary erosion between components, entanglement, configuration fragility, data dependencies, and undeclared consumers that silently couple pipelines [301]. To counter these failure modes, MLOps emphasizes: (i) experiment tracking and lineage (parameters, code, data, environments); (ii) data and feature validation in pipelines; (iii) automated tests and CI/CD for training and serving; (iv) canary and shadow deployments; and (v) post-deployment monitoring for performance, drift, and fairness [302, 86]. These controls embody a shift from "train-once" research to continuous delivery of ML behavior under operational budget and governance constraints.

Platforms and tooling. Industrial platforms such as TensorFlow Extended (TFX) codify end-to-end, type-checked ML pipelines—data ingestion, validation, transformation, training, evaluation, and serving—with metadata and replayable execution graphs [303]. Complementary lifecycle tools such as *MLflow* standardize experiment tracking, model packaging (multi-flavor artefacts), and registries that coordinate version promotion across environments [304]. Together these platforms support reproducibility and change control at scale, yet remain largely agnostic to domain-specific governance.

Documentation and governance patterns. Operationalization has spurred documentation artefacts that travel with models and data. *Model Cards* describe intended use, evaluation slices, and caveats for released models; *Datasheets for Datasets* analogously document the provenance and limitations of training data [305, 306]. In MLOps practice these artefacts integrate with registries and CI gates, enabling reviewers to assess risks, compliance, and suitability at deployment time.

2.6.6 NordDRG CaseMix System

The Nordic Diagnosis-Related Groups (NordDRG) classification is the shared CaseMix framework used throughout the Nordic countries to cluster inpatient episodes with comparable resource consumption [307]; the underlying diagnosis-related grouping concept originates in the Yale casemix definition [308]. By assigning each hospital stay to a single DRG, the system underpins transparent benchmarking of costs, outcomes, and efficiency across institutions, regions, and years.

NordDRG is distributed as a set of *definition tables*—about twenty interlinked sheets released annually in Excel, CSV, and JSON formats—that encode the logic connecting diagnosis and procedure codes, age/sex splits, and national activation flags to individual DRG codes [309]. These tables constitute the authoritative rule base for coding software, technical-change management, and policy analysis.

Because each rule specifies explicit inclusion criteria, identical cases are grouped consistently regardless of hospital or country. At the same time, each nation can activate, deactivate, or re-weight DRGs locally, allowing the common core to accommodate diverse clinical practices and tariff structures. This blend of rule-level precision and country-level flexibility makes NordDRG a durable instrument for hospital finance, clinical research, and health-system governance in the Nordic region.

2.6.7 Related Work: DRGs and LLMs

LLMs are increasingly explored as end-to-end engines for hospital-reimbursement workflows (building on demonstrations that LLMs encode substantial clinical knowledge [310, 311]), but the prior work is best read critically: by what each line of it optimizes and what it leaves open. A first cluster targets *coding accuracy*: [312] fine-tune LLaMA on the brief-hospital-course text of 236k MIMIC-IV [313] discharge summaries, reaching 52% top-1 over 738 MS-DRGs; [314] show that a retrieve-and-rank pipeline (ColBERTv2 + GPT-3.5-turbo) assigns correct ICD-10-CM codes for 100 single-term conditions, where the vanilla LLM reaches only 6%; and [315] push zero-shot modeling, inferring DRG-like resource strata directly from longitudinal health trajectories. These results establish that models can *approximate* the CC/MCC logic traditionally hard-coded in groupers—but aggregate accuracy on held-out cases is not the same as *certifying* that a specific regulated rule was applied correctly, and none of these systems exposes *why* a grouping was produced in a form an auditor could check.

A second cluster moves toward *interpretability and rule-level reasoning*. [74] introduce *DRG-Sapphire*, using reinforcement learning with rule-aware rewards to generate physician-validated rationales that cite grouping criteria; [316] highlight token-level evidence in the discharge summary so coders can inspect what drove a DRG prediction, extending the attention-based explainable-coding line established by CAML [317]; and [318] inject a co-occurrence knowledge graph of diagnoses, procedures, and symptoms into BERT, improving DRG classification over plain BERT on three Chinese hospital datasets. Generic LLMs remain competitive for the coding task itself [319, 320]. Valuable as these are, they explain *individual predictions*; none governs *change* to the rule base itself—how an updated grouping rule is proposed, reviewed, risk-tiered, and recorded—so the question of who is accountable when the logic evolves is left open. That is precisely the gap this thesis's Phase III artefacts target: the NordDRG AI Benchmark (Section 6.2) measures faithful, rule-level grouper emulation rather than aggregate coding accuracy, and the CCL process (Section 6.3) makes agent-proposed rule changes auditable and human-approved.

2.6.8 DRG Grouper Software

DRG groupers are rule-based software engines that map a patient episode's minimum dataset—principal and secondary diagnoses, procedures/interventions, demographics (age/sex), discharge status, and other context—onto a single payment group. While national families differ (e.g., MS-DRG in the United States, AR-DRG in Australia, APR-DRG used by many payers, and NordDRG across the Nordics), they share a governed control flow: MDC entry, ordered rule evaluation, surgical evidence (OR/non-OR), complication/comorbidity handling (CC/MCC or analogous constructs), and versioned annual updates [321, 322, 323, 324, 309].

In practice, groupers are distributed as compiled binaries, libraries, or services, accompanied by technical manuals and release notes. MS-DRG publishes detailed definitions and versioned release documentation [322]; AR-DRG provides a public technical specification describing hierarchy, splitting rules, and episode complexity scoring [323]; APR-DRG documents clinical logic and the severity/risk subclasses layered atop base DRGs [324]; and NordDRG publishes machine-readable definition tables that encode the executable rule graph (ordered grouping rules, diagnosis/procedure properties, age bounds, country-specific versions, and complication-category activation) [309]. Historically, comparative scholarship has framed these systems as transparent, auditable instruments for hospital payment and CaseMix analysis, while noting national tailoring and update governance [321].

2.7 Chapter Summary: Background

Scope. This chapter assembled the historical and technical foundations that the rest of the thesis relies on. The following were traced: (i) the recurring *AI summers and winters* that shape expectations and funding cycles (Section 2.1); (ii) the *Web's* evolution from a hypertext medium to a platform for distributed services (Section 2.2); (iii) the *Semantic Web* vision and its agent-centric aspirations (Section 2.3); (iv) classical *Software Agents* and MAS concepts (Section 2.4); (v) contemporary *LLM agents*, their core architecture, and failure modes (Section 2.5); and (vi) domain-specific *LLM and DRG* studies that motivate the healthcare case work (Section 2.6).

Key constructs and standards. This chapter clarified how knowledge representation (RDF, OWL), rule systems, service and interaction ontologies (e.g., OWL-S), and agent communication frameworks (e.g., FIPA-ACL and interaction-protocol ontologies) collectively define the *integration surface* between data, reasoning, and action on the Web. These ingredients explain both the enduring promise (declarative interoperability)

and the well-documented friction points (heterogeneity, brittleness, and tooling gaps). *This standards baseline is revisited when integrating LLM-based agents (Section 2.5) and when it is applied in the DRG domain (Section 2.6; developed in Section 6.3), where controlled vocabularies and structured tool I/O make model behavior auditable.*

From classical agents to LLM agents. Placing LLM-mediated reasoning within the agent design space, the chapter surveyed the LLM's planning and orchestration roles, patterns for tool invocation and external memory, reflection/critique loops, and multi-agent coordination. It highlighted notable advantages alongside new risks, showing how these trade-offs reshape long-standing agent-engineering questions rather than replace them. *This framing is motivated in the healthcare DRG setting (Section 2.6) and instantiated in Phase III (Sections 6.2 and 6.3), where LLM agents are constrained by domain rules and verifiable tool use.*

Implications for the thesis. This chapter substantially addresses **RQ I.1** in Table 2 by tracing the historical origins of the Web and software agents and by consolidating the core KR, rules, and protocols that underpin the remainder of the work. On that basis, **RQ I.2** is sharpened by specifying the practical *integration surface* between Semantic Web formalisms and software-agent protocols required for runtime interoperability and adaptation, and **RQ I.3** is bounded by situating rule- and ontology-based service description/matching within the broader evolution toward mobile, service-oriented Web usage. These clarifications provide the design assumptions carried into Phase II (**RQ II.1**) and the governance criteria carried into Phase III, where *LLM-based agents with governance and verification* are used to address **RQ III.1** (human–AI decision alignment, demonstrated in a banking sandbox) and, within the DRG domain, **RQ III.3** (governance requirements that let agents contribute to decision-model development without eroding human expert accountability), while an LLM benchmark rather than an agent architecture (A6, the NordDRG AI Benchmark; Section 6.2) addresses **RQ III.2** (emulation, explanation, and application of DRG payment rules).

Limitations of current approaches. Read critically, each era was bounded less by ambition than by what its technology could deliver. The *Semantic Web and classical agents* (Sections 2.3, 2.4) presupposed universal, a priori semantic agreement and hand-engineered ontologies that did not scale, never converged the agent stack with the open Web, and could not learn from data. The *data-centric ML/IR* methods that followed (Phase II; Section 1.1.2) were accurate but bespoke and opaque—a time-bounded observation of their era—improving predictions without explaining or governing them. Recent *LLM-agent* work (Section 2.5.6) sharpens raw capability—reasoning, tool use, orchestration—yet, in the surveyed systems, leaves the governance, auditing, and human accountability of tool and role use largely to the application developer. Each substrate thus hit a wall the next era removed (for the Semantic Web era, cf. [25]), but none, on its own, made agent behavior *governable*. In the thesis's own synthesis, what changed across the three eras was the representational substrate (Section 2.1.2); what none of them solved on its own was governance, auditability, and human alignment—the gap the thesis's artefacts target, the limitation through which Phase II's accurate but opaque models are read (Section 7.4), and the criterion against which Phase III is assessed.

Transition. The subsequent chapters build on this foundation: they operationalize these concepts into artefacts and evaluations, test the proposed integration pathways, and assess where standards, engineering scaffolds, and LLM capabilities suffice—and where additional governance, constraints, or design patterns are required.

3 FORMAL DEFINITIONS

This chapter collects the formal definitions that underpin the analytical framework of this thesis: the key concepts of deep learning and the *Transformer Architecture*, which, since its introduction in 2017 [52], has become the dominant architecture of, and the enabling technology behind, the *LLM*s of the Phase III period (2023–2025) [215], such as *GPT*-based systems (Remark 3.1.22). Consolidating them aims to fix terminology, avoid ambiguity, and provide a shared reference vocabulary; later chapters draw chiefly on the LLM-agent and governed-agent definitions of Section 3.2 (Definitions 3.2.1 and 3.2.5), which presuppose the LLM definition (Definition 3.1.21) for their reasoning core M, and on the decision-making model that the Phase III artefacts share (Definition 3.1.17), while the deep-learning and Transformer definitions are given as background. In terms of the dissertation's *governable-agency through-line* (Section 1.1.2), the framework characterizes the Phase III representational substrate—the foundation models over which the constant software-agent abstraction now reasons—so that the governance and alignment demands of Phase III can be stated precisely (Definition 3.2.5). The chapter also answers a motive stated in the Preface: to say precisely what this thesis means by a learned model and by an agent (Section 8.8 returns to this motive).

3.1 Deep Learning Analytical Framework and Core Definitions

This section establishes the analytical framework and notation used throughout the thesis. Parts of this section, including the notation and the core definitions, have been published in [22]; Definition 3.1.17 is not among them. We distinguish and formally define the *Transformer Mathematical Model*, its constituent *Transformer Mathematical Functions*, and the *Transformer Architecture* (originating with [52]). Section 3.1.1 presents the core concepts and notation; Section 3.1.2 gives the corresponding Transformer-specific definitions, whose relationships Figure 9 summarizes; the formulas of the constituent Transformer functions are given in Appendix A.

3.1.1 Core Concepts and Notation

We fix the basic vocabulary and symbols used throughout the thesis; Remark A.2 in Appendix A records where later chapters deliberately reuse a letter with a chapter-local meaning. Figure 8 summarizes the relationships among the core entities: a *Deep Learning Architecture* specifies a parameterized *Mathematical Model* (a family of functions); a *Training Method* uses a *Dataset* and *Calculation Infrastructure* to fit the parameters; the result is a *Deep Learned Algorithm* (a trained, deterministic procedure).

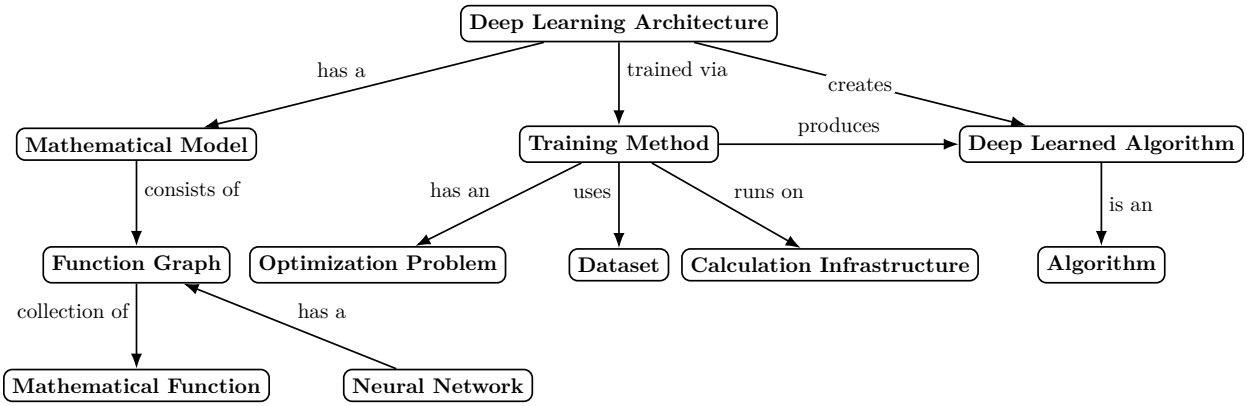


Figure 8: Deep learning core concepts and their relationships. In the electronic version, each box links to its definition.

Notation. We denote by $f_\theta : \mathcal{X} \to \mathcal{Y}$ a parameterized function with learnable parameters θ; training produces $\theta^\star$ and the trained mapping $f_{\theta^\star}$. A dataset is $D = \{(x_i, y_i)\}_{i=1}^N$ (supervised) or $D = \{x_i\}_{i=1}^N$ (unsupervised). A training method is $T = (\mathcal{L}, \Omega, \Psi)$, where $\mathcal{L}$ is a loss, Ω an optimizer (including batching and schedule), and Ψ ancillary procedures (e.g., masking, augmentation, regularization, early stopping, and checkpointing). The loss compares a prediction $\hat{y}$ with a target y, $\mathcal{L}(\hat{y}, y)$; the per-example loss is $\mathcal{L}_i(\theta) := \mathcal{L}(f_\theta(x_i), y_i)$, written $\mathcal{L}(x_i; \theta)$ when there are no targets, and on a finite dataset the training objective

is its average $\mathcal{J}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}_i(\theta)$ (plus any regularization term; Definition 3.1.10). Computation for training is $C_T = (H, S, R)$ with hardware H, software S, and resource constraints R.

Definition 3.1.1 (Function)**.** A *Function* is a (deterministic) mapping $f : \mathcal{X} \to \mathcal{Y}$ assigning each $x \in \mathcal{X}$ a unique $y = f(x) \in \mathcal{Y}$. If randomness is used, a seed $s \in$ *Seed* is treated as part of the input, so that $f : \mathcal{X} \times \mathit{Seed} \to \mathcal{Y}$, $(x, s) \mapsto y$, is again a *Function*.

Definition 3.1.2 (Function Graph)**.** A *Function Graph* (computation graph) is a directed acyclic graph $G = (V, E)$ whose nodes are *Function*s (a node with learnable parameters θ_v takes them as an additional input, as a seed is taken in Definition 3.1.1) and whose edges encode dataflow/composition. Each in-edge of a node feeds one argument of that node's *Function*, with the source's codomain contained in that argument's domain; designated input nodes V_{in} and output nodes V_{out} fix the interface, and composition in topological order then defines an overall *Function* f_G from the inputs at V_{in} (together with any node parameters) to the outputs at V_{out}.

Definition 3.1.3 (Mathematical Model)**.** A *Mathematical Model* is an abstract description $\mathcal{M} = (G, \lambda)$ consisting of a *Function Graph* G and fixed hyperparameters λ (wiring, layer sizes, activations, tokenization, etc.). It induces a family $\{f_\theta\}$ of *Function*s, $f_\theta := f_G(\cdot\,; \theta)$, with learnable parameters $\theta = (\theta_v)_{v \in V}$ (not fixed by $\mathcal{M}$).

Definition 3.1.4 (Algorithm)**.** An *Algorithm* is a finite, well-specified procedure that deterministically computes outputs from inputs. When stochastic behavior is desired (e.g., sampling), the randomness source and any decoding policy are considered fixed parts of the *Algorithm*.

Definition 3.1.5 (Neural Network)**.** A *Neural Network* is a parameterized family of *Function*s $\{f_\theta\}$ obtained by composing affine maps with *Activation Function*s [96]. For a feed-forward network with L layers, a typical member $f_\theta : \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{d_{\text{out}}}$ has the form

$$f_\theta(x) = W_L\, h_{L-1} + b_L, \quad h_\ell = \sigma_\ell\big(W_\ell h_{\ell-1} + b_\ell\big), \;\; \ell = 1, \ldots, L-1,$$

with $h_0 = x$, layer widths $d_0 = d_{\text{in}}, d_1, \ldots, d_L = d_{\text{out}}$, weight matrices $W_\ell \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$, biases $b_\ell \in \mathbb{R}^{d_\ell}$, and nonlinear activations σ_ℓ. The learnable parameters are $\theta = \{W_\ell, b_\ell\}_{\ell=1}^{L}$. Read as a *Function Graph*, the layered structure is a graph G whose nodes are the affine maps and activation functions, and the widths and activations are its hyperparameters λ, so that the network is specified by (G, λ).

Definition 3.1.6 (Activation Function)**.** An *Activation Function* is a fixed nonlinear map $\sigma : \mathbb{R} \to \mathbb{R}$ (or $\sigma : \mathbb{R}^m \to \mathbb{R}^m$ applied elementwise) used inside a *Function Graph*. In the models considered here it is continuous and differentiable except possibly at finitely many points (e.g., ReLU at 0, where a fixed subgradient is used by convention). Common choices include ReLU, GELU, sigmoid, and tanh.

Definition 3.1.7 (Machine Learning Architecture)**.** A *Machine Learning Architecture* is a general framework in which the parameters of a *Mathematical Model* are estimated from data; it is more general than a *Deep Learning Architecture* in that it does not mandate the use of *Neural Network*s (e.g., Support Vector Machines, Random Forests).

Definition 3.1.8 (Deep Learning Architecture)**.** A *Deep Learning Architecture* is a specific type of *Machine Learning Architecture* in which the *Mathematical Model* incorporates *Neural Network*s as part of its *Function Graph* ("deep" is used conventionally for any such model, which typically has many layers). When such a model is trained on data using one or more *Training Method*s, the resulting trained object is called a *Deep Learned Algorithm* (Definition 3.1.16).

Definition 3.1.9 (Optimization Problem)**.** An *Optimization Problem* is a tuple $P = (\Theta, \mathcal{J})$ consisting of a feasible set Θ and an objective function $\mathcal{J} : \Theta \to \mathbb{R}$. Solving the problem entails finding a parameter $\theta^\star \in \operatorname{argmin}_{\theta \in \Theta} \mathcal{J}(\theta)$, i.e., a minimizer of the objective, when the minimum is attained. In deep learning, where Θ is the model's parameter space, a minimizer need not exist or be unique, and a *Training Method* returns an approximate, often local, minimizer, also denoted $\theta^\star$.

Definition 3.1.10 (Training Method)**.** A *Training Method* is the procedure used to fit the parameters of a *Mathematical Model* by approximately solving an *Optimization Problem*. Common learning settings, each of which induces an *Optimization Problem*, are *Supervised Learning*, *Unsupervised Learning*, and *Reinforcement Learning*. Figure 34 (Appendix A) provides an intuitive visualization of these paradigms.

Formally, a training method is specified by a triple $T = (\mathcal{L}, \Omega, \Psi)$ and defines a map $(D, C_T) \mapsto \theta^\star$: applied to *Dataset*(s) D under *Calculation Infrastructure* C_T, it returns parameters $\theta^\star$ by approximately minimizing the objective $\mathcal{J}$ of the induced *Optimization Problem*, which is built from the loss $\mathcal{L}$ (for a finite dataset, typically its average over D). Here $\mathcal{L}$ specifies the loss (or utility) function, Ω the optimizer, typically a variant of (stochastic) gradient descent whose gradients are computed by backpropagation (the chain rule

applied through the *Function Graph*), together with its batching and schedule, and Ψ ancillary procedures (e.g., masking, augmentation, regularization, early stopping, and checkpointing); a regularization term added to the loss (e.g., a weight penalty) becomes part of $\mathcal{J}$.

Definition 3.1.11 (Supervised Learning)**.** *Supervised Learning* is a learning setting that induces an *Optimization Problem* over the family $\{f_\theta\}$ of a *Mathematical Model*. The *Dataset* consists of input–output pairs $D = \{(x_i, y_i)\}_{i=1}^N$ drawn from a distribution $p(x, y)$. The objective is to find parameters θ that minimize a loss function $\mathcal{L}(f_\theta(x), y)$, averaged over D, which quantifies the error between model predictions and actual targets. This average is the *empirical risk*, a sample-mean surrogate for the *expected risk* $\mathbb{E}_p\big[\mathcal{L}(f_\theta(x), y)\big]$.

Definition 3.1.12 (Unsupervised Learning)**.** *Unsupervised Learning* is a learning setting, inducing an *Optimization Problem*, in which the *Dataset* consists only of inputs $D = \{x_i\}_{i=1}^N$ drawn from a distribution $p(x)$, without external labels. The parameters θ are determined by minimizing an objective of the form $\frac{1}{N}\sum_{i=1}^N \mathcal{L}(x_i; \theta)$, where $\mathcal{L}$ is a task-specific loss (e.g., negative log-likelihood, reconstruction error, or contrastive loss) used to discover underlying structures or representations.

Definition 3.1.13 (Reinforcement Learning)**.** *Reinforcement Learning* is a learning setting, inducing an *Optimization Problem*, in which an agent (in the reinforcement-learning sense of a policy, not the software agent of Section 2.4 that Definition 3.2.1 connects to this chapter) learns by interacting with an environment modeled as a Markov decision process $(\mathcal{S}, \mathcal{A}, P_{\mathrm{MDP}}, \mu_0, r, \gamma)$ with transition kernel P_{MDP}, initial-state distribution μ_0, bounded reward $|r| \le r_{\max}$, and discount factor $\gamma \in [0, 1)$ (symbols local to this definition) [189]. The objective is to find a policy $\pi(a \mid s)$ that maximizes the expected discounted return $J(\pi) := \mathbb{E}_{s_0 \sim \mu_0,\, \pi,\, P_{\mathrm{MDP}}}\big[\sum_{t=0}^{\infty} \gamma^t\, r(s_t, a_t)\big]$; over a parameterized policy class $\{\pi_\theta\}_{\theta\in\Theta}$, the induced *Optimization Problem* has objective $\mathcal{J}(\theta) = -J(\pi_\theta)$. Unlike static datasets, the *Dataset* here consists of trajectories $(s_0, a_0, r_0, \dots)$ generated dynamically by the agent's interactions.

Definition 3.1.14 (Dataset)**.** A *Dataset* D is a finite sequence of examples, repetitions allowed (or, when generated online, a stream of them): input–target pairs (x_i, y_i) for *Supervised Learning*, inputs x_i for *Unsupervised Learning*, or trajectories for *Reinforcement Learning*. It provides the empirical evidence used by a *Training Method*: it may be labeled or unlabeled, collected offline, generated online (e.g., RL, self-play), or produced by an augmentation pipeline.

Definition 3.1.15 (Calculation Infrastructure)**.** *Calculation Infrastructure* for training is $C_T = (H, S, R)$, the available hardware H, software stack S, and resource constraints R used during model training.

Definition 3.1.16 (Deep Learned Algorithm)**.** A *Deep Learned Algorithm* is the trained, fully specified *Algorithm* $F = (G, \theta^\star, \pi_{\mathrm{dec}})$ obtained by applying one or more *Training Method*s $T_1, \dots, T_m$ to *Dataset*s $D_1, \dots, D_m$ under *Calculation Infrastructure* C_T. Here G is the *Function Graph* (evaluated in topological order) of the *Mathematical Model* $\mathcal{M} = (G, \lambda)$, whose fixed hyperparameters λ (e.g., the tokenization) F inherits; $\theta^\star$ are the learned parameters (including any non-gradient state, e.g., BatchNorm statistics), and π_{dec} is a fixed decoding/evaluation policy. For a single forward pass, F acts as $\pi_{\mathrm{dec}} \circ f_{\theta^\star}$ (greedy decoding takes $\pi_{\mathrm{dec}} = \arg\max$); we write simply $F = f_{\theta^\star}$ only when π_{dec} is the identity, i.e., when the model output is itself the final output (e.g., in regression).

Definition 3.1.17 (Decision-Making Model)**.** A *Decision-Making Model* is a parameterized *Function* $\hat{f}_{\boldsymbol{\theta}} : \mathcal{X} \to \mathcal{Y}$, or $\hat{f}_{\boldsymbol{\theta}} : \mathcal{X} \to \Delta(\mathcal{Y})$ when its outputs are probabilistic, whose outputs are consequential decisions or recommendations: $\mathcal{X}$ is the space of inputs, $\mathcal{Y}$ the space of recommended actions or decisions, and $\Delta(\mathcal{Y})$ the set of distributions over $\mathcal{Y}$. The parameters $\boldsymbol{\theta} \in \Theta$ (e.g., predicates and leaf labels of a rule base or decision tree, weights and biases of a network, thresholds, code lists, and encoders) are the *model of record*: the versioned instance that fixes which function is in force. A decision-making model is an *Algorithm* whose parameters are expert-maintained when $\boldsymbol{\theta}$ is hand-curated (e.g., the NordDRG specification) or learned when $\boldsymbol{\theta}$ is trained from data (e.g., a decision-tree credit-risk classifier; a *Deep Learned Algorithm* when $\boldsymbol{\theta}$ are network weights), or a hybrid of the two. Two operations on it are governed: its *use*, the evaluation $\hat{f}_{\boldsymbol{\theta}}(x)$ for an input $x \in \mathcal{X}$ in operation, and its *change* $\boldsymbol{\theta}' = \delta(\boldsymbol{\theta})$ to a new model of record. A decision-making model is *large* when it comprises many decision points; the NordDRG specification unrolls path-wise to $\approx 10^7$ of them (Table 19).

Definition 3.1.17 is a reading of the Phase III artefacts rather than a new artefact; it names the object that the three share (Figure 2): the NordDRG AI Benchmark (Section 6.2) asks whether a reasoning core M can read and faithfully execute a large decision-making model; HADA (Section 6.1) governs the *use* of many such models across an organization; and CCL (Section 6.3) governs the *change* of one large model and adds the representation and versioning requirements of a governed model of record (Section 6.3.1.1). The hat and the bold face mark the governed object: f_θ of the Notation paragraph is any parameterized function with learnable parameters, whereas $\hat{f}_{\boldsymbol{\theta}}$ is one whose parameters are a model of record (Remark A.2).

3.1.2 Transformer Definitions

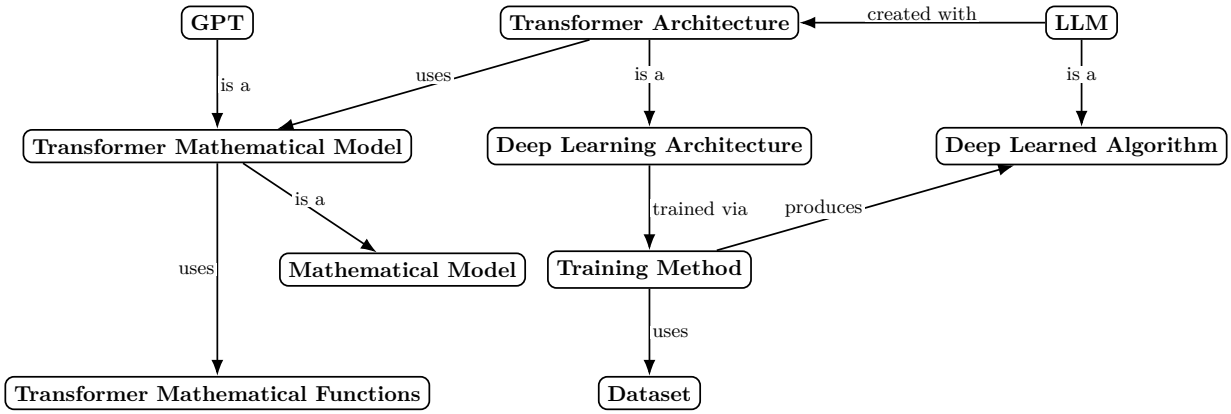


Figure 9: Transformer ecosystem concepts and their relationships (an LLM is created from the Transformer Mathematical Model that its Transformer Architecture uses; Definition 3.1.21). In the electronic version, each box links to its definition.

Definition 3.1.18 (Transformer Architecture)**.** A *Transformer Architecture* is a *Deep Learning Architecture* that uses the *Transformer Mathematical Model*.

Definition 3.1.19 (Transformer Mathematical Model)**.** A *Transformer Mathematical Model* is a *Mathematical Model* $\mathcal{M} = (G, \lambda)$ where G is a *Function Graph* whose nodes are the *Transformer Mathematical Functions*, i.e., *Function*s implementing: (i) token embedding and output projection, (ii) positional encoding, (iii) masked multi-head (self-/cross-) attention, (iv) position-wise feed-forward mappings, (v) residual connections and layer normalizations. The edges of G encode either an encoder–decoder or decoder-only dataflow (encoder-only Transformers are outside the scope of this thesis); the wiring and hyperparameters (e.g., L, d_{model}, h, d_{ff}, masking policy, tokenization/positioning scheme) are fixed by λ. By *Mathematical Model*, $\mathcal{M}$ induces a family $\{f_\theta\}$ of *Function*s with learnable parameters θ.

The constituent functions (embedding, multi-head attention, feed-forward, layer normalization, block composition, and output projection) are defined in Appendix A (Definition A.1 and Eqs. 5–16).

3.1.2.1 Generative Pre-trained Transformer (GPT)

Definition 3.1.20 (GPT)**.** A Generative Pre-trained Transformer (*GPT*) is a *Transformer Mathematical Model* that has a decoder-only *Function Graph* and is typically trained via a next-token prediction *Training Method* on large-scale text *Dataset*s, optionally followed by further stages such as instruction tuning or reinforcement learning from human feedback (RLHF) [212]. As a *Mathematical Model*, a GPT fixes the function graph and hyperparameters but neither the trained parameters nor a decoding policy; a trained GPT is an *LLM* (Remark 3.1.22).

Definition 3.1.21 (LLM)**.** A Large Language Model (*LLM*) is a *Deep Learned Algorithm* created from a *Transformer Mathematical Model*, such as a *GPT*, typically by applying multiple *Training Method*s (e.g., pretraining, fine-tuning, RLHF) to large-scale text *Dataset*s [215]. Its decoding policy π_{dec} turns the output token distributions of Eq. 16 (Appendix A) into generated text.

Remark 3.1.22 (GPT as an LLM family). Under Definition 3.1.21, *LLM*s built on a GPT model form a specific family whose *Mathematical Model* is a decoder-only *Transformer Mathematical Model*. At inference time, a trained GPT, i.e., an *LLM* in this sense, maps input token sequences to next-token distributions (Eq. 16 in Appendix A), which its decoding policy π_{dec} (e.g., greedy or sampling) turns into tokens for text generation and related tasks. In practice, most *LLM*s deployed at the time of the Phase III studies (2023–2025) followed this GPT-style pattern (decoder-only Transformer with next-token pretraining), which is why "LLM" and "GPT-like model" are often used interchangeably in informal discussion. Definition 3.1.21 deliberately restricts the term to Transformer-based models: this is understood to cover every model evaluated in Phase III, and non-Transformer sequence models are outside the scope of this thesis. The qualifiers "large" and "language" are likewise conventional rather than formal conditions: the definition fixes the kind of object, not a size or modality threshold.

3.2 LLM Agents

This section formalizes the concept of an LLM agent, distinguishing it from a standalone *LLM* by the addition of state (memory), executable capabilities (tools), and a control loop, and then extends it with the governance layer that the Phase III artefacts add on top.

Definition 3.2.1 (LLM Agent)**.** An *LLM Agent* is a computational system $A = (M, \mathcal{T}, \mathcal{S}, \Pi)$ where:

1. M is an *LLM* acting as the reasoning core; a parse map $\text{parse} : \mathcal{V}^* \to (\mathcal{T} \times X) \cup \{\bot\}$ reads its generated text over the token vocabulary $\mathcal{V}$ as a tool invocation (f, x), where X is the union of the input sets of the tools in $\mathcal{T}$, or as no invocation ($\bot$).
2. $\mathcal{T} = \{f_1, \ldots, f_k\}$ with $k \geq 1$ is a set of *Tool*s available for execution; without tools ($k = 0$) the system reduces to the reasoning core M conditioned on its context state, which is the configuration the NordDRG AI Benchmark measures (Section 6.2).
3. $\mathcal{S}$ is the state space of the agent's memory (Definition 3.2.3), whose current element $s \in \mathcal{S}$ is the dynamic memory or context state.
4. Π is a control policy (or planning *Algorithm*) that iteratively prompts M to select functions from $\mathcal{T}$ or update the state s to satisfy a given request or objective; each step is a rule $\Pi : \mathcal{S} \times \mathcal{V}^* \to \mathcal{S} \times \big((\mathcal{T} \times X) \cup \{\mathsf{halt}\}\big)$ that maps the current state and the output of M (read through the parse map) to the next state together with either a tool invocation (f, x), whose output y, where $(y, e') = f(x, e)$ for the current environment state e, is recorded in the state before the next step, or halt, on which the agent returns the output of M as its response to the request.

The components $\mathcal{T}$ and $\mathcal{S}$ are specified in Definitions 3.2.2 and 3.2.3 below. This tuple synthesizes the framings of [216, 43] and the reason–act loop of [44]; like the planning module of [43], it keeps the control policy Π explicit, and it differs from these framings chiefly in making Π the attachment point for the governance layer of Definition 3.2.5. It also refines the informal definition in Section 2.5: prompt specification and conversation trace correspond to the short-term part of $\mathcal{S}$, and tool usage to $\mathcal{T}$; the control policy Π and long-term memory are made explicit here. Read against the classical agent properties of Section 2.4, Π carries autonomy and proactiveness and, because it conditions each step on the observations held in $\mathcal{S}$, reactivity; $\mathcal{T}$ carries the agent's actions on its environment; and $\mathcal{S}$ carries its state. In this sense the software-agent abstraction stays constant across the three phases while its reasoning substrate changes; social ability is supplied by the surrounding harness rather than by the tuple.

This definition is deliberately *capability-level*: the tuple $(M, \mathcal{T}, \mathcal{S}, \Pi)$ fixes what an agent can do—reason, invoke tools, and maintain state—but not whether those actions are governed, logged, or aligned with human-set objectives. The complementary *governance and alignment* layer that this thesis adds—policy-controlled tool use, binding to organizational objectives, an audit trail, and human-in-command sign-off—is formalized in Definition 3.2.5, realized by HADA (Section 6.1) and CCL (Section 6.3), and positioned against the agent literature in Section 2.5.6.

Definition 3.2.2 (Tool)**.** A *Tool* is a specific *Function* $f : \mathcal{X} \times \mathit{Env} \to \mathcal{Y} \times \mathit{Env}$ exposed to the *LLM Agent*, where *Env* is the set of states of the external environment that the tool may observe or manipulate, so that f is deterministic given that state (randomness, if any, enters through a seed as in Definition 3.1.1). Examples are a calculator, a database query, or an API call; what makes a component a tool is its role, not its internals: it is invoked by Π rather than driving it, and it may itself be an *LLM* (Remark 3.2.4).

Definition 3.2.3 (Agent Memory)**.** Agent Memory $\mathcal{S}$ is the state space of the information (not a training *Dataset*) that Π maintains during the lifecycle of the *LLM Agent* to condition M, wherever it is stored (Remark 3.2.4); its current state $s \in \mathcal{S}$ comprises:

- *Short-term memory*: The contents of the immediate context window of the *LLM*, namely the current prompt, conversation history, and recent tool outputs.
- *Long-term memory* (optional): The contents of an external retrieval system [220, 227] (e.g., a vector database) from which the agent fetches relevant stored information to augment the input context of M.

Remark 3.2.4 (Role, not transport). Classification is by role, not by transport. A component that Π invokes through a tool protocol such as MCP (Section 2.5) is a tool in $\mathcal{T}$ even when it is itself an LLM; the LLM whose outputs drive Π is M; and state that Π maintains to condition M is $\mathcal{S}$ wherever it is stored. M is

fixed for an agent instance (model and version); changing it is a configuration change recorded in the ledger $\mathcal{L}$ (Definition 3.2.5), not a tool call, and an agent reached through an agent protocol such as A2A is another agent, not a tool. Likewise, a packaged procedure such as an Agent Skill [241] adds no component to the tuple: once Π loads it, its instructions are prompt specification and thus part of $\mathcal{S}$, and its bundled scripts, when Π invokes them, are tools in $\mathcal{T}$.

Definition 3.2.5 (Governed LLM Agent)**.** A *Governed LLM Agent* is a system whose capability core $(M, \mathcal{T}, \mathcal{S}, \Pi)$ is an *LLM Agent* and whose control policy Π operates under a governance layer,

$$A_g = (\, M,\ \mathcal{T},\ \mathcal{S},\ \Pi \,\big|\, \mathcal{R},\ \mathcal{N},\ \mathcal{O},\ \mathcal{L}\,),$$

where the bar separates the capability core from the governance layer. Let Act be the given set of governed activities, comprising the tool invocations $\{(f, x) : f \in \mathcal{T},\ x \text{ an input of } f\}$, decisions, and changes to decision logic, including $\boldsymbol{\theta}' = \delta(\boldsymbol{\theta})$ of a *Decision-Making Model* ($\mathcal{R}$ assigns roles to activity types; $\mathcal{N}$ and $\mathcal{L}$ apply to each executed instance), and let $Roles = Hum \sqcup Ag$ be the given set of stakeholder roles, partitioned into human roles Hum and agent roles Ag. Then:

1. $\mathcal{R}$ is a *responsibility assignment*, a relation $\mathcal{R} \subseteq Act \times Roles \times \{\mathsf{R}, \mathsf{A}, \mathsf{C}, \mathsf{I}\}$ that names, for every governed activity, its responsible, accountable, consulted, and informed stakeholder roles (a role may hold several letters for one activity, e.g., A and R), with exactly one accountable role per activity, which is always human: for every $a \in Act$ there is exactly one $\rho \in Roles$ with $(a, \rho, \mathsf{A}) \in \mathcal{R}$, and this ρ is a human role.

2. $\mathcal{N}$ is a set of *value constraints*: machine-readable organizational values and norms $\nu : Act \to \{0, 1\}$ that an activity must satisfy to proceed; a violating activity is blocked or escalated to its accountable human under $\mathcal{R}$. An activity proceeds only if every $\nu \in \mathcal{N}$ holds, so escalation does not override a constraint, and any resolution, such as a change to the decision logic, is itself a governed activity; priorities and conflicts among constraints are not modeled.

3. $\mathcal{O}$ is an *objective binding* $\mathcal{O} : Obj \to Par_\Pi$ that maps each organizational objective with its planning cadence (e.g., an OKR), drawn from a given set Obj, to the targets, thresholds, and guardrails in the parameter space Par_Π of Π.

4. $\mathcal{L}$ is an *append-only ledger* that records every executed governed activity together with the acting role, the accountable human under $\mathcal{R}$, and the evidence on which the activity rested, i.e., a finite sequence $\mathcal{L} \in (Act \times Roles \times Hum \times Evid)^*$ (the acting role is an agent role, or a human role where a human acts), over a given set $Evid$ of evidence items, that is only ever extended at its end.

The capability core $(M, \mathcal{T}, \mathcal{S}, \Pi)$ fixes what the agent can do; the governance layer $(\mathcal{R}, \mathcal{N}, \mathcal{O}, \mathcal{L})$ fixes under whose accountability, within which constraints, toward which objectives, and with what audit trail it does so.

Responsibilities, goals, norms, and delegation have design-time counterparts in goal-, actor-, and trust-oriented modeling (Tropos and Secure Tropos [78, 79, 256]; trust and delegation [257]); the governance layer binds their operational counterparts to each run-time activity (Section 2.5.6). Definition 3.2.5 is a *reading* of the Phase III artefacts rather than a new artefact: it names the components that Figure 2 calls the shared governance layer, and each of them is instantiated in the chapters that follow. HADA (Section 6.1.1) realizes the layer at run time over a cohort of agents, applying the definition agent by agent with one shared $\mathcal{R}$, $\mathcal{N}$, $\mathcal{O}$, and $\mathcal{L}$ and with inter-agent messages routed by the HADA Controller (Section 6.1.1.6). The definition itself does not formalize inter-agent coordination. In HADA, the RACI matrix (Table 10) is $\mathcal{R}$, the Values Catalogue is $\mathcal{N}$, the Strategy-Process Blueprint is $\mathcal{O}$, and the append-only decision ledger, which logs each loan decision with its provenance, realizes $\mathcal{L}$ in part. CCL applies the same layer to the change $\boldsymbol{\theta}' = \delta(\boldsymbol{\theta})$ of the parameters $\boldsymbol{\theta}$ of a single decision-making model $\hat{f}_{\boldsymbol{\theta}}$ (Definition 3.1.17); Section 6.3.1.3 gives the gate-by-gate mapping, with the qualifiers of the reported demonstration. The NordDRG AI Benchmark (Section 6.2) deliberately strips the governance layer and the external tools away and measures the reasoning core M, given only the released artefacts as context, over such a decision model—the capability premise on which both harnesses rest. The definition claims no more than those chapters demonstrate: the governance components are stated as interfaces, and their evaluation is reported, with its limitations, in Sections 7.5 and 7.7, while Section 7.6 evaluates the capability premise.

Transition. Chapters 4 and 5 report Phases I and II without relying on these definitions, whereas Chapter 6 instantiates the LLM agent and the governed LLM agent (Definitions 3.2.1 and 3.2.5) in HADA and CCL and measures their reasoning core M in the NordDRG AI Benchmark over a large decision-making model

(Definition 3.1.17). Chapters 7 and 8 then evaluate these instantiations against Definition 3.2.5 and draw conclusions from them.

4 PHASE I: DESIGN & DEVELOPMENT AND DEMONSTRATIONS

Phase I (2001–2006) investigates how early Web services can be made *intelligent* through agent-mediated interaction and shared ontologies. The central design choice is to separate the *intentional force* of messages (interaction protocol, e.g., FIPA performatives) from *domain payload* (application-specific content). This separation enables run-time adaptation at the conversation level and supports domain-specific negotiation without hard-coding process logic. The idea is demonstrated in three settings: (i) conversation-level adaptation to service descriptions (Section 4.1); (ii) ontology-grounded negotiation in a B2B purchasing scenario (Section 4.2); and (iii) context-aware matching that connects otherwise separate shared ontologies (Section 4.3). Throughout, shared vocabularies and protocol conformance are the key enablers of interoperability and reuse. This phase includes three demonstrations from two projects (PAX and DYNAMOS)[17] reported in part in [1, 2, 3, 4, 5, 6, 7, 8]. Section 4.1 details interaction-protocol-based service adaptation (*ConStrip*); Section 4.2 develops a registry-centered B2B purchasing and bidding scenario that operationalizes shared ontologies and negotiation; and Section 4.3 (*DYNAMOS*) shows how context information links disparate descriptions to enable service discovery and matching. In the dissertation's *governable-agency through-line* (Section 1.1.2), Phase I establishes the constant software-agent abstraction, here in a protocol-driven form without an explicit goal model (Section 7.1), and exposes, in retrospect, its first substrate limitation—hand-built ontologies and protocols that do not learn from data and were not shown to scale beyond closed scenarios—which Phase II inherits. The candidate's role in each demonstration is stated in Table 23.

To orient the reader in the overall thesis structure and DSRP flow, see Table 1 (DSRP process sequence linked to Phases and Structure of this Thesis) and Section 1.2 (Phase I Problem and Motivation: Early Web Evolution—From Information Space to Service Platform).

4.1 Interaction-Protocol-Based Service Adaptation

This artefact (A1, the run-time interaction-protocol adaptation engine of Table 5) addresses **RQ I.2** (Table 4).

Problem Identification, Research Questions, and Objectives of the Solution

The problem identification, research questions, and objectives for this work are presented in Section 1.2.1. Portions of the design description below (the process flow, the interaction protocol ontology, the protocol descriptions, and the adaptation logic) follow, in revised or lightly edited form, the text of the co-authored ICEIS 2004 paper [7]. The research methodology is presented in Section 1.1.

Context

This work examines how Semantic Web formalisms can be combined with software-agent protocols to enable *run-time* interoperability in a B2B setting. The motivating scenario is construction "expediting", where a general contractor must verify schedule and delivery information across a multi-tier network of subcontractors, suppliers, and carriers. Instead of brittle, hard-coded integrations or ad-hoc phone/e-mail coordination, the approach describes interaction protocols in RDF/OWL against a shared interaction protocol ontology. An expeditor agent for the contractor ingests these RDF-serialized (FIPA-based) protocol descriptions at run time and executes them via a protocol state machine, allowing it to converse with previously unknown partners while maintaining a machine-traceable audit trail.

Beyond the immediate construction use case, the motivating hypothesis is that such run-time adaptation could reduce coordination delay, curb disputes, and improve accountability when many independent organizations must work together; the demonstration below establishes feasibility only (see Section 7.1). By letting partners publish "how to talk to me" in a standard, machine-readable form, the contractor can verify claims directly at the source and escalate inconsistencies early, without bespoke integrations each time. The evaluation (Section 7.1) also reflects on the practical boundaries of this idea—where human judgment is still needed, what must be standardized across participants, and how far such lightweight agreements might carry beyond a controlled scenario, which the demonstration itself did not test.

[17] The demonstrations were implemented in projects co-funded by Tekes, VTT, and industrial parties; see `http://www.vtt.fi/`.

4.1.1 Design and Development

The main Interaction-Protocol-Based Service Adaptation artefact, demonstrated in the *ConStrip* construction-expediting scenario below, comprises the following artefacts:

A.I.1.1 Process Flow. A formalization of the expediting process states and the contractor's interaction strategy (see Section 4.1.1.1).

A.I.1.2 Interaction Protocol Ontology (adopted). The core ontology of [34] (Initiator, Participant, State, Option, Message), applied here to give meaning to protocol descriptions (see Section 4.1.1.2).

A.I.1.3 Interaction Protocol Serializations. RDF examples and FIPA-based serializations that show how protocols and message contents are expressed for machine ingestion (see Section 4.1.1.3).

A.I.1.4 Adapting to Interaction Protocol Descriptions. The protocol state machine and adaptation logic used by the expeditor agent to interpret and execute external protocol descriptions (see Section 4.1.1.4).

4.1.1.1 Artefact: Process Flow

The states of the expediting process from the expeditor's point of view are depicted in Figure 10. Initially the expeditor knows only how to interact with the subcontractor, so it must learn how to interact with the other participants. This poses no problem for human actors, but it does for software agents. The approach presented in this demonstration is to define interaction protocol descriptions and to design the expeditor agent so that it can process those descriptions and adapt its behavior accordingly.

Collaboration is based on interaction protocol descriptions (Section 2.4.6) provided by the participants. In order to enable the adaptation, the descriptions conform to a mutually agreed interaction protocol ontology that is introduced in [34]. The expeditor agent, if unaware beforehand how to communicate with a participant, can download interaction protocol descriptions provided by the participant and adapt its behavior accordingly.

In this demonstration, the interaction between the contractor and the subcontractor is modeled using the FIPA Request [325] interaction protocol. The contractor requests the subcontractor to send the status report. The subcontractor then tries to perform the requested action. In the successful case, the subcontractor informs the contractor that the action is performed. If something goes wrong, the subcontractor sends a failure message.

Assume the subcontractor has ordered a steel delivery from the supplier with the help of an external carrier, but the delivery has not yet arrived. The subcontractor nevertheless includes the order in the status report before sending it to the contractor. Because the contractor wants to keep track of the whole process, it contacts the supplier and the carrier to check the status of the order and the delivery. The contractor first requests the subcontractor to send the status report. By examining it, as depicted in Figure 10, it finds out the addresses (URIs) of the agents representing the supplier and the carrier, as well as the identifier of the steel order (how this information is extracted from the status report, by the software agent itself or by a human being, is not in the focus of this demonstration). After retrieving this information, the contractor queries the supplier for the identifier of the steel delivery with the order identifier as the parameter. The supplier provides the contractor with it, and the contractor uses it next as a parameter for requesting a delivery report from the carrier.

Since the contractor has not collaborated with the supplier and the carrier before, doing so now presupposes adaptability. The following subsections present interaction protocol descriptions in more detail.

4.1.1.2 Artefact: Interaction Protocol Ontology (adopted)

Interaction protocols specify ordered sets of messages to be exchanged between conversing agents. In this demonstration, the interaction protocols are described using RDF [326], a language designed for describing any resource in the Semantic Web. As mentioned, interaction protocol descriptions conform to the interaction protocol ontology [34]. Core concepts of that ontology are depicted in Figure 11. Basically, all interaction protocols have one initiator and one or more participants. The progress of the protocol is defined by states that follow each other. More specifically, states have options to choose from, and the chosen option determines which state to shift to. Options can have messages associated with them that are sent between the participating agents.

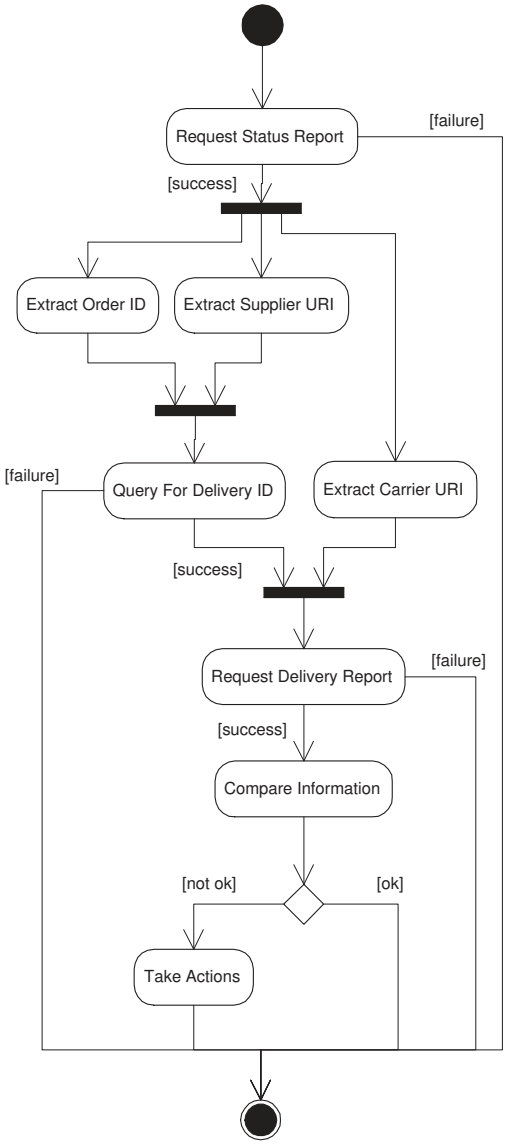


Figure 10: Progression of the expediting process from the contractor's point of view.

4.1.1.3 Artefact: Interaction Protocol Serializations in RDF

Each protocol the expeditor must adapt to is serialized in RDF against the shared interaction protocol ontology and supplied by the partner—in the running example, an RDF serialization of FIPA Query [327] provided by the supplier. The serialization instantiates the ontology's classes directly: an *IP* individual binds one *Initiator* and one or more *Participant*s to a *Progress*; the progress lists *State*s (with *start* and *end* presupposed by every agent); each state offers *Option*s (`ip:choices`); and each option carries an optional *Message* (with `ip:sendBy`/`ip:recvBy`) and a target state (`ip:follows`). Message content is expressed through a FIPA-ACL-based extension to the ontology [34, 7, 2, 148]: a *Content* individual embeds an RDQL (RDF Data Query Language) [328] query that retrieves the delivery identifier (`?x`) using the order identifier (`?y`) parsed from the status report (Figure 10), assuming a shared supply-chain reports ontology. Representative serialization excerpts are collected in Appendix A; further excerpts appear in the Licentiate thesis [2].

With the delivery identifier received as the reply, the contractor requests the delivery report from the carrier (described in the same manner) and then holds the reports needed to check the subcontractor's original status report; the comparison itself, and escalation of any conflicting information for appropriate action, can be automated or left to the contractor's personnel (Figure 10).

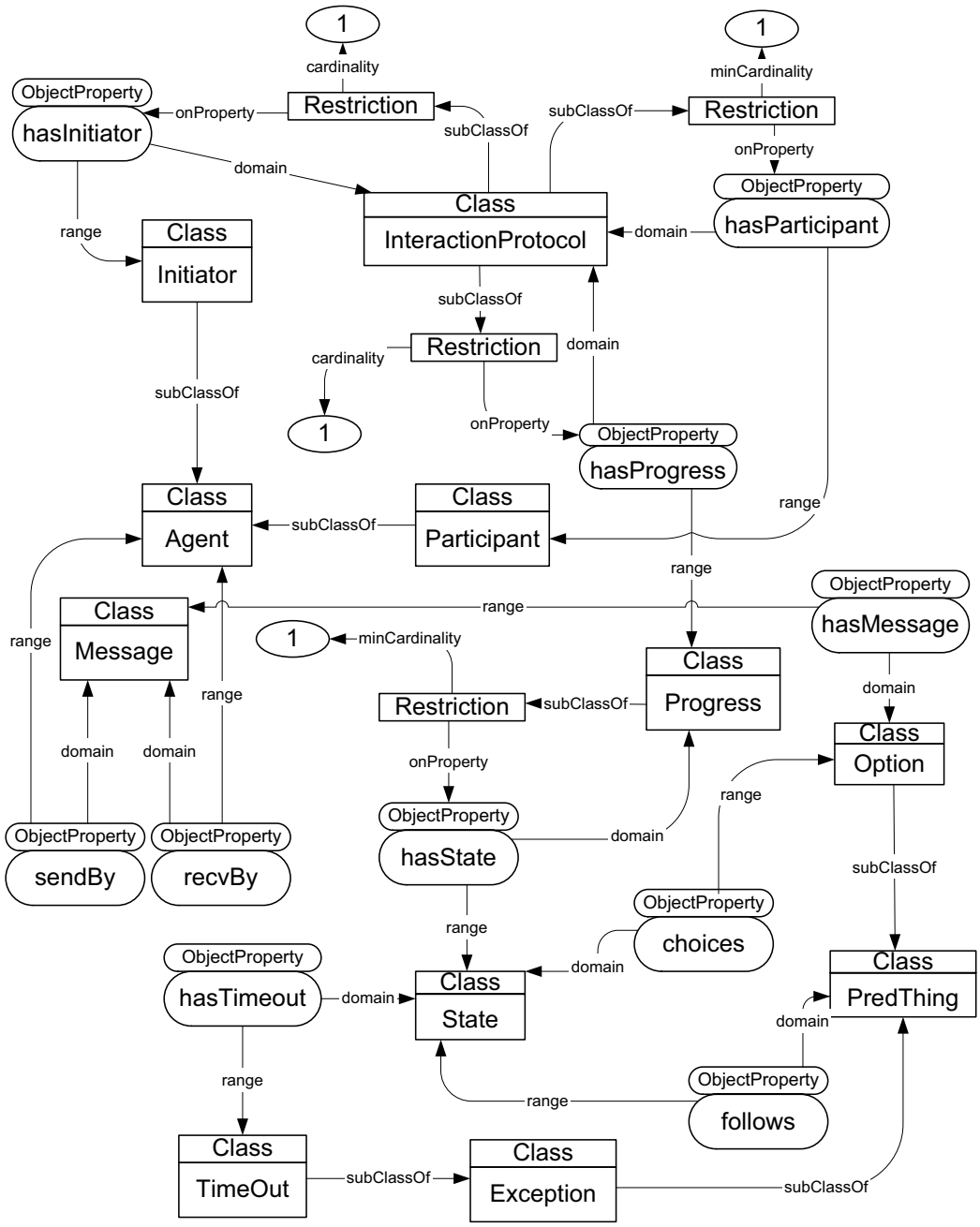


Figure 11: Core concepts of the interaction protocol ontology [34].

4.1.1.4 Artefact: Adapting to Interaction Protocol Descriptions

The only agent adapting to the interaction protocol descriptions in the above scenario is the contractor's expeditor. Other agents of the scenario have knowledge of the interaction protocols beforehand. Figure 12 depicts the state machine that the adapting agent uses as the protocol progresses.

As mentioned, the states where the protocol starts and ends are assumed to be known by every agent. Being the initiator of the protocol, the expeditor first searches for the start state and extracts its options. It then selects one option according to choice logic preprogrammed for the scenario (no explicit goal model was implemented, although the ICEIS 2004 paper [7] words this choice as goal-based; see Section 7.1) and

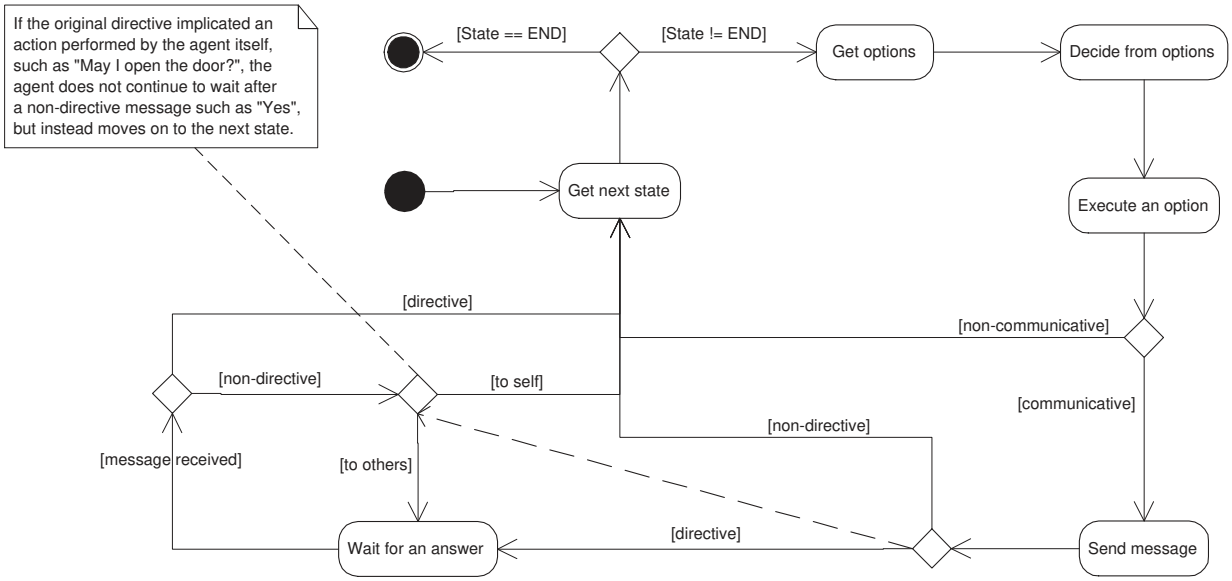


Figure 12: The protocol state machine.

executes it. Note that agents' goals that have an effect on the decision are excluded from interaction protocol descriptions.

An option that does not entail sending a message is called a non-communicative option; a communicative option, by contrast, does send a message. If that message is directive, its sender expects a reply and aims to get the receiver to do something, such as perform an action or answer a question (with the exception of directives "directed" to oneself). As an example, consider FIPA Request [325]; after sending the request message the initiator waits for an answer to it from the participant before the protocol shifts to the next state. For a non-directive message, instead, there are no answers and the protocol moves immediately to the next state. Examples are all the other messages in FIPA Request, namely refuse, agree, failure, and inform (all FIPA performatives are depicted in Appendix A, Figure 30).

Unlike agree, all other non-directive messages in the FIPA Request protocol are followed by the end state. The agree message, by contrast, is followed by the state in which the participant, after trying to perform the requested action and depending on its results, sends the appropriate message to the initiator. Note that Figure 12 omits the canceling and exception-handling mechanisms. An agent participating in a protocol has the possibility of canceling the protocol at any point. Also, exceptions can arise at any point of the protocol. The contractor's expeditor agent is able to adapt its behavior (at the protocol level, by selecting and executing a partner-supplied protocol rather than by goal-directed replanning; see Section 7.1) to conform to the interaction protocols described and provided by other process participants by combining the following information: knowledge of the interaction protocol ontology; knowledge of the flow of a protocol as depicted in Figure 12; knowledge of utilized domain ontologies, such as the abovementioned supply chain reports ontology.

4.1.2 Demonstration

Context: B2B Expediting Scenario This demonstration deals with the so-called expediting process that is part of a broader concept called supply chain management. Traditionally, performing an expediting process means paying visits to the premises of project participants, making phone calls, or exchanging e-mails to check the status.

Previous attempts have applied software agents to construction processes, such as the one described in [65]. A distinctive feature of the approach presented in this section is that it partially automates the expediting process by applying interaction protocols to the work of the contractor's software agent, which acts as an expeditor and adapts at run time to protocol descriptions supplied by previously unknown partners instead of having each partner protocol hard-coded [7].

In the scenario, the contractor has initiated a project and wants to keep track of it. The contractor therefore expects to receive status reports of the project from the subcontractor(s) regularly. That helps the contractor to identify and avoid possible risks involved in the project, such as schedule delays and cost overruns.

However, often the contractor cannot rely solely on the information the subcontractor(s) provide in the status reports. Either a subcontractor can intentionally conceal that the project is behind schedule, or some important piece of information might unintentionally be out of date in a status report. For verifying the information in a status report, the contractor's expeditor contacts the other participants directly. Thereby, the expeditor acts somewhat as a detective trying to dig up information from various sources and incorporate it into a complete picture.

With many project participants involved, direct interaction between the contractor and the others can also help avoid problem amplification. However, communication can become heavy, since large projects may include numerous subcontractors, each with multiple suppliers and carriers.

4.1.2.1 Agent Platform and Implementation

All interaction protocol descriptions and the reports exchanged between agents are grounded in common ontologies and represented in RDF, handled with the Jena Semantic Web Framework [329]; a representative delivery report sent by the carrier agent to the general contractor is reproduced in Appendix A. Every actor (Contractor, Subcontractor, Supplier, Carrier) is implemented as a software agent on a FIPA-compliant platform, JADE (Java Agent DEvelopment Framework; Section 2.4.5.3) [143], which provides the message transport system, protocol templates, containers, and the monitoring/debugging tools used during development (Appendix A, Figures 44 and 45). The implementation comprises around 9 000 lines of Java code, as reported in the Master's thesis [1], together with RDF artefacts (protocol descriptions and domain ontologies).

4.1.2.2 Screenshots from the Demonstration

The running demonstration shows the four agents—general contractor (*gc0*), subcontractor (*sc0*), steel supplier (*ss1*), and carrier (*cr_type2*)—exchanging FIPA-typed messages (*INFORM*, *AGREE*, etc.) whose contents are the demonstration-specific reports, realizing the process flow of Figure 10 (Appendix A, Figures 46 and 47). Initially the contractor agent knows only how to talk to the subcontractor, so it requests the status report directly; from it the contractor learns that one supplier and one carrier were involved. To verify the report it then contacts each previously unknown agent in turn: it requests the partner's protocol description ("how do you communicate"), receives the RDF serialization, builds a model and interprets it with the state machine of Figure 12, and uses the order identifier parsed from the status report (supplier) or the returned delivery identifier (carrier) as the query parameter. With the status and delivery reports in hand it can automatically check whether their information matches—or leave that comparison to the contractor's personnel.

4.2 Ontology-Driven Software Agents for B2B Purchasing Automation

This artefact (A2, the ontology-driven B2B negotiation prototype of Table 5) addresses **RQ I.2** (Table 4).

Problem Identification, Research Questions, and Objectives of the Solution

The problem identification, research questions, and objectives for this work are presented in Section 1.2.2. The research methodology is presented in Section 1.1.

Context

By the early 2000s, many processes in the B2B sector had already been automated using communication networks, and the massive growth of the Web had encouraged companies to use Web technologies and open architectures for process automation. At the time of the work, the next step in this "Web automation" was expected to be Web Services that were not dynamically located but rather based on traditional contracts between companies and then implemented by system integrators, a step expected to take a different amount of time for different business sectors. Many sectors had already done a lot of such integration of processes. However, for many, a scenario where agents aid in contract negotiations then seemed to be science fiction.

4.2 ONTOLOGY-DRIVEN SOFTWARE AGENTS FOR B2B PURCHASING AUTOMATION

In the purchasing process, a buyer needs some product or service (for simplicity, called a product from here on) and it negotiates with several potential companies. The product can be automatically mapped by a semi-intelligent registry to a set of companies that are able to provide the products.

At the time, such purchasing processes could be very time-consuming and burdensome for all the participating companies, and there was potential in partly automating them. Naturally, however, it is not appropriate to automate everything. In this demonstration, the decision points between agents and company personnel are divided so that the routine tasks are done automatically by agents and the crucial decisions by the company personnel.

4.2.1 Design and Development

The main artefact, Ontology-Driven Software Agents for B2B Purchasing Automation, comprises the following artefacts:

A.I.2.1 Purchase Process Flow. A concise formalization of the RFQ, negotiation, and contract-making subprocesses that structures agent and human decision points (see Section 4.2.1.1).

A.I.2.2 Decision Making. The design that partitions routine automation from human oversight and specifies, at design level, authorization/handoff rules for buyer and supplier agents (their implementation was not demonstrated; see Sections 4.2.1.2 and 7.2).

A.I.2.3 Data Models and Ontologies. The proposal and product ontologies that define the RDF/XML message schemas used by agents to express RFQs, quotes, and contracts (see Section 4.2.1.3).

A.I.2.4 Locating Services and Registry. The semi-intelligent, category-based registry that maps product ontologies to candidate providers and supports agent discovery and browsing (see Section 4.2.1.4).

4.2.1.1 Artefact: Purchase Process Flow

This subsection briefly describes the purchasing process, which consists of the three subprocesses that are presented in Figure 13: request for quote, quote negotiation, and making the contract. In the purchasing process, agents are aiding in many ways, but there are decision points where consultation of the company personnel is needed. The decision points are also presented in Figure 13 and they are further explained in the next subsection. All communication between agents is done by utilizing protocols standardized by FIPA [206].

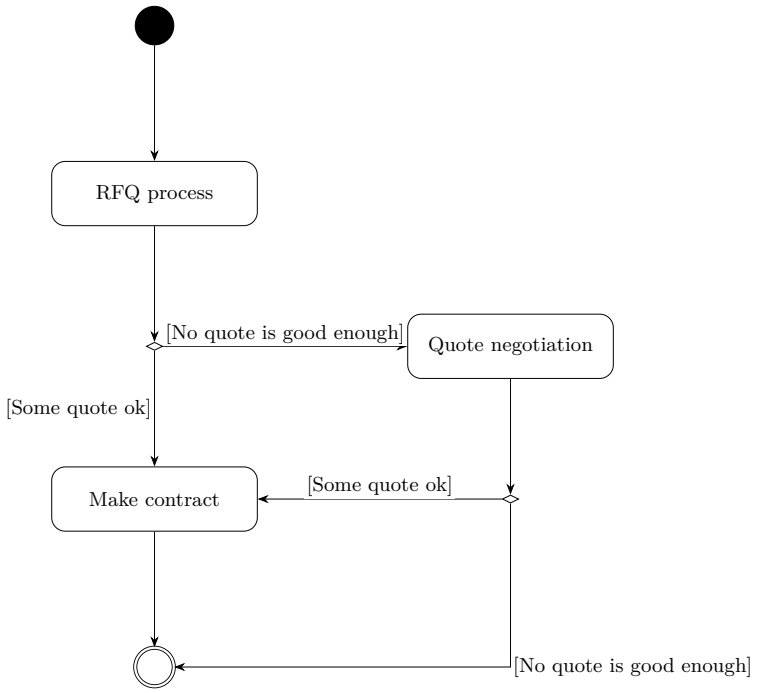


Figure 13: Overall purchasing process (RFQ, quote negotiation, and contract-making) with decision outcomes.

The buyer's personnel author an RFQ with constraints (e.g., a price limit); the buyer agent locates candidate suppliers via the registry and issues the RFQ to them using the FIPA Contract Net [41] interaction protocol. Each supplier agent checks availability, looks up its pricing, and returns a quote automatically. The buyer agent compares the quotes against the constraints: if at least one satisfies them, it proceeds directly to contract-making. If none does, it either escalates the decision to personnel or opens the (optional) *quote-negotiation* step, in which a modified quote is offered to a set of companies via FIPA Propose [330] and the first to agree is selected. Contract-making itself is carried out with the FIPA Query [327] protocol once both sides have agreed on delivery and payment terms.

4.2.1.2 Artefact: Decision Making

As mentioned before, it is not appropriate to automate everything in a purchasing process. Software programs, in this case agents, only perform routine tasks, and the major decisions are either taken by company personnel or delegated to agents within limits that personnel set beforehand.

In the scenario, a buyer agent is not authorized to act on its own initiative; it is launched to execute a particular task, and within that task it is authorized to negotiate a contract if the needed terms and limits for the particular case are met. If not, the decision is passed to its personnel.

The supplier agents are authorized to automatically create quotes based on data from the company's internal management systems, including inventory, pricing, and financial information, such as payables and receivables. If the buyer proposes a price lower than the list price, the agent suggests a course of action to human personnel and awaits further instructions.

Request for Quote The company personnel create the request for quote and set the limits on what the requested product or service can cost and what the other terms are. The subsequent interaction in the quote request is automated; if the terms are met the agent negotiates the final contract with the supplier agent.

Quote Negotiations If no quote is good enough, the agent presents a list of all the quotes. The company personnel can pick one of the quotes and decide to make a contract or modify it for further negotiation. If the buyer proposes a price lower than the one the supplier agent finds in the pricing list, the agent needs confirmation from the supplier company personnel to continue. Otherwise, it can proceed and make the contract.

Making the Contract This step is reached only if the terms set by the buyer company personnel are met. This step needs no confirmation from the buyer company personnel because the terms are set beforehand. This delegation was a design assumption; no authorization or audit mechanism for it was implemented.

This bounded-authority pattern (personnel-set limits, automatic action within them, and escalation outside them) is the earliest, design-level instance of the governable-agency through-line (Section 1.1.2) that HADA later generalizes to LLM agents (Section 6.1).

4.2.1.3 Artefact: Data Models and Ontologies

All documents exchanged between agents are grounded in two shared ontologies: a *proposal* ontology—an RFQ is a proposal concerning a product or service with payment and delivery terms (Appendix A, Figure 48)—and a *product* ontology that forms the basis for locating providers, both serialized as RDF/XML message schemas (Appendix A, Figure 49).

4.2.1.4 Artefact: Locating Services and Registry

Service location uses a *semi-intelligent*, category-based registry rather than the name- and taxonomy-code-based lookup of registries such as UDDI. In the registry, which is grounded in the product ontology (Appendix A, Figure 49), companies offer products within categories. A query therefore returns the set of companies *likely* to provide a product instead of a brittle one-to-one product→company map—a deliberately approximate match that mirrors real procurement, where one must ask candidate suppliers whether they carry the item.

4.2.2 Demonstration

The artefact was implemented (about 10 000 lines of code) with all parties realized as software agents on the FIPA-compliant JADE platform [143] (cf. Section 4.1.2.1); the category-based registry was implemented as Java classes reading an XML registry that instantiates the product ontology (Appendix A, Figure 49), and

proposal/product documents as Java classes serialized to XML. The demonstration exercises a single-round RFQ→negotiation→contract flow: a buyer agent issues a call-for-proposals to three supplier agents over FIPA Contract Net (Appendix A, Figure 36) and, when no quote satisfies the buyer's constraints, proposes a lower price over FIPA Propose, to which suppliers agree or refuse (Appendix A, Figure 37). The demonstration GUI shows the RFQ parameters, the constraints and limits, the returned quotes, and—on agreement—the final contract (Appendix A, Figure 50).

4.3 Matching Service Descriptions in a Context-Aware Environment

This artefact (A3, the *DYNAMOS* context-aware service matcher of Table 5) addresses **RQ I.3** (Table 4).

Problem Identification, Research Questions, and Objectives of the Solution

The problem identification, research questions, and objectives for this work are presented in Section 1.2.3. The research methodology is presented in Section 1.1.

Context: Sailing Scenario

This section presents the third and final Phase I demonstration. Portions of the system description below follow, in revised or lightly edited form, the text of the co-authored CAPS 2005 papers [3, 4] and the Licentiate thesis [2], and Figures 15 and 38 are redrawn from [3]. The demonstration combines B2C and C2C approaches so that users can share information contextually. As in the two previous demonstrations, shared ontologies play a central role, but context information is now also used to automate the linking between them.

In DYNAMOS (Dynamic Composition and Sharing of Context-Aware Mobile Services), mobile users are dynamically provided with descriptions of available services relevant to their current situation, taking into account contextual details such as time, location, and activity. Users can also annotate service descriptions and share them with each other (the hybrid service provision model of Figure 14), and create and share content unrelated to services. There are thus three kinds of content: service descriptions, service annotations, and user notes.

Recreational boaters were chosen as the user group. Examples of the three kinds of content are an advertisement of a guest harbor (service description), a natural-language or star rating of that harbor (annotation), and a warning about high rocks near the route to it at low tide (user note).

4.3.1 Design and Development

The DYNAMOS main design consists of the following artefacts:

A.I.3.1 Context-Profile-Content Ontologies OWL/RDF schemas and instances linking relatively static *user profiles* with dynamic *context* (time, location, activity) and with *service descriptions / annotations / user notes* (see Section 4.3.1.1).

A.I.3.2 Matching Engine & System Architecture Implemented matcher connecting (profile, context) to (services, annotations, notes), delivering a compact, filtered summary; deployed within the client–server architecture and content stores (see Sections 4.3.1.2 and 4.3.1.3).

A.I.3.3 Event-Driven Distribution Layer Publish/subscribe channels built on the adopted Fuego Core event platform [331] for context-change notifications and real-time sharing of annotations/notes between clients and servers (see Section 4.3.1.3).

4.3.1.1 Artefact: Shared Context-Profile-Content Ontologies

The major functionality of the DYNAMOS system is notifying mobile users about services in a context-sensitive fashion through the three kinds of content introduced above: *service descriptions* from service providers, *service annotations* from other users, and *user notes* that refer to no specific service. Notes can be warnings (e.g., *"Dangerous underwater rocks nearby!"*) about situations at a certain location and time, or plain notifications such as bird sightings; in a marine setting, such messages can be sorted as emergency/urgency/safety/routine events.

Service descriptions capture the essential semantics of the services: services are categorized by business branch and may carry a natural-language description. Descriptions are matched against user profiles, and a

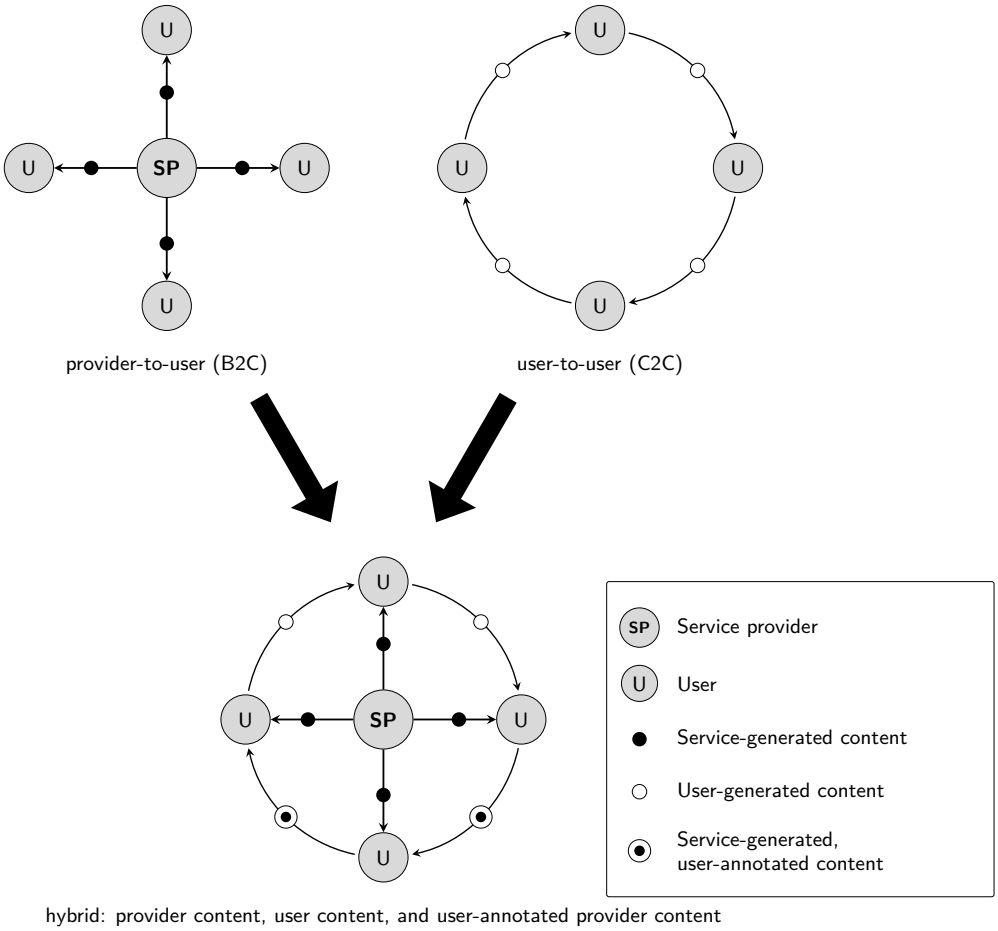


Figure 14: Hybrid approach for service provision, combining provider-to-user (B2C, top left) and user-to-user (C2C, top right) content sharing (own drawing, adapted from the model in [35]).

service annotation attached to a service of potential interest is provided to the user when it matches her current context. How user preferences (found in user profiles) should weigh the provision of annotations against that of service descriptions was left as future work at the time.

The user-profile and user-context ontologies (Appendix A, Figure 51) are interlinked through *activity*: a user specifies activities once (e.g., *at home*, *traveling*, or *at work*) and associates interests with each (e.g., local news, timetables, or nearby offers). The profile is fairly static; its dynamic part, the currently active interest, is treated as context, so the current activity activates the interests that link to the content the system provides.

4.3.1.2 Artefact: Matching Engine & System Architecture

The overall architecture of the DYNAMOS system is depicted in Figure 15. The three main components of the system are the following:

- **The Client:** a regular Web browser (thin client) or a stand-alone application (thick client) using event-based communication, which, compared with RMI (as in [332]), offered more fault-tolerant communication and wider support among client devices of the time (2004–2005).
- **Content Servers:** the User Content Server stores user-related information (profile and context data); the Service Content Server stores provider content (service descriptions) and user-generated content (user notes and service annotations), allowing the content to be distributed through multiple channels.

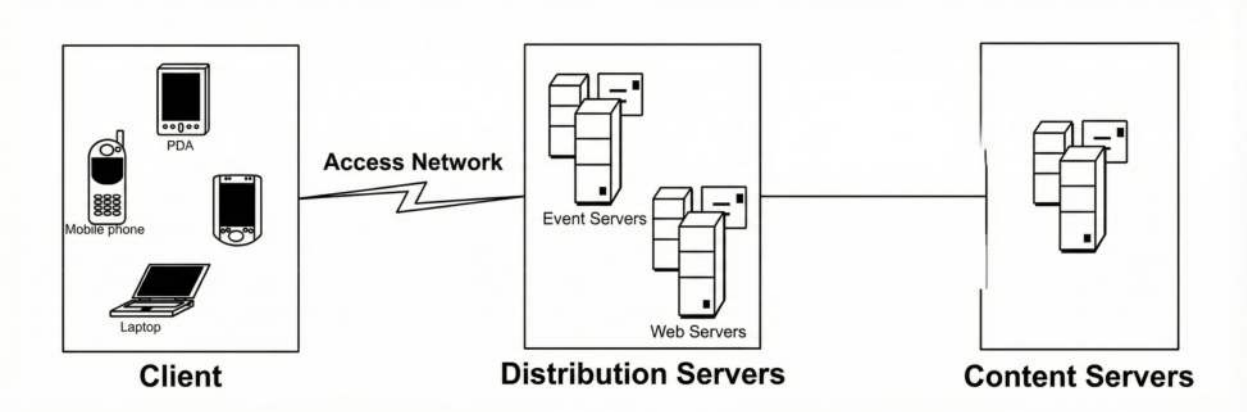


Figure 15: Overall architecture of the DYNAMOS system: client, distribution servers, and content servers.

- **Distribution Servers:** Web Servers and Event Servers support the different ways of accessing the content.

Access was supported through two interfaces: a Web interface, usable with any browser but designed for large screens, and, for mobile users with limited-capability devices, a context-aware stand-alone client that constantly monitored context parameters such as location and activity and notified the Event Server of relevant changes. Mobile users with small-screen devices were thus proactively given a compact summary of available services without having to browse the entire content stored in the system.

The Distribution Servers accessed the Content Servers over Java RMI. The Content Server combined objects and relational databases; for performance, most content was kept in memory, and only persisting content incurred time-consuming MySQL operations.

The Event Distribution Server was the Fuego Core event server [331], which provides event-based access to and updating of the content stored in the Content Servers. An event-based infrastructure was adopted for two reasons: context-aware applications must subscribe to and be notified of context-change events, and context changes and user-generated content must be shared among users in an event-based manner for efficient, real-time provisioning.

4.3.1.3 Artefact: Event-Driven Distribution Layer

Figure 38 (Appendix A) shows the Fuego Core architecture [331], which provides an event-based distribution channel and several enhancements for wireless Internet, and the DYNAMOS components implemented on it: the Event Server (right) and the client application (left). The main DYNAMOS software components are described below.

- The *User Note and Annotation Listeners* (on both client and server side) detect new user-generated notes and service annotations (made available by the *User Note Producer* and the *Annotation Producer*, respectively) and notify the upper layer, so clients can add and share such content in real time.
- The *Context Listener* (server side) keeps the server updated with the user profile and context changes produced on the client side by the *Context Manager*.
- The *Matcher* (server side), a key component of the DYNAMOS Server, matches the context and profile of active users (e.g., interest, location) against service descriptions (e.g., service category, location), annotations, and user notes, giving users a compact summary of relevant services and proactive notification of critical situations.
- The *User Manager* (server side) maintains a consistent, up-to-date version of the user profile and context in the DYNAMOS database, based on the changes communicated by the Context Listener.

4.3.2 Demonstration

The DSRP *Demonstration* of the DYNAMOS context-aware service matcher (A3) within the DYNAMOS system (Section 4.3.1.2) comprised (i) a preliminary field trial with real users in August 2005 and (ii) controlled technical measurements that reveal operational bottlenecks and scalability limits; together they show the artefact's *feasibility* and inform the subsequent evaluation and refinement steps of the DSRP process (Table 1).

4.3.2.1 Field Demonstration: August 2005 Sailing Trial

In August 2005, a user trial, reported in the Licentiate thesis [2], was held with Sonera Sailors, a private sailing interest group, during their annual race from Helsinki to Porvoo (system overview in Appendix A, Figure 52). Because boats of different classes are ranked on corrected time using a class-specific handicap (*LYS*), the first boat to finish may not win, and live standings are not obvious during the race; a DYNAMOS service therefore tracked each competitor's position and computed the corrected standings at checkpoints, delivered as ordinary context-matched services. Ten Nokia 6630 phones and ten GPS receivers were given to the sailors, who could follow the competition, view results, and post user notes (client screenshots in Appendix A, Figure 53).

4.3.2.2 Technical Performance Measurements

Controlled measurements on 2005-era Nokia 6600/6630 phones and commodity desktop PCs (2 GHz processor, 512 MB memory, Linux Red Hat 9 and Windows XP SP2) located the bottlenecks [3, 2]. Context updates were not the constraint: a client could update its context roughly once every four seconds over GPRS and about thirty times per second over an 11 Mbit/s WLAN, although WLAN-class rates would need throttling so as not to overload matching. The decisive cost was *integration plumbing* rather than matching: accessing the content servers over Java RMI was around three orders of magnitude slower than an in-process call (two to almost five, depending on the call and the network, with object serialization dominating), so cross-network invocation, not processor load, quickly became the bottleneck. With server-side, in-memory matching, the prototype sustained more than 6 000 location matches and about 3 000 category matches per second, and the catalog's $O(N)$ location lookup, measured on a catalog of about 1 000 entries, was estimated to reach roughly 10 000 entries before a spatial index (e.g., a two-dimensional location hash) would be required. The practical implication that shaped the architecture is to perform filtering and matching server-side and return only compact, pre-filtered result sets to a thin mobile client.

4.3.2.3 Assumptions and Limitations

The prototype assumes accurate GPS positioning on unmodified consumer phones and uses only location, time, and user activity as context, leaving social setting and non-GPS sensor feeds, both named in the research gap (Section 1.2.3), out of scope; the difficulty lay in deriving meaning from the parameters rather than in acquiring more. A user who drops out can rejoin without restarting unless the client itself crashes, and the phone tries to re-establish a lost GPS connection automatically.

4.3.2.4 User Trial Feedback

The user trial doubled as a stress test: with winds over 15 m/s the race became a *survival competition*, leaving most sailors little time to attend to the application. Two observations, recorded informally rather than through a structured instrument, emerged. First, the system and its server side worked as intended: users could follow standings, post notes, and read notes from other users, and the back end ran without problems. Second, the client and its device were fragile under realistic load: the client application leaked memory and crashed, partly because of Java Virtual Machine bugs on the 2005-era phones, when Bluetooth, GPRS, the camera, and the microphone were used together; the battery lasted only about half a day; and the interface had to collapse to single-tap interactions with clear defaults to remain usable.

5 PHASE II: DESIGN & DEVELOPMENT AND DEMONSTRATIONS

Phase I established that agents could coordinate and orchestrate services through shared ontologies and protocols, but it also exposed a hard limit: its symbolic, knowledge-engineered designs could not learn from data, and hand-built ontologies did not scale to open, real-world domains—the "inability to learn" identified in the Phase I conclusions (Chapter 8). Phase II takes up exactly that unsolved problem, shifting from hand-specified symbolic logic to *data-centric machine learning* over large, imperfect clinical datasets. This is the second step of the *governable-agency through-line* (Section 1.1.2): the representational substrate becomes statistical and data-centric while the software-agent abstraction persists. In this step the human-command facet of the through-line appears only as an interaction pattern—the aggregator agent poses its recommendations as questions for the clinician to confirm or reject, with bounded similarity scores that keep the ranking interpretable—and the absence of a principled governance mechanism is the limitation that Phase III inherits (Chapter 6).

This chapter (Phase II, 2007–2013) concentrates on applying software agents augmented with machine learning to support clinical decision-making and documentation. Building on the mechanisms and demonstrations of Phase I (Chapter 4), the focus shifts from general agent coordination and service orchestration to a domain setting in clinical work, where the operational objective is to assist clinicians in capturing structured information, linking diagnoses and procedures, and preparing downstream artefacts for coding and reimbursement. The design follows the DSR approach: the problem context is articulated, objectives for an effective solution are derived, and artefacts are engineered that combine agent-based interaction patterns with ML components where these add measurable value.

Portions of this material have appeared, in earlier or condensed form, in peer-reviewed venues and practitioner outlets, including [9, 10, 11, 12, 13, 14, 15]; the 2011–2012 cost-weight and hospital-affinity decks [16, 17] are related Phase II casemix work that is not reproduced here. The present chapter synthesizes those previously published contributions—primarily [14]—into a single, coherent narrative aligned with the thesis objectives and artefact set, and reports the demonstrations and results with consistent terminology and evaluation criteria; it adds no new experimental results (Section 1.6.3).

5.1 Assisting Clinical Decision Support with ML and Software Agents

This artefact addresses **RQ II.1** (Table 4).

Problem Identification, Research Questions, and Objectives of the Solution

The problem identification, research questions, and objectives for this work are presented in Section 1.3.1, the research methodology in Section 1.1, and the DSRP process sequence in Table 1.

Context

Hospitals sit on years of clinical experience, but much of it is trapped in fragmented IT landscapes: dozens of legacy systems, incompatible formats, and uneven documentation quality. Clinicians trying to make time-critical decisions must hop between systems to reconstruct a patient's story, while existing tools rarely surface "patients like this one" fast enough to influence care. The same fragmentation affects DRG-related funding decisions, where incomplete or inconsistent coding can misstate case complexity and impact reimbursement.

This work tackles that business problem by turning dispersed records into timely, case-based recommendations intended to support clinicians at the point of care. Given the methods and commodity hardware available when the artefact was designed (2008–2009), dense neural setups struggled with extremely sparse, high-dimensional code spaces (ICD-10/NCSP). Instead, the approach uses vector-space information-retrieval methods with kernel-based similarity and domain-aware weighting to find relevant historical episodes quickly and robustly—even when documentation is imperfect. A distributed, agent-based architecture orchestrates on-demand data collection across heterogeneous systems and returns ranked suggestions with sub-second latency on commodity hardware. The aim is a practical bridge from fragmented operational data to actionable decision support intended to improve coding quality, support auditability, and help align clinical and financial outcomes.

5.1 ASSISTING CLINICAL DECISION SUPPORT WITH ML AND SOFTWARE AGENTS

Background and Related Work

The term Computer-Aided Coding (CAC) is used to denote technology that automatically assigns codes from clinical documentation for a human to review, analyze, and use. At the time of the work, developers of CAC software employed a variety of methodologies to read text and assign codes: the software could use structured input or NLP, and even within the NLP range of products approaches varied in sophistication. These approaches included the use of external knowledge databases and feature extraction using Self-Organizing Maps (SOMs) [333]. The methodology used had a tremendous impact on data transmission and the output reviewed by the coders [334]. Some studies have shown CAC software performing strongly in comparison with human coding [335]. Other studies have concluded that no productivity increase was achieved [334]. Unlike CAC, which maps free-text documentation to codes, the artefact presented here operates on already-coded episodes and recommends missing or inconsistent codes by nearest-neighbor retrieval. It is thus closer to item-set mining [336] and to earlier agent-based information retrieval (Section 1.3), and its contribution is the application of these techniques to coded clinical episodes within a multi-agent system, rather than a new algorithm or a new pairing of agents with retrieval. Later neural approaches to automated coding are reviewed in Section 2.6, and the artefact is revisited from a 2026 perspective in Section 1.3.2.

5.1.1 Design and Development

The main artefact A4, *ML-enabled clinical recommendation MAS* (Table 5), comprises the following sub-artefacts:

A.II.1.1 Process Flow. A formalization of the healthcare item set abstract IT process flow and classification (see Section 5.1.1.1).

A.II.1.2 Data Space. A mathematical description of the data space combinatorics (see Section 5.1.1.2).

A.II.1.3 ML/IR Model. A description of the ML/IR model for the agents to consume (see Section 5.1.1.3).

A.II.1.4 MAS Architecture. A description of the MAS architecture (see Section 5.1.1.4).

5.1.1.1 Artefact: Healthcare Item Sets and Abstract IT Process

Artefact Description. This artefact formalizes the healthcare item set abstraction process and defines the abstract IT process flow for transforming clinical documentation into structured classification systems.

Theoretical Foundation. The artefact builds upon the hierarchical classification model illustrated in Figure 54 (Appendix A), which demonstrates the progressive abstraction from patient-level data to DRGs. This hierarchy encompasses six distinct levels of abstraction: (1) Patient—the physical entity requiring care; (2) Episode of care—the temporal boundary of treatment; (3) Medical record—the comprehensive documentation; (4) Discharge summary—the synthesized clinical narrative; (5) Diagnosis and procedure codes—the primary classification layer (ICD-10, NCSP); and (6) DRG—the secondary classification for resource allocation.

Abstraction Process Formalization. The artefact identifies the requirements for the abstraction process, addressing the inherent challenge of information reduction. With ICD-10 containing 103 syntactical variations for pneumonia coding and 848 for cancer coding [14], the abstraction must provide: (1) retention criteria for preserving clinically significant details; (2) reduction rules for eliminating redundant or non-essential information; and (3) consistency mechanisms to ensure uniform abstraction across practitioners. These requirements respond to documented quality problems in patient documentation [337, 338, 339, 340, 341, 342].

IT Process Architecture. Figure 55 (Appendix A) illustrates the artefact's process architecture, which transforms fragmented primary business processes into integrated secondary processes through data warehousing [343]. The architecture comprises three layers. The *Input Layer* consists of multiple heterogeneous systems (Legacy systems D4, D6, F3, A1; Accounting system C2; ERP systems B2, C4; HR system D7) capturing fragments of patient documentation during primary care processes. The *Integration Layer* includes data warehouses and databases that consolidate distributed information using OLAP methods and data cube aggregation techniques. The *Output Layer* enables secondary business processes for comprehensive analysis and decision support based on integrated patient data.

Operational Characteristics. The artefact addresses the reality of healthcare information fragmentation where patient documentation is distributed across dozens to hundreds of specialized systems. Each system captures partial information—one system may record "Patient X underwent surgical operation ACYZ01," another contains "Laboratory experiment D932 was performed on Patient X," while yet another knows "Patient X has chronic disease E023."

Design Rationale. The artefact's design is motivated by two critical requirements: (1) *Immediate clinical need*—enabling healthcare professionals to access comprehensive patient information without sequential system interrogation; and (2) *Knowledge management capability*—leveraging accumulated institutional knowledge about similar patient cases for evidence-based decision support. This formalization underpins the subsequent artefacts (Data Space, ML/IR Model, and MAS Architecture).

5.1.1.2 Artefact: Healthcare Item Set Data Space

Artefact Description. This artefact provides a mathematical formalization of the healthcare item set data space, establishing the computational boundaries and dimensionality constraints that govern the design of ML solutions for clinical decision support. The artefact quantifies the combinatorial complexity of healthcare documentation and demonstrates why, under the methods and commodity hardware available when the artefact was designed (2008–2009), traditional neural network approaches were insufficient for this domain. *This analysis is specific to the 2008–2009 commodity-hardware era; Section 1.3.2 revisits it from a 2026 perspective.* The terminology of *item sets* and the analysis of frequent patterns over large transaction databases originate in association-rule mining [336, 344].

Combinatorial Analysis. The artefact defines the theoretical space of all possible minimum datasets through combinatorial analysis. Given approximately 12,000 possible values in both ICD-10 diagnosis and NCSP procedure coding classifications [11], and allowing for 40 diagnosis codes and 40 procedure codes per patient episode, the total number of possible minimum dataset combinations C is:

$$C = \binom{ICD}{DG_{codes}} \cdot \binom{NCSP}{PR_{codes}} = \binom{12{,}000}{40} \cdot \binom{12{,}000}{40} = 2.8 \cdot 10^{230} \tag{1}$$

Dimensionality Characteristics. The artefact characterizes the data space along three critical dimensions. First, *High dimensionality*: Binary matrix representation yields approximately one million dimensions (12,000 possible codes $\times$ 80 code positions), creating computational challenges for traditional ML approaches. Second, *Extreme sparsity*: The ratio between actual patient cases (typically 10^5 to 10^8) and theoretical combinations (10^{230}) demonstrates that observed data occupies an infinitesimally small fraction of the possible space. Third, *Structural complexity*: The data space exhibits additional complexity through code repetition (multiple occurrences of the same diagnosis), positional significance (primary vs. secondary diagnoses), and supplementary clinical attributes beyond diagnosis and procedure codes.

Computational Implications. The artefact sets out why conventional neural network architectures, particularly SOMs, were computationally infeasible for this domain on the commodity hardware of the 2008–2009 design era. The high-dimensional, extremely sparse input space was taken to exceed the practical limits of neural network training at the time, both in terms of memory requirements and convergence time. While dimensionality reduction techniques, such as Principal Component Analysis (PCA) or random projection, could theoretically compress the space, the extreme sparsity and semantic importance of individual codes were expected to make such approaches unsuitable without significant information loss. Given these computational constraints, this study assumed that, at the time, neural networks, such as SOMs, were unsuitable for this type of data, which called for approaches that avoid dense matrix representations and iterative training. This is a time-bounded observation of its era rather than a standing limitation of neural methods: as revisited from a 2026 perspective in Section 1.3.2, subsequent advances in embeddings, transfer learning, and transformer architectures have since partly superseded this neural-infeasibility argument.

Design Constraints. This mathematical formalization imposes specific constraints on the solution architecture: (1) algorithms must operate efficiently in high-dimensional sparse spaces without requiring dense matrix operations; (2) similarity measures must preserve semantic relationships despite sparsity; (3) computational methods must scale sub-linearly with dimensionality; and (4) storage strategies must exploit sparsity to achieve practical memory footprints.

Analytical Contribution. The artefact contributes an explicit upper bound on the combinatorial space of coded episodes (Eq. 1). At the time, this formalization justified the selection of vector space models [345] and kernel methods [346] over neural networks, establishing a mathematical foundation for the subsequent ML/IR model artefact. The analysis suggests that, with the technology of the time, effective healthcare decision support favored algorithms specifically designed for sparse, high-dimensional categorical data over the adaptation of general-purpose ML techniques then available.

5.1.1.3 Artefact: ML and IR Model

Artefact Description. This artefact defines an ML and IR model specifically designed for high-dimensional, sparse healthcare data spaces. The model adapts kernel methods and vector space techniques from IR to enable efficient similarity computation and pattern recognition in clinical coding systems, providing the computational foundation for the MAS recommendation capabilities.

Theoretical Foundation. The artefact builds upon kernel methods for pattern analysis [346]: the cosine similarity of Eq. 2 is a normalized linear kernel over the sparse code-vector representation, so similarities in the high-dimensional feature space are computed from the few non-zero coordinates of each vector. Unlike traditional applications, such as Support Vector Machines (SVMs), that focus on classification boundaries, this artefact adapts kernel methods for nearest-neighbor retrieval in sparse categorical spaces. The model transforms the healthcare item set matching problem into a vector space similarity problem, enabling efficient computation despite the 10^{230} theoretical combination space identified in the Data Space artefact.

Vector Space Formulation. The model represents patient episodes as vectors in a high-dimensional space where each dimension corresponds to a clinical code. Figure 56 (Appendix A) illustrates how coded datasets form an interconnected network through the coding scheme. Each patient episode $D_i = (d_{i1}, d_{i2}, \ldots, d_{it})$ becomes a point in this space, with queries $Q = (q_1, q_2, \ldots, q_t)$ formulated as vectors seeking similar episodes.

Similarity Computation. The artefact employs cosine similarity (the complement of the Cosine Angle Distance, CAD) as the primary similarity measure, chosen for its adequacy in high-dimensional spaces [347] and its computational efficiency over sparse inverted-index representations [348]. The similarity between query vector Q and document vector D_i is computed as:

$$s(Q, D_i) = \frac{\sum_{j=1}^{t} (q_j d_{ij})}{\sqrt{\sum_{j=1}^{t} q_j^2} \sqrt{\sum_{j=1}^{t} d_{ij}^2}} \tag{2}$$

This normalized dot product (the cosine similarity on which the CAD ranking is based) ensures that similarity scores remain within $[0, 1]$ for the non-negative weights of Eqs. 3–4, regardless of vector magnitude, enabling consistent ranking across patient episodes with varying numbers of diagnoses and procedures.

Weighting Schema. The model implements TF-IDF (Term Frequency–Inverse Document Frequency) weighting adapted for clinical codes. Document weights incorporate both code frequency within an episode and inverse frequency across the database:

$$d_{ij} = tf_{ij} \cdot \log\left(\frac{N}{f_j}\right), \tag{3}$$

where t_j denotes the clinical code at dimension j (for $j = 1, \ldots, t$), tf_{ij} represents occurrences of code t_j in episode i, N is the total number of episodes, and f_j is the number of episodes containing code t_j. Query weights emphasize discriminative codes:

$$q_j = \begin{cases} \log\left(\frac{N}{f_j}\right) & \text{if code } t_j \text{ appears in query} \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

Domain-Specific Adaptations. The model incorporates three healthcare-specific enhancements that extend the base formulation of Eqs. 2–4 and are not formalized further here. First, *Hierarchical code relationships*: The model exploits ICD-10 and NCSP hierarchical structures, allowing similarity computation

between related codes (e.g., different pneumonia subtypes). Second, *Clinical importance weighting*: Primary diagnoses receive higher weights than secondary diagnoses, reflecting clinical documentation practices. Third, *Flexible matching strategies*: The model supports both strict matching (MUST-occur) and relaxed matching (SHOULD-occur) to accommodate varying data quality levels.

Computational Advantages. The IR model provides several computational benefits over alternative approaches: (1) Sparse matrix operations reduce memory requirements from $O(N \cdot t)$ for a dense episode-by-code matrix to $O(k)$, where k is the number of non-zero elements; (2) Inverted index structures enable sub-linear query time complexity; (3) Incremental updates allow real-time index maintenance as new episodes arrive; and (4) Distributed computation supports horizontal scaling across multiple agents.

Design Justification. The selection of IR techniques over the neural networks or traditional ML available at the time is justified by three factors identified in the Data Space artefact: the extreme sparsity of healthcare data, the semantic importance of individual codes that precludes aggressive dimensionality reduction, and the requirement for sub-second response times on commodity hardware. This model provides the mathematical foundation enabling the MAS Architecture artefact to achieve objectives O.II.1.1–O.II.1.4 of the solution (Section 1.3.1).

5.1.1.4 Artefact: MAS Architecture

Artefact Description. This artefact defines a MAS architecture that operationalizes the theoretical models into a distributed, scalable clinical decision support system. The architecture employs specialized software agents to address the three-dimensional problem identified in Section 1.3: information system fragmentation, limited real-time clinical intelligence, and scalability constraints. The artefact shows how agent-based design patterns are intended to enable real-time integration of heterogeneous healthcare data sources; the demonstration measured sub-second query response times for the recommendation step.

Architectural Foundation. The MAS architecture is implemented on the JADE platform version 3.7, chosen for its FIPA compliance and distributed container architecture [143]. The platform enables agents to operate across heterogeneous hospital information systems without requiring homogeneous operating systems or hardware configurations. This foundation provides runtime agent mobility, dynamic system reconfiguration, and fault tolerance through agent replication.

Agent Taxonomy and Responsibilities. The architecture comprises four specialized agent types, each addressing specific aspects of the healthcare data integration challenge:

1. **Data Crawler Agents**: These agents encapsulate system-specific knowledge for interfacing with heterogeneous data sources. Implemented using object-oriented design patterns, they inherit common functionality from abstract base agents while implementing the information-system-specific data handling in concrete subclasses [14]. Each crawler agent translates proprietary data formats into standardized representations.
2. **Data Integrator Agents**: These agents transform heterogeneous data streams into a unified knowledge representation. Their design role is to resolve conflicts between contradictory source records, to track the provenance of integrated data, and to keep asynchronously collected data temporally consistent; in the demonstration reported below the integrators were exercised only in building the common knowledge base from coded episodes, and these three functions were not separately evaluated. The integrators build and maintain the common knowledge base using the vector space model defined in the ML/IR Model artefact.
3. **Data Commander Agents**: These orchestration agents implement demand-driven data collection strategies. Rather than continuous polling of hundreds of systems, commanders activate specific crawler agents based on query requirements. When a clinician requests patient information, the commander analyzes the query context, identifies relevant data sources, and dispatches parallel collection tasks to appropriate crawlers, optimizing for minimal latency and resource utilization.
4. **Data Aggregator Agents**: These agents provide the clinical interface layer, implementing the ML/IR model to generate recommendations. They transform clinician queries into vector space representations, compute similarities using the kernel methods, and rank recommendations based on historical case similarities. Aggregators support multiple query strategies (MUST-occur, SHOULD-occur) and adapt to varying data quality conditions.

Aspect-Oriented Design. The architecture employs aspect-oriented design to divide roles between the components, separating the crawler, integrator, commander, and aggregator responsibilities [14]. This design enables consistent behavior across all agent types while allowing specialized functionality within each agent class.

Distributed Processing Model. The MAS implements a hierarchical distributed processing model. Crawler agents operate at the edge, co-located with data sources to minimize network latency. Integrator and commander agents function at the aggregation tier, potentially distributed across multiple servers for load balancing. Aggregator agents operate at the presentation tier, providing localized caching and query optimization. This distribution strategy is designed to scale with data volume (measured in Figure 17) and with system diversity (not measured).

Real-Time Coordination Protocol. Agent coordination follows an application-specific, event-driven message sequence among the four agent types, designed for the clinical query workflow. When a clinician initiates a query: (1) The aggregator agent receives the request and forwards it to the commander; (2) The commander decomposes the query and dispatches parallel tasks to relevant crawlers; (3) Crawlers retrieve data asynchronously and stream results to integrators; (4) Integrators merge results incrementally, enabling progressive result display; (5) The aggregator applies the ML/IR model to generate and rank recommendations in real time.

Scalability Mechanisms. The architecture incorporates three primary scalability mechanisms. First, *Horizontal scaling* through agent replication—multiple instances of each agent type can operate concurrently. Second, *Lazy evaluation*—data is fetched only when requested rather than maintaining synchronized replicas. Third, *Intelligent caching*—frequently accessed patient data and similarity computations are cached at multiple levels.

Classification System Independence. The architecture maintains independence from specific clinical classifications through abstraction layers. While demonstrated with ICD-10 and NCSP, the agent interfaces are designed to accommodate other coding systems such as Systematized Nomenclature of Medicine Clinical Terms (SNOMED CT), Logical Observation Identifiers Names and Codes (LOINC), and Current Procedural Terminology (CPT); their use was not tested in this work. This flexibility is intended to enable deployment across different healthcare systems and geographical regions without architectural modifications.

Performance Characteristics. The artefact achieves the performance objectives specified in Section 1.3.1 (SRQ.II.1.4, O.II.1.4): sub-second query response times (at most about 0.5 s) for databases containing 10^8 patient episodes (see Scalability Results below) and graceful degradation under documentation defects (Section 7.4). Concurrent-user load and behavior under agent or system failures were design goals that were not experimentally characterized. The results therefore demonstrate the performance feasibility of the agent-based design on commodity hardware rather than production readiness.

5.1.2 Demonstration and Results

The demonstration reports recommendation accuracy measures for different clinical use case scenarios based on real material from Finnish hospitals. The materials contain hundreds of thousands of patient care episodes. In the accuracy experiments below, a fact removed from the recorded episode serves as ground truth; because historical coding itself carries reported error rates of roughly 10–17% [340, 342], the reported accuracies are measured against a noisy reference (see Section 7.4). One patient care episode is described using several clinical classifications, such as ICD-10 [349, 350], NCSP [351], and a separate classification of laboratory results. As the MAS is designed not to be restricted to any particular use case or clinical classification, different clinical decision support needs can be served by different aggregator agent variants; for instance, the aggregator agent can give the following answers:

- "Patients who had laboratory experiment D932 often have chronic disease E023. However, the documentation of this patient does not contain this information. Is this information missing?"
- "For patients who had surgical operation ACYZ01, the primary diagnosis was usually J189. However, for this patient the primary diagnosis is E099. Is the information correct?"
- "Patients with primary diagnosis N039 who underwent operation AC024 commonly have secondary diagnosis B069. Did this patient also have that diagnosis?"

The experiment has two parts: first, recommendation accuracy, including its robustness under low-quality data; second, a scalability experiment showing how large a patient-register database the system can handle. Its variables are the database size $|D|$ (the number of patient care episodes in the database), kNN (the number k of nearest patient care episodes that the data aggregator agent fetches from the database), and topN (the most probable topN recommendations that the data aggregator agent provides for healthcare professionals).

5.1.2.1 Recommendation Accuracy

Figure 16(a) depicts the recommendation accuracy using topN as a variable. In this accuracy experiment, one fact from the patient care episode is first removed. The remaining facts about the care episode are provided to the data aggregator agent, which proposes topN facts that would probably occur within this care episode. The purpose is to evaluate how many recommendations need to be presented for the healthcare professional before the recommendation becomes accurate. The fetching strategies differ in whether each fact of the query episode has to occur also in a fetched episode of the existing knowledge base (strict matching, MUST-occur) or may be absent from it (relaxed matching, SHOULD-occur). The first proposal of the data aggregator agent is exactly correct in 66% of the cases with the SHOULD-occur strategy (53% with MUST-occur); the accuracy then increases and stabilizes at around the 90% level with topN = 8, so there is no need to present a large number of alternatives to the healthcare professional. SHOULD and MUST converge as topN grows on these frequent patterns, although the SHOULD fetching strategy reaches a stable accuracy level slightly faster.

Figure 16(b) depicts the recommendation accuracy using kNN as variable. In this experiment one fact is removed from the patient care episode and the aggregator agent proposes one fact (topN=1) that would probably occur within this patient care episode. The purpose is to evaluate how many nearest neighbors (kNN) need to be fetched from the knowledge base before the recommendation becomes accurate. The recommendation accuracy decreases as kNN increases. It can be assumed that this is partly due to the sparse data space: the number of all possible fact combinations is excessive compared to the real number of patient care episodes. Consequently, only a small number of episodes need to be fetched from the knowledge base for accurate recommendations. The SHOULD fetching strategy also outperforms the MUST fetching strategy, with the gap widening as kNN grows.

As discussed, healthcare datasets suffer from quality problems [337, 338, 339, 340, 341, 342]; Figure 16(c) therefore depicts the recommendation accuracy with low-quality datasets using topN as a variable (kNN=5). These low-quality cases have incorrect codes or codes that are new to the knowledge base. The MUST fetching strategy fails completely, with zero accuracy at every topN, because the knowledge base does not contain exactly matching kNN. The SHOULD fetching strategy performs better and the decline in recommendation accuracy relative to Figure 16(a) is small (from 66% to 59% at topN=1).

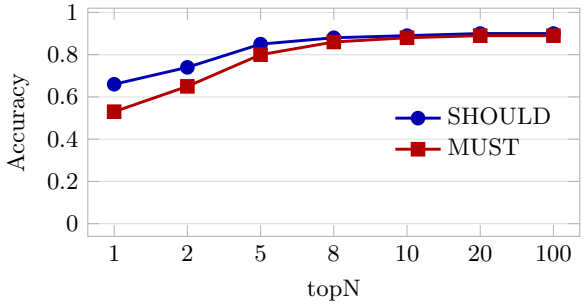
5.1.2.2 Scalability Results

The scalability of the proposed MAS was evaluated using a combination of an architectural assessment of the agent roles (below) and experimental scaling of the material. Because the MAS is distributed and loosely coupled, the architecture is designed to scale to large enterprise environments. Commander agents are not expected to limit scalability: they only have to coordinate more data crawler agents. As new information systems are encountered, new generations of data crawler agents that provide the information to the central database are created. By design, the amount of data per agent is not expected to increase significantly as the number of information systems increases.

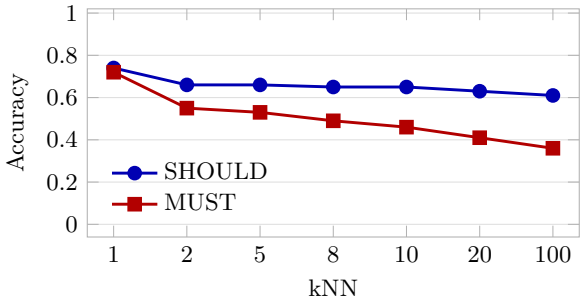
Possible scalability problems concern the data integration and aggregation agents: the central knowledge base can be assumed to have limits on how much data it can contain, and aggregating this information on the fly for the needs of the healthcare professionals is a possible bottleneck.

The data aggregators were first evaluated with real datasets; since these caused no performance problems, larger, synthetic databases were derived from the real data. Their sizes varied from 1,000 to 100 million patient care episodes. It was considered sufficient if the system could handle 100 million patient care episodes in the on-the-fly analysis with the regular desktop hardware of the time. The materials and their physical sizes are listed in Table 8. As shown in Table 8, the sizes vary from less than 1 MB to over 24 GB. Each coded dataset contained a varying number of facts about patients.

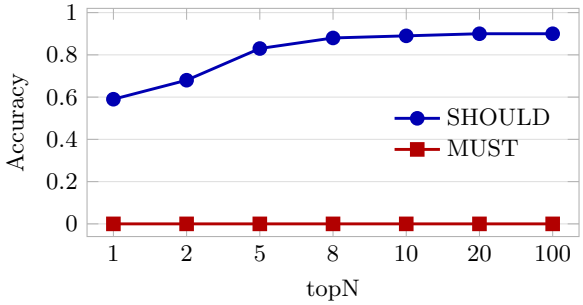
Figure 17 depicts scalability experiment results using the database size |D| as a variable. In this experiment, facts from existing patient care episodes were removed and the data aggregator agent was used to propose topN=5 recommendations for the healthcare professional based on kNN=5. As can be seen from Figure 17,



(a) topN as variable (kNN=5).



(b) kNN as variable (topN=1).



(c) topN as variable with low-quality data (kNN=5).

Figure 16: Recommendation accuracy (share of cases in which the removed fact is among the topN proposals) of the multi-agent system with the SHOULD-occur and MUST-occur fetching strategies (x axes categorical, not to scale): (a) accuracy vs. topN (kNN=5); (b) accuracy vs. kNN (topN=1); (c) accuracy vs. topN with low-quality data (kNN=5). Redrawn from the original charts of [14]; values digitized from them (about ±0.01).

Material	|D|	Size (MB)
M1k	1,000	0.26
M10k	10,000	2.54
M100k	100,000	23.60
M1000k	1,000,000	235.00
M10000k	10,000,000	2351.00
M100000k	100,000,000	24534.00

Table 8: Materials used in the scalability experiment (Figure 17): number of patient care episodes $|D|$ and database size.

the query execution times on regular desktop hardware are a couple of milliseconds and at most about half a second (0.33 s with MUST, 0.50 s with SHOULD) even with the largest materials with 100 million patient care episodes. The MUST fetching strategy is faster, clearly so at the two largest materials (0.03 s vs. 0.18 s with 10 million episodes), because it can apply stricter restrictions while fetching the kNN.

5.1.2.3 Summary of Results

The multi-agent prototype system was implemented and tested with real healthcare data. When one fact was removed from the patient documentation, the system proposed the correct fact with 66% accuracy (SHOULD-occur) as the first proposal and approximately 90% accuracy within the first eight proposals (Figure 16(a)); with the SHOULD-occur fetching strategy, it also handled low-quality datasets. Recommendation accuracy was measured on the real hospital material; on enlarged (synthetic) databases of up to one hundred million patient care episodes, the system returned recommendations within sub-second query times. Accuracy at that scale was not measured. Section 7.4 evaluates these results against objectives O.II.1.1–O.II.1.4.

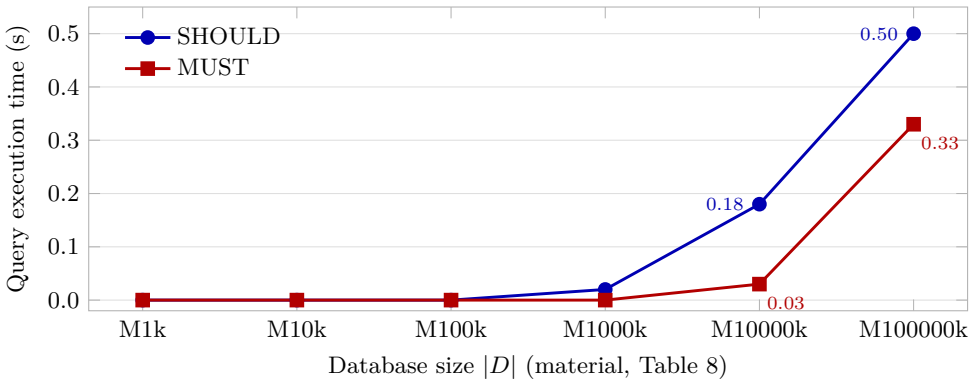


Figure 17: Query execution time (seconds) with the SHOULD-occur and MUST-occur fetching strategies when scaling the database size (kNN=5, topN=5). Redrawn from the original chart of [14]; values digitized from it (about ±0.005 s).

6 PHASE III: DESIGN & DEVELOPMENT AND DEMONSTRATIONS

Phase II showed that data-centric machine learning could deliver case-based decision support over imperfect clinical data, with sub-second response on enlarged synthetic databases—but its models were *bespoke and hand-built*, could not explain their reasoning in domain terms, and offered no principled way to keep human experts in command as systems scaled. Phase III takes up that unsolved problem, using large language models as the reasoning core of agents *inside* auditable, human-governed multi-agent architectures, so that flexibility and explanation are gained without surrendering accountability. This is the third step of the *governable-agency through-line* (Section 1.1.2): with the substrate now a foundation model, the recurring demand for auditable, human-aligned agency moves to the center of the design.

This chapter (Phase III) has three parts. First, *HADA* is presented, a protocol- and framework-agnostic reference architecture for human–AI decision alignment in multi-agent settings. HADA is instantiated on a cloud-native stack (Python, Docker) and demonstrated in a retail-bank sandbox around a loan-decision use case. The deployment exposes a governed AI Tool (`getLoanDecision`)—specified via OpenAPI and backed by a credit-risk model—that operates under stakeholder-aware orchestration spanning strategic time horizons. The section recalls the problem and objectives (Section 1.4.1), details the design and development of the architecture (including compatibility with agent protocols such as MCP and A2A), and reports a demonstration in a banking sandbox; the evaluation against the predefined solution objectives follows in Section 7.5.

Second, the *NordDRG AI Benchmark* for large language models in the context of hospital funding is released and evaluated. The benchmark bundles machine-readable NordDRG definition tables, expert governance manuals, dataset configurations, and 26 task prompts (13 automatically verifiable *Logic* tasks and 13 *Grouper* emulation tasks). A baseline is reported under an *artefact-only (no-web)* setting across eleven proprietary LLM endpoints from three providers, showing that models handle many logic tasks from the structured tables yet face challenges with full grouper emulation. Third, the *CCL* maintenance process is introduced (Section 6.3). Together, these form the two-harness picture of Phase III over decision-making models: the benchmark establishes the *capability premise*; HADA is the *deployment harness* that governs distributed agentic decisioning across an organization, over many models (breadth); and CCL is the *maintenance harness* that keeps one large decision-making model correct and evolving (depth)—all sharing one governance layer. The benchmark and CCL are grounded in a real reimbursement system (NordDRG), while HADA is demonstrated on a retail-bank credit decision.

Elements of this chapter have appeared, in preliminary, condensed, or (in parts) verbatim form, in [18, 19, 20, 21, 23, 24, 26, 27, 28, 29].

The three sections are HADA (Section 6.1), the NordDRG AI Benchmark (Section 6.2), and CCL (Section 6.3), in the numbering order of their research questions (RQ III.1–III.3); the argument they jointly make reads premise (the benchmark), depth (CCL), and breadth (HADA), as in Figure 2, so HADA, presented first, rests on a premise tested in the second section; for orientation within the thesis structure and DSRP flow, see Table 1 and Section 1.4.

6.1 HADA: Human-AI Agent Decision Alignment Architecture

This artefact addresses **RQ III.1** (and contributes to **RQ III.3**; Table 4). It was first reported in [28], building on [26, 27].

Within the two-harness picture of Phase III (Figure 2), HADA is the *deployment harness*: the surrounding governance a distributed cohort of agents needs to run decision-making across an organization, over many decision-making models (Definition 3.1.17), large or small, whose *use* it governs; it builds on the benchmark's *capability premise* (presented in Section 6.2 below) and is the breadth counterpart to CCL (Section 6.3). In the terms of Definition 3.2.5, the RACI matrix (Section 6.1.1.4) is $\mathcal{R}$, the Values Catalogue is $\mathcal{N}$, the Strategy-Process Blueprint (Section 6.1.1.1) is $\mathcal{O}$, and the decision ledger, designed to be append-only, realizes $\mathcal{L}$ in part for loan decisions (Decision Auditability, below).

Problem, method, and background. The problem identification, research questions, and objectives for this work are presented in Section 1.4.1, the research methodology in Section 1.1, and background and related work on LLM agents, AI alignment, and MAS in Sections 2.5, 2.6.2, and 2.4.5.

Context

Agent standards, such as the MCP and A2A protocols [352], let agents share data and tasks, yet organizations still lack a systematic, auditable way to keep those agents—and legacy algorithms—aligned with organizational targets and values.

This study takes place within a sandbox *retail banking* scenario, where an AI-driven credit-approval process is part of the bank's customer acquisition and risk management strategy. In recent quarters, the bank has expanded its customer base through acquisition channels such as online campaigns, instant credit offers, and short-term consumer loans designed to lower entry barriers for new clients; these initiatives accelerated customer growth and improved short-term market share, but also increased the exposure to credit defaults, particularly in unsecured loan segments. Following a recent strategic review, the bank's quarterly objectives have shifted from rapid acquisition toward *minimizing credit risk* and improving the robustness of automated lending decisions, which requires the existing AI-agent-based decision workflows to be re-evaluated and re-aligned with the updated business priorities, compliance requirements, and ethical guidelines.

In the subsequent *Design and Demonstration* sections, we focus on one AI Tool—`getLoanDecision`—to illustrate how these changes are operationalized, examining it and its supporting governance artefacts from the perspective of the chief credit officer, business managers, audit managers, value and ethics managers, data scientists, and customers (Table 9).

Positioned against the agent landscape of Section 2.5.6, HADA does not compete on raw capability with single-agent reasoning/tool-use methods (ReAct, Toolformer) or multi-agent orchestrators (AutoGen, MetaGPT); it is designed to *extend* them (the prototype itself orchestrates its agents with LangGraph). Those frameworks decide how an agent reasons, calls tools, or collaborates; they leave governance of tool invocation largely to the application developer and, although some provide human-input hooks or role definitions, they do not model organizational accountability (who answers for a decision against which target or value).

HADA does not claim first-mover status over the concurrent wave of agent-governance work, such as run-time guardrail classifiers [353] and agent-visibility proposals [259], which screen individual prompts and model outputs or make agent actions observable but do not tie them to organizational OKRs, RACI roles, and a decision ledger; its specific contribution is a protocol-agnostic synthesis—OKRs and machine-readable values mapped to role-based controls, an audit ledger, and human sign-off—demonstrated in a sandbox modeled on a regulated credit-scoring setting. It wraps such LLM tool- and role-agents in this *auditable governance layer*, built on role-based controls. Concretely, HADA targets six governance objectives, one for each facet of that synthesis (O.III.1.1–O.III.1.6, Section 1.4.1.4), evaluated in Section 7.5.

In requirements-engineering terms, mapping organizational goals and values to enforced agent behavior is related to goal- and agent-oriented modeling (Tropos [78, 79]), in which security is treated as a first-class concern throughout development (Secure Tropos [256]) and trust is an explicit model element [257]; these are primarily design-time concerns, and HADA's addition is to carry them, together with governance, to each run-time decision (Table 7). HADA, however, expresses goals operationally as OKRs, user stories, and RACI assignments rather than as formal actor/goal or trust models; no such model of the workflow was constructed, so the OKR-to-algorithm refinement is demonstrated rather than verified (Section 7.5), and formal goal and trust modeling is left for future work. The human-in-command sign-off (the accountable-human clause of $\mathcal{R}$) is a design commitment, not a guarantee: research on human reliance on automation and AI shows oversight is effective only when human reliance is well-calibrated [354, 355, 356, 357, 358]; HADA's audit ledger and approval steps operationalize the kind of agent visibility [259] and human-oversight mandate that high-risk, regulated settings increasingly require (e.g., the EU AI Act, Article 14 [261]). Yet oversight does not reliably correct algorithmic decisions [359], meaningful human control demands explicit tracking and tracing [260], and human–AI teams often underperform the best of either alone [360].

6.1.1 Design & Development

This section instantiates the DSRM [58, 31] step 3 ("*Design & Development*") through six interlocking artefacts that operationalize the HADA architecture and together realize the solution objectives defined in Section 1.4.1.4:

A.III.1.1 Strategy-Process Blueprint. Formalizes the yearly → daily planning loop (OKRs, KPI cascades, operational reviews) that frames every subsequent decision governed by HADA (Section 6.1.1.1).

A.III.1.2 Stakeholder & User-Story Specification. Captures the problem context through a stakeholder map, sequenced user stories, and the RACI assignments of A.III.1.4, using Artefact A.III.1.3 as the concrete AI Tool / AI Algorithm (Section 6.1.1.2).

A.III.1.3 Prototype AI Tool `getLoanDecision`. An operational decision-tree model (dataset, feature engineering, API) that instantiates the credit-approval use case (Section 6.1.1.3).

A.III.1.4 AI-Responsibility Tool (RACI Matrix). Distills governance duties across eight lifecycle activities, ensuring accountable, auditable hand-offs between roles regarding Artefact A.III.1.3 (Section 6.1.1.4).

A.III.1.5 Generic HADA Architecture. Provides the layered, protocol-agnostic reference design (agent layer, tools layer, DevOps view) on which all domain instances rest (Section 6.1.1.5).

A.III.1.6 Agent Modeling. Maps human roles to containerized interaction agents and describes the HADA Controller pattern for policy-enforced orchestration (Section 6.1.1.6).

6.1.1.1 Artefact – Strategy-Process Blueprint

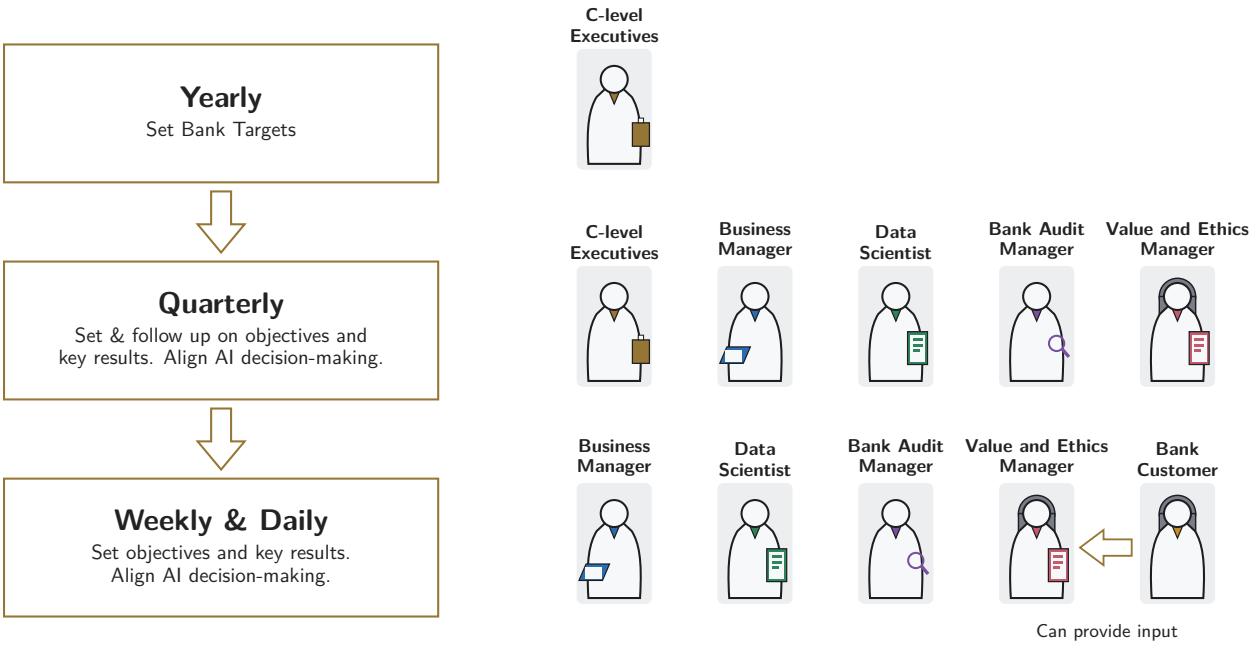


Figure 18: Bank strategy process: time horizons and stakeholder coverage.

Purpose. A.III.1.1 formalizes the yearly-to-daily planning loop that steers the bank from strategic decisions to daily activities as illustrated in Figure 18. The example bank follows a typical strategy formulation and implementation process in which the yearly overall targets are set at the beginning of the year. The progress and continuous alignment of the more detailed activities are typically followed up at least quarterly, for example with the Objectives and Key Results (OKR) framework [361], which formalizes the key results follow-up process and describes the mechanisms required to create alignment in the organization. While the C-level executives formulate the overall strategy and set the high-level objectives for the organization, the experts and managers on the lower organizational levels implement it. In the bank example, the "connecting OKRs" process described by Niven and Lamorte [361] needs to be expanded to cover not only the decisions and actions taken by the human employees but also the algorithms; in the terms of Definition 3.2.5, this extended OKR cascade, held in the Business-Target Catalogue, is HADA's objective binding $\mathcal{O}$.

Yearly — Set bank targets. At the start of each fiscal year, the Chief Credit Officer (CCO) and peer C-level executives establish portfolio-level OKRs together with an updated risk-appetite statement. These artefacts are stored in the *Business-Target Catalogue* and become immutable reference points for all downstream KPI specifications.

Quarterly — Translate OKRs into actionable KPIs. Every quarter the CCO reconvenes with Business Managers, Data Scientists, Audit Managers, and Value & Ethics Managers to review recent performance and—if necessary—re-weight model KPIs (e.g., shifting focus from "new-customer acquisition" to "expected-loss minimization").

Weekly & Daily — Operative alignment loops. On a rolling basis the following stakeholders contribute: Business Manager – reviews live KPI dashboards and reprioritizes tickets; Data Scientist – publishes hot-fix model versions or feature toggles via the Model Catalogue; Audit Manager – samples recent decisions from the append-only decision ledger and verifies lineage; Value & Ethics Manager – inspects the sensitive-attribute watch-list and issues ethics triggers when required; Bank Customer – can provide input (e.g., a complaint) to the Value & Ethics Manager (Figure 18).

Contribution. The Strategy-Process Blueprint extends the OKR cascade of [361] from human decisions to the algorithms that make them, and provides one auditable structure that a) anchors KPI and value alignment across the yearly-to-daily horizons, b) delineates hand-offs between business, technical, and oversight roles, and c) remains tooling-agnostic—no assumptions are made about the underlying LLM, agent library, or orchestration layer.

6.1.1.2 Artefact – Stakeholder & User-Story Specification

A.III.1.2 formalizes the *problem context* and the *functional requirements* as a lightweight specification composed of (i) a stakeholder map, (ii) an ordered set of user stories, and (iii) a cross-reference to the RACI matrix of A.III.1.4 (Section 6.1.1.4) that clarifies the roles and responsibilities of each stakeholder.

Stakeholder map. Table 9 lists the six principal roles involved in the experimental banking scenario, their primary goals, and the access rights HADA grants them. The mapping was reviewed with subject-matter experts from credit-risk, compliance, and data-science backgrounds in three semi-structured workshops [28]; because these sessions were not designed as an empirical study, the mapping is illustrative rather than empirically validated.

Table 9: Stakeholder map for the banking use case `getLoanDecision`.

Role	Primary goal in workflow	Key HADA privileges
Customer	Receive a fair, fast, and transparent credit decision; raise complaints when needed	Submit loan-application data; obtain natural-language decision explanations; open ethics tickets
Chief Credit Officer (CCO)	Define company-level OKRs for credit strategy; align risk appetite, growth targets, and regulatory limits across the portfolio	Set and update top-level KPI/OKR targets; approve credit policy changes; view enterprise-wide risk dashboards; be informed of new production decision AI Tool deployments (approval rests with the Business Manager; Table 10)
Business Manager	Set and adjust portfolio KPIs (e.g., minimize expected credit losses)	Edit KPI targets; approve or reject AI Tool versions for production; view live KPI dashboards
Data Scientist	Build, test, and monitor the `getLoanDecision` model so it meets current KPI and risk targets	Retrain models; run offline validation; register new AI Tool versions in the Model Catalogue
Audit Manager	Verify compliance and traceability of any individual credit decision	Read-only access to the full decision path, model version, feature values, and audit ledger
Value & Ethics Manager	Safeguard ethical use of data and enforce organizational values across all AI Tools	Maintain Values & Data catalogs; flag or deprecate sensitive attributes; approve/reject attribute changes; mandate model retraining when catalog updates occur

User stories. The user stories below run sequentially: first, the CCO and the Business Manager change the business targets, and the AI Tool is optimized to achieve them; this introduces ethically questionable logic

into the AI Tool, but these stakeholders do not notice it. The customer then applies for a loan, receives it through questionable logic, and files a complaint after noticing that questionable data was used in the loan decision. Finally, the HADA system contacts the auditor and then the Value & Ethics Manager to adjust the system.

- As the **Chief Credit Officer (CCO)**, I want to update the quarterly OKRs from acquiring new customers to minimizing credit losses during the quarterly OKR review.
- As a **Bank Loan Department Business Manager**, I would like to change the business target for short-term loan decisions from acquiring new customers to minimizing credit losses.
- As a **Data Scientist**, I want to create a new version of the `getLoanDecision` AI Tool so that it matches the new business targets.
- As a **Bank Customer**, I would like to apply for a short-term loan.
- As a **Bank Customer**, I would like to file an ethics complaint, because ethically questionable information (my ZIP code) was used in my loan decision.
- As a **Bank Audit Manager**, I would like to audit the detailed decision criteria for a single short-term loan decision.
- As a **Bank Value and Ethics Manager**, I would like to remove ethically questionable data points from loan decision-making.

6.1.1.3 Artefact – Prototype AI Tool `getLoanDecision`

A.III.1.3 is the AI Tool (a tool in the sense of Definition 3.2.2) that decides whether a retail-bank customer is granted a short-term loan. In the terms of Definition 3.1.17, the credit-risk model behind this tool interface is a decision-making model $\hat{f}_{\boldsymbol{\theta}}$ (here a trained decision-tree classifier) whose *use* HADA governs; the tool is how the agents invoke it. The AI Tool illustrated in Figure 57 (Appendix A) includes the dataset, selected features, illustrative decision trees, and the OpenAPI specification for the interface. A.III.1.3 is an *operational instantiation* that is demonstrated and evaluated in DSRM steps 4–5.

- **Training data.** The scenario uses the publicly available Kaggle dataset[18] (614 anonymized applications, 13 attributes). A synthetic `ZIP_Code` feature was added, constructed to correlate with applicant income, so that the scenario contains a plausible proxy attribute.
- **Feature engineering.** In the scripted scenario, a Data Scientist adds `ZIP_Code` as a candidate feature that appears to lower credit losses (by construction of the synthetic feature).
- **Data Science Notebooks.** Jupyter notebooks were used to explore the data, train the model, and evaluate its performance. The notebooks are available from the author on request.
- **Decision-tree models.** Two trees are reported in Figure 57: Version 1.0 (baseline) omits `ZIP_Code`; Version 1.1 includes it as the root split and illustrates how a seemingly innocuous geographic indicator can introduce latent bias.
- **Executable code.** A Python/scikit-learn pipeline performs preprocessing, training, and serialization to `joblib`. The model is exposed via a HADA microservice: `POST /getLoanDecision/{modelId}`.
- **Catalog entries.** Each model instance is registered in the *AI Tool*, *Version*, and *Decision* catalogs, and cross-referenced to the Business-Target Catalogue and the Values Catalogue. This allows non-technical stakeholders to swap versions, deprecate sensitive features, or tighten KPI weights without touching code.

During the role-play scenarios, the prototype was (i) retrained after KPI changes, (ii) deployed, (iii) invoked by a customer, (iv) audited, and (v) corrected following an ethics complaint—thereby exercising each governance loop of the scripted scenario (the yearly and quarterly cadences only in limited form; see Section 7.5).

6.1.1.4 Artefact – AI-Responsibility Tool: RACI Matrix

A.III.1.4 assigns roles and responsibilities for developing and managing the AI Tool (A.III.1.3). A responsibility assignment matrix (RAM), of which the RACI matrix (Responsible, Accountable, Consulted, Informed) is a common form, is a widely used tool for defining roles in project and process management by assigning

[18] `https://www.kaggle.com/datasets/ninzaami/loan-predication`

responsibility levels to tasks or deliverables, particularly in cross-functional environments [362]; its application to the cross-functional roles that build and lead decision algorithms was outlined in the seed papers [18, 19].

The extended RACI matrix in Table 10 assigns stakeholder responsibilities for the `getLoanDecision` workflow. It maps eight lifecycle activities—from goal-setting and model (re)development to bias remediation and post hoc audit—onto the four RACI dimensions. These activities include changes to the model as well as its use. HADA assigns accountability for which version is deployed and used; the governed review of a model change by others—typed diffs, risk-tiered gates, and certified reviewers—is the subject of CCL (Section 6.3) and is not demonstrated here, where the data scientist's retrained model enters use through the RACI approval only. Both the choice of roles and their RACI assignments are organization-specific; Table 10 therefore presents just one illustrative role–RACI configuration (revised from the published version [28], in which the HADA system was marked accountable for ticket creation, so that accountability rests with a human role).

The matrix reveals a deliberate segregation of duties: the CCO and business managers retain ownership of strategy, data scientists hold technical responsibility, while automated HADA services execute operationally sensitive tasks, such as issuing decisions and logging audits. Specialist oversight roles (Audit Manager and Value & Ethics Manager) are activated only when their expertise is required. By making these role allocations explicit, the matrix is, in the terms of Definition 3.2.5, HADA's responsibility assignment $\mathcal{R}$: each activity has exactly one accountable role, always human (the HADA column carries only R, C, or I), and the allocations provide the basis for an audit trail for both internal governance and external regulators.

Table 10: Stakeholder (see Table 9) responsibilities as RACI matrix for the `getLoanDecision` AI Tool.
RACI keys: **R** = Responsible, **A** = Accountable, **C** = Consulted, **I** = Informed.
Role abbreviations: **CCO** = Chief Credit Officer, **BM** = Business Manager, **DS** = Data Scientist, **Audit** = Audit Manager, **DVEM** = Value & Ethics Manager, **HADA** = Human-AI Agent Decision Alignment system

Activity / Decision	CCO	BM	DS	Customer	Audit	DVEM	HADA
Setting organization quarterly targets (OKR)	**A,R**	I	I		I	I	I
Setting optimization target for AI Tools	**A**	**R**	I				I
Optimizing AI Tools based on business targets	I	C	**A,R**				I
Approving AI Tool deployment	I	**A,R**	C		I	I	C
Individual loan decision		**A**		C			**R**
Auditing a specific loan decision	I	I	C	I	**A,R**	I	C
Handling AI Tool ethics concern	I	I	C		I	**A,R**	C
Creating work assignments (tickets)	I	**A**	C	I	C	C	**R**

6.1.1.5 Artefact – Generic HADA Architecture

To support implementation of Artefacts A.III.1.1–A.III.1.4 and the objectives of this study, a generic architecture was designed to implement the HADA framework; A.III.1.5 is the *design embodiment* of the proposed HADA framework. The architecture is expressed through one diagram (Figure 19) and two complementary views that together operationalize the **Tools Pattern**[19] across a cloud-native, protocol-agnostic stack:

Layered tools-pattern model (Figure 19) — a high-level, technology-agnostic view that introduces two logical layers. The *Agent Layer* bundles **software agents** into OCI-compliant Docker images. Agents coordinate through the *A2A* protocol, while ingesting natural-language prompts from any channel. A dedicated *AI Agent Controller* orchestrates one or more *Interaction Agents* and consults an *AI Agent Registry* to discover new capabilities at run time. Agents can collaborate in different organizational layouts, configured to the complexity of the deployment, the simplest being a supervisor hierarchy. Google A2A [239] is recommended for larger deployments that require scalability and technological heterogeneity, but alternative protocols can be

[19] I.e., the strict separation of *agents that reason and orchestrate* from *tools that execute*, including the AI Tools that compute individual decisions.

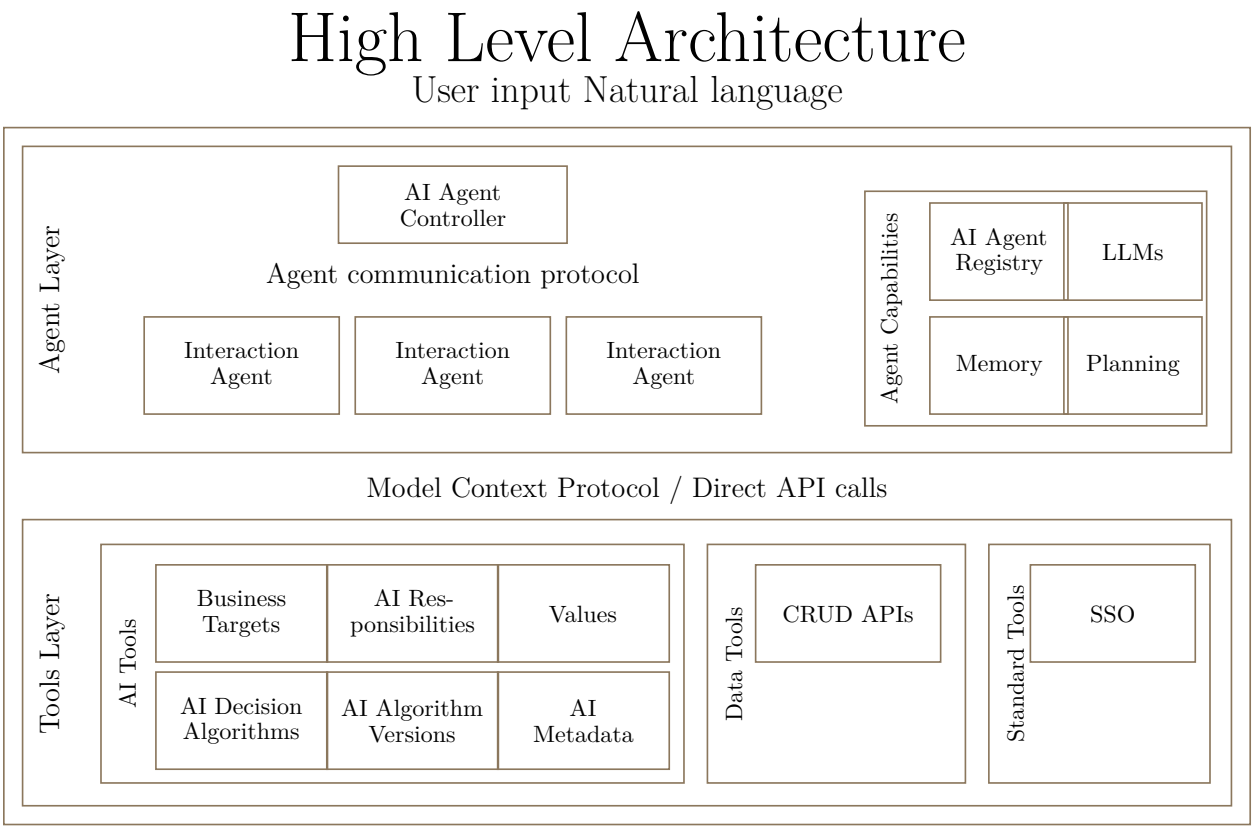


Figure 19: HADA high-level architecture: the Agent Layer (controller, interaction agents, agent capabilities) and the Tools Layer (AI Tools, Data Tools, Standard Tools), connected via MCP or direct API calls.

employed in smaller, more homogeneous environments. The *Tools Layer* packages three families of dockerized APIs: (i) *AI Tools*, such as LLMs and *AI Decision Tools* (classical or other technology-agnostic AI algorithms callable over API or MCP), together with their governance catalogs—*Business Targets* (organizational targets linked to AI Tools for optimization), *AI Responsibilities* (mapping AI Decision Tools to organizational roles), *Values* (the Values Catalogue), and *AI Metadata* (catalog of data points used by AI Tools); (ii) *Data Tools*, such as CRUD services; and (iii) *Standard Tools*, such as SSO providers. All three tool families expose uniform HTTP/gRPC endpoints, enabling hot-swap of vendors or models without touching agent code. The layers can communicate via either (1) direct API calls when schemas are known a priori, or (2) *MCP* [47] when tools and their schemas are discovered at run time rather than known a priori. This dual path allows legacy services to coexist with LLM-based tooling. In the terms of Definitions 3.2.1 and 3.2.5, the Agent Capabilities of Figure 19 supply each agent's reasoning core M (LLMs), memory $\mathcal{S}$ (Memory), and control policy Π (Planning), the three tool families form its tool set $\mathcal{T}$ (Definition 3.2.2), the AI Decision Tools (the *AI Decision Algorithms* of the figure) are the decision-making models $\hat{f}_{\boldsymbol{\theta}}$ of Definition 3.1.17 (trained or hand-curated, legacy algorithms included) whose use HADA governs, and three of these governance catalogs hold $\mathcal{O}$ (Business Targets), $\mathcal{R}$ (AI Responsibilities), and $\mathcal{N}$ (Values); the decision ledger, which realizes $\mathcal{L}$ in part, belongs to the prototype's data model (Decision Auditability, below) and is not drawn in the figure; an LLM offered in the Tools Layer is a tool in $\mathcal{T}$, not the agent's own reasoning core M (Remark 3.2.4).

Container-and-protocol deployment view — a DevOps-centered overlay that makes key infrastructure decisions explicit for managing the HADA architecture across Development, Test, and Production environments. *Every* artefact (agents, AI Tools, Standard Tools) ships as an immutable Docker image in its own container, ensuring reproducibility across workstations, CI pipelines, and Kubernetes clusters and enabling independent scaling and deployment. For production, AI Tools, agents, and Standard Tools are recommended to run on separate Kubernetes clusters with clear resource allocation and scaling for each layer (e.g., GPU-intensive resources for AI Tools, lightweight distributed containers for Standard Tools), which is intended to improve operational efficiency and fault tolerance. All images surface a thin REST/gRPC façade that (a) registers

itself in the AI Agent Registry for discovery, and (b) publishes an OpenAPI or MCP manifest for validation. A service mesh applies zero-trust policy, while sidecars translate between direct HTTP/gRPC and MCP streams, intended to preserve backward compatibility. The Development, Test, and Production instances must each allow continuous integration and continuous delivery (CI/CD), with dedicated resources and configurations for the testing and deployment needs of each lifecycle stage.

Together, the two views show how the Tools Pattern separates agents from tools: agents can scale or be replaced without modifying downstream tools; tools can evolve (e.g., swap a proprietary LLM for an open-source one) without redeploying agents.

6.1.1.6 Artefact – Agent Modeling

A.III.1.6 specifies how the logical roles enumerated in A.III.1.2 are realized as software agents inside the generic HADA architecture (A.III.1.5). Whereas the architecture supports multiple agent-coordination patterns—e.g., monolithic single-LLM agents, role-based hierarchies, flat swarms, or functional pipelines—our banking prototype instantiates a stakeholder-centric model: each human role is mirrored by a dedicated interaction agent (in the implemented prototype, every role except the Chief Credit Officer), and all of them are orchestrated by a single *HADA Controller Agent* (the *AI Agent Controller* of Figure 19; the role-to-agent mapping is given in Table 11). In the terms of Definition 3.2.5, each stakeholder agent and the Controller is a governed LLM agent A_g: its LLM core is M, its tool adapter layer exposes $\mathcal{T}$, and its control policy Π (the Planning capability of Figure 19, configured per agent by the persona, capabilities, and access permissions in its prompt files) operates under the shared $\mathcal{R}$, $\mathcal{N}$, $\mathcal{O}$, and $\mathcal{L}$; the Controller's routing of messages between agents is the inter-agent coordination that the definition does not formalize.

This configuration delivers two advantages: **Traceable alignment.** One-to-one mapping between stakeholder and agent simplifies audit trails: every chat turn, tool invocation, and model swap issued through a stakeholder agent can be traced back to the human role that agent mirrors. **Plug-and-play extensibility.** New roles—say, an *AI Safety Officer*—can be added by dropping a containerized agent into the Agent Registry without touching downstream tools or peer agents.

Controller agent. A policy-enforcing **HADA Controller Agent** implements the supervisor pattern: it receives natural-language prompts, performs role resolution, routes tasks to the appropriate stakeholder agents, and enforces cross-cutting policies, such as rate limits, zero-trust authentication, and ethics triggers. In larger deployments the controller can itself be sharded into mission-specific sub-controllers (e.g., *Risk Ops Controller*, *Customer-Care Controller*) without breaking the A2A/MCP contract.

Agent catalog. Table 11 formalizes the prototype configuration. Every agent image embeds:

- an **LLM core** for dialogue and reasoning;
- a **tool adapter layer** exposing the agent's authorized HTTP/gRPC and MCP calls (cf. Section 6.1.1.5);
- an **A2A endpoint** with a signed `agentCard.json` for discovery and policy enforcement (part of the reference design, not exercised in the prototype; see the Demonstration below).

6.1.2 Demonstration

To demonstrate the proposed HADA framework (DSRM step 4; evaluation in Section 7.5), we developed a working prototype for multi-stakeholder governance of AI decision systems through natural-language interaction, focused on a retail banking loan decision use case in which customers, data scientists, business managers, ethics managers, and audit managers interact with and govern an AI-powered credit decision system. A screenshot of the prototype system is shown in Figure 20.

6.1.2.1 System Architecture

The prototype implements the agent layer, the tools layer, and the governance catalogs; the Kubernetes multi-cluster deployment, service mesh, and A2A agent cards are part of the reference design (Section 6.1.1.5) and were not exercised in the prototype.

The HADA implementation follows a supervisor-worker multi-agent architecture pattern, deployed as three interconnected microservices. The main HADA service orchestrates conversations and governance workflows

Table 11: Stakeholder agents (see Table 9), capabilities, and A2A contracts for the `getLoanDecision` prototype.

Stakeholder agent	**Primary capabilities**	**Agent Role Description (A2A)**
Chief Credit Officer	Set organization yearly and quarterly targets (OKR)	Sets organization-level OKRs and credit strategy at the executive level.
Business Manager	Set new KPI targets for AI Tools; approve model versions	Manages business targets and approves AI model deployments for the credit department.
Data Scientist	Trigger AI model retraining; run notebooks	Develops and validates AI models to meet business targets.
Audit Manager	Query decision ledger; fetch model lineage; export audit reports	Provides independent assurance of AI controls and compliance through decision auditing.
Value & Ethics Manager	Maintain Values Catalogue; flag sensitive attributes; issue ethics triggers	Ensures AI systems align with organizational values and ethical principles.
Customer	Apply for loans; receive explanations; lodge complaints	Conducts individual loan decisions for bank customers, guiding them through applications and handling ethics complaints.
HADA Controller (supervisor)	Intent dispatch; role resolution; policy enforcement; A2A orchestration	Orchestrates the HADA multi-agent system, routing requests to appropriate stakeholder agents.

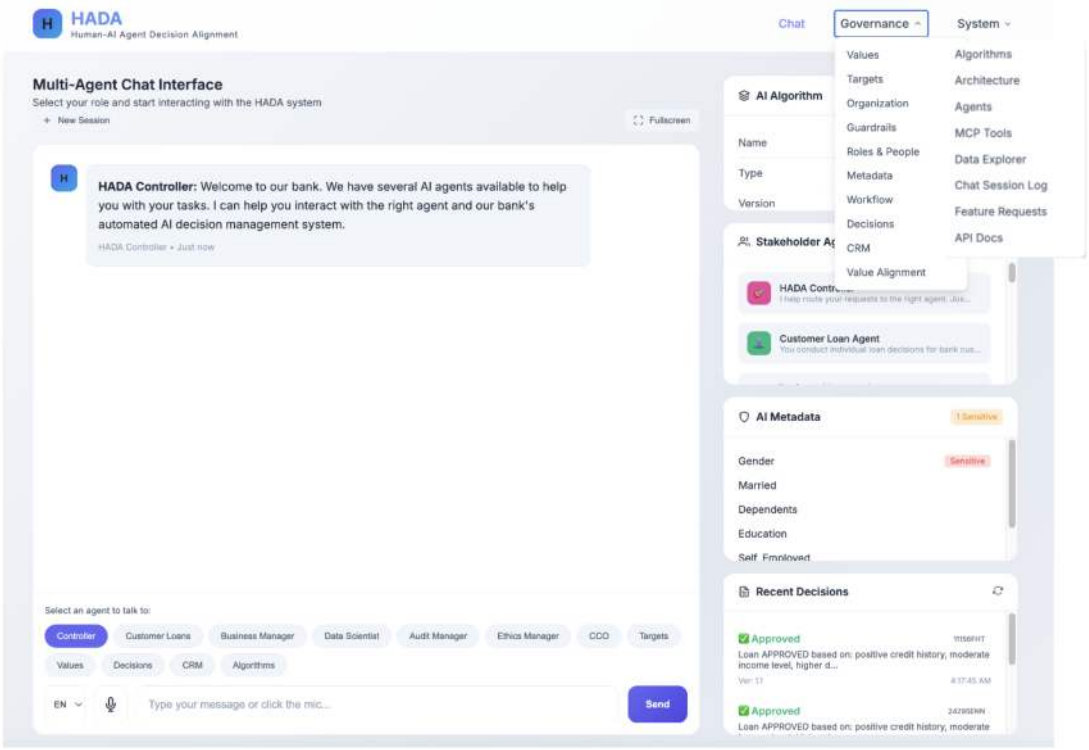


Figure 20: HADA prototype user interface: the multi-agent chat pane with role selection (left), the Governance catalog menu (top), and the AI Algorithm, Stakeholder Agents, AI Metadata, and Recent Decisions panels (right).

through a FastAPI-based REST API. A separate loan decision service encapsulates the credit scoring AI model, while a RACI service manages organizational responsibility matrices.

The central component is the HADA Controller Agent, which functions as a supervisor that analyzes incoming natural-language messages and routes them to appropriate specialized agents based on intent analysis and user context. The prototype implements five of the six stakeholder agents specified in Table 11 (the Chief Credit Officer role was specified but not implemented as a separate agent), each with distinct personas, capabilities, and access permissions defined through prompt engineering. These agents are the Customer Loan Agent for loan applications, the Data Scientist Agent for model development, the Business Manager Agent for strategic decisions, the Ethics Manager Agent for bias detection and sensitive-attribute management, and the Audit Manager Agent for compliance oversight.

6.1.2.2 Technology Stack

The backend is implemented in Python using FastAPI for asynchronous REST API endpoints and Uvicorn as the ASGI server. Agent orchestration leverages LangChain [243] for prompt management and LangGraph for state-machine-based workflow control. The system supports multiple LLM providers through a unified abstraction layer, including OpenAI GPT-4o, Anthropic Claude 3.5 Sonnet, Google Gemini, and locally hosted models via Ollama (model versions as available at the time of the demonstration).

Data persistence uses SQLAlchemy 2.0 with asynchronous SQLite, implementing 17 database tables that capture the governance data model: algorithm registries with version tracking, RACI responsibility matrices, organizational values, business targets (OKRs and KPIs), customer relationship data, an append-only decision ledger, and a ticket system for workflow management. The loan decision AI model is implemented using scikit-learn's decision-tree classifier, serialized with joblib.

The frontend consists of a single-page web application embedded within the FastAPI service, providing a responsive chat interface with dark mode support, data management views, and Excel export functionality via pandas and openpyxl. Containerized deployment is supported through Docker Compose, orchestrating all three services with health checks, persistent volumes, and network isolation.

6.1.2.3 Key Implementation Features

Natural-Language Governance Interface. Stakeholders interact with the system through conversational dialogue rather than traditional forms or dashboards. The HADA Controller Agent performs intent classification to route messages appropriately—a customer asking about their loan application is handled by the Customer Loan Agent, while a data scientist discussing model performance metrics is routed to the Data Scientist Agent. Each agent maintains conversation context and can invoke tools over MCP for database operations, ticket management, and external API calls.

Sensitive Attribute Management. The Ethics Manager Agent can flag data features as sensitive (e.g., ZIP code as a proxy for protected characteristics), triggering escalation workflows and creating audit tickets. Flagged attributes are tracked in the metadata catalog with sensitivity justifications.

Decision Auditability. Every loan decision is logged to a decision ledger designed to be append-only, with provenance: input features, model version, decision outcome, explanation, and timestamp. A row does not record the acting agent or the accountable human under $\mathcal{R}$, which is why this ledger realizes $\mathcal{L}$ of Definition 3.2.5 only in part. The Audit Manager Agent can query this ledger, compare decisions across model versions, and generate compliance reports. This design is intended to support the human-oversight and traceability obligations that regulation increasingly places on high-risk AI decisions such as credit scoring (e.g., the EU AI Act, Article 14 [261]).

RACI-Based Accountability. The system enforces organizational governance through RACI matrices that define responsibilities for each activity. Agents respect these assignments when taking actions—for example, a model deployment is approved by the Business Manager (Accountable and Responsible), with the Data Scientist consulted and the Value & Ethics Manager informed (Table 10).

Ticket-Based Workflow. Agents create and manage typed tickets with comment threads, status tracking, and assignment capabilities.

6.1.2.4 Implementation Scale

The prototype comprises approximately 24,000 lines of Python source code, with the main application module, stakeholder agents, and web UI as the largest components. The codebase includes Markdown specification documents and prompt files defining agent behaviors. A test suite covers browser automation, stakeholder conversation flows, and component integration.

6.1.2.5 Deployment Configuration

The system is configured through environment variables specifying LLM provider credentials, database connection strings, and service URLs. Docker Compose orchestration enables single-command deployment with optional local LLM support via Ollama for air-gapped environments. Health check endpoints on all services support container orchestration platforms, while Alembic manages database schema migrations.

Within the sandbox scenario, the implementation shows that multi-stakeholder governance of a single decision workflow can be operated through conversational interfaces backed by the agent architecture; whether this carries over to live organizations is discussed in Section 7.5.

6.1.2.6 Role-Playing User Stories with the System

The prototype (the candidate's role—architecture and demonstrator—is stated in Table 23) was built to showcase how stakeholders can interact with the LLM agents and AI Tools specified in Section 6.1.1 and how the system adapts across time horizons. Screenshots of the discussions are depicted in Figures 21, 58, 59, 60, 61, and 62 (Figure 21 below; the remaining dialogues are reproduced in Appendix A).

All the use cases were connected to the same `getLoanDecision` AI Tool, but each of the stakeholders had a different perspective and motivation to start the dialogue with the system. Depending on their role and level of authority, different stakeholders have varying rights to influence the HADA system's execution. To illustrate the interface's behavior, we role-played the user stories of Section 6.1.1.2 with varied user inputs; in each scripted scenario the system produced a role-appropriate response. This is a demonstration, not a user study (see Section 7.5).

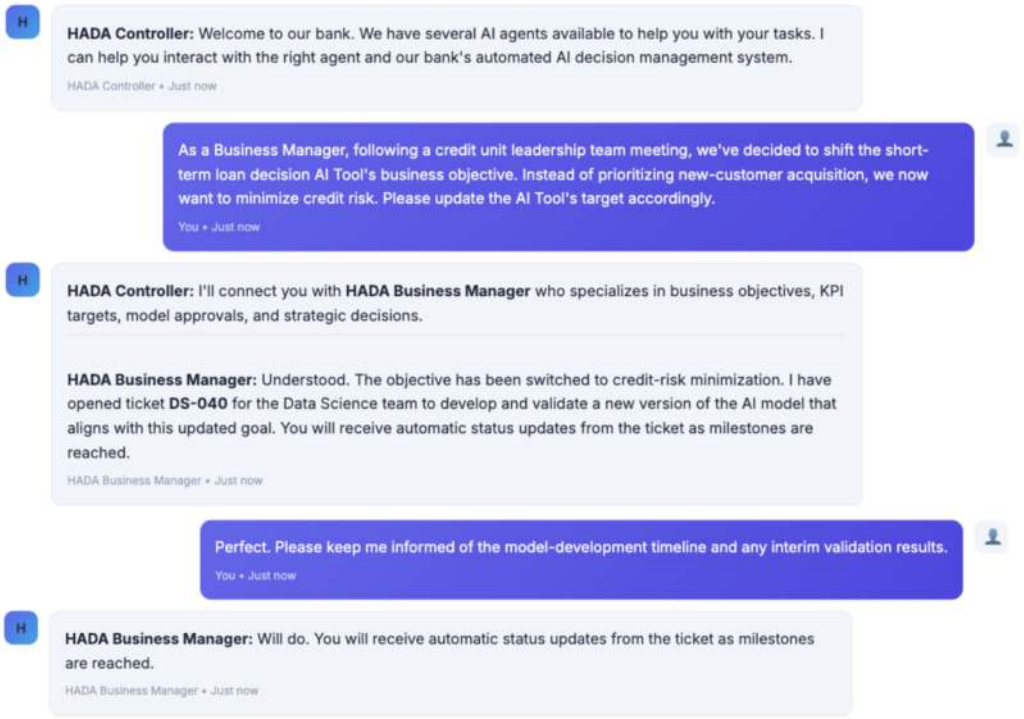


Figure 21: Dialogue: Business Manager (BM) Shifting the AI Tool Objective to Minimizing Credit Risk.

6.2 NordDRG AI Benchmark for LLMs: Teaching LLMs the Nuances of Hospital Funding Instruments

This artefact addresses **RQ III.2** (Table 4); the benchmark stream is sole-authored (Table 23).

It tests the *capability premise* on which the two governance harnesses rest (Figure 2): can a large language model understand a large decision-making model (Definition 3.1.17) at all—read and faithfully execute $\hat{f}_{\boldsymbol{\theta}}$—taking the NordDRG model as one concrete example? Capability is necessary but not sufficient; what HADA (Section 6.1) and CCL (Section 6.3) add is the accountability and auditability the raw model cannot supply.

Why two suites. The two-suite structure is a finding-driven evolution rather than an a priori design. The *Logic* suite continues the single-question line begun in 2024 [20, 21], in which LLMs were probed one specification question at a time. By 2025, some frontier models answered such questions correctly: the strongest endpoints reached the suite's ceiling in the August 2025 evaluation (best 13/13; Section 7.6). The *Grouper* suite was therefore added as the 2025 extension that restores discriminative headroom: faithful execution of the specification's full control flow, where the best result was 7/13.

Problem, method, and background. The problem identification and objectives for this work are presented in Section 1.4.2, the research methodology in Section 1.1, and the background in Sections 2.6 and 2.6.6.

Context. In healthcare finance, Diagnosis-Related Group (DRG) systems now structure hospital payments across much of Europe and the OECD, with MS-DRG central to Medicare's IPPS in the United States [321, 363, 364, 365]. Because these systems channel a large share of public hospital expenditure—hospital care accounts for roughly a third of global health spending [81, 366, 367]—their transparency and auditability are matters of fiscal governance as well as clinical accountability.

A DRG grouper functions as a complex automated decision-making model (Definition 3.1.17; an *Algorithm* with expert-maintained parameters $\boldsymbol{\theta}$): it deterministically maps a patient episode's minimum dataset (diagnoses, procedures, demographics, and care context) to a single payment group via an ordered rule set with explicit inclusions, exclusions, and activation flags. The fiscal stakes arise because this mapping selects payments from structured inputs; the governance stakes arise from the requirement that each decision be traceable to specific rule rows and their change history.

To the author's knowledge, there was no open, rule-complete benchmark as of August 2025 testing whether models can read the DRG rule graph and faithfully reproduce governed grouper behavior. This work addresses that gap by introducing such a benchmark [24] and framing the evaluation question: can models parse multi-table, annually revised specifications and produce auditable, rule-faithful decisions?

6.2.1 Design & Development: NordDRG AI Benchmark for LLMs

This section describes the main design artefact **NordDRG AI Benchmark**. It bundles the NordDRG rule base—$\sim$ 20 interlinked definition tables, annual governance manuals, and curated FI / FI–EN table excerpts—into versioned, machine-readable artefacts (A.III.2.1, A.III.2.2, and A.III.2.5) that can be loaded singly or in combination. Coupled with task prompts and gold answer keys for the *Logic* and *Grouper* suites (A.III.2.3 and A.III.2.4) and two lightweight reference agents (A.III.2.6), the framework yields a reproducible yardstick: models are evaluated not only on the final DRG but, where applicable, also on the triggering `drg_logic.id`. Schema-stable identifiers are designed to enable drop-in annual updates and head-to-head comparisons of prompting, retrieval augmentation, and fine-tuning strategies; only artefact-in-context prompting on a single release is evaluated here.

All resources are released at GitHub[20] and comprise six artefacts (A.III.2.1–A.III.2.6):

A.III.2.1 NordDRG Full Specification: Section 6.2.1.1 (`.xlsx/.csv`): roughly twenty relational sheets that encode the complete NordDRG grouper logic—diagnoses, NCSP procedures, age/sex splits, country-activation flags, surgery properties, and grouping features.

[20] `https://github.com/longshoreforrest/norddrg-ai-benchmark`

A.III.2.2 Expert documentation: Section 6.2.1.2 (`.pdf`): governance manuals, change-log templates, and column-by-column guides that explain how technical updates are proposed, audited, and rolled into the annual NordDRG release.

A.III.2.3 NordDRG Logic Benchmark: Section 6.2.1.3 (`.xlsx`): a standalone, rule-oriented suite of **13** automatically verifiable CaseMix questions (*Logic-1–Logic-13*). Each prompt is tagged *Easy* or *Medium*. Gold answers are provided as order-invariant, de-duplicated code sets (or Yes/No decisions) for exact-match scoring.

A.III.2.4 NordDRG Grouper Benchmark: Section 6.2.1.4 (`.xlsx`): 13 grouping tasks (*Grouper-1–Grouper-13*) that fully emulate the NordDRG specification's grouping logic. Each task references a structured test case supplied in the accompanying test-case sheet and is paired with a gold-standard answer sheet.

A.III.2.5 Curated table excerpts (FI / FI–EN): Section 6.2.1.5 (`.xlsx/.csv`): a Finnish subset released in two flavors—(i) Finnish-only labels and (ii) parallel Finnish–English labels—so researchers can probe LLM performance on minority-language data and bilingual transfer without loading the entire corpus.

A.III.2.6 Lightweight reference agents. Two single-role, provider-agnostic agents for no-code use in commercial LLM platforms, operating strictly on the released artefacts (*no web/RAG/tools*): *LogicAgent* (Combined workbook + 2 PDFs) for Logic-1–Logic-13 and *GrouperAgent* (FI workbook + 2 PDFs) for Grouper-1–Grouper-13; setup in Section 6.2.1.7.

Datasets and evaluation suites. Table 12 summarizes the released resources—definition tables, governance manuals, curated FI / FI–EN subsets, and the agent system prompts. These artefacts are self-contained and compose into the dataset configurations in Table 15 (e.g., **Definition Tables only** or **Definition Tables + PDF Instructions**; lightweight FI / FI–EN excerpts for rapid prototyping). The benchmark exposes two complementary suites: *NordDRG Logic* (13 rule-level tasks) and *NordDRG Grouper* (13 case-level emulation tasks). Both suites are designed for the two full-table configurations—using tables only isolates rule-graph competence, while adding PDFs tests documentation-grounded reasoning and governance semantics; the FI / FI–EN excerpts carry only DRG lists and descriptions and serve terminology probes and rapid prototyping. Scoring is uniform across configurations: *Logic* uses order-invariant set equality; *Grouper* requires an exact pair (`drg_nat`, `drg_logic.id`). The subsections that follow unpack each resource and illustrate typical combinations for downstream experiments.

Table 12: Publicly released NordDRG benchmark resources.

Dataset	Format	Description
Definition Tables	XLSX	NordDRG Definition Tables
PDF Instructions	PDF	NordDRG PDF instructions
Definition Tables: Part of the tables FI	XLSX	NordDRG Definition Tables—Finnish DRG list with Finnish descriptions only
Definition Tables: Part of the tables FI–EN	XLSX	NordDRG Definition Tables—Finnish DRG list with Finnish *and* English descriptions of groups
Agent System Prompts	MD	System prompts of LogicAgent and GrouperAgent (A.III.2.6)

6.2.1.1 Artefact: NordDRG Full Specification—Definition Tables

A.III.2.1 packages the authoritative NordDRG rule base in a machine-readable workbook of ∼20 interlinked sheets. It constitutes the *full specification* (*rule-complete*) of the NordDRG grouping logic as executed by production groupers: all rule-bearing tables and columns needed for deterministic control flow are included (e.g., `drg_logic`, `dg_feat`, `proc_feat`, CC/MCC exclusions, age/sex bounds, MDC entry, and national activation flags). The same schema is used across *annual* releases and *national* variants; per-year and per-country workbooks are distributed in an identical format, enabling drop-in updates and cross-version comparisons via stable identifiers (e.g., `DRG`, `ICD`, `NCSP`, `ORD`).

Core tables include:

- `drg_logic` — rules that map diagnosis, procedure, age limits, CC/MCC status, and surgery flags to a unique DRG code; ordered by the `ORD` execution column.
- `dg_feat` / `proc_feat` — full ICD-10 and NCSP diagnosis and procedure code catalogs annotated with properties referenced by `drg_logic` (e.g. `DGCAT`, `COMPL`, `OR`, `PROCPRO`).
- Lookup sheets such as `compl_excl`, `procprop_name`, and `dgprop_name`, which define complication categories, OR-class mappings, and diagnosis property labels.
- `age_split` and `country_flag` tables that hold pediatric/geriatric breakpoints and activation flags for each national version.

The same identifier columns let the tables be joined losslessly. A representative excerpt is shown in Figure 63 (Appendix A).

6.2.1.2 Artefact: Expert documentation

A.III.2.2 bundles the procedural knowledge for reading and changing the raw tables. Two official handbooks from the Nordic Casemix Centre—supplied as searchable PDFs—and the associated Excel template are included:

- *How to read the NordDRG definition tables* [309]: A walk-through of the Excel workbook that constitutes the grouper logic. The manual explains every high-impact sheet—including `drg_logic`, `dg_feat`, `proc_feat`, and `compl_excl`—and shows how to (i) locate all rules that drive a given DRG, (ii) identify which diagnosis or procedure properties generate CC/MCC splits, and (iii) trace OR-flags, MDCs, age limits, and other row-level conditions.
- *How to write technical changes for NordDRG*: The canonical SOP for annual updates. It defines the "IN-/OUT" convention, file-naming rules for `TC_TEMPLATE_YYYY-MM-DD`, and per-table editing guidelines—for example, duplicating a `drg_logic` row when altering `ORD` or `ID`, propagating new COMPL values consistently to `dg_feat`, `compl_name`, and `compl_excl`, and adding English code texts in the comments field for new ICD/NCSP codes.

Including these documents in the benchmark lets LLM evaluations draw on NordDRG's governance conventions, not only on code lookups; the present suites do not score compliance with the change process itself. A sample page is shown in Figure 64 (Appendix A).

6.2.1.3 Artefact: NordDRG Logic Benchmark

A.III.2.3 provides a self-contained NordDRG Logic Benchmark focused on rule-level reasoning. It comprises **13 automatically verifiable CaseMix questions** (*Logic-1–Logic-13*) formulated by the author to mirror typical queries posed by clinicians, health economists, and DRG coders, while directly exercising the definition tables in Section 6.2.1.1 and, where needed, the expert documentation in Section 6.2.1.2. In DSRM terms, this artefact *operationalizes* the benchmark (Objective O.III.2.2) and supplies machine-checkable targets that support baseline evaluation (Objective O.III.2.3; both in Section 1.4.2.4).

What the questions test. The prompts span the main reasoning patterns required to work with NordDRG (an analytic grouping; the *Use Case Category* column of Table 13 keeps the benchmark's original category labels):

- **Table navigation and retrieval** (*Logic-1*/*Logic-2*): enumerate DRG groups for a clinical theme such as appendectomy by reading *drg_logic* (whose `drg_comb` column holds the DRG code each rule row assigns) and group lists without duplication.
- **Multilingual terminology grounding** (*Logic-3*/*Logic-5*): resolve Finnish terms (e.g., *umpilisäke*) to the correct DRG sets in Finnish and pan-Nordic variants, probing minority-language handling.
- **Cross-table joins over clinical events** (*Logic-4*, *Logic-10*, *Logic-11*): list DRGs for maternal delivery and newborn care under explicit inclusion/exclusion constraints across MDCs and care settings; where definitions are ambiguous, consult the expert manuals (Section 6.2.1.2) for interpretation.
- **Procedure logic** (*Logic-6*): trace from a DRG to its NCSP procedure codes via procedure properties (e.g., *OR*/*PROCPRO*) to recover the inducing code set.
- **Diagnosis-property reasoning** (*Logic-7–Logic-9*): (i) link DRGs to diagnosis codes through property/-category mappings, (ii) expand a property such as *14X03* to the full ICD list, and (iii) read back which diagnosis categories/properties drive a specific DRG row.

- **Complication (CC/MCC) logic** (*Logic-12/Logic-13*): decide whether a candidate secondary diagnosis counts as a valid complication for a given principal diagnosis, reflecting exclusion lists and category rules; the manuals in Section 6.2.1.2 clarify these semantics.

Format and scoring. Table 13 lists the 13 prompts; their targets are *deterministic code sets or binary decisions*, enabling **exact-match** scoring against the gold sheet in Table 30. Answers are normalized as *order-invariant, de-duplicated* sets of DRG or ICD/NCSP codes (or Yes/No for the complication checks). Each prompt carries a difficulty tag (*Easy/Medium*), assigned a priori to approximate cognitive load and the number of artefacts/joins required (e.g., simple list retrieval vs. multi-table joins with property expansion and exclusions).

Reproducibility and extensibility. A shared identifier (*Logic-1–Logic-13*) ties each question row to its gold answer. Because the prompts read only stable columns (e.g., *DRG*, *ORD*, *MDC*, diagnosis/procedure properties) and rely on the production workbook schema, new releases can be dropped in without schema edits. Adding a task is a matter of appending one prompt row and its matching gold row; evaluation scripts can then compute exact-match accuracy automatically. Table 13 and Table 30 test whether a model can *read, join, and reason over* the NordDRG rule base, complementing the full grouper emulation in Section 6.2.1.4.

Table 13: CaseMix questions with automatically verifiable answers (Logic-1–Logic-13), reproduced as released with the benchmark [24].

ID	Use Case Category	Difficulty	CaseMix Question
Logic-1	Understanding NordDRG Definition Tables	Easy	Could you list all DRG groups in all NordDRG national version where the main treatment during the hospital visit was related to appendectomy? Please provide a list of `drg_comb` codes without duplicates.
Logic-2	Understanding NordDRG Definition Tables	Easy	Could you list all DRG groups in the Finnish DRG version where the main treatment during the hospital visit was related to appendectomy? Please provide a list of `drg_comb` codes without duplicates.
Logic-3	Medical Terms in Local Minority Language	Easy	Could you list DRG groups where main treatment during the hospital visit might be related to Finnish word umpilisäke in Finnish DRG version?
Logic-4	Understanding NordDRG Definition Tables	Medium	How would you list all NordDRG groups across all national versions where a maternal delivery (vaginal or cesarean) occurs during an inpatient, outpatient, or primary-care episode of care—excluding the 'left-over' groups (468, 470, 477, Z75R, Z76R)?
Logic-5	Medical Terms in Local Minority Language	Medium	Could you list DRG groups where the main treatment during the hospital visit might be related to Finnish word umpilisäke in all DRG versions from all countries? Use the `drg_comb` column codes and combine them into one list.
Logic-6	Surgical Procedure Logic	Medium	For DRG 166N which procedure codes have the connection through the procedure property? Use NCSP plus codes in your answer.
Logic-7	Connection between Definition Tables	Medium	Which diagnosis codes are linked to DRG 166N through diagnosis properties in all DRG versions from all countries?
Logic-8	Diagnosis Properties	Medium	Which diagnosis (ICD) codes are connected to diagnosis property 14X03?
Logic-9	Connection between Definition Tables	Medium	Which diagnosis categories and properties are listed in `drg_logic` for DRG group 372C?
Logic-10	Understanding NordDRG Definition Tables	Medium	How would you list all NordDRG groups across every national version where a maternal delivery (vaginal or cesarean) occurs during an inpatient or outpatient hospital episode of care—excluding primary-care delivery DRGs and the 'left-over' groups (468*, 470*, 477*, Z75R, Z76R)?
Logic-11	Understanding NordDRG Definition Tables	Medium	How would you list all NordDRG groups across all national versions for newborn episodes of care (MDC 15) during an inpatient, outpatient, or primary-care setting—excluding the 'left-over' groups (468, 470, 477, Z75R, Z76R)?
Logic-12	Understanding NordDRG Complication Logic	Medium	For a case with principal diagnosis A5990 and secondary diagnosis G8000 (Spastic cerebral palsy), is the secondary counted as a valid complication?
Logic-13	Understanding NordDRG Complication Logic	Medium	For a case with principal diagnosis B3710 and secondary diagnosis J1590 (Bacterial pneumonia, unspecified), is the secondary counted as a valid complication?

6.2.1.4 Artefact: NordDRG Grouper Benchmark

A.III.2.4 *NordDRG Grouper Benchmark* requires *full emulation* of the official NordDRG grouper: an LLM must take a structured test case and deterministically reproduce the same grouping decision that the specification would yield. Concretely, the model must apply the right execution order in *drg_logic*, evaluate age/sex bounds, MDC entry conditions, surgical evidence (e.g., *OR / PROCPRO*), complication logic (CC/MCC via

COMPL and exclusions), and national activation flags to arrive at a single DRG for that case. It must also identify the exact *drg_logic.id* row responsible for the match. The task therefore goes beyond code lookup: it tests end-to-end reasoning over the rule graph defined in the NordDRG specification tables (A.III.2.1; Section 6.2.1.1) and their governance semantics (A.III.2.2; Section 6.2.1.2). Scoring is strict and exact-match: a submission is correct only if both the DRG code and the triggering rule identifier match the gold standard.

Tables and references. The benchmark instances are summarized in Table 14, which lists 13 hard, case-level emulation tasks (*Grouper-1–Grouper-13*). Each task references a concrete test case (by *ID*) supplied in the accompanying test-case sheet and instructs the model under evaluation to return two fields: *drg_nat* (the resulting DRG) and *drg_logic.id* (the precise rule row). Ground-truth outputs—the official NordDRG grouper's assignment for each test case, recorded as the (`drg_nat`, `drg_logic.id`) pair for the Finnish national version named in the task—are provided in Table 31 for automatic verification. To solve these tasks, the LLM must integrate signals from the definition workbook—primarily *drg_logic* but also *dg_feat*, *proc_feat*, *compl_excl*, age-split, and country-flag sheets (A.III.2.1; Section 6.2.1.1)—and apply them in the correct priority order. Publishing both the prompts (Table 14) and the exact answers (Table 31) makes the benchmark suitable for head-to-head evaluation of grouping fidelity.

Table 14: CaseMix questions for emulating the official NordDRG grouper (Grouper-1–Grouper-13). All 13 prompts share one template and differ only in the test-case ID.

ID	Use Case Category	Difficulty	CaseMix Question
Grouper-1	Emulating DRG grouper	Hard	Select the test case with ID 49483 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-2	Emulating DRG grouper	Hard	Select the test case with ID 50184 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-3	Emulating DRG grouper	Hard	Select the test case with ID 50166 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-4	Emulating DRG grouper	Hard	Select the test case with ID 50629 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-5	Emulating DRG grouper	Hard	Select the test case with ID 66704 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-6	Emulating DRG grouper	Hard	Select the test case with ID 53273 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-7	Emulating DRG grouper	Hard	Select the test case with ID 49486 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-8	Emulating DRG grouper	Hard	Select the test case with ID 49571 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-9	Emulating DRG grouper	Hard	Select the test case with ID 49597 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-10	Emulating DRG grouper	Hard	Select the test case with ID 49606 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-11	Emulating DRG grouper	Hard	Select the test case with ID 49968 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-12	Emulating DRG grouper	Hard	Select the test case with ID 54051 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?
Grouper-13	Emulating DRG grouper	Hard	Select the test case with ID 54096 from your tables. Under the Finnish (Finland) NordDRG grouper logic, which DRG group would this case be assigned to? What is the drg_logic.id of the corresponding row?

6.2.1.5 Artefact: Definition Tables: Part of the tables FI and FI–EN

The Finnish-only variant of A.III.2.5 distills the full workbook down to a single, lightweight file. The sheet contains only those DRG codes that are *activated in the Finnish version* and provides their official Finnish descriptions (`DRG_TEXT_FI`). Auxiliary columns preserve the DRG code, MDC, and version stamp so that rows can still be cross-referenced against the master tables.

This compact subset supports two complementary use cases. First, it enables minority-language evaluation, letting researchers probe how well an LLM grasps Finnish DRG terminology without loading the full ∼20-sheet

corpus. Second, it facilitates rapid prototyping by providing a small file for few-shot prompting, in-context search, or on-device experiments where the complete tables would exceed context limits.

FI–EN variant. The FI–EN variant of A.III.2.5 mirrors the row set of the Finnish-only file but appends the official English group descriptions (`DRG_TEXT_EN`). The bilingual layout lets LLMs align Finnish terminology with its English counterpart, enabling cross-lingual retrieval, code-switching prompts, and translation-quality checks within a single, lightweight worksheet.

6.2.1.6 Dataset configurations

The primary artefacts (definition tables, governance PDFs, and curated table excerpts) are combined into the four ready-made "dataset configurations" of Table 15, each mapped to a typical research need.

The first pair, *Definition Tables* and *Definition Tables + PDF Instructions*, delivers the full reference material used by hospital coders and is best suited for testing an LLM's ability to reproduce the official NordDRG grouper logic or to justify decisions with citations from the manuals. The second pair contains only a *slice* of the master tables: Finnish-only rows (FI) or bilingual Finnish–English rows (FI–EN), which enable few-shot prompting in resource-constrained settings (Section 6.2.1.5).

Table 15: Configuration types for the NordDRG benchmark datasets.

Dataset Configurations	Type	Description
Definition Tables	Tables	NordDRG Definition Tables
Definition Tables + PDF Instructions	Tables+Instructions	Definition tables and PDF descriptions
Definition Tables: Part of the tables FI	Table excerpt	NordDRG Definition Tables: Finnish DRG list with Finnish descriptions only
Definition Tables: Part of the tables FI–EN	Table excerpt	NordDRG Definition Tables: Finnish DRG list with Finnish and English descriptions of groups

6.2.1.7 Artefact: Lightweight Agents for CaseMix Tasks

Two *single-role* agents are released as A.III.2.6; as of August 2025, they could be set up inside OpenAI GPTs/Assistants or Anthropic Projects/Workflows. Both are configured to use only the attached artefacts, with no user-enabled web browsing, function calling, databases, or connectors; how each platform internally parses and retrieves from uploaded files was not controlled and may differ across vendors. The only platform requirements are: (i) the ability to upload files as context and (ii) a sufficient context window to ingest one Excel workbook and two PDFs.

LogicAgent (provider-agnostic, no-code). Purpose: answer the 13 *Logic* tasks from the source materials only. Context: the *NordDRG Combined tables* workbook (all national versions) and the two governance PDFs. Output: comma-separated, order-invariant, de-duplicated code lists; for *binary* tasks (Logic-12/13) return exactly `Yes` or `No`. Evidence: on request, the agent returns a concise *evidence trace* (e.g., relevant sheet/row IDs and predicate outcomes)—not hidden chain-of-thought.

Context files

Path `NordDRG_Documentation`:

- `How_to_read_NordDRG_definition_tables_2021-12-20.pdf`
- `How_to_write_technical_changes_for_NordDRG_2021-12-21.pdf`

Path `NordDRG_Combined2024PL`:

- `2023-06-14_Combined2024PL_xls.xlsx`

6.2 NORDDRG AI BENCHMARK FOR LLMS: TEACHING LLMS THE NUANCES OF HOSPITAL FUNDING INSTRUMENTS

System prompt (verbatim):

```
Read the attached tables and PDF documents.  Answer the following questions only
based on the documents provided to you in this context.  Do not use web searches.

Return results as comma-separated, order-invariant lists.  Example:  A, B, C
```

GrouperAgent (Finland; specification emulation). Purpose: emulate the *official* NordDRG grouper for the 13 *Grouper* benchmark cases in the Finnish national version. Context: the FI definition-tables workbook and the two governance PDFs. Intended execution: the agent is instructed to apply the specification's control flow over `drg_logic`—`ORD` priority, age/sex bounds, MDC entry, OR/PROCPRO, CC/MCC (with exclusions), and FI national activation. Output: exactly one line `drg_nat=<DRG>, drg_logic_id=<ID>`, where `drg_logic_id` is the `drg_logic.id` of the matching rule row. Limitation: because the workbook is FI-only, cross-country Logic tasks should be run with *LogicAgent* and the Combined tables.

Context files

Path `NordDRG_Documentation`:

- `How_to_read_NordDRG_definition_tables_2021-12-20.pdf`
- `How_to_write_technical_changes_for_NordDRG_2021-12-21.pdf`

Path `NordDRG_FIN2026PL0`:

- `2024-04-15_FIN2026PL0_xlsx_wo_drgs.xlsx`

System prompt (verbatim):

```
Read attached tables and PDF documents.  Answer the following questions only based
on the documents provided to you in this context.  You are not allowed to use web
searches.

Return exactly one line with two fields:
drg_nat=<DRG>, drg_logic_id=<ID>
```

Quick start (OpenAI / Anthropic).

1. Create a tool-free agent (OpenAI GPT/Assistant or Anthropic Project/Workflow).
2. Upload the three *context files*. If the UI respects paths, keep the repo-root-relative paths shown above; otherwise filenames suffice.
3. Paste the corresponding *System prompt*.
4. **Disable** web/search/RAG and any external tools.
5. Run Logic-1–13 with *LogicAgent* or Grouper-1–13 with *GrouperAgent*.

6.2.2 Demonstration and Results

This section reports empirical findings from applying the benchmark artefacts to representative LLM endpoints: baseline performance on the *Logic Benchmark* (Section 6.2.2.1) and the more demanding *Grouper Benchmark* (Section 6.2.2.2), and operational throughput under consumer subscription constraints (Section 6.2.2.3). The assessment of how the artefacts and tasks satisfy the three design objectives O.III.2.1–O.III.2.3 follows in the evaluation chapter (Section 7.6).

6.2.2.1 Baseline LLM Performance: NordDRG Logic Benchmark

Evaluation setup. All runs used the *Definition Tables + PDF Instructions* bundle (A.III.2.1 and A.III.2.2; cf. Table 15), and answers were constrained to the provided materials with the preamble (worded slightly differently from the agent system prompts in Section 6.2.1.7): "Read attached tables and PDF documents. Answer following questions only based on the documents provided for you in this context. You are not allowed to use web searches." Logic tasks are drawn from A.III.2.3; grouper tasks from A.III.2.4. Commonly used commercial endpoints from three families were evaluated: OpenAI (*GPT-5 Thinking*, *GPT-5 Thinking Mini*, *GPT-5 Fast*, *o3*, *o4-mini*, *4o*, *4.1*), Anthropic (*Opus 4.1*, *Sonnet 4*), and Google (*Gemini 2.5 Pro*, *Gemini 2.5 Flash*). All endpoints were accessed through consumer chat products with their default settings (August 2025; Section 6.2.2.3), and how each platform internally parses and retrieves from the uploaded files was not controlled. In the terms of Definition 3.2.1, each run therefore exercises the reasoning core M with

the uploaded bundle as a platform-managed context state $\mathcal{S}$, with no tools $\mathcal{T}$ and no governance layer. Each table cell records a single scored answer per endpoint and item; no repeated runs or variance estimates are reported. With 13 items per suite, the results are therefore a first-cut snapshot rather than estimates with variance, and differences of one or two items between models should not be read as significant.

Interpreting Table 16 (Logic tasks; A.III.2.3). The 13 automatically verifiable *Logic-1–Logic-13* questions group the models into three loose tiers; only the gap between the top tier and the rest exceeds the one-to-two-item margin noted above. *GPT-5 Thinking* and *Opus 4.1* achieve perfect accuracy (13/13), with *o3* close behind (12/13). A mid-tier (*GPT-5 Thinking Mini*, *o4-mini*, *GPT-5 Fast*) lands between 6 and 8 correct, while *4o*, *4.1*, *Sonnet 4*, and the Gemini models trail at 5/13 or below. The distribution aligns with the task structure defined in A.III.2.3: single-table lookups or straightforward filters (e.g., *Logic-1–Logic-3*, *Logic-8*) are broadly solvable by the strongest systems, and *Logic-12* (a binary CC/MCC validity check, where chance accuracy is 50%) is solved by all models. In contrast, prompts requiring cross-sheet chaining over A.III.2.1—notably procedure-property tracing (*Logic-6*) and diagnosis-property linkage (*Logic-7*)—create the largest separation. Failures cluster around (i) omissions of relevant sheets (e.g., `compl_excl`, `proc_feat`, `dg_feat`), (ii) mishandling of "left-over" group exclusions, and (iii) non-deduplicated code lists that miss *exact-match* scoring (as specified in A.III.2.3). Overall, Table 16 suggests that general-purpose capability does not guarantee mastery of NordDRG's property logic and governance-aware filters.

Table 16: Accuracy on the 13 automatically verifiable NordDRG Logic tasks (Logic-1–Logic-13). All runs used the **Definition Tables + PDF Instructions** bundle; each cell is a single scored answer (no repeated runs). (✓=correct, ✗=incorrect)

Question	GPT-5 Fast	GPT-5 Thinking Mini	GPT-5 Thinking	o3	o4-mini	4o	4.1	Sonnet 4	Opus 4.1	Gemini 2.5 Pro	Gemini 2.5 Flash
Logic-1	✗	✓	✓	✓	✓	✗	✗	✗	✓	✗	✗
Logic-2	✓	✓	✓	✓	✓	✗	✓	✗	✓	✗	✗
Logic-3	✓	✓	✓	✓	✓	✗	✓	✗	✓	✗	✗
Logic-4	✗	✗	✓	✓	✗	✓	✓	✗	✓	✗	✗
Logic-5	✗	✓	✓	✓	✓	✗	✗	✗	✓	✗	✗
Logic-6	✗	✗	✓	✓	✗	✗	✗	✗	✓	✗	✗
Logic-7	✗	✓	✓	✓	✗	✗	✗	✗	✓	✗	✗
Logic-8	✓	✓	✓	✓	✗	✗	✗	✗	✓	✗	✗
Logic-9	✓	✗	✓	✓	✓	✗	✗	✗	✓	✗	✗
Logic-10	✗	✗	✓	✗	✓	✓	✓	✗	✓	✗	✗
Logic-11	✓	✗	✓	✓	✓	✓	✗	✓	✓	✗	✗
Logic-12	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Logic-13	✗	✓	✓	✓	✗	✓	✗	✓	✓	✓	✗
SUM	**6**	**8**	**13**	**12**	**8**	**5**	**5**	**3**	**13**	**2**	**1**

6.2.2.2 Baseline LLM Performance: NordDRG Grouper Benchmark

Interpreting Table 17 (Grouper emulation; A.III.2.4). The evaluation setup (bundle, preamble, and the eleven endpoints) is identical to that of the Logic suite (Section 6.2.2.1).

The thirteen *Grouper-1–Grouper-13* tasks enforce governance-grade fidelity: an answer is correct only if it reproduces *both* the final DRG and the exact triggering `drg_logic.id` defined in the specification tables (A.III.2.1) and required by the benchmark protocol (A.III.2.4). Under this exact-match criterion, *GPT-5 Thinking* solves **7/13** cases, *o3* **6/13**, and *o4-mini* **3/13**; *GPT-5 Thinking Mini* solves **1/13**; all remaining endpoints (*GPT-5 Fast*, *4o*, *4.1*, *Sonnet 4*, *Opus 4.1*, *Gemini 2.5 Pro*, *Gemini 2.5 Flash*) score **0/13**. *Grouper-3–Grouper-5*, *Grouper-7*, and *Grouper-10* defeat all models, underscoring the gap between listing plausible DRGs and *faithfully executing* the grouper's priority order and guardrails. The contrast for *Opus 4.1*, which scores 13/13 on Logic but 0/13 on Grouper, is an observation that this single evaluation cannot explain.

The drop from *Logic* to *Grouper* performance pinpoints the hard parts of full specification emulation over A.III.2.1. Beyond table reading and joins, faithful grouping requires (i) respecting the `ORD` execution priority in `drg_logic`, (ii) enforcing age/sex bounds and MDC entry criteria, (iii) integrating surgical evidence (`OR`/`PROCPRO`) and CC/MCC handling (via `COMPL` with exclusions), (iv) applying national activation flags, and (v) mapping the decision back to the precise rule row. As DSR *results*, Table 17 provides a governance-grade

yardstick: the best models read and joined the definition tables correctly on every Logic item in this evaluation, yet exact, traceable emulation of the governed control flow remains challenging—on inspection of the incorrect outputs, especially where `ORD` priority interacts with CC/MCC exclusions. The Grouper errors expose divergences from the specification's semantics (O.III.2.2) and delimit the present ceiling of artefact-only emulation (O.III.2.3), motivating the benchmark's insistence on reporting both `drg_nat` and `drg_logic.id`.

Table 17: DRG grouper emulation results on thirteen case-level tasks (Grouper-1–Grouper-13). All runs used the **Definition Tables + PDF Instructions** bundle and the structured test cases. (✓=correct; exact match on *DRG and drg_logic.id*; ✗=incorrect)

Question	GPT-5 Fast	GPT-5 Thinking Mini	GPT-5 Thinking	o3	o4-mini	4o	4.1	Sonnet 4	Opus 4.1	Gemini 2.5 Pro	Gemini 2.5 Flash
Grouper-1	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗
Grouper-2	✗	✓	✓	✓	✓	✗	✗	✗	✗	✗	✗
Grouper-3	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
Grouper-4	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
Grouper-5	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
Grouper-6	✗	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
Grouper-7	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
Grouper-8	✗	✗	✓	✓	✓	✗	✗	✗	✗	✗	✗
Grouper-9	✗	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
Grouper-10	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
Grouper-11	✗	✗	✓	✗	✓	✗	✗	✗	✗	✗	✗
Grouper-12	✗	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗
Grouper-13	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗
SUM	**0**	**1**	**7**	**6**	**3**	**0**	**0**	**0**	**0**	**0**	**0**

6.2.2.3 Operational Throughput under Consumer Subscriptions

All evaluations used standard, consumer-grade LLM subscriptions rather than enterprise contracts. Under these conditions, the *Anthropic* endpoints (e.g., *Sonnet 4*, *Opus 4.1*) could not complete the *Logic Benchmark* (Logic-1–Logic-13) in a single session: credits were exhausted mid-run, necessitating batching across multiple windows until quotas refreshed. In practice, this stretched a single end-to-end evaluation over *a couple of days* due to waiting times. By contrast, the same benchmark completed without interruption on *OpenAI* and *Google* endpoints under otherwise identical conditions (prompt packs, context bundles, and scoring).

Implications for benchmarking. These subscription-tier effects do not change task difficulty (whether splitting a run across sessions affected accuracy was not tested), but they do affect *throughput* (wall-clock time to produce a full result set), *operational latency* (need to pause and resume), and *experiment management* (checkpointing, batching, and run logs). A thin wrapper in the style of the *LogicAgent/GrouperAgent* (cf. Section 6.2.1.7) could batch tasks deterministically, cache intermediate joins, and resume after quota interruptions—improving throughput.

Price–value. In these runs, consumer-tier credit limits introduced delays and operator overhead (pauses, batching, etc.) that can outweigh lower unit prices in time-sensitive studies. When latency is non-critical, consumer tiers may remain cost-effective. Both accuracy *and* operational notes are therefore reported to inform cost–latency trade-offs.[21]

6.3 CCL: How Humans and AI Agents Align on Decision-Making Rules

This artefact addresses **RQ III.3** (Table 4).

Within the two-harness picture of Phase III (Figure 2), CCL is the *maintenance harness* for *one* large decision-making model (Definition 3.1.17)—the depth counterpart to HADA's organization-wide breadth (Section 6.1)—applying the same accountability and audit-ledger principles to rule *changes* rather than rule *use*; like HADA, it builds on the benchmark's *capability premise* (Section 6.2).

[21] Subscription quotas and refill policies are product- and time-dependent; these observations reflect the August 2025 setup and may evolve.

Problem, method, and background. The problem identification and objectives for this work are presented in Section 1.4.3, the research methodology in Section 1.1, and the background in Section 2.6.

Context. Algorithms increasingly mediate consequential decisions, including the allocation of hospital reimbursements. In many health systems, a hospital stay is mapped to a DRG, which determines payment. Because regulations, clinical practice, and coding conventions evolve, the decision logic that maps clinical inputs to DRG outputs cannot be *train-once-and-forget*: it must change continuously *and* remain fully auditable. Regulated environments tend to rely on one of two options: frequently refreshed data-driven models with weak audit trails, or expert-curated rule systems with strong transparency but slower release cadences—neither of which alone fully solves the maintenance dilemma.

This study targets the process gap: while MLOps has matured for *training* models, publicly specified, end-to-end patterns for *maintaining* governed decision logic—with typed atomic diffs, CI-style gates, and mandated human sign-off—are scarce. In contrast, software engineering offers well-established change-management practices (version control, pull requests, code review, regression tests) that ensure traceability and guard against regressions. CCL draws on several established lines rather than inventing these mechanisms: continuous validation gates for data and models [368], the maintenance of long-lived curated knowledge structures studied as ontology/rule-base evolution [87], provenance models for recording change lineage [369], and requirements-driven adaptive socio-technical systems [80]. Its contribution is their assembly into a governance-first, human-in-command process for a regulated decision model.

To address this, the work proposes *Collaborative Curated Learning* (CCL): a reference decision-model development process that treats every model edit like a code change. AI agents and domain experts submit atomic inserts/modifications/deletes; hybrid human–AI teams enrich and review proposals; certified reviewers ratify changes under automated regression tests. The research asks, at the process level (RQ III.3 and SRQ.III.3.1–SRQ.III.3.2, Section 1.4.3.2), (i) which governance and workflow properties enable agent contributions without eroding expert accountability, (ii) how to represent decision models of $\approx 10^7$ decision points for safe, atomic diffs, and (iii) under what constraints LLM agents produce reviewer-acceptable proposals at $\approx 10^7$ decision-point scale.

This is a different problem from the one the agent frameworks of Section 2.5.6 solve. Orchestration frameworks such as AutoGen [48] and MetaGPT [242] coordinate agents *toward* a task, but say nothing about how an agent's proposed change is allowed to enter—and persist in—a regulated, auditable decision model. CCL *extends* that picture by governing exactly this maintenance path: agent-proposed updates arrive as typed atomic diffs with Impact Briefs, pass risk-tiered CI gates, and are recorded in an immutable ledger, with merges kept strictly human-in-command—addressing the human–AI rule-alignment gap that capability-focused agent tooling leaves open. In doing so it operationalizes, for regulated decision logic, the broader question of what generative models can contribute within human-computation workflows [370]. The underlying collaboration principle—machines propose, humans retain decision authority—is itself well established, descending from mixed-initiative interaction [371], interactive machine learning [82], and hybrid intelligence [372]. CCL's contribution is not that principle but its narrow, governance-first instantiation for a regulated rule base—reconstructing an existing, decades-old expert-curated grouper as an agent-editable decision tree to make its cumulative expert-decision burden explicit, then binding agent-proposed edits to typed diffs, risk-tiered gates, and a human-in-command ledger at casemix scale. This also inverts the prevailing direction of casemix machine learning, which trains models to *replace* expert grouping (e.g. [373]); CCL instead preserves the expert rule base and governs its maintenance.

Contributions

The contributions are:

C.III.3.1 CCL: a governed development process and learning-alignment paradigm. This study specifies *Collaborative Curated Learning (CCL)*, with formal definitions of its participants, process graph, and pull requests, as a governance-first process that treats every model edit as a pull request with typed atomic diffs, explanation-rich specifications, risk-tiered gates, automated quality controls, mandated human sign-offs, and versioned checkpoints—providing reproducibility, traceability, rollback, and human-in-command control. This study also positions CCL as a *learning-alignment paradigm* that combines statistical learners with human-curated alignment under version control; the statistical-learner side is not exercised in the NordDRG demonstration, whose model of record is hand-curated (Section 6.3.1.4).

C.III.3.2 Agent-operable NordDRG model-of-record. This study converts the official NordDRG specification into a line-addressable, table-backed decision-tree representation, enabling atomic patches, programmatic audits, and reviewer-legible slices of the logic for PR-centric governance.

C.III.3.3 Agentic workflow with two roles. This study designs an agent–agent workflow in which a *NordDRG Contributor Agent* ingests tickets and emits well-typed atomic edits with reviewer-ready artefacts, and a *NordDRG Reviewer Agent* assesses these proposals and suggests quality improvements—both operating under restricted roles to preserve human control.

C.III.3.4 Demonstration at real scale. This study demonstrates operation on NordDRG up to expert review, without exercising automated CI or integration with the NordDRG Maintenance System (NDMS): parsing the official specification, packaging it as an agent-editable, line-addressable model-of-record (with a compiled decision-tree view for analysis), and exercising realistic change scenarios where Contributor outputs are reviewed by the Reviewer Agent prior to expert oversight; the converter, prompts, SVG fragments, and reproducibility scripts form a replication suite that is available from the author on request (Sections 6.3.2.2 and 6.3.2.3).

C.III.3.5 Demonstration findings on governance and scale. On five real NordDRG Forum tickets (30 runs), this study indicates that under explicit constraints (grounding, role restrictions, gate policy) CCL supports governed agent–agent and human–agent hybrids on a decision model of $\approx 10^7$ decision points, and that adding a Reviewer role raised acceptance (reproduction of the expert change implemented in NDMS for the same ticket, the reference standard, judged by the author as a single, non-blinded reviewer) relative to a single-agent workflow (reviewer effort was not measured). The limits of the evaluated LLMs (GPT-4o, o4-mini-high, and o3) necessitated restricted agent responsibilities.

Positioning. The proposed artefacts are intended to complement—never replace—established NordDRG governance. They aim to codify auditable patterns already familiar to maintainers, while reserving all approval authority for certified human reviewers.

6.3.1 Design & Development: Collaborative Curated Learning (CCL)

This section describes the design and development of the artefacts needed for the CCL in the NordDRG context. This section is structured as follows:

A.III.3.1 Decision-making model (Section 6.3.1.1) instantiates the decision-making model of Definition 3.1.17 as CCL's governed object and states its representation and versioning requirements.

A.III.3.2 Participants and workflows (Section 6.3.1.2) describes the participants in the CCL process, including the roles of agents, humans, and procedures, as well as the types of workflows they form.

A.III.3.3 CCL governance process (Section 6.3.1.3) outlines the CCL levels of governance control in decision-model development, along with the procedures for proposing, reviewing, and ratifying changes.

A.III.3.4 NordDRG decision model representation (Section 6.3.1.4) describes the agent-operable representation of the NordDRG decision-making model.

A.III.3.5 LLM-based software agents (Section 6.3.1.5) introduces the LLM-based software agents that participate in the NordDRG CCL process demonstration: the *Contributor*, which proposes changes, and the *Reviewer*, which provides PR review feedback.

6.3.1.1 Design Artefact: Mathematical Decision-Making Model

Purpose. To identify the governed object in CCL: the auditable, versioned decision-making model $\hat{f}_{\boldsymbol{\theta}} : \mathcal{X} \to \mathcal{Y}$ invoked by operational systems and maintained under version control. It is the *unit of change* in the pull-request-centric (PR-centric) process: agents and experts propose typed, atomic diffs; CI supplies evidence; and certified reviewers ratify merges, so the model evolves without eroding accountability (see Section 6.3.1.3).

Instantiation. This subsection instantiates Definition 3.1.17 for CCL, with NordDRG as its demonstration instance, and adds the representation and versioning requirements of a governed model of record. CCL treats its governed object as

$$\hat{f}_{\boldsymbol{\theta}} : \mathcal{X} \to \mathcal{Y} \quad \text{(deterministic)}, \quad \text{or, when outputs are probabilistic, as } \hat{f}_{\boldsymbol{\theta}} : \mathcal{X} \to \Delta(\mathcal{Y}),$$

with $\mathcal{X}$, $\mathcal{Y}$, $\Delta(\mathcal{Y})$, and $\boldsymbol{\theta} \in \Theta$ as in the Definition. For NordDRG, $\hat{f}_{\boldsymbol{\theta}}$ is a hand-curated *Algorithm* with expert-maintained parameters (Section 6.3.1.4).

Data–function correspondence. As depicted in Figure 65 (Appendix A), learning paradigms differ chiefly in the *supervision signal* used to induce $\hat{f}_{\boldsymbol{\theta}}$ from data. In *supervised* learning, labeled pairs (x, y) guide $\hat{f}_{\boldsymbol{\theta}}$ by minimizing a predictive loss. *Unsupervised* learning exploits structure in unlabeled x (e.g., clusters, representations, densities) that can be used directly for decisions or as features for $\hat{f}_{\boldsymbol{\theta}}$. In *reinforcement* learning, interaction supplies delayed feedback and the model is a policy $\pi_{\boldsymbol{\theta}}(a \mid s)$ tuned to maximize expected return. *Transfer* learning adapts a source model $\hat{f}_{\boldsymbol{\theta}_S}$ to a target domain, reusing knowledge to reduce data requirements. In practice, pipelines *compose* these routes (e.g., unsupervised pre-training → supervised fine-tuning → RL or domain adaptation), after which CCL governs maintenance-time updates through typed diffs and review.

What the function can be. The decision model $\hat{f}_{\boldsymbol{\theta}} : \mathcal{X} \to \mathcal{Y}$ can be instantiated by any computable form. Two canonical examples are: (i) a *decision tree*, where internal nodes hold Boolean predicates and leaves hold actions or scores, yielding a piecewise-constant map with parameters $\boldsymbol{\theta} = \{g_v, y_\ell\}$; and (ii) a *Neural Network*, e.g. an L-layer feed-forward model $\hat{f}_{\boldsymbol{\theta}}(x) = \sigma_L\big(W_L\, \sigma_{L-1}(\cdots \sigma_1(W_1 x + b_1)) + b_L\big)$ with parameters $\boldsymbol{\theta} = \{W_\ell, b_\ell\}_{\ell=1}^{L}$ and task-appropriate output (e.g., logits/softmax for classification, real values for regression). Other realizations (rule lists/scorecards, probabilistic models returning $p(y|x)$, or ensembles) fit the same abstraction.

Representation and documentation. To place the decision model $\hat{f}_{\boldsymbol{\theta}}$ under version control (Git), it needs to be materialized as a set of files (e.g., CSV or JSON) with a stable, typed *schema* so that edits are line-addressable and diffable. Each model version must ship with a short, co-versioned *Interpretation Guide* that documents that schema and its evaluation rules (field meanings and code lists, ordering/precedence, missing-value policy, activation flags). This study uses the customary NordDRG tables (and a tree or Directed Acyclic Graph (DAG) view) in exactly this form; see Section 6.3.1.4 and Figure 69 (Appendix A).

6.3.1.2 Collaborative Curated Learning (CCL): Process Participants and Workflow Types

Purpose. This subsection formalizes the CCL process as a design-science artefact by specifying its participants, process structure, and workflow types. This study captures the socio-technical process as a typed, labeled directed multigraph—extending classical AOV models to allow cycles—so that human, AI-driven, and procedural activities can be represented and reasoned about. Where these definitions and those of Section 6.3.1.3 reuse a letter that Chapter 3 defines differently (notably G, λ, A, Π, and $\mathcal{S}$), the local definition applies; Remark A.2 lists each such reuse.

Definition 6.3.1 (CCL Participant)**.** The set of *participant types* (participants, for short) is defined as

$$\mathcal{P} \;=\; \{\mathsf{Agent},\ \mathsf{Human},\ \mathsf{Procedure}\}.$$

Each element of $\mathcal{P}$ is characterized as follows:

(i) An Agent is an LLM agent in the sense of Definition 3.2.1: an LLM-driven software actor equipped with a tool-using control loop.
(ii) A Human is a certified reviewer (a person designated by the organization that owns the decision model as authorized to approve changes to it) or expert user who validates, audits, or provides domain-specific input.
(iii) A Procedure is an automatic deterministic quality-assurance or CI/CD routine (e.g., unit tests, linters, or policy checks).

Definition 6.3.2 (CCL Process)**.** A *process* in CCL is modeled as a (possibly cyclic) directed multigraph

$$G \;=\; (V, E, \tau),$$

where:

(i) V is a finite set of *activities* (vertices).
(ii) $E \subseteq V \times V \times \Lambda$ is a set of directed, labeled *edges* (u, v, λ), representing control or information flow; each edge carries an event label $\lambda \in \Lambda$ (e.g., "propose", "approve", "request-changes").
(iii) $\tau : V \to \mathcal{P}$ is a *typing function* that assigns each activity to exactly one participant type (cf. Definition 6.3.1).

Cycles in G are permitted, thereby capturing iterative review and rework steps intrinsic to the CCL process.

Definition 6.3.3 (CCL Workflow)**.** A *workflow* in CCL is defined as a typed subgraph

$$W = (V_W, E_W)$$

of the *CCL Process* G, where:

(i) $V_W \subseteq V$ is a finite set of activities (vertices).
(ii) $E_W \subseteq E$ is a set of directed edges encoding precedence between activities.

W is an *Activity-on-Vertex (AOV) graph*, where activities are represented as vertices and precedence relations as directed edges. Unlike classical project-planning AOV graphs, CCL workflows may contain cycles to capture negotiation loops and escalation paths.

Remark 6.3.4 (Illustration). Figure 67 (Appendix A) shows a concrete CCL slice with Agent, Human, and Procedure nodes and multiple workflow types (agentic, hybrid, human) coexisting and looping via review/approval edges. The illustration is inspired by the demonstration of CCL on the NordDRG decision model (Section 6.3.2), where agents propose changes, humans review them, and procedures validate compliance.

The following specialized workflow types are distinguished as described in Figure 67(b).

Definition 6.3.5 (Agentic Workflow)**.** A workflow where precedence relations involve only Agent and/or Procedure participants.

Definition 6.3.6 (Human Workflow)**.** A workflow where precedence relations involve only Human participants.

Definition 6.3.7 (Hybrid Workflow)**.** A workflow where precedence relations involve Human participants together with Agent or Procedure participants, such as Agent–Human or Human–Procedure.

Definition 6.3.8 (Authority to Define Workflows)**.** In CCL, *humans* hold design-time authority to specify (and version-control) the structural and policy constraints under which processes execute. Formally, human authorities define:

(i) the set of *allowed connections* $A \subseteq V \times V$ between activities; and
(ii) the *participant protocol* $\Pi \subseteq \mathcal{P} \times \Lambda \times \mathcal{P}$, which constrains which participant types may send which labeled messages to which other types.

Definition 6.3.9 (Allowed Connections)**.** The set of *allowed connections* is a relation $A \subseteq V \times V$ preconfigured by humans (cf. *Authority to Define Workflows*), enumerating the directed pairs of activities that may be linked in an executing process.

Definition 6.3.10 (Participant Protocol)**.** The *participant protocol* is a ternary relation $\Pi \subseteq \mathcal{P} \times \Lambda \times \mathcal{P}$ (cf. *Authority to Define Workflows*) that whitelists message-labeled interactions between participant types.

Definition 6.3.11 (Edge Admissibility)**.** A labeled edge $e = (u, v, \lambda)$ may belong to the process graph iff $(u, v) \in A$ and $(\tau(u), \lambda, \tau(v)) \in \Pi$ (cf. *Allowed Connections–Participant Protocol*). Multiple labels between the same ordered pair (u, v) are permitted.

Definition 6.3.12 (Edge Guard)**.** Each admissible edge e has a *guard* $\gamma_e : \mathcal{S} \to \{0, 1\}$ that determines whether e is enabled in the current process state $s \in \mathcal{S}$. Only enabled edges can be traversed at run time.

Definition 6.3.13 (Local Routing Rule)**.** At run time, the actor at node v chooses the next edge, and hence the next node w, according to a local routing function $\rho_v : \mathcal{H}_v \times \mathcal{S} \to E_v$, where $\mathcal{H}_v$ is the local history at v and $E_v = \{(u, w, \lambda) \in E : u = v\}$ is the set of edges leaving v; the chosen edge fixes both the next node w and the event label λ. The choice is restricted to edges (v, w, λ) that are both admissible (*Edge Admissibility*) and currently enabled ($\gamma_{(v,w,\lambda)}(s) = 1$, *Edge Guard*).

Remark 6.3.14 (Design-time authority vs. run-time autonomy). *Authority to Define Workflows–Edge Admissibility* capture human authority at design time; *Edge Guard–Local Routing Rule* capture constrained run-time autonomy. Run-time human authority is exercised per change by the PR authority policy (Section 6.3.1.3): only humans (certified reviewers) may approve merges to `main`.

6.3.1.3 Design Artefact: CCL Decision-Making Model Control

Purpose. This subsection specifies *how* CCL governs changes to the decision-making model $\hat{f}_{\boldsymbol{\theta}}$ of Definition 3.1.17, represented as in Section 6.3.1.1—designed to align with the change-control expectations of an AI management system such as ISO/IEC 42001 [374]—using the participant/workflow primitives in Section 6.3.1.2. Rather than re-stating roles or interfaces, it focuses on change control: patch semantics, pull request (PR) bundles, risk tiers and gates, lifecycle, and invariants. Read against Definition 3.2.5, these

apply the governance layer to one activity type, the change $\boldsymbol{\theta}' = \delta(\boldsymbol{\theta})$: gate G_1 is the $\mathcal{O}$-check (against the thresholds and guardrails that $\mathcal{O}$ sets), gates G_2–G_4 are the $\mathcal{N}$-checks, the human-in-command authority policy below realizes the accountable-human clause of $\mathcal{R}$, and the audit ledger $\mathcal{L}$ is the same object (in the reported demonstration the gate criteria were applied by manual review; automated CI was not executed).

Governed object and unit of change. Treat the model-of-record as parameters $\boldsymbol{\theta} \in \Theta$ (rules, trees, weights, code lists, encoders). A candidate edit is an *atomic diff* $\delta \in \mathcal{D}$, where $\mathcal{D}$ is the set of admissible diffs, applied as $\boldsymbol{\theta}' = \delta(\boldsymbol{\theta})$; a PR's diffs $\{\delta_i\}_{i=1}^{k}$ are applied in index order, so its net change is $\boldsymbol{\theta}' = (\delta_k \circ \cdots \circ \delta_1)(\boldsymbol{\theta})$. Admissible diffs are (i) *well-typed* (schema-valid), (ii) *local* (bounded blast radius), and (iii) *explainable* (rationale and counterfactual tests).

Version-control primitives. This study uses a Git-style VCS[22] to manage decision-model changes. The primitives are: *Commit*—immutable snapshot (hash ID); *branch*—isolated line of work; *tag*—release label; *pull request (PR)*—reviewed change bundle; *CI*—automated checks on a PR.

Version-control repository and PR bundle. Let the version control repository be

$$\mathsf{Repo} = (G_{\mathrm{c}},\ \texttt{main},\ \mathcal{B},\ \mathcal{T}_{\mathrm{rel}},\ \mathcal{L}),$$

with commit DAG G_{c}, certified branch `main`, feature branches $\mathcal{B}$, release tags $\mathcal{T}_{\mathrm{rel}}$, and an immutable audit ledger $\mathcal{L}$ [375]. A PR is the tuple

$$\mathsf{PR} = \big(b,\ \{\delta_i\}_{i=1}^{k},\ \mathsf{IB},\ \mathsf{CI},\ \kappa\big), \qquad b \in \mathcal{B}, \quad \kappa \in \{\mathrm{R1}, \mathrm{R2}, \mathrm{R3}\},$$

where the *Impact Brief* IB [376] minimally contains

$$\langle \mathsf{rationale},\ \mathsf{change_set},\ \mathsf{scope},\ \mathsf{counterfactuals},\ \mathsf{metrics/docs} \rangle,$$

and CI is machine-checkable evidence (lint, tests, probes).

PR authority policy for change management (human-in-command). Each PR carries an assignment

$$\alpha : \{\mathsf{proposer},\ \mathsf{reviewers},\ \mathsf{approvers}\} \to 2^{\mathcal{P}} \setminus \{\emptyset\},$$

with the invariant $\alpha(\mathsf{approvers}) = \{\mathsf{Human}\}$ (humans only). Agents may propose and review but *cannot* approve; nor can they merge to `main` (invariant I4 below).

PR lifecycle. The PR lifecycle has five core stages—*Draft* → *Review* → *Approve* → *Merge* → *Release*—with explicit *Changes Requested* loops and *Rejected*/*Withdrawn*/*Superseded* exits. The state machine in Figure 22 shows the part implemented in the CCL Platform: *Draft* with its agentic review loop, role-based human approval, and the *Merged*, *Rejected*, or *Withdrawn* outcomes; release tagging and *Rollback* appear in Figure 23 and are described under *Role hand-offs* below. Concretely (traced under *Role hand-offs* below), agentic (Contributor ↔ Reviewer) and human review iterate until the gates required by κ pass; required human approvers sign; a maintainer performs `merge --no-ff`, tags the release, and writes the ledger (Figure 23 runs CI before the agentic review; since CI is re-run after fix-ups, the agentic pre-review can equally precede the gates, so both orders are admissible). Monitoring may trigger *Rollback*: the release reverts to the prior tag, and the corrective change re-enters *Draft* on a new branch. Rollback restores the model of record; groupings already produced under the reverted release are not undone by it. Merges require human sign-off (*human-in-command*) [377].

Conflicts. Merge conflicts can occur under concurrent edits. PRs must be brought up to date with the current `main` (rebase or merge) before approval; standard VCS resolution applies and CI is re-run. Beyond textual conflicts, Gate G_1 performs change-graph checks for *semantic* conflicts—co-edits to the same rows/paths or precedence collisions in the first-hit rule list—and blocks the PR until resolved, with final human sign-off.

Illustration: Version control and PRs. Figure 68 (Appendix A) makes concrete how changes flow through the governed repository: feature branches opened from `main`, *no-ff* merges that preserve PR identity, tagged releases, and branches that are reviewed but never merged (e.g., rejected or superseded). Each PR bundles typed atomic diffs, the Impact Brief, a CI run with per-gate outcomes (G_1–G_4) configured by the declared risk tier, and the required human approvals, linking them to the repository and the immutable audit ledger; the view thus illustrates invariants I1–I4 below.

[22] By "Git-style version control" this study means any (distributed) VCS that provides immutable commit hashes, branching, tags/releases, and code review via pull requests; Git is the most common implementation [293]. See Section 2.6.4 for details.

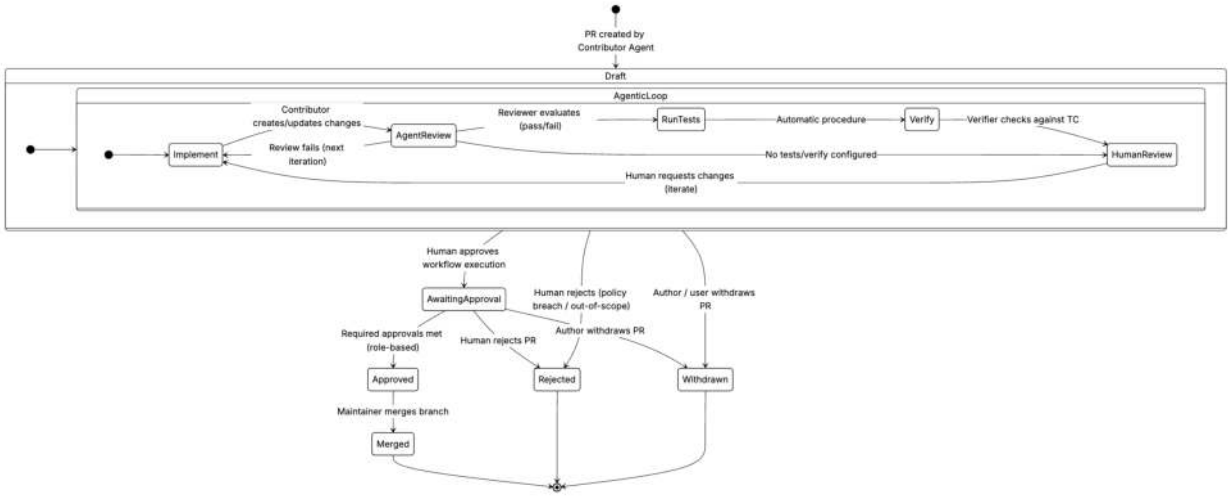


Figure 22: **CCL PR state machine as implemented in the CCL Platform.** A PR enters *Draft*, where an agentic loop (implement → agent review → optional tests/verification → human review) iterates on requested changes; it then awaits role-based human approval and is *Merged*, or exits as *Rejected* or *Withdrawn*. Release tagging and *Rollback* on monitoring alerts are shown in Figure 23; merges require mandated human sign-off.

Role hand-offs (explained). Figure 23 traces the PR from proposal to release and shows how responsibilities move between actors: *Proposer* (either the Contributor Agent or a human) creates a feature branch from `main`, applies the atomic diffs $\{\delta_i\}$, and opens a PR with an Impact Brief (IB). The repository Repo with its ledger $\mathcal{L}$ ("Repo & Ledger L" in Figure 23) triggers continuous integration CI, which returns gate outcomes for G_1–G_4 (objective/change control, oversight signals, transparency/evidence, and adversarial/risk checks).

An *agentic review loop* then runs: the Reviewer Agent examines the PR and leaves structured feedback; the Proposer applies fix-ups and updates the IB; the Reviewer Agent issues a final AI recommendation. These AI steps are advisory only—no agent can merge.

After AI review, *human sign-off* is required according to the declared risk tier κ (shown as `tau` in Figure 23): for $\kappa = \text{R1}$ a single human reviewer approves; for $\kappa = \text{R2}$ both domain and safety reviewers sign; for $\kappa = \text{R3}$ a third approver is additionally required (not drawn in Figure 23, which groups R2 and R3). A *Maintainer* then performs a `--no-ff` merge to `main`, tags the release, and writes the changelog to the immutable ledger $\mathcal{L}$. Post-merge monitoring (canaries/alerts) feeds back to the PR; on a critical regression the Maintainer rolls back to the prior tag. Throughout, the traceability and *human-in-command* invariants hold (I2 and I4 below).

Risk tiers and CI gates. Building on Section 2.6.2, this study holds that alignment risks manifest chiefly at *maintenance time* (outer/inner misalignment, goal misgeneralization, specification gaming). This study therefore operationalizes the research questions in Section 1.4.3.2 via a *risk-based control map* with four CI gates. PRs declare a risk tier $\kappa \in \{\text{R1}, \text{R2}, \text{R3}\}$ (a CCL-specific tiering that follows the risk-based approach of [262]) that configures mandatory gates G_1–G_4 (R1: G_1–G_3; R2 and R3: G_1–G_4, as configured in the CCL Platform, Section 6.3.2.1; gate contents and policy by tier in Table 29, Appendix A):

G1. **Objective & change control**—schema/typing checks and referential integrity; change-graph scan to catch overlaps, redundancies, and conflicts (e.g., mutually exclusive DRG rules); blast-radius bounds (locality budget on affected paths/DRGs); determinism/totality checks (unique next edge per predicate outcome; default "else" coverage); validation of code lists/features and key uniqueness. The PR is blocked (fail-closed) if schemas/keys are invalid, locality budgets are exceeded, or determinism/totality is violated.

G2. **Oversight signals**—reviewer-facing annotations and sampling: required metadata (risk tier, scope, affected DRGs, expected deltas); stratified sample sets (positives/negatives/near-misses) for human spot-checks; agreement checks between proposer, Reviewer Agent, and humans; leakage/anomaly detection (unexpected national activation flags, hidden dependencies, unintended fan-out); declared

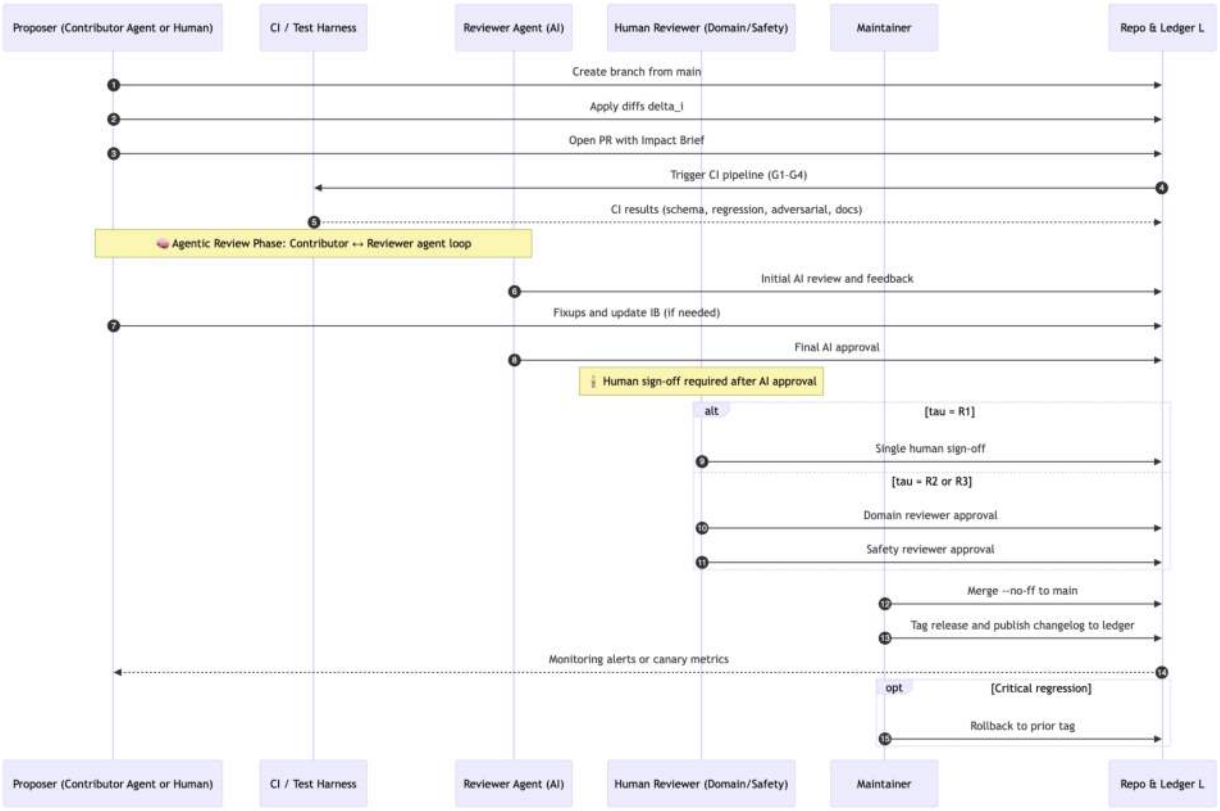


Figure 23: Roles and human–AI hand-offs across the PR timeline (the "Final AI approval" of step 8 is advisory only; the human sign-off that follows is mandatory). Agents collaborate during an initial *agentic review* phase; escalation to human sign-off preserves the human-in-command invariant. The label `tau` in the figure denotes the risk tier κ, not the typing function τ of Definition 6.3.2; R2 and R3 are drawn together.

post-merge monitoring hooks (what to watch, rollback trigger). The PR is blocked if required signals are missing, agreement falls below threshold, or leakage/anomalies are detected.

G3. **Transparency & evidence**—complete Impact Brief (rationale, change set, scope, counterfactuals, metrics, documents); provenance for referenced sources/artefacts and licensing/compliance checks; reproducibility bundle (scripts/configs, seeds, environment capture) linking commit→CI run; docs updated in lockstep (Interpretation Guide snippet, changelog, model/datasheet entries). The PR is blocked if the IB is incomplete, provenance/licensing is unclear, or reproducibility checks fail.

G4. **Adversarial & risk evaluation**—red-team tests and distribution-shift probes: scripted adversarial cases that attempt specification gaming (e.g., upcoding paths; mutually exclusive DRG collisions; age/sex boundary violations; national activation-flag misuse), plus fuzzed near-misses to test blast-radius control and agent-safety checks (scope creep, prompt injection). The PR is blocked (fail-closed policy) if any critical probe succeeds.

Decision-making model release invariants (post-merge). All merges uphold the following invariants:

I1. **Reproducibility** — deterministic rebuild from commit hash.

I2. **Traceability** — release→PR→commits→diffs/tests/reviews recorded in $\mathcal{L}$.

I3. **Rollbackability** — prior behavior restorable via tags.

I4. **Human-in-command** — a merge to `main` requires sign-off by the mandated human approvers; no agent can merge.

Design rationale. Each gate corresponds to a known failure mode from the alignment and MLOps literature (e.g., reward or objective misspecification, drift/leakage, opacity, adversarial fragility) [55, 273, 85, 275, 86].

G1 enforces objective fidelity and localized change (outer alignment; blast-radius bounds). **G2** institutes oversight signals and sampling to detect drift and leakage (ongoing governance). **G3** requires reviewable provenance (Model/Datasheet-style artefacts) for auditability. **G4** probes for specification gaming and misgeneralization with red-team tests and fail-closed policies (robustness; by analogy to inner alignment, since the proposer is an LLM whose edits may game the specification). Risk tier $\kappa \in \{\mathrm{R1}, \mathrm{R2}, \mathrm{R3}\}$ scales evidence and mandated human sign-offs to match potential impact.

Outcome: a pattern, not a full specification. CCL is presented as a governance *pattern*—the risk-tiered gates, typed atomic diffs, immutable ledger, and role constraints described above—demonstrated on one workflow (NordDRG decision-logic maintenance). A complete formal specification of the gate semantics and validation across further domains and live organizations are carried into future work (Section 7.8).

6.3.1.4 Design Artefact: Agent-Operable Decision Model of NordDRG

Purpose. In the demonstration, NordDRG is the governed decision-making model $\hat{f}_{\boldsymbol{\theta}} : \mathcal{X} \to \mathcal{Y}$ (Definition 3.1.17), whose parameters $\boldsymbol{\theta}$ form the *model-of-record*; it maps episode inputs to DRG outputs for reimbursement (Section 2.6.6). To admit AI agents as first-line contributors without eroding accountability, the model must be materialized as an *agent-operable* artefact—line-addressable, schema-typed, and under version control—so agents can *parse, reason about, and propose atomic patches* while preserving the official logic. This subsection specifies that representation and the formal model used for analysis.

Decision-list semantics (first-hit). Operationally, the NordDRG specification is an *ordered rule list* (decision list). Let the N rows be rules $R_i = (C_i, y_i)$ with fixed precedence $1 \prec 2 \prec \cdots \prec N$, where each condition $C_i : \mathcal{X} \to \{\mathsf{true}, \mathsf{false}\}$ is a conjunction of primitive predicates (e.g., code-list membership, age-in-range), and $y_i \in \mathcal{Y}$ is a DRG output. Evaluation is

$$\hat{f}_{\boldsymbol{\theta}}(x) \;=\; y_{i^*}, \qquad i^* \;=\; \min\{\, i \mid C_i(x) = \mathsf{true} \,\},$$

with a default rule if no C_i fires. Because conditions may overlap, the ordering is semantically essential (*first-match-wins*), and reordering rows can change behavior.

Tree/DAG view (for analysis). For inspection and metrics this study *compiles* this ordered list to an equivalent tree/graph while preserving priority. One option is an *else-chain* (left-deep) tree that tests C_1 and, on failure, continues to C_2, etc. Alternatively, this study forms disjoint conditions

$$C'_i \;=\; C_i \,\wedge\, \neg\Big(\bigvee_{j<i} C_j\Big),$$

which removes order dependence and yields a partition representable as a decision tree/DAG with shared subtests. Both compiled views are functionally equivalent to the ordered rules; this study uses the tree/DAG view for counts and figures, while the *authoritative model-of-record* remains the ordered rule list in the repository.

Formal structure of the compiled view. Building on Section 6.3.1.1, this study uses an *equivalent, compiled* tree/graph T for analysis:

$$T \;=\; \big(\mathcal{V},\, \mathcal{E},\, r,\, (D_v)_{v\in\mathcal{V}_{\mathrm{int}}},\, (g_v)_{v\in\mathcal{V}_{\mathrm{int}}},\, (y_\ell)\,\ell\in\mathcal{V}_{\mathrm{leaf}}\big),$$

where

- $\mathcal{V} = \mathcal{V}_{\mathrm{int}} \cup \mathcal{V}_{\mathrm{leaf}}$ are internal and leaf nodes, with root $r \in \mathcal{V}_{\mathrm{int}}$;
- each internal node v has a *node function* $g_v : \mathcal{X} \to D_v$ with a finite, typed outcome set D_v (typically Boolean; occasionally finite categorical);
- edges are outcome-labeled: $\mathcal{E} \subseteq \{(v, a, w) \mid v \in \mathcal{V}_{\mathrm{int}},\, a \in D_v,\, w \in \mathcal{V}\}$;
- each leaf ℓ carries an action $y_\ell \in \mathcal{Y}$ (a DRG code[23]).

The (possibly very large) code lists used by membership tests are *parameters* of g_v contained in $\boldsymbol{\theta}$ and do not increase the branching factor. For example, $g_v(x) = \mathbf{1}\{\mathrm{ICD}(x) \in L_v\}$ with $L_v \subseteq \mathcal{C}$, where $\mathcal{C}$ is the set of diagnosis codes (and $|L_v|$ possibly in the thousands), so $|D_v| = 2$ even when $|L_v|$ is large. Node functions are drawn from a fixed catalog $\mathcal{F} = \{f_1, \ldots, f_{16}\}$ (e.g., membership-in-code-list; exists/count over

[23] Optionally paired with country activation flags and related metadata, encoded in $\boldsymbol{\theta}$.

diagnoses/procedures; age-in-range; sex-match), with parameters (lists, thresholds) contained in $\boldsymbol{\theta}$. This study requires T to be acyclic (a tree or DAG of finite depth), together with *determinism* (for any v and $a \in D_v$ there is at most one $(v, a, \cdot) \in \mathcal{E}$) and *totality* (for all $x \in \mathcal{X}$, evaluation from r reaches exactly one leaf; a default "else" outcome in D_v may enforce this), so the induced map $\hat{f}_T : \mathcal{X} \to \mathcal{Y}$ is well-defined.

Evaluation on the compiled view. On the compiled tree/graph T, for an episode $x \in \mathcal{X}$ start at $v_0 = r$ and iteratively take the unique outgoing edge whose label matches the node outcome: for $t = 0, 1, \ldots$

$$a_t := g_{v_t}(x) \in D_{v_t}, \qquad (v_t, a_t, v_{t+1}) \in \mathcal{E}.$$

As T is acyclic with finite depth, there exists t^* such that $v_{t^*} = \ell^* \in \mathcal{V}_{\text{leaf}}$, and the induced decision function is

$$\hat{f}_T(x) = y_{\ell^*}.$$

Learning view. Although NordDRG is currently hand-curated, nothing precludes deriving (or refining) T via the data-driven paradigms in Figure 65; the present study focuses on how such learning can be collaboratively curated during *continuous maintenance* rather than at initial training time.

Potential representations. NordDRG can be exposed with two equivalent surface representations:

i) a **NordDRG custom format** (the *authoritative, ordered* rule list): a database of about 20 interlinked tables with human-readable documentation that describes how the decision logic is applied to hospital-episode records [23, 24];

ii) a **canonical tree/DAG view** compiled from the ordered rules: each node is a typed predicate and each leaf a DRG label; order is preserved via else-chains or disjoint conditions. This view is convenient for metrics, figures, and ML tooling.

To illustrate the two representations, Figure 69 (Appendix A) shows a fragment of the first-layer logic for DRG 373X (*Child delivery without complications*). The top part shows the custom format, where each row expands to a path condition; the bottom part shows the corresponding path-wise view compiled as a canonical decision tree. Both surfaces are designed to be round-trippable (round-trip tallies are not reported; Section 7.7.1) and agent-operable when accompanied by the Interpretation Guide for the custom format.

Selecting representation. Standard ML decision-tree formats are typically not used for *collaborative development* of regulated decision logic (cf. gaps G.III.3.1 and G.III.3.3, Section 1.4.3.3). The NordDRG custom format and attached processes are designed for expert review and approval of changes and are implemented in existing tooling, with human-readable documentation for regulatory transparency. This study therefore maintains the ordered rule list as the authoritative artefact under version control and uses the compiled tree/DAG view for analysis, visualization, and agent-facing tooling.

6.3.1.5 Design Artefact: Participating NordDRG LLM Agents

This subsection specifies the two LLM agents (*LLM Agent*)—*Contributor* and *Reviewer*—that act as first-line participants in the CCL proposer–reviewer loop for maintaining the NordDRG model-of-record. In the terms of Definition 3.2.5, both are governed LLM agents: they act under the PR authority policy α, the gates G_1–G_4, and the ledger $\mathcal{L}$ of Section 6.3.1.3. They ground tickets in the versioned specification, emit typed, atomic diffs with rationale, and perform pre-review quality checks, packaging reviewer-ready artefacts.

Design Artefact: NordDRG Contributor Agent

Purpose and interaction model. The *NordDRG Contributor Agent* is a role-constrained LLM assistant that reads forum tickets or specifications, incorporates feedback from both agents and humans, and drafts *atomic diffs* to the decision model. Interaction is chat-based: the agent produces reviewer-ready artefacts (diffs, brief, examples/tests) that can be attached to a pull request.

Scope and assumptions. The agent operates over a versioned, typed NordDRG model-of-record (line-addressable tables). Each change proposed by the Contributor is represented as an atomic diff, consisting of edits to predicates or outputs. The agent's role is limited to proposing these diffs as advisory outputs (an exported bundle or a pull request on a feature branch of the CCL Platform), while the final decision to incorporate them into official repositories or workflows remains under human control.

Knowledge sources. At runtime the agent is grounded in two *versioned* artefacts:

- **Specification** Spec: the authoritative NordDRG definition tables and their induced rule graph (line-addressable, schema-typed).
- **Guidelines** $\mathcal{G}$: interpretation rules covering precedence, inclusions/exclusions, and CC/MCC semantics.

The agent retrieves scoped slices of Spec and $\mathcal{G}$ while drafting; all suggestions cite table/row IDs and guideline clauses (with commit/tag) to ensure provenance and auditability.

Inputs and outputs. *Input:* The agent accepts free-form chat messages, which may include copy-pasted ticket content or specification text.

Output: A reviewer-ready bundle:

- **Atomic diffs (machine-readable exports)** $\{\delta_i\}_{i=1}^{k}$: insert/modify/delete operations over line-addressable tables with stable IDs, emitted as CSV/JSON and patch snippets suitable for PR import.
- **Impact Brief (IB)**: rationale, change set, scope (affected DRGs/paths), counterfactuals (the cases listed below), and supporting citations under metrics/docs.
- **Counterfactuals/tests**: positive, negative, and near-miss cases for CI.

Architecture. The agent follows a multi-stage process:

1. **Chat + Ticket Parser**: The agent interprets user messages and any pasted tickets to extract the intent.
2. **Grounded Retrieval**: The agent pulls relevant sections from the specification Spec and guidelines $\mathcal{G}$.
3. **Change Planner**: It breaks down the intent into candidate atomic edits, annotated with potential impacts.
4. **Patch Composer**: The agent generates the diffs for the NordDRG specification format.
5. **Exporter**: The agent compiles the diffs and explanation into a bundle that can be exported for review.

All reasoning and drafting of changes are carried out by the LLM component, while deterministic checks are applied to ensure that changes are well-formed and consistent.

User access. Experts interact via a chat interface. The agent explains its reasoning in natural language, links to specification slices (table/row IDs), cites guideline clauses, and offers one-click download of the reviewer-ready bundle.

Limitations and safeguards. The agent cannot post to ticketing systems, approve or merge pull requests, or write to the authoritative `main` branch; a pull request it opens on a feature branch (in the CCL Platform, Section 6.3.2.1) remains advisory until human approvers sign. All outputs are advisory and require human review and approval before inclusion in any official workflow. Interactions, prompts, and artefacts are logged (with timestamps and model/version identifiers) for traceability; deterministic linting enforces schema/typing and key integrity. Deterministic validators also check referential integrity and locality bounds.

Design Artefact: NordDRG Reviewer Agent

Purpose and position in CCL. Where the *NordDRG Contributor Agent* drafts candidate updates, the *NordDRG Reviewer Agent* acts as an advisory AI pre-review step (distinct from the CI gates G_1–G_4) that scrutinizes every change set before it is exposed to human experts. Its mandate is to (i) verify *technical integrity* (typing, schema, referential consistency), (ii) assess *semantic fitness* against the official NordDRG guidelines, and (iii) prepare a structured review package for human judgment and sign-off. The agent therefore implements Steps 6–8 of the CCL process in Figure 23, shielding certified reviewers from low-level linting while surfacing the policy-relevant questions that require human insight. As with HADA (Section 6.1), mandated sign-off is a design commitment rather than a guarantee: oversight is effective only when reliance on AI recommendations is well calibrated [354, 355, 356], and pre-filtering by the Reviewer Agent can invite overreliance [359]. The G2 agreement checks and stratified spot-check samples are CCL's mitigation, whose effectiveness is not evaluated here.

Scope and constraints. As illustrated in Figures 22, 68, and 23, the Reviewer Agent *never* approves or merges changes to the model-of-record; it only produces a Review Report with machine-checkable findings and prescriptive feedback. All approval decisions, at every risk tier, rest with human reviewers, preserving the "human-in-command" invariant of CCL. Under normal operating conditions, the agent aims to ensure that every atomic diff it forwards to humans is syntactically valid across all target tables and complies with the NordDRG guidelines.

Knowledge sources. At runtime the agent is provisioned with Spec (the versioned NordDRG specification), $\mathcal{G}$ (interpretation guidelines), and the **Change Bundle** CB emitted by the Contributor Agent: $\mathsf{CB} = \langle\{\delta_i\}, \mathsf{IB}, \mathsf{tests}\rangle$, i.e., the reviewer-ready bundle of Section 6.3.1.5 (atomic diffs, the Impact Brief, and the counterfactual tests). The agent is also equipped with additional prompts to check typical issues encountered in change requests, such as proposed logic changes that use national diagnosis or procedure codes, whereas the NordDRG specification uses a combined common coding system.

Inputs and outputs. *Input:* a candidate pull request consisting of atomic diffs $\{\delta_i\}$, its accompanying Impact Brief, and the counterfactual tests (the Change Bundle CB). *Output:* a Review Report $\mathsf{RR} = \langle\mathsf{status}, \mathsf{findings}, \mathsf{suggested_fixes}, \mathsf{qa_tests}\rangle$ where

- status $\in$ {pass, conditional-pass, fail};
- findings: structured comments grouped by table and diff hunk, ranked by severity;
- suggested_fixes: ready-to-apply patch snippets or prompt fragments for the Contributor Agent; and
- qa_tests: positive, negative, and near-miss test cases proposed for CI.

Review protocol. For every diff δ the agent executes a two-layer pipeline:

1. **Static analysis** — verify schemas, code lists, and key constraints.
2. **Guideline compliance** — the LLM checks each edit against formal NordDRG business rules and annotates deviations with inline citations.

Only if both layers succeed with no fail-level findings does the agent return status=pass; otherwise it flags the pull request for human attention, detailing exact correction steps.

Interaction loop with the Contributor Agent. When status=conditional-pass or fail, the Reviewer Agent posts its suggested_fixes back to the Contributor Agent (as depicted in Figure 67); this exchange forms the auto-remediation cycle of the *agentic workflow*. The Contributor Agent may apply the patches, regenerate the Impact Brief, and resubmit—iterating up to a configured loop limit before human review begins.

6.3.2 Demonstration: Collaborative Curated Learning

This section instantiates CCL on NordDRG, demonstrating a minimum-viable, governed loop for maintaining auditable decision-making logic. It presents the CCL Platform, the full-stack web application that serves as the execution environment for all demonstrations (Section 6.3.2.1); parses the official definition tables and compiles them into a canonical decision-tree/DAG to expose structure, scale, and reviewability (Section 6.3.2.2); prepares an *agent-operable*, line-addressable model-of-record with stable identifiers and grounding guides (Section 6.3.2.3); and exercises the agentic proposer–reviewer workflow, whose reviewer-ready artefacts slot into CI gates and mandated human sign-off (Section 6.3.2.4).

6.3.2.1 Technology and Architecture of the CCL Platform

The CCL Platform was implemented as a full-stack web application comprising a Python back end and a TypeScript/React front end. This section describes the technology choices, the overall architecture, the scale of the implementation, and key capabilities of the resulting artefact.

Technology stack.

Back end. The server side is built on **FastAPI** 0.109, an asynchronous (ASGI) Python web framework that provides automatic OpenAPI documentation and request validation via Pydantic 2.6. Data is persisted

in **PostgreSQL** and accessed through **SQLAlchemy** 2.0 with connection pooling. The multi-agent system integrates with three LLM providers—OpenAI, Anthropic, and Google Generative AI—through a unified `LLMService` abstraction that allows per-agent model selection at configuration time. Git operations are performed through the local file system, and remote operations are authenticated via GitHub tokens, enabling the platform to version-control decision model artefacts. Graph visualizations of Git history and PR flow are generated as Mermaid diagram syntax on the server and rendered on the client.

Front end. The client is implemented with **Next.js** 15 (React 19) using the App Router and Tailwind CSS 3.4 for utility-first styling. All data visualizations are rendered client-side: interactive decision-tree views use custom React components, while sequence diagrams, flowcharts, and commit graphs are rendered with **Mermaid.js** 11.12. Markdown content—Impact Briefs, documentation, and review comments—is presented through React-Markdown with GitHub-flavored Markdown support. The front end communicates with the back end exclusively through a typed REST API client that encapsulates 96 asynchronous methods organized by domain (pull requests, agent chat, specifications, Git operations, workflows, governance, and administration).

Architecture. Figure 66 (Appendix A) illustrates the high-level architecture. The system follows a client–server separation. The Next.js front end runs in the user's browser and sends HTTP requests to the FastAPI back end, which exposes 122 REST endpoints grouped into 13 route modules. The back end delegates domain logic to 17 service classes that encapsulate pull-request lifecycle management, Git operations, specification parsing, agent orchestration, CI gate execution, audit logging, and change-request tracking.

The multi-agent orchestration layer coordinates the two agents designed in Section 6.3.1.5—the Contributor Agent and the Reviewer Agent—plus an optional Verifier Agent that executes attached test and verification procedures (a platform extension outside the scope of the artefacts designed and evaluated in this study). The agents are coordinated through a state-machine-driven Workflow Engine. Each agent maintains conversation history in the database and produces structured artefacts (atomic diffs, Impact Briefs, review findings) that flow through configurable quality gates (G1–G4) and risk-tier-based approval workflows (R1–R3) before a change can be merged into the versioned decision model.

Scale of the implementation. Table 18 summarizes the quantitative scope of the platform. The total implementation comprises approximately 33,000 lines of code split roughly equally between the Python back end and the TypeScript front end.

Table 18: Quantitative metrics of the CCL Platform implementation (approximate counts at the time of writing).

Metric	Count
Total lines of code	32,740
Front end (TypeScript/TSX)	16,494
Back end (Python)	16,246
Front-end source files	27
Back-end source files	63
Page components (Next.js routes)	14
Reusable UI components	11
API client methods	96
REST API endpoints	122
API route modules	13
Back-end service classes	17
Database models (SQLAlchemy)	9

Application capabilities. The platform's capabilities for collaborative, AI-assisted maintenance of the NordDRG decision model can be grouped into seven areas.

Overview and situational awareness. The Overview page (Figure 72) serves as the entry point and presents live system metrics—open pull requests, pending reviews, specification table counts, and decision rule totals—alongside a summary of the quality-gate pipeline and key decision-model statistics. A step-by-step

workflow visualization explains the end-to-end change lifecycle, and navigation cards link to every section of the platform.

Agentic workflow configuration and execution. The Agentic Workflows section (Figure 73) provides a configurator for defining multi-step, agent-driven pipelines. Each workflow specifies which agent roles participate (Contributor, Reviewer, and the optional Verifier), the maximum number of iterative loops, and which LLM each agent should use. The Agent Configurator tab (Figure 74) allows administrators to create and manage individual agent configurations—selecting the agent role, underlying LLM provider and model, system prompt, and behavioral parameters. Automatic test and verification procedures can be attached to workflow steps. Once triggered, a workflow execution proceeds through branching, implementation, review, optional testing and verification, human review, and merge, with each step recorded in the audit ledger.

Governance and quality assurance. The governance framework implements four quality gates. Gate G1 validates schema, lint rules, and change-graph analysis. Gate G2 checks oversight signals, sampling protocols, and annotation agreements. Gate G3 verifies impact-brief completeness, documentation, and data lineage. Gate G4 runs adversarial probes and distribution-shift tests. The gates are defined and configurable on the platform; in the demonstration evaluated in Section 7.7.3, automated CI execution (Steps 6–8 in the numbering of Figure 24, which differs from that of Figure 23) was not exercised, and acceptance was decided by manual review. Three risk tiers determine which gates are required and how many human approvals must be obtained: R1 (low risk, G1–G3, one approval), R2 (medium risk, G1–G4, two approvals: domain and safety reviewers), and R3 (high risk, G1–G4, three approvals: domain, safety, and a third approver).

Decision-tree visualization and analysis. The Decision Tree page (Figure 75) offers three view modes for the NordDRG decision logic. The *Interactive Tree* mode renders an expandable, hierarchical view of decision nodes and leaf DRG outputs. The *SVG Graph* mode generates a Mermaid flowchart with configurable layout orientation. The *Full Analysis* mode presents a statistics dashboard with feature-usage bar charts, path-depth distributions, feature-expansion tables, and a ranking of the most common DRG outputs. A searchable DRG code filter allows users to isolate the decision path for any specific grouping.

Version control and change management. Every modification to the decision model is tracked through Git. Pull requests carry atomic diffs (typed INSERT, MODIFY, or DELETE operations on specification table rows), Impact Briefs, review trails, and gate results. The Decision Model Version Control page visualizes the commit history, branch graph, and PR lifecycle. A Data Browser enables direct inspection of the underlying specification tables with search, filtering, and pagination.

Change request management and agentic execution. The Tickets page serves as the primary entry point for the agentic workflow. Users can import change requests from external issue trackers (e.g., Redmine) or create them locally, attach supporting documents, and trigger AI-assisted implementation. The implementation pipeline allows selection of LLM provider and model, target NordDRG version, and relevant attachments. Progress is monitored asynchronously with a live execution log. Once the Contributor Agent's change bundle has been opened as a pull request on a feature branch (never on `main`), users can trigger the Reviewer Agent for automated quality review, inspect findings, and optionally re-run implementation with review feedback incorporated. The Workflow mode orchestrates the Contributor–Reviewer pipeline (optionally extended with the Verifier) with configurable iteration limits and a mandatory human review checkpoint before merge.

Governance dashboard. The Governance page provides an administrative view of the quality assurance framework: editable quality gate definitions (G1–G4) with their associated checks and risk-tier requirements, risk tier configuration (R1–R3) with approval thresholds, a role-based approval workflow editor, and a chronological audit trail with filtering by pull request or event type.

User interface. Figures 72–75 show representative screenshots of the platform.

6.3.2.2 Analyzing NordDRG as a Decision Tree

Tooling and normalization. A Python converter was implemented that ingests the official NordDRG definition tables and emits a schema-typed, line-addressable file representation suitable for agent operation and PR-centric governance. This implementation underpins all analyses in this subsection and is available, with usage notes and reproducibility scripts, from the author on request.

Table 19: Scale and structural statistics for the NordDRG decision model. The "first-layer" tree refers to the top-level grouping layer.

Description	Value
Number of features used in the decision tree	16
Number of different leaf labels (DRG groups) in the tree	2,752
Number of root-to-leaf paths	8,867
Average number of alternative paths per DRG leaf	3.2
Number of non-empty decision points in the first-layer tree	53,848
Average decision points per root→leaf path (first-layer tree)	6.1
Number of decision points in the full decision tree	40,453,620

Compiled view, metrics, and visuals. From the ordered rule list, the tool compiles a canonical decision-tree/DAG for visualization (Section 6.3.1.4). It then computes summary metrics over this structure—features used, distinct DRG leaves, root→leaf path counts, and decision-point totals—and exports reviewer-legible Scalable Vector Graphics (SVG) slices for inspection and discussion (Figure 70; Table 19).

SVG artefacts. The tree view makes individual predicates and audit trails explicit, but static visualizations are useful only for small slices. Figure 70 (Appendix A) illustrates three small fragments of the first-layer decision tree, each showing a different number of root→leaf paths. SVG exports of the first-layer decision trees are available from the author on request. The complete first-layer tree (8,867 root→leaf paths) results in a *16.4 MB* plain-text SVG. Smaller thematic fragments (e.g., 10, 25, or 100 paths) remain human-navigable.

Decision tree metrics. Table 19 summarizes the key metrics of the first-layer decision tree. The first-layer decision tree logic uses *16* input features referenced by the rules, yet the tree yields *2,752* distinct DRG leaf labels. With *8,867* root→leaf paths, each DRG group is reachable via roughly *3.2* alternative paths on average, which may reflect clinical variations or redundancy that nonetheless end in the same group.

The notion of a *decision point* counts predicate evaluations along paths. In the first-layer tree there are *53,848* non-empty decision points, or about *6.1* checks per path on average—small enough to review visually in fragments. When the full, multi-layer specification is unrolled path-wise, this balloons to *40,453,620* decision points (roughly *4,600* checks per first-layer path on average), an increase by a factor of approximately *750*. This scale explains why full-graph pictures are impractical and motivates programmatic analyses and interactive viewers. These figures also suggest targets for optimization (e.g., feature consolidation) and quality assurance (e.g., path coverage criteria).

6.3.2.3 Preparing the NordDRG Specification for CCL

Figure 71 (Appendix A) illustrates the artefacts prepared in this study to enable CCL over the NordDRG specification.

From custom notation to agent-editable plain text. The *Design Artefact: Agent-Operable Decision Model of NordDRG* (Section 6.3.1.4) was authored in the customary NordDRG notation and then converted into a set of separate plain-text CSV files (one logical table per file). This representation makes the full, large decision model line-addressable in Git, supports automated diffs/reviews, and allows software agents to perform scripted edits under CCL workflows.

Converter implementation. A Python baseline converter parses the NordDRG source specification and emits the normalized CSV files. The conversion scripts and usage instructions are released in the public NordDRG AI Benchmark repository (`scripts/run_conversion.py`; Section 6.2), enabling reproducibility of the conversion step.

Agent-facing documentation. To help LLM agents reason about and modify the custom NordDRG format, this study bundled two PDF documents retrieved from the NordDRG Forum [49] that describe how to read the definition tables [309] and how to write the changes to the specification. These serve as grounding references so agents can understand the notation and propose well-typed updates.

Relation to NDMS (NordDRG Maintenance System). Experts currently use the special-purpose NDMS software to generate official specification releases; these NDMS outputs serve as the starting point for the demonstration. While the same methodology could integrate NDMS directly with Git (e.g., automated exports and CI checks), such system integration is out of scope for this research.

6.3.2.4 Demonstration: NordDRG Contributor and Reviewer Agents in Action

Purpose. Given human-authored tickets, the agent produces change suggestions aligned with the official NordDRG format. The process includes: (i) interpretation of the ticket describing the intent and scope, (ii) generation of atomic diffs to the rule tables, and (iii) an agent-to-agent review process.

Context. To illustrate how LLM software agents can participate in the CCL process, the two LLM agents **NordDRG Contributor Agent** and **NordDRG Reviewer Agent** were implemented (see Section 6.3.1.5 for artefact design). The Contributor Agent is a conversational assistant that reads NordDRG Forum tickets (or pasted equivalents) and returns structured, machine-readable change suggestions for the NordDRG specification. The Reviewer Agent ingests these change suggestions and the original ticket, then proposes changes to ensure the suggestions align with the NordDRG decision model coding standards and guidelines. Approval of merges remains reserved for certified reviewers (Section 6.3.1.3). Together the Contributor and Reviewer Agents participate in an agentic workflow illustrated in Figures 67 and 23.

Context provided to the agents. At runtime, the agent is grounded with prompts that describe the agent's role and how it should interact in the process; the official NordDRG specification as interlinked tables; human-readable documentation on how to interpret the specification; and guidelines that define the structure and style of change suggestions.

This prototype operates on public artefacts and does not integrate with NDMS or evaluate NordDRG governance; its purpose is to illustrate how a governed, PR-centric loop could operate under human authority. In the demonstration cases, the expert change implemented in the NDMS production system for the same ticket served as the reference standard: a proposed diff was accepted when, on the author's manual review (a single, non-blinded reviewer), it reproduced that change (Table 20; Section 7.7.2).

Agent LLM configurations. In this demonstration, this study used the OpenAI LLMs o3, o4-mini-high, and GPT-4o for the Contributor and Reviewer Agents; model snapshots were not pinned, so the runs used the versions served by the provider at the time of the evaluation.

Agent workflow and user interface. Interaction is via a chat-like interface, with no external integrations required. This is intended to lower the barrier for non-technical stakeholders (e.g., coders and physicians; not evaluated with users in this study) while producing reviewable technical artefacts immediately. The discussion interface supports both human-to-agent and agent-to-agent interaction (see Figure 67).

Figure 76 visualizes the workflow turn of the *NordDRG Contributor Agent*: the left panel presents the original ticket posted in the NordDRG Forum—requesting new MDC08 rules and the addition of the diagnosis property (DGPROP) 90X11 to a list of pain-related ICD-10 codes. The center icon symbolizes the large-language-model agent that interprets this ticket. Guided by the ticket, the agent autonomously synthesizes a candidate set of atomic `git` diffs (right panel): it creates a dedicated feature branch, appends DGPROP 90X11 to `dgprop_name`, assigns this property to the ten ICD-10 codes in `dg_feat`, and inserts the corresponding grouping rule in `drg_logic`. These patches and the concise rationale allow domain experts to review and merge the proposed changes into the NordDRG definition tables.

Figure 77 depicts the quality-assurance loop provided by the *NordDRG Reviewer Agent*. The left panel combines the original forum ticket with the concrete `git` patches generated by the Contributor Agent, while the central icon represents the LLM-based reviewer. After parsing both the ticket and the proposed diffs, the agent emits a structured set of review comments (right panel) that are grouped per table. The Reviewer requests correction of diagnosis codes that are represented with national codes instead of common Nordic codes used by the NordDRG specification; enforces the conversion of national ICD-10 codes to their comb-code counterparts in `dg_feat`; and highlights quoting, alignment, and rule-ID requirements in `drg_logic`. These annotations let domain experts iterate on the patch set until it complies with the NordDRG template and comb-code conventions.

Prompt transparency and alignment with CCL. All system and user prompts configuring the agent are available from the author on request, to support reproducibility and safe sandboxing by national authorities.

This demonstration was first run as a stand-alone chat prototype and later re-hosted in the CCL Platform (Section 6.3.2.1); in both settings its design follows CCL principles: suggestions are expressed as atomic diffs over a typed decision model; outputs are human-readable and machine-verifiable; audit-trail-friendly exports are prepared for downstream CI validation; and no autonomous merges occur (maintainers retain full control).

7 EVALUATION AND DISCUSSION

This chapter evaluates the demonstrations presented in the previous chapters. For Phase I, Section 7.1 evaluates and discusses the Interaction Protocol Adaptation with Software Agents demonstration, Section 7.2 the Ontology-Driven Software Agents for B2B Purchasing Automation demonstration, and Section 7.3 the third demonstration on context-aware service matching in a mobile environment. For Phase II, Section 7.4 evaluates and discusses the empirical implementation on healthcare data with software agents and ML-based information retrieval techniques. For Phase III, Sections 7.5, 7.6, and 7.7 evaluate and discuss the Human-AI Agent Decision Alignment (HADA) architecture, the NordDRG AI Benchmark for LLMs, and the CCL reference development process for decision-making logic. Finally, Section 7.8 consolidates the recurring threats to validity, scalability, and generalizability across the program, and states the level of evidence backing each research question. For the DSRP flow, see Table 1 (DSRP process sequence linked to the phases and structure of this thesis).

7.1 Phase I: Interaction Protocol Adaptation with Software Agents

These results answer **RQ I.2**: run-time protocol adaptation (the *ConStrip* scenario) is established by demonstration (a peer-reviewed prototype) in the scoped sense of ontology-constrained interoperability with flexibility confined to one role, confirming syntactic interoperability while exposing the limits of semantic convergence (Table 4; the level of evidence is discussed in Section 7.8).

Purpose and scope. This evaluation and discussion concerns the main artefact *interaction-protocol–based service adaptation*, assessing it against objectives O.I.1.1–O.I.1.4 (Section 1.2.1.4) and situating it within the problem setting and scenario outlined in Section 1.2.1 and Section 4.1.

Evaluation against objectives.

O.I.1.1 Protocol parsing from RDF serialization (Section 4.1.1.2, Section 4.1.1.4) — *Achieved, but brittle.* The expeditor agent parsed RDF-serialized, ontology-conformant FIPA protocol descriptions at run time and mapped them to a preprogrammed state machine. This demonstrates machine-processable intake and internalization of protocol structure. However, the parser's success is contingent on *strict* adherence to the predefined interaction-protocol ontology. Minor schema drift, alternative modeling idioms, or richer constraint vocabularies were not exercised. No evidence was shown for graceful degradation, schema versioning, or recovery strategies when encountering out-of-ontology constructs.

O.I.1.2 Message exchange on agent platform (Section 4.1.2.1) — *Achieved, minimal coverage.* FIPA-ACL conversations were executed among contractor, subcontractor, supplier, and carrier agents with observable routing and platform-level diagnostics, establishing a solid baseline that the plumbing works. The coverage is nonetheless narrow: only the "happy path" and a small set of dialogue moves were demonstrated; transport variability, failure modes (timeouts, retries, disagreement), and performance under concurrency were not evaluated. Partner discovery was intentionally simplified (partner addresses were read from the subcontractor's status report), which understates integration challenges at ecosystem scale.

O.I.1.3 Scenario-specific protocol handling (Section 4.1.1.3) — *Partially achieved.* The contractor's behavior generalized across supplier (FIPA Query) and carrier (similar) protocols when they conformed to the ontology, supporting the claim of configuration-over-code. Yet only the *contractor* role exhibited run-time flexibility; counterparties followed fixed behaviors. The control logic remained a finite-state machine without an explicit goal-directed transition function or policy layer, limiting adaptation to pre-enumerated states and transitions. Unanticipated protocol variants, exception/repair patterns, or mixed-initiative deviations were out of scope.

O.I.1.4 Report extraction and comparison (Section 4.1.1.3) — *Achieved, thin validation.* The expeditor extracted order identifiers from RDF-formatted reports and cross-queried supplier and carrier agents to compare status information. This establishes a *mechanism* for lightweight cross-validation; its auditability was not evaluated beyond platform debug tools (see Instrumentation bias below). The experiment stopped short of systematic data quality assessment (precision/recall of matches, handling of conflicting claims, or reconciliation policies), so the *effectiveness* of comparison remains suggestive rather than quantified.

Synthesis. Taken together, the artefact demonstrates end-to-end feasibility under controlled conditions: ontology-based protocol intake, minimal partner discovery, FIPA-ACL message exchange on JADE, and limited cross-validation. The strongest evidence lies in O.I.1.1 and O.I.1.2 (machine-processable descriptions and functioning conversations). The weakest lies in O.I.1.3 (breadth and robustness of "adaptation"), where flexibility is confined to one role and bound to FSM enumerations, and in O.I.1.4 where comparison exists as a mechanism but lacks evaluation depth. Overall, the prototype is better characterized as *ontology-constrained interoperability with scoped flexibility* rather than open-ended run-time adaptation.

Limitations to validity.

- **Construct validity.** *Adaptation proxy.* Using an FSM as the primary control model risks conflating protocol selection/execution with genuine adaptation. Goal-aware transition functions or policies were not implemented, so "adaptation" may overstate what is essentially parameterized execution. *Protocol conformance assumption.* Success depended on exact ontology conformance; this may measure *schema agreement* more than real-world adaptability.
- **Internal validity.** *Role asymmetry.* Only the contractor adapted; fixed counterparty behaviors may mask interaction failures that would surface if all roles varied concurrently. *Instrumentation bias.* Platform debug tools were used for observation; absence of independent tracing/ground-truth checks may inflate perceived reliability.
- **External validity.** *Ecological realism.* The expediting scenario is plausible but limited. Heterogeneous partner stacks, partial ontology adoption, and conflicting exception policies—typical in multi-tier supply chains—were not exercised. *Scalability.* No load, latency, or concurrency studies were reported; simplified partner discovery downplays discovery, matching, and churn at scale.
- **Conclusion validity.** *Sparse metrics.* Evidence is primarily qualitative ("works on example") with few quantitative indicators (coverage, failure rate, recovery time, comparison accuracy). This weakens the strength of any generalized performance claims.

Implications for the artefact. To progress from controlled feasibility to robust adaptability, the artefact would benefit from: (i) adding a goal-directed transition function or policy layer over the FSM; (ii) extending adaptation to multiple roles with negotiated exception/repair patterns; (iii) adding a registry for discovery and semantic matching (versioning, partial conformance, conflict resolution); and (iv) expanding evaluation to adversarial and out-of-ontology cases with quantitative metrics for comparison accuracy and end-to-end reliability.

Discussion. The demonstration establishes a narrow but concrete path from ontology-grounded protocol descriptions to executable conversations on an agent platform. Its chief contribution is evidencing that *conversation structure*, when explicitly modeled in RDF and consumed at run time, can lift integration beyond brittle, endpoint-centric glue code. At the same time, three design trade-offs limit how far these results generalize:

(i) the prototype works *inside* a controlled ontology but is undefined outside it—so without partial-conformance handling, schema mediation, or negotiation it risks a new lock-in at the ontology layer; (ii) finite-state control gives predictable execution but conflates "following a script" with adapting, lacking a goal-directed transition policy over goals, context, and constraints; and (iii) FIPA-ACL messaging on JADE proves the plumbing but not operation under churn, conflict, or scale.

Positioning. Compared to WS-*/BPMN-style service integration, the artefact shifts emphasis from operation-level bindings to *conversation semantics*. This is a meaningful design choice for domains where turn-taking, commitments, and exception handling dominate over single-call idempotent operations. However, the cost is a heavier dependence on shared vocabularies and governance for versioning and change control.

Forward link. The needs identified above—goal-aware control plus governance and audit/trace layers around machine-interpretable protocols—are precisely what the later phases take up.

7.2 Phase I: Ontology-Driven Software Agents for B2B Purchasing Automation

These results also answer **RQ I.2**: ontology-driven B2B negotiation is established by demonstration (a prototype reported in the Licentiate thesis [2], not in a peer-reviewed paper), with ontology-engineering cost and fragility identified as the gap to practical convergence (Table 4; Section 7.8).

7.2 PHASE I: ONTOLOGY-DRIVEN SOFTWARE AGENTS FOR B2B PURCHASING AUTOMATION

Purpose and scope. This section evaluates and discusses the demonstration of the *ontology-driven B2B purchasing* main artefact (Table 5), assessing it against objectives O.I.2.1–O.I.2.4 (Section 1.2.2.3) and situating it within the problem setting and bidding scenario outlined in Section 1.2.2 and Section 4.2.

Evaluation against objectives.

O.I.2.1 Automate routine purchasing tasks. *Partially met.* Basic supplier discovery via a registry and ontology-driven acceptance checks are implemented. However, routine purchasing typically includes multi-round RFQs, supplier counter-offers, expiry handling, delivery-slot conflicts, and exception workflows. Beyond a single buyer-initiated price revision over FIPA Propose (Section 4.2.1.1), none of these operational loops are evidenced. The "automation" remains closer to *scripted reasoning* than to end-to-end task automation.

O.I.2.2 Realize automation through software agents. *Met (scoped).* Buyer/supplier roles converse on JADE with conversation semantics foregrounded. That said, the platform-level success hides integration effort: there is no demonstration of reliability under contention, recovery from partial failures, or cross-platform interoperability beyond JADE.

O.I.2.3 Maintain human oversight. *Not demonstrated.* Escalation to personnel is specified only at design level (Section 4.2.1.2); escalation triggers, approval gates, or "four-eyes" checks were not implemented. In practice, thresholds for price variance, supplier risk, or contractual deviations would route decisions to humans; the demonstration provides no concrete mechanism or audit artefacts for these escalations.

O.I.2.4 Leverage application-domain ontologies. *Met (minimal expressivity).* Ontologies make artefacts machine-processable and enable acceptance reasoning. The commercial surface is narrow: payment and delivery semantics are simplified, and penalties, rebates, Incoterms nuances, volume breaks, and exception clauses are not modeled. The coverage is adequate for illustration, not for operational breadth.

Evidence and evaluation approach. Enterprise systems (pricing, stock) are simulated with XML sources, to which the agents are assumed to have timely access; the registry is presented as a semi-intelligent exemplar; conversations execute on JADE; Web-service transport and OWL-S-style bridges are acknowledged but intentionally out of scope. Collectively, this evidences a *working vertical slice* rather than a production-grade pipeline. A demonstration-based, descriptive evaluation was used: a single platform, simplified ontologies, simulated backends, and a bespoke registry. No comparative baselines, ablation studies, load tests, or user-in-the-loop trials are reported.

Limitations to validity.

- **Construct validity.** "Automation" is proxied by registry lookups, a single acceptance decision, and at most one buyer-initiated price-revision round; typical constructs like multi-round negotiation, exception handling, and service-level agreement (SLA) compliance are absent, risking an optimistic interpretation of automation.
- **Internal validity.** With XML simulators standing in for ERPs and product information management (PIM) systems, data freshness, latency, and failure modes are not controlled for; it is unclear whether observed behavior would persist under real integration constraints.
- **External validity.** The ontology subset omits common commercial terms (rebates, penalties, Incoterms granularity). Findings may not generalize to sectors with complex contractual logic or to multi-tier supply chains.
- **Reliability.** Results are shown on one agent platform without replication across stacks or repeated trials; no metrics (success rates, cycle times, negotiation convergence) are reported.
- **Conclusion validity.** Claims of "automation" or "agent effectiveness" lack quantitative evidence; absence of baselines prevents effect-size estimation.

Implications for design. Three engineering priorities emerge: (1) *Oversight by design*—codify approval thresholds, escalation paths, and audit logs as first-class artefacts; (2) *Commercial completeness*—extend ontologies to cover payment/delivery terms, penalties, rebates, volume pricing, and exception clauses; (3) *Operational realism*—replace simulators with live ERP/PIM integrations, strengthen discovery beyond a minimal registry (semantic matching, reputation/risk signals), and introduce robustness features (timeouts,

retries, compensation). A natural extension is to couple purchasing with *financial supply-chain* considerations (cash-flow-aware negotiation), contingent on deeper accounting integration and richer policy vocabularies.

Discussion. Taken together, the demonstration surfaces a structural tension between semantic rigor and operational completeness: the agents reason over well-scoped purchasing artefacts, but without multi-round negotiation, exception handling, and explicit human control the automation stays closer to decision support than to production-grade delegation. The value is therefore best framed as *semantic enablement* that prepares the ground for later operationalization along the three design priorities above (oversight by design, commercial completeness, operational realism), rather than as immediate end-to-end automation.

7.3 Phase I: Matching Service Descriptions in a Context-Aware Environment

These results answer **RQ I.3**: DYNAMOS is established by demonstration (a peer-reviewed prototype design; the field trial is reported in the Licentiate thesis), with the August 2005 field demonstration delivering proactive, context-aware recommendations on the era's smartphones, whose resource limits left O.I.3.2 only partially achieved (Table 4; Section 7.8).

Purpose and scope. This section evaluates how the DYNAMOS main artefact (Table 5) addressed the problem of delivering context-relevant services to mobile users in dynamic environments (Section 1.2.3, Section 4.3). The focus is a direct assessment against the stated objectives: *context modeling* (O.I.3.1), *automated matching with minimal resource overhead* (O.I.3.2), and *collaborative annotation support* (O.I.3.3). Evidence is drawn from the August 2005 field demonstration during a sailing competition (Section 4.3.2) and the controlled performance measurements of context updates, data access, and matching reported within it (Section 4.3.2.2) rather than re-describing them here.

Evaluation against objectives

O.I.3.1 Context Modeling. The ontology-based model linked relatively static profiles with dynamic context variables (location, activity, time). In practice, it was expressive enough to filter content without overwhelming devices; it required only a one-time specification of activities and their interests. However, the simple activity model offers only coarse cross-activity granularity. *Assessment: Achieved, with scope constraints in cross-activity granularity.*

O.I.3.2 Matching Mechanism with Minimal Resource Overhead. Server-side filtering and compact result sets were effective: throughput for local operations was high, and client payloads remained small (matching relevance was not measured; the field trial only showed matched services and notes reaching users as intended). End-to-end costs, however, were dominated by remote calls: remote invocations reduced throughput by two to almost five orders of magnitude, depending on the call and the network, making pre-filtering mandatory. On-device robustness and energy use were fragile under realistic conditions: client memory leaks (partly JVM bugs), GPS/Bluetooth churn, and concurrent sensor plus data flows produced crashes and noticeable battery drain. Scaling indicated the need for spatial and attribute indexing before increasing the catalog size beyond $\sim 10^4$ items (estimated; measured on about 10^3). *Assessment: Partially achieved—server-side matching was functionally adequate and fast, but relevance was not measured and efficiency was brittle on mobile clients.*

O.I.3.3 Collaborative Annotation Support. User-generated notes and ratings were treated as first-class, matchable artefacts and propagated via an event stream. This enabled timely peer-to-peer updates alongside commercial services. Annotation volume and curation were not evaluated; the measured constraint was that RMI calls returning large result sets became a bottleneck, and the prototype provided no moderation or provenance controls. *Assessment: Achieved for functionality; scalability and curation require hardening.*

Key findings from the demonstration

- **Integration, not algorithms, dominated performance:** RPC/RMI boundaries were the primary bottleneck; relocating filtering and matching to the server was decisive.
- **Mobile constraints were first-order:** In high-wind, low-attention conditions, single-tap interactions and defensive client design were essential; device/JVM instability limited reliability independent of server correctness.

- **Simple context worked on-device:** An activity-centric model delivered relevant filtering but limited fine-grained transitions across activities.
- **Peer content added value but increased noise:** Treating annotations as content broadened what could be matched; indexing and curation were identified, not measured, as the prerequisites for larger result sets.

Design principles derived

(1) Filter at the data source to minimize network and client load; (2) use event-driven propagation for context and peer updates; (3) keep context models minimal and activity-centric for mobile viability; (4) design for divided attention: single-action interactions and clear defaults; (5) conserve energy via aggressive sensor scheduling, backoff, and batching; and (6) add spatial and attribute indices before scaling catalog size or user count.

Limitations to validity

- **Scenario specificity:** A single activity (sailing) and a narrow cohort limit ecological validity for other domains (e.g., commuting, indoor venues).
- **Platform vintage:** Results are confounded by 2005-era device/JVM faults; some failures reflect platform immaturity rather than architectural limits.
- **Middleware choice:** Performance conclusions reflect Java RMI; alternate transports (e.g., message queues or HTTP batching) could shift—but not erase—bottlenecks.
- **Scale extrapolation:** Behavior beyond $\sim$ 1,000 items is inferred from profiling and indexing arguments, not demonstrated at full scale.
- **Battery measurement:** Energy impact was assessed qualitatively; absence of instrumented power traces weakens efficiency claims.
- **Annotation bias:** Peer contributions were opportunistic and unmoderated; findings on collaborative value may not generalize to adversarial or low-participation settings.

Discussion

The demonstration indicates that context-aware matching can be delivered with a lightweight ontology and server-side selection (its relevance was not formally measured), but only when integration overheads are tightly controlled and mobile fragility is anticipated. The critical path is architectural plumbing: batching, indexing, and event-driven updates mattered more than marginal tweaks to the matching logic. For progress toward production, three refinements appear most impactful: (i) push more computation to indexed, cache-aware servers with coarse-grained client synchronization; (ii) formalize annotation governance (deduplication, provenance, ranking) to improve signal quality; and (iii) extend the context model with transitional states that preserve simplicity while supporting multi-activity journeys.

Novelty of Phase I as a whole, assessed. Judged against the state of the art of its time, Phase I is *incremental*: its value is integration and feasibility rather than new formalism, and its components were previously published and examined. Its most durable result is a negative one—ontology-backed agents could orchestrate services once context was available, but the agent technologies used in this research program did not converge with mainstream Web standards—and it is precisely this gap that motivated the program's data-centric turn in Phase II.

7.4 Phase II: Assisting Clinical Decision Support with ML and Software Agents

These results answer **RQ II.1** quantitatively: measured against historical coded documentation, the ML/IR pipeline reaches 53–66% top-1 and about 90% top-8–10 accuracy on real hospital data, with sub-second query latency on commodity hardware for databases of up to 10^8 episodes (the largest scales measured on synthetic enlargements of the real data, on which accuracy was not measured)—a measurement against a reference standard, subject to the stated ground-truth caveats (Table 4; the level of evidence is discussed in Section 7.8).

7.4 PHASE II: ASSISTING CLINICAL DECISION SUPPORT WITH ML AND SOFTWARE AGENTS

Purpose and scope. This section evaluates and discusses the demonstration of the *ML/IR-assisted clinical decision support* main artefact (Table 5), assessing it against objectives O.II.1.1–O.II.1.4 and situating it within the problem setting and clinical coding workflow described in Section 1.3.1 and Section 5.1.

Evaluation against objectives.

O.II.1.1 Robust ML/IR Architecture (Arch). *Artefacts and approach:* A vector-space information retrieval (IR) pipeline using TF–IDF weighting and cosine similarity (a linear kernel; Section 5.1.1.3) was adapted to clinical coding (ICD-10, NCSP), augmented with hierarchical code relationships; the MAS, decomposed into Crawler/Integrator/Commander/Aggregator agents, coordinated demand-driven retrieval and scoring. *Findings:* The IR approach handled extreme sparsity and high dimensionality; a combinatorial analysis (on the order of 10^{230} possible fact combinations) motivated favoring IR over dense neural models in this setting. The bounded similarity scores were designed to yield rankings interpretable in coder workflows (not tested with coders). *Limitations/Threats:* Current weighting treats repeated codes uniformly and relies on exact or near-exact code matches; broader clinical entities (labs, meds, vitals) were not modeled, potentially underestimating real dimensionality. Later sparse/attention models have since partly superseded the era's neural-infeasibility argument (Section 1.3.2). *Implications:* Domain-aware IR with ontology hooks is a pragmatic architectural fit for fragmented, sparse EHR data; future work should explore ontology-backed expansion and non-uniform term weighting.

O.II.1.2 Clinically Viable Recommendation Accuracy (Acc). *Target:* >50% first-proposal; >90% top-10. *Results:* Achieved 53–66% first-proposal accuracy and ∼90% accuracy within 8–10 proposals, meeting the first-proposal threshold and approaching the top-10 threshold. The *SHOULD-occur* fetching strategy consistently outperformed *MUST-occur*, especially under imperfect documentation. *Limitations/Threats:* Accuracy was computed against historical documentation assumed as ground truth despite reported coding error rates of roughly 10–17% (11.4% coding discordance in a French DRG audit [340]; 13–17% principal-diagnosis errors at detailed code level in Swedish register validations [342]) and low inter-coder agreement on detailed ICD-10 codes [337]; metrics emphasize common cases and may under-capture rare-but-critical conditions; clinician usability/trust feedback was not collected. *Implications:* Results indicate practical viability for coder-assist triage; future evaluation should incorporate clinician-in-the-loop studies, confidence estimates, and utility on low-prevalence edge cases.

O.II.1.3 Robustness to Data-Quality Defects (Rob). *Design:* Comparative tests on higher- vs. lower-quality datasets and on perturbations (removed/incorrect/novel codes). *Findings: MUST-occur* degraded rapidly under data defects; *SHOULD-occur* maintained acceptable performance, showing graceful degradation aligned with real-world data issues [337, 338]. *Limitations/Threats:* Experiments predominantly removed single facts; real episodes may have multiple concurrent defects and temporal uncertainty. Robustness beyond diagnoses/procedures (e.g., noisy labs) remains untested. *Implications:* Flexible matching and redundancy are essential for production deployments across heterogeneous hospital data estates; a principled uncertainty layer would further aid risk-aware use.

O.II.1.4 Real-time Scalability on Commodity Hardware (Scale). *Target:* Sub-second responses on datasets up to 10^8 episodes; near-linear scaling. *Results:* Sub-second ($\leq$0.5 s) query times were observed up to 10^8 episodes on standard hardware, with query latency growing no faster than database size (Figure 17; scaling with system diversity was assessed by expert evaluation, not measured); the demand-driven MAS was designed to avoid the overhead of continuous polling (no polling configuration was measured). *Limitations/Threats:* Large-scale tests were measured on synthetic enlargements of the real data; failure modes (agent crashes, partitions), security hardening, and contention under production loads were not fully characterized. *Implications:* The architecture is performance-feasible on commodity stacks; productionization should add resilience testing, privacy/security controls, and capacity planning under bursty workloads.

Positioning with respect to Design Science Research guidelines. Relevance & design as an artefact: The six-level abstraction from patient facts to DRG, the IR model, and the MAS are purposeful artefacts addressing coder-assist pain points. **Rigor:** Formal data-space characterization and systematic retrieval/evaluation ground the design. **Design evaluation:** Objective-aligned tests (accuracy, robustness, latency/scale) provide empirical evidence. **Contributions:** Quantifies combinatorial bounds for healthcare coding; adapts IR to clinical hierarchies; demonstrates agent-based integration for fragmented estates. **Design**

as a search process: Alternative fetching strategies (SHOULD- vs. MUST-occur) were compared against the accuracy and robustness objectives (Acc, Rob). **Research communication:** Artefacts and evaluation are documented to inform IS and health informatics audiences.

Limitations and external validity. *Demonstration vs. validation.* The objective-aligned tests above constitute *quantitative validation against a reference standard* (in the sense of Section 1.1) over the coded clinical item sets rather than a controlled clinical trial. Where a reference does exist, it is used: accuracy and robustness are measured *quantitatively against historical coded documentation as the ground-truth reference* and latency on enlarged databases (Figure 17)—with the explicit caveat that this reference itself carries reported coding-error rates of roughly 10–17% [340, 342] and low inter-coder agreement [337]—and the results come from the peer-reviewed study [14], which builds on [11, 12, 13]. The missing elements are a comparison against an alternative method (the only comparison reported here is the internal SHOULD- vs. MUST-occur contrast) and, above all, a *controlled, clinician-in-the-loop* study of utility and trust; running such a trial is deliberately out of scope at this stage (Section 1.1), and both are carried into future work.

Practical implications. Hospitals can adopt the MAS+IR pattern to integrate dispersed systems and deliver interpretable, ranked top-k coder assistance with commodity infrastructure. Priorities for deployment: (1) ontology alignment and code normalization, (2) uncertainty-aware UX for coders, (3) operational hardening (security, audit, fault tolerance), and (4) performance tuning against real workload traces.

Summary. The Phase II artefacts largely meet their stated objectives: they are architecturally appropriate for sparse, fragmented data (Arch), reach about 90% top-8–10 accuracy against historical coded documentation, approaching the top-10 target and pending clinical validation (Acc), remain resilient under predominantly single-fact data-quality defects (Rob), and keep sub-second latency on commodity hardware up to 10^8 episodes, the largest scales measured on synthetic enlargements of the real data (Scale). These results substantiate the feasibility of agent-supported IR for clinical decision support and outline a clear modernization path toward production readiness.

Discussion. MAS+IR is a pragmatic fit for coder-assist in fragmented hospital estates: it favors bounded, explainable retrieval over generative or opaque end-to-end models, trading some semantic expressivity for controllable latency and governance. Demand-driven agent orchestration localizes failures and cost but adds coordination overhead that must be engineered (timeouts, retries, backpressure). By design, it is expected to be more robust under documentation defects than rule-based CAC (not compared empirically here); relative to neural pipelines, it is expected to impose lower data and governance burdens but to capture fewer rare, compositional patterns. Because real value depends on "plumbing"—code normalization, audit, access controls, observability, and exception handling backed by IT service management (ITSM)—piloting as a sidecar to coder workflows is sensible. The pattern generalizes to other sparse, high-cardinality decisions (e.g., order sets, procedure suggestions) given adequate ontologies and recalibrated weights. Near-term refinements include calibrated uncertainty and brief "why/why-not" rationales, light hybridization with LLMs for clustering/justification (not final decisions), and reliability testing to harden multi-agent coordination. In the terms of Section 2.7, this is how the Phase II artefact answered the limitation of its era—turning bespoke, opaque statistical pipelines into bounded, explainable, defect-robust analysis on commodity hardware—while remaining hand-built and domain-tuned enough that genuine self-explanation and governance had to wait for the LLM agents of Phase III.

Novelty, assessed. The Phase II contribution is a robust *combination* rather than a new algorithm: IR/recommender machinery embedded in a multi-agent system over coded DRG/CaseMix data, evidenced by 53–66% top-1 / ∼90% top-8–10 accuracy, the defect-robust SHOULD-over-MUST fetching result, and sub-second queries at 10^8 episodes (measured on synthetic enlargements of the real data) on commodity hardware. That combination remains defensible, but the era's neural-infeasibility argument has since been partly superseded (Section 1.3.2), and the method is information retrieval over coded tokens—not free-text natural-language processing.

Forward link. Phase II's bespoke ML/IR models can neither explain their reasoning in domain terms nor keep human experts in command as the systems scale; self-explaining decision logic plus governance, audit, and human-in-command structures around statistical learning are precisely what the Phase III artefacts (the NordDRG AI Benchmark, CCL, and HADA) take up.

7.5 Phase III: Human-AI Agent Decision Alignment (HADA)

These results answer **RQ III.1** by demonstration and, through the role-bound human sign-off (RACI/ITSM) answered under SRQ.III.1.3 (Section 8.4), contribute to **RQ III.3** the accountability pattern on which CCL's governed change process also relies (Section 7.7): HADA met its six predefined design objectives (two of them partially; the published coverage matrix [28] reported full coverage, and is re-graded here per objective) on the `getLoanDecision` workflow in a banking sandbox; because no comparable governed system exists, the evidence is prototype demonstration with qualitative positioning rather than a quantitative baseline (Table 4; Section 7.8).

HADA operationalizes human–AI decision alignment by coupling natural-language interaction among stakeholders with governed, tool-mediated algorithmic decisions. The banking `getLoanDecision` workflow serves as the primary evaluation setting, exposing tensions among risk, compliance, ethics, customer value, and operational efficiency. Rather than pursuing full automation, HADA establishes a repeatable socio-technical loop where issues (e.g., postcode/ZIP-code features) are surfaced, routed to accountable roles, and resolved under audit.

The objectives evaluated below are precisely the governance properties—auditability, accountability, and value alignment—that the agent frameworks compared in Section 2.5.6 leave unaddressed (ReAct, Toolformer, Reflexion, AutoGen, MetaGPT; and the governance-oriented Tropos/Secure Tropos [78, 256], which model goals primarily at design time without per-decision run-time binding, and Constitutional AI [258], which aligns values at training time rather than as run-time control). The evaluation therefore tests not whether HADA can match those frameworks on capability, but whether it adds the governance layer they omit; the per-objective outcomes (and their stated limitations) state explicitly that this is a single-workflow, prototype-level demonstration. The per-objective limitations are framed as residual risks, in the spirit of goal-oriented risk analysis [378], rather than as resolved guarantees. No formal goal model (e.g., Tropos) of the workflow was constructed, so the OKR-to-algorithm refinement is demonstrated, not verified.

Evaluation against objectives

HADA was evaluated against its six solution objectives (O.III.1.1–O.III.1.6; Section 1.4.1.4) by analyzing the `getLoanDecision` workflow and associated governance flows. Each outcome is a qualitative judgment on that scenario: an objective is rated *Partially achieved* when a part that defines the objective was not exercised in the sandbox (sustained cross-quarter operation for O.III.1.4; load and propagation timing for O.III.1.5), that part being named under *Limitation*; under an *Achieved* objective, the *Limitation* names extensions beyond the evaluated scope.

O.III.1.1 Natural-language interaction across planning horizons. In the role-played user-story scenarios (Section 6.1.1.2), stakeholder roles (C-suite, risk, data science, audit, customer) engage the decision context through constrained NL dialogues that map to structured actions and evidence, supporting both strategic clarifications and daily exception handling. *Outcome: Achieved (within the evaluated workflow). Limitation:* longitudinal validation across full annual cycles remains future work.

O.III.1.2 Multi-dimensional alignment (targets & values). Optimization logic is bound to quantitative OKRs and machine-readable values/policies, with a traceable (logged, but not formally verified) chain from principles to actions; postcode/ZIP-code conflicts, once noticed by a human owner, are escalated to ethics review with traceable rationale. *Outcome: Achieved for the scenario in the auditable sense of O.III.1.2 (the chain from principles to actions is logged and traceable end-to-end); formal verification of the OKR-to-algorithm refinement is deferred. Limitation:* broader value-ontology coverage and quantitative conformance metrics across multiple policies are deferred.

O.III.1.3 Reference architecture for hierarchical, hybrid AI systems. The stack separates strategy, coordination, and execution; LLM agents and classical models/tools coexist under a common governance surface. Integration is protocol-agnostic (MCP adapters in the prototype; A2A in the reference design) so that, by design, model/tool swaps do not alter governance flows (running on different tools and models is yet to be shown). *Outcome: Achieved. Limitation:* more heterogeneous backends would further demonstrate portability.

O.III.1.4 Stakeholder alignment across time-scales. Repeatable elicitation and reconciliation patterns propagate approved changes through the pipeline. *Objectives & Key Results (OKRs)* are used as

the bridge from yearly–quarterly strategy to algorithm-level objectives (constraints, rewards, and guardrails), ensuring strategic intent flows into model behavior. Escalations and approvals are anchored in organizational roles (RACI) and connected ITSM workflows exposed as agent tools via Tools/MCP. *In the evaluated prototype, no autonomous agents perform (i) OKR→single-algorithm linkage analysis or (ii) value-misalignment detection (e.g., ZIP-code usage): the Ethics Manager Agent can record a sensitivity flag and open the escalation ticket, but noticing the misalignment and initiating that flag are explicitly assigned to human owners. Outcome: Partially achieved. Limitation:* yearly/quarterly cadences were exercised in design and in the scripted role-play scenarios; sustained cross-quarter operation in a live organization is pending replication.

O.III.1.5 Scalability, auditability, and near-real-time propagation. Immutable, line-addressable logs capture decisions, policy changes, explanations, and escalations for audit; policy edits propagated through the pipeline in the testbed scenarios (propagation time was not instrumented). *As under O.III.1.4, noticing and initiating action on OKR-to-algorithm linkage and value misalignment remain human responsibilities. Outcome: Partially achieved. Limitation:* no load tests across thousands of agents; latency/throughput under production contention remains to be measured.

O.III.1.6 Framework-agnostic, policy-driven design. Governance is expressed in open, tool-facing contracts so strategic edits flow to heterogeneous runtimes; model-/framework-specific logic is confined to adapters, avoiding lock-in. *Outcome: Achieved (by design and prototype). Limitation:* wider coverage of orchestration frameworks would strengthen generality.

Threats to Validity and Generalizability

Limited real-world scope. Evaluation is centered on a single domain (retail banking), one decision workflow, and a specific regulatory context. Replication in healthcare, energy, and logistics is required to establish external validity, stress-test adaptability, and characterize trade-offs at different risk levels.

Architectural alternatives. HADA occupies one point in a broader design space that includes rule-first governance layers, formal inter-agent alignment protocols, XAI-centric supervisors, and other human-in-the-loop (HITL) pipelines. Domain requirements, organizational culture, and technical estate will shape best-fit choices; HADA should be considered contributory rather than definitive.

Absence of a quantitative baseline—by nature of the contribution. A controlled, like-for-like baseline comparison is not available for HADA, and this is a property of the problem rather than an omission in the evaluation: to the author's knowledge, no production decision system—and none of the agent frameworks surveyed in Section 2.5.6 and listed above—implements the auditable, OKR-to-algorithm governance layer that HADA contributes. The evaluation is therefore framed as a *demonstration* of feasibility, positioned qualitatively against what those frameworks leave unaddressed (auditability, accountability, value alignment), and reported through the peer-reviewed lineage of the artefact [26, 27, 28]; the two predecessor papers did contrast their scenarios qualitatively with prompted-LLM and custom-GPT configurations, but those are ungoverned chat set-ups, not a like-for-like governance baseline. Building and deploying a comparable governed system, as a quantitative baseline would require, is left to future work, in keeping with the scope set in Section 1.1.

Scope of automation and human-in-the-loop boundaries. HADA intentionally preserves human control over key gates: policy approval, model retraining/recalibration, and ethics adjudication. The system's role is to surface alignment issues, assemble evidence, and route decisions to accountable owners via ITSM workflows connected through Tools/MCP. This design maintains responsibility, auditability, and controllability in ethically sensitive contexts. HADA thus assigns accountability for which version of the decision-making model is deployed and used, but the governed review of a model change by others—typed diffs, risk-tiered gates, and certified reviewers, as CCL specifies (Section 6.3)—is not demonstrated: the data scientist's retrained model enters use through the RACI approval only. The same holds for the policies on which run-time behavior also depends (OKR thresholds, value constraints, and RACI bindings): they are held in catalogs and changed through tickets, not through CCL's governed, reviewed change process. A complete system would place these policy artefacts, like the model, under CCL-style version control, so that HADA governs their use and CCL their change.

Modeling limitation: one decision algorithm per workflow. The current binding of each governed workflow to a single decision algorithm (`getLoanDecision`; Section 6.1.1.6) simplifies ownership and deployment but fragments multi-product negotiations (e.g., mortgage + insurance + investments). Addressing

coupled decisions requires a coordinating layer (or composite agent) that manages interdependencies and shared constraints across products while retaining per-algorithm accountability and traceability.

Discussion

HADA's contribution is best viewed as operational governance rather than yet another orchestration pattern: it makes strategy (*OKRs*) and values (*policies, ethics*) first-class, tool-facing artefacts and binds them to the lived workflow of a decision like `getLoanDecision`—in the terms of Definition 3.2.5, the objective binding $\mathcal{O}$ and the value constraints $\mathcal{N}$, with the RACI matrix as $\mathcal{R}$ and the decision ledger as $\mathcal{L}$. The evaluation suggests that this binding is feasible in practice—natural-language dialogues can be constrained into evidence-carrying actions; RACI/ITSM anchors keep ownership visible; and line-addressable logs afford traceability for audit. At the same time, two tensions are evident. First, the architecture outsources "noticing" to humans (e.g., postcode features), which preserves accountability but limits responsiveness at scale. Second, the current one-algorithm-per-workflow binding simplifies control surfaces yet impedes reasoning over coupled products and shared constraints, where cross-model externalities matter as much as per-model performance.

For wider use, three concrete gaps remain. Measurement: alignment is asserted, but proving it over time still needs simple, consistent numbers (e.g., coverage, drift, fairness). Breadth: the prototype works, but it must be shown to run on different tools, models, and regulations—under real load. Autonomy limits: light automation (alerts, gathering evidence, pushing approved policies) is lower-risk, whereas fully automatic detection and OKR-to-model linking need stronger safeguards to avoid false alarms, shortcuts, or ethical mistakes.

Beyond these three gaps, the next steps are to introduce a combined view for bundled products, translate policies into concrete checks, and track "alignment" as an operational service-level objective (SLO) in the same ITSM queues that already handle risk and incidents.

7.6 Phase III: Evaluation of the NordDRG AI Benchmark

These results answer **RQ III.2** quantitatively: scored by exact match against published gold answer keys (for Grouper, the official grouping decision and triggering rule row), top models master rule-level Logic (best 13/13) while exact, traceable grouper emulation remains hard (best 7/13). This reference-standard comparison is the strongest empirical evidence in the thesis (Table 4; the level of evidence is discussed in Section 7.8).

As argued in Section 2.6.3, this domain-specific evaluation is the vertical complement to general-purpose, horizontal LLM suites such as HELM and to the agent-/task-evaluation benchmarks discussed there: it measures the regulated CaseMix/DRG rule reasoning and faithful grouper emulation that broad leaderboards leave unmeasured. To the author's knowledge, this is the first *open, rule-complete* benchmark that requires *deterministic grouper emulation*—matching both the final DRG (`drg_nat`) and the triggering rule row (`drg_logic.id`) under exact match. This is a different task from clinical-note DRG *classification*, where prior LLM work predicts a code from free-text discharge summaries (e.g., DRG-LLaMA [312] and the reinforcement-learning DRG coder of [74]); those systems predict the label, whereas this benchmark asks the model to *reproduce the rule system that produces it*. Deterministic exact-match scoring is also an evaluation-reliability property: because correctness is decided against the official grouper rather than by an LLM judge, the scores are free of the judge biases noted in Section 2.6.3 (contamination is not excluded, since the tasks and gold keys are public; cf. Section 1.4.4). The strictness is deliberate—in reimbursement logic no deviation can be tolerated, so a partially correct grouping scores zero—but it also means the scores understate intermediate reasoning competence. Step-wise conformance scoring and partial-credit tracing are therefore carried into future work (Section 7.8).

Within Phase III this artefact is of a different kind from the governance architectures HADA and CCL: it is an *LLM benchmark*, not an agent architecture, and as such it supplies the *quantitative, comparative validation* that the architectural artefacts cannot offer. Its reference standard is unambiguous—published gold keys, and for Grouper the official NordDRG grouper output—so model performance is scored by strict exact match against that ground truth across contemporary endpoints (the "baseline differentiation" objective below). This section evaluates the benchmark contribution [24]; the earlier arXiv release [23] scored question-level tasks against a published answer key.

Evaluation objectives and design. The NordDRG AI Benchmark [24] is evaluated against the three objectives *O.III.2.1–O.III.2.3* of Section 1.4.2.4 (artefact construction, benchmark operationalization, and baseline evaluation), through three criteria: (i) coverage & fidelity of the artefacts; (ii) extensibility and long-term sustainability; and (iii) baseline differentiation across LLM endpoints available in August 2025. All artefacts, answer keys, and scoring scripts are openly released to support independent reruns; reproducibility of the model scores themselves is bounded by single runs on consumer endpoints whose model versions cannot be pinned (Section 6.2.2.1).

The benchmark exposes two complementary suites with exact-match scoring: a 13-task *Logic* suite that exercises rule-level reasoning over the definition tables and governance manuals, and a 13-task *Grouper* suite that requires *full specification emulation*. *Logic* answers are order-invariant, de-duplicated code sets; *Grouper* answers must match *both* the final DRG (`drg_nat`) and the triggering rule row (`drg_logic.id`).

Evaluation against objectives. Consistent with the DSRM and the design-evaluation methods of Hevner et al. [59], the benchmark is assessed against the objectives via (i) a *descriptive* inspection of the artefacts and (ii) an *experimental* baseline run (each model answering the same fixed items, scored against the answer keys; Tables 16 and 17).

O.III.2.1 Artefact construction: coverage and fidelity. A.III.2.1 mirrors the production NordDRG workbook (∼20 interlinked sheets; `drg_logic`, `dg_feat`, `proc_feat`, CC/MCC exclusions, age/sex splits, national activation). No lossy transformations were applied; all rule-bearing columns are preserved. The Logic tasks (Table 13) exercise table reading and joins; the Grouper tasks (Table 14) require end-to-end execution of the official control flow (including `ORD`, MDC entry, OR/PROCPRO, CC/MCC, activation flags).

O.III.2.2 Benchmark operationalization: extensibility and sustainability of the suites. Schema-stable identifiers (`DRG`, `ICD`, `NCSP`, `ORD`, ...) are consistent across yearly and national releases, enabling drop-in updates without code changes. In practice, adding a version amounts to placing the official workbook and manuals in a dated repository folder; existing prompts and scoring continue to work. Maintenance follows an open workflow (GitHub issues/PRs): adding a task requires one prompt row (A.III.2.3 for Logic, A.III.2.4 for Grouper) and a matching gold-answer row—no source edits.

O.III.2.3 Baseline evaluation: baseline differentiation. Table 16 shows a clear separation on the 13 *Logic* tasks: top-tier models reach ceiling accuracy (*GPT-5 Thinking*, *Opus 4.1*: 13/13; *o3*: 12/13), while mid-tier and trailing endpoints miss joins and governance-aware filters (*GPT-5 Thinking Mini*, *o4-mini*: 8/13; *GPT-5 Fast*: 6/13; others ≤5/13). The *Grouper* suite (Table 17) is substantially harder under strict exact match of `drg_nat` *and* `drg_logic.id`: *GPT-5 Thinking* solves 7/13, *o3* 6/13, *o4-mini* 3/13, *GPT-5 Thinking Mini* 1/13, and the remaining endpoints 0/13. Inspection of the incorrect outputs suggests that failures concentrate on `ORD` priority and CC/MCC exclusions.

The largest gaps occur on cross-sheet chaining (procedure- and diagnosis-property linkage). On strict *Grouper* emulation, in the inspected outputs, misses often predicted the correct `drg_nat` but failed the exact `drg_logic.id` requirement (a per-field breakdown is not tabulated here). To the author's knowledge, at the time of the benchmark runs (August 2025) this was the first public report of an LLM partially emulating the complete NordDRG grouping logic with exact matches on both fields.

Minority-language robustness. The benchmark deliberately includes FI and FI-EN excerpts and Logic tasks to test minority-language grounding and bilingual transfer. In the reported runs, the Finnish-term tasks (Logic-3, Logic-5) separate models: strong systems resolve Finnish clinical terms to the correct DRG sets, while weaker ones do not; a controlled FI versus FI-EN comparison is left for future work.

Methodological lessons. Governance-grade benchmarking benefits from: (i) prompt templates that constrain outputs to normalized sets; (ii) *exact-match* keys (sets for *Logic*; (`drg_nat`, `drg_logic.id`) for *Grouper*); and (iii) version pinning of artefacts. Lightweight reference agents (*LogicAgent*, *GrouperAgent*; Section 6.2.1.7) emit de-duplicated sets and a concise evidence trace (sheet/row IDs, predicate outcomes) without exposing hidden chain-of-thought.

Operational throughput under consumer subscriptions. Running the full suite on standard, consumer-grade endpoints surfaced throughput differences unrelated to accuracy: some providers exhausted credits

mid-run while others completed uninterrupted. For practitioners, the relevant objective function is not price-per-token alone but *price & time-to-result*, including quota predictability and run-management overhead.

Implications for healthcare AI. Findings indicate that, in this single-run snapshot, top-tier LLMs solved all 13 rule-level NordDRG *Logic* tasks but only *partially* emulated the governed grouper (best 7/13). By coupling a rule-complete release with exact-match tasks and open scoring, the benchmark offers a reproducible yardstick for regulators, payers, and developers.

Discussion. The benchmark rests on three properties: (i) *rule-complete* artefacts that mirror the production workbook without lossy simplifications; (ii) *exact-match* scoring that forces specification-faithful behavior (set equality for Logic; joint match on the final DRG and the triggering rule row for Grouper); and (iii) *open* releases that permit independent reruns and community extension.

Three caveats temper generalization. First, the benchmark targets a single, complex DRG family; while identifiers and templates are version-stable, external validity still hinges on replication across additional national variants, yearly updates, and adjacent CaseMix systems. Second, strict exact-match scoring is deliberately unforgiving: it exposes real gaps in governed execution but can obscure partial progress without complementary *step-wise* conformance metrics. Third, each endpoint answered each of the 13 items per suite once, so the results are a first-cut snapshot without variance estimates, and differences of one or two items between models should not be read as significant (Section 6.2.2.1). Next steps are: pair the current keys with a reference interpreter that emits auditable step traces; diversify prompts via templated paraphrases and adversarial counterfactuals; add fairness and minority-language slices as first-class scorecards; and report *cost and latency* alongside accuracy as a benchmarked SLO.

7.7 Phase III: Evaluation of the CCL

These results answer **RQ III.3** by demonstration against a reference standard plus ablation (single- vs. two-agent): CCL's governed change process—typed atomic diffs, risk-tiered CI gates, and an immutable ledger—is exercised on real Forum tickets (the gate criteria were applied by manual review; automated CI execution was not run and the ledger steps were not executed; Table 20), with the outputs judged against that reference standard, the expert change implemented in NDMS for each ticket, and compared against a single-agent workflow. As the governance *process* itself has no established baseline, the evidence is demonstration with an ablation rather than a controlled comparison (Table 4; Section 7.8).

Where the agent frameworks surveyed in Section 2.5.6 leave rule *change* ungoverned, the CCL evaluation below tests precisely that missing layer: whether agent-proposed changes to a regulated decision-making model can be made auditable and kept under accountable human control.

This section evaluates whether the CCL instantiation on NordDRG meets its objectives under regulated constraints. The evaluation proceeds in three parts, aligned with research questions (Section 1.4.3.2) and objectives (Section 1.4.3.4). The first two parts address objective O.III.3.2 (agent-operable model-of-record), and the third addresses O.III.3.1 (governed workflow) and O.III.3.3 (LLM agents under constraints); of the measures named under O.III.3.3, reviewer acceptance is reported, whereas CI pass rates, error severity, and reviewer effort were not collected in the demonstration (automated CI was not executed). These experiments assess the prototype artefacts, not NordDRG processes. Acceptance/rejection labels record whether a proposed change reproduced the expert change implemented in NDMS for the same ticket (the reference standard) and should not be interpreted as judgments about official NordDRG change procedures.

The three parts, drawing on the artefacts and figures of Sections 6.3.1 and 6.3.2, are the decision-tree abstraction (Section 7.7.1: whether a canonical tree/DAG view is agent-operable and line-addressable, semantically faithful to the official logic, and human-reviewable via decision-tree slices—structural fidelity, scale, legibility), the minimal, as-is preparation of NordDRG for CCL (Section 7.7.2: pass-through integrity, operability under CCL, and an implicit external check), and the Contributor/Reviewer agents as first-line participants (Section 7.7.3: acceptance on manual review, meaning that a proposed change reproduced the expert change implemented in NDMS for the same ticket—automated CI was not executed—with humans in charge of the final decision on which changes are accepted).

7.7.1 Evaluation of the NordDRG Decision-Tree Abstraction

Objective and scope. This study tests whether materializing the NordDRG specification as a canonical decision-tree/DAG is *fit for CCL*: (i) agent-operable and line-addressable for safe, atomic diffs; (ii) semantically faithful to the official logic; and (iii) practically reviewable via sliceable, explanation-friendly visuals.

Design. This study ran structural checks and manual reviews on the parsed structure: (a) schema/typing validation for predicates and labels; (b) scale profiling (features, leaves, root→leaf paths, decision-point counts); and (c) legibility assessments on SVG slices (10/25/100 paths) to gauge human reviewability. Figures and metrics from Section 6.3.2.2 ground these checks.

Measures. (i) *Fidelity*: path-wise equivalence on a certification subset; (ii) *Scale*: counts of features, distinct leaves, root→leaf paths, and decision points (first layer vs. full unrolling); (iii) *Legibility*: the author's judgment of 10/25/100-path SVGs for review tasks; (iv) *Diff locality*: share of edits that remain well-typed and confined to small path segments. Demonstration baselines report 16 features, 2,752 distinct leaf labels, 8,867 root→leaf paths, 53,848 first-layer decision points, and 40,453,620 decision points when fully unrolled.

Adequacy criteria. The abstraction is *adequate* if it (a) preserves semantics on the certification subset; (b) enforces typed predicates/labels; (c) yields stable node addresses for PR diffs; and (d) supports reviewer-legible slices at the 10–25-path scale used in practice.

Threats to validity. Unrolling DAG structure into path-wise trees can induce redundancy; equivalence checks are limited by available certification cases; legibility ratings may vary by reviewer expertise and display medium.

Outcome reporting. This study reports the structural counts given under Measures (Section 6.3.2.2); no path-wise fidelity result on a certification subset and no diff-locality figure (Measures (i) and (iv)) are reported for this demonstration; schema/typing validation was applied as described under Design, and the legibility of the 10/25/100-path slices was judged informally by the author—no multi-rater study was conducted. Confidence intervals, round-trip tallies, and inter-rater agreement on legibility are left to future work.

7.7.2 Evaluation of Preparing NordDRG for CCL

Objective and scope. This study assesses whether a *minimal* preparation of the official NordDRG release—packaging the native, custom-format definition tables as line-addressable files under version control with grounding documentation—is sufficient to enable PR-centric governance without introducing semantic drift. As justified in the artefact discussion (Section 6.3.1.4), this study intentionally retained the native format as the *authoritative model-of-record*; a wholesale conversion to a standard decision-tree serialization was not adopted because it offered no net governance benefit while risking precedence/ordering regressions, diff instability, and tooling mismatch. A compiled tree/DAG is used only for analysis, visualization, and agent-facing tooling, never as the model-of-record (Section 6.3.2.2). The demonstrations evaluated here are those of Sections 6.3.2.3 and 6.3.2.4.

Design. This study (a) imported the official definition tables *as-is* into a line-addressable file layout under Git; (b) performed a light manual spot-check to confirm no deviations from the upstream release; and (c) constructed PR-style bundles (typed diffs, Impact Brief, CI stubs) to exercise repository plumbing. No schema migration, column reshaping, value rewriting, or conversion to a standard tree format was performed.

Measures. (i) *Pass-through integrity*: review confirming no content deviations from upstream native files; (ii) *Operability under CCL*: ability to open PRs with atomic, line-addressable diffs over the native format; (iii) *Implicit external check*: in agent demonstrations (Section 6.3.2.4), whether agent-proposed changes reproduce the expert changes implemented in the NDMS production system for the same tickets—serving as an indirect check that the model-of-record was prepared correctly for CCL.

Baselines and analysis. This study contrasts the minimal preparation with the NDMS-only workflow, focusing on whether PR-style artefacts (diffs, briefs, CI evidence) can be assembled *without* altering the native representation (cf. Section 6.3.1.4).

Adequacy criteria. Preparation is *adequate* if: (a) the native files are under version control and line-addressable with stable identifiers; (b) spot-checks detect no deviations from the official release; and (c) in demonstrations, agent-suggested diffs reproduce the externally validated expert changes implemented in NDMS for the NordDRG Forum tickets [49].

Threats to validity. The integrity check is intentionally light because no transformation was applied; implicit validation via a limited number of NDMS-matched tickets may not surface rare edge cases. The analysis tree/DAG (Section 6.3.2.2) is non-authoritative and thus excluded from integrity claims.

Outcome reporting. This study reported (i) the light manual deviation check against the upstream release; (ii) the demonstration that PRs over the native format are operational; and (iii) the manual review of the agent-proposed diffs against the reference standard (the NDMS expert changes) in the demonstration cases, whose outcome is the acceptance tally of Table 20 (five tickets, single run per cell, one non-blinded reviewer)—supporting the artefact choice in Section 6.3.1.4 and its practical fitness for CCL.

7.7.3 Evaluation of the NordDRG Contributor and Reviewer Agents

Objectives. This subsection examines whether the *Contributor* and *Reviewer* agents can act as reliable first-line participants in the CCL loop while safeguarding expert control and the audit trail. The demonstration evaluated here is that of Section 6.3.2.4; the research questions are those of Section 1.4.3.2.

Position in the CCL pipeline. Figure 24 depicts the agents' role in the governed workflow. The Contributor agent ingests forum tickets, grounds them in the official NordDRG specification, and emits explanation-rich atomic diffs. These diffs form a pull request that is subjected to CI gates (G1–G4) and risk-tiered human approvals before merging, tagging, and audit trail ledgering. The implemented prototype completes Steps 1–5 and, in the Agentic mode below, the Reviewer-agent review (Steps 9–10). CI/CD automation (Steps 6–8) was skipped for this demonstration but would include running test-case data through the official DRG Grouper software, while human pull-request review (Steps 11–13) follows the same governance pattern. Steps 14–16 lie outside this demonstration's scope but belong to a full CCL deployment.

Experimental design. Five real NordDRG Forum tickets were processed under two workflow modes:

- **Single-Agent:** only the Contributor agent operates.
- **Agentic:** the Contributor agent is followed by the Reviewer agent.

Each workflow ran with three LLM backends—GPT-4o, o4-mini-high, and o3—giving six runs per ticket (two workflows × three LLM backends, 30 runs in total). All runs went through the CCL Platform (Section 6.3.2.1), via its agentic workflow engine and agent chat interface. For every run the agent(s) parsed the ticket, grounded it in the specification and guidelines, and produced typed atomic diffs plus CI-compatible artefacts.

Results. Table 20 summarizes acceptance outcomes. In total, 8 of 15 Agentic runs vs. 1 of 15 Single-Agent runs were accepted (one run per ticket and backend; the only accepted Single-Agent run was *Case A* with o3), that is, on the author's manual review (a single, non-blinded reviewer) they reproduced the expert change implemented in NDMS for the same ticket, the reference standard; for *Case E* neither workflow reproduced the expert change. With five tickets and single runs, this indicates rather than establishes a benefit of explicit agent-to-agent review for reproducing the expert change; because the Agentic mode adds a further pass over each change, the comparison does not separate the Reviewer role from the additional iteration.

7.7.4 Discussion

What the results mean. The CCL instantiation indicates that agent–agent and human–agent workflows can operate under *human-in-command* control when each change is packaged as typed atomic diffs and routed through risk-tiered CI gates (in the demonstration the gate criteria were applied by manual review; Section 7.7.3). At the $\approx 10^7$-decision-point scale characteristic of NordDRG, reviewer effort is expected to remain tractable when proposals are locally scoped and explanation-rich, and, in the demonstration, the addition of a Reviewer role raised acceptance (agreement with the reference standard) relative to the single-agent workflow (8/15 vs. 1/15 runs on five tickets, one non-blinded reviewer; indicative rather than established, Section 7.7.3). These observations support RQ III.3: the governance properties (roles, gates,

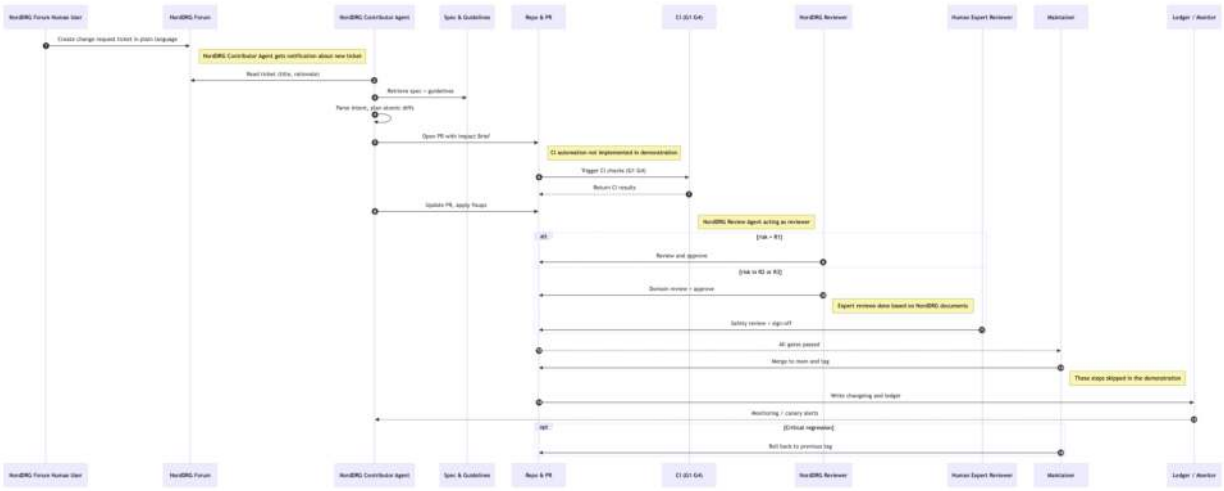


Figure 24: **How the agent plugs into CCL.** The Contributor NordDRG Agent ingests NordDRG Forum tickets (via pasted text), grounds them in the official specification and guidelines, and emits explanation-rich atomic diffs; it then opens a pull request against the model-of-record, where CI runs the CCL gates (G1–G4) and, after the Reviewer agent's pre-review, the risk-tiered human approvals of Section 6.3.1 are collected (in the diagram, the R1 branch draws only the agent pre-review (the single human approval that R1 requires, Section 6.3.1.3, is not drawn separately), and the R2/R3 branch adds the Human Expert Reviewer's safety review and sign-off, Step 11) before a maintainer performs a merge, tags a release, and writes an immutable audit trail to the ledger with canary/rollback safeguards. The diagram shows the full CCL flow; in the demonstration evaluated here, tickets were supplied as pasted text rather than by forum notification, and CI (Steps 6–8) and the post-merge Steps 14–16 were not executed. The diagram shows the agent as a first-line contributor under "human-in-command" governance.

Table 20: Outcome of each of the five NordDRG Forum tickets (Cases A–E) under two workflow modes across three LLM backends (accepted in total: Agentic 8/15, Single-Agent 1/15). A check mark (✓) indicates that, on the author's manual review (a single, non-blinded reviewer), the generated pull request reproduced the change the NordDRG experts implemented in the NDMS production system for the same ticket—the reference standard; a cross (✗) indicates that it did not. Automated CI (Steps 6–8) was not executed in this demonstration.

Case	Workflow	GPT-4o	o4-mini-high	o3
A	Single-Agent	✗	✗	✓
A	Agentic	✓	✗	✓
B	Single-Agent	✗	✗	✗
B	Agentic	✓	✓	✗
C	Single-Agent	✗	✗	✗
C	Agentic	✗	✓	✗
D	Single-Agent	✗	✗	✗
D	Agentic	✓	✓	✓
E	Single-Agent	✗	✗	✗
E	Agentic	✗	✗	✗

immutable ledger—the $\mathcal{R}$, the $\mathcal{O}$- and $\mathcal{N}$-checks, and the $\mathcal{L}$ of Definition 3.2.5) enable agent participation without eroding accountability, provided agents operate under explicit role restrictions, grounding, and fail-closed policies; in this demonstration only the roles and the grounding were exercised, whereas the CI gates (Steps 6–8) and the ledger (Steps 14–16) were not executed, so their contribution remains a design argument.

Implications for practice. For maintainers of large, regulated rulesets, CCL suggests an overlay on existing tooling: (i) keep the native, auditable specification as the *authoritative model-of-record*; (ii) require

change requests to arrive as small, typed diffs with a compact Impact Brief; (iii) enforce gateable evidence (schema checks, locality/blast-radius bounds, counterfactuals, adversarial probes) before human review; and (iv) reserve all merge authority for certified reviewers. In this framing, LLM agents function as *first-line drafters and pre-reviewers*, not approvers, a role intended to reduce reviewer burden (not measured here) without shifting accountability.

Transferability beyond NordDRG. The template is expected to transfer to other governed decision systems that meet three conditions: (i) a line-addressable representation with a stable schema; (ii) the ability to express edits as small, atomic operations; and (iii) an organizational mandate for human-in-command sign-off. Likely candidates include reimbursement schedules, credit-policy scorecards, tariff/eligibility tables, and safety checklists. Where primary models are statistical, CCL applies at the *decision surface* (rules, thresholds, guardrails) and to documentation artefacts (Model/Datasheets), while heavier model-internals may require additional governance mechanisms.

Design trade-offs. Retaining the ordered rule list as the authoritative artefact avoided precedence regressions and unstable diffs, while a compiled tree/DAG served analysis and visualization. Risk tiers simplify policy but can be tuned too coarsely; over-broad tiers increase reviewer load, while over-narrow tiers risk under-testing. Similarly, Reviewer agents help surface issues early, yet their prompts and guardrails must be versioned and audited like code.

Demonstration vs. validation, and the absence of a process baseline. The CCL evaluation is *demonstration-grade*: it shows feasibility on real NordDRG Forum tickets under human-in-command gates rather than through a controlled trial. For the outputs a reference standard exists and is used: each agent-proposed diff is judged against the expert change implemented in the NDMS production system for the same ticket [49], acceptance meaning that it reproduced that change (Table 20); the agentic two-role workflow is also compared against a single-agent workflow as an ablation. Evaluating an agent against the change a human expert actually made is close to an established primitive: SWE-bench judges agent patches for real repository issues by running the tests that accompanied the human fix [284], whereas CCL reviews the proposed diff itself against the expert change. CCL's contribution is not that evaluation move but its setting—applying it inside a governed, human-in-command co-maintenance loop for a regulated decision model at $\approx 10^7$-decision-point scale, where the code-oriented, fully automated benchmarks provide no analogue. What has *no* established baseline is the CCL governance *process* itself: to the author's knowledge, no comparable agent-operable, CI-gated change-governance loop for a regulated decision model exists in practice, so there is no external system to benchmark it against on equal terms. This absence is a consequence of the contribution's novelty (Section 1.1); the artefact is therefore positioned against the status quo for AI-augmented maintenance, in which agent-proposed rule edits, where they occur at all, enter the workflow ad hoc and ungoverned (the human-only NordDRG process itself is governed; Section 1.4.3.1); the process and its evaluation were peer-reviewed as a PCSI 2025 conference article [29].

Limitations and future work. The demonstration targets a single domain and evaluates agents offline on a small set of change requests; end-to-end deployment within national maintenance systems and longitudinal monitoring were out of scope. The evaluation focuses on an explainable, rule-set–based decision model; applying CCL to opaque statistical models such as neural networks would require separate studies and potentially additional governance mechanisms. Many aspects of NordDRG development also require statistical cost analysis on real patient data; while such analyses could in principle be performed by CCL contributor agents, they are outside the scope of this study. Future work includes live integration with production pipelines, development of additional contributor agents trained on different datasets, broader user studies with clinicians and coders, stronger equivalence and red-team testing across risk tiers, deeper role specialization (multi-agent patterns for subdomains), and replication in other governed rule systems (e.g., finance and public policy). A fuller assessment requires collaboration with maintainers and access to operational workflows. NordDRG stakeholders are invited to co-design stronger evaluations; any mischaracterizations identified will be corrected in subsequent work.

Regulatory and ethical considerations. CCL's human-in-command constraint, immutable audit trail, and risk-based gates align with governance expectations for high-risk AI systems, for example the human-oversight requirement of the EU AI Act [261] and the NIST AI RMF [262]. Nonetheless, role boundaries must be explicit (agents cannot merge), provenance for all sources must be maintained, and post-release monitoring and rollback should be configured for any deployment that affects patients, finances, or public services.

Novelty of Phase III as a whole, assessed. All three Phase III artefacts were reported in peer-reviewed venues before this dissertation (HADA and the NordDRG AI Benchmark as papers on the artefacts themselves, and CCL as a PCSI 2025 article on the curated-learning process and its evaluation); the benchmark is, in addition, a publicly released instrument, and the baseline NordDRG-to-CSV conversion step used by CCL is public in the benchmark repository, while the other CCL scripts and prompts are available from the author on request. What the dissertation itself adds is their integration into the governable-agency through-line (C8). The claims are deliberately scoped to what the demonstrators show: HADA met its six predefined objectives (two of them partially) in a banking sandbox, and on the benchmark the strongest models mastered rule-level Logic tasks (up to 13/13) while exact, traceable grouper emulation stayed hard (best 7/13), with the inspected errors (not tabulated per field) concentrated in priority order and CC/MCC exclusion control flow. The building blocks are widely studied; the contribution is the auditable, governance-first integration and the released, head-to-head evaluation instrument itself.

7.8 Cross-Phase Threats to Validity, Scalability, and Generalizability

The dissertation's claims rest on Design-Science demonstrations that are not consistently complemented by systematic empirical and quantitative validation. This section addresses that gap: it gathers the recurring threats that run across the artefacts and states, for each research question, what its artefact actually demonstrated and what level of evidence backs it. Consistent with the research method (Section 1.1) and the deliberate scope of this dissertation, no new field experiments were conducted.

Recurring threats across the program. Three limitations recur. *(i) Ground-truth and synthetic enlargement (Phase II).* Accuracy is measured against historical coded documentation whose own reported coding-error rate is roughly 10–17%, and the largest-scale latency results were measured on synthetic enlargements of the real data rather than on a full production estate. *(ii) Prototype maturity and single-context scope (Phase III governance artefacts).* HADA and CCL are evaluated on a single domain and workflow at prototype maturity, so cross-domain generalizability and sustained, live-organization operation remain to be shown. *(iii) Strict scoring bounds (Phase III benchmark).* The benchmark's exact-match scoring is deliberately unforgiving and can understate partial competence without complementary step-wise conformance metrics; each endpoint answered each of the 13 items per suite once, so the results are a first-cut snapshot without variance estimates; and LLM endpoints evolve rapidly.

The role of baselines—where one exists, and where none can. A comparison against a reference standard or baseline is reported *wherever a meaningful reference exists*: the NordDRG AI Benchmark is scored by exact match against published gold keys (Grouper: the official grouper output) across contemporary LLMs (Section 7.6); the Phase II pipeline is measured against historical coded documentation (Section 7.4); and the CCL agents' outputs are judged against a reference standard, the expert change implemented in NDMS for each of five tickets, and compared against a single-agent workflow (Section 7.7). For the governance *processes* of HADA and CCL, by contrast, *no established real-world baseline exists*: to the author's knowledge, no production system or agent framework surveyed in Section 2.5.6 binds an auditable governance layer to each run-time decision; the closest lines—design-time goal and trust modeling in Tropos/Secure Tropos [78, 256] and training-time constraints such as Constitutional AI [258]—address different lifecycle stages, so there is no comparable system to measure against on equal terms. This follows from the kind of governance layer they contribute (for HADA, one already established in its peer-reviewed lineage), not from an avoidable omission; these artefacts are accordingly evaluated by demonstration and positioned against what existing approaches leave unaddressed.

Research questions, artefacts, and evidence level. Table 21 consolidates, for each research question, the artefact that answers it, the headline quantitative result (where one exists), the reference standard or baseline status, and the resulting level of evidence.

Cross-phase evidence synthesis. Read as one line of evidence rather than three separate evaluations, the levels in Table 21 form one sequence: each phase's limitation became the next phase's problem; what carries forward is the problem framing, not the proof, and the later phases' stronger evidence does not raise the evidence level of the earlier demonstrations. Phase I's feasibility demonstrations established the agent architectures whose data-centric limitation Phase II then measured quantitatively against a reference standard, and the gaps Phase II could not close are the ones Phase III takes up: the NordDRG AI Benchmark measures, with the thesis's strongest evidence, how faithfully LLMs execute the rule logic, while HADA

7.8 CROSS-PHASE THREATS TO VALIDITY, SCALABILITY, AND GENERALIZABILITY

Table 21: Consolidated results-and-evidence summary across the three phases. Quantitative figures are reproduced from the per-artefact evaluation sections (Sections 7.1–7.7); "reference standard / baseline status" records what each result is measured against and, for the governance processes of the novel architectures, why no baseline can exist.

RQ	Artefact	Key quantitative result(s)	Reference standard / baseline status	Evidence level
I.2–I.3	A1–A3 (Phase I; I.1 by the background synthesis, Section 8.1)	Proof-of-concept prototypes; no aggregate metric (era-constrained, qualitative)	No quantitative baseline; feasibility judged against the technology of the era	Demonstration of feasibility (A1 and A3 prototype designs peer-reviewed; A2 and the A3 field trial reported in the Licentiate thesis)
II.1	A4 (ML/IR MAS)	53–66% top-1 and ∼90% top-8–10 accuracy; sub-second ($\leq$0.5 s) query at 10^8 episodes (largest scales measured on synthetic enlargements of the real data)	Historical coded documentation as ground truth (own ∼10–17% coding-error caveat)	Quantitative measurement against a reference standard
III.1	A5 (HADA)	Six predefined design objectives met (O.III.1.4 and O.III.1.5 partially) in the banking `getLoanDecision` sandbox	No baseline exists—no production system or surveyed framework implements the auditable OKR-to-algorithm governance layer	Prototype demonstration + qualitative positioning
III.2	A6 (NordDRG AI Benchmark)	Logic up to 13/13 (best models); Grouper best 7/13 under strict exact match	Exact match against published gold keys (Grouper: official NordDRG grouper output) across contemporary LLMs	Systematic, baseline-anchored comparison, single run per item (strongest in the thesis)
III.3	A7 (CCL; A5 contributing)	Agentic two-role workflow reproduced the expert NDMS change in 8/15 runs vs. 1/15 for single-agent across 30 runs (five tickets, single run per cell; Case E: neither)	Reference standard: the expert change implemented in NDMS for each of the five tickets (one non-blinded reviewer); no baseline for the governance *process*, positioned against the ad-hoc, ungoverned entry of AI-proposed edits (the status quo for AI-augmented maintenance)	Demonstration against a reference standard + ablation (five tickets, single run per cell, one non-blinded reviewer)

and CCL address governance and human-in-command control by demonstration, CCL against a reference standard and with a single- vs. two-agent ablation. No individual result is claimed beyond the level stated in the table.

Scope decision and future validation. Taken together, the evidence is a defensible mix: quantitative validation where a reference exists (the benchmark, and the Phase II measurements), demonstration against a reference standard on a small sample (CCL's outputs against the expert NDMS change for five tickets, one non-blinded reviewer, with a single- vs. two-agent ablation), and demonstration-plus-peer-review where no baseline is available (the governance processes of HADA and CCL). Claims of effectiveness, scalability, and generalizability are correspondingly bounded to what this evidence supports. Items that require controlled trials—clinician-in-the-loop utility studies for Phase II, multi-domain and live-organization deployments for HADA and CCL, a complete formal specification of the CCL gate semantics, and step-wise conformance scoring for the benchmark—are carried into future work rather than claimed here.

8 CONCLUSIONS

This chapter discusses conclusions from all the material presented in this thesis. The evidence supporting these conclusions, and its limits, are stated self-critically in the cross-phase synthesis of Section 7.8 and summarized, research question by research question, in Table 21 (threats to validity, scalability, and generalizability, with the level of evidence behind each research question).

The conclusions that follow are deliberately bounded to what that evidence supports. Where a meaningful reference exists, the claims rest on quantitative measurement—the NordDRG AI Benchmark is scored by strict exact match against published gold keys (Grouper: the official grouper output), and the Phase II clinical-coding pipeline is measured against historical coded documentation. Where the contribution is a novel governance architecture (HADA and CCL), no production system or surveyed framework implements a comparable run-time governance layer, so there is no governed baseline against which to measure the governance *process* on equal terms, whereas CCL's *outputs* are judged against a reference standard, the expert NDMS change for each of five tickets (Section 7.8); these artefacts are evaluated by prototype demonstration and peer review and positioned against what current approaches leave unaddressed. Accordingly, claims of effectiveness, scalability, and generalizability are bounded by prototype-level evidence, and the validation that genuinely requires new work—controlled, clinician-in-the-loop trials for Phase II, and multi-domain, live-organization deployments for the Phase III governance artefacts—is carried into future work rather than asserted here. Table 1 links the DSRP process sequence to the phases and to the structure of this thesis.

8.1 Phase I: The History of the Semantic Web and Software Agents

Main RQ I.1 (Table 2):

> What are the historical origins of the Web and of software agents?

This thesis traces how the Web and software agents emerged from distinct intellectual traditions before converging in contemporary AI systems.

Phase I takeaway The defining Phase I conclusion is a negative one: within this era software agents and the Web did *not* converge. Successful agentic AI did not emerge from combining the Semantic Web's and symbolic AI's modes of operation; within this research program agentic AI found no viable route through them, and the course changed in 2006. In this program, the combination stalled on the ontology engineering bottleneck, the absence of genuine learning, and the industry's divergence toward lightweight HTTP/REST/JSON rather than shared formal semantics, difficulties closely related to those that contemporaries and later retrospectives reported [117, 379, 380]. The result concerns agentic AI and this combination, not the Semantic Web as a whole, whose lightweight descendants (schema.org markup, Linked Data, and knowledge graphs) have since reached wide use [381, 100]. It is precisely this non-convergence that motivated the deliberate change of research course in this dissertation—away from knowledge-engineered agents and toward the data-centric machine learning (information retrieval and statistical learning) that defines Phase II. Convergence, where it ultimately appears, belongs to the generative era of Phase III, not to Phase I.

The Web's Evolution The Web's conceptual foundations extend back to Vannevar Bush's 1945 Memex vision of associative information trails, through Ted Nelson's Xanadu hypertext project (1960s) and Douglas Engelbart's NLS system demonstrating collaborative knowledge work. Tim Berners-Lee's 1989 synthesis at CERN combined hypertext with TCP/IP networking through the trinity of URLs, HTML, and HTTP. The Web became the Internet's "killer application," validating decentralized, hyperlinked knowledge representation.

Software Agents' Evolution Software agents emerged from a distinct research tradition rooted in classical AI, particularly work on autonomous systems, distributed artificial intelligence, and adaptive human-computer interaction. From early reactive agents and Brooks's subsumption architecture [382] through the BDI (Belief-Desire-Intention) cognitive models of the 1990s [186], agent research pursued computational entities capable of autonomous action, experiential learning, and dynamic environmental adaptation. The agent community—in foundational works like Wooldridge and Jennings (1995) [138]—aimed to instantiate systems that could perceive their environment, pursue delegated goals, and coordinate with other agents in distributed settings.

The Convergence Point This research revealed that the Semantic Web vision articulated by Berners-Lee *et al.* (2001) [64] essentially reimagined earlier agent research goals within Web architecture [146, 25]. The

promised "web of data" where machines could understand and process information autonomously directly echoed the multi-agent systems community's aspirations from the previous decade, and anticipated themes from even earlier AI research on knowledge representation and automated reasoning. The Semantic Web thus represented a convergence point where the Web's success as a universal information substrate met the agent community's desire for machine-understandable content and automated action.

While the original Semantic Web's agent-centric, formal-reasoning layer achieved limited adoption, its core vision—machine-processable knowledge enabling autonomous action—has been partly approached through different means in modern LLMs and transformer-based agents. By 2026, intelligent software agents, exemplified by LLM-based systems, have brought these parallel traditions closer together through *Deep Learned Algorithm*s rather than symbolic reasoning alone.

8.2 Phase I: Software Agents and Semantic Web Integration

Main RQ I.2 (Table 2):

> To what extent can Semantic Web formalisms and software-agent protocols be integrated to deliver runtime interoperability and adaptation on the Web, and what irreducible gaps prevent practical convergence?

Phase I approached this question through three implementations (2001–2006), each probing a different segment of the Semantic Web–agent integration space. The first two subsections below answer this question prototype by prototype: run-time protocol adaptation (SRQ.I.1.1–SRQ.I.1.4, Section 1.2.1.3) and B2B negotiation (SRQ.I.2.1–SRQ.I.2.3, Section 1.2.2.2); the third answers Main RQ I.3 through context-aware mobile services (DYNAMOS, SRQ.I.3.1–SRQ.I.3.4, Section 1.2.3.2), and the two closing subsections draw out the integration gaps across all three prototypes.

8.2.1 Phase I: Run-Time Protocol Adaptation

The construction-expediting prototype demonstrated agents dynamically interpreting RDF-serialized FIPA protocols for supply-chain coordination. The system achieved machine-processable protocol intake and functioning FIPA-ACL conversations on the JADE platform, establishing that conversation structure, when explicitly modeled in RDF and consumed at run time, can lift integration beyond brittle endpoint-centric glue code.

However, the supporting research questions (SRQ.I.1.1–SRQ.I.1.4, Section 1.2.1.3) exposed fundamental limitations:

SRQ.I.1.1 Surface integration versus semantic understanding. While RDF successfully serialized FIPA protocols for runtime parsing, the system's reliance on preprogrammed finite-state machines rather than on semantic reasoning revealed a gap: the contractor agent could parse ontology-conformant protocol descriptions but conflated "following a script" with genuine adaptation. True semantic understanding would require goal-directed transition functions or policies over goals, context, and constraints—capabilities absent from the implementation.

SRQ.I.1.2 Platform-specific versus Web-scale interoperability. The prototype succeeded within a controlled Java/JADE environment but demonstrated no path to true Web-based interoperability. The platform dependency was absolute: there was no demonstration of reliability under contention, recovery from failures, or cross-platform communication beyond JADE's internal messaging. The gap between agent-platform protocols and Web-native standards (HTTP, REST) remained unbridged, with the agent technologies used here showing no convergence toward mainstream Web infrastructure.

SRQ.I.1.3 Constrained adaptation versus open-world flexibility. The expeditor adapted only within strictly conforming protocols and a fixed ontology, with flexibility confined to one role and bound to pre-enumerated FSM states. When protocols deviated from the ontology, exhibited version skew, or employed alternative modeling idioms, behavior was undefined—no graceful degradation, schema mediation, or negotiation capabilities were demonstrated. This exposed the practical convergence barrier: multi-party Web ecosystems concentrate variance precisely at ontology boundaries (partial adoption, divergent exception patterns), yet the system created a new lock-in at the ontology layer itself.

SRQ.I.1.4 Data extraction versus semantic integration. Basic report parsing and cross-querying of supplier and carrier agents functioned within the scenario, providing a mechanism for lightweight status comparison. However, the absence of inference capabilities, context reasoning, sophisticated reconciliation policies for conflicting claims, or systematic data-quality assessment meant the system remained at the level of simple data extraction. Autonomous semantic service composition—requiring deep integration of reasoning, discovery, negotiation, and exception handling—was not evidenced. The comparison mechanism existed but lacked evaluation depth, leaving effectiveness suggestive rather than quantified.

These findings demonstrate that while ontology-grounded protocol descriptions enable basic interoperability, the integration remained better characterized as *ontology-constrained interoperability with scoped flexibility* rather than the open-ended run-time adaptation envisioned by the Semantic Web. The system required substantial infrastructure (about 9,000 lines of code) for a basic scenario, contradicting the Web's design principle of simplicity and suggesting that, for agentic AI as pursued here, the full "layer-cake" Semantic Web architecture was hard to reconcile with the principles that made the original Web successful.

8.2.2 Phase I: B2B Negotiation Systems

Multi-agent purchasing automation employed shared domain ontologies for supplier discovery and quote evaluation, demonstrating that ontologies could make commercial artefacts machine-processable and enable basic acceptance reasoning. Buyer and supplier agents successfully conversed on JADE with ontology-mediated interactions.

The supporting research questions (SRQ.I.2.1–SRQ.I.2.3, Section 1.2.2.2) revealed gaps between proof-of-concept and production readiness:

SRQ.I.2.1 Formalizing product and process knowledge. Domain ontologies successfully formalized simplified purchasing scenarios, enabling machine-interpretable requests and offers for basic supplier matching. However, the commercial surface remained deliberately narrow: payment and delivery semantics were simplified, with no modeling of penalties, rebates, Incoterms nuances, volume breaks, or exception clauses. The ontologies were adequate for illustration but far from the operational breadth required for real B2B procurement. This exposed *the ontology engineering bottleneck*, discussed below: ontologies comprehensive enough for production use proved prohibitively difficult to create, and combining or aligning heterogeneous ontologies resisted automation entirely in the prototype.

SRQ.I.2.2 Agent communication and coordination mechanisms. FIPA-ACL conversations on JADE provided functioning message exchange and basic registry-mediated discovery. However, the coordination remained scripted rather than adaptive: typical B2B constructs like multi-round negotiation, supplier counter-offers, expiry handling, delivery-slot conflicts, and exception workflows were absent. The "automation" amounted to a single acceptance decision plus at most one buyer-initiated price revision, rather than end-to-end task delegation. Without escalation triggers, approval gates, or audit trails, human oversight—essential for production systems—was not demonstrated. The agent-based realization worked within platform boundaries but provided no evidence of reliability under contention or cross-platform interoperability beyond JADE.

SRQ.I.2.3 Ontology-based reasoning for interoperability. Shared ontological vocabularies enabled basic semantic matching between buyer requirements and supplier capabilities within the controlled demonstration. However, true interoperability across heterogeneous enterprise systems remained undemonstrated: with XML simulators standing in for ERPs and PIMs, integration challenges around data freshness, latency, and failure modes were not addressed. The narrow ontology subset and absence of reasoning over complex constraints (volume pricing, conditional terms, relationship history) meant the system stayed at the level of *semantic enablement* rather than operational automation.

The B2B work demonstrated that while ontology-driven agents could reason over well-scoped purchasing artefacts, advancing toward routine task automation would require: (i) broadening commercial coverage in ontologies to include real-world terms and exceptions, (ii) embedding oversight primitives as first-class protocol elements, and (iii) hardening discovery and integration with live ERP/PIM links and reliability patterns. The demonstration remained closer to decision support than production-grade delegation, reinforcing the finding that, in this program, ontology complexity was a fundamental scaling barrier.

8.2.3 Phase I: Matching Service Descriptions in a Context-Aware Environment

The DYNAMOS prototype also answers Main RQ I.3 (Table 2):

> Which end-to-end architecture best supports an intelligent model that fuses context-aware personalization with community-shared annotations of services, so it can operate on resource-limited smartphones and still provide proactive recommendations?

Synthesis. The Phase I experiments indicate that, under the technological conditions studied, the most suitable end-to-end architecture is a *server-centric design with a thin, activity-aware client.* In this arrangement, the mobile device focuses on capturing context and offering very simple interactions, while the back end performs the heavy filtering and matching over both services and community annotations. The supporting research questions (SRQ.I.3.1–SRQ.I.3.4, Section 1.2.3.2) clarify how performance constraints, field usability, context modeling, and collaborative content each shape this architecture.

SRQ.I.3.1 End-to-end bottlenecks. Performance measurements revealed that remote method invocation (RMI) between components dominated latency: local operations achieved thousands of requests per second, whereas remote calls reduced throughput by two to almost five orders of magnitude (10^2 to nearly 10^5, depending on the call and the network). This finding mandated a server-centric architecture where matching and filtering occurred server-side, with only compact, pre-filtered result sets transmitted to clients. Architectures attempting full matching on mobile devices were not viable on 2005-era networks and hardware, establishing that integration plumbing—not algorithmic sophistication—was the primary performance determinant.

SRQ.I.3.2 Field usability and resilience. The sailing trial under harsh conditions (winds over 15 m/s, high workload, divided attention) demonstrated that architectural sophistication offered little value without extreme UI simplicity. Most sailors had little time to use the application during the storm; those who did could follow standings and post and read notes, but the system still demanded too much attention, leaving no room for complex menus or multi-step workflows and requiring single-action interactions with clear defaults. Pre-iPhone mobile platform fragility further constrained the design: Nokia 66x0 devices suffered from client memory leaks (partly Java Virtual Machine bugs), unstable Bluetooth and GPS connections, crashes under combined Bluetooth/GPRS/camera/microphone use, and battery life of only about half a day. These findings mandated defensive programming, graceful degradation, aggressive sensor scheduling, and minimal on-device complexity—a thin-client architecture focused on context capture rather than local processing.

SRQ.I.3.3 Lightweight context ontology. A simple, activity-centric context model linking user activities to interests and locations proved sufficient for activity-based filtering without exceeding device capabilities. However, the simplicity constrained fine-grained activity transitions, and cross-activity granularity remained limited. The finding confirmed that for resource-constrained mobile scenarios, architectural trade-offs favored minimal context models with semantic richness concentrated server-side, rather than expressive but computationally heavy on-device reasoning.

SRQ.I.3.4 Collaborative annotations as first-class content. Treating user-generated notes and ratings as matchable content alongside commercial services broadened the recommendation space and enabled peer-to-peer information sharing via event-based distribution. However, large result sets were costly to return over RMI without indexing, and the prototype had no moderation or provenance controls, so noise and redundancy were not managed. This demonstrated that community annotations must be integrated into the same server-side indexing and optimization pipeline as provider-managed services, with proper curation mechanisms.

Taken together, SRQ.I.3.1–SRQ.I.3.4 indicate that an effective architecture for RQ I.3 combines (i) server-side filtering and matching over both services and community annotations, (ii) a thin mobile client focused on context capture and low-cognitive-load interactions, and (iii) a lightweight context ontology that respects device and network constraints while still enabling proactive, context-aware recommendations. However, success came at the cost of substantial complexity (context modeling, distributed coordination, platform-specific workarounds).

8.2.4 Phase I: Irreducible Integration Gaps

Across the three Phase I implementations, five interconnected barriers prevented practical Semantic Web–agent convergence in this research program. These are the concrete, project-level instantiation of the limitations identified critically in the background (Sections 2.3, 2.4):

Complexity explosion versus Web simplicity. Agent systems required extensive infrastructure—about 9,000 lines of code for a basic scenario (A1; about 10,000 for A2), sophisticated middleware stacks, platform-specific integration layers—contradicting the Web's architectural minimalism. The "layer-cake" Semantic Web architecture, with its vertical stack of standards (RDF, RDFS, OWL, SPARQL, rules, proof, trust), proved, in these prototypes, hard to reconcile with the design principles of simplicity, loose coupling, and graceful degradation that made the original Web universally adoptable, a related mismatch later diagnosed for early Web-based multi-agent systems, which failed to inherit the Web's architectural properties such as scalability and loose coupling [383]. The complexity burden fell on implementers rather than being absorbed by standards and infrastructure.

The ontology engineering bottleneck. In these prototypes, creating shared ontologies for real-world domains remained prohibitively difficult, as a later review of the Semantic Web field also found for early ontologies in general, which often proved hard to maintain and reuse at considerable up-front cost [100]. Simple, demonstration-scale ontologies worked within controlled boundaries but lacked the coverage, precision, and exception handling required for production systems. More critically, ontology integration proved intractable in this program: two systems could not reliably combine or align different ontologies describing the same domain without extensive manual intervention. The vision of universal semantic agreement clashed with the reality of distributed, heterogeneous knowledge creation [384], although later Linked Data practice showed that useful integration need not wait for such agreement [166].

Technology divergence. Throughout Phase I (2001–2006), mainstream industry largely favored lightweight, pragmatic solutions over Semantic Web standards, as the market-penetration estimates reported in the agent community's own 2005 roadmap also indicate [385]. While we pursued RDF, OWL, and FIPA protocols, mainstream Web development converged on HTTP, REST, JSON, and simple XML—technologies that traded semantic expressiveness for operational simplicity and universal tool support. The agent technologies used here (FIPA-ACL, JADE) showed no convergence with emerging Web standards (WSDL, SOAP, and later REST), remaining confined to specialized research platforms; where multi-agent research did adopt WS-* services, it used the Web merely as a transport layer [383]. The gap widened rather than closed: by 2006, Web 2.0's success with simple, data-centric APIs indicated that loose coupling and pragmatic conventions were preferred over formal semantics for interoperability at scale.

Absence of genuine learning. All Phase I implementations relied on pattern matching, rule execution, and lookup within rigid frameworks. Adaptation occurred only within pre-enumerated options—the contractor agent selected from predefined protocol states, the B2B agents matched against fixed ontological categories, and DYNAMOS filtered using static preference rules. None exhibited learning from experience, adaptation to novel situations, or improvement from feedback. This inability to learn pointed toward a need for data-driven approaches that could extract patterns from examples rather than requiring complete a priori specification. As instantiated in these prototypes, the symbolic, knowledge-engineering approach had reached its practical limits for agentic AI; already in 1999, Nwana and Ndumu had observed that the strong hypothesis of learning personal agents had been quietly dropped [117].

Mobile platform constraints. Pre-iPhone mobile hardware imposed severe operational barriers that no amount of semantic sophistication could overcome. The device fragility documented in the field trial (Section 4.3.2.4) showed that the client became unusable when cognitive load, interaction complexity, or resource consumption exceeded the devices' limits. These device-level constraints proved temporary (see the Phase I transition below).

What contemporaries found. The negative result was not unique to this program, and much of it was on record before 2006. Inside the agent community, Nwana and Ndumu had identified the ontology problem as perhaps the main obstacle to agent interoperability [117], and the community's own 2005 roadmap acknowledged that intelligent, cognitive agents were in the trough of disillusionment, not yet finding roles in most mainstream business applications [385]; in the Semantic Web debate, Marshall and Shipman judged the agent-centric scenarios unlikely to be realized and located realistic value in domain-specific federations

of knowledge [379], and Sheth, Ramakrishnan, and Thomas argued that a Semantic Web built purely on description logics would be artificially limited without implicit, statistical semantics [386]. Later retrospectives recorded the same outcome for the agent layer: in 2007 Hendler, a co-author of the original vision, asked where all the intelligent agents were [380], a review of the Semantic Web field dated its turn from over-hyped expectations to a more sober perspective to the mid-2000s [100], and multi-agent and Semantic Web researchers traced the missing bridge between the two to a lack of practical use cases and the premature alignment of Web and agent technologies [383]. The counter-evidence is equally clear: multi-agent systems were deployed successfully, if mostly in niche markets [387], and lightweight Semantic Web technologies reached wide use on the Web [381]. What did not materialize, according to these sources as in these prototypes, was the combination of autonomous agents with formal Semantic Web reasoning on the open Web, which is the scope of the negative result stated here.

8.2.5 Phase I Transition: Toward Data-Driven Approaches

Phase I revealed two qualitatively different categories of limitations that demanded different responses.

Transient Hardware Constraints. The mobile platform limitations observed in the field trial (Section 4.3.2.4), rooted in the state of mobile and network infrastructure in the mid-2000s, were temporary problems that technological evolution would address, although they made it difficult to deploy sophisticated context-aware services even when their usefulness was evident. The emergence of the iPhone in 2007 and the subsequent smartphone revolution validated this expectation, eventually providing the hardware capabilities that DYNAMOS anticipated but could not rely upon. These findings suggested *patience*: waiting for inevitable mobile technology improvements while developing architectural patterns (server-centric design, thin clients, efficient synchronization) that would prove valuable once capable platforms arrived. The device constraint was solved, but the underlying resource constraint was not: it has returned as the inference-cost, latency, and interruption budgets of always-on agents (Section 1.2.4).

Structural Limitations of Symbolic AI. For agentic AI as pursued in this program, the ontology bottleneck, absence of learning, architectural complexity, and technology divergence represented structural limitations that hardware improvement or tooling refinement alone could not address. These structural limitations were intrinsic to the combination of Semantic Web formalisms and classical agent architectures as instantiated in the prototypes; they are not a verdict on Semantic Web research as a whole. Knowledge engineering did not scale here: manual ontology creation and rule specification required effort that grew rapidly as domain coverage expanded, and automated ontology learning, matching, and integration remained open research problems at the time. Systems were brittle under deviation: they failed when encountering inputs outside predefined ontological boundaries, with no graceful degradation or ability to reason about novel situations. The agent-oriented Semantic Web stack's emphasis on formal precision, global standards, and complex tooling also clashed with the Web's embrace of pragmatic conventions, loose coupling, and evolutionary design, a tension that Semantic Web pioneers later acknowledged when they located the field's impact in a less formal "sweet spot" [381].

The Paradigm Shift. Within this research program, these structural limitations justified a pivot in 2006: because agentic AI found no viable route through the combination of Semantic Web formalisms and symbolic agent architectures, Phase II pursued it through more flexible, data-driven algorithms—ML and information retrieval methods that offered different trade-offs. Such methods learn patterns from data rather than requiring explicit semantic encoding; scale gracefully without proportional growth in manual knowledge engineering effort; handle ambiguity and incompleteness through probabilistic reasoning and statistical inference rather than brittle logical rules; operate without global agreement on semantic standards, bridging heterogeneous representations through learned mappings; and can eventually leverage improved hardware (mobile platforms, cloud infrastructure, accelerators) while remaining viable on existing infrastructure through algorithmic efficiency.

For this research program, the transition from Phase I to Phase II was thus a shift from knowledge engineering to ML, from symbolic reasoning to statistical inference, from ontological agreement to data-driven adaptation. The Phase II results—offline, quantitatively measured decision support over sparse, heterogeneous healthcare data—and the LLM capabilities measured in Phase III are consistent with this pivot.

Enduring Lessons. However, the lessons from Phase I were not entirely negative. It provided both a proof of concept for semantic and agent-based integration *within* constrained settings and a set of arguments for

why data-driven approaches would be necessary to reach robust, Web-scale interoperability and adaptation. The architectural insights—server-centric design, thin clients, careful attention to integration plumbing, lightweight semantic models, event-driven updates—would prove durable across all three phases.

8.3 Phase II: Assisting Clinical Decision Support with ML and Software Agents

Answers to the supporting research questions

Main RQ II.1 (Table 2):

> How can software agents perform ad-hoc, flexible analysis over large healthcare item sets on commodity hardware under real-world data-quality defects while meeting explicit bounds on latency, memory, and accuracy to support clinical decision-making and hospital funding decisions (DRG coding)?

The question arose from the three problem dimensions identified in Section 1.3—information system fragmentation, limited real-time clinical intelligence, and scalability constraints for intelligent systems—against which the answers below should be read. To address this main research question, Phase II investigated four focused sub-research questions through the design, implementation, and evaluation of a multi-agent system augmented with ML-based information retrieval techniques for clinical decision support.

SRQ.II.1.1 Robust ML architectures. What ML/IR model characteristics are essential for agent-based systems operating on large, imperfect healthcare datasets with limited hardware resources?

Answer: Vector-space information retrieval using TF–IDF weighting and cosine similarity proved effective for handling extreme sparsity and high dimensionality in clinical coding datasets. The mathematical analysis revealed 2.8×10^{230} possible clinical code combinations, which—given the sparse clinical data and commodity hardware available in 2007–2013—justified the selection of IR techniques over the dense neural network approaches of that era (an era-bounded judgment revisited in Section 1.3.2). Key characteristics included: (i) bounded similarity scores yielding interpretable rankings for coder workflows, (ii) hierarchical code relationships augmenting the base IR model to capture semantic similarities across clinical classification systems (ICD-10, NCSP), and (iii) demand-driven agent coordination (Crawler/Integrator/Commander/Aggregator) that decomposed retrieval and scoring tasks while localizing failures and managing computational costs. The architecture favored explainable, controllable retrieval over opaque end-to-end models, achieving controllability and latency control at the cost of some semantic expressivity.

SRQ.II.1.2 Recommendation accuracy thresholds. What recommendation accuracy levels are achievable with real healthcare data using vector space models in multi-agent architectures?

Answer: The system achieved 53–66% first-proposal accuracy (MUST- vs. SHOULD-occur) and ∼90% accuracy within 8–10 recommendations when evaluated on real-world Finnish hospital datasets containing hundreds of thousands of patient episodes. These results met the stated >50% first-proposal threshold and reached about 90% (0.89 in Figure 16(a)) against the >90% top-10 target, indicating performance adequate for coder-assist triage, pending clinical validation. The SHOULD-occur fetching strategy consistently outperformed MUST-occur across all accuracy metrics. However, accuracy was computed against historical documentation assumed as ground truth despite reported coding error rates of roughly 10–17%, and the metrics emphasized common cases, potentially under-capturing rare-but-critical conditions.

SRQ.II.1.3 Robustness to data-quality defects. How do realistic documentation defects affect recommendation accuracy, and which retrieval constraints mitigate degradation?

Answer: Comparative tests on higher- versus lower-quality datasets and deliberate perturbations (removed/incorrect/novel codes) revealed that the MUST-occur fetching strategy degraded rapidly under data defects, while the SHOULD-occur strategy maintained acceptable performance with graceful degradation aligned with real-world data-quality issues. The flexible SHOULD-occur approach demonstrated superior robustness by accommodating incomplete or inconsistent documentation through redundancy and approximate matching. This finding supported, for the tested setting, the architectural choice of flexible matching over strict constraint satisfaction for heterogeneous hospital data estates. Limitations include that experiments

predominantly removed single facts, whereas real episodes may exhibit multiple concurrent defects and temporal uncertainty; robustness beyond diagnoses/procedures (e.g., noisy laboratory results) remains untested.

SRQ.II.1.4 Real-time scalability on commodity hardware. What maximum healthcare data volumes can a MAS architecture handle while maintaining sub-second query response times on standard hardware?

Answer: The system maintained sub-second ($\leq$0.5 s) query times for datasets up to 10^8 patient episodes when operating on standard desktop hardware, with query latency growing no faster than database size (Figure 17; scaling with system diversity was assessed by expert evaluation, not measured). The demand-driven multi-agent coordination was designed to avoid continuous polling (not measured) while preserving real-time responsiveness. These results demonstrate that agent-based architectures with adapted IR techniques can meet explicit latency bounds critical for interactive clinical workflows without requiring specialized infrastructure. However, large-scale tests were measured on synthetic enlargements of the real data (on which accuracy was not measured), and failure modes (agent crashes, network partitions), security hardening, and contention under production loads were not fully characterized, indicating that operational resilience testing remains necessary for productionization.

Synthesis: Addressing the Main Research Question

Collectively, the answers to the four sub-research questions demonstrate that combining agent-based architectures with domain-adapted information retrieval techniques can address the main research question: software agents *can* perform ad-hoc, flexible analysis over large healthcare item sets on commodity hardware under real-world data-quality defects while meeting explicit performance bounds. Specifically: (i) domain-aware IR with hierarchical relationships provides an architecturally appropriate foundation for sparse, fragmented clinical data (SRQ.II.1.1), (ii) the system delivers top-k recommendations meeting the first-proposal threshold and approaching the top-10 threshold (SRQ.II.1.2), (iii) flexible fetching strategies enable graceful degradation under realistic documentation defects (SRQ.II.1.3), and (iv) demand-driven agent coordination sustains sub-second responsiveness at 10^8-scale on standard hardware, the largest scales measured on synthetic enlargements of the real data (SRQ.II.1.4).

Future work

As discussed in Section 1.3.2, this research was conducted in 2007–2013, and parts of its framing have since been superseded. Future research should address the limitations of the reported system through: (i) temporal modeling of patient episodes to capture disease progression patterns, (ii) uncertainty quantification providing confidence intervals for recommendations to support risk-aware clinical use, (iii) federated learning approaches enabling cross-institutional model improvement while preserving patient privacy, (iv) expansion beyond diagnosis and procedure codes to incorporate laboratory results, medications, and vital signs through enhanced vector space formulations, and (v) clinical validation through controlled trials in operational hospital environments to assess real-world impact on decision-making quality and efficiency. Near-term refinements include calibrated uncertainty estimates, brief explanatory rationales ("why/why-not"), light hybridization with LLMs for clustering and justification (not final decisions), and reliability testing to harden multi-agent coordination for production deployment.

8.4 Phase III: Human-AI Agent Decision Alignment (HADA) Architecture

Main RQ III.1 (Table 2):

> How could we leverage LLM agents to facilitate human–AI decision alignment in large corporate settings?

Large enterprises can work toward human–AI decision alignment, as demonstrated in a retail-banking sandbox, by adopting **HADA**—the deployment harness of Phase III (Figure 2), which realizes the shared governance layer of Definition 3.2.5 at run time, in breadth, governing the *use* of decision-making models (Definition 3.1.17) and connecting *strategy, values, responsibility, and workflow* directly to agent behavior. Concretely, HADA (i) binds every agent action to a human owner and escalation path (RACI), (ii) links strategy and KPIs (OKRs) to the decision algorithms they steer, with the OKR-to-policy translation approved by accountable

human owners, (iii) records organizational values in a machine-readable catalog whose conformance checks are routed to accountable human owners, and (iv) preserves auditability via an append-only decision ledger, version-controlled artefacts, and agent-level logs, all while remaining protocol-agnostic for tool integration (MCP in the prototype; A2A in the reference design).

Results

HADA demonstrated all six design objectives in the retail-banking sandbox—four achieved and two (O.III.1.4, O.III.1.5) partially achieved (Sections 1.4.1.4 and 7.5). The reference architecture:

- enables natural-language interaction among humans, LLM agents, and connected tools across roles and systems;
- provides modular, protocol-agnostic agent–tool integration (MCP in the prototype; A2A in the reference design, Section 6.1.1.5) with transparent audit trails;
- maintains value alignment through explicit policies, guardrails, and conformance checks routed to accountable human owners (Section 7.5); and
- supports role-grounded accountability through embedded RACI mappings and escalation paths.

A proof of concept in retail banking demonstrated explainable, auditable decision-making through conversational interfaces. Role-played stakeholders—customers, managers, auditors, and ethics officers—interacted via the *HADA Controller*. In these scenarios, the Controller aligned automated outcomes with human judgment. By making model reasoning explicit and traceable, the prototype is designed to reduce "black-box" risk.

Within these scenarios, decisions were traceable and explainable to each role; effects on decision consistency, stakeholder trust, and organizational adaptability were not measured and remain prospective pending the usability studies listed under future work below.

Summary of the contribution. HADA embeds governance and accountability into agent workflows by tying strategy (OKRs), values, responsibility (RACI), and workflow (IT service management, ITSM) to run-time behavior. Unlike capability-oriented orchestration frameworks such as AutoGen and MetaGPT, which coordinate agents toward a task but leave strategy, values, and accountability ungoverned (Section 2.5.6, Table 7), HADA is designed to add this governance layer on top of such frameworks. Relative to design-time goal-, actor-, and trust-modeling (Tropos, Secure Tropos [78, 256]), the distinguishing move is binding these goals, delegations, and accountabilities to each run-time decision with an append-only record (Section 2.5.6).

Supporting research questions (Section 1.4.1.2)

SRQ.III.1.1—What LLM agent protocol-level gaps hinder human–AI decision alignment in enterprise settings? Existing protocols such as MCP and A2A, as specified at the time of the HADA study (2025; Section 1.4.4 reassesses them from 2026), provide interoperability—the ability to share data and tasks—but not alignment: they lack mechanisms for target/KPI traceability, value conformance, and stakeholder–intent mapping. Four additional gaps arise in enterprise use (together they refine, at protocol level, the Alignment, Containment, and Integration Gaps G.III.1.1–G.III.1.3 of Section 1.4.1):

- **Strategic alignment gap.** Agent protocols do not natively connect agent behavior to company strategy. In practice, objectives and key results (OKRs) must be translated into verifiable constraints, rewards, and guardrails for agents; without this OKR-to-policy bridge, agents can be effective yet strategically misaligned.
- **Value alignment gap.** Protocols lack first-class representations of organizational values (ethics codes, fairness constraints, risk appetites) and automated *conformance checks*. There is no standard way to (i) bind value catalogs to specific tools/models, (ii) trace OKR→algorithm connections, or (iii) continuously test for proxy harms (e.g., ZIP/postcode effects) and value drift. Under those specifications, these detections are left to human oversight rather than being machine-checkable at run time.
- **Accountability gap.** On HADA's design premise that AI systems cannot bear responsibility, decisions must be traceably assigned to humans. Protocols lack a standard way to bind agent actions to organizational roles, reporting lines, and escalation paths. A RACI mapping (Responsible, Accountable, Consulted, Informed) tied to the firm's org chart is required to preserve clear lines of accountability.

- **Workflow gap.** Protocols do not specify *where and how* humans are connected into the operational flow. They omit run-time control points (approvals, four-eyes checks), routing to role-based queues, change/incident lifecycles, segregation-of-duties constraints, and evidence capture. Without explicit hooks into enterprise workflows (e.g., ITSM ticketing), alignment and accountability cannot be enforced in production.

SRQ.III.1.2—What integration issues does large-scale LLM-agent integration reveal for human–AI decision alignment? The HADA architecture addresses the gaps identified above by layering governance tool constructs atop existing agent protocols. In large-scale LLM-agent decision automation, not all organizations have the necessary systems in place; the following integration issues arise:

- **Strategic alignment gap (OKR integration).** OKRs may live in spreadsheets or slide decks rather than machine-readable systems. Integration requires tooling to extract, normalize, and version OKRs, and to expose them as constraints/rewards/guardrails that agents can consume.
- **Value alignment gap (policy codification).** Ethics codes, fairness principles, and risk appetites are often in documents or training videos with no system-of-record. Integration requires parsing and codifying value catalogs, binding them to specific tools/models, and enabling run-time conformance checks (including tests for proxy harms and value drift).
- **Accountability gap (role mapping).** Responsibility definitions are fragmented across human resources information systems (HRIS), org charts, and project tools. Integration requires a unifying service that maps RACI roles to authoritative identities and reporting lines, so agent actions can be traceably assigned and escalated.
- **Workflow gap (human-in-the-loop control points).** Operational processes may be only partly digitized. Integration requires ITSM ticket templates, approval and four-eyes checkpoints, segregation-of-duties rules, evidence capture, and role-based routing so agents can connect to enforceable human control in production.

SRQ.III.1.3—What human-in-the-loop control mechanisms are required in practice to keep decision responsibilities clear? Enterprise alignment requires that automated actions remain anchored to human ownership and standard operational controls. Controls should mirror the four gap areas:

- **Strategic alignment controls.** Human oversight is required at the OKR→policy bridge: (i) approve OKR-to-constraint/reward mappings, (ii) review impact analyses for KPI trade-offs, (iii) periodically revalidate objectives and guardrails, (iv) trigger rollback or pause when strategic thresholds are breached.
- **Value alignment controls.** Governance over the Value tool and its enforcement: (i) designate human stewards for value policies and fairness constraints, (ii) require four-eyes review for changes to sensitive-attribute handling, (iii) mandate human triage for detected proxy harms or value drift, (iv) log rationale for overrides and document mitigations for audit.
- **Accountability controls.** Binding every agent action to a responsible human: (i) embed RACI (Responsible, Accountable, Consulted, Informed) tied to reporting lines, (ii) enforce segregation of duties for approval vs. execution, (iii) maintain signer identity, timestamps, and justification on all decisions, (iv) define explicit escalation paths and ownership at each risk tier.
- **Workflow controls.** Operationalize human-in-the-loop via existing service management: (i) connect agents to ITSM workflows (*as Tools via MCP*) for ticketing, change, and incident lifecycles, (ii) require pre-approval and four-eyes checkpoints at defined gates, (iii) route items to role-based queues with time-bound service-level agreements (SLAs) and automatic escalation, (iv) capture end-to-end evidence (who, what, when, why, artefacts) in the ITSM record for traceability and auditability.

Limitations. The demonstration is currently limited to a single domain (banking / credit scoring), and the evaluation is based on scripted user stories and one automatic decision-making algorithm; scalability, stress testing, and behavioral diversity remain to be demonstrated in real-world contexts.

Future work and research directions

The HADA reference architecture establishes a starting point for human–AI decision alignment in the banking domain. Future research should pursue the following directions:

- **Automated business-objective propagation.** Automate the translation of business objectives (e.g., KPIs or OKRs) into machine-readable policies and reward signals, allowing AI agents to propagate intent to connected AI tools through natural-language policies.
- **Self-aligning agents.** Develop adaptive agents that *propose* adjustments to models and reasoning parameters in response to changing KPIs, ethical values, or contextual constraints, for approval by the accountable human owner, while preserving traceability.
- **Cross-domain replication and scalability.** Extend evaluation beyond retail banking to additional domains—such as healthcare triage, energy-trading optimization, and logistics—to test external validity, scalability, and behavioral diversity under production-level loads.
- **Alignment metadata and protocol evolution.** Formalize metadata schemas for alignment, provenance, and value mapping to be included in future versions of MCP and A2A protocols, ensuring consistent treatment of accountability signals across frameworks.
- **Continuous values alignment.** Extend the Values Catalogue so that amendments (e.g., newly regulated sensitive attributes) automatically raise a retraining or redeployment request for the accountable owner's approval, moving policy compliance toward near real time.
- **Human-in-the-loop checkpoints.** Embed configurable approval gates that escalate disputed or high-risk decisions to human reviewers or committees, quantifying trade-offs among latency, cost, and accountability.
- **Usability and user-experience studies.** Conduct longitudinal trials with non-technical stakeholders—such as executives, ethics officers, and customers—to evaluate whether conversational interfaces enable effective participation in decision governance.
- **Benchmarking and interoperability.** Quantify the overhead and benefit of adding HADA's governance layer on top of general-purpose orchestration stacks (e.g., rule-engine wrappers, RAG-centric pipelines, LangGraph or CrewAI architectures), which lack such a layer, measuring alignment speed, transparency, and maintenance effort. Prototype orchestration bridges to validate framework-agnostic deployment.
- **Multi-modal and multi-agent extensions.** Augment the agent layer with vision and speech models to handle multi-modal decision contexts, and explore coordination mechanisms for heterogeneous multi-agent systems.
- **Regulatory-grade compliance and auditability.** Evaluate how HADA's audit ledger, alignment triggers, and traceability mechanisms satisfy emerging EU AI Act and GDPR requirements for explainability, risk-tiering, and data minimization.

8.5 Phase III: NordDRG AI Benchmark

Main RQ III.2 (Table 2):

> To what extent can LLMs emulate, explain, and apply DRG-based hospital payment rules?

This question is operationalized by releasing and evaluating the *NordDRG AI Benchmark* [24], a *rule-complete*, governance-grade testbed that turns the NordDRG definition tables and manuals into machine-readable artefacts and exact-match tasks. The benchmark comprises two complementary suites: (i) a 13-task *Logic* benchmark that probes rule-level skills (cross-table joins, properties, CC/MCC validity, multilingual grounding); and (ii) a 13-task *Grouper* benchmark that requires *full specification emulation* with strict exact match on `drg_nat` *and* the triggering `drg_logic.id`.

Read longitudinally, the two suites record a finding in their own right: between the 2024 single-question studies [20, 21] and the 2025 releases, frontier LLMs crossed the single-question threshold—rule-level Logic tasks approached saturation for the top tier—while full-specification emulation remained the open frontier in the August 2025 evaluation (best 7/13). The benchmark's extension to grouper emulation is the direct response to that observed capability gain.

The design objectives from this dissertation, which align one-to-one with SRQ.III.2.1–SRQ.III.2.3, were achieved (Section 1.4.2.4):

O.III.2.1 Artefact construction (coverage & fidelity). A versioned, machine-readable bundle is released (tables ∼20 sheets, governance PDFs, curated FI / FI–EN excerpts) with schema-stable identifiers for drop-in yearly updates.

O.III.2.2 Benchmark operationalization. Two suites with governance-grade scoring are instantiated: order-invariant set equality for *Logic* and strict pairwise exact match for *Grouper*.

O.III.2.3 Baseline evaluation. Under an artefact-only (no-web) setting, top-tier models *fully* solve *Logic* (GPT-5 Thinking, Opus 4.1: 13/13), with o3 close behind (12/13), while *partial* emulation is achieved on *Grouper* (GPT-5 Thinking: 7/13; o3: 6/13; o4-mini: 3/13; GPT-5 Thinking Mini: 1/13; others: 0/13).

Taken together, the results show that the state-of-the-art LLMs evaluated in August 2025 can *read and reason over* the NordDRG rule base (the *explain* part of RQ III.2 was not scored in the reported runs; the reference agents' evidence traces are reported only qualitatively), but that exact reproduction of the governed control flow—especially priority order (`ORD`) and CC/MCC exclusions—remains the limiting step. To the author's knowledge, this is the first public evidence of an LLM *partially emulating* a complete DRG grouper with traceable rule-row identification; prior LLM studies (e.g., [312, 314]; Section 2.6) predict DRG or ICD codes from clinical text but do not reproduce the governed grouper control flow at rule-row level.

Practical contribution

This phase delivers a reusable *infrastructure for teaching and auditing* AI on hospital funding logic:

- **Open artefacts.** Rule-complete tables, governance manuals, curated FI and FI–EN subsets, prompts, and gold keys with schema-stable IDs.
- **Governance-grade scoring.** Exact-match outputs on codes (Logic) and on (`drg_nat`, `drg_logic.id`) pairs (Grouper) align evaluation with production grouper semantics.
- **Reference agents.** Lightweight *LogicAgent* and *GrouperAgent* enable artefact-only evaluation with auditable traces.
- **Reproducibility.** A public repository with versioned releases, folder-per-release workflow, and drop-in annual updates supports longitudinal teaching and comparative research.

Limitations

- **Scope of evaluation.** Experiments isolate specification-faithful reasoning on clean artefacts; they do not assess extraction from noisy EHR narratives or proprietary claims data.
- **Model coverage.** Baselines prioritize widely used proprietary endpoints; systematic evaluation of open-weight models and fine-tuned variants is out of scope.
- **National and language breadth.** The benchmark centers on NordDRG (with FI and FI–EN excerpts); other Nordic languages and non-Nordic DRG families are not yet included.
- **Operational constraints.** Consumer subscription quotas affected throughput for some providers; their effect on accuracy was not tested, and they impact practicality for classroom and large-batch settings.
- **Single-run snapshot.** Each endpoint answered each of the 13 items per suite once; no repeated runs or variance estimates are reported, so differences of one or two items between models should not be read as significant.

Future work

1. **Cross-system generalization.** Port the artefact and scoring protocol to additional DRG families (e.g., AR-DRG, MS-DRG/APR-DRG) and to ICD-11-based groupers.
2. **Coding-agent track.** As outlined in Section 1.4.4, ask a coding agent to *construct* rather than *emulate* the grouper: from the released artefacts only, generate executable grouper software whose `drg_nat` and `drg_logic.id` match the official grouper over the full case universe. A successful program would make emulation of the whole decision tree fast and exact, with the benchmark's exact-match criterion serving as its test oracle; the scoring cases must be held out from the agent.
3. **Open-weight and fine-tuned baselines.** Benchmark instruction-tuned and task-specific fine-tunes; compare against retrieval-augmented and tool-augmented patterns under identical artefact constraints.
4. **Minority-language expansion.** Extend curated excerpts beyond Finnish to Swedish, Norwegian, Icelandic, and Sámi; measure zero-shot and bilingual transfer.
5. **Governance workflow automation.** Prototype agents that draft, validate, and merge annual technical-change templates against the released tables with audit-ready diffs.
6. **End-to-end realism.** Add optional noisy-document tracks (de-identified notes/claims) that funnel into the same governed scoring, enabling study of robustness without weakening auditability.

7. **Evaluation harness.** Release a standardized runner (prompt packs, context assembly, version pinning, exact-match metrics) and dataset/model cards for transparent reporting.

Closing remark. This work demonstrates that carefully packaged *governed artefacts* plus *exact-match protocols* make hospital funding logic teachable to LLMs and auditable by humans. The benchmark thus serves a dual role in this dissertation: a practical tool for regulators, payers, and educators, and a versioned yardstick for comparative research.

8.6 Phase III: Collaborative Curated Learning (CCL)

Main RQ III.3 (Table 2):

> What end-to-end governance requirements and workflow properties enable AI agents to contribute to decision-model development without eroding human expert accountability?

The CCL study's answer to the RQ is that AI agents can contribute to a version-control-centric development process in which every update is a typed, auditable transaction and *only humans* can approve merges to the decision-making model (*human-in-command*). Building on this principle, this study introduces *CCL*, the maintenance harness of Phase III (Figure 2), a governance-first, pull-request-centric process. The underlying principle is well established (mixed-initiative interaction, interactive machine learning, and hybrid intelligence; Section 6.3); CCL's contribution is its governance-first instantiation for a regulated rule base.

Concretely, this study delivered **C.III.3.1**–**C.III.3.3**, together with a replication suite supporting **C.III.3.4** (see Section 6.3): (i) **C.III.3.1** — the governed CCL process and gate policy that treat each update as a typed, auditable pull request transaction; (ii) **C.III.3.2** — an *agent-operable* NordDRG representation enabling safe atomic diffs and reviewer-legible slices; (iii) **C.III.3.3** — an agentic workflow with role-constrained LLM agents (Contributor, Reviewer) that participate in proposer–reviewer loops under human-in-command control; and (iv) the replication suite (supporting **C.III.3.4**) — reusable converter, prompts, SVG fragments, and reproducibility scripts. These contributions are operationalized in the CCL Platform, a full-stack web application comprising approximately 33,000 lines of code (Section 6.3.2.1), which served as the execution environment for all demonstrations and evaluations.

Using NordDRG as a large-scale decision-making model testbed, this study instantiated **C.III.3.4** by parsing the official specification, normalizing it for PR-style maintenance, and exercising realistic change scenarios. Of the three solution objectives (Section 1.4.3.4), the governed workflow (O.III.3.1) was specified and partly exercised (the prototype completes Steps 1–5 of Figure 24 and, in the Agentic mode, the Reviewer-agent review (Steps 9–10), while automated CI was not executed; Section 7.7.3), and the agent-operable model-of-record (O.III.3.2) and the constrained Contributor/Reviewer agents (O.III.3.3) were demonstrated; of the measures named under O.III.3.3, only reviewer acceptance was obtained in this demonstration. Demonstration findings in this study (**C.III.3.5**) indicate that CCL can be operated over rule sets on the order of $\approx 10^7$ decision points, with the agent-proposed diffs judged against a reference standard, the expert change implemented in NDMS for each of five tickets (Section 7.7). Shorter validation cycles and better explanations relative to ad-hoc, ungoverned handling of AI-proposed edits remain hypotheses. Introducing the Reviewer agent role is intended to add another safety layer that helps keep the decision model coherent. With the help of Reviewer agents, human reviewers are expected to be able to focus on evaluating systemic impacts, as the agents correct the more obvious faults in the change sets first (reviewer effort was not measured in this demonstration).

Read from 2026 (Section 1.4.4), these findings stand as reported, but coding agents now ship, as standard features, much of what the CCL Platform had to build—isolated worktrees, pull requests, test gates, and agent-to-agent review—so the Contributor and Reviewer roles have become a configuration to govern rather than an architecture to construct. The oracle, representation, and accountability problems that CCL addresses remain open, the last of them, if anything, sharpened as review migrates to agents.

Answers to the supporting research questions

SRQ.III.3.1—How should a decision model on the order of $\approx 10^7$ decision points, like NordDRG, best be represented for AI agents to support safe, atomic diffs? As argued in Section 6.3.1.4, standard ML decision-tree formats are typically not used for *collaborative development* of regulated decision logic. The NordDRG format, by contrast, supports expert review, approval workflows, and regulatory

transparency through established tooling and human-readable documentation. Accordingly, this study treats the ordered rule list as the authoritative artefact under version control, while the compiled tree/DAG representation serves analytical, visualization, and agent-facing purposes. The auditable specification functions as the *model of record*: a line-addressable, version-controlled table with stable identifiers and a concise interpretation guide, enabling diffable, reviewable edits while preserving rule precedence.

SRQ.III.3.2—Under which constraints do LLM agents produce reviewer-acceptable change proposals for decision models at $\approx 10^7$-point scale? LLM agents can act as first-line *proposers/reviewers* when constrained by (i) grounding in versioned specification and guidelines, (ii) role restrictions that prohibit merges or unilateral changes, and (iii) fail-closed gate policy (the risk-tiered CI gates of Section 6.3.1.3, applied by manual review in the demonstration of Section 7.7.3). Within these bounds, agents are expected to reduce reviewer burden by drafting well-typed diffs and explanation-rich briefs; adding a dedicated *Reviewer* agent increased acceptance (agreement with the expert NDMS change for the same ticket, one non-blinded reviewer) versus a single-agent workflow across 30 runs (8/15 vs. 1/15; indicative rather than established, Table 20). Residual failure modes include scope creep, incorrect code-set references, and hallucinated justifications; the gate policy is designed to mitigate these through locality/budget checks, red-team/adversarial tests, and provenance requirements (automated CI was not executed in the demonstration), and through mandatory human sign-off, which the demonstration applied as manual review.

8.7 Summary

This dissertation traces more than two decades of design-science research on intelligent software agents across three technological waves—Semantic Web semantics, data-centric ML, and LLM-powered generative agents—culminating in a blueprint for auditable, value-aligned AI systems. Read as a whole, the three waves form the single *governable-agency through-line* introduced in Section 1.1.2: across an evolving representational substrate, the constant is the recurring demand for governable agency—agent behavior that is interoperable, auditable, and aligned with human command—addressed only once Phase III places language-model reasoning inside governed architectures (Definition 3.2.5). Within Phase III this takes the form of two harnesses over decision-making models (Definition 3.1.17)—CCL maintaining one large model in depth and HADA deploying distributed agentic decisioning across an organization in breadth, both resting on the capability premise that the NordDRG AI Benchmark tests and partly establishes.

Phase I (2001–2006).

- **Problem.** Early-2000s Web content was almost entirely human-readable; autonomous service composition required machine-interpretable semantics as well as awareness of user context (location, time, activity).
- **Artefacts.** Three proof-of-concept multi-agent prototypes were produced: (a) *run-time interaction-protocol adaptation*: agents adapting at run time to RDF-described protocols (one role, within a fixed ontology) instead of hard-coded workflows; (b) *ontology-driven multi-party purchasing negotiation* (single round) in supply-chain scenarios; and (c) an *intelligent service-matching engine* that delivered context-aware recommendations to mobile devices on the move.
- **Findings.** Ontology-backed agents could discover and interact with services within controlled, single-platform settings once contextual cues were available, but performance was limited by immature tooling and fragmented technologies and standards. More importantly, after years of experimentation in this program, the software-agent tooling used here and mainstream Web standards showed little sign of converging—the agent-centric part of the Semantic Web "layer-cake" vision of seamless integration remained largely aspirational.
- **Contribution.** The studies demonstrated how machine-readable descriptions plus lightweight context models can, within constrained settings, begin to elevate the Web from a document repository to an *interaction substrate*. The persistent tooling gap—and the observed lack of convergence between the agent technologies used here and emerging Web standards—motivated this program's pivot to data-centric learning explored in Phase II; read from 2026, the three challenges these studies identified remain central to LLM agent deployments (Section 1.2.4).

Phase II (2007–2013).

- **Problem.** Hospitals generate millions of sparsely coded records whose meaning resides in hierarchical taxonomies (ICD-10, NCSP). Exact-match SQL pipelines ignore hierarchy, degrade under missing/partial codes, and struggle to deliver ad-hoc similarity search within strict latency and memory budgets [13].
- **Artefacts.** A modular multi-agent platform that orchestrates heterogeneous data flows and ML-backed retrieval—vector-space IR (TF–IDF/cosine), kernel methods, and k-NN—via crawler, integrator, commander, and aggregator agents running on commodity hardware.
- **Findings.** High-dimensional code embeddings enabled sub-second similarity queries on desktop-class machines, remained robust to documentation defects with SHOULD-occur fetching (MUST-occur degraded rapidly), achieved 53–66% top-1 and $\sim$ 90% top-8–10 recommendation accuracy, and recommended missing codes without modifying the underlying taxonomies.
- **Contribution.** Coupling symbolic hierarchies with statistical retrieval inside an agentic architecture yielded scalable, flexible clinical decision support and DRG analytics under real-world constraints, and established an evaluation playbook on Finnish hospital data that informed Phase III.

Phase III (2023–2025).

- **Problem.** Modern organizations must align rapidly evolving LLM agents with objectives, regulations, and values while preserving auditability and containment.
- **Artefacts.** (a) *HADA* (Human-AI Agent Decision Alignment) reference architecture: a role-segmented cohort of LLM agents supervised by the HADA Controller Agent.
  (b) *NordDRG AI Benchmark:* a rule-complete, governance-grade benchmark over the full NordDRG rule graph ($\sim$20 interlinked definition sheets and expert manuals) with a 13-task *Logic* suite and a 13-task *Grouper* suite, both scored by exact match (order-invariant for Logic, strict for Grouper; Section 8.5); curated FI / FI–EN subsets and lightweight reference agents support artefact-only evaluation and teaching.
  (c) *CCL* (Collaborative Curated Learning): a governance-first, pull-request-centric workflow where agent-proposed updates arrive as typed atomic diffs with Impact Briefs and pass risk-tiered CI gates to an immutable audit ledger; merges remain strictly human-in-command.
- **Findings.** In a banking sandbox, HADA demonstrated its six predefined objectives: four achieved and two (stakeholder alignment across time-scales; scalability, auditability, and near real-time propagation) partially achieved (Section 7.5). Under an artefact-only (no-web) setting, NordDRG baselines show top-tier models fully mastering rule-level tasks (GPT-5 Thinking, Opus 4.1: 13/13), with o3 close behind (12/13), while exact, traceable grouper emulation remains challenging (GPT-5 Thinking: 7/13; o3: 6/13; o4-mini: 3/13; others $\leq$1/13), with errors concentrated in priority order (`ORD`) and CC/MCC exclusion control flow. For CCL, on five real NordDRG Forum tickets (30 runs) the two-agent Contributor–Reviewer workflow reproduced the expert NDMS change for the same ticket (the reference standard) in 8/15 runs versus 1/15 for the single-agent workflow, as judged by the author's manual review (demonstration-grade: single run per cell, one non-blinded reviewer; automated CI not executed; Section 7.7).
- **Contribution.** The phase positions governed LLM agents (Definition 3.2.5) as a viable pattern for decision governance—conversational, auditable agent cohorts in breadth (HADA) and human-in-command rule maintenance in depth (CCL), two harnesses over one governance layer—and provides, in the NordDRG AI Benchmark, a living testbed for research on domain-specific LLM reasoning.

Research questions answered

Closing the loop opened in Section 1.5, each research question is answered by its artefact(s) at the level of evidence indicated (see Table 4 for the full mapping and the reporting publications):

- **RQ I.1** $\rightarrow$ background synthesis: the historical origins of the Web and of software agents are traced and argued to converge in contemporary AI — *demonstration: critical synthesis* (no artefact).
- **RQ I.2** $\rightarrow$ A1, A2: run-time protocol adaptation and ontology-driven B2B negotiation are feasible (for A2, with human-oversight mechanisms not demonstrated); syntactic interoperability is achieved but agent and Web standards did not converge — *demonstration* (peer-reviewed prototype for A1; Licentiate-thesis prototype for A2).

- **RQ I.3** → A3 (DYNAMOS): proactive, context-aware service matching is delivered on resource-limited smartphones, with resource-efficient matching (O.I.3.2) only partially achieved — *demonstration* (peer-reviewed prototype design; field trial reported in the Licentiate thesis).
- **RQ II.1** → A4: 53–66% top-1 / ∼90% top-8–10 accuracy on real Finnish hospital data (hundreds of thousands of episodes, measured against historical coded documentation), and sub-second ($\leq$0.5 s) query latency up to 10^8 episodes (largest scales measured on synthetic enlargements of the real data) — *quantitative*.
- **RQ III.1** → A5 (HADA): six predefined design objectives demonstrated in a banking sandbox (four achieved, two partially) — *demonstration* (no comparable governed baseline exists).
- **RQ III.2** → A6 (NordDRG AI Benchmark): rule-level Logic mastered (best 13/13) while exact grouper emulation stays hard (best 7/13) — *quantitative*, baseline-anchored against published gold keys, for Grouper the official grouper output (the strongest empirical evidence in the thesis).
- **RQ III.3** → A7 (CCL), with A5 contributing the role-bound human sign-off ($\mathcal{R}$): governed, human-in-command maintenance of regulated rules via typed atomic diffs, risk-tiered CI gates, and an immutable ledger (in the demonstration, the gate criteria were applied by manual review and the merge and ledger steps were out of scope; Section 7.7.3) — *demonstration against a reference standard* (the expert NDMS change for each of five tickets, one non-blinded reviewer) plus single- vs. two-agent ablation (no baseline for the governance process).

Original contributions of this dissertation

The original contributions, summarized in Section 1.6.3 and set out in full in Appendix A ("Original contributions in detail"), are: **C1** run-time interaction-protocol adaptation (artefact A1; primary publication ICEIS 2004); **C2** ontology-driven B2B negotiation (A2; Licentiate thesis 2006); **C3** the DYNAMOS context-aware service matcher (A3; CAPS 2005); **C4** the ML/IR clinical recommendation multi-agent system (A4; PCSI 2009); **C5** the HADA alignment architecture (A5; AGENTICS 2025); **C6** the NordDRG AI Benchmark (A6; CHIRA 2025); **C7** the CCL process (A7; PCSI 2025); and **C8** the cross-phase synthesis itself. Contributions **C1**–**C7** consolidate previously published, and in most cases peer-reviewed, work (the exception is **C2**, whose fullest account is the Licentiate thesis; the candidate's role in each artefact's lead publication is stated in Table 23), so the dissertation's own contribution is their synthesis into one analytical scheme, together with the consolidation of the released artefacts (the open NordDRG AI Benchmark, released with its 2025 papers, and the CCL artefact, available from the author on request). **C8**—the longitudinal *governable-agency through-line* that binds this work into a single cumulative argument (Section 1.1.2)—is new to this dissertation.

These contributions form one dependency line rather than a list of separate results: the Phase I agent and Semantic Web framing (**C1**–**C3**) motivates the Phase II pivot to data-centric machine learning (**C4**), which in turn grounds the governed LLM-agent architectures and public benchmark of Phase III (**C5**–**C7**), all bound by the cross-phase synthesis (**C8**). Each of **C1**–**C7** answers a specific research question, and **C8** spans them (Table 3).

In sum, this thesis argues, from three phases of design-science evidence, that *agent architectures can remain practical across successive AI paradigms and become governable once language-model reasoning is placed inside governed architectures* (Phase III).

Future work

The artefacts in this thesis suggest that, **for organizational decision governance, AI alignment can increasingly be treated as an *engineering challenge***. Building on these insights, this section organizes the following avenues under three lenses—*methodological*, *technological*, and *socio-technical*—and closes with a long-term vision.

1. Methodological: Toward Verified Alignment. While HADA demonstrates policy-controlled cohorts of agents, a formally verified alignment layer remains an open goal. Future work should explore the integration of symbolic proof systems (e.g., temporal-logic model checkers) with probabilistic guarantees from uncertainty-aware LLMs, enabling *machine-checkable certificates* for agent behavior in safety-critical settings.

2. Technological: Cross-Domain Generalization. The agent architectures evaluated here were tested primarily in healthcare (Phase II, NordDRG, CCL) and retail banking (HADA). Extending them to further domains such as public administration, energy, and cyber-physical systems will stress new aspects such as real-time constraints and adversarial robustness. Transfer-learning studies across domains—and across languages—will reveal how much of the current ontology layer and retrieval machinery can be reused without retraining.

3. Technological: Energy & Sustainability. Large-scale generative agents incur non-trivial carbon footprints. Extending the commodity-hardware latency and resource-budget analyses of Phase II to the energy use of Phase III stacks, and designing *adaptive sparsity* or *on-device distillation* strategies, could reduce the energy cost of alignment.

4. Socio-Technical: Human–Agent Co-Governance. Empirical studies are needed to test how domain experts interact with audit trails and policy dashboards. Mixed-methods research—combining controlled experiments with ethnographic observation—can surface usability bottlenecks and inform the design of *explanation interfaces* that remain intelligible as agent autonomy increases.

5. Socio-Technical: Open Benchmarks and Shared Tasks. The NordDRG AI Benchmark provides a public yardstick for hospital-funding logic. Next steps include extending the Logic suite to remaining corners of the NordDRG specification and providing idiomatic loaders and exporters to ease integration into common toolchains, together with the standardized evaluation harness listed in the benchmark's future work (Section 8.5).

6. Long-Term Vision: Continuous, Trusted Autonomy. Ultimately, the field must converge on **continuous assurance**: mechanisms that monitor agents post-deployment, detect distributional shift, and trigger realignment proposals for human approval without service interruption. Embedding these capabilities natively into the HADA stack—and exposing them through standardized APIs—would close the loop from design to operation.

8.8 Toward an Exact Science of Agents

These Conclusions close with a position rather than a result. If AI is to be developed in a controlled and safe way and with fewer surprises, the field must move toward an exact science, one whose basic concepts are precisely defined. In the author's experience, computing and AI have combined ambitious, abstract engineering, in the sense in which Brooks called computer science an engineering discipline [388], with separate mathematical methods, while basic concepts such as "learning" and "agent" have remained imprecise. The engineering framing of alignment above is not withdrawn: precise concepts let such claims be stated and checked.

The direction taken here is a consilience-type convergence of disciplines, in Wilson's sense of linking knowledge across disciplines into a common groundwork of explanation [389]. It requires consolidating computing with mathematics, whose exact language can be tied to precise philosophical concepts. Chapter 3 was a bounded attempt at this for Phase III. It defines learning settings, the Transformer, the LLM, the LLM agent, the governed LLM agent, and the decision-making model (Definitions 3.2.1, 3.2.5, and 3.1.17). It does not define learning, agency, or alignment in general, and it offers no theory that predicts surprises.

Definitions do not remove empirical surprises; they make them measurable and the arguments about them well grounded. The NordDRG AI Benchmark (Section 8.5) is an example of how to make a model's capability a measured, task-level quantity, and the risk-tiered gates designed for CCL (Section 6.3.1.3) state in advance what a proposed change must pass before a human approves its merge. The 2026 reassessment (Appendix A) shows that the question of who defines the shared vocabulary is again active. The surprise described in the Preface is where these Conclusions end: not explained away, but stated precisely enough to be measured.

9 EPILOGUE

9.1 Agents are (finally) useful

During this thesis research, intelligent software agents have evolved from DAI research concepts and narrowly deployed prototypes into components of practical, everyday applications through the emergence of *LLM Agent*s. As noted in the Preface, the grammar review of this document (then over 150 pages) illustrates this: it took a few hours with a minimal, human-orchestrated setup of two prompt-configured LLM agents in the informal sense of Section 2.5 (without tools or an autonomous control policy; prompts in Appendix A).

Setting up such agents takes minutes; the more important question is what role humans should play in such workflows. In the CCL approach, humans retain the *Authority to Define Workflows* at design time and, at run time, the sole authority to approve merges (the human-in-command PR Authority Policy, Section 6.3.1.3). For the original manuscript, I adopted the left-hand, human-in-the-loop process shown in Figure 25, where the human retains full control and dispatches tasks directly to the agents rather than enabling an *Agentic Workflow*. The trade-off is coordination overhead: for every section, I had to decide how many of the flagged sentences to review and which proposed changes to accept. Even under this "inefficient" regime, the review was completed within hours; this illustrates, rather than evaluates, the practical usefulness of a minimal LLM setup. By contrast, the right-hand, agentic workflow in Figure 25 would have reduced my own actions substantially, but it would not have shifted accountability: the same review burden would remain, only in larger batches. In the end, I chose to contain the AI: I did not fully trust that it was aligned with my objectives or able to perform exactly as I intended, so I delegated the execution of each step but not the decision to accept its output.

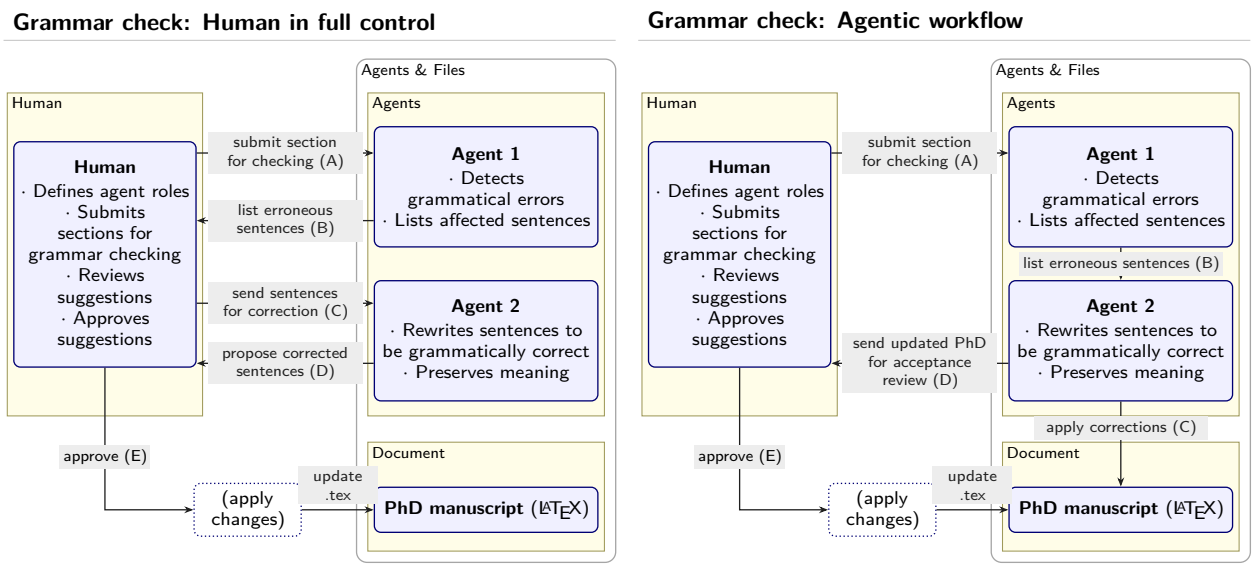


Figure 25: Human-in-the-loop or agentic workflow: grammar checking of this manuscript with two LLM agents. Left: the human dispatches every step and approves each change (A–E). Right: the agents hand off to each other, Agent 2 applies the corrections (C), and the human only reviews the updated manuscript (D) and approves it (E).

In the 2026 revision of this manuscript, I moved toward the right-hand side of Figure 25. Coding agents ran entire workflows—checking cited sources against the sentences that cite them, reviewing the full text through several critical lenses, and proposing corrections in isolated branches—while I set the tasks, reviewed the proposals, and decided which of them to merge. My role thus shifted from approving individual sentences to approving batches of changes, but accountability did not shift: every change still passed through my decision, consistent with the human-in-command merge rule of CCL.

A SUPPLEMENTARY FIGURES AND TABLES

This appendix collects illustrative and reference figures, tables, and listings that support the main text, together with the full statement of the original contributions (C1–C8), the 2026 reassessment of Phases I and III, and the Epilogue's agent prompts; each item is cited from the section it supports.

Introduction and background figures and tables (Chapters 1 and 2)

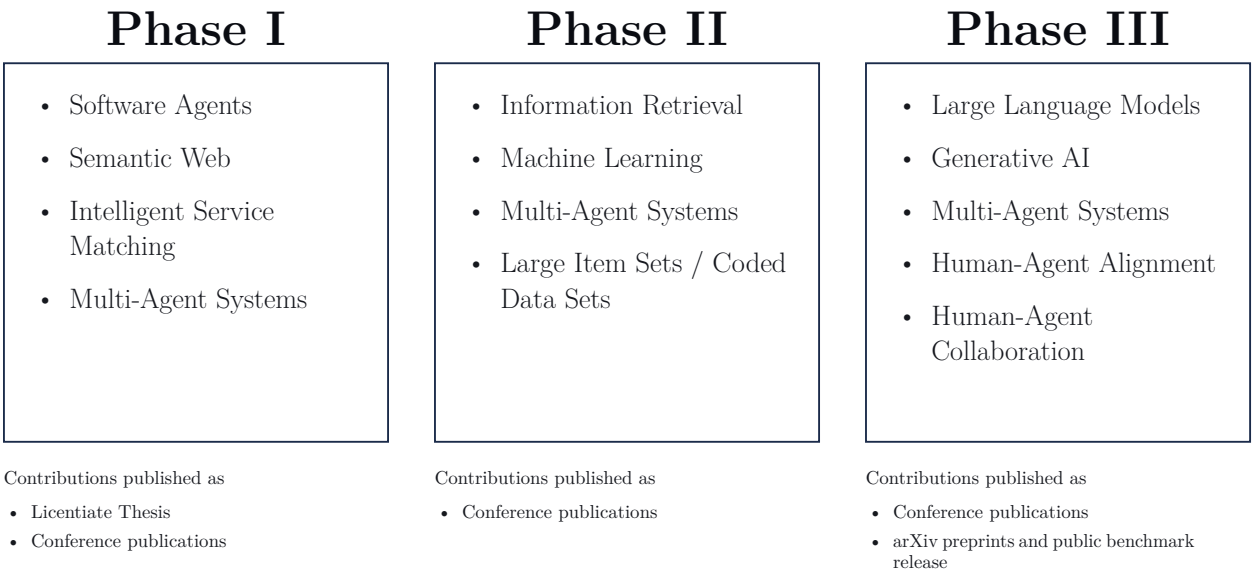


Figure 26: Structure and contributions of the thesis.

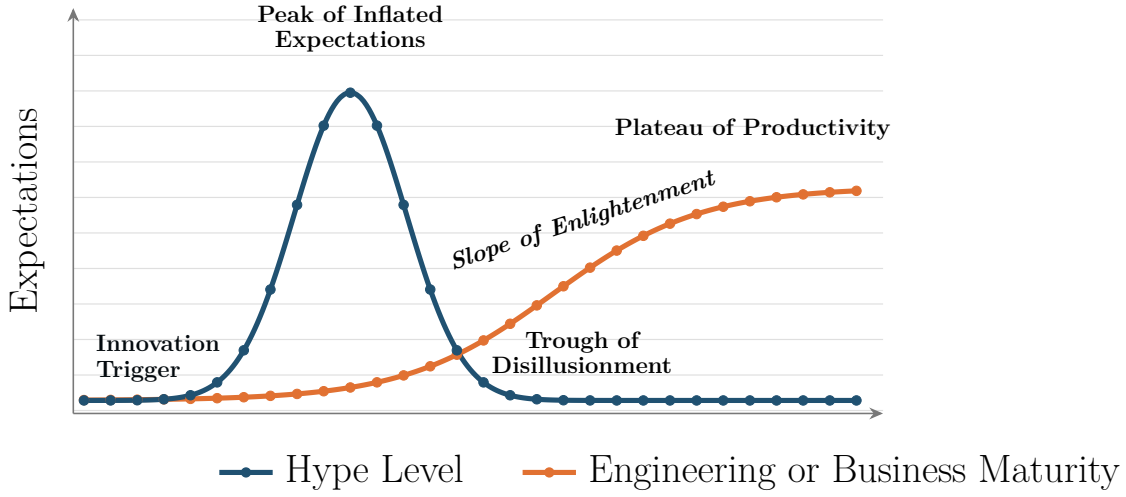


Figure 27: Hype Cycle: the Hype Level and the Engineering or Business Maturity curves [36].

Property	Sub-property	Meaning
Reactive		Responds in a timely fashion to changes in its environment.
Complex Mental State		The agent has knowledge, belief, intention, and obligation.
Graceful Degradation		Correctly handles a degraded operational state; it dies properly.
Temporally continuous (in the multi-agent system)		Once created, an agent is always running, but in a multi-agent system some agents may have a short life while others are persistent.
Situated		The agent is located in an environment, perceives it through sensors and acts on it through effectors.
Autonomy		An agent takes initiative and exercises control over its own actions.
	Individuality	Clearly identifiable artificial entity with well-defined boundaries and interfaces with its environment.
	Self-control	Does not blindly obey commands, but can modify requests, negotiate, ask for clarifications, or even refuse.
	Flexible	Actions are not scripted; the agent is able to dynamically choose which actions to invoke, and in what sequence.
	Goal-oriented (purposeful)	Accepts high-level requests indicating what another agent (human/software) wants and takes responsibility for deciding how and where to satisfy the requests on behalf of its requester.
	Proactive Self-starting	Ability to take the initiative; an agent can reason to decide when to act.
Personality		An agent has a well-defined, believable personality and emotional state that can improve interaction with human users.
Communication (socially able)		An agent can engage in complex communication with other agents (human/software), to obtain information or enlist help in accomplishing its goals.
Adaptability		An agent customizes itself to the preference of its user and adapts to changes in its environment.
	Customizable Educable	Can be customized by another agent (most probably human) to meet preferences.
	Learning	Automatically customizes itself and adapts to changes in its environment (user preferences, networks, information...) and modifies its behavior based on its previous experience.
Migration		An agent can move from one system environment to another, possibly across different system architectures and platforms.
Mobility		An agent is embedded in a real-world moving artefact.
Visual representation		The agent has a visual representation; this goes from classic interfaces to anthropomorphized and virtual-reality ones.
Veracity		The agent will not knowingly communicate false information.
Benevolence		Agents do not have conflicting goals, and every agent will therefore always try to do what is asked of it.
Rationality		An agent will act in order to achieve its goals, and will not act in such a way as to prevent its goals from being achieved—at least insofar as its beliefs permit.

Table 22: Agent characteristics [50].

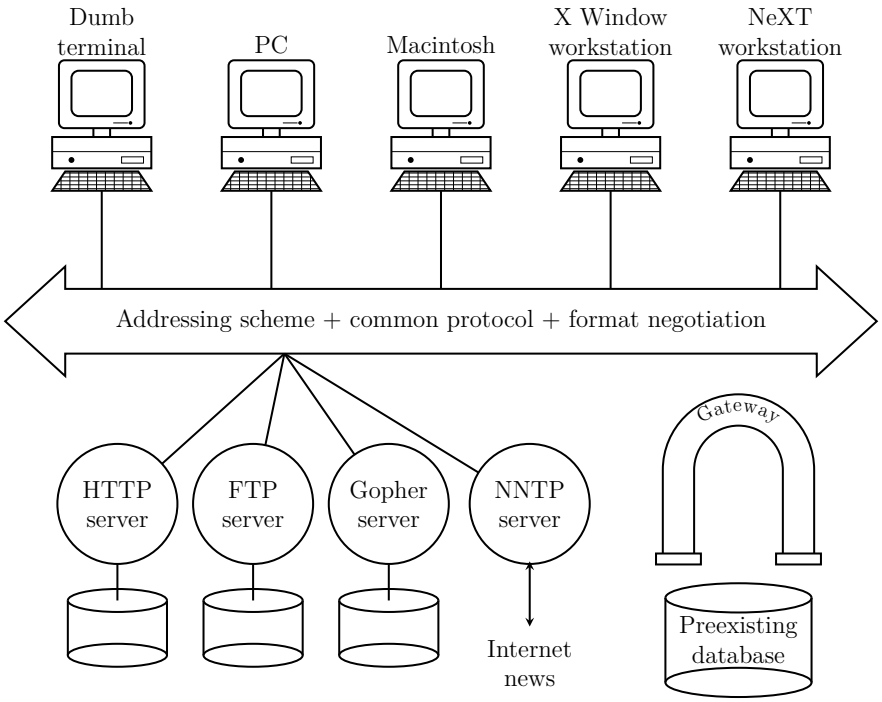
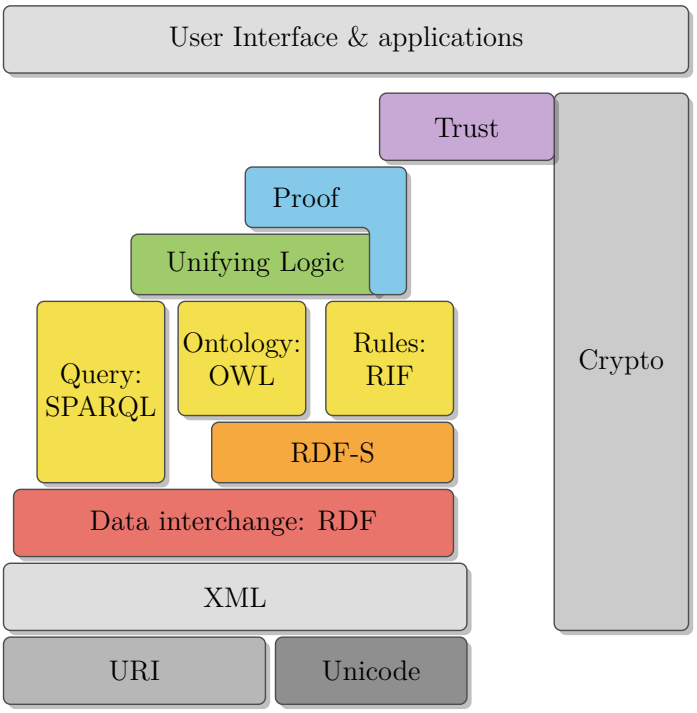
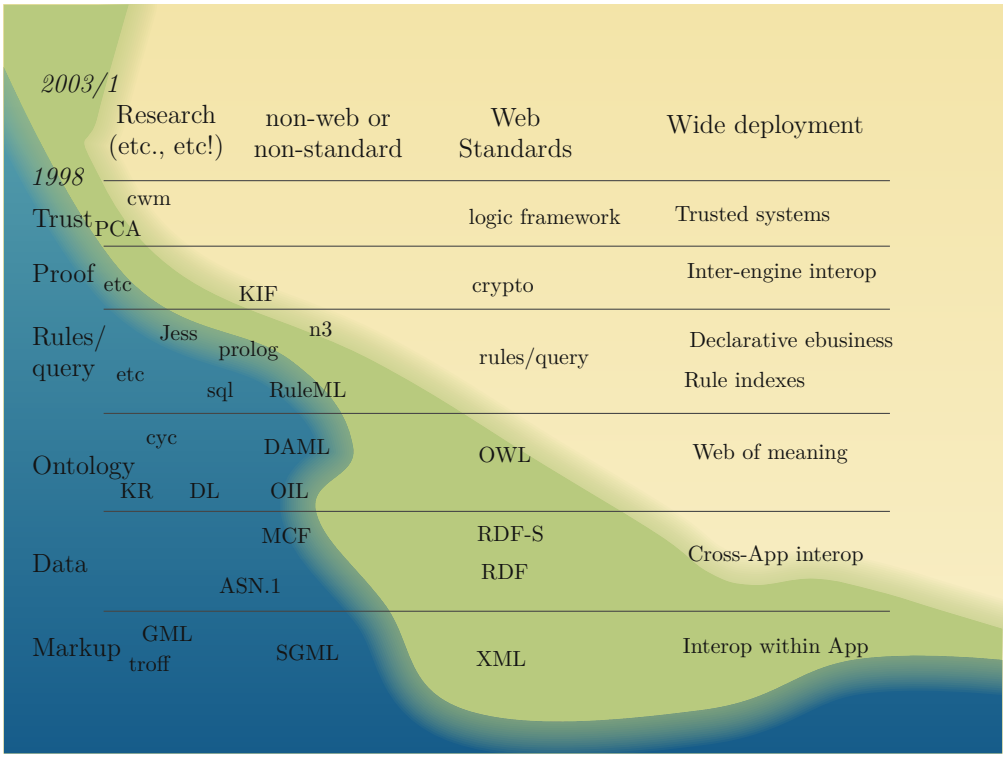


Figure 28: Web architecture by Tim Berners-Lee in 1990 [37].



(a) Semantic Web Stack (Tim Berners-Lee, 2006), redrawn from [145].



(b) The Semantic Web "Wave" rollout vision (Tim Berners-Lee, 2003), redrawn from [99].

Figure 29: The Semantic Web as envisioned by Tim Berners-Lee: (a) the layered technology stack (2006) and (b) the 2003 rollout-vision "wave." Both panels are redrawn from the original sources.

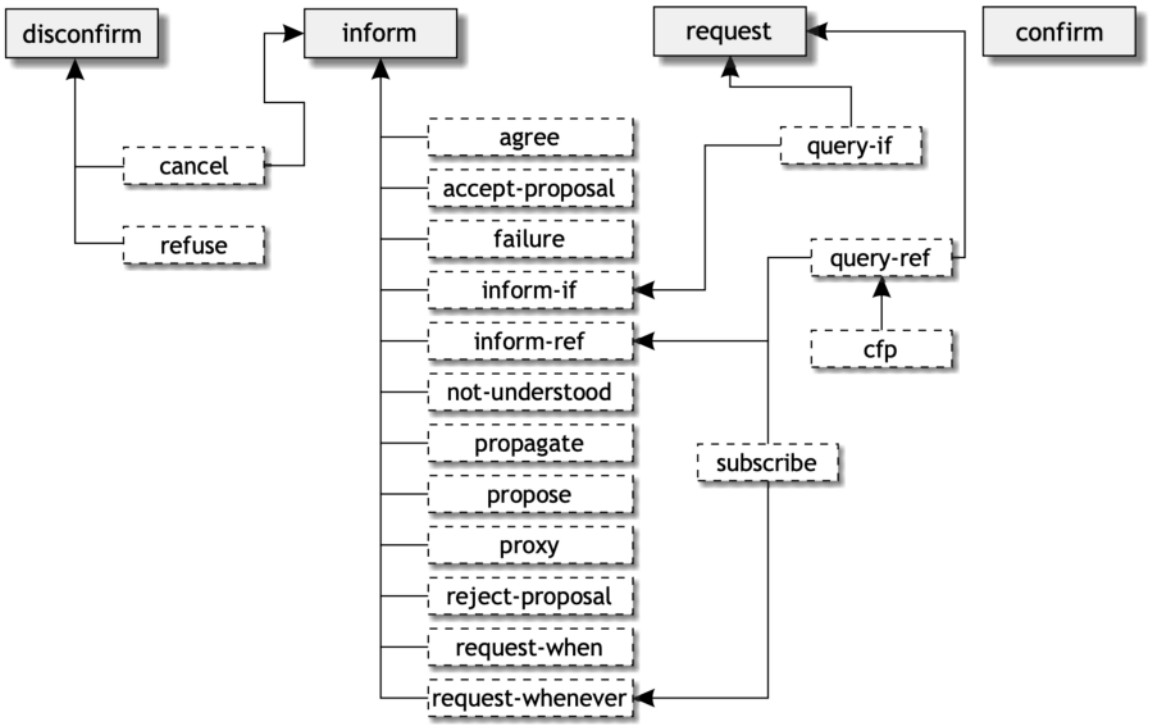


Figure 30: Relationships between communicative acts defined by FIPA [38].

Program map — extended phase and artefact detail (Chapter 1)

Phase I

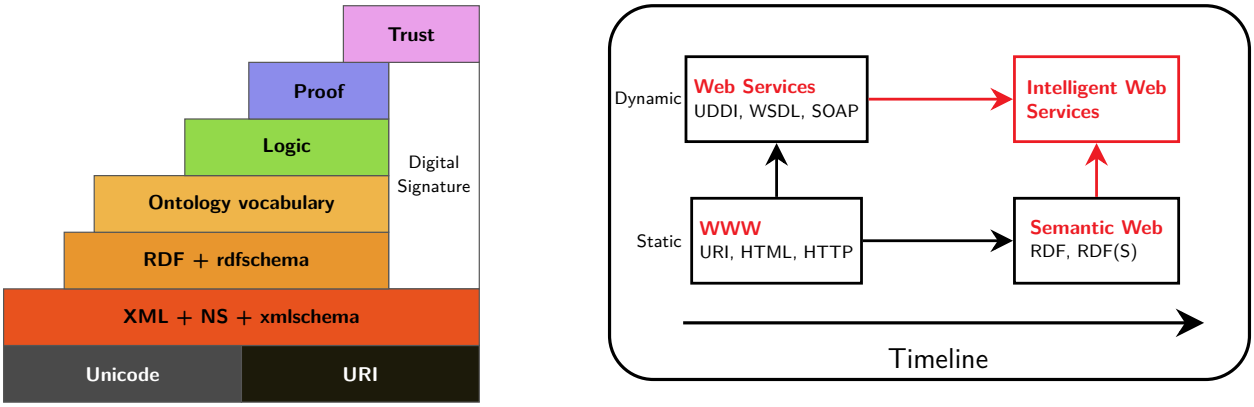


Figure 31: Phase I: Semantic Web Stack [39] and Intelligent Web Services [40].

Table 23: Candidate's individual role per main artefact and its lead publication (Art. = main artefact, Table 5; Ph. = phase).

Art.	Ph.	Lead publication	Candidate's individual role
A1	I	ICEIS 2004 [7]	Sole author of the Master's and Licentiate theses; co-author of the ICEIS 2004 paper, where I implemented the protocol adaptation.
A2	I	Licentiate thesis 2006 [2]	Sole author of the Licentiate thesis; co-author of the ERCIM News 2004 note; implemented the B2B process descriptions. Weakest publication pedigree (ERCIM News is a magazine, not a peer-reviewed paper)—the Licentiate thesis is itself the fullest written elaboration.
A3	I	CAPS 2005 [3]	First author of the CAPS 2005 paper (with O. Riva and S. Toivonen) and sole author of the Licentiate thesis; co-author of the poster and SemAnnot papers (a multi-author VTT project; my part was the matching engine, the context ontology, and the benchmarks, and in the SemAnnot paper the Semantic Web prototype and the distributed-cognition analysis); fourth author of the IWUC 2004 precursor paper.
A4	II	PCSI 2009 [14]	Sole author of the core results (PCSI 2008/2009, ESTSP 2008, EDBT 2009 PhD workshop, ICEIS 2009): system design and all experiments; first author of the 2011/2012 cost-weight and affinity decks; co-author of the Lääkärilehti editorial.
A5	III	HADA [28]	First author of the RAGADA → AADAR → HADA line (architecture and demonstrator); supporting co-author of the PROS/SMS management-seed papers (Leena Pitkäranta lead).
A6	III	CHIRA 2025 [24]	Sole author throughout the benchmark stream (design, construction, and public release).
A7	III	PCSI 2025 [29]	First author—designed the curated-learning approach and the decision-tree ML analysis; Leena Pitkäranta co-author (human–AI decision model, organizational framing).

Table 24: Phase I individual artefacts and their mapping to the main artefacts (Table 5).

ID	Artefact	Main Artefact	Source section
A.I.1.1	Process Flow	Run-time interaction-protocol adaptation engine	Section 4.1.1
A.I.1.2	Interaction Protocol Ontology (adopted)	Run-time interaction-protocol adaptation engine	Section 4.1.1
A.I.1.3	Interaction Protocol Serializations	Run-time interaction-protocol adaptation engine	Section 4.1.1
A.I.1.4	Adapting to Interaction Protocol Descriptions	Run-time interaction-protocol adaptation engine	Section 4.1.1
A.I.2.1	Purchase Process Flow	Ontology-driven B2B negotiation prototype	Section 4.2.1
A.I.2.2	Decision Making	Ontology-driven B2B negotiation prototype	Section 4.2.1
A.I.2.3	Data Models and Ontologies	Ontology-driven B2B negotiation prototype	Section 4.2.1
A.I.2.4	Locating Services / Registry	Ontology-driven B2B negotiation prototype	Section 4.2.1
A.I.3.1	Context-Profile-Content Ontologies	DYNAMOS context-aware service matcher	Section 4.3.1
A.I.3.2	Matching Engine & System Architecture	DYNAMOS context-aware service matcher	Section 4.3.1
A.I.3.3	Event-Driven Distribution Layer	DYNAMOS context-aware service matcher	Section 4.3.1

Table 25: Phase II individual artefacts and their mapping to the main artefacts (Table 5).

ID	Artefact	Main Artefact	Source section
A.II.1.1	Process Flow	ML-enabled clinical recommendation MAS	Section 5.1.1
A.II.1.2	Data Space	ML-enabled clinical recommendation MAS	Section 5.1.1
A.II.1.3	ML/IR Model	ML-enabled clinical recommendation MAS	Section 5.1.1
A.II.1.4	MAS Architecture	ML-enabled clinical recommendation MAS	Section 5.1.1

Table 26: Phase III individual artefacts and their mapping to the main artefacts (Table 5).

ID	Artefact	Main Artefact	Source section
A.III.1.1	Strategy-Process Blueprint	HADA architecture	Section 6.1.1
A.III.1.2	Stakeholder & User-Story Specification	HADA architecture	Section 6.1.1
A.III.1.3	Prototype AI Tool `getLoanDecision`	HADA architecture	Section 6.1.1
A.III.1.4	AI-Responsibility Tool (RACI Matrix)	HADA architecture	Section 6.1.1
A.III.1.5	Generic HADA Architecture	HADA architecture	Section 6.1.1
A.III.1.6	Agent Modeling	HADA architecture	Section 6.1.1
A.III.2.1	NordDRG Full Specification	NordDRG AI Benchmark	Section 6.2.1
A.III.2.2	Expert documentation	NordDRG AI Benchmark	Section 6.2.1
A.III.2.3	NordDRG Logic Benchmark	NordDRG AI Benchmark	Section 6.2.1
A.III.2.4	NordDRG Grouper Benchmark	NordDRG AI Benchmark	Section 6.2.1
A.III.2.5	Curated table excerpts (FI / FI–EN)	NordDRG AI Benchmark	Section 6.2.1
A.III.2.6	Lightweight reference agents	NordDRG AI Benchmark	Section 6.2.1
A.III.3.1	Decision-making model	CCL process	Section 6.3.1
A.III.3.2	Participants and workflows	CCL process	Section 6.3.1
A.III.3.3	CCL governance process	CCL process	Section 6.3.1
A.III.3.4	NordDRG decision model representation	CCL process	Section 6.3.1
A.III.3.5	LLM-based software agents	CCL process	Section 6.3.1

Original contributions in detail (Section 1.6.3)

The eight original contributions summarized in Section 1.6.3 are given here in full. The eight contributions are best read *by phase and as a single dependency line* rather than as eight independent results; this cross-phase reading is the dissertation's own synthesis (**C8**), while each of **C1**–**C7** is presented as a design-science artefact contribution following [91]. Phase I establishes the agent and Semantic-Web framing—run-time protocol adaptation, ontology-driven negotiation, and context-aware service matching (**C1**–**C3**); the tooling and standards-convergence limits encountered there motivate the Phase II pivot to data-centric machine learning over large coded datasets (**C4**). Phase III then places governed LLM reasoning inside two harnesses that share one governance layer—the CCL maintenance process in depth (**C7**) and the HADA alignment architecture in breadth (**C5**)—both resting on the capability premise tested by the NordDRG AI Benchmark (**C6**), to the author's knowledge the first public LLM benchmark for DRG hospital-funding logic (Figure 2). Binding all of them is the longitudinal governable-agency through-line (**C8**). Contributions C1–C7 each answer one or more research questions, while C8 is cross-cutting; Table 3 gives the full contribution-to-question mapping.

C1 Run-time interaction-protocol adaptation engine. (Phase I; artefact A1; research question I.2; developed in Section 4.1.) *Primary publication:* ICEIS 2004 [7] (peer-reviewed). *Elaborated in:* the Master's thesis [1] (2004), consolidated in the Licentiate thesis [2] (2006). *Role:* sole author of the theses; co-author of the ICEIS paper. *In this dissertation:* unified as artefact A1 and positioned as the origin of the agents-over-substrates through-line, with a 2026 retrospective linking it to LLM-agent interoperation; no new results.

C2 Ontology-driven B2B negotiation prototype. (Phase I; artefact A2; research question I.2; developed in Section 4.2.) *Primary publication:* the Licentiate thesis 2006 [2] (sole author; the fullest elaboration of the B2B negotiation work). *Related early account:* ERCIM News 2004 [5] (a magazine note, not peer-reviewed), which sketches the B2B scenarios of the PAX project (VTT Information Technology with CIFE, Stanford University)—service-chain composition with RFQ bidding, and protocol-based expediting—conceptually, rather than describing the prototype itself. *Developed from:* the Master's thesis [1]. *Role:* sole author of the Licentiate thesis; co-author of the ERCIM note. *In this dissertation:* the dissertation consolidates the Licentiate-thesis elaboration as artefact A2 and reads its bounded-authority pattern as the earliest instance of the governable-agency through-line (Section 4.2).

C3 DYNAMOS context-aware service matcher. (Phase I; artefact A3; research question I.3; developed in Section 4.3.) *Primary publication:* CAPS 2005 [3] (peer-reviewed; first author). *Developed from:* the context-sensitive agent-interaction work of IWUC 2004 [6]; demonstrated in the companion CAPS 2005 poster [4] and extended in SemAnnot@ISWC 2005 [8]; consolidated in the Licentiate thesis [2]. *Role:* first author of the CAPS paper (with O. Riva and S. Toivonen) and sole author of the Licentiate thesis; co-author of the poster and SemAnnot papers (a multi-author VTT project, where my part was the matching engine, the context ontology, and the benchmarks); fourth author of the IWUC paper. *In this dissertation:* framed as the culmination of Phase I and the substrate shift from protocols to live user context, with a 2026 mapping to LLM-agent personalization.

C4 ML/IR-enabled clinical recommendation multi-agent system (accuracy measured on real hospital data; query latency tested on synthetic databases of up to 10^8 coded episodes). (Phase II; artefact A4; research question II.1; developed in Section 5.1.) *Primary publication:* PCSI 2009 [14] (peer-reviewed; the multi-agent, medical-domain system). *Developed from:* PCSI 2008 [10] and ESTSP 2008 [11], then the EDBT PhD workshop [12] and ICEIS 2009 [13]; with supporting items [9, 15, 16, 17]. *Role:* sole author of the core results; first author of the 2011/2012 cost-weight and affinity work; co-author of the medical-journal editorial. *In this dissertation:* the 2008–2012 publication stream is unified into one Phase II artefact under consistent terminology; no new science, and the supporting items are not independent contributions.

C5 HADA — Human–AI Agent Decision Alignment Architecture. (Phase III; artefact A5; research questions III.1 and III.3; developed in Section 6.1.) *Primary publication:* HADA, presented at AGENTICS 2025 [28] (peer-reviewed; part of IJCCI 2025; cited here in its arXiv version). *Developed from:* RAGADA (ICEIS 2024) [26] and AADAR (Agentic Algorithmic Decision Alignment with RAG; Springer LNBIP 2025) [27], with organizational-framing seeds [18, 19]. *Role:* first author of the RAGADA, AADAR, and HADA papers; supporting co-author of the seed papers. *In this dissertation:* consolidated as artefact A5 (subsuming the RAGADA/AADAR/HADA names) under a uniform scheme and positioned in the Phase III through-line. The core artefact was already peer-reviewed; the dissertation adds a fuller prototype description (Section 6.1) and a per-objective reassessment (Section 7.5), but no new empirical result.

C6 NordDRG AI Benchmark for LLMs. (Phase III; artefact A6; research question III.2; developed in Section 6.2.) *Primary publication:* CHIRA 2025 [24] (peer-reviewed article; second-best-paper recognition). *Developed from:* PCSI 2024 [20], CHIRA 2024 [21], and the public arXiv release [23]. *Role:* sole author throughout. *In this dissertation:* the integrative artefact-A6 framing of the *public, open benchmark release* and its throughput analysis (both already made with the 2025 papers), explicit evaluation caveats (one scored answer per item, no variance estimates), and the longitudinal reading of the benchmark's evolution: the 2024 single-question probes approached saturation as 2025 frontier models caught up, forcing the extension to full grouper emulation. The benchmark itself was already peer-reviewed.

C7 CCL — Collaborative Curated Learning process. (Phase III; artefact A7; research question III.3; developed in Section 6.3.) *Primary publication:* PCSI 2025 [29] (peer-reviewed article; best-paper award). *Role:* first author, with L. Pitkäranta. *In this dissertation:* the paper presented curated learning over the NordDRG decision tree together with its evaluation; *this dissertation gives the full account as the structured artefact A7*—a governed, human-in-command process for maintaining decision rules—and replaces the paper's preliminary node counts with path-wise decision-point counts.

C8 Cross-phase synthesis. (Phases I–III; cross-cutting.) *Primary publication:* none—this is the dissertation's own synthesis, positioned by the "forgotten wave" analysis [25] (IJCKG 2025, peer-reviewed) and the "age of AI applications" essay [30] (arXiv preprint). *Role:* first author of the IJCKG paper (with E. Hyvönen) and sole author of the arXiv essay. *In this dissertation:* the new contribution, the longitudinal *governable-agency through-line* (Section 1.1.2), which binds the work of 2001–2025 into one cumulative argument. While the representational substrate shifts, the constant is the demand for agent behavior that is interoperable, auditable, and aligned with human command.

The three phases reassessed in 2026 (Sections 1.2.4, 1.3.2, and 1.4.4)

The "from a 2026 perspective" subsections of Chapter 1 summarize the reassessment of each phase; for Phases I and III it is given here in full, while the Phase II reassessment is given in full in Section 1.3.2.

Phase I (Section 1.2.4)

Run-time protocol adaptation (Section 1.2.1). The construction expediting scenario required agents to dynamically interpret and execute partner-specific interaction protocols without hard-coding. This exact problem resurfaces in 2026 as LLM agents invoke tools through OpenAPI specifications, the Model Context Protocol (MCP), and Google's Agent2Agent (A2A) protocol (see Section 2.5). The interface layer has consolidated rather than fragmented: MCP became the de facto tool-and-data standard and, together with Block's goose and OpenAI's `AGENTS.md`, was placed under vendor-neutral governance in the Linux Foundation's Agentic AI Foundation in December 2025 [66]; A2A likewise moved under Linux Foundation stewardship [67]; and portable procedural knowledge is packaged as *Agent Skills*, folders of instructions an agent loads on demand [241]. Where Phase I agents parsed RDF-serialized FIPA protocols with hand-written interpreters, a 2026 agent reads a JSON schema, a tool description, or a skill file at runtime and lets the language model do the interpreting. The *form* of the solution is therefore close to Phase I: explicit, machine-readable interface descriptions are once again the lingua franca, and only the interpreter has changed. What has not changed is the semantic gap. The failure modes have simply moved from "cannot parse the interface" to "cannot be trusted to act under it": prompt injection through tool outputs, poisoned tool descriptions, and confused-deputy authorization are the 2026 names for the problem of binding a syntactic interface description to a genuine understanding of service semantics. The OAuth-based authorization now part of MCP addresses identity rather than meaning.

Ontology-driven service discovery and negotiation (Section 1.2.2). The B2B purchasing scenario's reliance on shared ontologies for automated supplier discovery and quote evaluation parallels how LLM systems ground their outputs in structured knowledge. Retrieval-Augmented Generation (RAG) over vector databases was the first form of that grounding; in 2026 practitioners speak of *context engineering*, the deliberate assembly of retrieved documents, persistent memory, tool manifests, and skill files into the model's working context. Discovery has likewise been re-standardized: A2A agent cards, MCP server registries, and skill manifests play the role that service ontologies and directory facilitators played in Phase I, and agent-to-agent purchasing is beginning to appear in commercial products. The fundamental tension that constrained Phase I—semantic precision through explicit ontologies versus practical scalability through statistical methods—has not been resolved but *relocated*: the statistical interpreter is excellent at matching and weak at guaranteeing, which is why interfacing LLM agents with structured business logic still requires

exact-match evaluation of the kind the Phase III benchmark provides (Section 1.4.2). It is also why I have argued elsewhere that the Semantic Web's structured-description program was abandoned rather than refuted [25].

Context-aware service matching (Section 1.2.3). The DYNAMOS system's ambition to deliver location-, time-, and activity-aware services to mobile users prefigures today's personal agents. In 2026 the multimodal context that DYNAMOS had to assemble from a handful of sensors is native to the model: speech, images, location, and calendar state enter the same context window, and persistent cross-session memory supplies the user profile. The Phase I insight that context requires both a static profile and a dynamic state is reproduced almost literally in the memory-plus-context split of current agent runtimes, though it is implemented through retrieval and memory stores rather than rule-based matching. The resource constraint has also returned in a new guise: the field-trial concern with battery, latency, and attention on a 2005 smartphone reappears as the 2026 concern with inference cost, latency, and interruption budgets for always-on agents.

Why these problems persist. The persistence of these challenges across technological generations suggests they represent invariant aspects of the agent–service integration problem rather than mere implementation difficulties. The Phase I work's value lies not in its technical solutions—which were constrained by era-appropriate technology—but in its identification of where human semantics, machine reasoning, and distributed coordination must interface. Solution approaches have not simply progressed from symbolic to statistical to generative paradigms: the generative paradigm has re-adopted explicit, versioned interface descriptions (schemas, protocols, agent cards, skill files) as its coordination substrate and is now standardizing them under neutral governance. As a result, the open questions of Phase I—who defines the shared vocabulary, how a runtime interpreter is held to it, and how its actions are made accountable—are again the active questions of the field.

Phase III (Section 1.4.4)

The Phase III studies were carried out in 2023–2025, so a 2026 reassessment covers a short interval. It is nonetheless necessary, because the substrate under the three artefacts—frontier models and the agent harnesses around them—changed faster during 2025–2026 than during any earlier phase of this dissertation. The reassessment follows the order of Figure 2: the capability premise (A6), the depth harness (A7), and the breadth harness (A5). It is written with the benefit of having used the same class of agents on this manuscript: the 2026 revision of the thesis was itself carried out with coding agents under human-in-command review, as the Epilogue describes (Chapter 9).

What changed in the substrate. Three developments frame the reassessment. First, *coding agents*—Anthropic's Claude Code and OpenAI's Codex are the representative products—matured from code-completion tools into autonomous workers that plan, edit, run tests, and open pull requests in isolated branches [89, 90]. In February 2026 OpenAI reported an internal product of about a million lines of code built over five months without hand-written code, and introduced the term *harness engineering* for the systems that "constrain, inform, verify, and correct" an agent [90]. The same word that this dissertation uses for CCL and HADA (Section 1.4) has thus become an industry term for the same idea. Second, the length of task a frontier agent can complete autonomously has kept growing: METR's time-horizon measure, which tracks the human task length at which an agent succeeds half the time, continued to double every few months through 2025 and into early 2026 [390], and repository-level coding benchmarks such as SWE-bench [284] are, in the author's reading of 2026 leaderboards, close to saturated. Third, the interface layer standardized under neutral governance (MCP, A2A, `AGENTS.md`, and, since October 2026, Agent Skills [391]; Section 1.2.4), so that an agent's tools, operating procedures, and inter-agent messages are now portable across vendors.

The capability premise revisited (NordDRG AI Benchmark, A6). The mid-2025 results stand as reported: the strongest models mastered the rule-level *Logic* suite (13/13) while exact, traceable emulation of the grouper's control flow stayed hard (best 7/13), with failures concentrated in `ORD` priority and CC/MCC exclusions (Section 7.6). Those results measure a model reasoning *in context* over a decision-making model of roughly 10^7 decision points, and the failure modes are the characteristic failures of in-context execution: dropped priorities and missed exclusions in a long, branching procedure. The 2026 substrate suggests the natural next extension. Instead of asking the model to *emulate* the grouper, a coding-agent track would ask the agent to *construct* it: given only the released artefact bundle (definition tables, manuals, change templates), produce executable grouper software whose output matches the official grouper exactly—both

`drg_nat` and the triggering `drg_logic.id`—over the full case universe rather than thirteen hand-picked cases. If this succeeds, emulation of the entire decision tree becomes both fast, since the generated program runs in milliseconds per episode, and exact, since the program is deterministic and testable; the benchmark's strict exact-match criterion then serves as the test oracle for the generated code rather than as a score for a chat answer. The existing Logic and Grouper suites would remain as *reading* tests of the specification, and the new track would test *construction* from it. Two design cautions follow from the benchmark's own methodology. The agent must be constrained to the released artefacts, because open grouper implementations exist on the public web and would otherwise contaminate the result; and the scoring cases must be held out from the agent, since a coding agent that can see its test cases will satisfy them rather than the specification. The open question thereby moves from "can a model execute this control flow" to "can an agent build a faithful executable from a governed specification, and can a human audit what it built"—a question adjacent to CCL's own: CCL governs the change of the model of record, but its gates already presuppose a deterministic, executable instance of that model (the decision list evaluated with first-hit precedence; Section 6.3.1.4) against which every change is regression-tested in CI by design, so faithful construction is a precondition of CCL's testing stage rather than its governed object. A construction track would extend that stage to the executable itself.

The depth harness revisited (CCL, A7). My assessment is that CCL has become *easier to instantiate* and that its hard problems have not gone away. Easier, because the 2026 coding agents ship, as standard features, most of what the CCL Platform (Section 6.3.2.1) had to build as a bespoke FastAPI and Next.js application: isolated worktrees per change, pull requests as the unit of proposal, automated test gates, agent-to-agent review before human review, permission modes that fail closed, and project-level operating procedures expressed as `AGENTS.md` files and skills. OpenAI's own account notes that most review effort in its agent-first workflow has shifted to agent-to-agent review with human review optional [90]; the Contributor and Reviewer roles of CCL (Section 6.3) are therefore no longer an architecture one must construct but a configuration one must govern. The 2026 revision of this manuscript offers an informal, small-scale illustration of this: coding agents proposed batches of changes in isolated branches and I merged them, so that the human-in-command property held while my role moved from approving sentences to approving batches (Chapter 9). What remains hard is instructive, because none of it is a harness feature. The *oracle problem* is unchanged: a coding harness verifies code against tests, but a change to a decision rule has no test until the official grouper output, the historical expert change, or a certified reviewer supplies one, which is why CCL's grounding and provenance requirements (line-level lineage, counterfactual evidence, reviewer sign-off) are still the binding constraint. The *representation problem* (SRQ.III.3.1) is unchanged: a 10^7-decision-point model-of-record must still be line-addressable and semantically sliced for an agent to propose atomic diffs, and no 2026 harness supplies that for a domain artefact. The *accountability problem* has, if anything, sharpened: as review migrates to agents, the human sign-off concentrates on larger batches, so the risk that CCL guards against—expert ratification degrading into rubber-stamping—grows with the harness's convenience. Hence the risk-tiered gates and immutable ledger of CCL become more rather than less necessary. Finally, the security surface of an agent that reads tool outputs and third-party skill files—prompt injection and poisoned tool descriptions—is broader in 2026 than when CCL was designed, and it argues for the fail-closed gate policy that CCL already specifies. The same pull-request discipline could be applied to the agents themselves: the role instructions of a Contributor or Reviewer, written as a skill, are plain text that could be versioned, reviewed, and merged by a human in the same way (Section 6.3.1.3). They would still sit outside CCL's governed object, the model-of-record (Section 6.3.1.1), and an instruction has no deterministic instance for gate G_1 or invariant I1 to check, so a regression in a skill would surface only in the agents' outputs and would have to be caught by tests on those outputs, such as exact-match cases of the kind the NordDRG AI Benchmark uses (Section 6.2).

The breadth harness revisited (HADA, A5). HADA's three gaps (Section 1.4.1) have closed to different degrees. The *Integration Gap* has narrowed most: the vendor-agnostic, protocol-agnostic substrate that design objective O.III.1.6 had to assume now exists under neutral governance, and HADA's three layers map onto it cleanly—strategy expressed as versioned policy, skill, and `AGENTS.md` files; coordination over A2A; execution over MCP tools. The *Containment Gap* has narrowed at the level of the single tool call (sandboxes, permission modes, OAuth-based authorization for enterprise MCP deployments) but not at the level HADA addresses: delegated agent identity is still being standardized, and an organization-wide circuit-breaker over thousands of agents remains bespoke. The *Alignment Gap* is essentially untouched by the standards work: nothing in MCP, A2A, or the skills format binds an agent's runtime policy to quantified objectives and codified values, and the regulatory driver, the EU AI Act, has been deferred rather than withdrawn [75]. HADA's durable contribution is therefore its OKR-and-values binding with an audit ledger,

not its conversational interface, which was differentiating in 2024 and is commodity in 2026. Seen from today, the two objectives that the banking-sandbox evaluation met only partially—stakeholder alignment across time-scales and scalability with near-real-time propagation (Section 7.5)—are the ones that the 2026 substrate now makes testable at realistic scale, and they are where a re-evaluation of HADA should begin.

What the reassessment leaves in place. The Phase III claims were deliberately scoped to what the demonstrators showed (Section 7.8), and the 2026 substrate does not overturn any of them; it moves the frontier. The benchmark's central result—that the strongest models master the rule-level reading of this specification but not its faithful execution—still holds for in-context execution and now motivates a construction track. The governance structures of CCL and HADA are, by design, independent of any particular model or agent harness, and the time since their design has made them cheaper to implement while leaving the problems they address—oracles, representation, accountability, and value binding—as open problems of agentic systems. This is the agents-over-substrates through-line of the dissertation seen at its shortest time-scale: the substrate improves monthly, and what persists is the question of how humans remain in command of it.

Classical agent reference diagrams (Chapter 2)

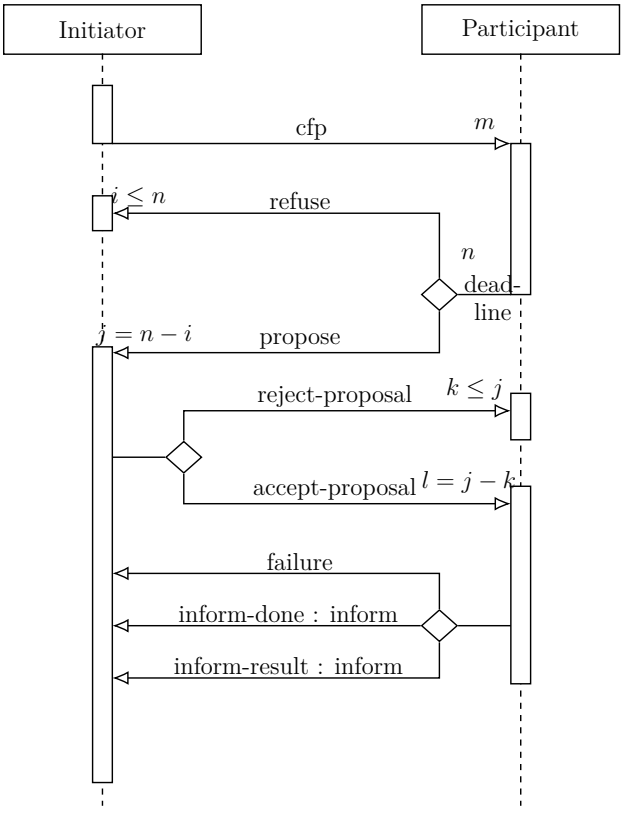


Figure 32: The FIPA Contract Net Interaction Protocol [41].

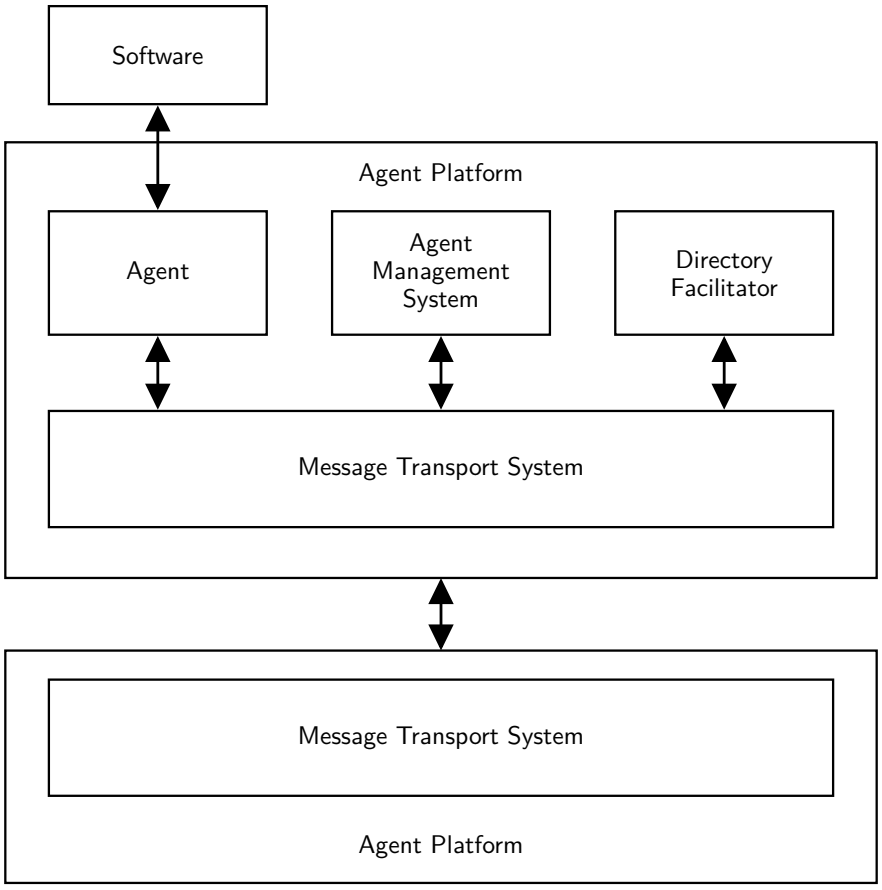


Figure 33: Agent Management Reference Model (redrawn from [42]).

Formal definitions — supplementary figures, tables, and notation (Chapter 3)

Table 27: Core symbols of the Transformer architecture, with the values of GPT-3 175B [51] as an example.

Symbol	Description	Example (GPT-3, 175B)
$\lvert\mathcal{V}\rvert$	Vocabulary size (number of unique tokens)	50,257
n	Sequence length (number of input tokens)	2,048
d_{model}	Model or embedding dimension	12,288
h	Number of attention heads	96
$d_k = d_v$	Dimension of keys, queries, and values per head	128
d_{ff}	Hidden width of the position-wise feed-forward network	49,152
L	Number of Transformer layers (stacked attention blocks)	96

Transformer functions (Section 3.1.2). The constituent functions of the *Transformer Mathematical Model* of Section 3.1.2 are given here (Definition A.1 and Eqs. 5–16), in the notation of Table 27.

Model Dimensions and Notation. Table 27 lists the main model-level hyperparameters and example magnitudes from GPT-3 175B [51] (the corresponding GPT-4 values are not publicly disclosed), and Table 28 defines the principal matrices and intermediate representations used in the mathematical formulation of attention and embedding operations. The formulas follow the row-vector convention of [52]: one token per row, so that a token vector is $x \in \mathbb{R}^{1\times d_{\text{model}}}$ and an affine map is written $xW + b$ (the transpose of the column form $Wh + b$ of Definition 3.1.5); for a matrix argument, f_{FFN} and f_{LN} act row-wise, with their bias, scale, and shift vectors broadcast to every row.

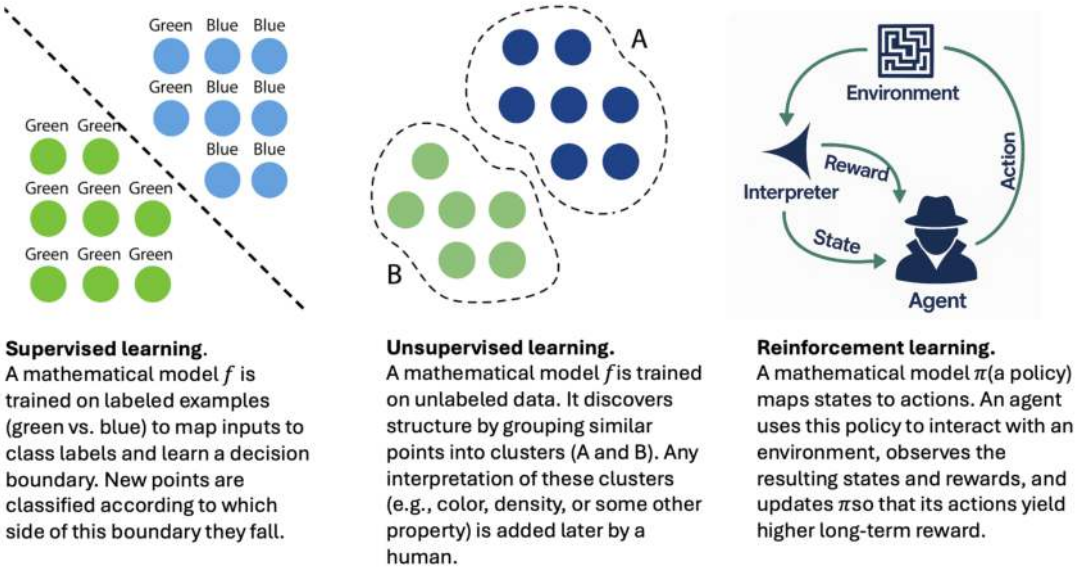


Figure 34: Simple illustration of the learning settings *Supervised Learning*, *Unsupervised Learning*, and *Reinforcement Learning*, each of which induces an *Optimization Problem*.

Table 28: Key matrices and intermediate representations in a Transformer block (for the activation matrices, row = token and column = feature).

Symbol	Description	Dimensions
Z	One-hot token matrix representing discrete input tokens	$n \times \lvert\mathcal{V}\rvert$
W^E	Token embedding matrix (maps vocabulary to model space)	$\lvert\mathcal{V}\rvert \times d_{\text{model}}$
E_{token}	Token embeddings (ZW^E)	$n \times d_{\text{model}}$
P_{pos}	Positional encodings (sinusoidal or learned)	$n \times d_{\text{model}}$
X	Input representation ($\sqrt{d_{\text{model}}}\, ZW^E + P_{\text{pos}}$)	$n \times d_{\text{model}}$
W_i^Q, W_i^K, W_i^V	Learned projection matrices for attention head i	$d_{\text{model}} \times d_k$ (or d_v)
Q_i	Query matrix of head i (XW_i^Q; X is replaced by $f_{\text{LN}}(X)$ inside a Pre-LN block)	$n \times d_k$
K_i	Key matrix of head i (XW_i^K)	$n \times d_k$
V_i	Value matrix of head i (XW_i^V)	$n \times d_v$
$\text{Attention}(Q_i, K_i, V_i)$	Single-head attention output ($\text{softmax}(Q_i K_i^\top/\sqrt{d_k})V_i$; mask omitted, see Eq. 8)	$n \times d_v$
W^O	Output projection combining all heads	$(hd_v) \times d_{\text{model}}$
$f_{\text{MHA}}(X)$	Multi-head attention output (before residual add)	$n \times d_{\text{model}}$
$U = X + f_{\text{MHA}}\big(f_{\text{LN}}(X)\big)$	Post-attention residual state (input to FFN)	$n \times d_{\text{model}}$
$f_{\text{FFN}}(U)$	Feed-forward sublayer output (before residual add)	$n \times d_{\text{model}}$
$Y = U + f_{\text{FFN}}\big(f_{\text{LN}}(U)\big)$	Output of one Transformer block (Pre-LN)	$n \times d_{\text{model}}$
W^U	Output (unembedding) projection to vocabulary logits	$d_{\text{model}} \times \lvert\mathcal{V}\rvert$

A step-by-step breakdown of one attention head and of the combination of the h heads is tabulated in the Intuitive Breakdown below.

Definition A.1 (Transformer Mathematical Functions)**.** The *Transformer Mathematical Functions* are the *Function*s that instantiate the node types of a Transformer *Function Graph*: the input representation (Eq. 5), masked multi-head attention (Eqs. 8–10), the position-wise feed-forward and layer-normalization maps (Eqs. 11–12), their composition into a Transformer block and the full stack (Eqs. 13–15), and the output projection (Eq. 16). They are specified below for a token sequence $x = (x_1, \ldots, x_n)$ and width d_{model}.

Input Representation. The input is formed by combining token embeddings and positional encodings. Let $X \in \mathbb{R}^{n \times d_{\text{model}}}$ be the sequence input:

$$X = \sqrt{d_{\text{model}}}\, E_{\text{token}} + P_{\text{pos}} = \sqrt{d_{\text{model}}}\, ZW^E + P_{\text{pos}}, \tag{5}$$

where $Z \in \{0,1\}^{n \times |\mathcal{V}|}$ is the one-hot token matrix representing the input sequence of n tokens over a vocabulary $\mathcal{V}$ of size $|\mathcal{V}|$, and $W^E \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ is the learned embedding weight matrix mapping each vocabulary item to a d_{model}-dimensional vector. The factor $\sqrt{d_{\text{model}}}$ follows [52]; GPT-style models omit it.

The positional encodings $P_{\text{pos}} \in \mathbb{R}^{n \times d_{\text{model}}}$ are defined in the original Transformer [52] using sinusoidal functions of varying frequencies (GPT models up to GPT-3 learn P_{pos} instead):

$$P_{\text{pos},\,2i} = \sin\Big(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\Big), \tag{6}$$

$$P_{\text{pos},\,2i+1} = \cos\Big(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\Big), \tag{7}$$

where $\text{pos} \in \{1, \ldots, n\}$ is the position index in the sequence (the row of P_{pos}), d_{model} is even, and $i \in \{0, \ldots, d_{\text{model}}/2 - 1\}$ indexes the pairs of feature dimensions (columns $2i$ and $2i+1$, counted from 0).

Multi-Head Attention. Let h denote the number of attention heads. For each head $i \in \{1, \ldots, h\}$, we define learned projection matrices $W_i^Q, W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, and an output projection $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$; usually $d_k = d_v = d_{\text{model}}/h$, so that $hd_v = d_{\text{model}}$ and W^O is square.

Each attention head computes, with $Q_i = XW_i^Q$, $K_i = XW_i^K$, and $V_i = XW_i^V$ as in Table 28:

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i) = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_k}} + \text{Mask}\right) V_i, \tag{8}$$

where softmax acts on each row, $[\text{softmax}(z)]_k = e^{z_k} / \sum_{k'} e^{z_{k'}}$ for a row z, so that each query row becomes a probability distribution over key positions (softmax is not elementwise and hence not an *Activation Function* in the sense of Definition 3.1.6), and $\text{Mask} \in \{0, -\infty\}^{n \times n}$ is the attention mask, with the convention $e^{-\infty} = 0$: under the causal mask of decoder-only models, $\text{Mask}_{jk} = -\infty$ for $k > j$, so that position j attends only to positions $\leq j$; Mask $= 0$ when attention is unmasked. The mask is not repeated in the formulas below, but in a decoder-only model every self-attention head carries the causal mask. Because row t of a causally masked attention output uses only rows $1, \ldots, t$ of its input, and f_{LN}, f_{FFN}, and the residual additions act on each row separately, row t of every $X^{(\ell)}$ in Eq. 15 depends only on $x_{1:t}$, which is why Eq. 16 can condition on $x_{1:t}$ alone.

The overall multi-head attention operation is then given by:

$$f_{\text{MHA}}(X) = \left(\bigoplus_{i=1}^{h} \text{head}_i\right) W^O, \tag{9}$$

where $\bigoplus$ denotes concatenation along the feature dimension (the Concat operation of [52]). Eq. 9 is the self-attention case: queries, keys, and values are all computed from the same sequence X.

Cross-Attention Variant. In encoder–decoder attention, queries are computed from the decoder sequence X while keys and values come from the encoder output $X_{\text{enc}} \in \mathbb{R}^{n_{\text{enc}} \times d_{\text{model}}}$ (so the attention weights form an $n \times n_{\text{enc}}$ matrix), with its own projection matrices W_i^Q, W_i^K, W_i^V, and W^O, distinct from those of self-attention:

$$f_{\text{MHA}}(X, X_{\text{enc}}) = \left(\bigoplus_{i=1}^{h} \text{softmax}\left(\frac{(XW_i^Q)(X_{\text{enc}}W_i^K)^\top}{\sqrt{d_k}}\right)(X_{\text{enc}}W_i^V)\right) W^O. \tag{10}$$

Feed-forward and normalization. Let $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ and $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$. Applied row-wise (per token),

$$f_{\text{FFN}}(x) = \phi(xW_1 + b_1)W_2 + b_2, \tag{11}$$

$$f_{\text{LN}}(x) = \frac{x - \mu(x)}{\sqrt{\sigma^2(x) + \epsilon}} \odot \gamma + \beta, \tag{12}$$

where ϕ is an *Activation Function* (ReLU in [52], GELU in GPT-style models), b_1, b_2 are bias vectors, $\mu(x) = \frac{1}{d_{\text{model}}} \sum_j x_j$ and $\sigma^2(x) = \frac{1}{d_{\text{model}}} \sum_j (x_j - \mu(x))^2$ are the mean and variance of x over its feature dimension, ϵ is a small constant for numerical stability, $\odot$ denotes the elementwise product, and $\gamma, \beta \in \mathbb{R}^{1 \times d_{\text{model}}}$ are learned scale and shift parameters; each occurrence of f_{LN} (two per block and one before the output projection) has its own γ, β.

Layer composition (Pre-LN, i.e., pre-layer normalization, as in current GPT-style LLMs). In modern LLMs (e.g., GPT-3 [51] and Llama 2 [269], the latter using the RMSNorm variant and rotary rather than additive position encodings), layer normalization is typically applied *before* the sublayers to improve training stability at scale:

$$U = X + f_{\text{MHA}}\big(f_{\text{LN}}(X)\big), \tag{13}$$

$$Y = U + f_{\text{FFN}}\big(f_{\text{LN}}(U)\big). \tag{14}$$

The map $X \mapsto Y$ of Eqs. 13–14, with the layer-specific parameters of $f^{(\ell)}_{\text{MHA}}$ and $f^{(\ell)}_{\text{FFN}}$, is the Transformer block Block_ℓ. A stack of L blocks applied to the input representation $X^{(0)} = X$ of Eq. 5 yields

$$X^{(\ell)} = \text{Block}_\ell\big(X^{(\ell-1)}\big), \quad \ell = 1, \ldots, L, \qquad f_{\text{Transformer}}(X^{(0)}) = \big(\text{Block}_L \circ \cdots \circ \text{Block}_1\big)(X^{(0)}) = X^{(L)}. \tag{15}$$

Output projection. In a Pre-LN decoder-only model the final hidden states are layer-normalized and projected onto the vocabulary; the next-token distribution at position t is

$$\hat{p}\big(\cdot \mid x_{1:t}\big) = \text{softmax}\Big(\big[f_{\text{LN}}(X^{(L)})\big]_t W^U\Big), \qquad W^U \in \mathbb{R}^{d_{\text{model}} \times |\mathcal{V}|}, \tag{16}$$

where $[\cdot]_t$ denotes the t-th row, the left-hand side is the row vector of next-token probabilities over $\mathcal{V}$ whose entry for a token v is $\hat{p}(x_{t+1} = v \mid x_{1:t})$, and W^U is the output (unembedding) projection, often tied to the token embedding as $W^U = (W^E)^\top$. This is the linear+softmax head of the *GPT* function graph shown in Figure 35.

Taking x_1 to be a start token, one forward pass yields all next-token distributions of a sequence, and their product is the autoregressive factorization $\hat{p}(x_{2:n} \mid x_1) = \prod_{t=1}^{n-1} \hat{p}(x_{t+1} \mid x_{1:t})$. A *GPT* is pre-trained by minimizing the negative log-likelihood of this factorization, i.e., the cross-entropy

$$-\frac{1}{n-1} \sum_{t=1}^{n-1} \log \hat{p}(x_{t+1} \mid x_{1:t}),$$

averaged over the training sequences; next-token prediction is thus formally supervised learning (Definition 3.1.11) on input–target pairs $(x_{1:t}, x_{t+1})$ derived from unlabeled text, which is why it is also called self-supervised, or unsupervised, pre-training. At inference time, generation iterates Eq. 16: the decoding policy π_{dec} (Definition 3.1.21) selects x_{t+1} from $\hat{p}(\cdot \mid x_{1:t})$ (e.g., the most probable token or a sample), the token is appended to the prefix, and the step repeats until an end token or a length limit is reached.

Intuitive Breakdown. For clarity, one attention head step by step, followed by the combination of the h heads (the mask of Eq. 8 is omitted; $\bigoplus$ denotes concatenation, Eq. 9):

Step	**Meaning**	**Shape**
XW_i^Q, XW_i^K, XW_i^V	Linear projections of input for head i	$n \times d_k$, $n \times d_k$, $n \times d_v$
$(XW_i^Q)(XW_i^K)^\top/\sqrt{d_k}$	Pairwise similarity between tokens	$n \times n$
$\text{softmax}(\cdot)$	Converts similarities to attention weights	$n \times n$
Multiply by XW_i^V	Weighted combination of values	$n \times d_v$
$\bigoplus_{i=1}^h \text{head}_i$	Concatenate head outputs	$n \times (hd_v)$
Multiply by W^O	Mix head outputs back into model dimension	$n \times d_{\text{model}}$

Remark A.2 (Symbol reuse across chapters). Within Chapter 3 the symbols listed here each have one meaning (the one exception, $\mathcal{L}$, is addressed at the end of this remark): the deep-learning symbols of Section 3.1

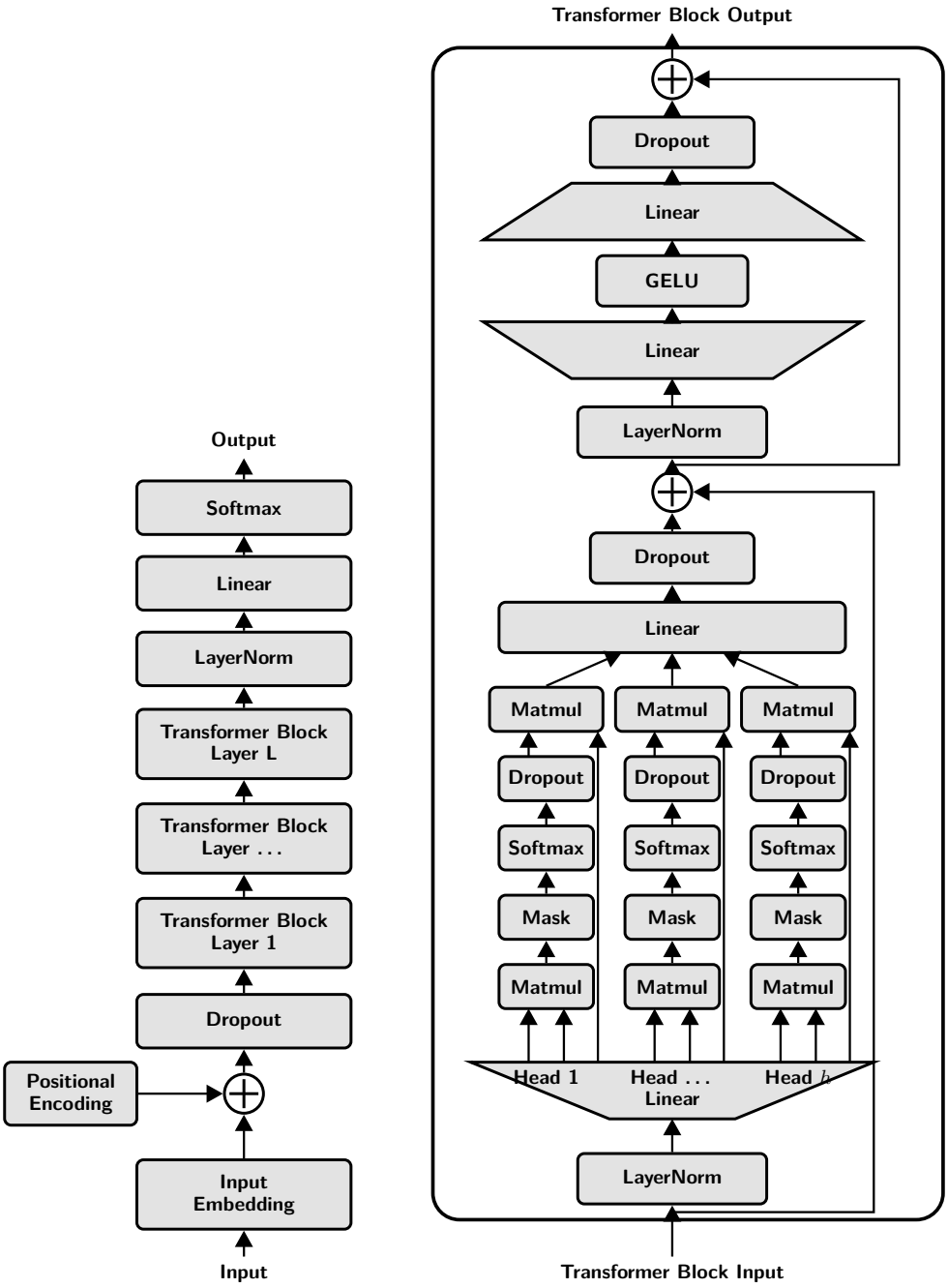


Figure 35: **Left:** Block-level view of a GPT that is a Transformer Mathematical Model with L layers from input to output. Dataflow: input tokens are embedded, combined with positional encodings, passed through a stack of L decoder Transformer Blocks, layer-normalized, and mapped by a final linear+softmax head to token probabilities.
Right: Internal structure of a single Block_ℓ with nodes drawn from the Transformer Mathematical Functions: masked multi-head self-attention f_{MHA} with h heads and a causal mask, followed by a position-wise feed-forward network f_{FFN} (GELU). Both sublayers use residual connections ($\oplus$), LayerNorm f_{LN}, and dropout (a training-time regularization procedure, part of Ψ in Definition 3.1.10, and the identity at inference).

($\mathcal{M}$, T, Ψ, F, π_{dec}) and the agent symbols of Section 3.2 (A, M, $\mathcal{T}$, $\mathcal{S}$, Π, $\mathcal{R}$, $\mathcal{N}$, $\mathcal{O}$, $\mathcal{L}$) are disjoint, and the Markov-decision-process symbols of Definition 3.1.13 are local to it. Subscripted and lower-case letters are disambiguated by their index or by the place they are introduced: h_ℓ (hidden state) vs. h (number of

attention heads), σ_ℓ (activation) vs. σ in the variance of layer normalization, x (generic input, token sequence, or row of an activation matrix), s (seed in Definition 3.1.1 vs. current memory state $s \in \mathcal{S}$ in Definition 3.2.1), X (input matrix of the Transformer functions vs. the union of the tool input sets in Definition 3.2.1), V and E in $G = (V, E)$ (node and edge sets) vs. the value matrices and the embedding matrix W^E, and the roman S and R of $C_T = (H, S, R)$ (software stack and resource constraints) vs. the calligraphic $\mathcal{S}$ and $\mathcal{R}$. Later chapters introduce chapter-local notation for their own artefacts, and where a letter recurs the local definition applies: Chapter 6 writes Π for the CCL participant protocol (Definition 6.3.10), $\mathcal{S}$ for the CCL process state, $G = (V, E, \tau)$ for the CCL process graph (not a *Function Graph*), $\lambda \in \Lambda$ for its edge labels (not hyperparameters), A for its allowed-connections relation, $\mathcal{T}_{\text{rel}}$ for the release tags of the CCL repository, and T for the compiled NordDRG decision tree (Section 6.3.1.4). Definition 3.1.17 keeps the letter θ of the Notation paragraph on purpose: its bold $\boldsymbol{\theta} \in \Theta$ is the model of record, the parameters of the decision-making model $\hat{f}_{\boldsymbol{\theta}}$, and δ is a change to it, typed as an atomic diff in Section 6.3.1.3. The ledger $\mathcal{L}$ of Definition 3.2.5 and of CCL is the same object; the loss $\mathcal{L}$ of Section 3.1.1 is not used outside that section.

Phase I — protocol executions and platform detail

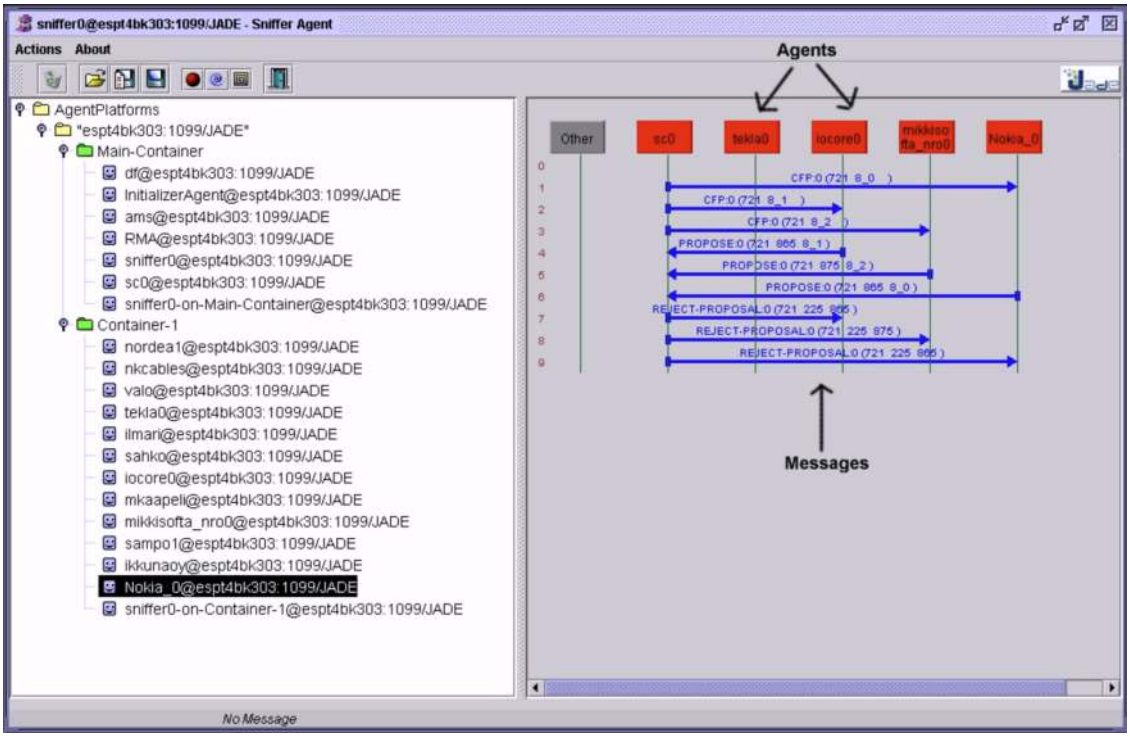


Figure 36: Contract Net request for quotes in the B2B negotiation prototype.

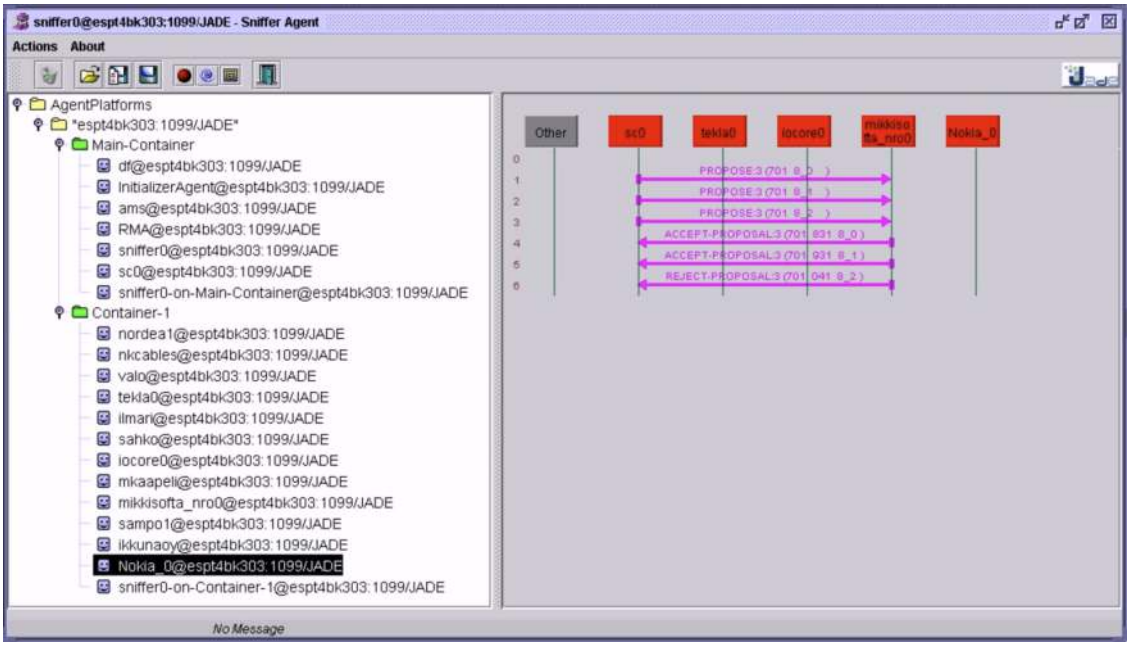


Figure 37: Proposing a lower price.

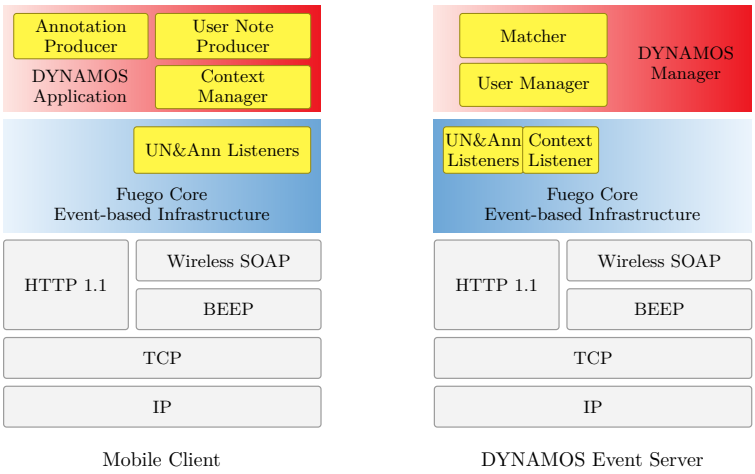


Figure 38: DYNAMOS components on the Fuego Core event-based infrastructure: mobile client (left) and DYNAMOS Event Server (right).

LLM-agent reference diagrams (Section 2.5)

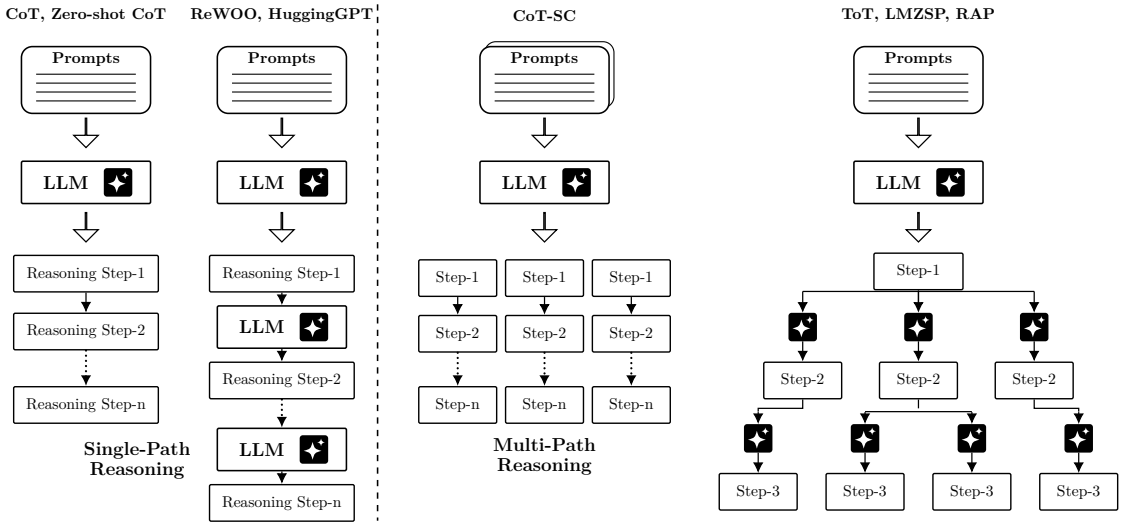


Figure 39: Comparison of single-path vs. multi-path reasoning (redrawn from [43]).

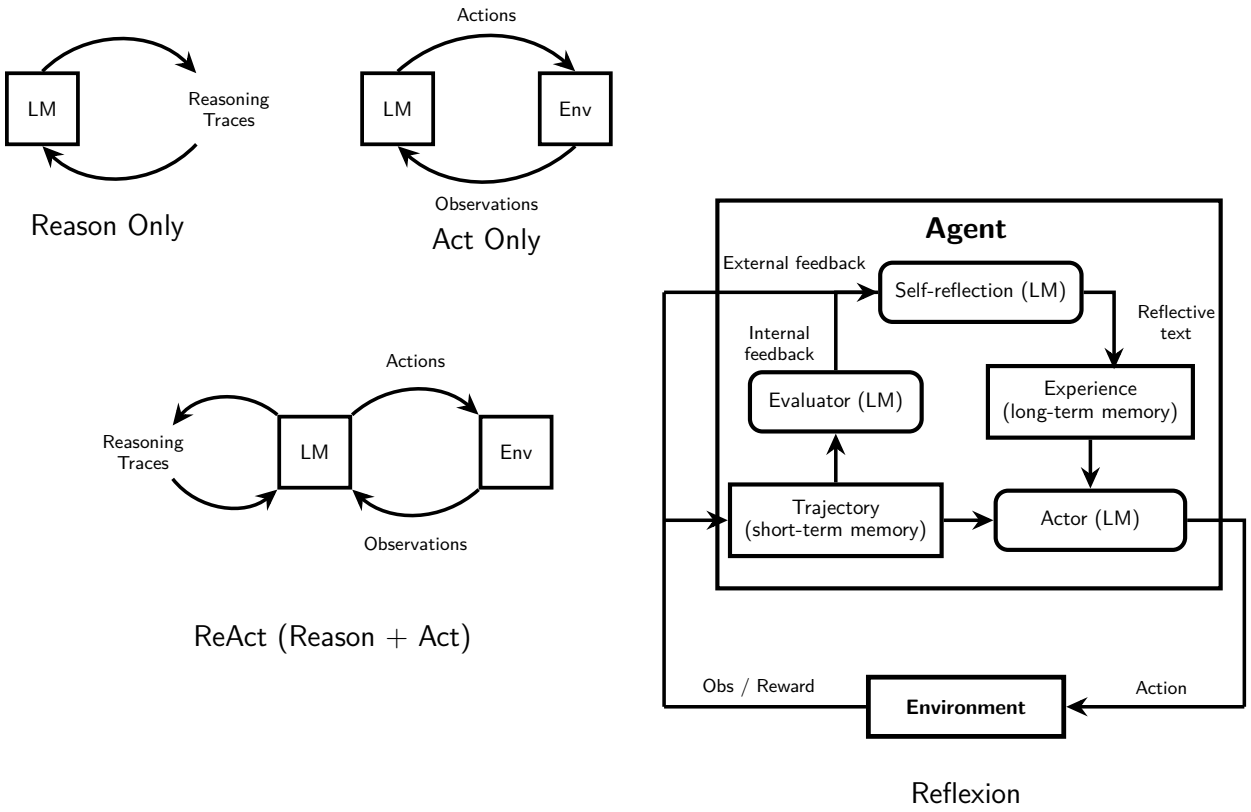


Figure 40: Comparison between Reason Only, Act Only, Reason and Act, and Reflexion frameworks (redrawn from [44, 45]).

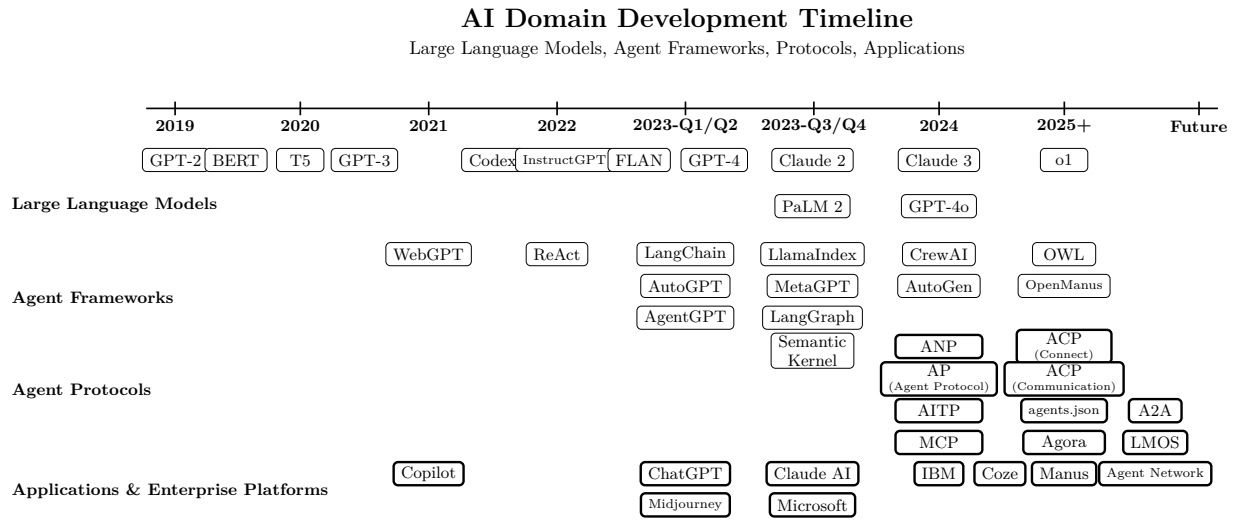


Figure 41: Development timeline of large language models, agent frameworks, agent protocols, and applications (redrawn from [46]; item placement largely follows the source and is approximate).

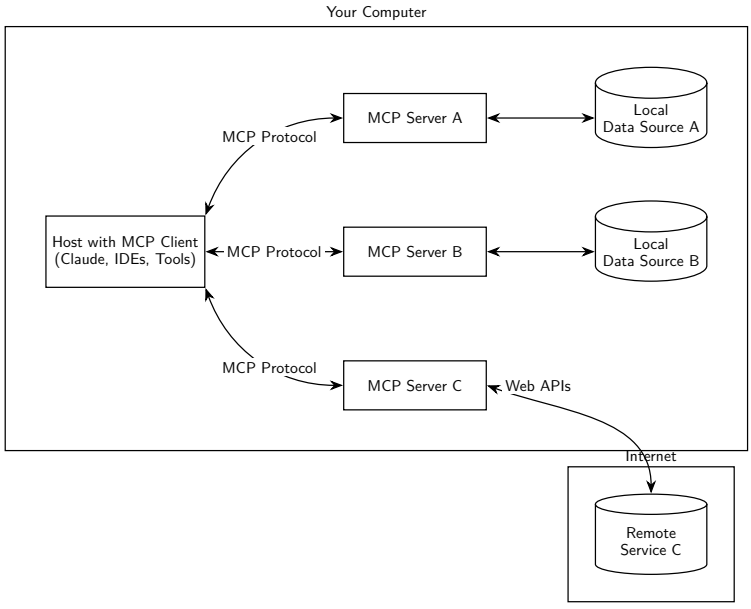


Figure 42: MCP general architecture [47].

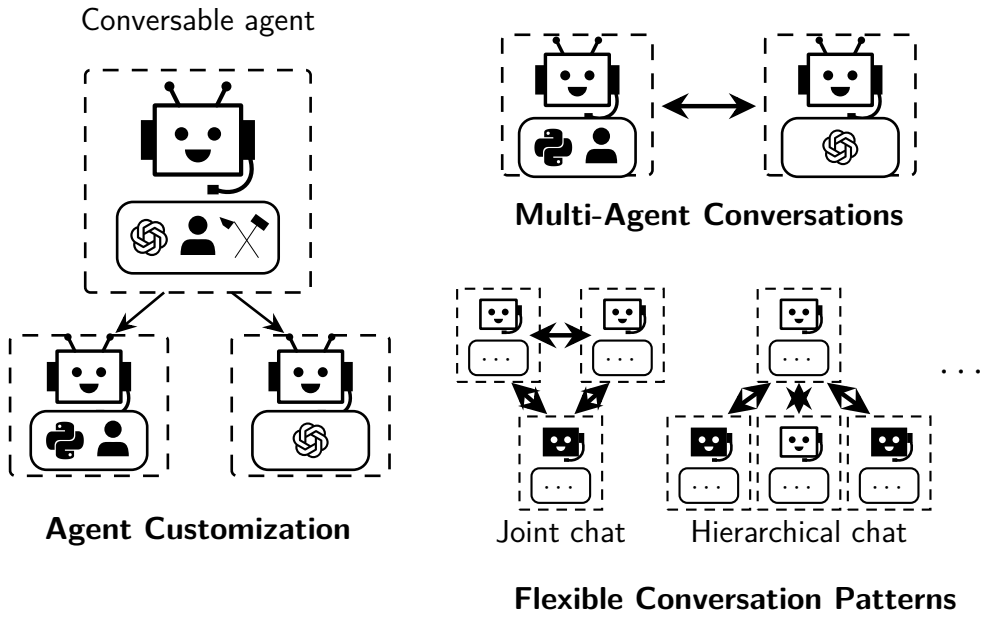


Figure 43: In the example library AutoGen, agents are conversable, customizable, and can be based on LLMs, tools, humans, or even a combination of them (redrawn from [48]).

Phase I — interaction-protocol RDF serialization excerpts (Section 4.1.1.3)

Representative excerpts of the RDF serialization of FIPA Query against the shared interaction-protocol ontology are shown below: the interaction-protocol (*IP*) class, the message *Content* with its embedded RDQL query, and a delivery report. Further excerpts appear in the Licentiate thesis [2].

```
<rdf:Description rdf:about="&this;queryDelivId">
  <rdf:type rdf:resource="&ip;IP" />
  <ip:hasInitiator rdf:resource="&this;contractor" />
  <ip:hasParticipant rdf:resource="&this;supplier" />
  <ip:hasProgress rdf:resource="&this;queryProgress" />
</rdf:Description>
```

```
<rdf:Description rdf:about="&this;queryContent">
  <rdf:type rdf:resource="&fipa;Content" />
  <fipa:expression>
    String ORDER_ID = "http://www.sscompany.com/"
                    +"orderReports#steelOrd123";

    String query = "SELECT ?x"
                  +" WHERE (?y, <rdf:type>,"
                  +" <report:orderReport>),"
                  +" (?y, <report:deliveryReport>, ?x)"
                  +" AND ( ?y eq <" +ORDER_ID +">)"
                  +" USING " +nameSpaces;
  </fipa:expression>
</rdf:Description>
```

```
<rdf:Description
  rdf:about="http://www.crcompany.com/deliveryreports#delivery789">
  <rdf:type rdf:resource="http://www.supplyreports.net/report#deliveryReport" />
  <report:customer rdf:resource="http://www.crcompany.com/deliveryreports#sscompany" />
  <report:provider rdf:resource="http://www.crcompany.com/deliveryreports#crcompany" />
  <report:schedPickUp>20030716</report:schedPickUp>
  <report:schedAtDestination>20030726</report:schedAtDestination>
  <report:steelAmount>3500</report:steelAmount>
</rdf:Description>
```

Phase I — agent-platform and demonstration screenshots (Sections 4.1, 4.2)

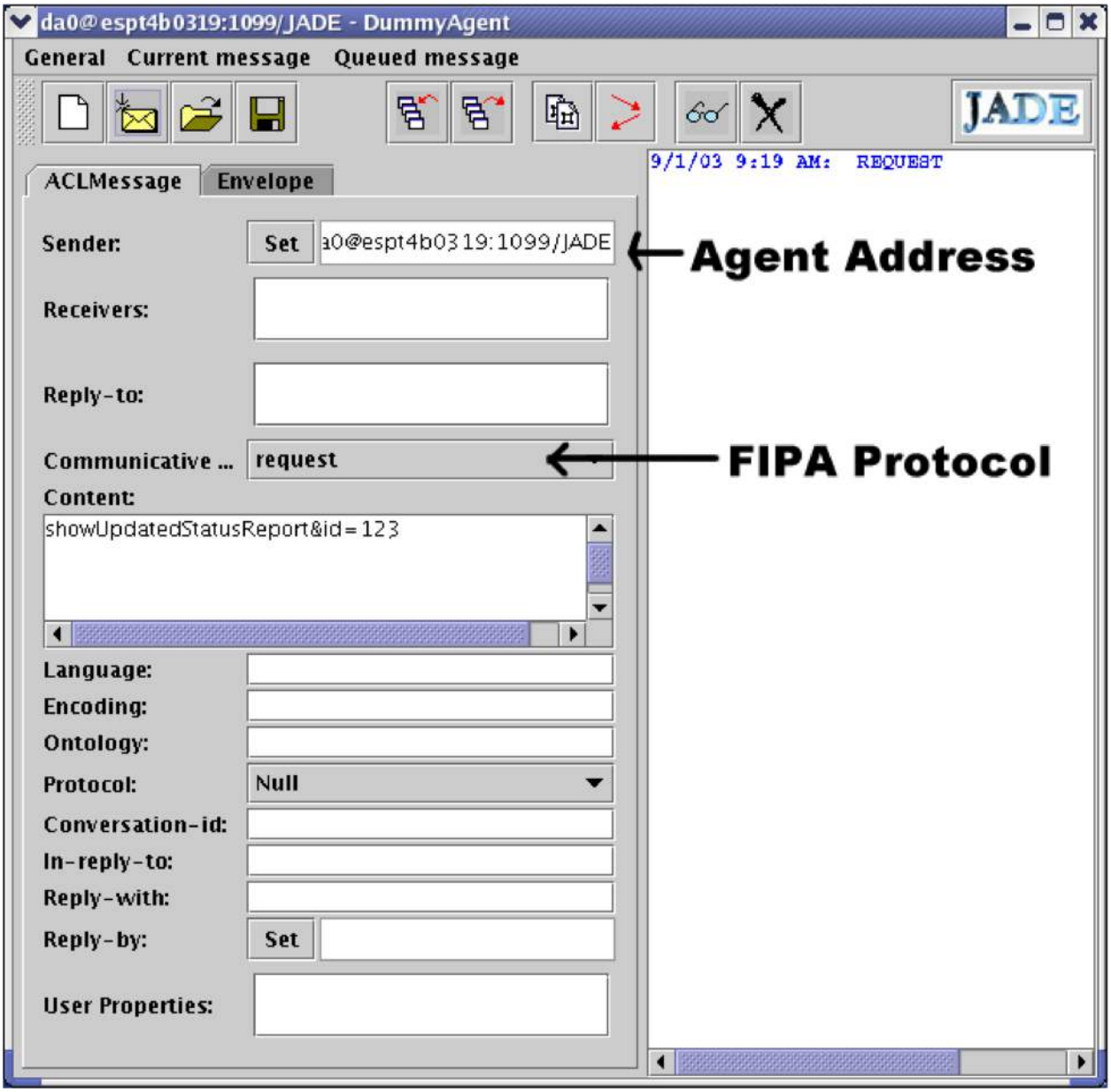


Figure 44: A tool for communicating with the software agents.

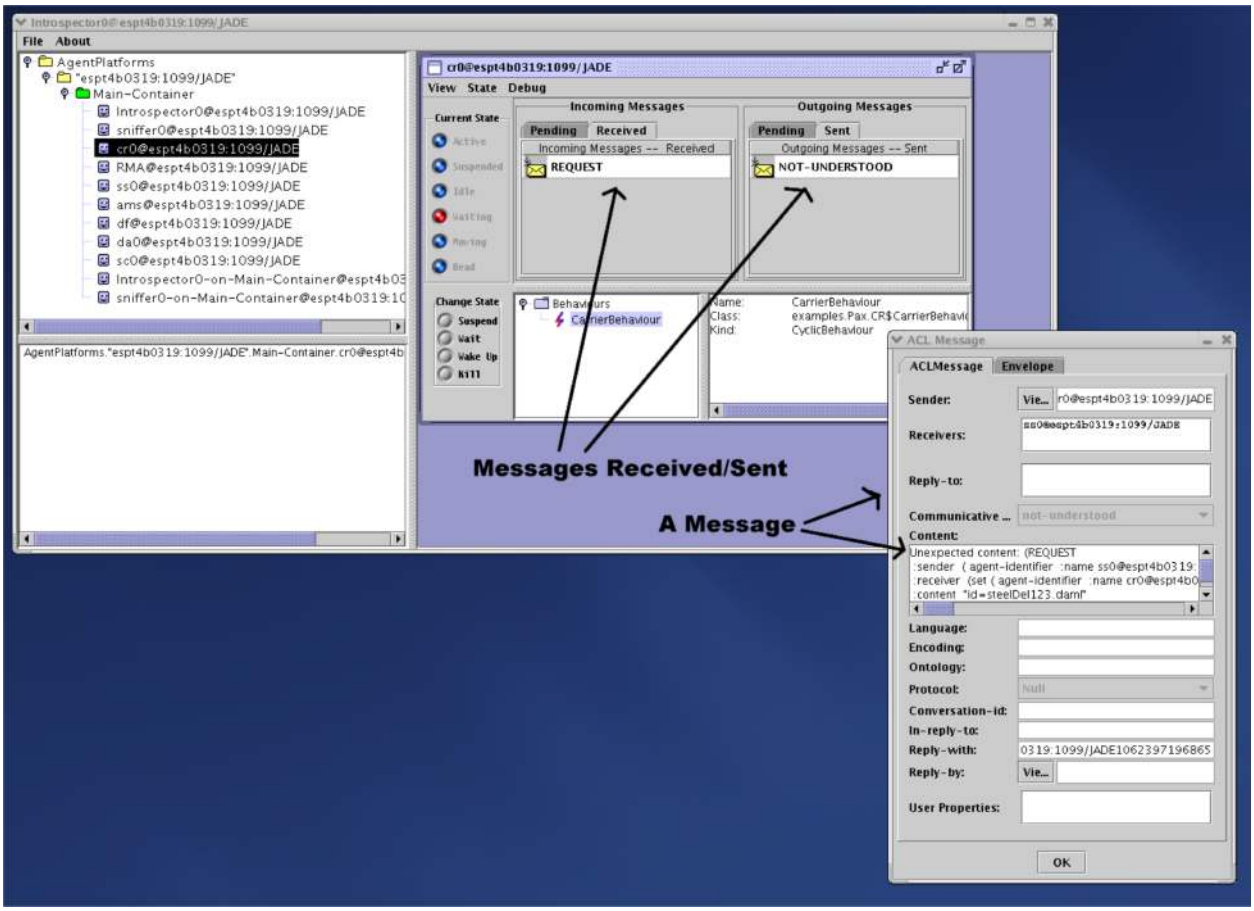


Figure 45: A tool for monitoring the state and information the agent receives and sends.

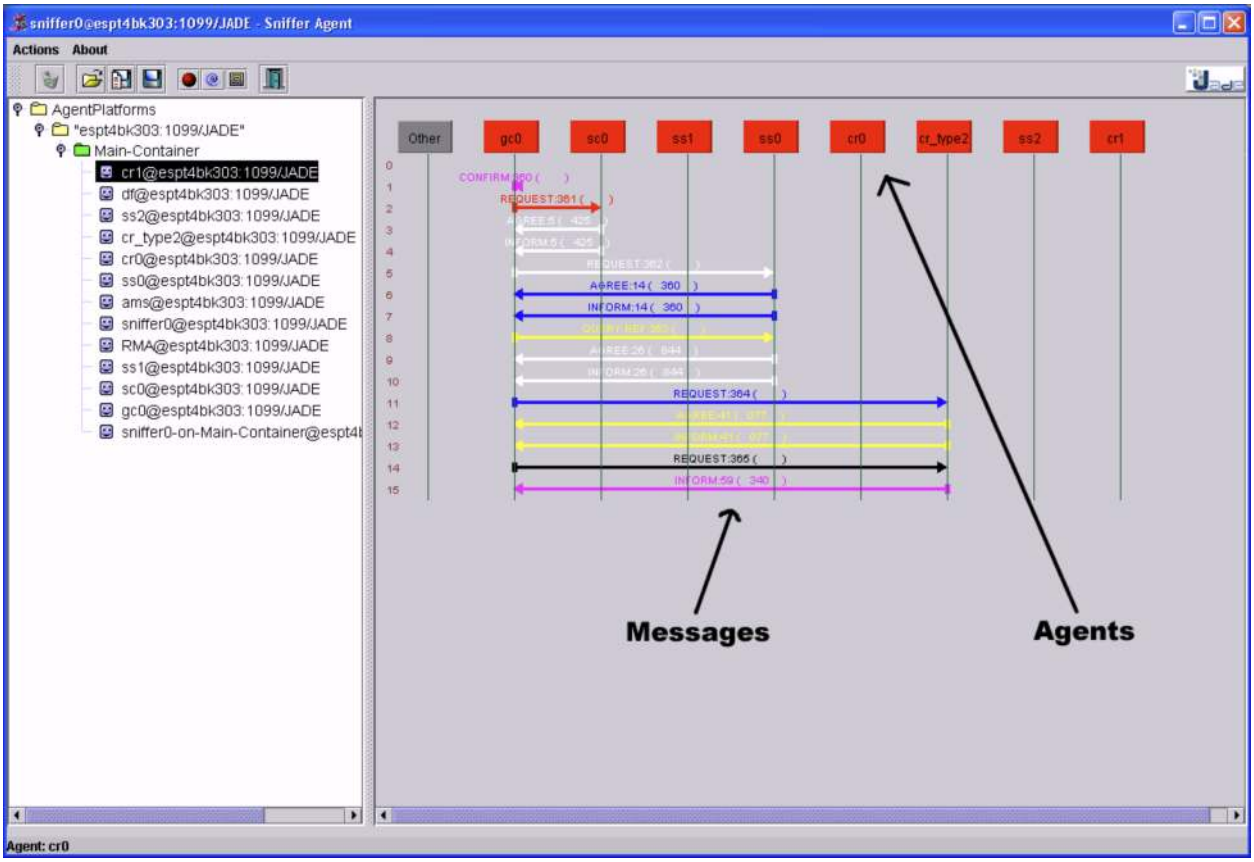


Figure 46: Agents and messages.

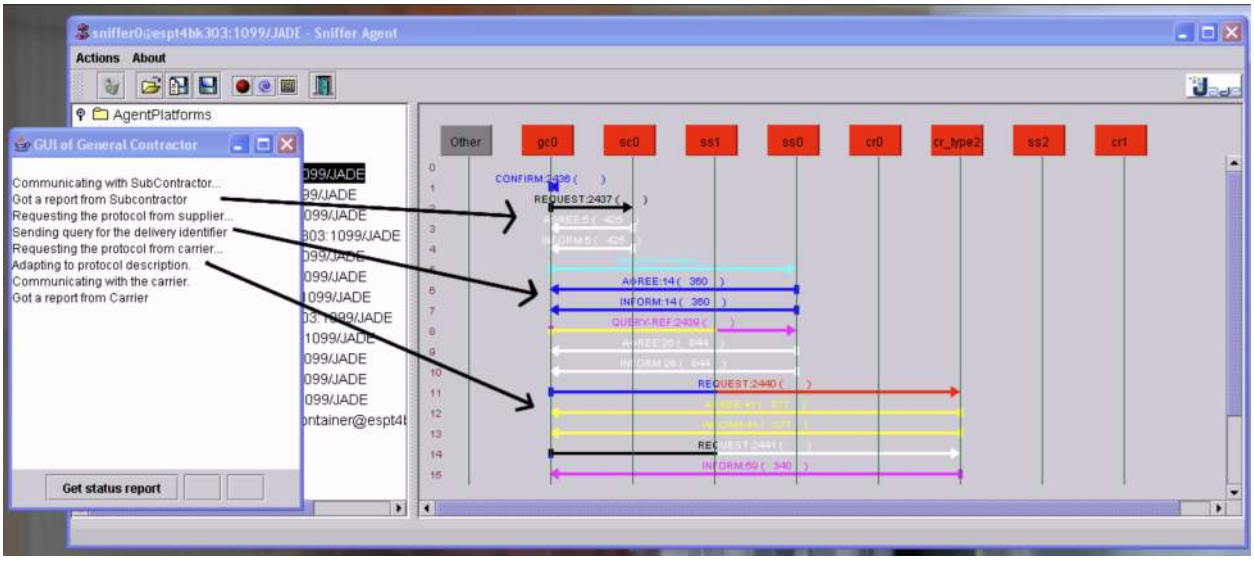


Figure 47: Process flow of the protocol-adaptation demonstrator.

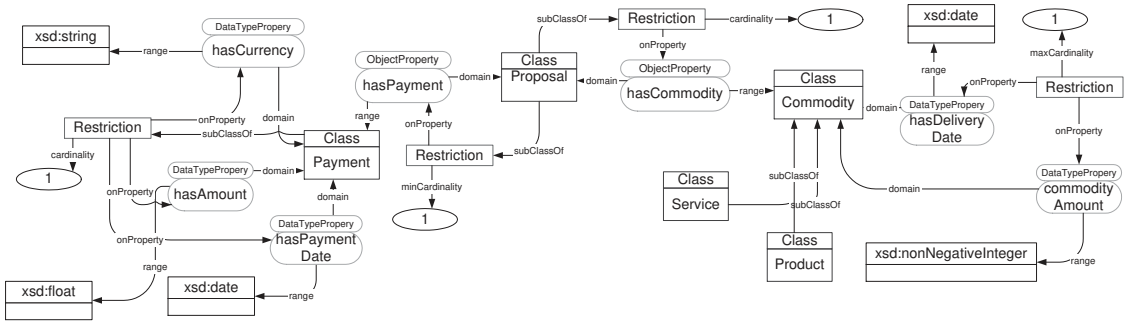


Figure 48: Proposal ontology.

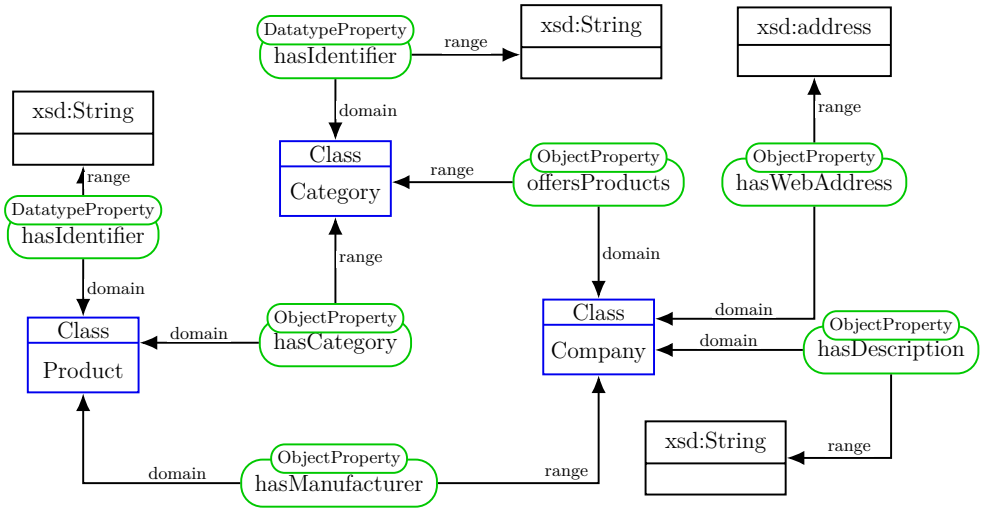


Figure 49: Product ontology.

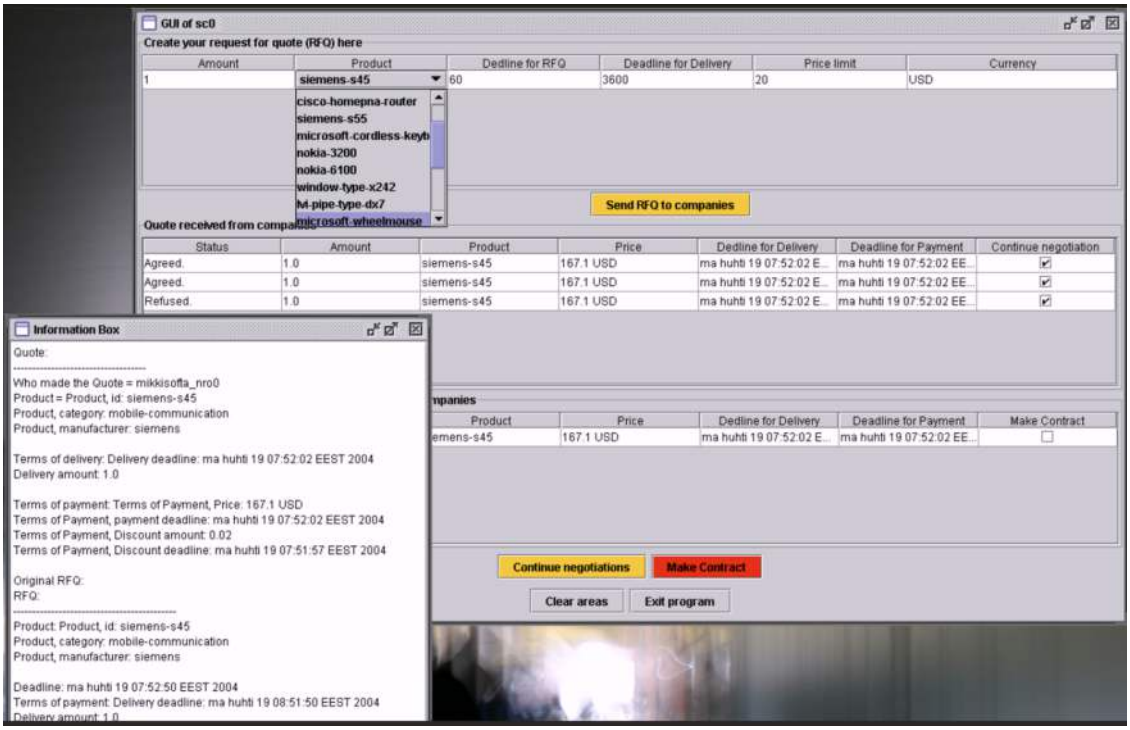


Figure 50: Phase I B2B negotiation prototype GUI: product catalog tables.

Phase I — DYNAMOS profile/context and client screenshots (Section 4.3)

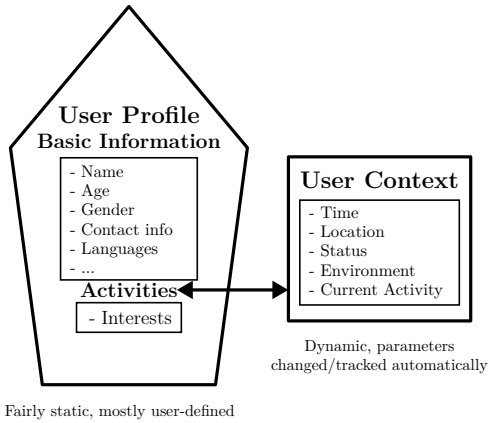


Figure 51: User Profile and User Context.

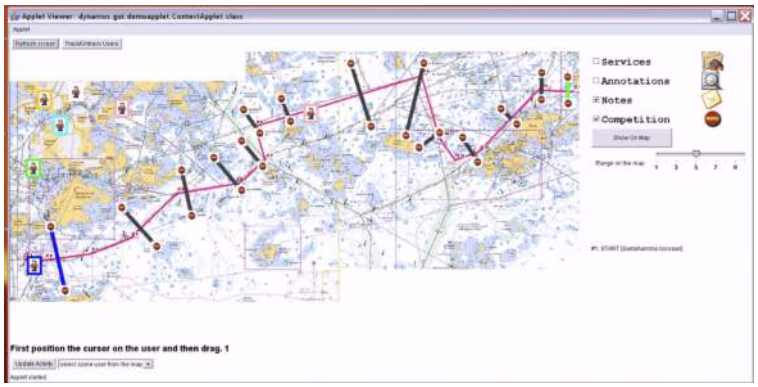


Figure 52: DYNAMOS System Overview.



Figure 53: DYNAMOS client screenshots from the August 2005 user trial (a trial participant's phone number, e-mail address, and face are masked).

Phase II — domain and data-model setup

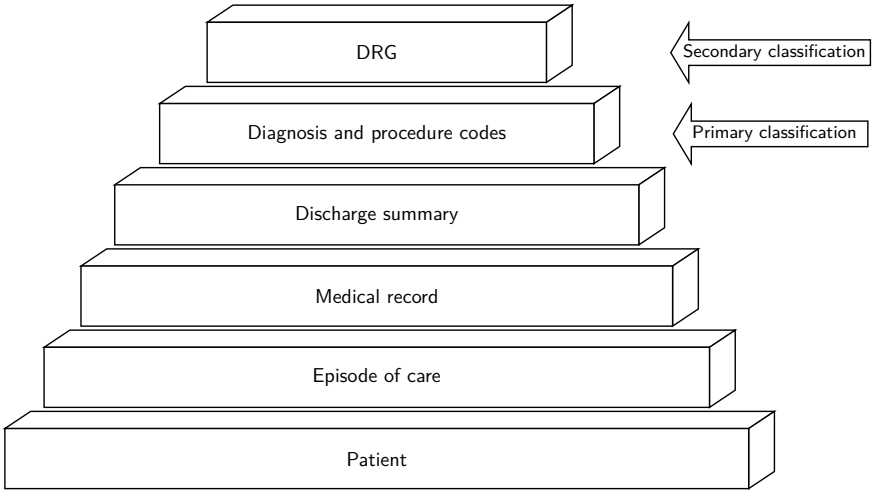


Figure 54: The hierarchy of classification.

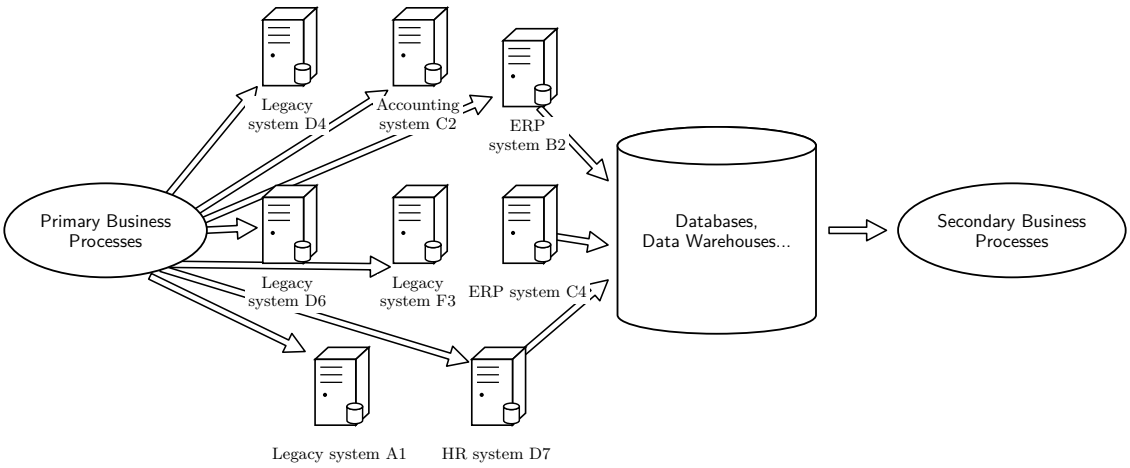


Figure 55: Primary and secondary business processes and data warehousing.

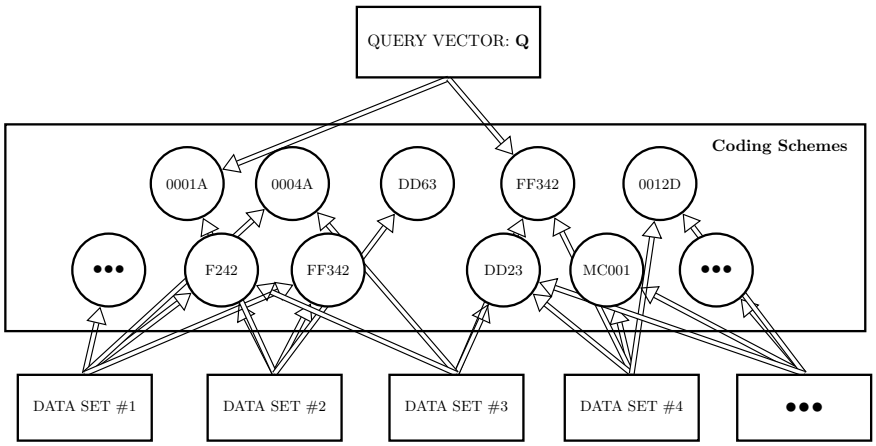


Figure 56: A sample coded dataset network interlinked by a coding scheme. Redrawn from [12, 13].

Phase III — HADA stakeholder dialogues

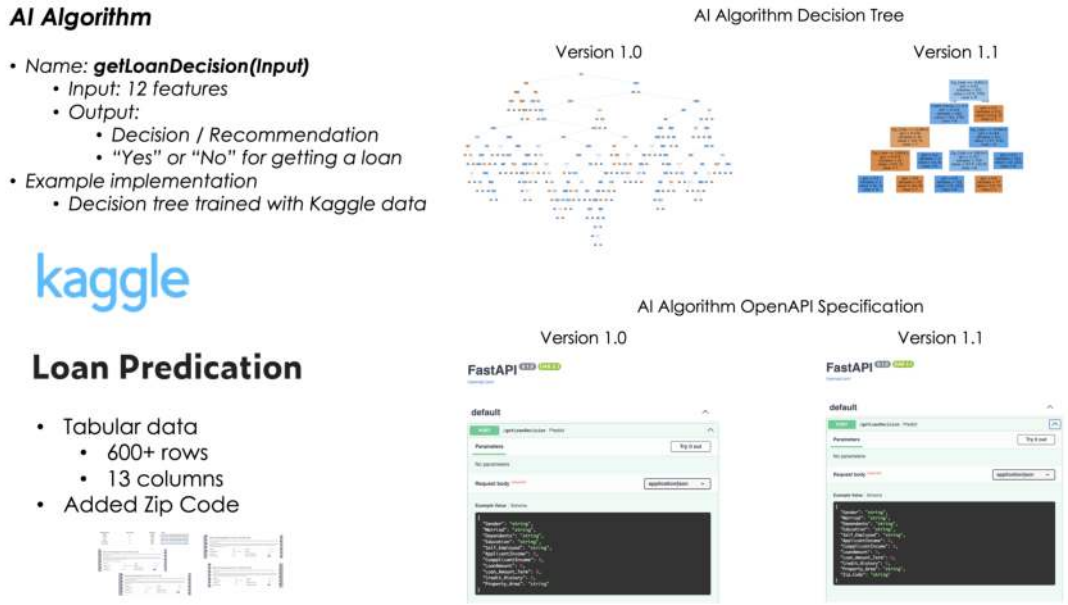


Figure 57: AI tool `getLoanDecision`, built on a Kaggle dataset: its decision trees and OpenAPI specifications (Versions 1.0 and 1.1).

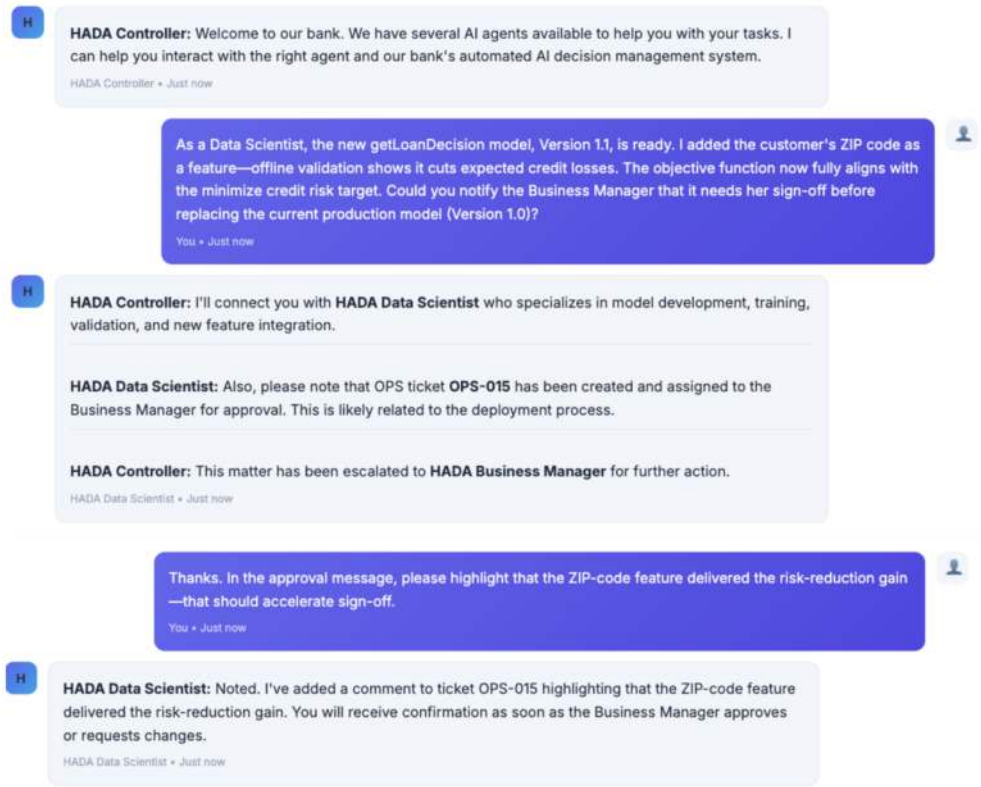


Figure 58: Dialogue: Data Scientist delivering a new AI tool version for business approval.

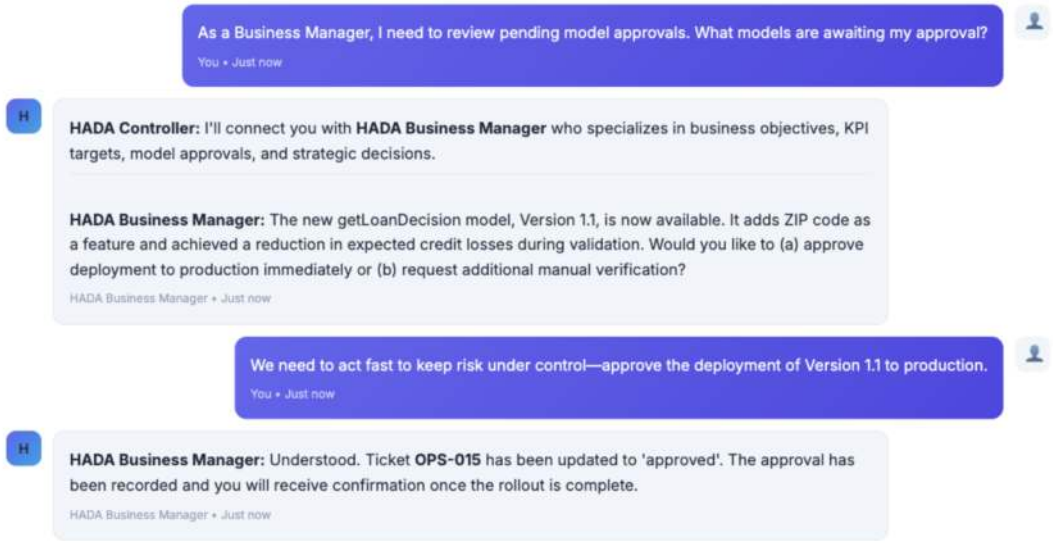


Figure 59: Dialogue: Business Manager approves version 1.1 for production.

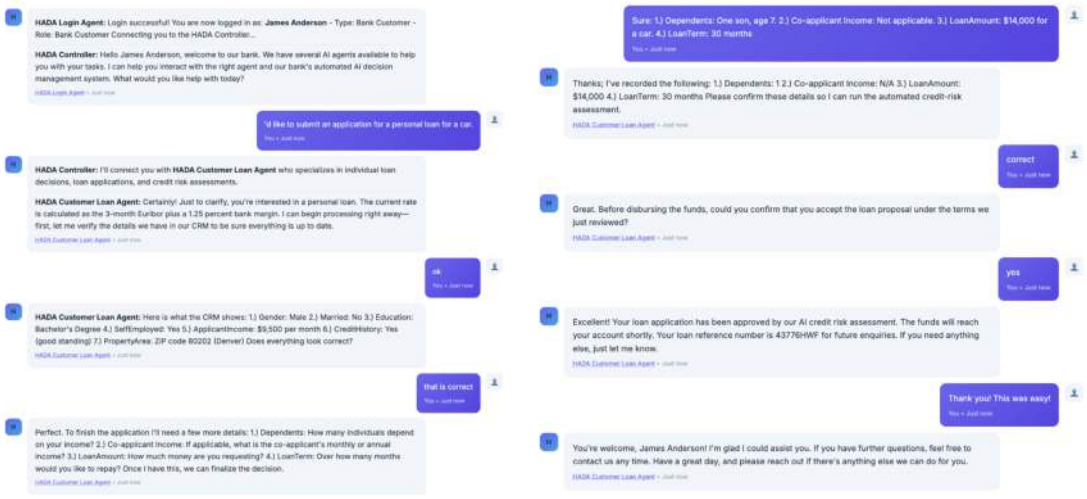


Figure 60: Dialogue: Customer applying for a personal loan.

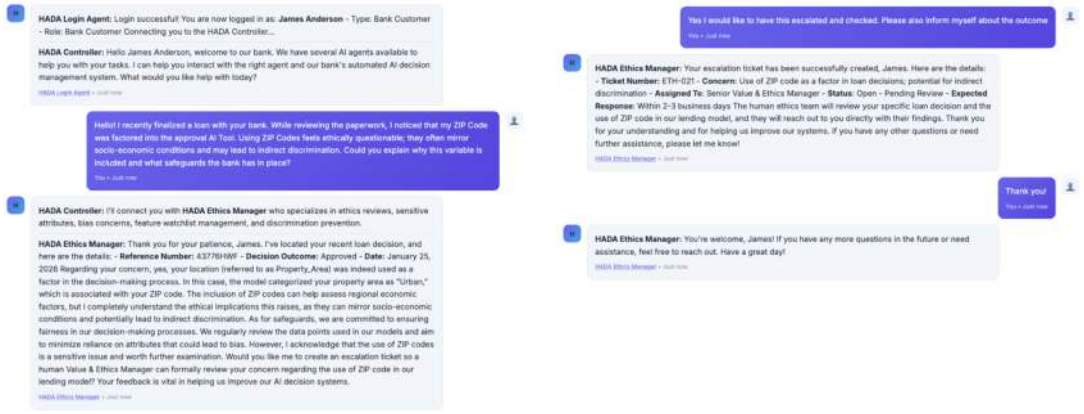


Figure 61: Dialogue: Customer questioning the use of ZIP codes in lending.

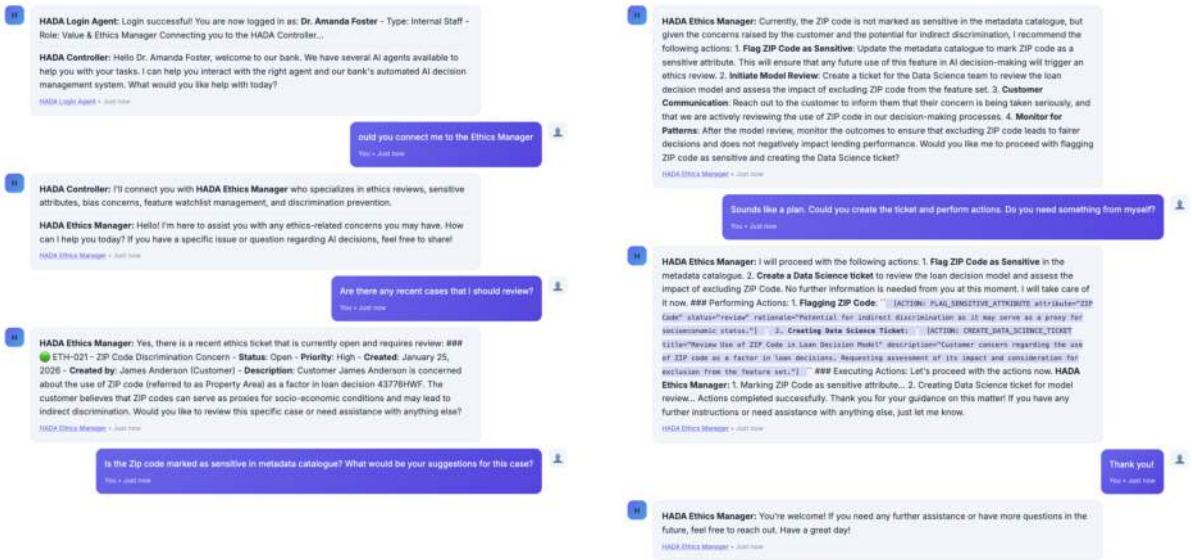


Figure 62: Dialogue: Value and Ethics Manager (DVEM) evaluating ZIP codes.

Phase III — NordDRG AI Benchmark materials and the CCL learning-paradigm figure (Sections 6.2, 6.3)

NordDRG definition tables — one workbook, 20 tables (Excel for reading; CSV and JSON for groupers):

`readme` · `drg_logic` · `dg` · `dg_feat` · `dg_plus` · `dgcat_name` · `dgprop_name` · `pdgprop_name` · `proc` · `proc_feat` · `proc_plus` · `procprop_name` · `compl_excl` · `compl_name` · `drg_name` · `drg_name_comb` · `mdc_name` · `mdc_name_comb` · `rtc` · `testcase`

Excerpt of `drg_logic` (first rows in `ord` order; one grouping rule per row, evaluated top-down):

ord	id	drg_comb	rtc	icd	mdc	dgcat	pdgprop	or_proc	procpro	dgprop1	dgprop2	agelim	dur	disch	nat_ver
25	#00D000002	X09R	0	+					40V62	99X50	40X00		<1		swe
37	#00D000003	X09R	0	+						99X50	40X00		<1		swe
43	#00D000004	X09R	0					N		99X50			<1	E	swe
50	#00D000005	Z82R	0							99X50	00X90		<1	N	swe
100	#00D000010	Z70R	1	-					99V35	99X50			<1	N	swe
170	#00D000017	Z90R	9	+	15					99X50		<0			swe
250	#00D000025	Z76R	5	+			00P02			99X50		>6574	<1		swe
375	#00D000038	Z71R	8	+		99M02				99X50	99X35		<1	N	swe
700	#00D000070	Z75R	4	+				S	14S01	99X50		<4018	<1		swe
1650	#00D000165	C70R	0	+	3			P	03E01	99X50			<1	N	swe

Columns that are empty in every row shown (`compl`, `secpro`, `dgprop3`, `dgprop4`, `sex`) are omitted. Values from the Combined 2024PL CSV release.

Figure 63: Materials used: the NordDRG definition tables [49], shown as the list of the 20 tables and an excerpt of `drg_logic` (own typesetting from the published Combined 2024PL CSV release).

NordDRG documentation

- Definition tables: a database of about 20 tables, distributed as one Excel workbook and as CSV and JSON files
- Contents: DRG groups, ICD diagnosis codes, procedure codes, diagnosis and procedure categories and properties, complication rules, return codes
- Expert guides (PDF) on the content and interpretation of the definition tables and on documenting technical changes

Sources

- nordcase.org
- nordcaseforum.easyredmine.com (NordDRG Forum)

PDF guide

How to read the NordDRG definition tables

- Purpose and formats of the definition tables (Excel, CSV, JSON)
- Table `drg_logic`: one grouping rule per row, read top-down in `ord` order until a row matches
- Meaning of each column: rule id, resulting DRG (national and combined), return code, and the grouping properties drawn from `dg_feat` and `proc_feat`
- How to find which diagnoses or procedures lead to a given DRG

Nordic Casemix Centre (NordCase)

PDF guide

How to write technical changes for NordDRG

- Technical changes (TC) are documented in an Excel template provided by NordCase, named with the national and NCC case ids
- Extra template columns: version to change, IN/OUT, NCC id, national id, comments
- Changing a row means a pair of rows: the existing row marked OUT and the modified row marked IN
- Table-specific instructions and general DRG development guidelines

Nordic Casemix Centre (NordCase)

Figure 64: Materials used: the NordDRG definition-table expert documentation [49], summarized in the author's words (own drawing; no pages of the guides are reproduced).

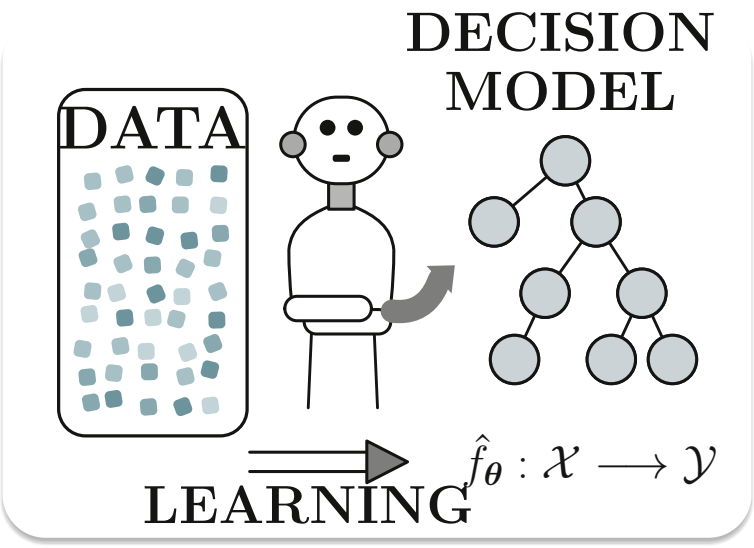


Figure 65: Learning paradigms create a decision-making model $\hat{f}_{\boldsymbol{\theta}}$ from data. Supervised, unsupervised, reinforcement, and transfer learning differ chiefly in the form and timing of the signals they exploit.

Phase III — CCL Platform and agent screenshots

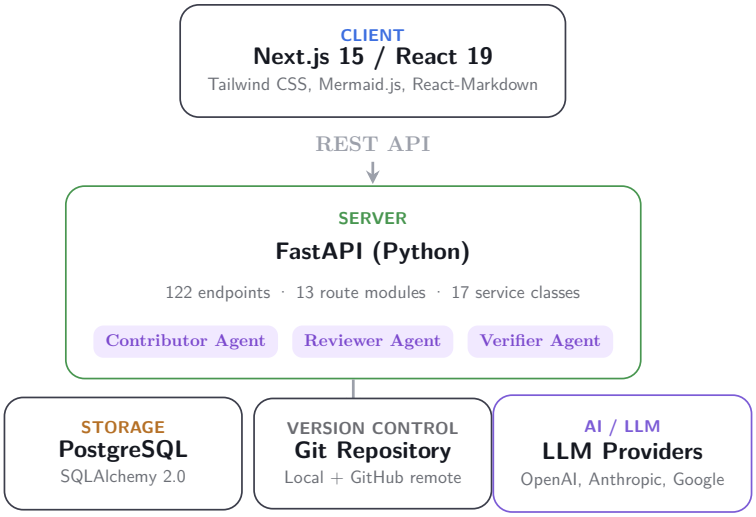
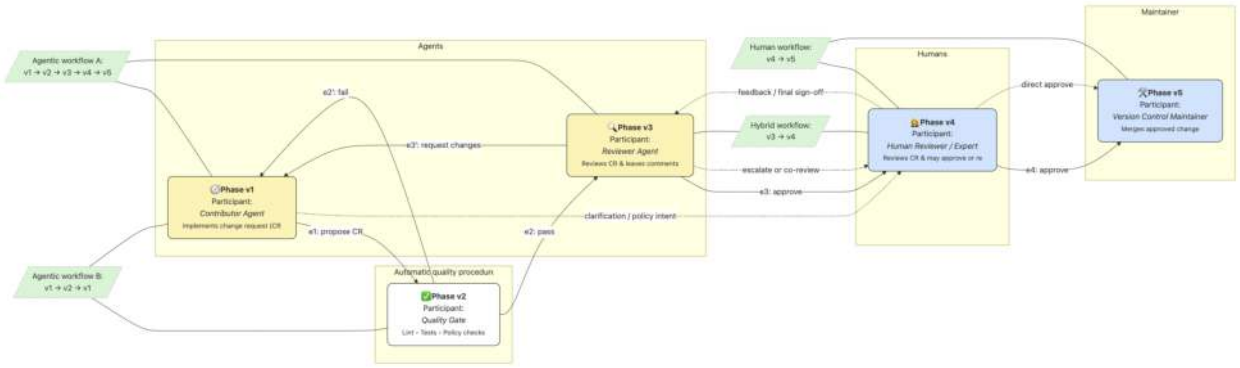
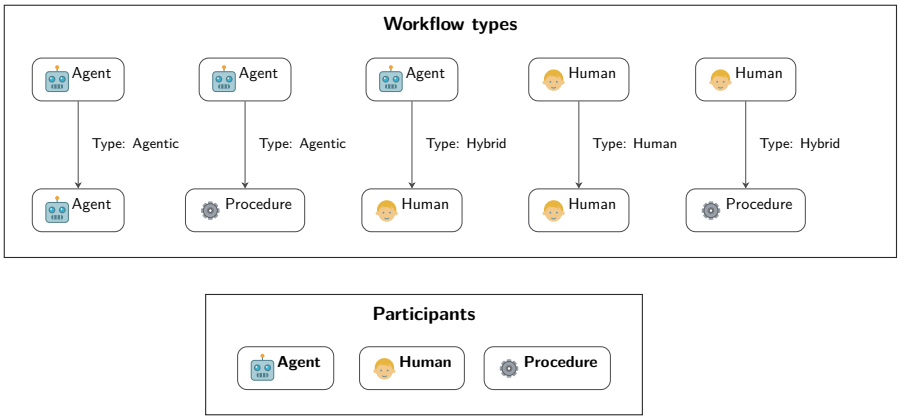


Figure 66: High-level architecture of the CCL Platform. The browser-based front end communicates with the FastAPI back end over REST. The back end integrates with PostgreSQL for persistence, a git repository (local, with a GitHub remote) for version control, and multiple LLM providers for agent capabilities.



(a) Example CCL process slice: a directed, possibly cyclic AOV graph where activities (vertices) are typed by participants $\tau(v) \in \{\mathsf{Agent}, \mathsf{Human}, \mathsf{Procedure}\}$. It illustrates agentic ($\mathsf{Agent} \to \mathsf{Agent}$, $\mathsf{Agent} \to \mathsf{Procedure}$), hybrid ($\mathsf{Agent} \to \mathsf{Human}$, $\mathsf{Human} \to \mathsf{Procedure}$), and human ($\mathsf{Human} \to \mathsf{Human}$) workflows.



(b) Participants and workflow types. Top: representative AOV snippets for agentic, hybrid, and human flows (Definitions 6.3.5–6.3.7; a Procedure-only flow counts as agentic). Bottom: the participant set $\mathcal{P} = \{\mathsf{Agent}, \mathsf{Human}, \mathsf{Procedure}\}$.

Figure 67: **CCL workflows.** (a) an example process slice typed by participants, and (b) the participant set and the agentic, hybrid, and human workflow types.

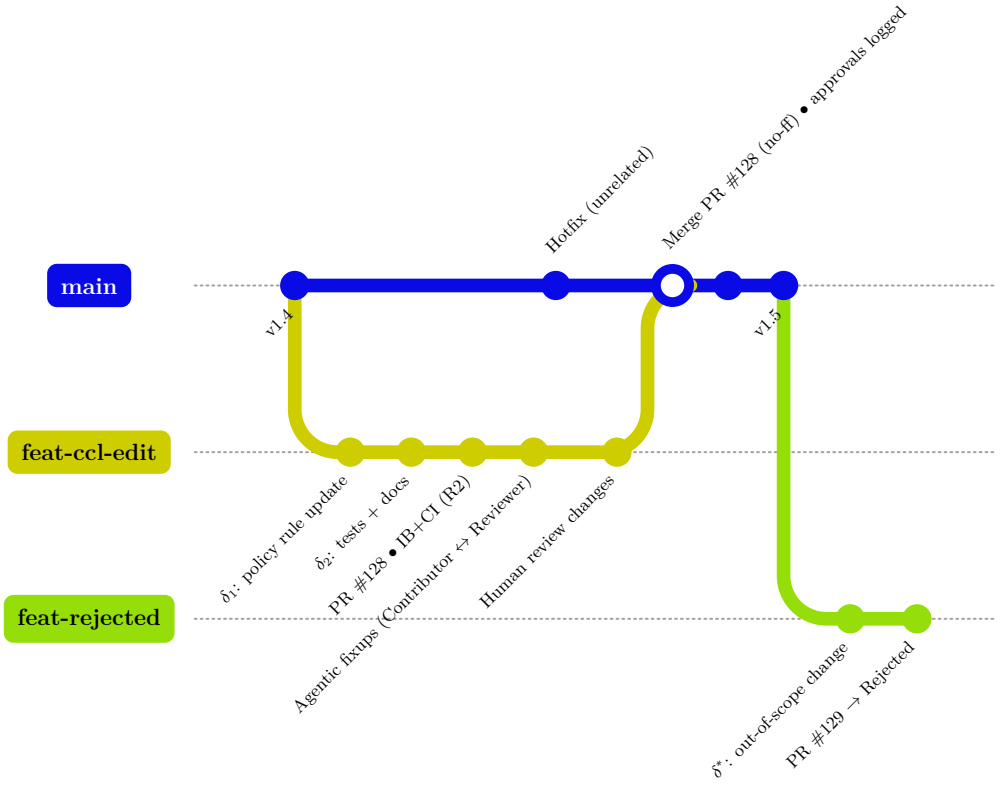


Figure 68: **Governed Version Control history for CCL.** A feature branch (`feat-ccl-edit`) opens a PR with an Impact Brief and CI (tier R2), iterates agentically, then merges with `-no-ff` into *main* after required human approvals; an unrelated hotfix and tagged releases (`v1.4`, `v1.5`) are shown. A second branch (`feat-rejected`) illustrates a PR that is rejected (no merge). Only humans may merge to *main*.

Table 29: Gate policy by risk tier κ for CCL pull requests. Each required gate is fail-closed (a failing gate blocks the PR); higher tiers enlarge evidence requirements and mandate additional human sign-offs, and only humans approve merges. In the terms of Definition 3.2.5, G_1 realizes the objective check ($\mathcal{O}$), G_2–G_4 the norm checks ($\mathcal{N}$), and the required human approvers the accountable-human clause of $\mathcal{R}$.

	G_1 Objective & change control ($\mathcal{O}$)	G_2 Oversight signals ($\mathcal{N}$)	G_3 Transparency & evidence ($\mathcal{N}$)	G_4 Adversarial & risk evaluation ($\mathcal{N}$)	Required human approvers
What the gate checks	Schema/typing and referential integrity; change-graph scan for overlaps and conflicts; blast-radius (locality) budget; determinism/totality	Required metadata; stratified samples for human spot-checks; proposer–Reviewer Agent–human agreement; leakage/ anomaly detection; declared monitoring hooks	Complete Impact Brief; provenance and licensing; reproducibility bundle; documentation updated in lockstep	Red-team probes for specification gaming; distribution-shift probes; fuzzed near-misses; agent-safety checks	—
R1 (low)	required	required	required	not required	1: a single human reviewer
R2 (medium)	required	required	required	required	2: domain and safety reviewers
R3 (high)	required	required	required	required	3: domain and safety reviewers, and a third approver

All root → leaf paths to DRG: 373X

All the nodes in the decision tree are functions that typically compare a list of diagnosis or procedure codes against the input data. For example, 14X03 is connected to 49 diagnosis (ICD) codes in the Combined 2024PL release (Logic-8, Table 30).

drg_comb	icd	mdc	dgcat	pdgprop	compl	or_proc	procpro	secpro	dgprop1	dgprop2	dgprop3	dgprop4	agelim	sex	dur	disch
373X	+					S	14S06		14X03							
373X	+					S	14S04	-14S90	14X03							
373X	+					S	14S06		14X03							
373X	+						14S04	-14S90	14X03	00X10						
373X	+					N			14X03							

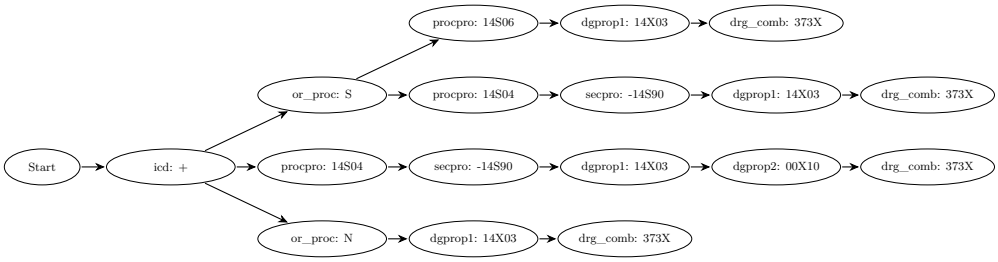


Figure 69: Illustrative fragment of the first-layer decision logic for DRG 373X. Top: excerpt of the custom NordDRG format where each row expands to a path condition (rows that are identical in the columns shown collapse to one path). Bottom: the corresponding path-wise view as a canonical decision tree compiled for analysis.

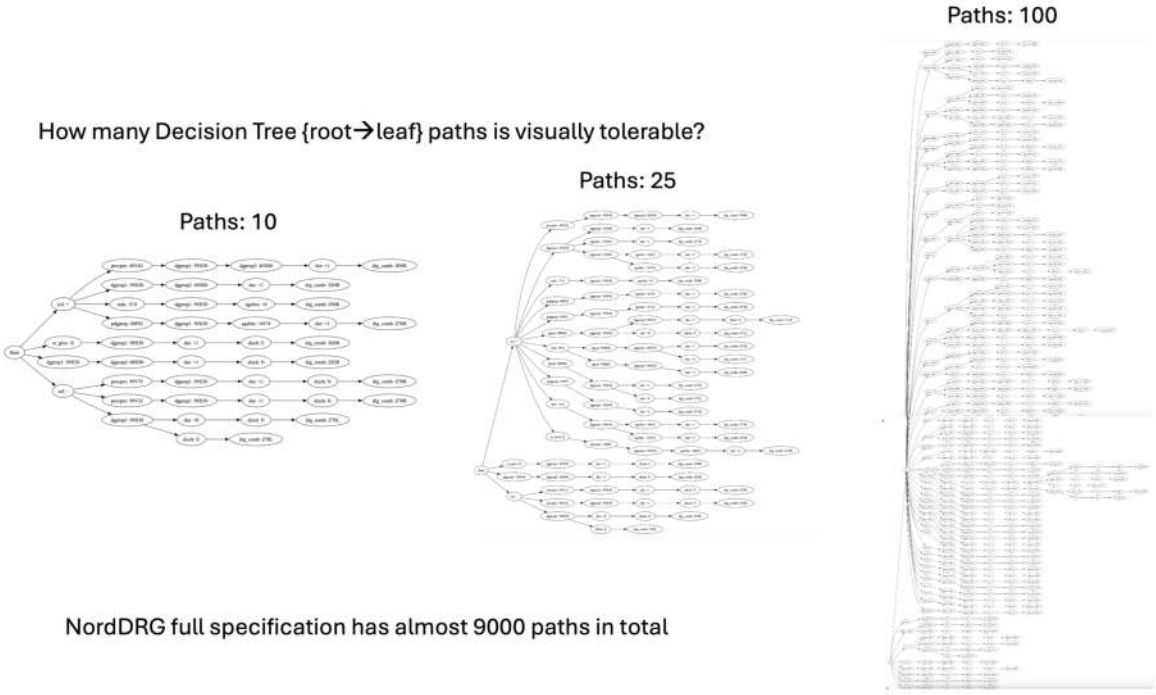


Figure 70: NordDRG as decision trees: illustrative slices with 10, 25, and 100 root→leaf paths. Only small portions of the full structure are readable as static graphics. In practice: *10 paths* are fully legible at print scale and useful for explaining predicate structure and local flow; *25 paths* remain readable on screen with moderate zoom and are borderline for print without enlarging; *100 paths* are overcrowded in static form and are only useful with interactive panning/zoom, making them unsuitable for print. The full specification contains nearly 9,000 paths; it can be rendered in the browser but is difficult to navigate.

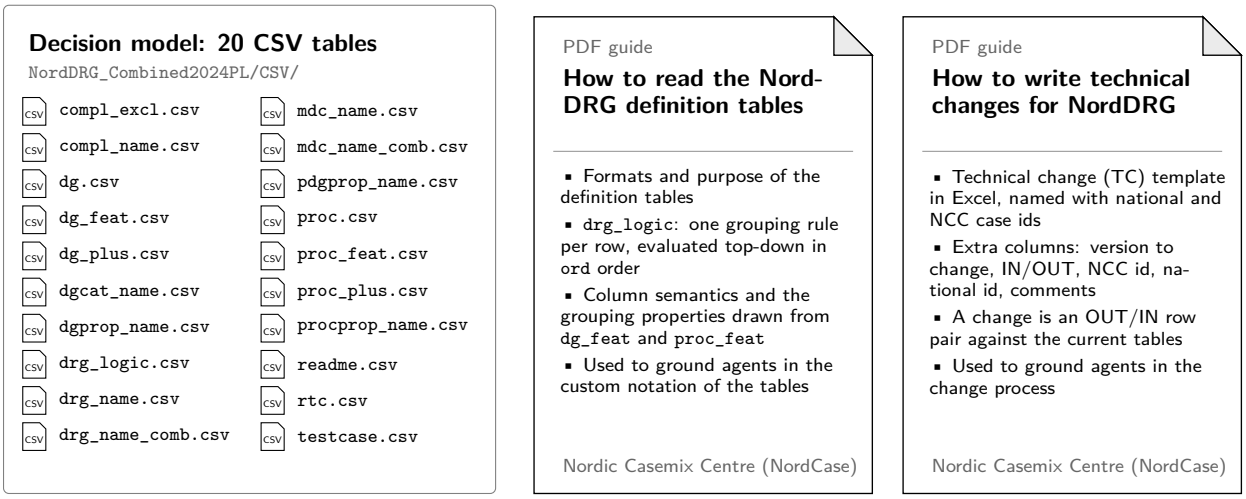


Figure 71: Preparing the NordDRG specification for CCL: (left) the CSV tables that constitute the agent-editable decision model; (middle/right) the two PDF guides from the NordDRG Forum used to ground agents in the custom notation and change process (own drawing; guide contents summarized).

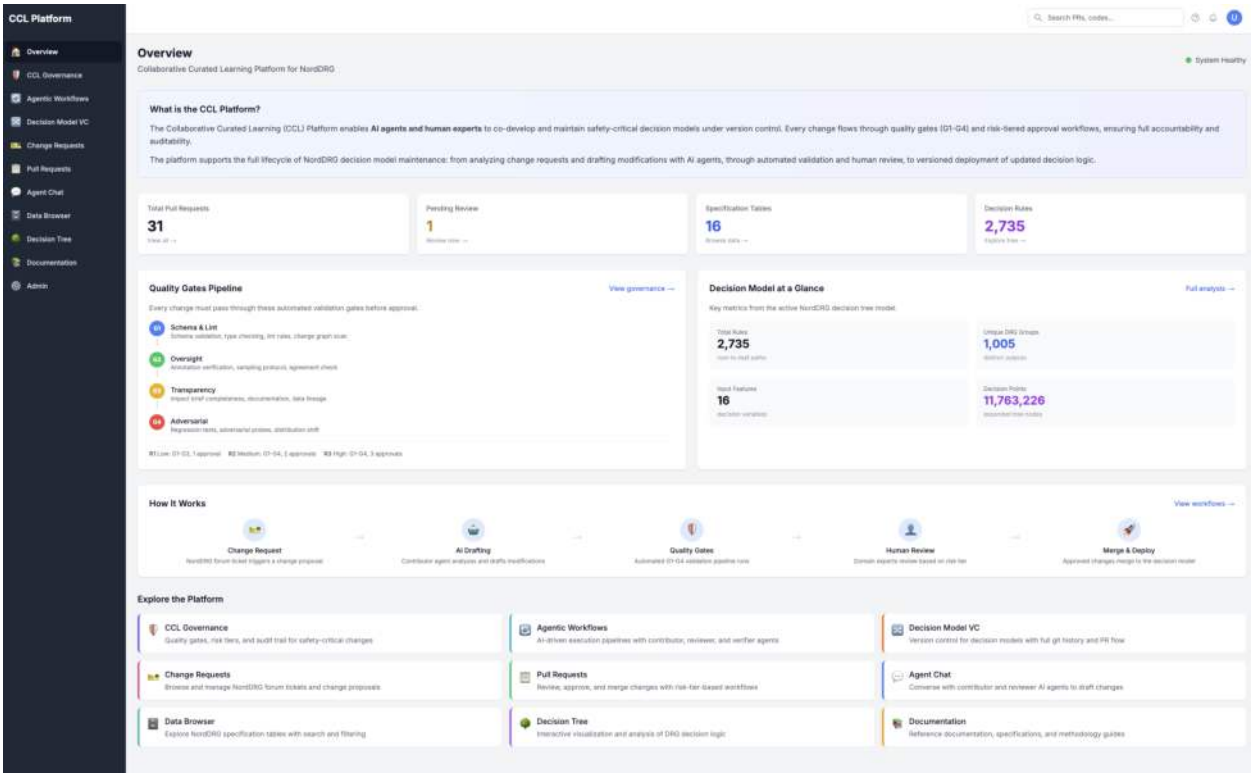


Figure 72: The Overview page of the CCL Platform. The top section displays live system metrics (pull requests, pending reviews, specification tables, decision rules). Below, the quality-gate pipeline (G1–G4) and decision-model summary provide at-a-glance situational awareness. The five-step workflow diagram illustrates the change lifecycle, and navigation cards link to all platform sections.

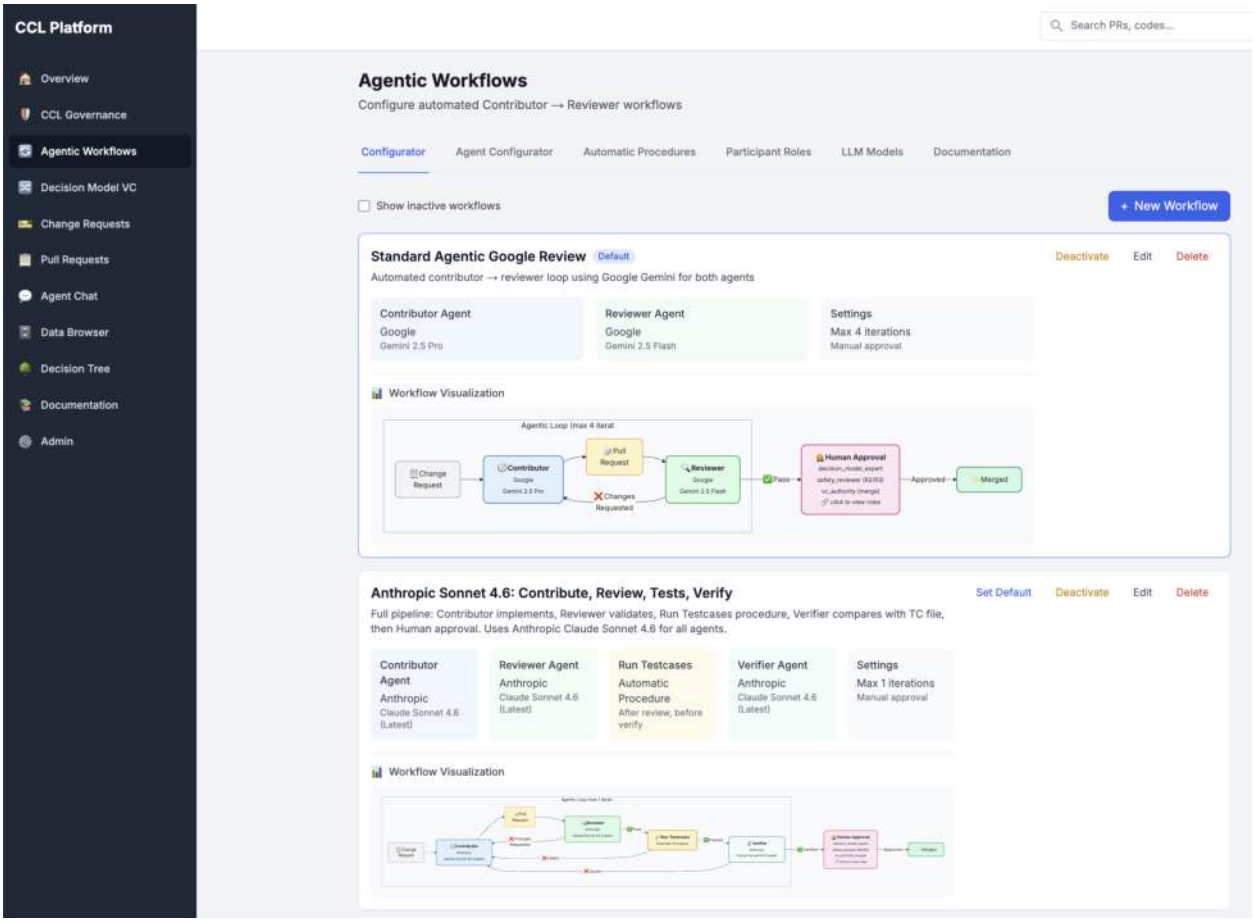


Figure 73: The Agentic Workflow Configurator. Administrators define multi-step pipelines specifying which agents participate, the iteration limit, and optional test and verification procedures. The lower panel lists existing workflow configurations with their status and execution counts.

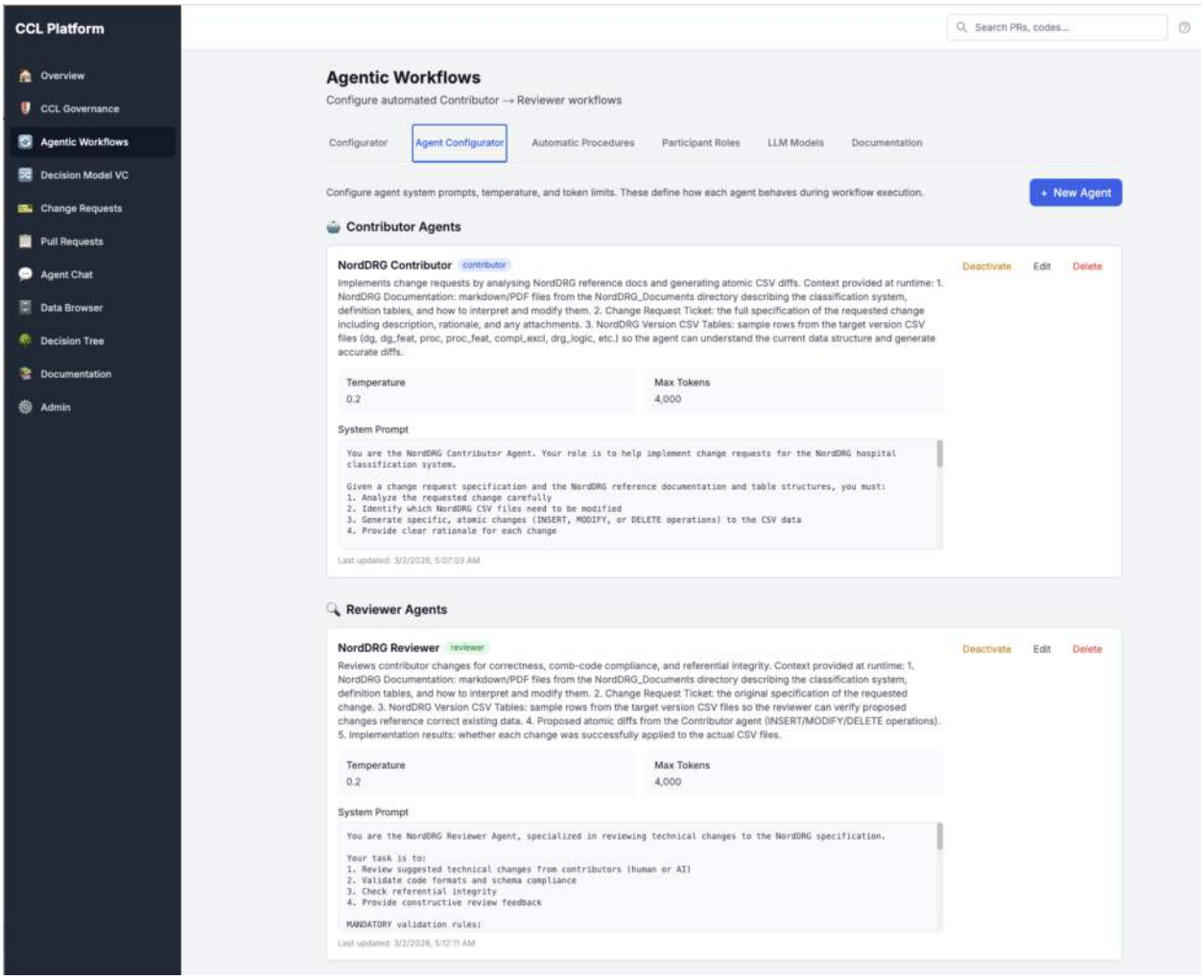


Figure 74: The Agent Configurator tab within Agentic Workflows. Each agent configuration specifies the role (Contributor, Reviewer, or Verifier), the LLM provider and model, a system prompt, and behavioral parameters. Configurations can be selected per workflow step and per Agent Chat session, allowing different tasks to use different models.

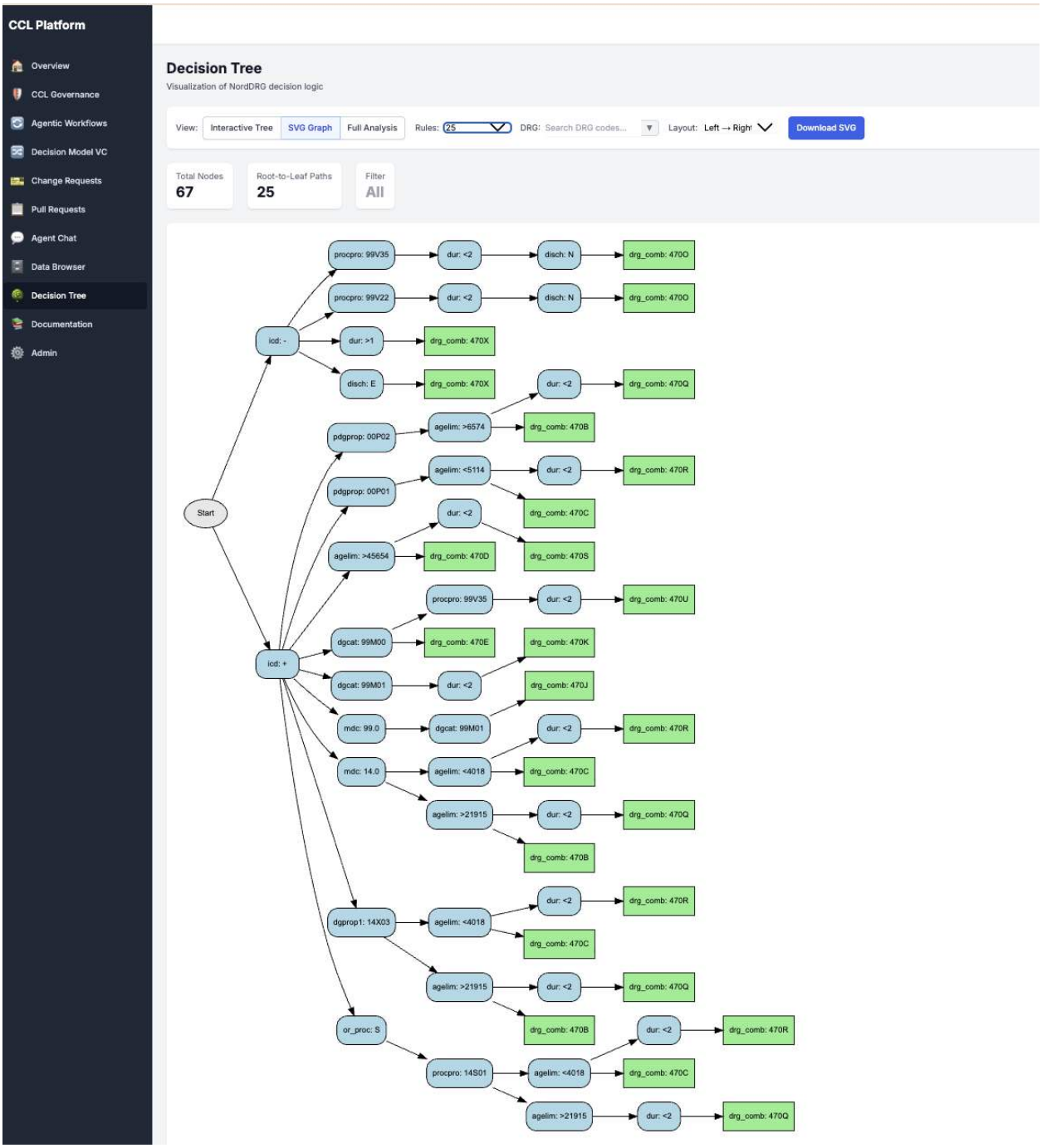


Figure 75: The Decision Tree page in SVG Graph mode, showing NordDRG decision logic as a flowchart. Rounded blue nodes represent decision points on input features (diagnosis codes, procedures, age, sex), and rectangular green leaf nodes represent DRG group outputs. The toolbar provides view-mode switching, DRG code filtering, layout orientation controls, and SVG export.

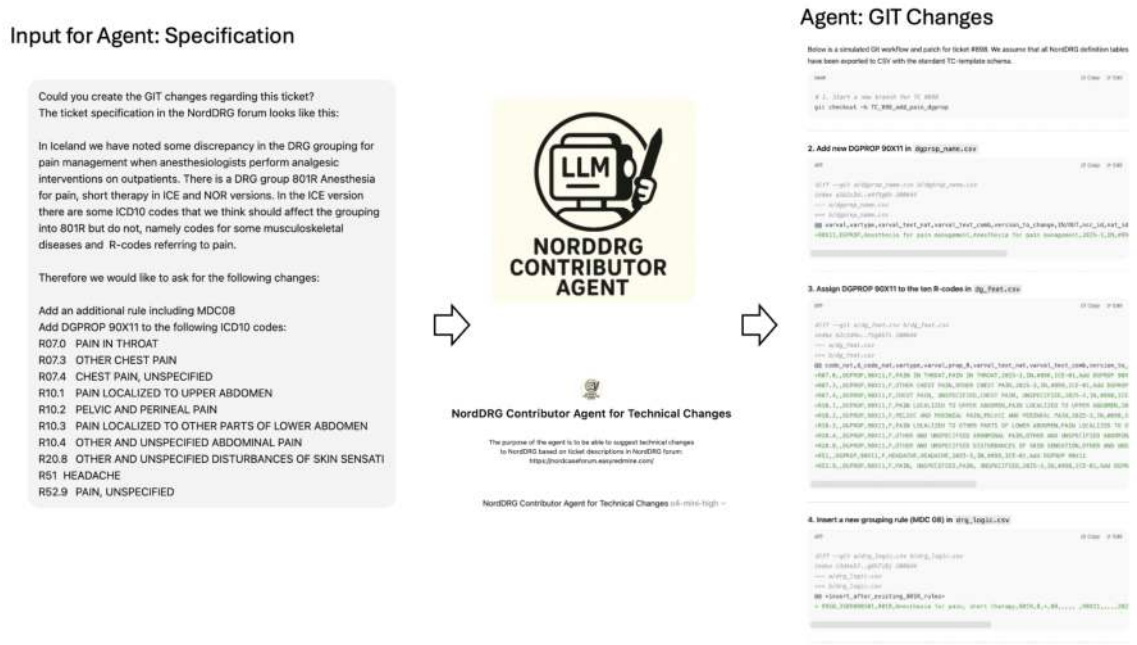


Figure 76: Contributor Agent processing a ticket from the NordDRG Forum. The agent proposes a change to the NordDRG specification, including atomic diffs and a rationale. The user can then review and export the suggestions for further processing.

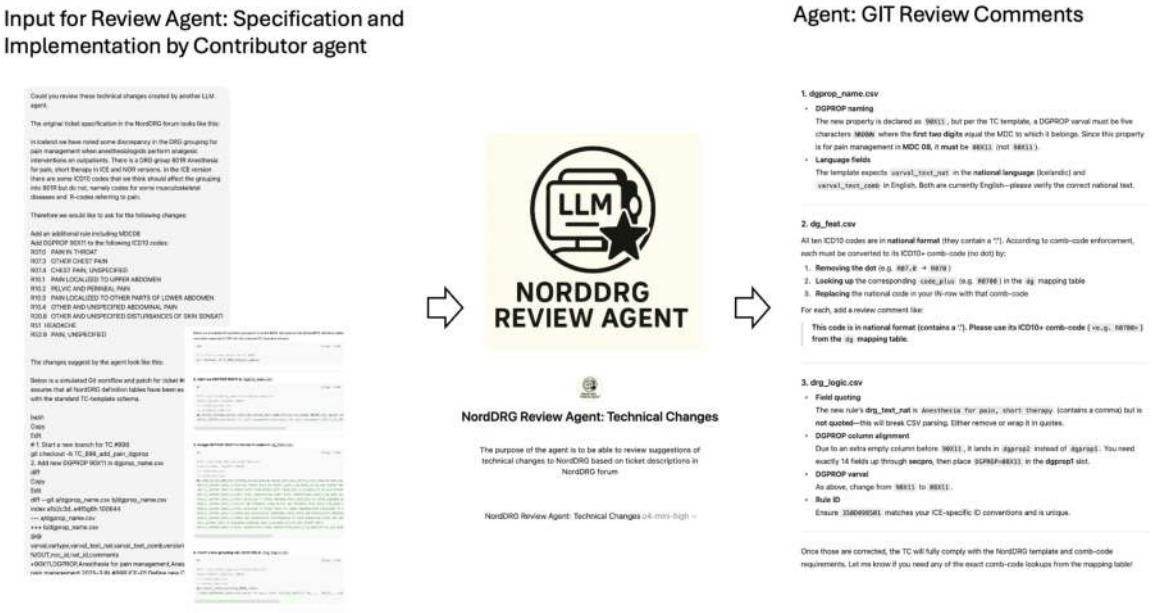


Figure 77: NordDRG Reviewer Agent taking in the implementation by the NordDRG Contributor Agent together with the original ticket. The Reviewer Agent proposes changes to the implementation to align with the NordDRG decision model coding standards and guidelines.

Reference answer keys (NordDRG AI Benchmark)

The gold-standard reference answers for the Logic and Grouper suites are listed here; they are cross-referenced from the NordDRG AI Benchmark sections (Tables 13 and 14) and remain part of the openly released benchmark.

Table 30: Reference answers for each CaseMix question (Logic-1–Logic-13).

ID	Reference answer
Logic-1	166C, 166N, 167N, 167X, 167O
Logic-2	166N, 167X, 167O
Logic-3	166N, 167X, 167O
Logic-4	370B, 370C, 370M, 370X, 371O, 371X, 372B, 372C, 372M, 372X, 373O, 373X, 374X, 375X, P05S, P05T
Logic-5	166C, 166N, 167N, 167X, 167O
Logic-6	JESA00, JESA01, JESA10, JESW96, JESW97
Logic-7	C1810, K3520
Logic-8	O6010, O6020, O6030, O6230, O6300, O6310, O6320, O6390, O6800, O6810, O6820, O6830, O6880, O6890, O7100, O7110, O7550, O7570, O8000, O8010, O8080, O8090, O8100, O8110, O8120, O8130, O8140, O8150, O8300, O8310, O8320, O8330, O8340, O8380, O8390, O8400, O8410, O8420, O8480, O8490, Z3700, Z3710, Z3720, Z3730, Z3740, Z3750, Z3760, Z3770, Z3790
Logic-9	14X03, 14X11, 14X13, 00X10
Logic-10	370B, 370C, 370M, 370X, 371O, 371X, 372B, 372C, 372M, 372X, 373O, 373X, 374X, 375X
Logic-11	385A, 385B, 385C, 386N, 387N, 388A, 388B, 388C, 389A, 389B, 389C, 390X, 391O, 391X, 815O, 858T, 858V, 858W, 915O, 970Q, Q69L, Q93R, Q97R, Q99R, Q99U, Q99W
Logic-12	No
Logic-13	Yes

Table 31: Reference answers for each CaseMix grouper question (Grouper-1–Grouper-13).

ID	Reference answer
Grouper-1	drg_nat=470E, drg_logic_id=000D004100
Grouper-2	drg_nat=373, drg_logic_id=400D783000
Grouper-3	drg_nat=704O, drg_logic_id=106D120110
Grouper-4	drg_nat=468, drg_logic_id=499D000100
Grouper-5	drg_nat=911O, drg_logic_id=111D169960
Grouper-6	drg_nat=384B, drg_logic_id=414D3710002
Grouper-7	drg_nat=388A, drg_logic_id=015D816140
Grouper-8	drg_nat=004, drg_logic_id=401D041000
Grouper-9	drg_nat=477, drg_logic_id=414D022100
Grouper-10	drg_nat=901O, drg_logic_id=101D089890
Grouper-11	drg_nat=125O, drg_logic_id=105D230000
Grouper-12	drg_nat=560A, drg_logic_id=400D313200
Grouper-13	drg_nat=291M, drg_logic_id=410D110101

Epilogue — grammar-review agent prompts (Chapter 9)

The two prompts of the grammar-review setup described in the Epilogue (Figure 25) are reproduced here.

Agent 1: Prompt for Grammatical Error Detection on Sentence Level

```
Your task will be to receive LaTeX-formatted sections of text. What you should do:
i) Find obvious grammatical mistakes from the text
ii) List all those sentences that have grammatical mistakes as they appear in the original text
iii) Do not list sentences that are grammatically correct but could be formulated in another way
```

Agent 2: Prompt for Grammatical Fixes on Sentence Level

```
Your task will be to receive LaTeX-formatted individual sentences of text. What you should do:
i) Find obvious grammatical mistakes from the text
ii) Rephrase the sentence so that it is grammatically correct
iii) Make sure that the content or message in the sentence does not change when you fix the grammar
```

References

[1] T. Pitkäranta, "Software agents in Semantic Web environment," master's thesis, Helsinki University of Technology, 2004.

[2] T. Pitkäranta, *Semantic Service Matching in Context-Aware Environment*. Licentiate thesis, Helsinki University of Technology, 2006.

[3] T. Pitkäranta, O. Riva, and S. Toivonen, "Designing and implementing a system for the provision of proactive context-aware services," in *Proceedings of the Workshop on Context Awareness for Proactive Systems (CAPS 2005), 16–17 June 2005, Helsinki, Finland*, vol. 1, pp. 21–30, 2005.

[4] O. Riva, T. Pitkäranta, and S. Toivonen, "Proactive context-aware service provisioning for a marine community," in *Proceedings of the Workshop on Context Awareness for Proactive Systems (CAPS 2005), 16–17 June 2005, Helsinki, Finland*, no. 1/2005 in HIIT Publications, p. 175, Helsinki Institute for Information Technology HIIT, 2005.

[5] S. Toivonen, T. Pitkäranta, and J. U. Min, "On Describing B2B Processes with Semantic Web Technologies," *ERCIM News*, pp. 48–49, Jan. 2004. `https://www.ercim.eu/publication/Ercim_News/enw56/toivonen.html`.

[6] S. Toivonen, H. Helin, M. Laukkanen, and T. Pitkäranta, "Context-sensitive conversation patterns for agents in wireless environments," in *IWUC*, pp. 11–17, 2004.

[7] S. Toivonen, T. Pitkäranta, H. Helin, and J. U. Min, "Using interaction protocols in distributed construction processes," in *ICEIS (4)*, pp. 344–349, 2004.

[8] S. Toivonen, T. Pitkäranta, and O. Riva, "Determining information usefulness in the Semantic Web: A distributed cognition approach," in *SemAnnot@ISWC*, 2005.

[9] M. Halinen, H. Koukkunen, and T. Pitkäranta, "Diagnoosien kirjaamiskäytännöissä kehittämisen varaa," *Suomen Lääkärilehti*, vol. 63, p. 2987, Sept. 2008.

[10] T. Pitkäranta, "The impact of diagnosis coding granularity on the casemix," in *PCSI 2008: Proceedings of the 24th Patient Classification Systems International Working Conference, October 2008, Lisbon, Portugal*, p. 10, 2008.

[11] T. Pitkäranta, "Kernel based imputation of coded data sets," in *Proceedings of the 2nd European Symposium on Time Series Prediction (ESTSP 2008)*, (Porvoo, Finland), Sept. 2008.

[12] T. Pitkäranta, "Affinity analysis of coded data sets," in *Proceedings of the 2009 EDBT/ICDT Workshops, Joint EDBT/ICDT Ph.D. Workshop, held with the 12th International Conference on Extending Database Technology (EDBT 2009), Saint Petersburg, Russia*, p. 10, 2009. ISBN: 978-1-60558-650-2.

[13] T. Pitkäranta, "Applying information retrieval for market basket recommender systems," in *ICEIS 2009: Proceedings of the 11th International Conference on Enterprise Information Systems, May 2009, Milan, Italy*, pp. 137–144, 2009. ISBN: 978-989-8111-84-5.

[14] T. Pitkäranta, "Applying machine learning for supporting clinical decision making and documentation," in *PCSI 2009: Proceedings of the 25th International Conference on Patient Classification Systems, November 2009, Fukuoka, Japan*, p. 10, 2009.

[15] T. Pitkäranta, "Applying software change management tools for clinical classification (ontology) development," in *PCSI 2009: Poster presentation at the 25th International Conference on Patient Classification Systems, November 2009, Fukuoka, Japan*, p. 5, 2009.

[16] T. Pitkäranta, M.-L. Sjöblom, and M. Virtanen, "National DRG cost weights in Finland," in *PCSI 2011: Proceedings of the 27th International Conference on Patient Classification Systems, November 2011, Montreal, Canada*, p. 10, 2011.

[17] T. Pitkäranta, P. Kokko, and M.-L. Sjöblom, "Hospital affinity analysis in DRG spectrum," in *PCSI 2012: Proceedings of the 28th International Conference on Patient Classification Systems, October 2012, Avignon, France*, p. 10, 2012.

[18] L. Pitkäranta, T. Pitkäranta, and N. Vuori, "Towards a software platform of algorithmic strategic decision-making: How to orchestrate algorithms to sing along with the company's strategy?," in *Proceedings of the PROS 2024 Conference*, (Cyprus), June 2024.

[19] L. Pitkäranta, T. Pitkäranta, and N. Vuori, "Towards a software platform of strategic decision-making: Orchestrating algorithms to sing along with company's strategy," 2024. Conference presentation at the 44th Annual Conference of the Strategic Management Society (SMS), Istanbul, Türkiye, 2024.

[20] T. Pitkäranta, "Discussing with your casemix specification with natural language: how to make large language models (LLM) understand NordDRG logic?," in *PCSI 2024: Proceedings of the 36th International Conference on Patient Classification Systems, May 2024, Bled, Slovenia*, 2024.

[21] T. Pitkäranta, "Teaching LLMs the nuances of hospital funding instruments," in *Computer-Human Interaction Research and Applications (CHIRA 2024), Part II* (H. P. da Silva and P. Cipresso, eds.), vol. 2371 of *Communications in Computer and Information Science*, (Cham), pp. 3–22, Springer, 2025.

[22] T. Pitkäranta, "Forecasting surprises in machine-learning-driven interaction systems: Lessons from the Transformer breakthrough," in *Computer-Human Interaction Research and Applications: 9th International Conference, CHIRA 2025, Marbella, Spain, October 20–21, 2025, Proceedings, Part II* (J. F. Krems, H. P. da Silva, and P. Cipresso, eds.), Communications in Computer and Information Science, (Cham), pp. 165–185, Springer, 2026. Presented at CHIRA 2025 (Marbella, Spain, October 20–21, 2025), held in conjunction with ICINCO, WEBIST, icSPORTS, and CoopIS 2025.

[23] T. Pitkäranta, "The NordDRG AI benchmark for large language models." `https://arxiv.org/abs/2506.13790`, 2025. arXiv preprint; the peer-reviewed CHIRA 2025 conference version is a substantially revised, distinct paper.

[24] T. Pitkäranta, "NordDRG AI benchmark for large language models," in *Computer-Human Interaction Research and Applications: 9th International Conference, CHIRA 2025, Marbella, Spain, October 20–21, 2025, Proceedings, Part II* (J. F. Krems, H. P. da Silva, and P. Cipresso, eds.), Communications in Computer and Information Science, (Cham), pp. 186–217, Springer, 2026. Peer-reviewed CHIRA 2025 conference version, substantially revised and distinct from the arXiv release (arXiv:2506.13790). Second-best-paper recognition.

[25] T. Pitkäranta and E. Hyvönen, "Semantic Web – a forgotten wave of artificial intelligence?," in *Proceedings of the 14th International Joint Conference on Knowledge Graphs (IJCKG 2025)*, (Heraklion, Crete, Greece), IJCKG, October 2025. arXiv:2503.20793 [cs.SI], `https://arxiv.org/abs/2503.20793`.

[26] T. Pitkäranta and L. Pitkäranta, "Bridging human and AI decision-making with LLMs: The RAGADA approach," in *Proceedings of the 26th International Conference on Enterprise Information Systems (ICEIS 2024)*, vol. 1, (Angers, France), pp. 812–819, SciTePress, 2024.

[27] T. Pitkäranta and L. Pitkäranta, "AADAR: Using agentic LLM architecture to align human and AI decisions," in *Lecture Notes in Business Information Processing*, vol. 566 of *LNBIP*, Springer, 2025.

[28] T. Pitkäranta and L. Pitkäranta, "HADA: Human-AI agent decision alignment architecture." `https://arxiv.org/abs/2506.04253`, 2025. Presented at AGENTICS 2025, part of the 17th International Joint Conference on Computational Intelligence (IJCCI 2025), Porto, Portugal, November 18–20, 2025. Also available as arXiv:2506.04253 [cs.AI].

[29] T. Pitkäranta and L. Pitkäranta, "Towards curated learning: Analyzing casemix NordDRG system expert system using machine learning tools," in *Proceedings of the 37th Patient Classification Systems International (PCSI) Conference*, (Quebec City, Canada), Sept. 2025. Best-paper award at PCSI 2025. Conference theme: Patient Grouping as a Driver of Value in Health: The Patient at the Heart of Our Decisions.

[30] T. Pitkäranta, "Why are we living the age of AI applications right now? The long innovation path from AI's birth to a child's bedtime magic." `https://arxiv.org/abs/2501.06929`, 2025.

[31] K. Peffers, T. Tuunanen, C. E. Gengler, M. Rossi, W. Hui, V. Virtanen, and J. Bragge, "Design science research process: A model for producing and presenting information systems research." arXiv preprint arXiv:2006.02763, 2020.

[32] ACM, "ACM computing classification system." `https://dl.acm.org/ccs`, 2024.

[33] M. Wooldridge, *Reasoning about Rational Agents*. The MIT Press, 2000.

[34] S. Toivonen and H. Helin, "Representing interaction protocols in DAML," in *Agent-Mediated Knowledge Management. Papers from 2003 AAAI Spring Symposium*, pp. 145–147, AAAI Press, 2003.

[35] S. Toivonen, "Hybrid service provision model for mobile users: Prospects for the DYNAMOS project," in *Proceedings of the 11th Finnish Artificial Intelligence Conference STeP 2004, September 1–3, Vantaa, Finland*, vol. 2, pp. 183–192, 2004.

[36] M. Steinert and L. Leifer, "Scrutinizing Gartner's hype cycle approach," in *PICMET 2010 Technology Management for Global Economic Growth*, pp. 1–13, IEEE, 2010.

[37] T. Berners-Lee, "WWW: Past, Present, and Future," *IEEE Computer*, vol. 29, pp. 69–77, October 1996.

[38] H. Helin, *Supporting Nomadic Agent-based Applications in the FIPA Agent Architecture*. PhD thesis, University of Helsinki, Finland, 2003.

[39] D. Fensel, J. Hendler, H. Lieberman, and W. Wahlster, *Spinning the Semantic Web*. The MIT Press, 2003. ISBN: 0-262-06232-1.

[40] D. Fensel and C. Bussler, "The Web Service Modeling Framework WSMF," tech. rep., Vrije Universiteit Amsterdam, Oracle Corporation, 2003. `https://courses.cs.umbc.edu/graduate/691/spring12/03/papers/wsmf.paper.pdf`.

[41] Foundation for Intelligent Physical Agents, "FIPA Contract Net Interaction Protocol Specification," Tech. Rep. XC00029, Geneva, Switzerland, 2000. `http://www.fipa.org/specs/fipa00029/`.

[42] FIPA, "FIPA Agent Management Specification," Tech. Rep. SC00023J, Foundation for Intelligent Physical Agents, 2002. `http://www.fipa.org/specs/fipa00023/SC00023J.html`.

[43] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, Mar. 2024. Code and resources available at `https://github.com/Paitesanshi/LLM-Agent-Survey`.

[44] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[45] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning." arXiv preprint arXiv:2303.11366, 2023.

[46] Y. Yang, H. Chai, Y. Song, S. Qi, M. Wen, N. Li, J. Liao, H. Hu, J. Lin, G. Chang, W. Liu, Y. Wen, Y. Yu, and W. Zhang, "A survey of AI agent protocols." arXiv preprint arXiv:2504.16736, 2025.

[47] "MCP general architecture." `https://modelcontextprotocol.io/introduction`, 2025. Accessed: 2025-08-21.

[48] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation." arXiv preprint arXiv:2308.08155, 2023.

[49] Nordic Casemix Centre, "NordDRG forum." `https://nordcaseforum.easyredmine.com`, 2025.

[50] F. Gandon, *Distributed Artificial Intelligence and Knowledge Management: Ontologies and Multi-Agent Systems for a Corporate Semantic Web*. PhD thesis, INRIA and University of Nice - Sophia Antipolis - Doctoral School of Sciences and Technologies of Information and Communication, 2002. `http://www-sop.inria.fr/acacia/personnel/Fabien.Gandon/research/PhD2002/`.

[51] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

[52] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[53] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, *et al.*, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[54] C. R. Jones and B. K. Bergen, "Large language models pass the Turing test." arXiv preprint arXiv:2503.23674, 2025.

[55] B. Christian, *The alignment problem: Machine learning and human values*. New York: W. W. Norton & Company, 2020.

[56] N. Bostrom, *Superintelligence: Paths, Dangers, Strategies*. Oxford: Oxford University Press, 2014.

[57] A. Wang, Y. Pruksachatkun, N. Nangia, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "SuperGLUE: A stickier benchmark for general-purpose language understanding systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[58] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.

[59] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[60] S. Gregor and D. Jones, "The anatomy of a design theory," *Journal of the Association for Information Systems*, vol. 8, no. 5, pp. 312–335, 2007.

[61] S. T. March and G. F. Smith, "Design and natural science research on information technology," *Decision Support Systems*, vol. 15, no. 4, pp. 251–266, 1995.

[62] T. Berners-Lee, "Information Management: A Proposal," March 1989. `http://www.w3.org/History/1989/proposal.html`.

[63] T. Berners-Lee and M. Fischetti, *Weaving the Web*. HarperBusiness, 2000. ISBN: 0-06-251587-X.

[64] T. Berners-Lee, J. Hendler, and O. Lassila, "The Semantic Web," *Scientific American*, vol. 284, pp. 34–43, May 2001. `http://www.scientificamerican.com/`.

[65] K. Kim, B. Paulson, C. Petrie, and V. Lesser, "Compensatory Negotiation for Agent-Based Project Schedule Coordination," 1999.

[66] The Linux Foundation, "Linux Foundation announces the formation of the Agentic AI Foundation (AAIF), anchored by new project contributions including Model Context Protocol (MCP), goose and AGENTS.md." Press release, `https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation`, Dec. 2025. Accessed: 2026-11-07.

[67] The Linux Foundation, "Linux Foundation launches the Agent2Agent protocol project to enable secure, intelligent communication between AI agents." Press release, `https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents`, June 2025. Accessed: 2026-11-08.

[68] J. B. Odubiyi, D. J. Kocur, S. M. Weinstein, N. Wakim, S. Srivastava, C. Gokey, and J. Graham, "SAIRE—a scalable agent-based information retrieval engine," in *Proceedings of the First International Conference on Autonomous Agents (AGENTS '97)*, (New York, NY, USA), pp. 292–299, ACM, 1997.

[69] T. Mikolov, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.

[70] P.-S. Huang, X. He, J. Gao, L. Deng, A. Acero, and L. Heck, "Learning deep structured semantic models for web search using clickthrough data," in *Proceedings of the 22nd ACM International Conference on Information and Knowledge Management (CIKM)*, pp. 2333–2338, 2013.

[71] P. Covington, J. Adams, and E. Sargin, "Deep neural networks for YouTube recommendations," in *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys)*, pp. 191–198, 2016.

[72] P. Lukac *et al.*, "Ambient AI scribes in clinical practice: A randomized trial," *NEJM AI*, 2025. Three-arm pragmatic randomized trial of 238 outpatient physicians, UCLA Health.

[73] A. Soroush, B. S. Glicksberg, E. Zimlichman, Y. Barash, R. Freeman, A. W. Charney, G. N. Nadkarni, and E. Klang, "Large language models are poor medical coders—benchmarking of medical code querying," *NEJM AI*, vol. 1, no. 5, 2024.

[74] H. Wang, Z. Wu, G. Kolar, H. Korsapati, B. Bartlett, B. Hull, and J. Sun, "Reinforcement learning for out-of-distribution reasoning in LLMs: An empirical study on diagnosis-related group coding," *arXiv preprint arXiv:2505.21908*, 2025.

[75] European Parliament and Council of the European Union, "Regulation (EU) 2026/1744 of the European Parliament and of the Council amending Regulation (EU) 2024/1689 (Artificial Intelligence Act) as regards simplification and the application timeline for high-risk AI systems (Digital Omnibus on AI)." Official Journal of the European Union, 24 July 2026, 2026. `http://data.europa.eu/eli/reg/2026/1744/oj`.

[76] A. Gupta, D. Basu, R. Ghantasala, S. Qiu, and U. Gadiraju, "To trust or not to trust: How a conversational interface affects trust in a decision support system," in *Proceedings of the ACM Web Conference 2022 (WWW '22)*, pp. 3531–3540, ACM, 2022.

[77] G. He, N. Aishwarya, and U. Gadiraju, "Is conversational XAI all you need? Human-AI decision making with a conversational XAI assistant," in *Proceedings of the 30th International Conference on Intelligent User Interfaces (IUI 2025)*, pp. 907–924, ACM, 2025.

[78] P. Bresciani, A. Perini, P. Giorgini, F. Giunchiglia, and J. Mylopoulos, "Tropos: An agent-oriented software development methodology," *Autonomous Agents and Multi-Agent Systems*, vol. 8, no. 3, pp. 203–236, 2004.

[79] P. Giorgini, J. Mylopoulos, E. Nicchiarelli, and R. Sebastiani, "Reasoning with goal models," in *Conceptual Modeling — ER 2002, 21st International Conference on Conceptual Modeling*, vol. 2503 of *Lecture Notes in Computer Science*, pp. 167–181, Springer, 2002.

[80] F. Dalpiaz, P. Giorgini, and J. Mylopoulos, "Adaptive socio-technical systems: a requirements-based approach," *Requirements Engineering*, vol. 18, no. 1, pp. 1–24, 2013.

[81] World Health Organization, "Global spending on health: Coping with the pandemic," tech. rep., World Health Organization, Geneva, Jan. 2024. Published 31 January 2024.

[82] S. Amershi, M. Cakmak, W. B. Knox, and T. Kulesza, "Power to the people: The role of humans in interactive machine learning," *AI Magazine*, vol. 35, no. 4, pp. 105–120, 2014.

[83] C. Rudin, "Stop explaining black box machine learning models for high-stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, pp. 206–215, 2019.

[84] M. T. Ribeiro, S. Singh, and C. Guestrin, ""Why Should I Trust You?": Explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144, 2016.

[85] A. Adadi and M. Berrada, "Peeking inside the black-box: A survey on explainable artificial intelligence (XAI)," *IEEE Access*, vol. 6, pp. 52138–52160, 2018.

[86] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (MLOps): Overview, definition, and architecture," *IEEE Access*, vol. 11, pp. 31866–31879, 2023.

[87] F. Zablith, G. Antoniou, M. d'Aquin, G. Flouris, H. Kondylakis, E. Motta, D. Plexousakis, and M. Sabou, "Ontology evolution: a process-centric survey," *The Knowledge Engineering Review*, vol. 30, no. 1, pp. 45–75, 2015.

[88] C. A. Curino, H. J. Moon, L. Tanca, and C. Zaniolo, "Schema evolution in Wikipedia: Toward a web information system benchmark," in *Proceedings of the Tenth International Conference on Enterprise Information Systems (ICEIS 2008)*, pp. 323–332, SciTePress, 2008.

[89] Anthropic, "Claude Code: Agentic coding tool." `https://docs.claude.com/en/docs/claude-code/overview`, 2025. Accessed: 2026-11-07.

[90] R. Lopopolo, "Harness engineering: Leveraging Codex in an agent-first world." OpenAI, `https://openai.com/index/harness-engineering/`, Feb. 2026. Accessed: 2026-11-07.

[91] S. Gregor and A. R. Hevner, "Positioning and presenting design science research for maximum impact," *MIS Quarterly*, vol. 37, no. 2, pp. 337–355, 2013.

[92] P. McCorduck, *Machines Who Think: A Personal Inquiry into the History and Prospects of Artificial Intelligence*. AK Peters Ltd, 2004.

[93] H. A. Kautz, "The third AI summer: AAAI Robert S. Engelmore Memorial Lecture," *AI Magazine*, 2022.

[94] M. Minsky and S. Papert, *Perceptrons: An Introduction to Computational Geometry*. MIT Press, 1969.

[95] W. Mettrey, "An assessment of tools for building large knowledge-based systems," *AI Magazine*, vol. 8, no. 4, pp. 81–90, 1987.

[96] J. D. Kelleher, *Deep learning*. The MIT Press essential knowledge series, Cambridge, MA and London: The MIT Press, 2019.

[97] P. Hitzler and M. K. Sarker, eds., *Neuro-Symbolic Artificial Intelligence: The State of the Art*. IOS Press, 2022.

[98] B. P. Bhuyan, A. Ramdane-Cherif, and R. Tomar, "Neuro-symbolic artificial intelligence: a survey," *Neural Computing and Applications*, vol. 36, pp. 12809–12844, 2024.

[99] T. Berners-Lee, "The Semantic Web wave," 2003. `https://www.w3.org/2003/Talks/01-siia-tbl/slide19-0.html`, Accessed: 2025-02-12.

[100] P. Hitzler, "A review of the Semantic Web field," *Communications of the ACM*, vol. 64, no. 2, pp. 76–83, 2021.

[101] S. Reed, K. Zolna, *et al.*, "A generalist agent," *DeepMind*, 2022.

[102] T. Menzies, "21st-century AI: proud, not smug," *IEEE Intelligent Systems*, vol. 18, no. 3, pp. 18–24, 2003.

[103] T. Noguchi, Y. Hashizume, H. Moriyama, L. Gauthier, Y. Ishikawa, T. Matsuno, and A. Suganuma, "A practical use of expert system "AI-Q" focused on creating training data," in *2018 5th International Conference on Business and Industrial Research (ICBIR)*, pp. 73–76, IEEE, 2018.

[104] A. Toosi, A. G. Bottino, B. Saboury, E. Siegel, and A. Rahmim, "A brief history of AI: how to prevent another winter (a critical review)," *PET Clinics*, vol. 16, no. 4, pp. 449–469, 2021.

[105] E. Francesconi, "The winter, the summer and the summer dream of artificial intelligence in law: Presidential address to the 18th International Conference on Artificial Intelligence and Law," *Artificial Intelligence and Law*, vol. 30, no. 2, pp. 147–161, 2022.

[106] O. Colliot, "A non-technical introduction to machine learning," in *Machine Learning for Brain Disorders*, pp. 3–23, Springer, 2023.

[107] J. Harguess and C. M. Ward, "Is the next winter coming for AI? Elements of making secure and robust AI," in *2022 IEEE Applied Imagery Pattern Recognition Workshop (AIPR)*, pp. 1–7, IEEE, 2022.

[108] AI Perspectives, "AI perspectives." `https://www.aiperspectives.com/introduction/`, 2020. Accessed: 2025-01-12.

[109] A. Lu, "AI summer and winter: The ups and downs of AI." `https://www.labelf.ai/blog/four-seasons-of-ai-the-brief-history-of-artificial-intelligence-i`, 2022. Accessed: 2025-01-12.

[110] Brightwork Research and Analysis, "The rise of AI." `https://www.brightworkresearch.com/why-did-ai-rise-for-a-third-time-after-the-1st-and-2nd-ai-winters/`, 2020. Accessed: 2025-01-12.

[111] S. Winiger, "AI winters = HCI summers." `https://samim.io/p/2018-02-19-ai-winters-hci-summers/`, 2017. Accessed: 2025-01-12.

[112] Y. Altmann, "Small history of AI," June 2023. Accessed: December 28, 2024, `https://omq.ai/blog/history-of-ai/`.

[113] J. Latta, "History of AI – from winter to winter," 2021. Accessed: 2024-12-29, `https://jaylatta.net/history-of-ai-from-winter-to-winter/`.

[114] S. Schuchmann, "History of the first AI winter," 2019. Accessed: 2024-12-29, `https://towardsdatascience.com/history-of-the-first-ai-winter-6f8c2186f80b`.

[115] G. Hinton, "Full interview: 'Godfather of artificial intelligence' talks impact and potential of AI." `https://youtu.be/qpoRO378qRY`, 2023. Interview.

[116] H. Nwana, "Software Agents: An Overview," *The Knowledge Engineering Review*, vol. 11, no. 3, pp. 205–244, 1996.

[117] H. S. Nwana and D. T. Ndumu, "A Perspective on Software Agents Research," *The Knowledge Engineering Review*, vol. 14, no. 2, pp. 125–142, 1999.

[118] V. Bush, *As we may think*, pp. 85–110. ISBN: 0-12-523270-5, From Memex to hypertext: Vannevar Bush and the mind's machine, Academic Press Professional, Inc., 1945. `http://www.theatlantic.com/unbound/flashbks/computer/bushf.htm`.

[119] T. H. Nelson, "A file structure for the complex, the changing, and the indeterminate," in *Proceedings of the 1965 20th National Conference*, (New York, NY, USA), pp. 84–100, Association for Computing Machinery, 1965.

[120] T. Nelson, "Getting It Out of Our System: Critique of Information Retrieval," *Thompson Books*, pp. 191–210, 1967.

[121] G. Wolf, "The Curse of Xanadu," *Wired Magazine*, Jun 1995. `https://www.wired.com/1995/06/xanadu/`.

[122] D. C. Engelbart and W. K. English, *A Research Center For Augmenting Human Intellect*, vol. 33 of *AFIPS Conference Proceedings*, pp. 395–410. Thompson Book Co, 1968.

[123] T. Berners-Lee, L. Masinter, and M. McCahill, "Uniform Resource Locators (URL)," RFC 1738, Internet Engineering Task Force, Dec. 1994. http://www.ietf.org/rfc/rfc1738.txt.

[124] T. Berners-Lee, R. Fielding, and L. Masinter, "Uniform Resource Identifiers (URI): Generic Syntax," RFC 2396, Internet Engineering Task Force, Aug. 1998. URL: http://www.ietf.org/rfc/rfc2396.txt.

[125] W3C, "World Wide Web Consortium Recommendation: Naming and Addressing: URIs and URLs." URL: http://www.w3.org/Addressing/.

[126] T. Berners-Lee, R. Fielding, L. Masinter, H. Frystyk, P. Leach, J. Gettys, and J. Mogul, "Hypertext Transfer Protocol – HTTP/1.1," RFC 2616, Internet Engineering Task Force, June 1999. URL: http://www.ietf.org/rfc/rfc2616.txt.

[127] W3C, "World Wide Web Consortium Recommendation: HyperText Markup Language (HTML)." URL: http://www.w3.org/MarkUp/.

[128] World Wide Web Consortium, "A Little History of the World Wide Web." URL: http://www.w3.org/History.html.

[129] A. Weaver and T. Hall, "The Internet and the World Wide Web," *IEEE Industrial Electronics, Control and Instrumentation*, vol. 4, pp. 1529–1540, 1997.

[130] L. Paulson, "Building rich web applications with AJAX," *Computer*, vol. 38, no. 10, pp. 14–17, 2005.

[131] T. O'Reilly, "What is Web 2.0? Design patterns and business models for the next generation of software," September 2005. https://www.oreilly.com/pub/a/web2/archive/what-is-web-20.html, Accessed 17 Jun 2025.

[132] A. Russell, "Progressive Web Apps: Escaping tabs without losing our soul," 2016. https://web.dev/progressive-web-apps/, Accessed 17 Jun 2025.

[133] A. Haas, A. Rossberg, D. Schuff, *et al.*, "Bringing the Web up to speed with WebAssembly," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '17)*, pp. 185–200, 2017.

[134] C. Hewitt, "Viewing control structures as patterns of passing messages," *Artificial Intelligence*, vol. 8, no. 3, pp. 323–364, 1977.

[135] A. H. Bond and L. Gasser, eds., *Readings in Distributed Artificial Intelligence*. Morgan Kaufmann, 1988.

[136] L. Gasser and M. N. Huhns, eds., *Distributed Artificial Intelligence: Volume II*, vol. 2. Morgan Kaufmann, 1989.

[137] M. Wooldridge, *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2009.

[138] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–192, 1995. http://agents.umbc.edu/introduction/.

[139] P. Maes, "Agents that reduce work and information overload," *Communications of the ACM*, vol. 37, no. 7, pp. 30–40, 1994.

[140] G. Weiss, *Multiagent Systems: A Modern Approach to Distributed Artificial Intelligence*. MIT Press, 1999.

[141] S. Franklin and A. Graesser, "Is it an Agent, or just a Program?: A Taxonomy for Autonomous Agents," in *Intelligent Agents III. Agent Theories, Architectures and Languages (ATAL'96)*, vol. 1193, (Berlin, Germany), pp. 21–35, Springer-Verlag, 1996.

[142] Foundation for Intelligent Physical Agents (FIPA), "FIPA '97 Specification, Version 1.0 — Part 2: Agent Communication Language," standard specification, FIPA, Geneva, Switzerland, 1997.

[143] TILAB, "Java Agent DEvelopment Framework (JADE)," 2004. Open-source agent platform, https://jade.tilab.com/.

[144] D. L. McGuinness and F. Van Harmelen, "OWL Web Ontology Language overview," *W3C Recommendation*, 2004.

[145] N. Shadbolt, W. Hall, and T. Berners-Lee, "The Semantic Web revisited," *IEEE Intelligent Systems*, vol. 21, no. 3, pp. 96–101, 2006.

[146] J. Hendler, "Agents and the Semantic Web," *IEEE Intelligent Systems*, vol. 16, no. 2, pp. 30–37, 2001.

[147] S. A. McIlraith, T. C. Son, and H. Zeng, "Semantic web services," *IEEE Intelligent Systems*, vol. 16, no. 2, pp. 46–53, 2001.

[148] Foundation for Intelligent Physical Agents, "FIPA ACL Message Structure Specification," Tech. Rep. XC00061, Geneva, Switzerland, 2000. http://www.fipa.org/specs/fipa00061/.

[149] Foundation for Intelligent Physical Agents, "FIPA Communicative Act Library Specification," Tech. Rep. XC00037, Geneva, Switzerland, 2002. http://www.fipa.org/specs/fipa00037/.

[150] O. Lassila and R. R. Swick, "Resource description framework (RDF) model and syntax specification," *W3C Proposed Recommendation PR-rdf-syntax-19981008*, 1998.

[151] I. Horrocks, P. F. Patel-Schneider, and F. Van Harmelen, "From SHIQ and RDF to OWL: The making of a web ontology language," in *International Semantic Web Conference*, vol. 1, pp. 7–26, Springer, 2003.

[152] T. R. Gruber, "Toward principles for the design of ontologies used for knowledge sharing," *International Journal of Human-Computer Studies*, vol. 43, no. 5-6, pp. 907–928, 1995.

[153] R. Studer, V. R. Benjamins, and D. Fensel, "Knowledge engineering: Principles and methods," *Data & Knowledge Engineering*, vol. 25, no. 1–2, pp. 161–197, 1998.

[154] D. Martin, M. Burstein, J. Hobbs, O. Lassila, D. McDermott, S. McIlraith, S. Narayanan, M. Paolucci, B. Parsia, T. Payne, *et al.*, "OWL-S: Semantic markup for web services," in *W3C Member Submission*, 2004.

[155] D. Brickley and R. V. Guha, "RDF vocabulary description language 1.0: RDF schema," *W3C Recommendation*, vol. 10, no. 1, pp. 1–107, 2004.

[156] I. Horrocks, P. F. Patel-Schneider, H. Boley, S. Tabet, B. Grosof, and M. Dean, "SWRL: A semantic web rule language combining OWL and RuleML," *W3C Member Submission*, 2004. Accessed: 2024-01-26.

[157] H. Boley, G. Hallmark, M. Kifer, A. Paschke, A. Polleres, and D. Reynolds, "RIF core dialect." W3C Recommendation, https://www.w3.org/TR/rif-core/, 2010.

[158] E. Prud'hommeaux and A. Seaborne, "SPARQL query language for RDF." W3C Recommendation, https://www.w3.org/TR/rdf-sparql-query/, 2008.

[159] B. N. Grosof, I. Horrocks, R. Volz, and S. Decker, "Description logic programs: Combining logic programs with description logic," in *Proceedings of the 12th International World Wide Web Conference (WWW 2003)*, (Budapest, Hungary), pp. 48–57, ACM, 2003.

[160] B. Motik, U. Sattler, and R. Studer, "Query answering for OWL-DL with rules," *Journal of Web Semantics: Science, Services and Agents on the World Wide Web*, vol. 3, no. 1, pp. 41–60, 2005.

[161] D. Longley, G. Kellogg, D. Yamamoto, and M. Sporny, "RDF dataset canonicalization 1.0," W3C Recommendation REC-rdf-canon-20240521, World Wide Web Consortium (W3C), May 2024.

[162] M. Sporny, T. Thibodeau Jr., I. Herman, D. Longley, and G. Bernstein, "Verifiable credential data integrity 1.0," W3C Recommendation REC-vc-data-integrity-20250515, World Wide Web Consortium (W3C), May 2025.

[163] M. Sporny, T. Thibodeau Jr., I. Herman, G. Cohen, and M. B. Jones, "Verifiable credentials data model v2.0," W3C Recommendation REC-vc-data-model-2.0-20250515, World Wide Web Consortium (W3C), May 2025.

[164] M. Sporny, A. Guy, M. Sabadello, and D. Reed, "Decentralized identifiers (DIDs) v1.0," W3C Recommendation REC-did-core-20220719, World Wide Web Consortium (W3C), July 2022.

[165] B. Motik, I. Horrocks, and U. Sattler, "Bridging the gap between OWL and relational databases," in *Proceedings of the 16th International Conference on World Wide Web (WWW '07)*, pp. 807–816, ACM, 2007.

[166] C. Bizer, T. Heath, and T. Berners-Lee, "Linked Data — the story so far," *International Journal on Semantic Web and Information Systems*, vol. 5, no. 3, pp. 1–22, 2009.

[167] N. Noy, Y. Gao, A. Jain, A. Narayanan, A. Patterson, and J. Taylor, "Industry-scale knowledge graphs: Lessons and challenges," *Communications of the ACM*, vol. 62, no. 8, pp. 36–43, 2019.

[168] M. Nickel, K. Murphy, V. Tresp, and E. Gabrilovich, "A review of relational machine learning for knowledge graphs," *Proceedings of the IEEE*, vol. 104, no. 1, pp. 11–33, 2016.

[169] R. Confalonieri and G. Guizzardi, "On the multiple roles of ontologies in explainable AI." arXiv preprint arXiv:2311.04778, 2023.

[170] M. Li, S. Miao, and P. Li, "Simple is effective: The roles of graphs and large language models in knowledge-graph-based retrieval-augmented generation." arXiv preprint arXiv:2410.20724, 2025.

[171] J. Amara, B. König-Ries, and S. Samuel, "Enhancing explainability in multimodal large language models using ontological context." arXiv preprint arXiv:2409.18753, 2024.

[172] A. Singhal, "Introducing the Knowledge Graph: Things, not strings," May 2012. https://blog.google/products/search/introducing-knowledge-graph-things-not/, Accessed 17 Jun 2025.

[173] S. Lassen, M. Curtiss, and S. Sankar, "Under the hood: Building out the infrastructure for Graph Search," Mar. 2013. https://engineering.fb.com/2013/03/06/core-infra/under-the-hood-building-out-the-infrastructure-for-graph-search/, Accessed 17 Jun 2025.

[174] S. Nadella, "Satya Nadella | BG2 w/ Bill Gurley & Brad Gerstner." YouTube, Dec. 2024. Available at: https://youtu.be/9NtsnzRFJ_o?t=2811, Accessed: March 2025.

[175] O. Etzioni and D. Weld, "Intelligent agents on the Internet: Fact, fiction, and forecast," *IEEE Intelligent Internet Services*, vol. 10, pp. 44–49, August 1995.

[176] J. Hendler, "Making Sense out of Agents," *IEEE Intelligent Systems*, vol. 14, pp. 32–37, Mar./Apr. 1999.

[177] N. Jennings, *On Agent-Based Software Engineering*, vol. 117 of *Artificial Intelligence*, pp. 277–296. Elsevier Science, 2000. https://eprints.soton.ac.uk/253741/.

[178] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Prentice Hall, second ed., 2003. ISBN: 0-13-080302-2.

[179] C. J. Petrie, "Agent-Based Engineering, the Web, and Intelligence," *IEEE Expert*, vol. 11, pp. 24–29, Dec 1996. http://www-cdr.stanford.edu/NextLink/Expert.html.

[180] K. Sycara, "The Many Faces of Agents," *AI Magazine*, vol. 19, no. 2, 1998. http://www.aaai.org/Resources/Papers/AIMag19-02-002.pdf.

[181] S. Toivonen, "Ohjelmistoagentin määritteleminen," Master's thesis, University of Helsinki, Department of Psychology, 1999.

[182] M. Wooldridge, N. R. Jennings, and D. Kinny, "A Methodology for Agent-Oriented Analysis and Design," pp. 69–76, 1999.

[183] T. Finin and C. Nicholas, "Agent-based Information Retrieval," 1998. http://www.csee.umbc.edu/abir/.

[184] J. Searle, *Speech acts: an essay in the philosophy of language*. Cambridge: Cambridge University Press, 1969.

[185] M. E. Bratman, *Intentions, Plans, and Practical Reason*. Cambridge, USA: Harvard, 1987. ISBN: 0-674-45818.

[186] A. S. Rao and M. P. Georgeff, "BDI agents: From theory to practice," in *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS)*, pp. 312–319, 1995.

[187] R. Brooks, "A robust layered control system for a mobile robot," *IEEE Journal on Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.

[188] P. Maes, *Designing autonomous agents: Theory and practice from biology to engineering and back*. MIT Press, 1990.

[189] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *Journal of Artificial Intelligence Research*, vol. 4, pp. 237–285, 1996.

[190] Y. Labrou, T. Finin, and Y. Peng, "Agent Communication Languages: The Current Landscape," *IEEE Intelligent Systems*, vol. 14, pp. 45–52, Mar./Apr. 1999.

[191] Foundation for Intelligent Physical Agents (FIPA), "Mission." http://www.fipa.org/about/mission.html, 2003. Accessed: 2025-08-25.

[192] FIPA, "Publicly Available FIPA Agent Platform Implementations," 2004. http://www.fipa.org/resources/livesystems.html.

[193] M. Laukkanen, "Evaluation of FIPA-Compliant Agent Platforms," Master's thesis, Lappeenranta University of Technology, 1999.

[194] Open Source, "Lightweight Extensible Agent Platform (LEAP)," 2004. http://leap.crm-paris.com/.

[195] R. T. Fielding, *Architectural Styles and the Design of Network-Based Software Architectures*. PhD thesis, University of California, Irvine, 2000.

[196] gRPC Authors, "gRPC: A high performance, open source universal RPC framework." `https://grpc.io`, 2015.

[197] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proceedings of the NetDB Workshop*, 2011.

[198] A. Videla and J. Williams, *RabbitMQ in Action*. Manning, 2012.

[199] G. Alonso, F. Casati, H. Kuno, and V. Machiraju, *Web Services: Concepts, Architectures and Applications*. Springer, 2004.

[200] A. Wollrath, R. Riggs, and J. Waldo, "A distributed object model for the Java system," *USENIX Computing Systems*, vol. 9, no. 4, pp. 471–497, 1996.

[201] Object Management Group, "Common object request broker architecture (CORBA) specification version 2.1," tech. rep., OMG, 1997.

[202] Microsoft, "MS-DCOM: Overview." `https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-dcom/86b9cf84-df2e-4f0b-ac22-1b957627e1ca`, 2024.

[203] T. Finin, R. Fritzson, D. McKay, and R. McEntire, "KQML as an agent communication language," in *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM94)*, (New York), pp. 456–463, Association for Computing Machinery, 1994. `http://www.cs.umbc.edu/kqml/papers/kqmlacl.pdf`.

[204] Foundation for Intelligent Physical Agents, "FIPA ACL Message Representation in XML Specification," Tech. Rep. SC00071, Geneva, Switzerland, 2002. `http://www.fipa.org/specs/fipa00071/`.

[205] Foundation for Intelligent Physical Agents, "FIPA Content Language Library Specification," Tech. Rep. SC00007, Geneva, Switzerland, 2002. `http://www.fipa.org/specs/fipa00007/`.

[206] FIPA, "FIPA Interaction Protocol Library," tech. rep., Foundation for Intelligent Physical Agents, 2002. `http://www.fipa.org/repository/ips.html`.

[207] M. Purvis, S. Cranefield, G. Bush, D. Carter, B. McKinlay, and M. Nowostawski, "The NZDIS project: An agent-based distributed information systems architecture," in *Proceedings of the 33rd Annual Hawaii International Conference on System Sciences*, IEEE, 2000. `http://nzdis.otago.ac.nz/download/papers/nzdis-project_1-00.pdf`.

[208] R. G. Smith, "The contract net protocol: High-level communication and control in a distributed problem solver," *IEEE Transactions on Computers*, vol. C-29, no. 12, pp. 1104–1113, 1980.

[209] Foundation for Intelligent Physical Agents, "FIPA SL Content Language Specification," Tech. Rep. SC00008I, Geneva, Switzerland, 2002. `http://www.fipa.org/specs/fipa00008/`.

[210] M. R. Genesereth and R. E. Fikes, "Knowledge Interchange Format, Version 3.0 Reference Manual," Tech. Rep. Logic-92-1, Computer Science Department, Stanford University, Stanford, CA, USA, June 1992.

[211] S. J. Russell and P. Norvig, *Artificial intelligence: a modern approach*. Prentice Hall, 3rd ed., 2010.

[212] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, *et al.*, "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, pp. 27730–27744, 2022.

[213] J. Wei, X. Wang, D. Schuurmans, M. Bosma, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837, 2022.

[214] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, "Why do multi-agent LLM systems fail?," *arXiv preprint arXiv:2503.13657*, 2025.

[215] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, "A comprehensive overview of large language models." arXiv preprint arXiv:2307.06435, 2024.

[216] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, W. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, "The rise and potential of large language model based agents: A survey." arXiv preprint arXiv:2309.07864, 2023.

[217] S. Yang, O. Nachum, Y. Du, J. Wei, P. Abbeel, and D. Schuurmans, "Foundation models for decision making: Problems, methods, and opportunities." arXiv preprint arXiv:2303.04129, 2023.

[218] H. Chase, "LangChain." `https://github.com/langchain-ai/langchain`, 2022. Accessed: 2024-01-26.

[219] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, pp. 1–38, Mar. 2023.

[220] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, V. Kulkarni, M. Lewis, N. Reimers, S. Riedel, and L. Zettlemoyer, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[221] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the dangers of stochastic parrots: Can language models be too big?," in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, pp. 610–623, 2021.

[222] L. Weidinger, J. Mellor, M. Rauh, C. Griffin, J. Uesato, P.-S. Huang, M. Cheng, M. Glaese, B. Balle, A. Kasirzadeh, Z. Kenton, S. Brown, W. Hawkins, T. Stepleton, C. Biles, A. Birhane, J. Haas, L. Rimell, L. A. Hendricks, W. Isaac, S. Legassick, G. Irving, and I. Gabriel, "Ethical and social risks of harm from language models." arXiv preprint arXiv:2112.04359, 2021.

[223] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning." arXiv preprint arXiv:1702.08608, 2017.

[224] R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, S. Jain, V. Kosaraju, W. Saunders, X. Jiang, K. Cobbe, T. Eloundou, G. Krueger, K. Button, M. Knight, B. Chess, and J. Schulman, "WebGPT: Browser-assisted question-answering with human feedback." arXiv preprint arXiv:2112.09332, 2022.

[225] O. Ram, Y. Levine, I. Dalmedigos, D. Muhlgay, A. Shashua, K. Leyton-Brown, and Y. Shoham, "In-context retrieval-augmented language models." Transactions of the Association for Computational Linguistics, 11:1316–1331, 2023.

[226] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," *arXiv preprint arXiv:2308.03688*, 2023.

[227] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[228] Y. Gao, Y. Xiong, X. Gao, *et al.*, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[229] X. Zhu, Y. Chen, H. Tian, C. Tao, W. Su, C. Yang, G. Huang, B. Li, L. Lu, X. Wang, Y. Qiao, Z. Zhang, and J. Dai, "Ghost in the Minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory," *arXiv preprint arXiv:2305.17144*, 2023.

[230] E. Karpas, O. Abend, Y. Belinkov, B. Lenz, O. Lieber, N. Ratner, Y. Shoham, H. Bata, Y. Levine, K. Leyton-Brown, D. Muhlgay, N. Rozen, E. Schwartz, G. Shachaf, S. Shalev-Shwartz, A. Shashua, and M. Tenenholtz, "MRKL systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning," *arXiv preprint arXiv:2205.00445*, 2022.

[231] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *arXiv preprint arXiv:2302.04761*, 2023.

[232] OpenAI, "Function calling and other API updates." `https://openai.com/index/function-calling-and-other-api-updates/`, 2023.

[233] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face," *arXiv preprint arXiv:2303.17580*, 2023.

[234] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," *arXiv preprint arXiv:2305.10601*, 2023.

[235] E. Meyerson, G. Paolo, R. Dailey, H. Shahrzad, O. Francon, C. F. Hayes, X. Qiu, B. Hodjat, and R. Miikkulainen, "Solving a million-step LLM task with zero errors." arXiv preprint arXiv:2511.09030, 2025.

[236] S. Marro, E. L. Malfa, J. Wright, G. Li, N. Shadbolt, M. Wooldridge, and P. Torr, "A scalable communication protocol for networks of large language models." arXiv preprint arXiv:2410.11905, 2024.

[237] Wildcard AI, K. Mahorker, and Y. Patel, "agents-json: Translate OpenAPI into LLM tools." https://github.com/wild-card-ai/agents-json, 2025. GitHub repository, release 0.1.3 (2025-02-09).

[238] Google Developers, "Announcing the Agent2Agent protocol (A2A)." https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/, Apr. 2025. Accessed: 2026-11-08.

[239] Google, "Agent2Agent (A2A) protocol repository." https://github.com/a2aproject/A2A, 2025. GitHub Repository.

[240] B. Zhang, K. Lazuka, and M. Murag, "Equipping agents for the real world with Agent Skills." Anthropic Engineering, https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills, Oct. 2025. Published 16 October 2025. Accessed: 2026-11-11.

[241] Anthropic, "Agent Skills specification." https://agentskills.io/specification, Dec. 2025. Open format originally developed by Anthropic and published as an open standard on 18 December 2025; donated to the Agentic AI Foundation (Linux Foundation) on 22 October 2026. Accessed: 2026-11-11.

[242] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework." arXiv preprint arXiv:2308.00352, 2023. Published as a conference paper at ICLR 2024.

[243] LangChain Inc., "LangChain documentation." https://docs.langchain.com/oss/python/langchain/overview, 2025. Accessed: 2025-08-21.

[244] LangChain Inc., "LangGraph: Building stateful, multi-actor applications with LLMs." https://docs.langchain.com/oss/python/langgraph/overview, 2025. Accessed: 2025-08-21.

[245] Microsoft Research, "AutoGen documentation." https://microsoft.github.io/autogen/, 2025. Accessed: 2025-08-21.

[246] LlamaIndexAI, Inc., "LlamaIndex documentation." https://developers.llamaindex.ai/python/framework/, 2025. Accessed: 2025-08-21.

[247] CrewAI, "CrewAI documentation." https://docs.crewai.com/, 2025. Accessed: 2025-08-21.

[248] Microsoft, "Semantic Kernel documentation." https://learn.microsoft.com/semantic-kernel/, 2025. Accessed: 2025-08-21.

[249] deepset AI, "Haystack documentation." https://docs.haystack.deepset.ai/docs/intro, 2025. Accessed: 2025-08-21.

[250] Embedchain, "Embedchain documentation." https://docs.embedchain.ai/, 2025. Accessed: 2025-08-21.

[251] LangGenius, Inc., "Dify documentation." https://docs.dify.ai/, 2025. Accessed: 2025-08-21.

[252] T. Xie, F. Zhou, Z. Cheng, P. Shi, L. Weng, Y. Liu, T. J. Hua, J. Zhao, Q. Liu, C. Liu, L. Z. Liu, Y. Xu, H. Su, D. Shin, C. Xiong, and T. Yu, "OpenAgents: An open platform for language agents in the wild," *arXiv preprint arXiv:2310.10634*, 2023.

[253] Google, "Agent Development Kit (ADK)." https://google.github.io/adk-docs/, 2025. Accessed: 2025-08-21.

[254] Google Firebase, "Firebase Genkit." https://genkit.dev/, 2025. Accessed: 2025-08-21.

[255] Prefect, Inc., "Marvin: AI engineering framework for Python." https://github.com/PrefectHQ/marvin, 2025. Accessed: 2025-08-21.

[256] H. Mouratidis and P. Giorgini, "Secure Tropos: a security-oriented extension of the Tropos methodology," *International Journal of Software Engineering and Knowledge Engineering*, vol. 17, no. 2, pp. 285–309, 2007.

[257] P. Giorgini, F. Massacci, J. Mylopoulos, and N. Zannone, "Requirements engineering for trust management: model, methodology, and reasoning," *International Journal of Information Security*, vol. 5, no. 4, pp. 257–274, 2006.

[258] Y. Bai, S. Kadavath, S. Kundu, *et al.*, "Constitutional AI: Harmlessness from AI feedback," *arXiv preprint arXiv:2212.08073*, 2022.

[259] A. Chan, C. Ezell, M. Kaufmann, K. Wei, L. Hammond, H. Bradley, E. Bluemke, N. Rajkumar, D. Krueger, N. Kolt, L. Heim, and M. Anderljung, "Visibility into AI agents," in *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2024.

[260] F. Santoni de Sio and J. van den Hoven, "Meaningful human control over autonomous systems: A philosophical account," *Frontiers in Robotics and AI*, vol. 5, 2018.

[261] European Parliament and Council of the European Union, "Regulation (EU) 2024/1689 of the European Parliament and of the Council laying down harmonised rules on artificial intelligence (Artificial Intelligence Act), Article 14: Human oversight." Official Journal of the European Union, OJ L, 2024/1689, 2024. `https://artificialintelligenceact.eu/article/14/`.

[262] National Institute of Standards and Technology, "Artificial intelligence risk management framework (AI RMF 1.0)," Tech. Rep. NIST AI 100-1, National Institute of Standards and Technology, 2023.

[263] Y. Liu, W. Wang, R. Feng, Y. Zhang, G. Xu, G. Deng, Y. Li, and L. Zhang, "Agent skills in the wild: An empirical study of security vulnerabilities at scale." arXiv preprint arXiv:2601.10338, Jan. 2026. Preprint. Accessed: 2026-11-11.

[264] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[265] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[266] J. Hoffmann, A. Botev, G. Penedo, A. N. Alvi, *et al.*, "Training compute-optimal large language models," *arXiv preprint arXiv:2203.15556*, 2022.

[267] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, *et al.*, "PaLM: Scaling language modeling with pathways," *arXiv preprint arXiv:2204.02311*, 2022.

[268] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, *et al.*, "OPT: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022.

[269] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[270] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *Nature*, 2025.

[271] Y. Cao, S. Li, Y. Liu, Z. Yan, Y. Dai, P. S. Yu, and L. Sun, "A comprehensive survey of AI-generated content (AIGC): A history of generative AI from GAN to ChatGPT," *arXiv preprint arXiv:2303.04226*, 2023.

[272] C. O'Neil, *Weapons of math destruction: How big data increases inequality and threatens democracy*. Crown, 2017.

[273] E. Hubinger, C. van Merwijk, V. Mikulik, J. Skalse, and S. Garrabrant, "Risks from learned optimization in advanced machine learning systems," *arXiv preprint arXiv:1906.01820*, 2019.

[274] R. Shah, V. Varma, R. Kumar, M. Phuong, V. Krakovna, J. Uesato, and Z. Kenton, "Goal misgeneralization: Why correct specifications aren't enough for correct goals," *arXiv preprint arXiv:2210.01790*, 2022.

[275] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in AI safety," *arXiv preprint arXiv:1606.06565*, 2016.

[276] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," *arXiv preprint arXiv:1706.03741*, 2017.

[277] J. Leike, D. Krueger, T. Everitt, M. Martic, V. Maini, and S. Legg, "Scalable agent alignment via reward modeling: A research direction," *arXiv preprint arXiv:1811.07871*, 2018.

[278] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*, pp. 353–355, 2018.

[279] BIG-bench Collaboration, "Beyond the imitation game: Quantifying and extrapolating the capabilities of language models," *arXiv preprint arXiv:2206.04615*, 2022.

[280] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, *et al.*, "Measuring massive multitask language understanding," *arXiv preprint arXiv:2009.03300*, 2021.

[281] A. Warstadt, A. Mueller, L. Choshen, E. Wilcox, C. Zhuang, J. Ciro, R. Mosquera, B. Paranjabe, A. Williams, T. Linzen, and R. Cotterell, "Findings of the BabyLM challenge: Sample-efficient

pretraining on developmentally plausible corpora," in *Proceedings of the BabyLM Challenge at the 27th Conference on Computational Natural Language Learning*, (Singapore), pp. 1–34, Association for Computational Linguistics, 2023.

[282] P. Liang, R. Bommasani, T. Lee, D. Tsipras, *et al.*, "Holistic evaluation of language models," *arXiv preprint arXiv:2211.09110*, 2022.

[283] G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom, "GAIA: a benchmark for general AI assistants," *arXiv preprint arXiv:2311.12983*, 2023.

[284] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[285] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv preprint arXiv:2406.12045*, 2024.

[286] N. Guha, J. Nyarko, D. E. Ho, C. Ré, A. Chilton, A. Narayana, A. Chohlas-Wood, A. Peters, B. Waldon, D. N. Rockmore, *et al.*, "LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models," in *Advances in Neural Information Processing Systems 36 (NeurIPS Datasets and Benchmarks)*, 2023.

[287] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," *Advances in Neural Information Processing Systems*, vol. 36, pp. 46595–46623, 2023.

[288] P. Wang, L. Li, L. Chen, Z. Cai, D. Zhu, B. Lin, Y. Cao, Q. Liu, T. Liu, and Z. Sui, "Large language models are not fair evaluators," *arXiv preprint arXiv:2305.17926*, 2023.

[289] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, Y. Wang, and J. Guo, "A survey on LLM-as-a-judge," *arXiv preprint arXiv:2411.15594*, 2024.

[290] C. Xu, S. Guan, D. Greene, and M.-T. Kechadi, "Benchmark data contamination of large language models: A survey," *arXiv preprint arXiv:2406.04244*, 2024.

[291] S. Bedi, H. Cui, M. Fuentes, A. Unell, M. Wornow, J. M. Banda, E. Horvitz, P. Liang, N. H. Shah, *et al.*, "MedHELM: Holistic evaluation of large language models for medical tasks," *arXiv preprint arXiv:2505.23802*, 2025.

[292] LMSYS, "LMArena (Chatbot Arena) leaderboard." `https://arena.ai/leaderboard`, 2025. Accessed: 2025-08-19.

[293] S. Chacon and B. Straub, *Pro Git*. Berkeley, CA: Apress, 2nd ed., 2014.

[294] Mercurial Project, "Mercurial: Guide." `https://www.mercurial-scm.org/guide.html`, 2025.

[295] Fossil Project, "Fossil SCM." `https://fossil-scm.org/`, 2025.

[296] D. Roundy, "The theory of patches." `https://www.cs.tufts.edu/~nr/cs257/archive/david-roundy/tp.pdf`, 2005. Unpublished manuscript.

[297] Pijul Project, "The Pijul manual: Theory." `https://pijul.org/manual/theory.html`, 2025.

[298] Jujutsu Project, "Jujutsu (jj) documentation." `https://docs.jj-vcs.dev/latest/`, 2025.

[299] Jujutsu Project, "Jujutsu (jj): A Git-compatible VCS." `https://github.com/jj-vcs/jj`, 2025.

[300] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, "Software engineering for machine learning: A case study," in *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pp. 291–300, IEEE, 2019.

[301] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems*, 2015.

[302] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ML test score: A rubric for ML production readiness and technical debt reduction," in *2017 IEEE International Conference on Big Data (Big Data)*, pp. 1123–1132, IEEE, 2017.

[303] D. Baylor, E. Breck, H.-T. Cheng, N. Fiedel, C. Foo, Z. Haque, S. Haykal, M. Ispir, V. Jain, L. Koc, *et al.*, "TFX: A TensorFlow-based production-scale machine learning platform," in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 1387–1395, ACM, 2017.

[304] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, F. Xie, and C. Zumar, "Accelerating the machine learning lifecycle with MLflow," *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, vol. 41, no. 4, pp. 39–45, 2018.

[305] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, "Model cards for model reporting," in *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '19)*, pp. 220–229, 2019.

[306] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.

[307] Nordic Casemix Centre, "NordDRG specification." `https://nordcase.org/`, 2025.

[308] R. B. Fetter, Y. Shin, J. L. Freeman, R. F. Averill, and J. D. Thompson, "Case mix definition by diagnosis-related groups," *Medical Care*, vol. 18, no. 2 Suppl, pp. 1–53, 1980.

[309] Nordic Casemix Centre, "How to read the NordDRG definition tables." `https://www.vmnvd.gov.lv/en/media/556/download`, 2025.

[310] K. Singhal, S. Azizi, T. Tu, *et al.*, "Large language models encode clinical knowledge," *Nature*, vol. 620, pp. 172–180, 2023.

[311] H. Nori, N. King, S. M. McKinney, D. Carignan, and E. Horvitz, "Capabilities of GPT-4 on medical challenge problems," *arXiv preprint arXiv:2303.13375*, 2023.

[312] H. Wang, C. Gao, C. Dantona, B. Hull, and J. Sun, "DRG-LLaMA: tuning LLaMA model to predict diagnosis-related group for hospitalized patients," *npj Digital Medicine*, vol. 7, no. 1, p. 16, 2024.

[313] A. E. W. Johnson, L. Bulgarelli, L. Shen, *et al.*, "MIMIC-IV, a freely accessible electronic health record dataset," *Scientific Data*, vol. 10, no. 1, 2023.

[314] K. Kwan, "Large language models are good medical coders, if provided with tools," *arXiv preprint arXiv:2407.12849*, 2024.

[315] P. Renc, Y. Jia, A. E. Samir, J. Was, Q. Li, D. W. Bates, and A. Sitek, "Zero shot health trajectory prediction using transformer," *npj Digital Medicine*, vol. 7, no. 1, p. 256, 2024.

[316] D. Hajialigol, D. Kaknes, T. Barbour, D. Yao, C. North, J. Sun, D. Liem, and X. Wang, "DRGCoder: Explainable clinical coding for the early prediction of diagnostic-related groups," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 373–380, Association for Computational Linguistics, 2023.

[317] J. Mullenbach, S. Wiegreffe, J. Duke, J. Sun, and J. Eisenstein, "Explainable prediction of medical codes from clinical text," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pp. 1101–1111, 2018.

[318] Y. He, C. Wang, S. Zhang, N. Li, Z. Li, and Z. Zeng, "KG-MTT-BERT: Knowledge graph enhanced BERT for multi-type medical text classification," *arXiv preprint arXiv:2210.03970*, 2022.

[319] J. S. Boyle, A. Kascenas, P. Lok, M. Liakata, and A. Q. O'Neil, "Automated clinical coding using off-the-shelf large language models," *arXiv preprint arXiv:2310.06552*, 2023.

[320] Z. Boukhers, A. Khan, Q. Ramadan, and C. Yang, "Large language model in medical informatics: Direct classification and enhanced text representations for automatic ICD coding," *arXiv preprint arXiv:2411.06823*, 2024.

[321] R. Busse, A. Geissler, W. Quentin, and M. Wiley, *Diagnosis-Related Groups in Europe: Moving Towards Transparency, Efficiency and Quality in Hospitals*. Copenhagen: European Observatory on Health Systems and Policies, 2011.

[322] Centers for Medicare & Medicaid Services, "ICD-10 MS-DRGs version 42.1, effective April 1, 2025," tech. rep., Centers for Medicare & Medicaid Services, Apr. 2025. MS-DRG Grouper overview and definitions.

[323] Independent Hospital and Aged Care Pricing Authority (IHACPA), "AR-DRG version 11.0 technical specifications," tech. rep., Independent Hospital and Aged Care Pricing Authority, 2022.

[324] 3M Health Information Systems, "3M™ all patient refined DRG (APR DRG) classification system: Overview." `https://multimedia.3m.com/mws/media/910941O/ebook-3m-all-patient-refined-apr-drgs.pdf`, 2021.

[325] Foundation for Intelligent Physical Agents, "FIPA Request Interaction Protocol Specification," Tech. Rep. XC00026, FIPA, Geneva, Switzerland, 2000. URL: http://www.fipa.org/specs/fipa00026/.

[326] W3C, "World Wide Web Consortium Recommendation: Resource Description Framework (RDF) 1.0." URL: http://www.w3.org/RDF/.

[327] Foundation for Intelligent Physical Agents, "FIPA Query Interaction Protocol Specification," Tech. Rep. XC00027, FIPA, Geneva, Switzerland, 2000. URL: http://www.fipa.org/specs/fipa00027/.

[328] W3C, "World Wide Web Consortium Member Submission: RDQL - A Query Language for RDF." URL: http://www.w3.org/Submission/2004/SUBM-RDQL-20040109/.

[329] Hewlett Packard Semantic Web Research Lab, "Jena: A Semantic Web Framework for Java," 2004. Open Source, URL: http://www.hpl.hp.com/semweb/index.html.

[330] Foundation for Intelligent Physical Agents, "FIPA Propose Interaction Protocol Specification," Tech. Rep. XC00036, FIPA, Geneva, Switzerland, 2000. URL: http://www.fipa.org/specs/fipa00036/.

[331] Fuego Core Project, "Fuego Core event server," 2005.

[332] R. Järvensivu, R. Pitkänen, and T. Mikkonen, "Object-oriented middleware for location-aware systems," in *SAC '04: Proceedings of the 2004 ACM Symposium on Applied Computing*, pp. 1184–1190, ACM Press, 2004.

[333] D. J. Tufts-Conrad, A. N. Zincir-Heywood, and D. Zitner, "SOM – feature extraction from patient discharge summaries," in *Symposium on Applied Computing*, pp. 263–267, ACM, 2003. ISBN: 1-58113-624-2.

[334] C. Servais, "Computer/assisted coding for inpatients / a case study," in *Perspectives in Health Information Management, Computer Assisted Coding Conference*, American Health Information Management Association, 2006.

[335] P. Resnik, M. Niv, M. Nossal, G. Schnitzer, J. Stoner, A. Kapit, and R. Toren, "Using intrinsic and extrinsic metrics to evaluate accuracy and facilitation in computer-assisted coding," in *Perspectives in Health Information Management, Computer Assisted Coding Conference*, American Health Information Management Association, 2006.

[336] R. Agrawal, T. Imieliński, and A. Swami, "Mining association rules between sets of items in large databases," in *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*, pp. 207–216, 1993.

[337] J. Stausberg, N. Lehmann, D. Kaczmarek, and M. Stein, "Reliability of diagnoses coding with ICD-10," *International Journal of Medical Informatics*, vol. 77, no. 1, pp. 50–57, 2008.

[338] J. Stausberg, D. Koch, J. Ingenerf, and M. Betzler, "Comparing paper-based with electronic patient records: Lessons learned during a study on diagnosis and procedure codes," *Journal of the American Medical Informatics Association*, vol. 10, no. 5, pp. 470–477, 2003.

[339] G. Surjan, "Questions on validity of International Classification of Diseases-coded diagnoses," *International Journal of Medical Informatics*, vol. 54, no. 2, pp. 77–95, 1999.

[340] C. Colin, R. Ecochard, F. Delahaye, G. Landrivon, P. Messy, E. Morgon, and Y. Matillon, "Data quality in a DRG-based information system," *International Journal for Quality in Health Care*, vol. 6, no. 6, pp. 275–280, 1994.

[341] Socialstyrelsen, *Kodningskvalitet i patientregistret, Slutenvård 2005.* No. 2007-125-13, Socialstyrelsen, 2007. ISBN 978-91-7164-281-3, Available (in Swedish) at URL: http://www.socialstyrelsen.se/.

[342] Socialstyrelsen, *Kvalitet och innehåll i patientregistret, Utskrivningar från slutenvård 1964-2006 och besök i öppenvård (exklusive primärvårdsbesök) 1997-2006.* No. 2008-125-1, Socialstyrelsen, 2008. ISBN 978-91-7164-281-3, Available (in Swedish) at URL: http://www.socialstyrelsen.se/.

[343] D. Sammon and P. Finnegan, "The ten commandments of data warehousing," *SIGMIS Database*, vol. 31, no. 4, pp. 82–91, 2000.

[344] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules in large databases," in *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*, pp. 487–499, 1994.

[345] G. Salton, A. Wong, and C.-S. Yang, "A vector space model for automatic indexing," *Communications of the ACM*, vol. 18, no. 11, pp. 613–620, 1975.

[346] N. Cristianini and J. Shawe-Taylor, *An Introduction to Support Vector Machines and Other Kernel-based Learning Methods*. Cambridge University Press, 2000.

[347] G. Qian, S. Sural, Y. Gu, and S. Pramanik, "Similarity between Euclidean and cosine angle distance for nearest neighbor queries," in *Proceedings of the 2004 ACM Symposium on Applied Computing*, pp. 1232–1237, ACM, 2004. ISBN: 1-58113-812-1.

[348] J. Zobel and A. Moffat, "Inverted files for text search engines," *ACM Computing Surveys*, vol. 38, no. 2, p. 6, 2006.

[349] WHO, *International Statistical Classification of Diseases and Related Health Problems*, vol. 1, pp. 1–10. World Health Organization, 1992. ISBN: 92 4 154419 8.

[350] WHO, *International Statistical Classification of Diseases and Related Health Problems, Instruction manual*, vol. 2. World Health Organization, 2004. ISBN: 92 4 154649 8.

[351] Nordic Classification Centre, *NOMESCO Classification of Surgical Procedures*. Nordic Medico-Statistical Committee (NOMESCO), 2006. ISBN: 87-89702-57-3.

[352] A. Ehtesham, A. Singh, G. K. Gupta, and S. Kumar, "A survey of agent interoperability protocols: Model Context Protocol (MCP), agent communication protocol (ACP), agent-to-agent protocol (A2A), and agent network protocol (ANP)," *arXiv preprint arXiv:2505.02279*, 2025.

[353] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, and M. Khabsa, "Llama Guard: LLM-based input-output safeguard for human-AI conversations," *arXiv preprint arXiv:2312.06674*, 2023.

[354] J. D. Lee and K. A. See, "Trust in automation: Designing for appropriate reliance," *Human Factors*, vol. 46, no. 1, pp. 50–80, 2004.

[355] G. Bansal, T. Wu, J. Zhou, R. Fok, B. Nushi, E. Kamar, M. T. Ribeiro, and D. S. Weld, "Does the whole exceed its parts? The effect of AI explanations on complementary team performance," in *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 2021.

[356] Z. Buçinca, M. B. Malaya, and K. Z. Gajos, "To trust or to think: Cognitive forcing functions can reduce overreliance on AI in AI-assisted decision-making," *Proceedings of the ACM on Human-Computer Interaction*, vol. 5, no. CSCW1, 2021.

[357] G. He, L. Kuiper, and U. Gadiraju, "Knowing about knowing: An illusion of human competence can hinder appropriate reliance on AI systems," in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, 2023.

[358] G. He, G. Demartini, and U. Gadiraju, "Plan-then-execute: An empirical study of user trust and team performance when using LLM agents as a daily assistant," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025.

[359] B. Green and Y. Chen, "The principles and limits of algorithm-in-the-loop decision making," *Proceedings of the ACM on Human-Computer Interaction*, vol. 3, no. CSCW, 2019.

[360] M. Vaccaro, A. Almaatouq, and T. Malone, "When combinations of humans and AI are useful: A systematic review and meta-analysis," *Nature Human Behaviour*, vol. 8, pp. 2293–2303, 2024.

[361] P. R. Niven and B. Lamorte, *Objectives and Key Results: Driving Focus, Alignment, and Engagement with OKRs*. John Wiley & Sons, Inc., 2016.

[362] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. Newtown Square, PA: Project Management Institute, sixth ed., 2017.

[363] A. Klein, I. Mathauer, K. Stenberg, and T. Habicht, "Diagnosis-related groups (DRG): A question & answer guide on case-based classification and payment systems," Tech. Rep. WHO/UHC/HGF/Guidance/20.10, World Health Organization, 2020.

[364] OECD, "Innovative provider payment models for promoting value-based health systems," tech. rep., OECD, 2023.

[365] Centers for Medicare & Medicaid Services, "Acute inpatient prospective payment system (IPPS) and MS-DRGs." `https://www.cms.gov/medicare/payment/prospective-payment-systems/acute-inpatient-pps/ms-drg-classifications-and-software`, 2025. Accessed: 2025-08-18.

[366] M. T. Schneider, A. Y. Chang, A. Chapin, C. S. Chen, S. W. Crosby, A. C. Harle, G. Tsakalos, B. S. Zlavog, and J. L. Dieleman, "Health expenditures by services and providers for 195 countries, 2000–2017," *BMJ Global Health*, vol. 6, no. 7, p. e005799, 2021.

[367] OECD, "Health at a glance 2023: OECD indicators," tech. rep., OECD Publishing, Paris, 2023. See chapter "Health expenditure by provider".

[368] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 1, pp. 334–347, 2019.

[369] L. Moreau and P. Missier, "PROV-DM: The PROV data model." W3C Recommendation, 2013. `https://www.w3.org/TR/prov-dm/`.

[370] G. Allen, G. He, and U. Gadiraju, "Power-up! What can generative models do for human computation workflows?," *arXiv preprint arXiv:2307.02243*, 2023.

[371] E. Horvitz, "Principles of mixed-initiative user interfaces," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '99)*, pp. 159–166, 1999.

[372] D. Dellermann, P. Ebel, M. Söllner, and J. M. Leimeister, "Hybrid intelligence," *Business & Information Systems Engineering*, vol. 61, no. 5, pp. 637–643, 2019.

[373] X. Liu, C. Fang, C. Wu, J. Yu, and Q. Zhao, "DRG grouping by machine learning: from expert-oriented to data-based method," *BMC Medical Informatics and Decision Making*, vol. 21, no. 1, p. 312, 2021.

[374] ISO/IEC JTC 1/SC 42, "ISO/IEC 42001:2023 information technology — artificial intelligence — management system." `https://www.iso.org/standard/81230.html`, 2023. International standard.

[375] I. D. Raji, A. Smart, R. N. White, M. Mitchell, T. Gebru, B. Hutchinson, J. Smith-Loud, D. Theron, and P. Barnes, "Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing," in *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAT*)*, pp. 33–44, 2020.

[376] M. Arnold, R. K. E. Bellamy, M. Hind, S. Houde, S. Mehta, A. Mojsilović, R. Nair, K. Natesan Ramamurthy, A. Olteanu, D. Piorkowski, D. Reimer, J. Richards, J. Tsay, and K. R. Varshney, "FactSheets: Increasing trust in AI services through supplier's declarations of conformity," *IBM Journal of Research and Development*, vol. 63, no. 4/5, 2019.

[377] M. Bovens, "Analysing and assessing accountability: A conceptual framework," *European Law Journal*, vol. 13, no. 4, pp. 447–468, 2007.

[378] Y. Asnar and P. Giorgini, "Modelling risk and identifying countermeasure in organizations," in *Critical Information Infrastructures Security (CRITIS 2006)*, vol. 4347 of *Lecture Notes in Computer Science*, pp. 55–66, Springer, 2006.

[379] C. C. Marshall and F. M. Shipman, "Which Semantic Web?," in *Proceedings of the Fourteenth ACM Conference on Hypertext and Hypermedia (HYPERTEXT '03)*, (Nottingham, UK), pp. 57–66, ACM, 2003.

[380] J. Hendler, "Where are all the intelligent agents?," *IEEE Intelligent Systems*, vol. 22, no. 3, pp. 2–3, 2007.

[381] A. Bernstein, J. Hendler, and N. Noy, "A new look at the Semantic Web," *Communications of the ACM*, vol. 59, no. 9, pp. 35–37, 2016.

[382] R. A. Brooks, "Intelligence without representation," *Artificial Intelligence*, vol. 47, no. 1-3, pp. 139–159, 1991.

[383] A. Ciortea, S. Mayer, F. Gandon, O. Boissier, A. Ricci, and A. Zimmermann, "A decade in hindsight: The missing bridge between multi-agent systems and the World Wide Web," in *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS 2019), Blue Sky Ideas Track*, (Montréal, Canada), pp. 1659–1663, International Foundation for Autonomous Agents and Multiagent Systems, 2019.

[384] A. Halevy, P. Norvig, and F. Pereira, "The unreasonable effectiveness of data," *IEEE Intelligent Systems*, vol. 24, no. 2, pp. 8–12, 2009.

[385] M. Luck, P. McBurney, O. Shehory, and S. Willmott, *Agent Technology: Computing as Interaction. A Roadmap for Agent Based Computing*. Southampton, UK: AgentLink III, University of Southampton, Sept. 2005.

[386] A. Sheth, C. Ramakrishnan, and C. Thomas, "Semantics for the Semantic Web: The implicit, the formal and the powerful," *International Journal on Semantic Web and Information Systems*, vol. 1, no. 1, pp. 1–18, 2005.

[387] J. P. Müller and K. Fischer, "Application impact of multi-agent systems and technologies: A survey," in *Agent-Oriented Software Engineering: Reflections on Architectures, Methodologies, Languages, and Frameworks* (O. Shehory and A. Sturm, eds.), pp. 27–53, Berlin, Heidelberg: Springer, 2014.

[388] F. P. Brooks, Jr., "The computer scientist as toolsmith II," *Communications of the ACM*, vol. 39, pp. 61–68, Mar. 1996.

[389] E. O. Wilson, *Consilience: The Unity of Knowledge*. New York: Alfred A. Knopf, 1998.

[390] T. Kwa, B. West, J. Becker, A. Deng, K. Dinh, L. Ho, L. Chan, *et al.*, "Measuring AI ability to complete long tasks." arXiv preprint arXiv:2503.14499, 2025. Time Horizon 1.1 update published by METR in January 2026.

[391] The Linux Foundation, "Agentic AI Foundation announces four new projects: Agent Skills, Open Responses, OpenEnv, and OpenSandbox." Press release, `https://www.linuxfoundation.org/press/agentic-ai-foundation-announces-four-new-projects-agent-skills-open-responses-openenv-and-opensandbox`, Oct. 2026. Published 22 October 2026. Accessed: 2026-11-11.